unirse al anillo podrá hacerlo. Si las primeras dos estaciones se encienden simultáneamente, el protocolo maneja esto permitiendo que soliciten la ficha usando el algoritmo estándar de conteo descendente binario modificado y los dos bits aleatorios.

Debido a los errores de transmisión y a las fallas de *hardware*, pueden surgir problemas con el anillo lógico o la señal. Por ejemplo, si una estación trata de pasar la ficha a una estación que se ha desactivado, ¿qué ocurre? La solución es directa. Tras pasar la ficha, una estación escucha si su sucesor transmite un marco o pasa la ficha; si no lo hace, se le pasa la ficha una segunda vez.

Si también eso falla, la estación transmite un marco WHO_FOLLOWS especificando la dirección de su sucesor. Cuando el sucesor de la estación fallida ve un marco nombrando a su antecesor, responde con un marco SET_SUCCESSOR a la estación a la que le falló el sucesor, nombrándose como nuevo sucesor. De esta manera, la estación fallida se retira de la red.

Ahora supongamos que una estación no pasa la ficha a su sucesor y tampoco logra localizar al sucesor del sucesor, que también podría haberse caído. La estación adopta una estrategia nueva enviando un marco SOLICIT_SUCCESSOR_2 para ver si queda *alguien* vivo. Nuevamente se ejecuta el protocolo estándar de contención, solicitando ahora un lugar todas las estaciones que quieren estar en el anillo. Finalmente, se restablece el anillo.

Ocurre otro tipo de problema si el poseedor de la ficha se desactiva y se lleva con él la ficha. Este problema se resuelve usando el algoritmo de inicialización del anillo. Cada estación tiene un temporizador que se restablece cada vez que aparece un marco en la red. Cuando este temporizador alcanza un valor límite, la estación envía un marco CLAIM_TOKEN, y el algoritmo de conteo binario modificado con bits aleatorios determina quién obtiene la ficha.

Un problema más es el de las fichas múltiples. Si una estación poseedora de la ficha nota una transmisión desde otra estación, se deshace de su ficha. Si había dos, ahora habrá una. Si había más de dos, este proceso se repetirá tarde o temprano hasta que todas las fichas, menos una, se desechen. Si por accidente se desecharan todas las fichas, la falta de actividad causaría que una o más estaciones trataran de reclamar la ficha.

4.3.3. Estándar IEEE 802.5: *token ring*

Las redes en anillo no son nada nuevo, pues (Pierce, 1972) se han utilizado desde hace mucho para redes tanto locales como de área amplia. Entre sus muchas características atractivas está el que un anillo no es realmente un medio de difusión, sino un conjunto de enlaces punto a punto individuales que, coincidentemente, forman un círculo. Los enlaces punto a punto implican una tecnología bien entendida y probada en el campo que puede operar en par trenzado, cable coaxial y fibra óptica. La ingeniería de anillos es casi completamente digital. En contraste, por ejemplo, el 802.3 tiene una componente analógica considerable para la detección de colisiones. Un anillo también es equitativo y tiene un límite superior conocido de acceso a canal. Por estas razones, IBM escogió el anillo como su LAN y el IEEE ha incluido el estándar ***token ring*** como el 802.5 (IEEE, 1985c; Latif *et al.*, 1992).

Un asunto fundamental en el diseño y análisis de cualquier red en anillo es la "longitud física" de un bit. Si la tasa de datos del anillo es de R Mbps, se emite un bit cada $1/R$ µseg. Con una velocidad de propagación de señal típica de unos 200 m/µseg, cada bit ocupa $200/R$ metros del anillo. Esto significa, por ejemplo, que un anillo de 1 Mbps cuya circunferencia es de 1000 metros puede contener sólo 5 bits a la vez. Las implicaciones del número de bits en el anillo se aclararán después.

Como se mencionó antes, un anillo en realidad consiste en un conjunto de interfaces de anillo conectadas por líneas punto a punto. Cada bit que llega a una interfaz se copia en un *buffer* de 1 bit y luego se copia en el anillo nuevamente. Mientras está en el *buffer*, el bit se puede inspeccionar y posiblemente modificar antes de enviarse. Este paso de copiado introduce un retardo de 1 bit en cada interfaz. En la figura 4-28 se muestra un anillo y sus interfaces.

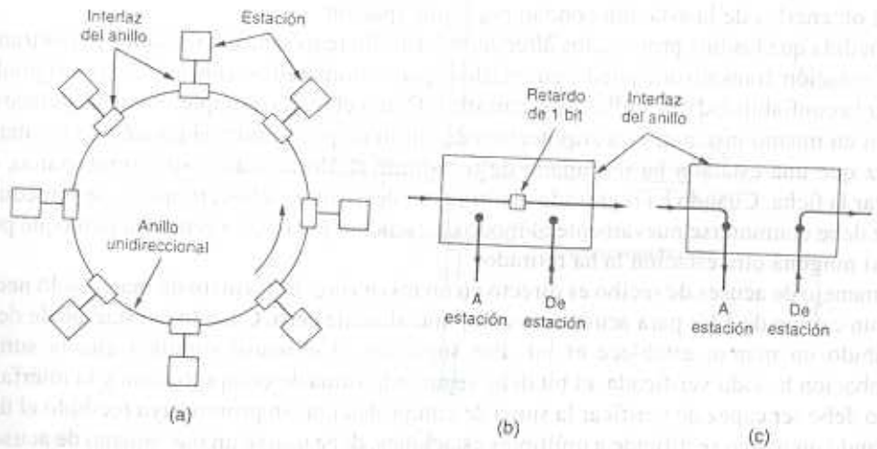


Figura 4-28. (a) Red en anillo. (b) Modo de escuchar. (c) Modo de transmitir.

En un *token ring* (anillo con ficha) circula un patrón de bit especial, llamado **ficha** (*token*) alrededor del anillo cuando todas las estaciones están inactivas. Cuando una estación quiere transmitir un marco, debe tomar la ficha y retirarla del anillo antes de transmitir. Esta acción se lleva a cabo invirtiendo un solo bit de la ficha de 3 bytes, lo que instantáneamente la convierte en los tres primeros bytes de un marco de datos normal. Debido a que sólo hay una ficha, sólo una estación puede transmitir en un instante dado, resolviendo por tanto el problema de acceso al canal de la misma manera en que lo resuelve el *token bus*.

Una implicación del diseño *token ring* es que el anillo mismo debe tener un retardo (*delay*) suficiente para contener una ficha completa que circule cuando todas las estaciones están inactivas. El retardo tiene dos componentes: el retardo de 1 bit introducido por cada estación y el retardo de propagación de señal. En la mayoría de los anillos, los diseñadores deben suponer que pueden apagarse estaciones en diferentes momentos, especialmente durante la noche. Si las interfaces se alimentan de energía o energizan desde el anillo, el apagado de la estación no

tiene ningún efecto sobre la interfaz, pero si las interfaces se energizan externamente deben estar diseñadas para conectar la entrada a la salida al interrumpirse la energía, con lo que se elimina el retardo de 1 bit. La cuestión aquí es que, en un anillo corto, podría ser necesario introducir un retardo artificial en el anillo por las noches para asegurar que pueda contener una ficha.

Las interfaces del anillo tienen dos modos operativos, escuchar y transmitir. En el modo de escuchar, los bits de entrada simplemente se copian en la salida, con un retardo de un tiempo de bit, como se muestra en la figura 4-28(b). En el modo de transmitir, al que se entra una vez que se tiene la ficha, la interfaz rompe la conexión entre la entrada y la salida, introduciendo sus propios datos en el anillo. Para poder conmutar del modo de escuchar al de transmitir en un tiempo de 1 bit, la interfaz generalmente necesita almacenar en su *buffer* uno o más marcos, en lugar de obtenerlos de la estación con tan poca anticipación.

A medida que los bits propagados alrededor del anillo regresan, son retirados por el transmisor. La estación transmisora puede guardarlos, para compararlos con los datos originales y verificar la confiabilidad del anillo, o descartarlos. Como el marco completo nunca aparece en el anillo en un mismo instante, esta arquitectura de anillo no pone límite al tamaño de los marcos. Una vez que una estación ha terminado de transmitir el último bit de su último marco, debe regenerar la ficha. Cuando ha regresado el último bit del marco, debe retirarse, y de inmediato la interfaz debe conmutarse nuevamente al modo de escuchar para evitar retirar la ficha que podría seguir si ninguna otra estación la ha retirado.

El manejo de acuses de recibo es directo en un *token ring*. El formato de marco sólo necesita incluir un campo de 1 bit para acuse de recibo, inicialmente cero. Cuando la estación de destino ha recibido un marco, establece el bit. Por supuesto, si el acuse significa que la suma de comprobación ha sido verificada, el bit debe seguir a la suma de comprobación y la interfaz con el anillo debe ser capaz de verificar la suma de comprobación tan pronto haya recibido el último bit. Cuando un marco se difunde a múltiples estaciones, debe usarse un mecanismo de acuse más complicado (si es que se usa uno).

Cuando el tráfico es ligero, la ficha pasará la mayor parte del tiempo circulando inactivamente alrededor del anillo. Ocasionalmente, una estación la tomará, transmitirá un marco, y entonces emitirá una ficha nueva. Sin embargo, cuando el tráfico es pesado, de modo que hay una cola en cada estación, tan pronto una estación termina su transmisión y regenera la ficha la siguiente estación verá y retirará la ficha. De esta manera, el permiso para enviar gira continuamente, por turno circular. La eficiencia de la red puede acercarse al 100% en condiciones de carga pesada.

Ahora pasemos de los *token rings* en general al estándar 802.5 en particular. En la capa física, el 802.5 especifica par trenzado operando a 1 o 4 Mbps, aunque IBM posteriormente introdujo una versión de 16 Mbps. Las señales se codifican usando codificación Manchester diferencial [véase la figura 4-20(c)], siendo alto (HI) y bajo (LO) señales positivas y negativas de magnitudes absolutas de 3.0 a 4.5 volts. Normalmente, la codificación Manchester diferencial usa HI-LO o LO-HI para cada bit, pero el 802.5 también usa HI-HI y LO-LO en ciertos bytes de control (por ejemplo, para marcar el inicio y el final de un marco). Estas señales que no son de datos ocurren en pares consecutivos y no introducen una componente de cc (*DC component*) en el voltaje del anillo.

Un problema con las redes de anillo es que, si se rompe el cable en alguna parte, el anillo se inhabilita. Este problema puede resolverse de manera muy elegante mediante el uso de un **centro de alambrado**, como se muestra en la figura 4-29. Si bien la red sigue siendo lógicamente un anillo, físicamente cada estación está conectada al centro de alambrado que contiene (cuando menos) dos pares trenzados, uno de datos a la estación y otro de datos de la estación.

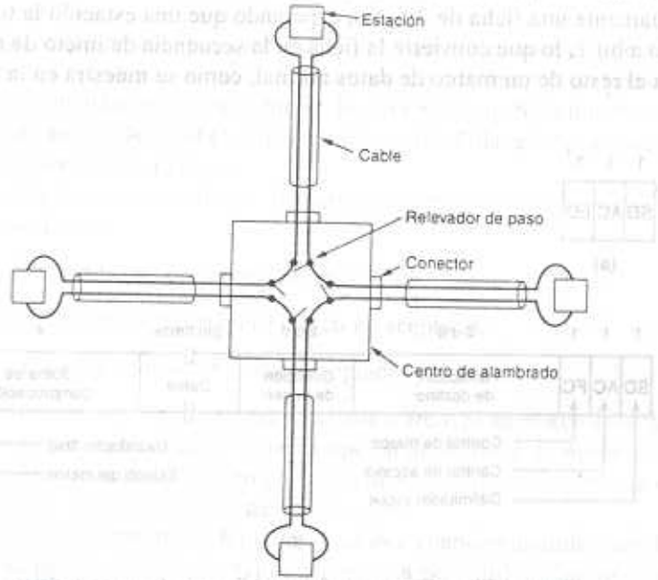


Figura 4-29. Cuatro estaciones conectadas mediante un centro de alambrado.

Dentro del centro de alambrado hay relevadores de paso (*bypass relays*) que se energizan mediante corriente de las estaciones. Si se rompe el anillo o se inactiva una estación, la pérdida de la corriente de operación desactivará el relevador, poniendo en puente la estación. Los relevadores también pueden operarse mediante *software* que permita a los programas de diagnóstico remover estaciones una a la vez para encontrar estaciones y segmentos de anillo con fallas. El anillo entonces puede continuar su operación teniendo en puente la sección dañada. Aunque el estándar 802.5 no requiere formalmente este tipo de anillo, llamado con frecuencia **anillo en estrella** (Saltzer *et al.*, 1983) de hecho la mayoría de las LAN 802.5 usan centros de alambrado para mejorar su confiabilidad y mantenimiento.

Cuando una red consiste en muchos grupos de estaciones muy separados, puede usarse una topología con muchos centros de alambrado. Sólo imagine que el cable a una de las estaciones de la figura 4-29 fuera reemplazado por un cable a un centro de alambrado distante. Aunque lógicamente todas las estaciones están en el mismo anillo, los requisitos de alambrado se

reducen significativamente. Un anillo 802.5 que utiliza un centro de alambrado tiene una topología semejante a una red 10Base-T basada en concertadores (*hubs*), pero los formatos y protocolos son diferentes.

Protocolo de subcapa MAC del *token ring*

La operación básica del protocolo de MAC es sencilla. Cuando no hay tráfico en el anillo, circula continuamente una ficha de 3 bytes, esperando que una estación la tome cambiando un bit 0 específico a bit 1, lo que convierte la ficha en la secuencia de inicio de marco. La estación entonces envía el resto de un marco de datos normal, como se muestra en la figura 4-30.

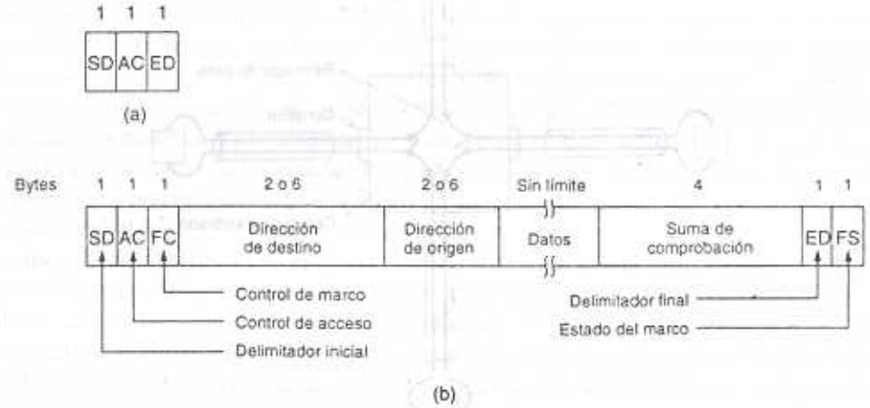


Figura 4-30. (a) Formato de ficha. (b) Formato de marco de datos.

En condiciones normales, el primer bit del marco recorrerá el anillo y regresará al transmisor antes de que el marco completo haya sido transmitido. Sólo un anillo muy grande será capaz de contener incluso un marco muy corto. En consecuencia, la estación transmisora debe drenar el anillo mientras continúa transmitiendo. Como se muestra en la figura 4-28(c), esto significa que los bits que han completado el viaje alrededor del anillo regresan al transmisor y ahí son removidos.

Una estación puede retener la ficha durante el **tiempo de retención de la ficha**, que es de 10 mseg a menos que una instalación establezca un valor distinto. Si queda suficiente tiempo tras la transmisión del primer marco para enviar más marcos, éstos pueden enviarse también. Una vez que han sido transmitidos todos los marcos pendientes, o si la transmisión de otro marco excedería el tiempo de retención de la ficha, la estación regenera el marco de ficha de 3 bits y lo pone en el anillo.

Los campos de *delimitador inicial* y *delimitador final* de la figura 4-30(b) marcan el inicio y el fin del marco. Cada uno contiene patrones Manchester diferenciales no válidos (HH y LL) para

distinguirlos de los bytes de datos. El byte de *control de acceso* contiene el bit de ficha y también el *bit de revisión*, los *bits de prioridad* y los *bits de reservación* (descritos más adelante). El byte de *control de marco* distingue los marcos de datos de los diferentes marcos de control posibles.

A continuación vienen los campos de *dirección destino* y *dirección origen*, que son iguales que en el 802.3 y el 802.4. A éstos siguen los datos, que pueden ser tan grandes como sea necesario, siempre y cuando el marco aún pueda transmitirse dentro del tiempo de retención de la ficha. El campo de *suma de comprobación*, como las direcciones de destino y origen, también es igual que en 802.3 y 802.4.

Un byte interesante que no está presente en los otros dos protocolos es el byte de *estado de marco*, que contiene los bits *A* y *C*. Cuando llega un marco a la interfaz de una estación con la dirección de destino, la interfaz enciende el bit *A* y lo pasa adelante. Si la interfaz copia el marco en la estación, también enciende el bit *C*. Una estación podría fallar al copiar un marco debido a falta de espacio de *buffer* u otras razones.

Cuando la estación transmisora drena el marco del anillo, examina los bits *A* y *C*. Son posibles tres combinaciones:

1. $A = 0$ y $C = 0$: destino no presente o no encendido.
2. $A = 1$ y $C = 0$: destino presente pero marco no aceptado.
3. $A = 1$ y $C = 1$: destino presente y marco copiado.

Este arreglo proporciona un acuse automático para cada marco. Si un marco es rechazado pero la estación aún está presente, el transmisor tiene la opción de intentar de nuevo posteriormente. Los bits *A* y *C* están presentes dos veces en el *estado de marco* para aumentar la confiabilidad, puesto que no están cubiertos por la suma de verificación.

El *delimitador final* contiene un bit *E* que se establece cuando cualquier interfaz detecta un error (por ejemplo, un patrón no Manchester donde no está permitido). También contiene un bit que puede servir para marcar el último marco de una secuencia lógica, algo parecido a un bit de fin de archivo.

El protocolo 802.5 tiene un elaborado esquema para manejar marcos con distintas prioridades. El marco de ficha de 3 bytes contiene un campo en el byte de enmedio que indica la prioridad de la ficha. Cuando una estación quiere transmitir un marco de prioridad n, debe esperar hasta que pueda capturar una ficha cuya prioridad sea menor o igual que n. Además, al pasar un marco de datos, una estación puede tratar de reservar la siguiente ficha escribiendo la prioridad del marco que quiere enviar en los *bits de reservación* del marco. Sin embargo, si se ha reservado ahí una prioridad mayor, la estación podría no hacer la reservación. Cuando el marco actual ha terminado, se genera la siguiente ficha con la prioridad reservada.

Un breve análisis demostrará que este mecanismo actúa como un trinquete, siempre aumentando la prioridad de la reservación. Para eliminar este problema, el protocolo incluye algunas reglas complicadas. La esencia del concepto es que la estación que eleva la prioridad tiene la obligación de disminuirla nuevamente cuando haya terminado.

Nótese que este esquema de prioridad es sustancialmente diferente del esquema del *token bus*, en el que cada estación siempre obtiene una parte equitativa del ancho de banda, sin importar

lo que hagan otras estaciones. En el *token ring*, una estación que sólo tiene marcos de prioridad baja puede morir de inanición esperando la aparición de una ficha de prioridad baja. Ciertamente ambos comités tenían gustos distintos en cuanto a sacrificar la justicia para todas las estaciones en aras de un buen servicio para el tráfico de prioridad alta.

Mantenimiento del anillo

El protocolo de *token bus* se esfuerza mucho por efectuar el mantenimiento del anillo de una manera completamente descentralizada. El protocolo de *token ring* maneja el mantenimiento de manera bastante diferente. Cada *token ring* tiene una **estación monitor** que supervisa el anillo. Si se inactiva el monitor, un protocolo de contención asegura que otra estación será electa rápidamente como monitor. (Cada estación tiene la capacidad de convertirse en monitor.) Cuando el monitor funciona adecuadamente, sólo él es responsable de ver que el anillo opere de manera correcta.

Cuando el anillo se activa o alguna estación nota que no hay monitor, puede transmitir un marco de control SOLICITAR FICHA. Si este marco recorre todo el anillo antes del envío de otros marcos SOLICITAR FICHA, el transmisor se convierte en el nuevo monitor (cada estación tiene capacidad de monitor integrada). Los marcos de control de *token ring* se muestran en la figura 4-31.

Campo de control	Nombre	Significado
00000000	Prueba dirección duplicada	Prueba si dos estaciones tienen la misma dirección
00000010	Indicación o faro	Usado para localizar rupturas en el anillo
00000011	Solicitar ficha	Intenta convertirse en monitor
00000100	Purga	Reinicializa el anillo
00000101	Monitor activo presente	Emitido periódicamente por el monitor
00000110	Monitor en espera presente	Anuncia la presencia de monitores potenciales

Figura 4-31. Marcos de control del *token ring*.

Entre las funciones del monitor están ver que no se pierda la ficha, emprender acciones cuando se rompe el anillo, limpiar el anillo después de la aparición de marcos alterados y buscar marcos huérfanos. Aparece un marco huérfano cuando una estación transmite un marco corto completo por un anillo grande y luego se inactiva o apaga antes de que el marco pueda ser drenado. Si no se hace nada, el marco circulará eternamente.

Para buscar fichas perdidas, el monitor tiene un temporizador que se establece al mayor intervalo posible sin ficha: cuando cada estación transmite durante el tiempo completo de retención de la ficha. Si se vence este temporizador, el monitor drena el anillo y envía una ficha nueva.

Al aparecer un marco alterado, el monitor puede detectarlo por su formato no válido o su suma de comprobación, abrir el anillo para drenarlo, y enviar una ficha nueva cuando el anillo se haya limpiado. Por último, el monitor detecta marcos huérfanos estableciendo el bit de *monitor* del byte de *control de acceso* cada vez que éste pasa por él. Si un marco que llega tiene este bit establecido, algo anda mal, pues significa que este mismo marco ha pasado dos veces por el monitor sin ser drenado, así que el monitor lo drena.

Una última función del monitor tiene que ver con la longitud del anillo. La ficha tiene 24 bits de longitud, lo que significa que el anillo debe tener el tamaño suficiente para contener 24 bits. Si los retardos de 1 bit en las estaciones más la longitud del cable suman menos de 24 bits, el monitor inserta bits de retardo adicionales para que pueda circular una ficha.

Una función de mantenimiento que no puede manejar el monitor es la localización de rupturas en el anillo. Cuando una estación nota que cualquiera de sus vecinas parece estar muerta, transmite un marco INDICACIÓN O FARO (*Beacon*) dando la dirección de la estación que tal vez está muerta. Cuando la señal se ha propagado tan lejos como puede, es posible ver cuántas estaciones están desactivadas y eliminarlas del anillo usando los relevadores de paso del centro de alambrado, todo sin intervención humana.

Es instructivo comparar los enfoques adoptados para controlar el *token bus* y el *token ring*. El comité del 802.4 le tenía pavor a los componentes centralizados que pudieran fallar de alguna manera inesperada y tirar el sistema al caerse ellos. Por tanto, diseñaron un sistema en el que el poseedor de la ficha actual tiene poderes especiales (por ejemplo, pedir solicitudes de ingreso al anillo), pero ninguna estación se distinguía de las demás en otros aspectos (por ejemplo, las responsabilidades administrativas de mantenimiento asignadas en el momento).

El comité del 802.5, por otro lado, pensó que tener un monitor centralizado haría mucho más sencillo el manejo de las fichas perdidas, los marcos huérfanos, etc. Además, en un sistema normal, las estaciones pocas veces se inactivan involuntariamente, por lo que tener que manejar ocasionalmente una contienda por un monitor nuevo no representa gran sufrimiento. El precio que se paga es que, si el monitor alguna vez realmente enloquece pero continúa enviando periódicamente marcos de control de MONITOR ACTIVO PRESENTE, ninguna estación se lo disputará. Los monitores no pueden ser impugnados.

Esta diferencia de enfoque surge de las distintas áreas de aplicación que tenían en mente los dos comités. El comité del 802.4 estaba pensando en términos de fábricas con grandes masas de metal en movimiento controladas por computadora. Las fallas de red podrían resultar en daños severos y tenían que evitarse a toda costa. El comité del 802.5 estaba interesado en la automatización de oficinas, donde podía tolerarse una falla de vez en cuando como el precio de tener un sistema más sencillo. Que el 802.4 sea, de hecho, más confiable que el 802.5 es tema de controversia.

4.3.4. Comparación del 802.3, el 802.4 y el 802.5

Al estar disponibles tres LAN diferentes e incompatibles, cada una con distintas propiedades, muchas organizaciones enfrentan la pregunta: ¿cuál debemos instalar? En esta sección veremos los tres estándares de LAN 802, señalando sus virtudes y sus defectos, comparándolos y poniéndolos en contraste.

Para comenzar, vale la pena hacer notar que los tres estándares de LAN usan una tecnología básicamente similar y obtienen un desempeño también similar. Si bien los científicos e ingenieros en computación pueden discutir sin parar los méritos del coaxial contra el par trenzado durante horas si se les permite, a la gente de los departamentos de mercadeo, personal o contabilidad probablemente no le importe mucho una cosa o la otra.

Comencemos por las ventajas del 802.3, el cual es, por mucho, el tipo más usado actualmente, con una base instalada enorme y con una considerable experiencia operativa. El protocolo es sencillo. Pueden instalarse estaciones al vuelo, sin tirar la red. Se usa un cable pasivo y no se requieren módems. Es más, el retardo con carga baja es prácticamente cero (las estaciones no tienen que esperar una ficha; simplemente transmiten de inmediato).

Por otra parte, el 802.3 tiene una componente analógica sustancial. Cada estación tiene que ser capaz de detectar la señal de la estación más débil, aun cuando esté transmitiendo ella misma, y todos los circuitos de detección de colisiones del transceptor son analógicos. Dada la posibilidad de que aborten marcos por colisión, el marco mínimo es de 64 bytes, lo que representa una carga extra sustancial cuando los datos consisten en un solo carácter enviado desde una terminal.

Además, el 802.3 es no determinístico, lo que con frecuencia es inapropiado para el trabajo en tiempo real [aunque es posible cierto trabajo de tiempo real simulando un *token ring* en *software* (Venkatramani y Chiueh, 1995)]. Esta LAN tampoco tiene prioridades. La longitud del cable está limitada a 2.5 km (a 10 Mbps) porque la longitud de ida y vuelta del cable determina el tiempo de ranura, y por tanto el desempeño. A medida que aumenta la velocidad, la eficiencia decae porque los tiempos de transmisión se reducen pero la ranura de contención no (el ancho de ranura es 2τ sin importar la tasa de datos). Alternativamente, el cable tiene que hacerse más corto. También, con carga alta, la presencia de colisiones se vuelve un problema mayor y puede afectar el rendimiento de manera seria.

Consideremos ahora el 802.4, el *token bus*. Esta LAN emplea equipo de televisión por cable altamente confiable, que se puede obtener sin necesidad de un pedido especial de numerosos proveedores. Es más determinístico que el 802.3, aunque las pérdidas repetidas de la ficha en momentos críticos pueden introducir más incertidumbre de la que están dispuestos a admitir sus partidarios. Esta LAN puede manejar marcos mínimos cortos.

El *token bus* también reconoce prioridades y puede configurarse para proporcionar una fracción garantizada del ancho de banda a tráfico con prioridad alta, como la voz digitalizada. Esta LAN también tiene rendimiento y eficiencia excelentes con carga alta, volviéndose efectivamente TDM. Por último, el cable de banda ancha puede manejar múltiples canales, no sólo de datos, sino también de voz y televisión.

Por el lado negativo, los sistemas de banda ancha emplean mucha ingeniería analógica e incluyen módems y amplificadores de banda amplia. El protocolo es extremadamente complicado y tiene un retardo sustancial cuando la carga es baja (las estaciones siempre deben esperar la ficha, aun en un sistema sin tráfico). Por otra parte, el *token bus* no es idóneo para las implementaciones de fibra óptica y tiene una base instalada de usuarios pequeña.

Consideremos ahora el *token ring*. Esta LAN maneja conexiones punto a punto, lo que significa que la ingeniería es más sencilla y puede ser completamente digital. Los anillos pueden construirse usando prácticamente cualquier medio de transmisión desde paloma mensajera hasta

fibra óptica. El par trenzado estándar es barato y sencillo de instalar. El uso de centros de alambrado hace al *token ring* la única LAN que puede detectar y eliminar automáticamente fallas de cableado.

Al igual que en el *token bus*, puede haber prioridades, aunque el esquema no es equitativo. También, como el *token bus*, son posibles los marcos cortos pero, a diferencia suya, también son posibles los arbitrariamente grandes, limitados sólo por el tiempo de retención de la ficha. Por último, el rendimiento y la eficiencia cuando la carga es alta son excelentes, como en el *token bus* y a diferencia del 802.3.

El punto negativo principal es la presencia de una función de supervisión centralizada, que introduce un componente crítico. Aun cuando un monitor muerto puede reemplazarse, uno enfermo puede causar dolores de cabeza. Además, como en todos los esquemas de paso de ficha, siempre hay un retardo a muy baja carga, porque el transmisor debe esperar la ficha.

También vale la pena señalar que ha habido numerosos estudios de las tres LAN. La conclusión principal que podemos sacar de estos estudios es que no podemos sacar conclusiones de ellos. Siempre se puede encontrar un grupo de parámetros que hacen que una de las LAN se vea mejor que las demás. En la mayoría de las circunstancias, las tres se desempeñan bien, por lo que probablemente son más importantes los factores distintos del desempeño al hacer una elección.

4.3.5. Estándar IEEE 802.6: *bus* doble de colas distribuidas

Ninguna de las LAN 802 que hemos estudiado hasta ahora es adecuada para usarse como MAN. Las limitaciones de longitud del cable y los problemas de desempeño cuando se conectan miles de estaciones las limitan a áreas de extensión pequeña. Para redes que cubren una ciudad completa, IEEE definió una MAN, llamada DQDB (*Distributed Queue Dual Bus*, bus doble de colas distribuidas), como el estándar 802.6. En esta sección estudiaremos su funcionamiento. Para información adicional, véase (Kessler y Train, 1992). En (Sadiku y Arvind, 1992) se da una bibliografía que lista 171 artículos sobre el DQDB.

En la figura 1-4 se ilustra la geometría básica del 802.6. Dos *buses* unidireccionales paralelos serpentean a través de la ciudad, con estaciones conectadas a ambos *buses* en paralelo. Cada *bus* tiene un *head-end*, que genera una cadena constante de células de 53 bytes. Cada célula viaja corriente abajo del *head-end*. Cuando la célula llega al final, sale del *bus*.

Cada célula lleva un campo de carga de 44 bytes, haciéndola compatible con algunos modos AAL. Cada célula contiene también dos bits de protocolo, *ocupado*, que se establece para indicar que la célula está ocupada, y *solicitud*, que puede establecerse cuando una estación quiere hacer una solicitud.

Para transmitir una célula, una estación necesita saber si el destino está a la izquierda o a la derecha de ella. Si el destino está a la derecha, el transmisor usa el *bus A*; de otra manera, usa el *bus B*. Los datos se introducen en cualquiera de los dos *buses* usando un circuito OR alambrado, de manera que la falla de una estación no desactive la red.

A diferencia de los otros protocolos de LAN 802, el 802.6 no es ávido. En todos los demás, si una estación tiene la oportunidad de enviar, lo hará. Aquí las estaciones se ponen en cola en el orden en el que van quedando listas para enviar y transmitir, en orden FIFO (*first in, first out*,

el que entra primero sale primero). La parte interesante del protocolo es la manera en la que logra el orden FIFO sin tener una cola central.

La regla básica es que las estaciones son amables: ceden el paso a estaciones más abajo de ellas. Esta amabilidad es necesaria para evitar una situación en la que la estación más cercana al *head-end* simplemente toma todas las células vacías a medida que pasan y las llena, haciendo que las otras más abajo mueran de hambre. Por sencillez, sólo examinaremos la transmisión en el *bus A*, pero la misma historia se aplica también al *bus B*.

Para simular la cola FIFO, cada estación mantiene dos contadores, *RC* y *CD*. *RC* (*Request Counter*, contador de solicitudes) cuenta el número de solicitudes pendientes corriente abajo hasta que la estación misma tiene un marco por enviar. En ese punto, *RC* se copia en *CD*, *RC* se restablece a 0 y ahora cuenta el número de solicitudes hechas después de que la estación quedó lista. Por ejemplo, si $CD = 3$ y $RC = 2$ para la estación *k*, las siguientes tres células vacías que pasen por la estación *k* se reservarán para estaciones corriente abajo, después la estación *k* podrá enviar, luego dos células más se reservarán para estaciones corriente abajo. Por sencillez, supondremos que una estación puede tener lista a la vez sólo una célula para transmisión.

Para enviar una célula, una estación primero debe hacer una reservación estableciendo el bit de *solicitud* en alguna célula en el *bus* de reversa (es decir, en el *bus B* para una transmisión que luego ocurrirá en el *bus A*). A medida que se propaga esta célula a través del *bus* en reversa, cada estación en el camino lo nota y aumenta su *RC*. Para ilustrar este concepto, usaremos un ejemplo. Inicialmente, todos los contadores *RC* están en 0, y no hay células en cola, como se muestra en la figura 4-32(a). Entonces, la estación *D* hace una solicitud, que causa que las estaciones *C*, *B* y *A* incrementen sus contadores *RC*, como se muestra en la figura 4-32(b). Tras eso, *B* hace una solicitud, copiando su valor *RC* actual en *CD* y conduciendo a la situación de la figura 4-32(c).

En este punto, el *head-end* del *bus A* genera una célula vacía. Cuando la célula pasa por *B*, esa estación ve que su $CD > 0$, por lo que no puede usar la célula vacía. (Cuando una estación tiene una célula en cola, *CD* representa su posición en la cola, siendo 0 el inicio de la cola.) En cambio, la estación disminuye *CD*. Cuando la célula aún vacía llega a *B*, esa estación ve que $CD = 0$, lo que significa que no hay nadie adelante de ella en la cola, por lo que hace un OR de sus datos en la célula y establece el bit de *ocupado*. Una vez que se ha efectuado la transmisión, tenemos la situación de la figura 4-32(d).

Cuando se genera la siguiente célula vacía, la estación *D* ve que ahora está al frente de la cola, y toma la célula (estableciendo un bit), como se ilustra en la figura 4-32(e). De esta manera, las estaciones se ponen en cola para tomar turnos, sin un administrador centralizado de colas.

Los sistemas DQDB ya están siendo instalados por muchas portadoras a través de ciudades enteras; típicamente operan hasta a 160 km a velocidades de 44.736 Mbps (T3).

4.3.6. Estándar IEEE 802.2: control lógico de enlace

Tal vez ahora sea el momento de dar un paso atrás y comparar lo que hemos aprendido en este capítulo con lo que estudiamos en el anterior. En el capítulo 3 vimos la manera en que dos máquinas se podían comunicar de manera confiable a través de una línea inestable usando varios

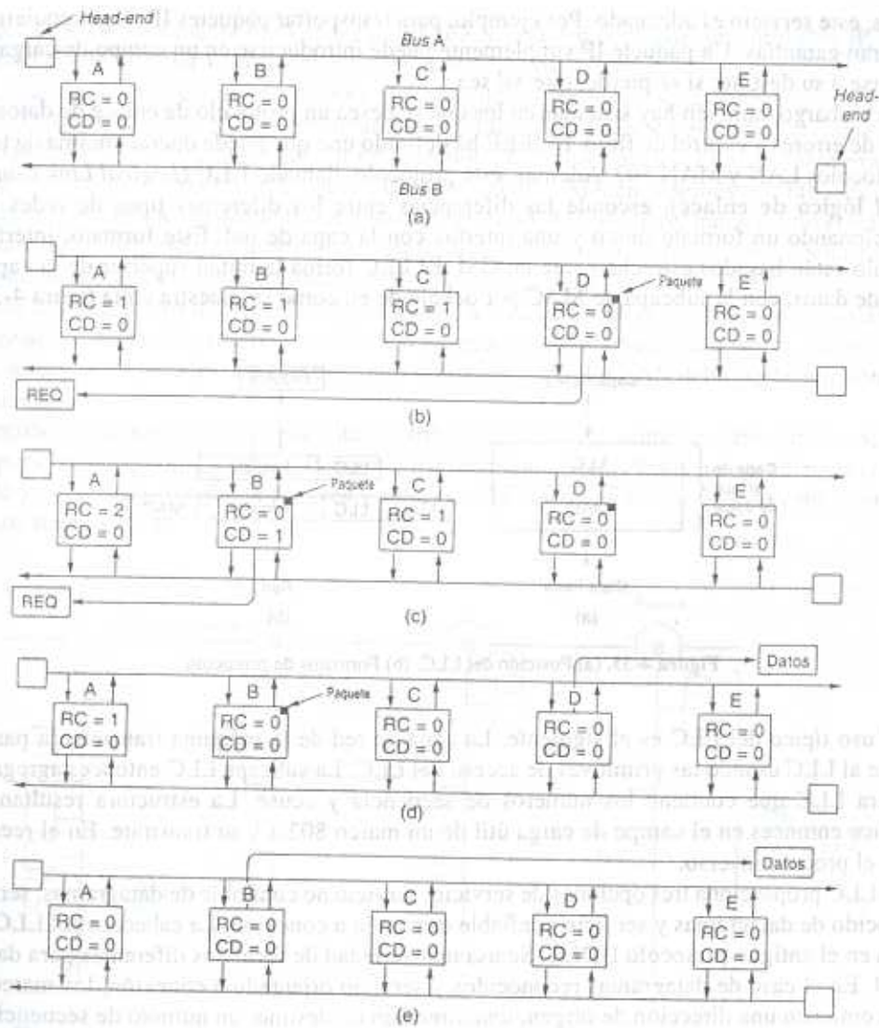


Figura 4-32. (a) Inicialmente, la MAN está inactiva. (b) Después de que D hace una solicitud. (c) Después de que B hace una solicitud. (d) Después de que D transmite. (e) Después de que B transmite.

protocolos de enlace de datos. Estos protocolos proporcionaban control de errores (mediante acuses) y control de flujo (usando una ventana corrediza).

En contraste, en este capítulo no hemos mencionado las comunicaciones confiables. Todo lo que ofrecen las LAN y las MAN 802 es un servicio de datagramas que hace su mejor esfuerzo.

A veces, este servicio es adecuado. Por ejemplo, para transportar paquetes IP no se requieren ni se esperan garantías. Un paquete IP simplemente puede introducirse en un campo de carga 802 y enviarse a su destino; si se pierde, que así sea.

Sin embargo, también hay sistemas en los que se desea un protocolo de enlace de datos con control de errores y control de flujo. El IEEE ha definido uno que puede operar encima de todos los protocolos LAN y MAN 802. Además, este protocolo, llamado **LLC** (*Logical Link Control*, **control lógico de enlace**), esconde las diferencias entre los diferentes tipos de redes 802, proporcionando un formato único y una interfaz con la capa de red. Este formato, interfaz y protocolo están basados estrechamente en OSI. El LLC forma la mitad superior de la capa de enlace de datos, con la subcapa de MAC por debajo de él, como se muestra en la figura 4-33.

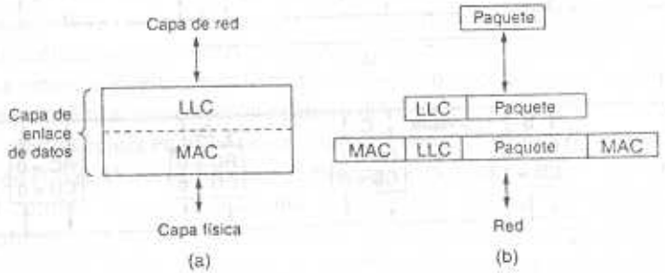


Figura 4-33. (a) Posición del LLC. (b) Formatos de protocolo.

El uso típico del LLC es el siguiente. La capa de red de la máquina transmisora pasa un paquete al LLC usando las primitivas de acceso del LLC. La subcapa LLC entonces agrega una cabecera LLC que contiene los números de secuencia y acuse. La estructura resultante se introduce entonces en el campo de carga útil de un marco 802.x y se transmite. En el receptor ocurre el proceso inverso.

El LLC proporciona tres opciones de servicio: servicio no confiable de datagramas, servicio reconocido de datagramas y servicio confiable orientado a conexión. La cabecera del LLC está basada en el antiguo protocolo HDLC. Se usa una variedad de formatos diferentes para datos y control. En el caso de datagramas reconocidos y servicio orientado a conexión, los marcos de datos contienen una dirección de origen, una dirección de destino, un número de secuencia, un número de acuse y unos cuantos bits misceláneos. En el servicio de datagramas no confiable, se omiten el número de secuencia y el número de acuse.

4.4. PUENTES

Muchas organizaciones tienen varias LAN y desean conectarlas. Las LAN pueden conectarse mediante dispositivos llamados **puentes** (*bridges*), que operan en la capa de enlace de datos. Este postulado implica que los puentes no examinan la cabecera de la capa de red y que pueden,

por tanto, copiar igualmente bien paquetes IP, IPX y OSI. En contraste, un enrutador IP, IPX u OSI puro puede manejar sólo sus propios paquetes nativos.

En las siguientes secciones veremos el diseño de los puentes, especialmente para conectar LAN 802.3, 802.4 y 802.5. Para un tratamiento detallado de los puentes y temas relacionados, véase (Perlman, 1992). Antes de entrar en la tecnología de los puentes, es importante examinar algunas situaciones comunes en las que se usan los puentes. Mencionaremos seis razones por las que una sola organización puede llegar a tener múltiples LAN. Primero, muchas universidades y departamentos corporativos tienen sus propias LAN, principalmente para interconectar sus propias computadoras personales, estaciones de trabajo y servidores. Dado que las metas de los distintos departamentos difieren, los departamentos escogen LAN diferentes, sin importarles lo que hagan los demás departamentos. Tarde o temprano surge la necesidad de interacción, para lo que se requieren puentes. En este ejemplo, surgieron múltiples LAN debido a la autonomía de sus dueños.

Segunda, la organización puede estar distribuida geográficamente en varios edificios, separados por distancias considerables. Puede ser más económico tener LAN independientes en cada edificio y conectarlas mediante puentes y enlaces infrarrojos que tender un solo cable coaxial a través de toda la zona.

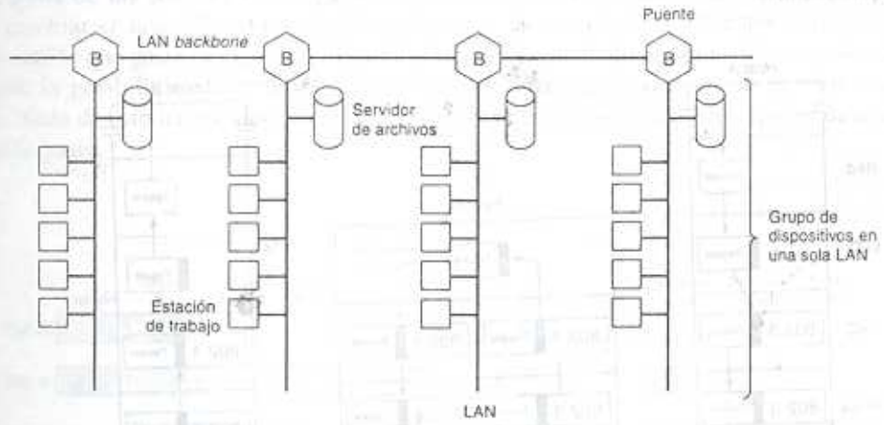


Figura 4-34. Varias LAN conectadas mediante un *backbone* para manejar una carga total mayor que la capacidad de una sola LAN.

Tercera, puede ser necesario dividir lo que lógicamente es una sola LAN en LAN individuales para manejar la carga. Por ejemplo, en muchas universidades miles de estaciones de trabajo están disponibles para los estudiantes y el cuerpo docente. Los archivos normalmente se guardan en máquinas servidoras de archivos, y son descargados a las máquinas de los usuarios a solicitud. La enorme escala de este sistema hace imposible poner todas las estaciones de trabajo en una sola LAN, pues el ancho de banda necesario es demasiado grande. En cambio, se usan varias

LAN conectadas mediante puentes, como se muestra en la figura 4-34. Cada LAN contiene un grupo de estaciones de trabajo con su propio servidor de archivos, por lo que la mayor parte del tráfico está restringida a una sola LAN y no agrega carga al *backbone*.

Cuarta, en algunas situaciones una sola LAN sería adecuada en términos de la carga, pero la distancia física entre las máquinas más distantes es demasiado grande (por ejemplo, mayor que 2.5 km para el 802.3). Aun si fuera fácil tender el cable, la red no funcionaría debido al retardo excesivamente grande del viaje de ida y vuelta. La única solución es segmentar la LAN e instalar puentes entre los segmentos. Usando puentes, puede aumentarse la distancia física total cubierta.

Quinta, está la cuestión de la confiabilidad. En una sola LAN, un nodo defectuoso que envíe constantemente una cadena de basura echará a perder la LAN. Pueden introducirse puentes en lugares críticos, como puertas para bloquear el fuego en un edificio, y así evitar que un solo nodo enloquecido tire el sistema completo. A diferencia de un repetidor, que sólo copia lo que ve, un puente puede programarse para ejercer algún arbitrio respecto a lo que envía y lo que no.

Sexta y última, los puentes pueden contribuir a la seguridad de la organización. La mayor parte de las interfaces de LAN tienen un **modo promiscuo**, en el que *todos* los marcos se entregan a la computadora, no sólo aquellos dirigidos a ella. Los espías y los intrusos aman esta característica. Al introducir puentes en varios lugares y tener cuidado de no reenviar tráfico delicado, es posible aislar partes de la red de manera que su tráfico no pueda escapar y caer en las manos equivocadas.

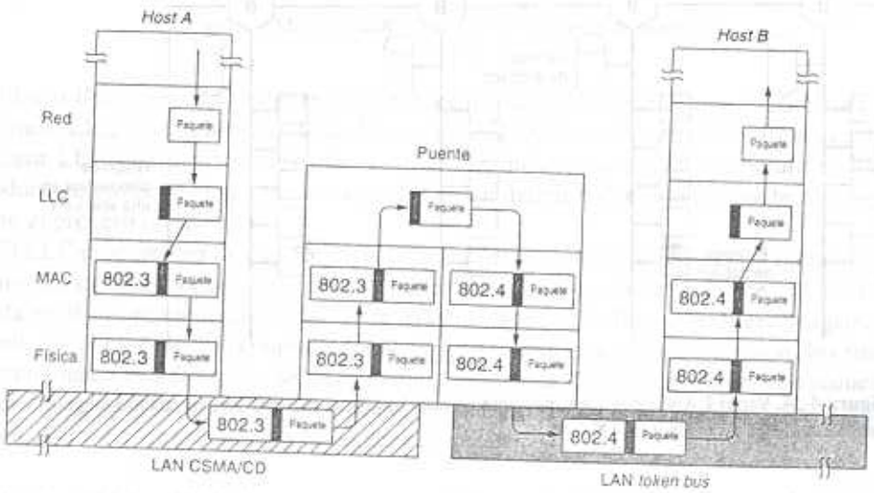


Figura 4-35. Operación de un puente LAN de 802.3 a 802.4.

Habiendo constatado la necesidad de los puentes, pasemos a la cuestión de su funcionamiento. En la figura 4-35 se ilustra la operación de un puente sencillo de dos puertos. El *host A*

tiene un paquete por enviar. El paquete desciende a la subcapa LLC y adquiere una cabecera LLC; luego pasa a la subcapa MAC y se le añade una cabecera 802.3 (también un apéndice, que no se muestra en la figura). Esta unidad sale al cable y en algún momento se pasa hacia arriba a la subcapa MAC del puente, donde se le quita la cabecera 802.3. El paquete solo (con la cabecera LLC) se entrega entonces a la subcapa LLC del puente. En este ejemplo, el paquete está destinado a una subred 802.4 conectada al puente, por lo que pasa al lado 802.4 del puente y sale por ahí. Nótese que un puente que conecta k LAN diferentes tendrá k subcapas MAC y k capas físicas, una para cada tipo.

4.4.1. Puentes de 802.x a 802.y

Usted podría pensar ingenuamente que un puente entre una LAN 802 y otra sería completamente trivial. Éste no es el caso. En el resto de esta sección indicaremos algunas de las dificultades que se presentan al tratar de construir un puente entre las distintas LAN 802.

Cada una de las nueve combinaciones de 802.x a 802.y tiene su propio grupo de problemas. Sin embargo, antes de tratar éstos uno por uno, veamos algunos problemas generales comunes a todos los puentes. Para comenzar, cada una de las LAN usa un formato de marco diferente (véase la figura 4-36). No hay razón técnica válida para esta incompatibilidad. Simplemente es que ninguna de las corporaciones que promovieron los tres estándares (Xerox, GM e IBM) quería cambiar *el suyo*. Como resultado, cualquier copiado entre las diferentes LAN requiere reformateo, lo que gasta tiempo de CPU, obliga a un nuevo cálculo de suma de comprobación e introduce la posibilidad de errores no detectados debido a bits erróneos en la memoria del puente. Nada de esto habría sido necesario si los tres comités hubieran sido capaces de acordar un solo formato.

Figura 4-36. Formatos de marco de los IEEE 802.

Un segundo problema es que las LAN interconectadas no necesariamente operan con la misma tasa de datos. Al enviar un grupo grande de marcos uno tras otro de una LAN rápida a una lenta, el puente no será capaz de deshacerse de los marcos a la misma velocidad a la que llegan; tendrá que ponerlos en *buffers*, esperando que no se le acabe la memoria. El problema también existe en alguna medida del 802.4 a 802.3 a 10 Mbps, pues se pierde un poco del ancho

de banda del 802.3 debido a colisiones. En realidad el 802.3 no tiene 10 Mbps, en tanto que el 802.4 sí lo tiene (bueno, casi). Los puentes que conectan a tres o más LAN tienen un problema parecido cuando varias LAN intentan alimentar simultáneamente a la misma LAN de salida.

Un problema sutil, pero importante, relacionado con el problema del puente como cuello de botella es el valor de los temporizadores (*timers*) en las capas superiores. Supóngase que la capa de red de la LAN 802.4 está tratando de enviar un mensaje muy largo como secuencia de marcos. Tras enviar el último, esta capa inicia un temporizador para esperar un acuse. Si el mensaje tiene que transitar por un puente a una LAN 802.5 más lenta, hay el peligro de que el temporizador termine antes de que el último marco haya sido reenviado a la LAN más lenta. La capa de red supondrá que el problema se debe a un marco perdido y simplemente retransmitirá de nuevo la secuencia completa. Tras n intentos fallidos, podrá darse por vencida e indicar a la capa de transporte que el destino está muerto.

Un tercer problema, que puede ser el más grave de todos, es que las tres LAN 802 tienen una longitud máxima de marco diferente. En el 802.3 esta longitud depende de los parámetros de la configuración pero, para el sistema estándar de 10 Mbps, la carga útil máxima es de 1500 bytes. En el 802.4 la longitud está fija en 8191 bytes. Para el 802.5 no hay límite superior, excepto que una estación no puede transmitir durante un tiempo mayor que el de retención de la ficha. Con el valor predeterminado de 10 mseg, la longitud máxima de marco es de 5000 bytes.

Surge un problema obvio cuando debe reenviarse un marco grande a una LAN que no puede aceptarlo. En esta capa es imposible la división del marco en partes. Todos los protocolos suponen que los marcos llegan o no. No se contempla el reensamblado de los marcos a partir de unidades más pequeñas. Esto no quiere decir que no se podrían desarrollar tales protocolos; se puede, y se ha hecho. La cuestión es que el 802 no tiene esta característica. Básicamente, no hay solución. Los marcos que son demasiado grandes para reenviarse deben descartarse. Así que habrá que olvidarse de la transparencia.

Consideremos ahora brevemente cada uno de los nueve casos de puentes 802.x a 802.y para ver qué otros problemas acechan entre las sombras. Del 802.3 al 802.3 es muy sencillo. Lo único que puede fallar es que la LAN de destino esté tan cargada que los marcos sigan entrando en el puente, pero el puente no pueda deshacerse de ellos. Si persiste esta situación durante un tiempo suficiente, se puede agotar el espacio de *buffer* del puente, y éste comenzará a tirar marcos. Dado que este problema siempre está potencialmente presente al reenviar a una LAN 802.3, no lo mencionaremos más. Con las otras dos LAN, cada estación, incluido el puente, tiene la garantía de adquirir la ficha periódicamente y no puede quedar excluido durante intervalos grandes.

Del 802.4 al 802.3 hay dos problemas. Primero, los marcos 802.4 llevan bits de prioridad que los marcos 802.3 no tienen. Como resultado, si dos LAN 802.4 se comunican mediante una LAN 802.3, la prioridad se perderá en la LAN intermedia.

El segundo problema es causado por una característica específica del 802.4: la entrega temporal de la ficha. Es posible que un marco 802.4 tenga un bit de cabecera puesto en 1 para pasar temporalmente la ficha al destino, a fin de permitirle enviar un marco de acuse. Sin embargo, si se reenvía uno de tales marcos desde un puente, ¿qué debe hacer el puente? Si envía él mismo un marco de acuse, estará mintiendo, pues en realidad aún no ha sido entregado el marco. De hecho, el destino puede estar muerto.

Por otra parte, si el puente no genera el acuse, el transmisor casi con seguridad concluirá que el destino está muerto e informará de la falla a sus superiores. No parece haber ninguna manera de resolver este problema.

Del 802.5 al 802.3 tenemos un problema parecido. El formato de marco 802.5 tiene los bits *A* y *C* en el byte de estado del marco. Estos bits los establece el destino para indicar al transmisor si la estación direccionada vio el marco, y si lo copió. Nuevamente, el puente puede mentir y decir que el marco ha sido copiado pero, si luego resulta que el destino está inactivo, pueden ocurrir problemas serios. En esencia, la inserción de un puente en la red ha cambiado la semántica de los bits. Es difícil imaginar una solución adecuada a este problema.

Del 802.3 al 802.4 tenemos el problema de qué poner en los bits de prioridad. Parece bastante sensato hacer que el puente retransmita todos los marcos a la prioridad máxima, dado que probablemente ya han sufrido suficiente retardo.

Del 802.4 al 802.4 el único problema es qué hacer con la entrega temporal de la ficha. Cuando menos, aquí tenemos la posibilidad de que el puente logre reenviar el marco a una velocidad suficiente para obtener la respuesta antes de que termine el temporizador. Aun así, es un riesgo. Al reenviar el marco a la prioridad más alta, el puente miente inocentemente, pero de esa manera aumenta la probabilidad de obtener la respuesta a tiempo.

Del 802.5 al 802.4 tenemos el mismo problema que antes con los bits *A* y *C*. También la definición de los bits de prioridad es distinta para las dos LAN, pero los mendigos no pueden escoger. Al menos las dos LAN tienen el mismo número de bits de prioridad. Todo lo que puede hacer el puente es copiar y pasar los bits de prioridad, y esperar lo mejor.

Del 802.3 al 802.5, el puente debe generar bits de prioridad, pero no hay otros problemas especiales. Del 802.4 al 802.5 hay un problema potencial con marcos que son demasiado grandes, y se presenta nuevamente el problema de la entrega de la ficha. Finalmente, del 802.5 al 802.5 el problema es qué hacer nuevamente con los bits *A* y *C*. En la figura 4-37 se resumen los distintos problemas que hemos analizado.

Cuando el comité del IEEE 802 se dio a la tarea de desarrollar un estándar de LAN, fue incapaz de acordar un estándar único, por lo que produjo *tres* estándares incompatibles, como acabamos de ver con algún detalle. Por este fracaso el comité ha sido unánimemente criticado. Cuando posteriormente se asignó al mismo comité la tarea de diseñar un estándar de puentes para interconectar sus tres LAN incompatibles, decidió hacerlo mejor y lo hizo: desarrolló *dos* diseños de puente incompatibles. Hasta ahora nadie le ha pedido diseñar un estándar de pasarela entre sus dos puentes incompatibles, pero cuando menos la tendencia es en la dirección correcta.

Esta sección ha tratado los problemas que surgen al conectar dos LAN IEEE 802 mediante un solo puente. Las siguientes dos secciones tratan los problemas de la conexión de interredes grandes que contienen muchas LAN y muchos puentes, y los dos enfoques del IEEE para diseñar estos puentes.

LAN de origen \ LAN de destino	802.3 (CSMA/CD)	802.4 (*Token bus*)	802.4 (*Token ring*)
802.3		1, 4	1, 2, 4, 8
802.4	1, 5, 8, 9, 10	9	1, 2, 3, 8, 9, 10
802.5	1, 2, 5, 6, 7, 10	1, 2, 3, 6, 7	6, 7

Acciones:
1. Reformatea el marco y calcula suma de comprobación nueva.
2. Invierte el orden de bits.
3. Copia la prioridad, sea significativa o no.
4. Genera una prioridad ficticia.
5. Desecha la prioridad.
6. Drena el anillo (de alguna manera).
7. Establece los bits A y C (mintiendo).
8. Preocupación por los congestionamientos (LAN rápida a LAN lenta).
9. Preocupación por el retardo o la imposibilidad del reconocimiento de la entrega de la ficha.
10. Pánico si el marco es demasiado grande para la LAN de destino.

Parámetros supuestos:
802.3: marcos de 1500 bytes 10 Mbps (menos colisiones)
802.4: marcos de 8191 bytes 10 Mbps
802.5: marcos de 5000 bytes 4 Mbps

Figura 4-37. Problemas encontrados en la construcción de puentes entre 802.x y 802.y.

4.4.2. Puentes transparentes

El primer puente 802 es un puente transparente o puente de árbol de extensión (Perlman, 1992). La preocupación principal de la gente que apoyó este diseño fue la transparencia completa. Desde su punto de vista, una instalación con varias LAN debería ser capaz de salir y comprar puentes diseñados para el estándar IEEE, enchufar los conectores a los puentes, y todo debería funcionar perfectamente al instante. No deberían requerirse cambios de *hardware*, cambios de *software*, ajustes de conmutadores de direccionamiento, carga de tablas o parámetros de enrutamiento, nada. Simplemente conectar los cables y dedicarse a otro asunto. Además, la operación de las LAN existentes no debería ser afectada por los puentes en lo absoluto. Sorprendente, pero lo lograron.

Un puente transparente trabaja en modo promiscuo, aceptando todo marco transmitido por cualquier LAN a la que está conectado. Por ejemplo, considere la configuración de la figura 4-38. El puente B1 está conectado a las LAN 1 y 2, y el puente B2 está conectado a las LAN 2, 3 y 4. Un marco que llega al puente B1 de la LAN 1, destinado a *A*, puede descartarse de inmediato, puesto que ya está en la LAN adecuada, pero un marco que llegue para *C* o *F* por la LAN 1 se debe reenviar.

Al llegar un marco, un puente debe decidir si lo descarta o lo reenvía y, de ser lo segundo, en qué LAN debe poner el marco. Esta decisión se toma buscando la dirección de destino en una

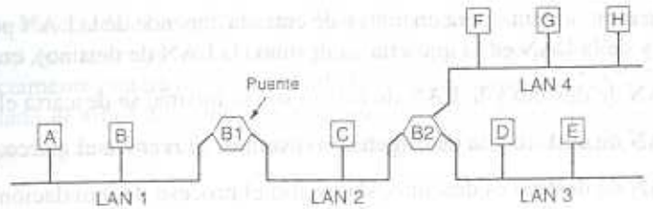


Figura 4-38. Configuración con cuatro LAN y dos puentes.

gran tabla (de dispersión) contenida en el puente. La tabla puede listar cada posible destino e indicar a qué línea de salida (LAN) pertenece. Por ejemplo, la tabla de B2 indicaría que *A* pertenece a la LAN 2, ya que todo lo que B2 tiene que saber es en qué LAN tiene que poner los marcos para *A*. El que se reenvíe posteriormente no le interesa.

Al conectarse inicialmente los puentes, todas las tablas de dispersión están vacías. Ninguno de los puentes sabe la ubicación de ninguno de los destinos, por lo que usa el algoritmo de inundación: cada marco de entrada para un destino desconocido se envía a todas las LAN a las que está conectado el puente, excepto a aquella por la que llegó. A medida que pasa el tiempo, los puentes aprenden la ubicación de los destinos, como se describe más adelante. Una vez que se conoce un destino, los marcos destinados a él se colocan en la LAN apropiada y ya no se efectúa el proceso de inundación.

El algoritmo empleado por los puentes transparentes es de **aprendizaje de lo sucedido**. Como se mencionó antes, los puentes operan en modo promiscuo, por lo que ven cada marco enviado por cualquiera de sus LAN. Examinando la dirección de origen, pueden saber qué máquina está accesible desde qué LAN. Por ejemplo, si el puente B1 de la figura 4-38 ve un marco en la LAN 2 que viene de *C*, sabe que se puede llegar a *C* a través de la LAN 2, por lo que hace una entrada en su tabla de dispersión anotando que los marcos que van a *C* deben usar la LAN 2. Cualquier marco subsiguiente dirigido a *C* que entre por la LAN 1 será reenviado, pero un marco para *C* que llegue por la LAN 2 será descartado.

La topología puede cambiar a medida que las máquinas y los puentes se encienden, se apagan y se mueven. Para manejar topologías dinámicas, cada vez que se hace una entrada en la tabla de dispersión se anota la hora de llegada del marco en la entrada. Cada vez que llega un marco cuyo destino ya está en la tabla, se actualiza su entrada con la hora actual. Por tanto, el tiempo asociado a cada entrada indica la última vez que se vio un marco de esa máquina.

Periódicamente, un proceso del puente recorre la tabla de dispersión y elimina las entradas que tienen más de unos cuantos minutos. De esta manera, si una computadora se desconecta de su LAN, se mueve a otra parte del edificio y se reconecta en otro lado, en pocos minutos entrará de nuevo en operación normal, sin intervención manual. Este algoritmo también significa que, si una máquina está quieta durante algunos minutos, cualquier tráfico enviado a ella tendrá que pasar por el proceso de inundación, hasta que ella misma envíe un marco.

El proceso de enrutamiento para un marco de entrada depende de la LAN por la que llega (la LAN de origen) y de la LAN en la que está su destino (la LAN de destino), como sigue:

1. Si la LAN de destino y la LAN de origen son la misma, se descarta el marco.
2. Si la LAN de destino y la de origen son distintas, se reenvía el marco.
3. Si la LAN de destino es desconocida, se usa el proceso de inundación.

A medida que llega cada marco, debe aplicarse este algoritmo. Existen *chips* VLSI especiales para realizar la búsqueda y actualización de la entrada de tabla, todo en unos cuantos microsegundos.

A fin de aumentar la confiabilidad, algunas instalaciones tienen dos o más puentes en paralelo entre pares de LAN, como se muestra en la figura 4-39. Este arreglo, sin embargo, también agrega algunos problemas adicionales, pues crea ciclos en la topología.

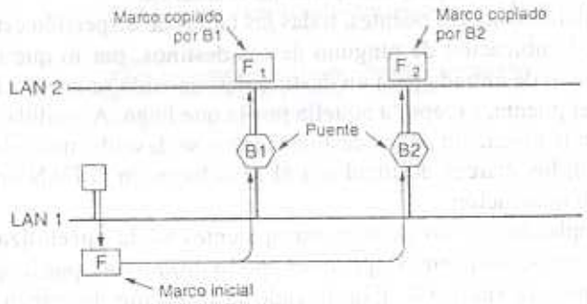


Figura 4-39. Dos puentes transparentes paralelos.

Un ejemplo sencillo de estos problemas es evidente al observar la manera en que se maneja un marco F con un destino desconocido en la figura 4-39. Cada puente, siguiendo las reglas normales de manejo de destinos desconocidos, usa inundación, que en este ejemplo significa simplemente copiar el marco en la LAN 2. Poco después, el puente 1 ve a F_2, un marco con un destino desconocido, el cual copia en la LAN 1, generando F_3 (que no se muestra). Del mismo modo, el puente 2 copia F_1 en la LAN 1, generando F_4 (que tampoco se muestra). El puente 1 ahora reenvía F_4 y el puente 2 copia F_3. Este ciclo continúa indefinidamente.

Puentes de árbol de extensión

La solución para este problema es comunicar los puentes entre sí y extender sobre la topología actual un árbol de extensión que alcance todas las LAN. En efecto, se ignoran algunas conexiones potenciales entre las LAN por el interés de construir una topología ficticia libre de ciclos. Por ejemplo, en la figura 4-40(a) vemos nueve LAN interconectadas por 10 puentes. Esta configuración puede abstraerse en un grafo con las LAN como nodos. Un arco conecta cada par

de LAN que están conectadas por un puente. El grafo puede reducirse a árbol de extensión eliminando los arcos que aparecen como líneas punteadas en la figura 4-40(b). En este árbol de extensión hay exactamente una trayectoria de cada LAN a cualquier otra LAN. Una vez que los puentes han acordado el árbol de extensión, todo el reenvío entre las LAN sigue al árbol de extensión. Dado que hay una trayectoria única de cada origen a cada destino, es imposible que ocurran ciclos.

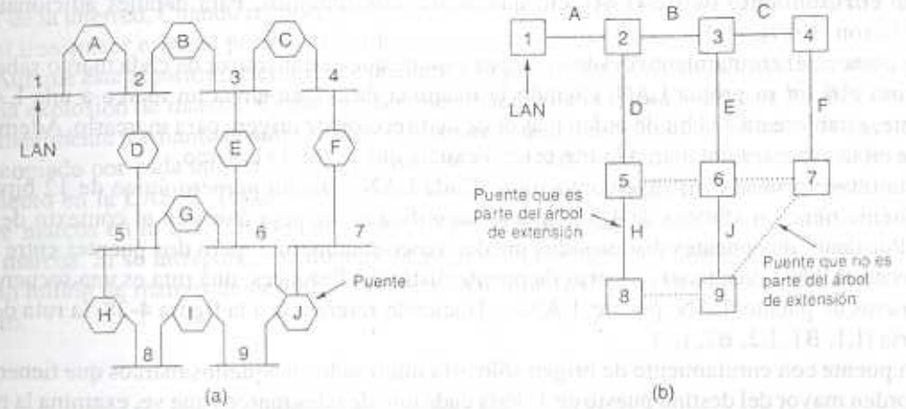


Figura 4-40. (a) LAN interconectadas. (b) Árbol de extensión que cubre las LAN. Las líneas punteadas no son parte del árbol.

Para construir el árbol de extensión, los puentes primero tienen que escoger un puente que sea la raíz del árbol; deben tomar esta decisión haciendo que cada uno difunda su número de serie, instalado por el fabricante y con garantía de ser único en el mundo. El puente con el número de serie menor se vuelve la raíz. A continuación, se construye un árbol de trayectorias mínimas de la raíz a cada puente y LAN. Este árbol es el árbol de extensión. Si falla un puente o una LAN, se calcula un árbol nuevo.

El resultado de este algoritmo es que se establece una trayectoria única de cada LAN hasta la raíz, y por tanto a todas las demás LAN. Aunque el árbol abarca todas las LAN, no necesariamente están presentes todos los puentes en el árbol (para evitar ciclos). Aun después de que se ha establecido el árbol de extensión, el algoritmo continúa operando a fin de detectar automáticamente cambios de topología y actualizar el árbol. El algoritmo distribuido que se usa para construir el árbol de extensión fue inventado por Perlman y se describe en detalle en (Perlman, 1992).

También pueden construirse puentes para conectar las LAN que están muy separadas. En este modelo, cada instalación consiste en un conjunto de LAN y puentes, de los cuales uno tiene conexión con una WAN. Los marcos para las LAN remotas viajan a través de la WAN. Puede usarse el algoritmo básico de árbol de extensión, de preferencia con ciertas optimizaciones para seleccionar un árbol que minimice la cantidad de tráfico en la WAN.

4.4.3. Puentes de enrutamiento desde el origen

Los puentes transparentes tienen la ventaja de ser fáciles de instalar. Simplemente se enchufan y a otra cosa. Por otra parte, no hacen uso óptimo del ancho de banda, ya que sólo utilizan un subconjunto de la topología (el árbol de extensión). La importancia relativa de estos dos (y otros) factores condujo a una división en el comité 802 (Pitt, 1988). La gente de CSMA/CD y *token bus* escogió el puente transparente. La gente de anillos (con apoyo de IBM) prefirió un esquema llamado **enrutamiento desde el origen**, que ahora describiremos. Para detalles adicionales, véase (Dixon, 1987).

En esencia, el enrutamiento desde el origen supone que el transmisor de cada marco sabe si el destino está en su propia LAN. Cuando la máquina de origen envía un marco a una LAN diferente, establece en 1 el bit de orden mayor de la dirección de origen, para marcarlo. Además, incluye en la cabecera del marco la trayectoria exacta que seguirá el marco.

Esta trayectoria se construye como sigue. Cada LAN tiene un número único de 12 bits, y cada puente tiene un número de 4 bits que lo identifica de manera única en el contexto de su LAN. Por tanto, dos puentes distanciados pueden tener el número 3, pero dos puentes entre las mismas dos LAN deben tener números de puente distintos. Entonces, una ruta es una secuencia de números de puente, LAN, puente, LAN, ... Haciendo referencia a la figura 4-38, la ruta de *A* a *D* sería (L1, B1, L2, B2, L3).

Un puente con enrutamiento de origen sólo está interesado en aquellos marcos que tienen el bit de orden mayor del destino puesto en 1. Para cada uno de tales marcos que ve, examina la ruta buscando el número de la LAN por la que llegó el marco. Si este número de LAN va seguido de su propio número de puente, el puente reenvía el marco a la LAN cuyo número sigue a su número de puente en la ruta. Si el número de la LAN entrante va seguido del número de algún otro puente, no reenvía el marco.

Este algoritmo se presta para tres posibles implementaciones:

1. *Software*: el puente opera en modo promiscuo, copiando todos los marcos en su memoria para ver si tienen establecido en 1 el bit de orden mayor del destino. De ser así, se sigue examinando el marco, de otro modo, no se continúa la inspección.
2. Híbrida: la interfaz de LAN del puente inspecciona el bit de orden mayor del destino y sólo acepta los marcos que tienen el bit establecido. Esta interfaz es fácil de incorporar en el *hardware* y reduce de manera importante la cantidad de marcos que debe inspeccionar el puente.
3. *Hardware*: la interfaz de la LAN no sólo revisa el bit de orden mayor del destino, sino que también examina la ruta para ver si este puente debe reenviar. Sólo los marcos que sí deben reenviarse son entregados al puente. Esta implementación requiere del *hardware* más complejo, pero no desperdicia ciclos de CPU del puente, porque se filtran todos los marcos que no son pertinentes.

Estas tres implementaciones varían considerablemente en costo y desempeño. La primera no tiene un costo adicional de *hardware* por la interfaz, pero puede requerir una CPU muy rápida para manejar todos los marcos. La última no requiere un *chip* VLSI, pues descarga gran parte del procesamiento del puente al *chip*, por lo que puede usarse una CPU más lenta o, alternativamente, el puente puede manejar más LAN.

Implícito en el diseño del enrutamiento desde el origen está el hecho de que cada máquina de la interred conoce o puede encontrar la mejor trayectoria a todas las demás máquinas. La manera de descubrir estas rutas es una parte importante del algoritmo de enrutamiento desde el origen. El concepto básico es que, si el origen desconoce un destino, difunde un marco preguntando dónde está. Este **marco de descubrimiento** es reenviado por cada puente de modo que llegue a todas las LAN de la interred. Cuando regresa la respuesta, los puentes registran en ella su identidad, por lo que el transmisor original puede ver la ruta exacta que siguió y escoger la mejor.

Aunque este algoritmo ciertamente encuentra la mejor ruta (encuentra *todas* las rutas), sufre de una explosión de marcos. Considere la configuración de la figura 4-41, con *N* LAN conectadas linealmente mediante puentes triples. Cada marco de descubrimiento enviado por la estación 1 es copiado por cada uno de los tres puentes de la LAN 1, produciendo tres marcos de descubrimiento en la LAN 2. Todos estos son copiados por cada uno de los puentes, resultando en nueve marcos en la LAN 3. Para el momento en que alcanzamos la LAN *N*, están circulando 3^{N-1} marcos. Si se interpone una docena de grupos de puentes, se tendrán que inyectar más de medio millón de marcos de descubrimiento en la última LAN, causando un congestionamiento severo.

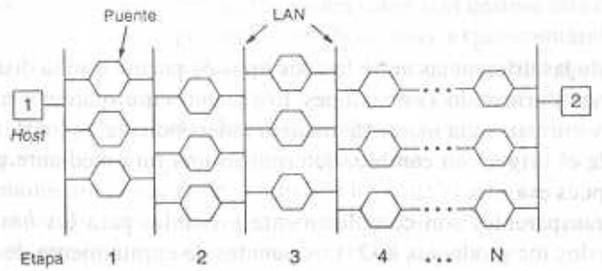


Figura 4-41. Serie de LAN conectadas mediante puentes triples.

Ocurre un proceso análogo con el puente transparente, sólo que no es tan grave. Cuando llega un marco desconocido, pasa por el proceso de inundación, pero sólo a través del árbol de extensión, por lo que el volumen total de marcos enviados es lineal respecto al tamaño de la red, no exponencial.

Una vez que el *host* ha descubierto una ruta a cierto destino, almacena la ruta en una memoria caché, para que el proceso de descubrimiento no tenga que ejecutarse la siguiente vez. Aunque este enfoque limita en gran medida el impacto de la explosión de marcos, impone un poco de carga administrativa a todos los *hosts*, y definitivamente el algoritmo completo no es transparente, lo que fue una de las metas originales, como se mencionó antes.

4.4.4. Comparación de los puentes 802

El puente transparente y el puente de enrutamiento desde el origen tienen sus ventajas y desventajas. En esta sección estudiaremos algunas de las principales que se presentan en resumen en la figura 4-42 y se cubren con mayor detalle en (Soha y Perlman, 1988, y en Zhang, 1988). Sin embargo, cabe señalar que cada uno de los puntos es materia de amplia discusión.

Aspecto	Puente transparente	Puente de enrutamiento desde el origen
Orientación	Sin conexiones	Orientado a conexión
Transparencia	Completamente transparente	No transparente
Configuración	Automático	Manual
Enrutamiento	Subóptimo	Óptimo
Localización	Aprendizaje en reversa	Marcos de descubrimiento
Fallas	Manejado por los puentes	Manejado por los *hosts*
Complejidad	En los puentes	En los *hosts*

Figura 4-42. Comparación del puente transparente y el puente de enrutamiento desde el origen.

En el corazón de las diferencias entre los dos tipos de puente está la distinción entre trabajo en red sin conexiones y orientado a conexiones. Los puentes transparentes no tienen el concepto de circuito virtual y enrutan cada marco de manera independiente de los demás. Los puentes de enrutamiento desde el origen, en cambio, determinan una ruta mediante marcos de descubrimiento y usan después esa ruta.

Los puentes transparentes son completamente invisibles para los *hosts* y son totalmente compatibles con todos los productos 802. Los puentes de enrutamiento desde el origen no son transparentes ni compatibles. Para usar enrutamiento desde el origen, los *hosts* deben estar perfectamente conscientes del esquema de puentes y deben participar activamente en él. La división de una LAN existente en dos LAN conectadas mediante un puente de enrutamiento desde el origen requiere cambios en el *software* del *host*.

Al usar puentes transparentes, no hay necesidad de administración de la red. Los puentes mismos se configuran para la topología de manera automática. Con los puentes de enrutamiento desde el origen, el administrador de la red debe instalar manualmente y los números de LAN de puente. Los errores, como la duplicación de un número de LAN o de puente, pueden ser muy difíciles de detectar y pueden causar que algunos marcos entren en un ciclo, pero no otros en rutas distintas. Además, al conectar mediante puentes transparentes dos interredes que antes no tenían conexión, no hay nada que hacer excepto conectarlas. En cambio, con el enrutamiento desde el origen, puede ser necesario cambiar manualmente muchos números de LAN para hacerlos únicos en la interred combinada.

Una de las pocas ventajas del enrutamiento desde el origen es que, en teoría, puede usar un enrutamiento óptimo, mientras que los puentes transparentes están restringidos al árbol de extensión. Además, el enrutamiento desde el origen también puede aprovechar los puentes paralelos entre dos LAN para dividir la carga. Que los puentes reales sean lo bastante listos como para hacer uso de estas ventajas teóricas es cuestionable.

En el puente transparente, la localización de destinos se hace mediante aprendizaje de lo sucedido, y en los puentes de enrutamiento desde el origen, mediante marcos de descubrimiento. La desventaja del aprendizaje de lo sucedido es que los puentes tienen que esperar hasta que llega un marco de una máquina en particular para saber dónde está esa máquina. La desventaja de los marcos de descubrimiento es la explosión exponencial en interredes de moderadas a grandes con puentes paralelos.

El manejo de fallas es bastante diferente en ambos esquemas. Los puentes transparentes se enteran con rapidez y automáticamente de las fallas de puentes y de LAN, así como de otros cambios de topología, con sólo escuchar los marcos de control de los demás. Los *hosts* no notan estos cambios en lo absoluto.

Con el enrutamiento desde el origen, la situación es bastante distinta. Al fallar un puente, las máquinas que enrutan a través de él notan inicialmente que sus marcos ya no están siendo reconocidos, por lo que terminan su temporización e intentan una y otra vez. Por fin concluyen que algo está mal, pero aún no saben si el problema es con el destino mismo o con la ruta actual. Sólo enviando otro marco de descubrimiento pueden saber si el destino está disponible. Desafortunadamente, al fallar un puente principal, muchos *hosts* experimentarán terminaciones de temporización (*time outs*) y enviarán nuevos marcos de descubrimiento antes de que se resuelva el problema, aun si hay una ruta alterna disponible. Esta mayor vulnerabilidad a las fallas es una de las debilidades principales de todos los sistemas orientados a conexiones.

Por último, llegamos a la complejidad y el costo, temas que provocan gran controversia. Si los puentes de enrutamiento desde el origen tienen un *chip* VLSI que lee sólo aquellos marcos que deben reenviarse, estos puentes experimentarán una carga de procesamiento de marcos menor y tendrán un desempeño mejor por una inversión dada en *hardware*. Sin este *chip*, el desempeño será peor, pues la cantidad de procesamiento por marco (la búsqueda de la ruta en la cabecera del marco) es sustancialmente mayor.

Además, el enrutamiento desde el origen agrega complejidad a los *hosts*: éstos deben almacenar las rutas, enviar marcos de descubrimiento y copiar información de ruta en cada marco. Todo esto requiere memoria y ciclos de CPU. Dado que típicamente los *hosts* superan en número a los puentes en uno a dos órdenes de magnitud, puede ser mejor agregar el costo y la complejidad extra a unos cuantos puentes, en lugar de a todos los *hosts*.

4.4.5. Puentes remotos

Un uso común de los puentes es la conexión de dos (o más) LAN distantes. Por ejemplo, una compañía puede tener plantas en varias ciudades, cada una con su propia LAN. Idealmente, todas las LAN deben estar interconectadas, por lo que el sistema completo actúa como una LAN grande.

Esta meta puede lograrse poniendo un puente en cada LAN y conectando los puentes en pares mediante líneas punto a punto (por ejemplo, líneas arrendadas a una compañía telefónica). En la figura 4-43 se ilustra un sistema sencillo con tres LAN. Aquí se aplican los algoritmos usuales de enrutamiento. La manera más sencilla de ver esto es considerar las tres líneas punto a punto como LAN sin *hosts*. Entonces tenemos un sistema normal de seis LAN interconectadas mediante cuatro puentes. Nada de lo que hemos estudiado hasta ahora indica que una LAN debe tener *hosts*.

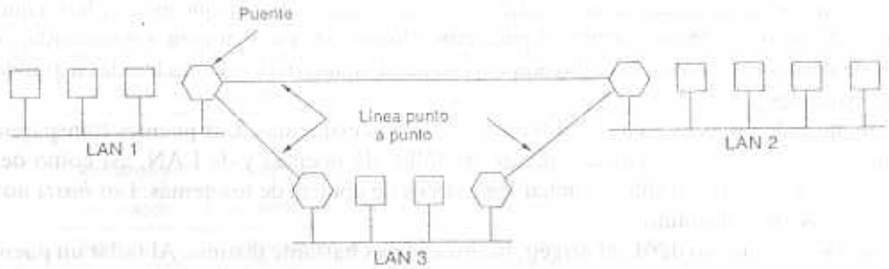


Figura 4-43. Se pueden usar puentes remotos para interconectar LAN distantes.

Se pueden usar varios protocolos en las líneas punto a punto. Una posibilidad es escoger algún protocolo de enlace de datos punto a punto estándar, poniendo marcos de MAC completos en el campo de carga útil. Esta estrategia funciona mejor si todas las LAN son idénticas, y el único problema es lograr que lleguen los marcos a la LAN correcta. Otra opción es eliminar la cabecera y el apéndice de MAC en el puente de origen y poner lo que queda en el campo de carga útil del protocolo punto a punto. En el puente de destino se pueden generar entonces una nueva cabecera y un nuevo apéndice de MAC. Una desventaja de este enfoque es que la suma de comprobación que llega al anfitrión de destino no es la calculada por el *host* de origen, por lo que podrían no detectarse los errores causados por bits erróneos en la memoria de un puente.

4.5. LAN DE ALTA VELOCIDAD

Todas las LAN y MAN 802 que acabamos de estudiar se basan en un alambre de cobre (dos alambres para el 802.6). Para velocidades bajas y distancias cortas esto es suficiente, pero para altas velocidades y mayores distancias las LAN deben basarse en fibra óptica o en redes de cobre altamente paralelas. La fibra tiene un gran ancho de banda, es delgada y ligera, no es afectada por la interferencia electromagnética de maquinaria pesada (lo que es importante cuando el cableado corre por cubos de elevador), descargas eléctricas ni relámpagos, y tiene una seguridad excelente porque es casi imposible hacer una derivación indetectable. En consecuencia, las LAN rápidas por lo regular usan fibra. En las siguientes secciones veremos algunas redes de área local que usan fibra óptica, así como una LAN de velocidad extremadamente alta que usa el tradicional alambre de cobre (aunque en grandes cantidades).

4.5.1. FDDI

La **FDDI** (*Fiber Distributed Data Interface*, **interfaz de datos distribuidos por fibra**) es una LAN *token ring* de fibra óptica de alto desempeño que opera a 100 Mbps y distancias de hasta 200 km con hasta 1000 estaciones conectadas (Black, 1994; Jain, 1994; Mirchandani y Khanna, 1993; Ross y Hamstra, 1993; Shah y Ramakrishnan, 1994, y Wolter, 1990). Esta red puede usarse de la misma manera que cualquiera de las LAN 802 pero, con su gran ancho de banda, otro uso común es como *backbone* para conectar varias LAN de cobre, como se muestra en la figura 4-44. La FDDI-II es la sucesora de la FDDI, modificada para manejar datos PCM sincrónicos de circuitos conmutados para voz o tráfico ISDN, además de datos normales. Nos referiremos a ambas simplemente como FDDI. Esta sección abarca tanto la capa física como la subcapa MAC de FDDI.

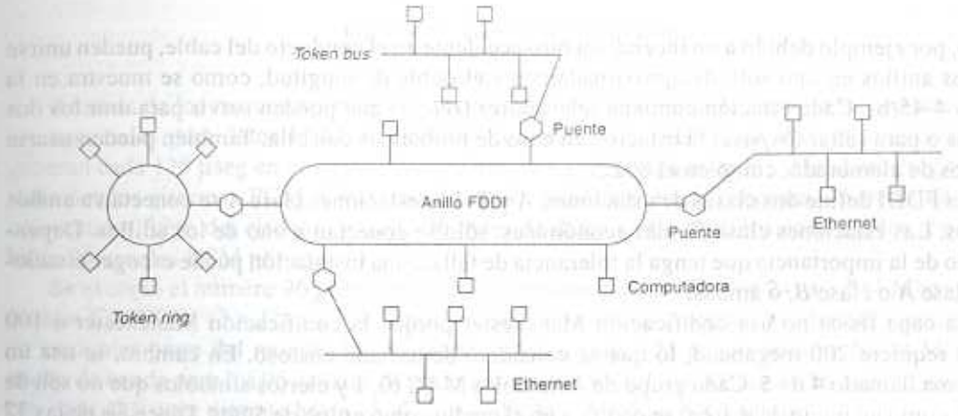


Figura 4-44. Anillo FDDI utilizado como *backbone* para conectar LAN y computadoras.

La FDDI usa fibras multimodo, pues el gasto adicional de las fibras monomodo no es necesario para redes que operan sólo a 100 Mbps; también usa LED (*Light-Emitting Diodes*, **diodos emisores de luz**), en lugar de láser, no sólo debido a su menor costo, sino también porque la FDDI a veces puede conectarse directamente a estaciones de trabajo de usuario. Hay el peligro de que usuarios curiosos ocasionalmente desenchufen el conector de la fibra y miren dentro de ella para ver el paso de los bits a 100 Mbps. Con un láser, el usuario curioso podría terminar con un agujero en su retina. Los LED son demasiado débiles para producir daño ocular, pero lo suficientemente intensos para transferir datos correctamente a 100 Mbps. La especificación de diseño de la FDDI pide no más de un error por cada 2.5×10^{10} bits. Muchas implementaciones se comportan aún mejor.

El cableado de la FDDI consiste en dos anillos de fibra, uno que transmite en dirección de las manecillas del reloj y otro en dirección contraria, como se muestra en la figura 4-45(a). Si se rompe cualquiera de ellos, se puede usar el otro como respaldo. Si ambos se rompen en el mismo

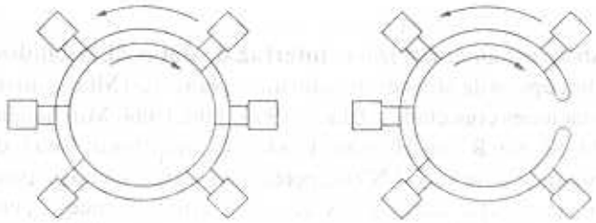


Figura 4-45. (a) La FDDI consiste en dos anillos que giran en direcciones opuestas. (b) En caso de fallar ambos anillos en un punto, pueden unirse los dos anillos para formar un solo anillo grande.

punto, por ejemplo debido a un incendio u otro accidente en el conducto del cable, pueden unirse los dos anillos en uno solo de aproximadamente el doble de longitud, como se muestra en la figura 4-45(b). Cada estación contiene relevadores (*relays*) que pueden servir para unir los dos anillos o para saltar (*bypass*) la estación en caso de problemas con ella. También pueden usarse centros de alambrado, como en el 802.5.

La FDDI define dos clases de estaciones, *A* y *B*. Las estaciones clase *A* se conectan a ambos anillos. Las estaciones clase *B*, más económicas, sólo se conectan a uno de los anillos. Dependiendo de la importancia que tenga la tolerancia de fallas, una instalación puede escoger estaciones clase *A* o clase *B*, o ambas.

La capa física no usa codificación Manchester porque la codificación Manchester a 100 Mbps requiere 200 megabaud, lo que se consideró demasiado costoso. En cambio, se usa un esquema llamado **4 de 5**. Cada grupo de 4 símbolos MAC (0, 1 y ciertos símbolos que no son de datos, como un inicio de marco) se codifica en el medio como grupo de 5 bits. Dieciséis de las 32 combinaciones son para datos, 3 son para delimitadores, 2 son de control, 3 para señalamiento al *hardware* y 8 no se usan (es decir, se reservan para versiones futuras del protocolo).

La ventaja de este esquema es que ahorra ancho de banda, pero la desventaja es la pérdida de la propiedad de autosincronización de la codificación Manchester. Para compensar esta pérdida, se usa un preámbulo grande para sincronizar el reloj del receptor con el reloj del transmisor. Además, se requiere que todos los relojes tengan una estabilidad de cuando menos 0.005%. Con esta estabilidad, se pueden enviar marcos de hasta 4500 bytes sin peligro de que el reloj del receptor se desincronice demasiado respecto a la corriente de datos.

Los protocolos FDDI básicos se basan en gran medida en los protocolos 802.5. Para transmitir datos, una estación debe primero capturar la ficha; luego transmite un marco y lo retira cuando regresa nuevamente. Una diferencia entre la FDDI y el 802.5 es que, en el 802.5, una estación no puede generar una ficha nueva hasta que su marco ha dado la vuelta completa y ha regresado. En la FDDI, con 1000 estaciones potenciales y 200 km de fibra, puede ser sustancial el tiempo perdido esperando que el marco recorra todo el anillo. Por esta razón, se decidió permitir que una estación ponga nuevamente una ficha en el anillo tan pronto ha terminado de transmitir sus marcos. En un anillo grande puede haber varios marcos a la vez.

Los marcos de datos FDDI son parecidos a los 802.5. El formato FDDI se muestra en la figura 4-46. Los campos de *delimitador inicial* y *delimitador final* marcan los límites del marco. El campo de *control de marco* indica el tipo de marco de que se trata (datos, control, etc.). El byte de *estado de marco* contiene bits de reconocimiento, parecidos a los del 802.5. Los otros campos son análogos al 802.5.

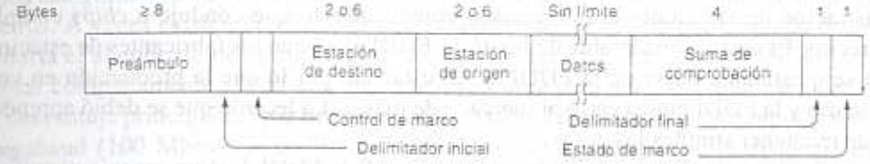


Figura 4-46. Formato de marco FDDI.

Además de los marcos normales (asincrónicos), la FDDI también permite marcos sincrónicos especiales para datos PCM de circuitos conmutados o ISDN. Los marcos sincrónicos se generan cada 125 µseg en una estación maestra para proporcionar las 8000 muestras/seg requeridas por los sistemas PCM. Cada uno de estos marcos tiene una cabecera, 16 bytes de datos conmutados fuera del circuito, y hasta 96 bytes de datos no de circuitos conmutados (es decir, hasta 96 canales PCM por marco).

Se escogió el número 96 porque permite que cuatro canales T1 (4 × 24) a 1.544 Mbps o tres canales CCITT E1 (3 × 32) a 2.048 Mbps quepan en un marco, haciéndolo adecuado para usarse en cualquier parte del mundo. Un marco sincrónico cada 125 µseg consume 6.144 Mbps de ancho de banda para los 96 canales de circuitos conmutados. Un máximo de 16 marcos sincrónicos cada 125 µseg permite hasta 1536 canales PCM y consume 98.3 Mbps.

Una vez que una estación ha adquirido una o más ranuras de tiempo en un marco sincrónico, se reservan esas ranuras para ella hasta que se liberan explícitamente. El ancho de banda total no usado por los marcos sincrónicos se reparte por demanda. En cada uno de estos marcos hay una máscara de bits para indicar qué ranuras están disponibles para la asignación por demanda. El tráfico asincrónico se divide en clases de prioridades, teniendo preferencia las prioridades mayores para recibir el ancho de banda sobrante.

El protocolo MAC de la FDDI usa tres temporizadores. El **temporizador de retención de la ficha** determina el tiempo que una estación puede continuar transmitiendo una vez que ha adquirido la ficha. Este temporizador evita que una estación acapare permanentemente el anillo. El **temporizador de rotación de la ficha** se reinicia cada vez que se ve la ficha. Si termina el temporizador, significa que no se ha visto la ficha durante una ranura demasiado grande; probablemente se ha perdido, por lo que inicia el procedimiento de recuperación de la ficha. Por último, el **temporizador de transmisión válida** se usa para terminar la temporización y recuperarse de ciertos errores transitorios del anillo.

La FDDI también tiene un algoritmo de prioridad semejante al de 802.4: determina las clases de prioridad que pueden transmitir durante un paso dado de la ficha. Si la ficha se adelanta

a lo programado, todas las prioridades pueden transmitir, pero si está retrasada, sólo pueden transmitir las mayores.

4.5.2. Ethernet rápido

Se suponía que la FDDI debía ser la LAN de la siguiente generación, pero realmente nunca se extendió su uso más allá del mercado de *backbone* (donde sigue desempeñándose bien). La administración de estaciones era demasiado complicada, lo que condujo a *chips* complejos y altos precios. El costo considerable de los *chips* FDDI evitó que los fabricantes de estaciones de trabajo se prestaran a hacer de la FDDI la red estándar, por lo que la producción en volumen jamás se dio y la FDDI nunca entró al mercado de masas. La lección que se debió aprender aquí fue la de mantener simples las cosas.

En cualquier caso, la incapacidad para despegar de la FDDI dejó un espacio para una LAN ordinaria con velocidades por encima de 10 Mbps. Muchas instalaciones necesitaban más ancho de banda y por tanto tenían muchas LAN de 10 Mbps conectadas mediante una maraña de repetidores, puentes, enrutadores y pasarelas, aunque para los administradores de red a veces parecía que se mantenían unidos con goma de mascar y alambre de gallinero.

Fue en este ambiente que el IEEE volvió a convocar en 1992 al comité del 802.3, con instrucciones de inventar una LAN más rápida. Una propuesta fue mantener el 802.3 exactamente como estaba, pero hacerlo más rápido. Otra propuesta fue rehacerlo por completo para darle muchas características nuevas, como tráfico en tiempo real y voz digitalizada, pero manteniendo su viejo nombre (por cuestiones de marketing). Tras algún forcejeo, el comité decidió mantener el 802.3 como estaba, pero hacerlo más veloz. La gente que apoyaba la propuesta perdedora hizo lo que haría cualquier persona en la industria de la computación en estas circunstancias: formó su propio comité y desarrolló el estándar de su propia LAN (que es el 802.12).

Las tres razones principales por las que el comité del 802.3 decidió diseñar una LAN 802.3 mejorada fueron:

1. La necesidad de compatibilidad hacia atrás con miles de LAN existentes.
2. El temor a que un protocolo nuevo tuviera problemas imprevistos.
3. El deseo de terminar el trabajo antes de que cambiara la tecnología.

El trabajo se hizo con rapidez (según las normas de los comités de estándares), y el resultado, el **802.3u**, fue aprobado oficialmente por el IEEE en junio de 1995. Técnicamente, el 802.3u no es un estándar nuevo, sino una adición al estándar 802.3 existente (para subrayar su compatibilidad hacia atrás). Ya que todo mundo lo llama **Ethernet rápido**, en lugar de 802.3u, nosotros también lo haremos.

El concepto principal en que se basa el Ethernet rápido es sencillo: mantener todos los formatos de paquete, interfaces y reglas de procedimientos anteriores, y simplemente reducir el tiempo de bit de 100 nseg a 10 nseg. Técnicamente, habría sido posible copiar el 10Base-5 o el 10Base-2 y aún detectar colisiones a tiempo con sólo reducir la longitud máxima del cable por un factor de 10. Sin embargo, las ventajas del alambrado 10Base-T fueron tan contundentes que

el Ethernet rápido se basa completamente en este diseño. Por tanto, todos los sistemas Ethernet rápido usan concertadores; no se permiten cables de derivación múltiple con derivaciones vampiro ni conectores BNC.

Sin embargo, aún tenían que tomarse algunas decisiones, siendo la más importante el tipo de cables a manejar. Un contendiente fue el par trenzado categoría 3. El argumento a su favor fue que prácticamente todas las oficinas del mundo occidental tienen cuando menos cuatro pares trenzados categoría 3 (o mejor) tendidos a un gabinete de alambrado telefónico en un alcance de 100 metros. A veces existen dos de tales cables. Por tanto, el uso de par trenzado categoría 3 posibilitaría el alambrado de computadoras de escritorio usando Ethernet rápido sin tener que realambrar completamente el edificio, una ventaja enorme para muchas organizaciones.

La desventaja principal del par trenzado categoría 3 es la incapacidad de conducir señales de 200 megabaud (100 Mbps con codificación Manchester) a 100 metros, la distancia máxima de computadora a concertador especificada para 10Base-T (véase la figura 4-17). En contraste, el par trenzado categoría 5 puede manejar fácilmente los 100 metros, y la fibra puede llegar mucho más lejos. La media escogida fue permitir las tres posibilidades, como se muestra en la figura 4-47, pero mejorar la solución de categoría 3 para darle la capacidad de conducción adicional necesaria.

Nombre	Cable	Segmento máximo	Ventajas
100Base-T4	Par trenzado	100 m	Usa UTP categoría 3
100Base-TX	Par trenzado	100 m	Dúplex integral a 100 Mbps
100Base-F	Fibra óptica	2000 m	Dúplex integral a 100 Mbps; tramos grandes

Figura 4-47. Cableado de Ethernet rápido.

El esquema UTP categoría 3, llamado **100Base-T4**, usa una velocidad de señalización de 25 MHz, sólo 25% más rápida que los 20 MHz del 802.3 estándar (recuerde que la codificación Manchester, como se indica en la figura 4-20, requiere dos periodos de reloj por cada uno de los 10 millones de bits por segundo). Para lograr el ancho de banda necesario, el 100Base-T4 requiere cuatro pares trenzados. Dado que el alambrado telefónico estándar ha tenido durante décadas cuatro pares trenzados por cable, la mayoría de las oficinas pueden manejar esto. Por supuesto, significa renunciar al teléfono de la oficina, pero eso seguramente es un precio bajo a pagar por un correo electrónico más rápido.

De los cuatro pares trenzados, uno siempre va al concertador, uno más siempre viene del concertador y los otros dos son conmutables a la dirección actual de transmisión. Para conseguir el ancho de banda necesario, no se usa codificación Manchester, pues con relojes modernos y distancias tan cortas ya no es necesario. Además, se envían señales ternarias, por lo que durante un solo periodo de reloj el alambre puede contener un 0, un 1 o un 2. Con tres pares trenzados en la dirección de transmisión y señalización ternaria, puede transmitirse cualquiera de 27 símbolos, posibilitando el envío de 4 bits con un poco de redundancia. La transmisión de 4 bits en cada

uno de los 25 millones de ciclos de reloj por segundo da los 100 Mbps necesarios. Además, siempre hay un canal en reversa de 33.3 Mbps que utiliza el par trenzado restante. Este esquema, conocido como **8B6T** (mapa de 8 bits a 6 trits), seguramente no ganará premios por su elegancia, pero funciona con la planta de alambrado existente.

Para el alambrado de categoría 5, el diseño, **100Base-TX**, es más sencillo, pues los alambres pueden manejar tasas de reloj de hasta 125 MHz o más. Sólo se usan dos pares trenzados por estación, uno al concertador y uno de él. En lugar de sólo usar codificación binaria directa, se usa un esquema llamado **4B5B** a 125 MHz. Cada grupo de cinco periodos de reloj se usa para enviar 4 bits a fin de tener cierta redundancia, proporcionar suficientes transiciones para permitir una fácil sincronización de los relojes, crear patrones únicos para delimitar marcos, y ser compatible con la FDDI en la capa física. En consecuencia, el 100Base-TX es un sistema dúplex integral; las estaciones pueden transmitir a 100 Mbps y recibir a 100 Mbps al mismo tiempo. Además, se pueden tener dos líneas telefónicas para comunicación real en caso de que la computadora esté completamente ocupada navegando la *Web*.

La última opción, **100Base-FX**, usa dos hilos de fibra multimodo, uno para cada dirección, por lo que también es dúplex integral con 100 Mbps en cada dirección. Además, la distancia entre una estación y el concertador puede ser de hasta 2 km.

Son posibles dos tipos de concertadores con 100Base-T4 y 100Base-TX, conocidos en conjunto como **100Base-T**. En un concertador compartido, todas las líneas de entrada (o cuando menos todas las líneas que llegan a una tarjeta) están conectadas lógicamente, formando un solo dominio de colisión. Se aplican todas las reglas estándar, incluido el algoritmo de retroceso binario, por lo que el sistema funciona igual que el 802.3 tradicional. En particular, sólo una estación puede transmitir a la vez.

En un concertador conmutado, cada marco de entrada se coloca en el *buffer* de una tarjeta de línea (*plug-in line card*). Aunque esta característica hace que el concertador y las tarjetas sean más caros, también significa que todas las estaciones pueden transmitir (y recibir) al mismo tiempo, mejorando de manera importante el ancho de banda total del sistema, con frecuencia en un orden de magnitud o más. Los marcos en *buffer* se pasan a través de un canal de alta velocidad de la tarjeta de origen a la tarjeta de destino. El canal de alta velocidad en plano posterior no está estandarizado, ni necesita estarlo, ya que está completamente escondido dentro del conmutador. Si la experiencia pasada sirve de guía, los fabricantes de conmutadores competirán fuertemente para producir canales de alta velocidad a plano posterior cada vez más rápidos a fin de mejorar el rendimiento del sistema. Dado que los cables 100Base-FX son demasiado largos para el algoritmo normal de colisiones de Ethernet, deben conectarse a concertadores conmutados con *buffers*, de modo que cada uno sea un dominio de colisión por sí mismo.

Como nota final, prácticamente todos los conmutadores pueden manejar una mezcla de estaciones de 10 Mbps y 100 Mbps, para hacer más fácil la modernización. A medida que una instalación consigue más estaciones de 100 Mbps, todo lo que tiene que hacer es comprar la cantidad necesaria de tarjetas de línea nuevas e introducirlas en el conmutador.

En (Johnson, 1996) puede encontrarse mayor información sobre el Ethernet rápido. Para una comparación de las redes de área local de alta velocidad, en particular FDDI, Ethernet rápido, ATM y VG-AnyLAN, véase (Cronin *et al.*, 1994).

4.5.3. HIPPI — Interfaz paralela de alto desempeño

Durante la guerra fría, el Laboratorio Nacional de Los Álamos, el centro de diseño de armas nucleares del gobierno de Estados Unidos, compró rutinariamente cada uno de los modelos de supercomputadora en venta. Los Álamos también acumuló periféricos complicados, como dispositivos de almacenamiento masivo y estaciones de trabajo gráficas especiales para visualización científica. En esos años, cada fabricante tenía una interfaz distinta para conectar periféricos a su supercomputadora, por lo que no era posible compartir periféricos entre máquinas o interconectar dos supercomputadoras.

En 1987, los investigadores de Los Álamos comenzaron a trabajar con una interfaz éstándar de supercomputadora, con la intención de conseguir su estandarización y luego convencer a todos los fabricantes de que la usaran. (Dado el tamaño del presupuesto de computación de Los Álamos, cuando ellos hablaban, los fabricantes escuchaban.) La meta era una interfaz que cualquiera pudiera implementar rápida y eficientemente. El principio rector fue la sencillez. La interfaz no debía tener opciones ni requerir el diseño de *chips* nuevos, y debía tener el desempeño de una manguera contra incendios.

La especificación inicial indicaba una tasa de datos de 800 Mbps, porque las películas de bombas estallando requerían marcos de 1024 × 1024 pixeles, con 24 bits por pixel y 30 marcos/seg, para una tasa de datos total de 750 Mbps. Después, se coló una opción: una segunda tasa de datos de 1600 Mbps. Cuando esta propuesta, llamada **HIPPI** (*HIgh Performance Parallel Interface*, **interfaz paralela de alto desempeño**) se ofreció al ANSI para su estandarización, los proponentes fueron considerados lunáticos, ya que las LAN en los años 80 implicaban Ethernets de 10 Mbps.

La HIPPI se diseñó originalmente como canal de datos, no como LAN. Los canales de datos operan punto a punto, de un maestro (una computadora) a un subordinado (un periférico), con alambres dedicados y sin conmutación. No hay contención y el entorno es completamente predecible. Después, se hizo evidente la necesidad de conmutar un periférico de una supercomputadora a otra, y se añadió un conmutador de barras cruzadas al diseño de la HIPPI, como se ilustra en la figura 4-48.

A fin de lograr un desempeño de tal nivel usando sólo *chips* comunes, la interfaz básica se hizo de 50 bits de ancho, 32 bits de datos y 18 bits de control, por lo que el cable HIPPI contiene 50 pares trenzados. Cada 40 nseg se transfiere una palabra en paralelo a través de la interfaz. Para lograr 1600 Mbps, se usan dos cables y se transfieren dos palabras por ciclo. Todas las transferencias son símplex. Para lograr comunicación de dos vías, se requieren dos (o cuatro) cables. A estas velocidades, la longitud máxima del cable es de 25 metros.

Tras recuperarse de su pasmo inicial, el comité ANSI X3T9.3 generó un estándar HIPPI basado en la información de Los Álamos. El estándar cubre las capas física y de enlace de datos. Todo lo que esté por arriba de ellas es responsabilidad de los usuarios. El protocolo básico es que, para comunicarse, un *host* primero pide al conmutador de barras cruzadas que establezca una conexión. Entonces (generalmente) envía un solo mensaje y libera la conexión.

Los mensajes se estructuran con una palabra de control, una cabecera de hasta 1016 bytes y una parte de datos de hasta $2^{32} - 2$ bytes. Por razones de control de flujo, los mensajes se dividen

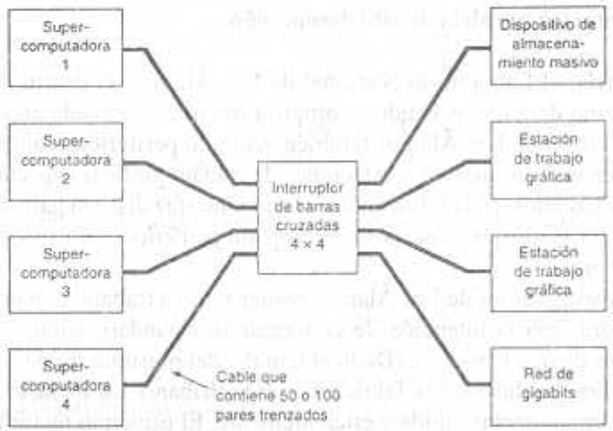


Figura 4-48. HIPPI que usa un interruptor de barras cruzadas.

en marcos de 256 palabras. Cuando el receptor es capaz de aceptar un marco, envía una señal al transmisor, quien entonces envía el marco. Los receptores también pueden pedir varios marcos a la vez. El control de errores consiste en un bit de paridad horizontal por palabra y una palabra de paridad vertical al final de cada marco. Las sumas de comprobación tradicionales se consideraron innecesarias y demasiado lentas.

La HIPPI fue implementada rápidamente por docenas de fabricantes y ha sido el estándar de interconexión de supercomputadoras durante años. Para mayor información sobre ella, véase (Hughes y Franta, 1994; Tolmie, 1992, y Tolmie y Renwick, 1993).

4.5.4. Canal de fibra

En la época en que se diseñó la HIPPI, la fibra óptica era demasiado cara y no se consideraba lo suficientemente confiable, por lo que la LAN más rápida construida hasta el momento se hizo con alambre telefónico de baja clasificación. Con el paso del tiempo, la fibra se volvió más barata y confiable, por lo que fue natural que tarde o temprano hubiera un intento por rehacer la HIPPI usando una sola fibra en lugar de 50 o 100 pares trenzados. Desafortunadamente, la disciplina que había establecido Los Álamos de parar las propuestas de características nuevas cada vez que aparecía una, se perdió en el camino. El sucesor de la HIPPI, llamado **canal de fibra** (*fibre channel*), es mucho más complicado y de implementación más cara. Queda por ver si disfrutará del éxito comercial de la HIPPI.

El canal de fibra maneja tanto conexiones de canal de datos como de red. En particular, puede usarse para conducir canales de datos que incluyen HIPPI, SCSI y el canal multiplexor empleado en las *mainframe* IBM. También puede conducir paquetes de red, incluidos IEEE 802, IP y ATM. Como en HIPPI, la estructura básica de los canales de fibra es un interruptor de barras

cruzadas que conecta las entradas con las salidas. Las conexiones pueden establecerse durante un solo paquete o por una ranura mucho mayor.

El canal de fibra apoya tres clases de servicio. La primera es la conmutación de circuitos pura, con entrega en orden garantizada. Los modos del canal de datos usan esta clase de servicio. La segunda clase es la conmutación de paquetes con entrega garantizada. La tercera clase es la conmutación de paquetes sin entrega garantizada.

El canal de fibra tiene una estructura de protocolos complicada, como se muestra en la figura 4-49. Aquí vemos cinco capas, que en conjunto cubren las capas física y de enlace de datos. La capa inferior tiene que ver con el medio físico. Hasta ahora, maneja tasas de datos de 100, 200, 400 y 800 Mbps. La segunda capa maneja la codificación de bits. El sistema empleado se parece un poco al de la FDDI, pero en lugar de usar 5 bits para codificar 16 símbolos válidos usa 10 bits para codificar 256 símbolos válidos, proporcionando algo de redundancia. Además, estas dos capas son equivalentes funcionalmente a la capa física de la OSI.

Figura 4-49. Capas de protocolo del canal de fibra.

La capa media define el formato de la distribución del marco y de la cabecera. Los datos se transmiten en marcos con cargas útiles de hasta 2048 bytes. La siguiente capa permitirá proporcionar servicios comunes a la capa superior posteriormente, según se requiera. Por último, la capa superior proporciona las interfaces con los distintos tipos de computadoras y periféricos reconocidos.

Como comentario al margen, aunque el canal de fibra fue diseñado en Estados Unidos, se escogió la pronunciación inglesa (*fibre channel*) en lugar de la estadounidense (*fiber channel*), porque el editor del estándar era británico. Se puede encontrar información adicional sobre el canal de fibra en (Tolmie, 1992). En (Tolmie, 1995) hay una comparación con la HIPPI y ATM.

4.6. REDES SATELITALES

Aunque la mayoría de los canales de acceso múltiple se encuentran en las LAN, hay un tipo de WAN que usa canales de acceso múltiple: las WAN basadas en comunicación satelital. En las siguientes secciones estudiaremos brevemente algunos de los problemas que se presentan en las

redes de área amplia basadas en satélite. También veremos algunos de los protocolos que se han diseñado para manejarlas.

Los satélites de comunicación por lo general tienen hasta una docena o más de transpondedores. Cada transpondedor tiene un haz que cubre una parte de la Tierra debajo de él, variando de un haz amplio de 10,000 km de diámetro hasta un haz localizado de sólo 250 km de diámetro. Las estaciones dentro del área del haz pueden enviar marcos al satélite en la frecuencia de enlace ascendente. El satélite entonces vuelve a difundirlos por la frecuencia de enlace descendente. Se usan diferentes frecuencias para el enlace ascendente y el descendente a fin de evitar que el transpondedor entre en oscilación. Los satélites sin procesamiento "a bordo", sino que simplemente repiten lo que escuchan (la mayoría de ellos), con frecuencia se llaman satélites de codo.

Cada antena puede enfocarse en un área, transmitir algunos marcos, y luego enfocarse en un área nueva. El enfoque es electrónico, pero aun así tarda algunos microsegundos. El tiempo durante el cual se apunta un haz sobre un área dada se llama el **tiempo de morada o permanencia** (*dwell time*). Para una máxima eficiencia, este tiempo no debe ser muy corto, porque se desperdiciará demasiado tiempo moviendo el haz.

Al igual que en las LAN, uno de los puntos clave del diseño es la manera de repartir los canales del transpondedor. Sin embargo, a diferencia de las LAN, es imposible la detección de portadora, debido al retardo de propagación de 270 mseg. Cuando una estación detecta el estado de un canal de enlace descendente, escucha lo que ocurrió hace 270 mseg. La detección del canal de enlace ascendente generalmente es imposible. Como resultado, los protocolos CSMA/CD (que suponen que una estación transmisora puede detectar colisiones en los primeros tiempos de bit, y retraerse si está ocurriendo una) no pueden usarse con los satélites. De ahí la necesidad de otros protocolos.

Se emplean cinco clases de protocolos en el canal de acceso múltiple (de enlace ascendente): sondeo, ALOHA, FDM, TDM y CDMA. Aunque los hemos estudiado a todos, la operación satelital a veces agrega nuevas complicaciones. El problema principal es con el canal de enlace ascendente, ya que el de enlace descendente sólo tiene un transmisor (el satélite) y por tanto no tiene el problema de reparto del canal.

4.6.1. Sondeo

La forma tradicional de repartir un solo canal entre usuarios competidores es que alguien los sondee. Hacer que el satélite sondee por turno cada estación para ver si tiene un marco es prohibitivamente caro, dado el tiempo de 270 mseg requerido para cada secuencia de sondeo/respuesta.

Sin embargo, si todas las estaciones de tierra también están conectadas a una red de conmutación de paquetes (típicamente de poco ancho de banda), es concebible una variación menor de este concepto. La idea es disponer todas las estaciones en un anillo lógico, de modo que cada estación conozca su sucesor. Por este anillo terrestre circula una ficha. El satélite nunca ve la ficha. Sólo se permite a una estación transmitir por el enlace ascendente cuando ha capturado la ficha. Si el número de estaciones es pequeño y constante, el tiempo de transmisión de la ficha es corto y las ráfagas enviadas por el canal de enlace ascendente son mucho más grandes que el tiempo de rotación de la ficha, el esquema es moderadamente eficiente.

4.6.2. ALOHA

El ALOHA puro es fácil de implementar: cada estación simplemente envía cuando quiere. El problema es que la eficiencia del canal es de sólo 18%. En general, un factor de uso tan bajo es inaceptable para satélites que cuestan decenas de millones de dólares cada uno.

El uso del ALOHA ranurado duplica la eficiencia, pero agrega el problema de cómo sincronizar todas las estaciones para que todas sepan en qué momento comienza cada intervalo. Afortunadamente el satélite mismo tiene la respuesta, ya que inherentemente es un medio de difusión. Una estación de tierra, la **estación de referencia**, transmite periódicamente una señal especial cuya retransmisión es usada por todas las estaciones de tierra como origen del tiempo. Si todas las ranuras de tiempo tienen una longitud de ΔT, cada estación sabe ahora que la ranura de tiempo k comienza en el momento $k\Delta T$ después del origen del tiempo. Puesto que los relojes operan con tasas ligeramente diferentes, es necesaria una resincronización periódica para mantener a todos en fase. Una complicación adicional es que el tiempo de propagación desde el satélite es diferente para cada estación de tierra, pero este efecto puede compensarse.

Para aumentar el aprovechamiento del canal de enlace ascendente por encima de $1/e$, podemos pasar del canal de enlace ascendente individual de la figura 4-50(a) al esquema de enlace ascendente doble de la figura 4-50(b). Una estación con un marco por transmitir escoge uno de los canales de enlace ascendente al azar y envía el marco en la siguiente ranura. Cada enlace ascendente opera entonces un canal ALOHA ranurado independiente.

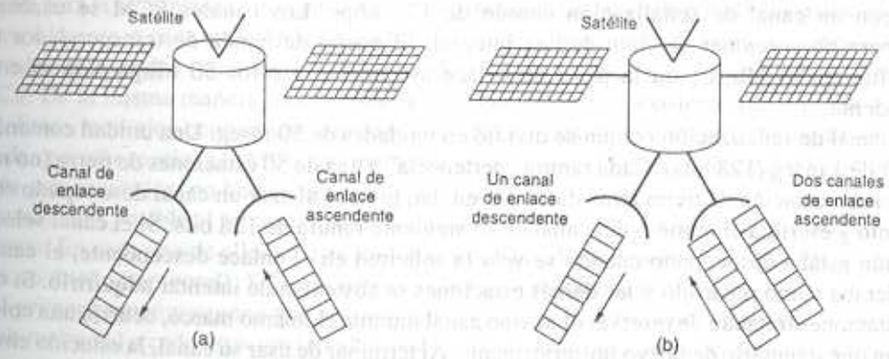


Figura 4-50. (a) Sistema ALOHA estándar. (b) Adición de un segundo canal de enlace ascendente.

Si uno de los canales de enlace ascendente contiene un solo marco, simplemente se transmite después en la ranura de enlace descendente. Si ambos canales tienen éxito, el satélite puede poner en *buffer* uno de los marcos y transmitirlo durante una ranura inactiva posterior. Calculando las probabilidades, puede demostrarse que, dada una cantidad infinita de espacio de *buffer*, puede elevarse la utilización del canal de enlace descendente hasta 0.736, al costo de aumentar los requisitos de ancho de banda en 50%.

4.6.3. FDM

La multiplexión por división en frecuencia es el esquema de reparto de canal más viejo y probablemente el más utilizado aún. Un transpondedor típico de 36 Mbps puede dividirse estáticamente en unos 500 canales PCM de 64,000 bps, operando cada uno en su propia frecuencia para evitar la interferencia con los demás.

Aunque es simple, la FDM también tiene algunos inconvenientes. Primero, se requieren bandas de seguridad (laterales) entre los canales para mantener separadas las estaciones. Este requisito existe porque no es posible construir transmisores que envíen toda su energía por la banda principal y nada por las bandas laterales. La cantidad de ancho de banda desperdiciado en las bandas laterales puede ser una parte sustancial del total.

Segundo, debe controlarse cuidadosamente la potencia de las estaciones. Si una estación pone demasiada potencia en la banda principal, también pondrá automáticamente demasiada potencia en las bandas laterales, ocurriendo un derramamiento a los canales adyacentes que causa interferencia. Por último, la FDM es una técnica completamente analógica que no se presta bien a una implementación en *software*.

Si el número de estaciones es pequeño y fijo, pueden repartirse por adelantado los canales de frecuencia. Sin embargo, si el número de estaciones, o la carga de cada una de ellas, puede fluctuar con rapidez, se necesita algún tipo de reparto dinámico de las bandas de frecuencia. Uno de tales mecanismos es el sistema **SPADE** empleado en algunos de los primeros satélites Intelsat. Cada repetidor SPADE se dividió en 794 canales de voz PCM símplex (de 64 kbps), junto con un canal de señalización común de 128 kbps. Los canales PCM se usaban en pares para proporcionar servicio dúplex integral. El ancho de banda de transpondedor total usado fue de 50 Mbps para la parte de enlace ascendente y otros 50 Mbps para el enlace descendente.

El canal de señalización común se dividió en unidades de 50 mseg. Una unidad contenía 50 ranuras de 1 mseg (128 bits). Cada ranura "pertenecía" a una de 50 estaciones de tierra (no más). Cuando una estación de tierra tenía datos por enviar, tomaba al azar un canal desocupado en ese momento y escribía el número del canal en su siguiente ranura de 128 bits. Si el canal seleccionado aún estaba desocupado cuando se veía la solicitud en el enlace descendente, el canal se consideraba como repartido y las demás estaciones se abstenían de intentar adquirirlo. Si dos o más estaciones trataban de reservar el mismo canal durante el mismo marco, ocurría una colisión y tenían que intentarlo de nuevo posteriormente. Al terminar de usar su canal, la estación enviaba un mensaje de liberación en su ranura del canal común.

4.6.4. TDM

Al igual que la FDM, la TDM es bien entendida y se usa ampliamente en la práctica; requiere sincronización de tiempo para las ranuras, pero esto puede proporcionarse mediante una estación de referencia, como se describió antes para el ALOHA ranurado. De modo parecido a la FDM, para un número pequeño e invariable de estaciones, la asignación de ranuras puede establecerse por adelantado y no cambiarse nunca, pero con un número variable de estaciones, o

un número fijo de estaciones con cargas variables en el tiempo, las ranuras de tiempo deben asignarse dinámicamente.

La asignación de ranuras puede hacerse de manera centralizada o descentralizada. Como ejemplo de asignación de ranuras centralizada, consideremos el ACTS (*Advanced Communication Technology Satellite*, **satélite de tecnología de comunicaciones avanzada**), que se diseñó para unas pocas docenas de estaciones (Palmer y White, 1990). El ACTS se lanzó en 1992 y tiene cuatro canales TDM independientes de 110 Mbps, dos de enlace ascendente y dos de enlace descendente. Cada canal está organizado como secuencia de marcos de 1 mseg, conteniendo cada marco 1728 ranuras de tiempo. Cada ranura tiene una carga útil de 64 bits, permitiendo a cada una contener un canal de voz de 64 kbps.

Los haces pueden conmutarse de un área geográfica a otra pero, dado que el movimiento del haz se lleva varios tiempos de ranura, los canales que se originan o terminan en la misma zona geográfica normalmente son asignados a ranuras contiguas para aumentar el tiempo de morada o permanencia y reducir al mínimo el tiempo perdido en el movimiento del haz. Por tanto, la administración de las ranuras de tiempo requiere un conocimiento profundo de la geografía de las estaciones para minimizar la cantidad de ranuras de tiempo desperdiciadas. Por ésta y otras razones, la administración de ranuras de tiempo la efectúa una de las estaciones de tierra, la MCS (*Master Control Station*, **estación de control maestro**).

La operación básica del ACTS es un proceso continuo de tres pasos, tardándose cada paso 1 mseg. En el paso 1, el satélite recibe un marco y lo almacena en una RAM interna de 1728 entradas. En el paso 2, una computadora a bordo copia cada entrada en la salida correspondiente (posiblemente para la otra antena). En el paso 3, el marco de salida se transmite por el enlace descendente.

Inicialmente, a cada estación se asigna cuando menos una ranura de tiempo. Si desea adquirir canales adicionales (para nuevas llamadas de voz), una estación envía una solicitud corta a la MCS. De la misma manera, la estación puede liberar un canal existente con un mensaje a la MCS. Estos mensajes ocupan una cantidad pequeña de bits de carga extra y proporcionan un canal de control especial a la MCS con una capacidad de unos 13 mensajes/seg por estación. Los canales son dedicados; no hay contención por ellos.

También es posible el reparto dinámico de ranuras TDM. A continuación analizaremos tres esquemas. En cada uno de ellos se dividen los marcos TDM en ranuras de tiempo, teniendo cada ranura un dueño (temporal). Sólo el dueño puede usar una ranura de tiempo.

El primer esquema supone que hay más ranuras que estaciones, por lo que puede asignarse una ranura inicial a cada estación (Binder, 1975). Si hay más ranuras que estaciones, las ranuras extra no se asignan a nadie. Si el dueño de una ranura no la quiere durante el grupo actual, se desperdicia. Una ranura vacía es una señal para todos los demás de que el dueño no tiene tráfico. Durante el siguiente marco, la ranura queda disponible para quien la quiera, con base en contención (ALOHA).

Si el dueño quiere recuperar "su" ranura inicial, transmite un marco, obligando a una colisión (si hay otro tráfico). Tras la colisión, todos, excepto el dueño, deben desistir de usar la ranura durante el siguiente marco. Por tanto, el dueño siempre puede comenzar a transmitir en un lapso de dos tiempos de marco, en el peor caso. Si la utilización del canal es baja, el sistema no se desempeña tan bien como el ALOHA ranurado normal ya que, después de una colisión, los

implicados deben abstenerse durante un marco para ver si el dueño quiere recuperar la ranura. En la figura 4-51(a) se muestra un marco con ocho ranuras, de las cuales siete pertenecen a *G*, *A*, *F*, *E*, *B*, *C* y *D*, respectivamente. La octava ranura no pertenece a nadie y cualquiera puede contender por ella.

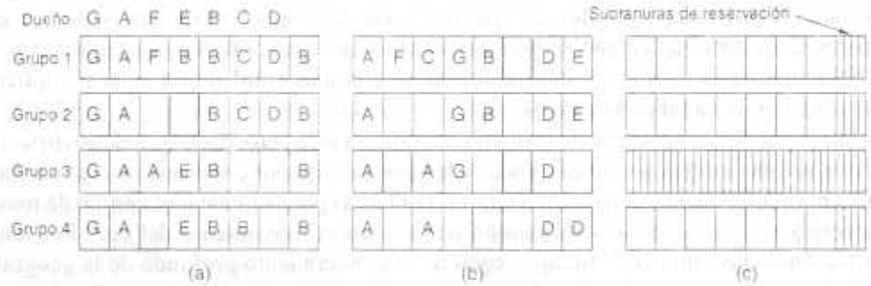


Figura 4-51. Esquemas de reservación. (a) Binder. (b) Crowther. (c) Roberts. Los marcos sombreados indican colisiones. Se muestran cuatro grupos consecutivos de ranuras para cada uno de los tres esquemas.

Es aplicable un segundo esquema aun cuando el número de estaciones es desconocido y variable (Crowther *et al.*, 1973). En este método, las ranuras no tienen dueños permanentes, como en el de Binder. En cambio, las estaciones compiten por ranuras usando ALOHA ranurado. Cuando tiene éxito una transmisión, la estación que hace la transmisión también tiene derecho a esa ranura durante el siguiente marco. Así, mientras una estación tenga datos por enviar, podrá continuar haciéndolo indefinidamente (sujeta a algunas reglas para evitar acaparamientos). En esencia, la propuesta permite una mezcla dinámica de ALOHA ranurado y TDM, variando según la demanda el número de ranuras destinadas a cada esquema. En la figura 4-51(b) también se muestra un marco con ocho ranuras. Inicialmente, *E* usa la última ranura, pero tras dos marcos ya no la necesita. La ranura permanece inactiva durante un marco y entonces *D* la toma y la conserva hasta haber terminado.

Un tercer esquema, debido a Roberts (1973), requiere que las estaciones hagan solicitudes por adelantado antes de transmitir. Cada marco contiene, digamos, una ranura especial [la última de la figura 4-51(c)], que se divide en *V* subranuras menores empleadas para hacer reservaciones. Cuando una estación quiere enviar datos, difunde un marco corto de solicitud en una subranura de reservación escogida al azar. Si la reservación tiene éxito (es decir, no hay una colisión), entonces se reserva la siguiente ranura normal (o más). Todos deben conocer la longitud de la cola (cantidad de ranuras reservadas) en todo momento, para que, al lograr una estación una reservación exitosa, sepa cuántas ranuras de datos deben pasar antes de transmitir. Las estaciones no necesitan saber *quién* está en la cola; simplemente necesitan conocer el tamaño de la cola. Cuando el tamaño de la cola cae a cero, todas las ranuras se revierten a subranuras de reservación a fin de acelerar el proceso de reservación.

Aunque la TDM se usa ampliamente, tanto con esquemas de reservación como sin ellos, tiene algunos inconvenientes. Por una parte, requiere que todas las estaciones se resincronicen en el tiempo, lo que en la práctica no es un asunto trivial, puesto que los satélites tienden a una deriva en su órbita, lo que cambia el tiempo de propagación a cada estación de tierra. También requiere que cada estación de tierra pueda alcanzar velocidades de ráfaga extremadamente altas. Por ejemplo, aun cuando una estación ACTS puede tener sólo un canal de 64 kbps, debe ser capaz de enviar una ráfaga de 64 bits en una ranura de tiempo de 578 nseg. En otras palabras, debe operar en realidad a 110 Mbps. A diferencia, una estación FDM de 64 kbps en realidad opera a 64 kbps.

4.6.5. CDMA

El esquema final es el CDMA. El CDMA evita el problema de sincronización de tiempo y también el problema de reparto del canal; es completamente descentralizado y totalmente dinámico.

Sin embargo, el CDMA tiene tres desventajas principales. Primero, la capacidad de un canal CDMA ante la presencia de ruido y de estaciones descoordinadas es típicamente menor de lo que se puede lograr con TDM. Segundo, con 128 *chips*/bit (que es un valor común), aunque la tasa de bits no sea alta, la tasa de *chips* sí lo es, necesitándose un transmisor rápido (léase caro). Tercero, pocos ingenieros entienden realmente el CDMA, lo que por lo general no aumenta la probabilidad de que lo usen, aun si es el mejor método para una aplicación en particular. No obstante, el CDMA ha sido usado por los cuerpos armados durante décadas, y ahora se está volviendo más común en aplicaciones comerciales.

4.7. RESUMEN

Algunas redes tienen un solo canal que se usa para todas las comunicaciones. En estas redes, el aspecto clave del diseño es el reparto del canal entre las estaciones competidoras que desean usarlo. Se han desarrollado muchos algoritmos de reparto de canal. En la figura 4-52 se presenta un resumen de algunos de los métodos de reparto de canal más importantes.

Los métodos de reparto más sencillos son la FDM y la TDM; son eficientes con un número de estaciones pequeño y tráfico continuo. Ambos esquemas se usan ampliamente en estas circunstancias; por ejemplo, para dividir el ancho de banda de los enlaces satelitales usados como troncales telefónicas.

Con un número grande y variable de estaciones, o con un tráfico en ráfagas, la FDM y la TDM son soluciones pobres. Se ha propuesto como alternativa el protocolo ALOHA, con y sin ranuras y control. El ALOHA y sus muchas variantes y derivaciones han sido ampliamente estudiados, analizados y usados en sistemas reales.

Cuando puede detectarse el estado del canal, las estaciones pueden evitar el comienzo de una transmisión mientras otra estación está transmitiendo. Esta técnica, la detección de portadora, ha producido varios protocolos que pueden usarse en redes LAN y MAN.

Se conoce una clase de protocolos que eliminan por completo la contención, o cuando menos la reducen considerablemente. El conteo descendente binario elimina por completo la

Método	Descripción
FDM	Dedica una banda de frecuencia a cada estación
TDM	Dedica una ranura de tiempo a cada estación
ALOHA puro	Transmisión no sincronizada en cualquier momento
ALOHA ranurado	Transmisión aleatoria en ranuras de tiempo bien definidas
CSMA persistente-1	Acceso múltiple con detección de portadora estándar
CSMA no persistente	Retardo aleatorio cuando se detecta ocupado el canal
CSMA persistente-p	CSMA, pero con una probabilidad p de persistir
CSMA/CD	CSMA, pero con aborto al detectar una colisión
Mapa de bits	Programación por turno circular usando un mapa de bits
Conteo descendente binario	La estación lista con el número más alto es la siguiente
Recorrido de árbol	Contención reducida mediante habilitación selectiva
División en longitud de onda	Esquema FDM dinámico para fibra
MACA, MACAW	Protocolos LAN inalámbricos
GSM	FDM más TDM para radio celular
CDPD	Radio en paquetes con un canal AMPS
CDMA	Todos hablan a la vez, pero en idiomas diferentes
Ethernet	CSMA/CD con retroceso binario exponencial
Token bus	Anillo lógico en un canal físico
Token ring	Captura la ficha para enviar un marco
DQDB	Encolamiento distribuido en una MAN de dos canales
FDDI	*Token ring* de fibra óptica
HIPPI	Barras cruzadas usando 50 o 100 pares trenzados
Canal de fibra	Barras cruzadas usando fibra óptica
SPADE	FDM con reparto dinámico del canal
ACTS	TDM con reparto centralizado de ranuras
Binder	TDM con ALOHA cuando el dueño de la ranura no está interesado
Crowther	ALOHA en el que el dueño de la ranura la conserva
Roberts	Tiempo de canal reservado por adelantado con ALOHA

Figura 4-52. Métodos de reparto de canal y sistemas para un canal común.

contención. El protocolo de recorrido de árbol la reduce dividiendo dinámicamente las estaciones en dos grupos separados, uno que puede transmitir y otro que no. Se intenta hacer la división de tal manera que sólo una estación lista para transmitir pueda hacerlo.

Las LAN inalámbricas tienen sus propios problemas y soluciones. El problema principal lo causan las estaciones ocultas, por lo que el CSMA no funciona. Una clase de soluciones, tipificadas por el MACA, intenta estimular las transmisiones en las cercanías del destino, para hacer que el CSMA funcione mejor.

En lo que toca a las computadoras móviles y los teléfonos, la tecnología en ascenso es la radio celular. El GSM, el CDPD y el CDMA se usan ampliamente.

Las LAN IEEE 802 son: CSMA/CD, *token bus* y *token ring*. Cada una tiene sus propias ventajas y desventajas, y cada una ha encontrado su propia comunidad de usuarios, así que probablemente continuará sirviendo a esa comunidad durante varios años más. La convergencia a un solo estándar LAN es poco probable. Una nueva adición a esta familia es DQDB, que se vende como MAN en muchas ciudades.

Una organización con varias LAN con frecuencia las conecta mediante puentes. Cuando un puente conecta dos o más tipos distintos de LAN, surgen nuevos problemas, algunos de ellos sin solución.

Aunque las LAN 802 son los caballos de batalla del día, los caballos de carrera son las FDDI, el Ethernet rápido, HIPPI y el canal de fibra. Todos estos ofrecen anchos de banda del orden de 100 Mbps o más.

Por último, las redes satelitales también emplean canales de acceso múltiple (para el enlace ascendente). Aquí se usan varios métodos de reparto de canal, que incluyen ALOHA, FDM, TDM y CDMA.

PROBLEMAS

1. Un grupo de N estaciones comparte un canal ALOHA puro de 56 kbps. La salida de cada estación es un marco de 1000 bits en promedio cada 100 seg, aun si el anterior no ha sido enviado (por ejemplo, las estaciones tienen *buffers*). ¿Cuál es el valor máximo de N?

2. Considere el retardo del ALOHA puro comparándolo con el ALOHA ranurado cuando la carga es baja. ¿Cuál es menor? Explique su respuesta.

3. Diez mil estaciones de reservaciones de una aerolínea compiten por un solo canal ALOHA ranurado. La estación promedio hace 18 solicitudes/hora. Una ranura dura 125 µseg. ¿Cuál es la carga aproximada total del canal?

4. Una gran población de usuarios de ALOHA genera 50 solicitudes/seg, incluidas tanto originales como retransmisiones. El tiempo se divide en ranuras de 40 mseg.
   (a) ¿Cuál es la posibilidad de éxito en el primer intento?
   (b) ¿Cuál es la probabilidad de exactamente k colisiones y luego una transmisión exitosa?
   (c) ¿Cuál es el número esperado de intentos de transmisión necesarios?

5. Mediciones en un canal ALOHA ranurado con una cantidad infinita de usuarios muestra que 10% de las ranuras están inactivas.
   (a) ¿Qué carga, G, tiene el canal?
   (b) ¿Cuál es el rendimiento?
   (c) ¿El canal está subcargado o sobrecargado?
6. En un sistema ALOHA ranurado de población infinita, la cantidad media de ranuras que espera una estación entre una colisión y su retransmisión es de 4. Grafique la curva de retardo contra rendimiento de este sistema.
7. Una LAN usa la versión de Mok y Ward del conteo descendente binario. En cierto momento, las 10 estaciones tienen los números de estación virtual 8, 2, 4, 5, 1, 7, 3, 6, 9 y 0. Las tres estaciones siguientes que van a enviar son: 4, 3 y 9, en ese orden. ¿Cuáles son los nuevos números de estación virtual una vez que las tres han terminado sus transmisiones?
8. Dieciséis estaciones contienden por un canal compartido que usa el protocolo de recorrido de árbol. Si todas las estaciones cuyas direcciones son números primos de pronto quedaran listas al mismo tiempo, ¿cuántas ranuras de bit se necesitan para resolver la contención?
9. Un conjunto de 2^n estaciones usa el protocolo de recorrido de árbol para arbitrar el acceso a un cable compartido. En cierto momento, dos de ellas quedan listas. ¿Cuál es el número de ranuras mínimo, máximo y medio para recorrer el árbol si $2^n \gg 1$?
10. Las LAN inalámbricas que estudiamos usaban protocolos como MACA en lugar de CSMA/CD. ¿En qué condiciones sería posible usar CSMA/CD?
11. ¿Qué propiedades tienen en común los protocolos de acceso a canal WDMA y GSM?
12. Usando la estructura de enmarcado del GSM de la figura 4-14, determine la frecuencia con que puede enviar marcos de datos cualquier usuario dado.
13. Suponga que A, B y C transmiten simultáneamente bits 0 usando un sistema CDMA con las secuencias de *chips* de la figura 4-16(b). ¿Cuál es la secuencia de *chips* resultante?
14. En el análisis sobre la ortogonalidad de las secuencias de *chips* del CDMA se indicó que, si $S \bullet T = 0$, entonces $S \bullet \overline{T}$ también es 0. Compruebe esto.
15. Considere una manera diferente de ver la propiedad de ortogonalidad de las secuencias de *chips* del CDMA. Cada bit de un par de secuencia puede ser igual o diferente. Exprese la propiedad de ortogonalidad en términos de igualdades y diferencias.
16. Un receptor CDMA recibe los siguientes *chips*: (−1 +1 −3 +1 −1 −3 +1 +1). Suponiendo las secuencias de *chips* definidas en la figura 4-16(b), ¿cuáles estaciones transmitieron y qué bits envió cada una?
17. Un edificio de oficinas de siete pisos tiene 15 oficinas adyacentes por piso. Cada oficina contiene un enchufe de pared para una terminal en la pared frontal, por lo que los enchufes forman una retícula triangular en el plano vertical. Suponiendo que es factible tender un cable recto entre cualquier par de enchufes, horizontal, vertical o diagonalmente, ¿cuántos metros de cable se necesitan para conectar todos los enchufes usando
   (a) una configuración en estrella con un solo enrutador en medio?
   (b) una LAN 802.3?
   (c) una red en anillo (sin centro de alambrado)?

18. ¿Cuál es la tasa de baud de la LAN 802.3 de 10 Mbps estándar?
19. Una LAN CSMA/CD (no la 802.3) de 10 Mbps y 1 km de largo tiene una velocidad de propagación de 200 m/μseg. Los marcos de datos tienen 256 bits de longitud, incluidos 32 bits de cabecera, suma de comprobación y un poco más de información extra. La primera ranura de bits tras una transmisión exitosa se reserva para que el receptor capture el canal y envíe un marco de acuse de 32 bits. ¿Cuál es la tasa de datos efectiva, excluyendo la carga extra, suponiendo que no hay colisiones?
20. Dos estaciones CSMA/CD intentan transmitir archivos grandes (multimarco). Tras el envío de cada marco, contienden por el canal usando el algoritmo de retroceso exponencial binario. ¿Cuál es la probabilidad de que la contención termine en la ronda k, y cuál es la cantidad media de rondas por periodo de contención?
21. Considere la construcción de una red CSMA/CD que opera a 1 Gbps a través de un cable de 1 km de longitud sin repetidores. La velocidad de la señal en el cable es de 200,000 km/seg. ¿Cuál es el tamaño mínimo de marco?
22. Dibuje la codificación Manchester para la cadena de bits 0001110101.
23. Dibuje la codificación Manchester diferencial para la cadena de bits del problema anterior. Suponga que la línea inicialmente está en el estado LO.
24. Un sistema *token bus* funciona como sigue. Cuando llega la ficha a una estación, se restablece un temporizador en 0. La estación entonces comienza a transmitir marcos de prioridad 6 hasta que el temporizador alcanza *T6*; luego se conmuta a los marcos de prioridad 4 hasta que el temporizador alcanza *T4*. Este algoritmo se repite entonces con prioridad 2 y prioridad 0. Si todas las estaciones tienen valores de temporización de 40, 80, 90 y 100 mseg para *T6* a *T0*, respectivamente, ¿qué fracción del ancho de banda total se reserva para cada clase de prioridad?
25. ¿Qué ocurre en un *token bus* si una estación acepta la ficha y se cae de inmediato? ¿Cómo maneja este caso el protocolo descrito en el texto?
26. Con una tasa de transmisión de 5 Mbps y una velocidad de propagación de 200 m/μseg, ¿a cuántos metros de cable equivale el retardo de 1 bit de una interfaz *token ring*?
27. El retardo total de un *token ring* debe ser suficiente para contener la ficha completa. Si el alambre no tiene la longitud suficiente, debe introducirse un retardo artificial. Explique por qué es necesario este retardo extra en el contexto de una ficha de 24 bits y un anillo con sólo 16 bits de retardo.
28. Un *token ring* de 1 km de longitud y 10 Mbps con carga muy alta tiene una velocidad de propagación de 200 m/μseg. Hay 50 estaciones a intervalos uniformes alrededor del anillo. Los marcos de datos son de 256 bits, incluidos 32 bits de carga extra. Se incorporan acuses a los marcos de datos, y por tanto se incluyen como bits de repuesto en los marcos de datos, por lo que efectivamente están libres. La ficha es de 8 bits. ¿La tasa de datos efectiva de este anillo es mayor o menor que la de una red CSMA/CD de 10 Mbps?
29. En un *token ring*, el transmisor retira el marco. ¿Qué modificaciones al sistema serían necesarias para hacer que el receptor retire el marco, y cuáles serían las consecuencias?
30. Un *token ring* de 4 Mbps tiene un valor de temporización de 10 mseg para la retención de la ficha. ¿De qué tamaño es el marco más grande que puede enviarse en este anillo?
31. ¿Tiene influencia el uso de un centro de alambrado en el desempeño del *token ring*?

32. Un *token ring* de fibra óptica empleado como MAN tiene 200 km de longitud y opera a 100 Mbps. Tras enviar un marco, la estación drena el marco del anillo antes de regenerar la ficha. La velocidad de propagación de la señal a través de la fibra es de 200,000 km/seg y el tamaño máximo de marco es de 1K bytes. ¿Cuál es la eficiencia máxima del anillo (ignorando todas las demás fuentes de carga extra)?

33. En la figura 4-32, la estación *D* quiere enviar una célula. ¿A qué estación quiere enviarla?

34. El sistema de la figura 4-32 está inactivo. Un poco después, las estaciones *C*, *A* y *B* quedan listas para enviar, en ese orden y en secuencia rápida. Suponiendo que no se transmiten marcos de datos hasta que las tres han enviado una solicitud corriente arriba, muestre los valores de *RC* y de *CD* tras cada solicitud y tras los tres marcos de datos.

35. A veces se dice que el Ethernet es inadecuado para la computación en tiempo real, pues la ranura de retransmisión de peor caso no está limitado. ¿En qué circunstancias puede aplicarse el mismo argumento al *token ring*? ¿En qué circunstancias tiene un peor caso conocido el *token ring*? Suponga que el número de estaciones del *token ring* es fijo y se conoce.

36. Los marcos de Ethernet deben tener cuando menos 64 bytes de longitud para asegurar que el transmisor aún esté transmitiendo en caso de haber una colisión en el otro extremo del cable. El Ethernet rápido tiene el mismo tamaño mínimo de marco de 64 bytes, pero puede enviar los bits a una velocidad 10 veces mayor. ¿Cómo es posible mantener el mismo tamaño mínimo de marco?

37. Imagine dos puentes LAN que conectan, ambos, un par de redes 802.4. El primer puente confronta 1000 marcos por segundo de 512 bytes que deben reenviarse. El segundo confronta 200 marcos por segundo de 4096 bytes. ¿Qué puente cree usted que necesitará una CPU más rápida? Explique.

38. Suponga que los dos puentes del problema anterior conectan, ambos, una LAN 802.4 con una LAN 802.5. ¿Afectaría ese cambio la respuesta anterior?

39. Un puente entre una LAN 802.3 y una 802.4 tiene problemas de errores de memoria intermitentes. ¿Puede causar esto errores no detectados en los marcos transmitidos, o serán atrapados los errores gracias a las sumas de comprobación de los marcos?

40. El departamento de ciencias de la computación de una universidad tiene tres segmentos Ethernet conectados mediante dos puentes transparentes a una red lineal. Un día, el administrador renuncia y lo reemplaza apresuradamente alguien del centro de cómputo, que es una instalación *token ring* de IBM. El nuevo administrador, notando que los extremos de la red no están conectados, de inmediato ordena un puente transparente nuevo y conecta ambos extremos sueltos a él, formando un anillo cerrado. ¿Qué ocurre después?

41. Un anillo FDDI grande tiene 100 estaciones y un tiempo de rotación de ficha de 40 mseg. El tiempo de retención de la ficha es de 10 mseg. ¿Cuál es la eficiencia máxima que puede tener el anillo?

42. Considere la construcción de una interconexión de supercomputadoras usando el enfoque HIPPI, pero tecnología moderna. La trayectoria de datos ahora es de 64 bits de ancho y puede enviarse una palabra cada 10 nseg. ¿Cuál es el ancho de banda del canal?

43. En el texto se indicó que un satélite con dos canales de ALOHA ranurado de enlace ascendente y uno de enlace descendente puede lograr una utilización del enlace descendente de 0.736, dada una cantidad infinita de espacio de *buffer*. Muestre la manera en que puede lograrse este resultado.

5

LA CAPA DE RED

La capa de red se encarga de llevar los paquetes desde el origen hasta el destino. Llegar al destino puede requerir muchos saltos por enrutadores intermedios. Esta función ciertamente contrasta con la de la capa de enlace de datos, que sólo tiene la meta modesta de mover marcos de un extremo del alambre al otro. Por tanto, la capa de red es la capa más baja que maneja la transmisión de punta a punta. Para mayor información sobre ella, véase (Huitema, 1995, y Perlman, 1992).

Para lograr su cometido, la capa de red debe conocer la topología de la subred de comunicación (es decir, el grupo de enrutadores) y escoger las trayectorias adecuadas a través de ella; también debe tener cuidado de escoger las rutas a modo de evitar la carga extra de algunas de las líneas de comunicación y de los enrutadores mientras deja a otros sin trabajo. Por último, cuando el origen y el destino están en redes diferentes, es responsabilidad de la capa de red el manejo de estas diferencias y la resolución de los problemas que causan. En este capítulo estudiaremos todos estos temas y los ilustraremos con nuestros dos ejemplos, Internet y ATM.

5.1. TEMAS DE DISEÑO DE LA CAPA DE RED

En las siguientes secciones presentaremos una introducción a algunos de los problemas que deben enfrentar los diseñadores de la capa de red. Estos temas incluyen el servicio proporcionado a la capa de transporte y el diseño interno de la subred.

5.1.1. Servicios proporcionados a la capa de transporte

La capa de red proporciona servicios a la capa de transporte en la interfaz capa de red/capa de transporte. Esta interfaz muchas veces tiene especial importancia por otra razón: con frecuencia es la interfaz entre la portadora y el cliente, es decir, el límite de la subred. La portadora suele controlar los protocolos y las interfaces hasta la capa de red, inclusive; su trabajo es entregar los paquetes entregados a ella por sus clientes. Por esta razón, esta interfaz debe estar especialmente bien definida.

Los servicios de la capa de red se diseñaron con las siguientes metas en mente.

1. Los servicios deben ser independientes de la tecnología de subred.
2. La capa de transporte debe estar aislada de la cantidad, tipo y topología de las subredes presentes.
3. Las direcciones de red disponibles para la capa de transporte deben seguir un plan de numeración uniforme, aun a través de varias LAN y WAN.

Dadas estas metas, los diseñadores de la capa de red tienen mucha libertad para escribir especificaciones detalladas de los servicios que serán ofrecidos a la capa de transporte. Esta libertad con frecuencia degenera en una batalla campal entre dos facciones en conflicto. La discusión se centra en la interrogante de si la capa de red debe proporcionar servicio orientado a conexión o servicio sin conexiones.

Un bando (representado por la comunidad Internet) alega que la tarea de la subred es mover bits de un lado a otro, y nada más. Desde su punto de vista (basado en casi 30 años de experiencia con una red de computadoras real y operativa), la subred es inherentemente inestable, sin importar su diseño. Por tanto, los *hosts* deben aceptar el hecho de que es inestable y efectuar ellos mismos el control de errores (es decir, detección y corrección de errores) y el control de flujo.

Este punto de vista conduce directamente a la conclusión de que el servicio de red debe ser sin conexiones, tener las primitivas ENVIAR PAQUETE (*SEND PACKET*) y RECIBIR PAQUETE (*RECEIVE PACKET*), y casi nada más. En particular, no debe efectuarse ningún ordenamiento de paquetes ni control de flujo, pues de todos modos los *hosts* lo van a efectuar y probablemente se ganaría poco haciéndolo dos veces. Además, cada paquete debe llevar la dirección de destino completa, porque cada paquete enviado se transporta independientemente de sus antecesores, si los hay.

El otro bando (representado por las compañías telefónicas) argumenta que la subred debe proporcionar un servicio (razonablemente) confiable, orientado a conexión. Indican que una buena guía son 100 años de experiencia exitosa del sistema telefónico mundial. Desde este punto de vista, las conexiones deben tener las siguientes propiedades:

1. Antes de enviar datos, un proceso de capa de red en el lado transmisor debe establecer una conexión con su igual en el lado receptor. Se usa entonces esta conexión, que recibe un identificador especial, hasta que se han enviado todos los datos, en cuyo momento se libera explícitamente.
2. Al establecerse una conexión, los dos procesos pueden comenzar a negociar los parámetros, calidad y costo del servicio a proporcionar.
3. La comunicación es en ambas direcciones y los paquetes se entregan en secuencia.
4. Se proporciona automáticamente control de flujo para evitar que un transmisor rápido descargue paquetes en el conducto con mayor rapidez que aquella con que puede retirarlos el receptor, lo que conduce a un desbordamiento.

Son opcionales otras propiedades, como la entrega garantizada, la confirmación explícita de la entrega y los paquetes de alta prioridad. Como indicamos en el capítulo 1, el servicio sin conexiones es como el sistema postal y el servicio orientado a conexión es como el sistema telefónico.

La controversia entre el servicio orientado a conexión y el servicio sin conexiones en realidad tiene que ver con dónde está la complejidad. En el servicio orientado a conexión, está en la capa de red (subred); en el servicio sin conexiones, está en la capa de transporte (*hosts*). Los defensores del servicio sin conexiones dicen que la capacidad de cómputo de usuario se ha vuelto barata, por lo que no hay razón para no poner la complejidad en los *hosts*. Además, argumentan que la subred es una inversión (inter)nacional importante que durará décadas, por lo que no debe sobrecargarse con características que podrían volverse obsoletas rápidamente, pero que tendrán que calcularse como parte de la estructura de precios durante muchos años. Además, algunas aplicaciones, como la voz digitalizada y la recolección de datos en tiempo real, pueden considerar mucho más importante la entrega *rápida* que la entrega *exacta*.

Por otra parte, los defensores del servicio orientado a conexión dicen que la mayoría de los usuarios no tienen interés en ejecutar protocolos complejos de capa de transporte en sus máquinas. Lo que quieren es servicio confiable y libre de problemas, y este servicio puede proporcionarse mejor con conexiones de capa de red. Además, es más fácil proporcionar algunos servicios, como el audio y el vídeo de tiempo real, a través de una capa de red orientada a conexiones que a través de una capa de red sin conexiones.

Aunque pocas veces la discusión es en estos términos, aquí están implicados dos temas diferentes. Primero, si la red está orientada a conexión (se requiere preparación) o es sin conexiones (no se requiere preparación). Segundo, si la red es confiable (sin paquetes perdidos, duplicados ni alterados) o no confiable (los paquetes pueden perderse, duplicarse o alterarse). En teoría, existen las cuatro combinaciones, pero las combinaciones dominantes son confiable orientada a conexión y no confiable sin conexiones, por lo que las otras dos tienden a perderse en el alboroto.

Estos bandos están representados por nuestros dos ejemplos operativos. El Internet tiene una capa de red sin conexiones, y las redes ATM tienen una capa de red orientada a conexión. Surge una pregunta obvia sobre la manera en que trabaja Internet cuando atraviesa una subred con portadora basada en ATM. La respuesta es que el *host* de origen establece primero una conexión

ATM de capa de red con el *host* de destino, y luego envía por ella paquetes independientes (IP), como se muestra en la figura 5-1. Aunque este enfoque funciona, es ineficiente debido a que cierta funcionalidad está en ambas capas. Por ejemplo, la capa de red ATM garantiza que los paquetes siempre se entreguen en orden, pero el código TCP de todos modos contiene el mecanismo completo de administración y reordenamiento de paquetes desordenados. Para mayor información sobre la manera de ejecutar IP sobre ATM, véase el RFC 1577 y (Armitage y Adams, 1995).

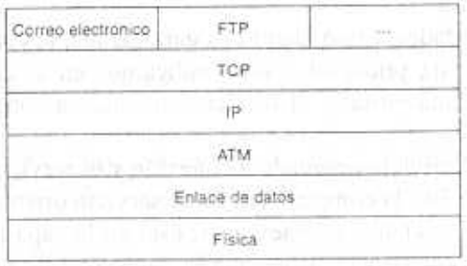


Figura 5-1. Ejecución de TCP/IP en una subred ATM.

5.1.2. Organización interna de la capa de red

Ahora que hemos visto las dos clases de servicio que puede proporcionar la capa de red a sus usuarios, es tiempo de ver su funcionamiento interno. Básicamente hay dos filosofías diferentes de organización de la subred, una que usa conexiones y otra que funciona sin conexiones. En el contexto de la operación *interna* de la subred, generalmente se llama **circuito virtual** a una conexión, por analogía con los circuitos físicos instalados por el sistema telefónico. Los paquetes independientes de la organización de tipo sin conexiones se llaman **datagramas**, por analogía con los telegramas.

Los circuitos virtuales generalmente se usan en subredes cuyo servicio primario está orientado a conexión, por lo que los describiremos en ese contexto. La idea en que se basan los circuitos virtuales es evitar tener que escoger una ruta nueva para cada paquete o célula enviada. En cambio, cuando se establece una conexión, se escoge y se recuerda una ruta de la máquina de origen a la de destino como parte del establecimiento de la conexión. Esa ruta se usa para todo el tráfico que fluye por la conexión, de la misma manera en que funciona el sistema telefónico. Al liberarse la conexión, también deja de existir el circuito virtual.

En contraste, en una subred de datagramas no se determinan rutas por adelantado, aun si el servicio está orientado a conexión. Cada paquete enviado se enruta de manera independiente de sus antecesores. Dos paquetes sucesivos pueden seguir rutas distintas. Si bien las subredes de datagramas tienen más trabajo que hacer, generalmente también son más robustas y se adaptan a

fallas y congestionamientos con mayor facilidad que las subredes de circuito virtual. Después estudiaremos las ventajas y desventajas de los dos enfoques.

Si los paquetes que fluyen a través de un circuito virtual dado siempre siguen la misma ruta a través de la subred, cada enrutador debe recordar a dónde enviar los paquetes para cada uno de los circuitos virtuales abiertos que pasan por él. Cada enrutador debe mantener una tabla con una entrada por circuito virtual abierto que pasa a través suyo. Cada paquete que viaja a través de la red debe contener un campo de número de circuito virtual en su cabecera, además de los números de secuencia, sumas de comprobación y demás. Al llegar un paquete al enrutador, el enrutador sabe la línea por la que llegó y el número del circuito virtual. Utilizando sólo esa información, debe enviar el paquete por la línea de salida correcta.

Al establecerse una conexión de red, se escoge un número de circuito virtual actualmente desocupado en esa máquina como identificador de la conexión. Dado que cada máquina escoge independientemente los números de circuito virtual, estos números tienen sólo significado local. Si tuvieran un significado global en toda la red, es posible que dos circuitos virtuales que tuvieran el mismo número de circuito virtual global pasaran a través de un mismo enrutador intermedio, provocando ambigüedades.

Dado que los circuitos virtuales pueden iniciarse desde cualquiera de los dos lados, ocurre un problema cuando se propagan establecimientos de llamada en ambas direcciones al mismo tiempo a lo largo de una cadena de enrutadores. En algún punto, habrán llegado a enrutadores adyacentes. Cada enrutador debe escoger ahora un número de circuito virtual para el circuito (dúplex integral) que está tratando de establecer. Si los enrutadores están programados para escoger el número menor no utilizado en el enlace, escogerán el mismo número, haciendo que dos circuitos virtuales no relacionados en la misma línea física tengan el mismo número. Cuando llega posteriormente un paquete de datos, el enrutador receptor no tiene manera de saber si se trata de un paquete de ida por un circuito o un paquete de regreso por el otro. Si los circuitos son símplex, no hay ambigüedad.

Nótese que se requiere que cada proceso indique cuándo ha terminado de usar un circuito virtual, de modo que el circuito virtual pueda purgarse de las tablas del enrutador y así recuperar el espacio. En las redes públicas, la motivación es el "palo", en lugar de la "zanahoria": a los usuarios siempre se les cobra por el tiempo de conexión, así como por los datos transportados. Además, debe tomarse alguna medida para manejar las máquinas que terminan sus circuitos virtuales con una caída abrupta, en lugar de liberarlos cortésmente.

Hasta aquí el uso de circuitos virtuales internos de la subred. La otra posibilidad es usar datagramas internamente, en cuyo caso los enrutadores no tienen una tabla con una entrada por cada circuito virtual abierto; en cambio, tienen un tabla que indica la línea de salida a usar para cada enrutador de destino posible. Estas tablas también son necesarias al usar internamente circuitos virtuales, a fin de determinar la ruta de un paquete de establecimiento.

Cada datagrama debe contener la dirección de destino completa. En una red grande, estas direcciones pueden ser bastante largas (por ejemplo, una docena de bytes o más). Al llegar un paquete, el enrutador busca la línea de salida a usar y envía el paquete. El establecimiento y liberación de conexiones de capa de red o de transporte no requiere ningún trabajo especial por parte de los enrutadores.

5.1.3. Comparación de las subredes de circuitos virtuales y de datagramas

Tanto los circuitos virtuales como los datagramas tienen sus seguidores y sus detractores. Ahora intentaremos resumir los argumentos de ambos bandos. Los aspectos principales se listan en la figura 5-2, aunque los puristas probablemente podrán encontrar ejemplos contrarios para todo lo indicado en la figura.

Asunto	Subred de datagramas	Subred de CV
Establecimiento del circuito	No necesaria	Requerida
Direccionamiento	Cada paquete contiene completas la dirección de origen y de destino	Cada paquete contiene un número de CV corto
Información de estado	La subred no contiene información de estado	Cada CV requiere espacio de tabla de la subred
Enrutamiento	Cada paquete se enruta independientemente	Ruta escogida cuando se establece el CV; todos los paquetes siguen esta ruta
Efecto de fallas del enrutador	Ninguno, exceptuando paquetes perdidos durante una caída	Terminan todos los CV que pasan a través del enrutador con falla
Control de congestionamientos	Difícil	Fácil si pueden asignarse por adelantado *buffers* suficientes a cada CV

Figura 5-2. Comparación de las subredes de datagramas y de circuitos virtuales.

Dentro de la subred, hay varias diferencias entre lo que sacrifican los circuitos virtuales y los datagramas. Una de ellas concierne al espacio de memoria del enrutador y el ancho de banda. Los circuitos virtuales permiten que los paquetes contengan números de circuito en lugar de direcciones de destino completas. Si los paquetes suelen ser bastante cortos, una dirección de destino completa en cada paquete puede representar una carga extra significativa y, por tanto, ancho de banda desperdiciado. El precio que paga por el uso interno de circuitos virtuales es el espacio de tabla en los enrutadores. Dependiendo del costo relativo de los circuitos de comunicación y de la memoria de enrutador, alguna de las dos cosas podría ser más barata.

Otro balance es el del tiempo de establecimiento contra el tiempo de análisis de la dirección. El uso de circuitos virtuales requiere una fase de establecimiento, que consume tiempo y recursos. Sin embargo, la determinación de lo que hay que hacer con un paquete de datos en una subred de circuitos virtuales es fácil: el enrutador simplemente usa el número de circuito para buscar en una tabla indizada y encontrar el lugar al que va el paquete. En una subred de datagramas se requiere un procedimiento más complicado para determinar a dónde va el paquete.

Los circuitos virtuales tienen algunas ventajas en cuanto a que evitan congestionamientos en la subred, pues los recursos pueden reservarse por adelantado al establecerse la conexión. Una vez que comienzan a llegar los paquetes, estarán ahí el ancho de banda y la capacidad de enrutamiento necesarios. Con una subred de datagramas, es más difícil evitar los congestionamientos.

Para los sistemas de procesamiento de transacciones (por ejemplo, las tiendas que llaman para verificar pagos con tarjeta de crédito), la carga extra requerida para establecer y terminar un circuito virtual puede ocupar mucho más tiempo que el uso real del circuito. Si la mayor parte del tráfico esperado es de este tipo, el uso de circuitos virtuales conmutados en la subred tiene poco sentido. Por otra parte, pueden ser de utilidad aquí los circuitos virtuales permanentes, establecidos manualmente y con duración de meses o años.

Los circuitos virtuales también tienen un problema de vulnerabilidad. Si se cae un enrutador, perdiéndose su memoria, tendrán que abortarse todos los circuitos virtuales que pasan por él, aun si se recupera un segundo después. Por el contrario, al caerse un enrutador de datagramas sólo sufrirán aquellos usuarios cuyos paquetes estaban encolados en el enrutador en el momento y, dependiendo de si ya habían sido reconocidos o no, tal vez ni siquiera todos. La pérdida de una línea de comunicación es fatal para los circuitos virtuales que la usan, pero puede compensarse fácilmente cuando se usan datagramas. Los datagramas también permiten que los enrutadores equilibren el tráfico a través de la subred, ya que las rutas pueden cambiarse a la mitad de una conexión.

Es importante indicar explícitamente que el servicio ofrecido (orientado a conexión o sin conexiones) es un asunto independiente de la estructura de la subred (circuitos virtuales o datagramas). En teoría, son posibles las cuatro combinaciones. Obviamente, son razonables la implementación con circuitos virtuales de un servicio orientado a conexión y la implementación con datagramas de un servicio sin conexiones. También tiene sentido la implementación de conexiones usando datagramas cuando la subred está tratando de proporcionar un servicio muy robusto.

La cuarta posibilidad, el servicio sin conexiones por una subred de circuitos virtuales, parece extraña, pero ciertamente ocurre. El ejemplo obvio es ejecutar IP en una subred ATM. Aquí se desea operar un protocolo sin conexiones existente sobre una nueva capa de red orientada a conexiones. Como se mencionó antes, ésta es más bien una solución *ad hoc* para un problema, no un buen diseño. En un sistema nuevo diseñado para operar en una subred ATM, normalmente no se pondría un protocolo sin conexiones, como IP, sobre una capa de red orientada a conexión, como ATM, agregando después un protocolo de transporte orientado a conexión encima del protocolo sin conexiones. En la figura 5-3 se muestran ejemplos de los cuatro casos.

5.2. ALGORITMOS DE ENRUTAMIENTO

La función principal de la capa de red es enrutar paquetes de la máquina de origen a la de destino. En la mayoría de las subredes, los paquetes requerirán varias escalas para completar el viaje. La única excepción importante son las redes de difusión, pero aun aquí es importante

Capa superior	Tipo de subred: Datagramas	Tipo de subred: Circuitos virtuales
Sin conexiones	UDP sobre IP	UDP sobre IP sobre ATM
Orientada a conexiones	TCP sobre IP	ATM AAL1 sobre ATM

Figura 5-3. Ejemplos de diferentes combinaciones de estructuras de servicios y subredes.

el enrutamiento si el origen y el destino no están en la misma red. Los algoritmos que escogen las rutas y las estructuras de datos que éstos usan son un área principal del diseño de la capa de red.

El **algoritmo de enrutamiento** es aquella parte del *software* de la capa de red encargada de decidir la línea de salida por la que se transmitirá un paquete de entrada. Si la subred usa datagramas internamente, esta decisión debe hacerse cada vez que llega un paquete de datos de entrada, dado que la mejor ruta podría haber cambiado desde la última vez. Si la subred usa circuitos virtuales internamente, las decisiones de enrutamiento se toman sólo al establecerse un circuito virtual nuevo. Por tanto, los paquetes de datos simplemente siguen la ruta previamente establecida. Este último caso a veces se llama **enrutamiento de sesión**, dado que una ruta permanece vigente durante la sesión de usuario completa (por ejemplo, durante una sesión interactiva desde una terminal, o durante una transferencia de archivo).

Sea que se escojan independientemente las rutas para cada paquete o sólo al establecerse nuevas conexiones, hay ciertas propiedades que son deseables en un algoritmo de enrutamiento: corrección, sencillez, robustez, estabilidad, equitatividad y optimalidad. La corrección y la sencillez apenas requieren comentarios, pero la necesidad de robustez puede ser menos obvia a primera vista. Una vez que entra en operación una red principal, puede esperarse que opere continuamente durante años sin fallas generales. Durante ese periodo habrá fallas de *hardware* y de *software* de todos tipos. Los *hosts*, enrutadores y líneas se activarán y desactivarán repetidamente y cambiará muchas veces la topología. El algoritmo de enrutamiento debe ser capaz de manejar los cambios de topología y tráfico sin requerir el aborto de todas las actividades en todos los *hosts* y el rearranque de la red con cada caída de un enrutador.

La estabilidad también es una meta importante del algoritmo de enrutamiento. Existen algoritmos de enrutamiento que nunca convergen en el equilibrio, sin importar el tiempo que permanezcan operativos. La equitatividad y la optimalidad pueden parecer algo obvio (ciertamente nadie se opondrá a ellas), pero resulta que con frecuencia son metas contradictorias. Para un ejemplo sencillo de este conflicto, véase la figura 5-4. Supóngase que hay suficiente tráfico entre A y A', entre B y B' y entre C y C' para saturar los enlaces horizontales. A fin de aumentar al máximo el flujo total, el tráfico de X a X' debe suspenderse por completo. Por desgracia, X y

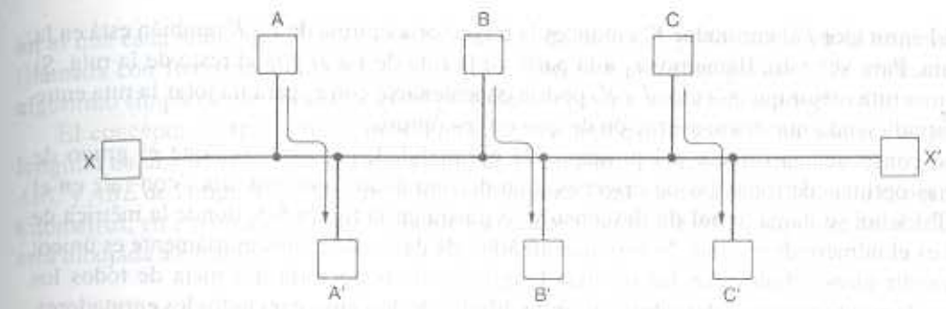


Figura 5-4. El conflicto entre equitatividad y optimización.

X' podrían no verlo de esta manera. Evidentemente se requiere una concesión entre la eficiencia global y la equitatividad hacia las conexiones individuales.

Antes de que podamos siquiera intentar encontrar un justo medio entre la equitatividad y la optimalidad, debemos decidir qué es lo que buscamos optimar. Un candidato obvio es la minimización del retardo medio de los paquetes, pero también lo es el aumento al máximo del rendimiento total de la red. Además, estas dos metas también están en conflicto, ya que la operación de cualquier sistema de colas cerca de su capacidad máxima implica un retardo de encolamiento grande. Como término medio, muchas redes intentan minimizar el número de escalas que tiene que hacer un paquete, puesto que la reducción de la cantidad de escalas tiende a reducir el retardo y también el consumo de ancho de banda, lo que tiende a mejorar el rendimiento.

Los algoritmos de enrutamiento pueden agruparse en dos clases principales: no adaptables y adaptables. Los **algoritmos no adaptables** no basan sus decisiones de enrutamiento en mediciones o estimaciones del tráfico y la topología actuales. En cambio, la decisión de qué ruta se usará para llegar de I a J (para todas las I y J) se calcula por adelantado, fuera de línea, y se carga en los enrutadores al iniciar la red. Este procedimiento se llama **enrutamiento estático**.

Los **algoritmos adaptables**, en contraste, cambian sus decisiones de enrutamiento para reflejar los cambios de topología, y generalmente también el tráfico. Los algoritmos adaptables difieren en el lugar de obtención de su información (por ejemplo, localmente, de los enrutadores adyacentes o de todos los enrutadores), el momento de cambio de sus rutas (por ejemplo, cada ΔT segundos, cuando cambia la carga o cuando cambia la topología), y la métrica usada para la optimalidad (por ejemplo, distancia, número de escalas o tiempo estimado de tránsito). En las siguientes secciones estudiaremos una variedad de algoritmos de enrutamiento, tanto estáticos como dinámicos.

5.2.1. Principio de optimación

Antes de entrar en algoritmos específicos, puede ser útil señalar que es posible hacer un postulado general sobre las rutas óptimas sin importar la topología o el tráfico de la red. Este postulado se conoce como **principio de optimación**, y establece que, si el enrutador J está en la trayectoria

óptima del enrutador I al enrutador K, entonces la trayectoria óptima de J a K también está en la misma ruta. Para ver esto, llamemos r_1 a la parte de la ruta de I a J, y r_2 al resto de la ruta. Si existiera una ruta mejor que r_2 entre J y K, podría concatenarse con r_1 para mejorar la ruta entre I y K, contradiciendo nuestra aseveración de que r_1r_2 es óptima.

Como consecuencia directa del principio de optimalidad, podemos ver que el grupo de trayectorias óptimas de todas las de orígenes a un destino dado forma un árbol con raíz en el destino. Tal árbol se llama **árbol de descenso** y se ilustra en la figura 5-5, donde la métrica de distancia es el número de escalas. Nótese que un árbol de descenso no necesariamente es único; pueden existir otros árboles con las mismas longitudes de trayectoria. La meta de todos los algoritmos de enrutamiento es descubrir y usar los árboles de descenso para todos los enrutadores.

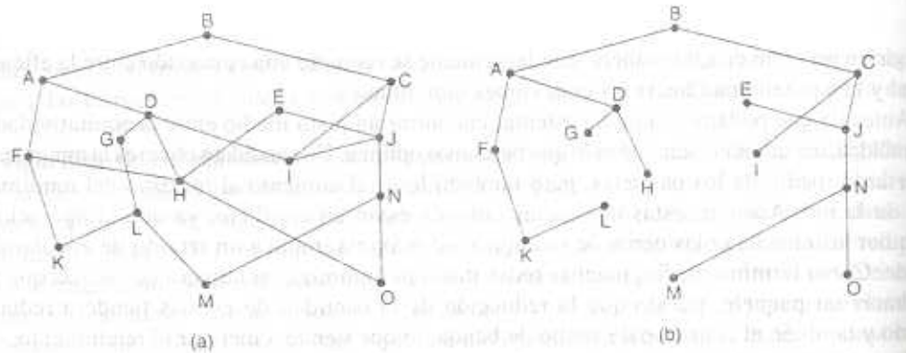


Figura 5-5. (a) Una subred. (b) Árbol descendente para el enrutador B.

Dado que un árbol de descenso ciertamente es un árbol, no contiene ciclos, por lo que cada paquete será entregado con un número de escalas finito y limitado. En la práctica, la vida no es tan fácil. Los enlaces y los enrutadores pueden caerse y reactivarse durante la operación, por lo que diferentes enrutadores pueden tener ideas distintas sobre la topología actual. También hemos evadido calladamente la cuestión de si cada enrutador tiene que adquirir individualmente la información en la cual basa su cálculo del árbol de descenso, o si esta información se obtiene por otros medios. Regresaremos a estos asuntos pronto. Con todo, el principio de optimalidad y el árbol de descenso proporcionan parámetros contra los que se pueden medir otros algoritmos de enrutamiento.

En las tres secciones siguientes veremos tres algoritmos estáticos de enrutamiento. Después, pasaremos a los adaptables.

5.2.2. Enrutamiento por la trayectoria más corta

Comencemos nuestro estudio de los algoritmos de enrutamiento con una técnica de amplio uso en muchas formas, porque es sencilla y fácil de entender. La idea es armar un grafo de la subred,

en el que cada nodo representa un enrutador y cada arco del grafo una línea de comunicación (llamada con frecuencia enlace). Para escoger una ruta entre un par dado de enrutadores, el algoritmo simplemente encuentra en el grafo la trayectoria más corta entre ellos.

El concepto de **trayectoria más corta** merece una explicación. Una manera de medir la longitud de una trayectoria es por la cantidad de escalas. Usando esta métrica, las trayectorias ABC y ABE de la figura 5-6 tienen la misma longitud. Otra métrica es la distancia geográfica en kilómetros, en cuyo caso ABC claramente es mucho mayor que ABE (suponiendo que la figura está dibujada a escala).

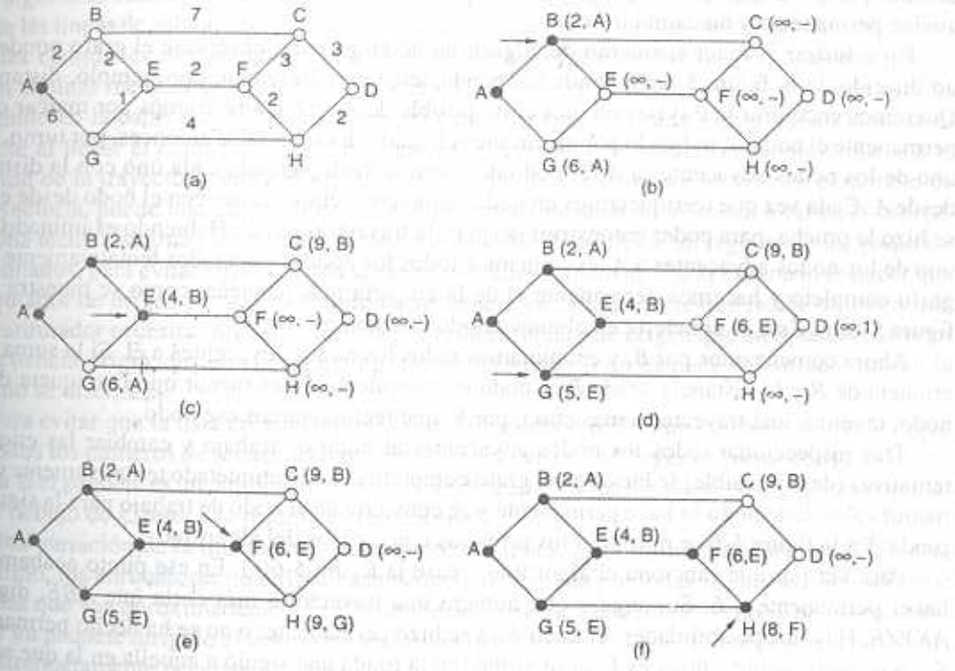


Figura 5-6. Los primeros cinco pasos del cálculo de la trayectoria más corta de A a D. Las flechas indican el nodo de trabajo.

Sin embargo, también son posibles muchas otras métricas además de las escalas y la distancia física. Por ejemplo, cada arco podría etiquetarse con el retardo medio de encolamiento y transmisión de un paquete de prueba estándar, determinado por series de prueba cada hora. Con estas etiquetas en el grafo, la trayectoria más corta es la trayectoria más rápida, en lugar de la trayectoria con menos arcos o kilómetros.

En el caso más general, las etiquetas de los arcos podrían calcularse como una función de la distancia, ancho de banda, tráfico medio, costo de comunicación, longitud media de las colas,

retardo medido y otros factores. Cambiando la función de ponderación, el algoritmo calcularía la trayectoria medida "más corta" de acuerdo con cualquiera de varios criterios, o una combinación de criterios.

Se conocen varios algoritmos de cálculo de la trayectoria más corta entre dos nodos de un grafo. Éste se debe a Dijkstra (1959). Cada nodo se etiqueta (entre paréntesis) con su distancia al nodo de origen a través de la mejor trayectoria conocida. Inicialmente no se conocen trayectorias, por lo que todos los nodos tienen la etiqueta infinito. A medida que avanza el algoritmo y se encuentran trayectorias, pueden cambiar las etiquetas, reflejando mejores trayectorias. Una etiqueta puede ser tentativa o permanente. Inicialmente todas las etiquetas son tentativas. Al descubrirse que una etiqueta representa la trayectoria más corta posible del origen a ese nodo, se vuelve permanente y no cambiar más.

Para ilustrar el funcionamiento del algoritmo de etiquetado, obsérvese el grafo ponderado no dirigido de la figura 5-6(a), donde las ponderaciones representan, por ejemplo, distancias. Queremos encontrar la trayectoria más corta posible de *A* a *D*. Comenzamos por marcar como permanente el nodo *A*, indicado por un círculo rellenado. Examinamos entonces, por turno, cada uno de los nodos adyacentes a *A* (el nodo de trabajo), reetiquetando cada uno con la distancia desde *A*. Cada vez que reetiquetamos un nodo, también lo etiquetamos con el nodo desde el que se hizo la prueba, para poder reconstruir después la trayectoria final. Habiendo examinado cada uno de los nodos adyacentes a *A*, examinamos todos los nodos etiquetados tentativamente en el grafo completo y hacemos permanente el de la etiqueta más pequeña, como se muestra en la figura 5-6(b). Éste se convierte en el nuevo nodo de trabajo.

Ahora comenzamos por *B*, y examinamos todos los nodos adyacentes a él. Si la suma de la etiqueta de *B* y la distancia desde *B* al nodo en consideración es menor que la etiqueta de ese nodo, tenemos una trayectoria más corta, por lo que reetiquetamos ese nodo.

Tras inspeccionar todos los nodos adyacentes al nodo de trabajo y cambiar las etiquetas tentativas (de ser posible) se busca en el grafo completo el nodo etiquetado tentativamente con el menor valor. Este nodo se hace permanente y se convierte en el nodo de trabajo para la siguiente ronda. En la figura 5-6 se muestran los primeros cinco pasos del algoritmo.

Para ver por qué funciona el algoritmo, véase la figura 5-6(c). En ese punto acabamos de hacer permanente a *E*. Supóngase que hubiera una trayectoria más corta que *ABE*, digamos *AXYZE*. Hay dos posibilidades: el nodo *Z* ya se hizo permanente, o no se ha hecho permanente. Si ya es permanente, entonces *E* ya se probó (en la ronda que siguió a aquella en la que se hizo permanente *Z*), por lo que la trayectoria *AXYZE* no ha escapado a nuestra atención.

Ahora considere el caso en el que *Z* aún está etiquetado tentativamente. O bien la etiqueta de *Z* es mayor o igual que la de *E*, en cuyo caso *AXYZE* no puede ser una trayectoria más corta que *ABE*, o es menor que la de *E*, en cuyo caso *Z*, y no *E*, se volverá permanente primero, permitiendo probar *E* desde *Z*.

Este algoritmo se da en la figura 5-7. La única diferencia entre el programa y el algoritmo antes descrito es que, en la figura 5-7, calculamos la trayectoria más corta posible comenzando por el nodo terminal, *t*, en lugar del nodo de origen, *s*. Dado que la trayectoria más corta posible de *t* a *s* en un grafo no dirigido es igual a la trayectoria más corta de *s* a *t*, no importa el extremo

por el que comencemos (a menos que haya varias trayectorias más cortas posibles, en cuyo caso la inversión de la búsqueda podría descubrir una distinta). La razón de una búsqueda en reversa es que cada nodo está etiquetado con su antecesor, en lugar de su sucesor. Al copiar la trayectoria final en la variable de salida, *path*, la trayectoria de salida se invierte. Al invertir la búsqueda, ambos efectos se cancelan, y la respuesta se produce en el orden correcto.

5.2.3. Inundación

Otro algoritmo estático es la **inundación**, en la que cada paquete de entrada se envía por cada una de las líneas de salida, excepto aquella por la que llegó. La inundación evidentemente genera grandes cantidades de paquetes duplicados; de hecho, una cantidad infinita a menos que se tomen algunas medidas para limitar el proceso. Una de tales medidas es un contador de escalas contenido en la cabecera de cada paquete, el cual disminuye en cada escala, descartándose el paquete al llegar el contador a cero. Idealmente, el contador de escalas debe inicializarse a la longitud de la trayectoria entre el origen y el destino. Si el transmisor no conoce el tamaño de la trayectoria, puede inicializar el contador al peor caso, es decir, el diámetro total de la subred.

Una técnica alterna para ponerle diques a la inundación es llevar un registro de los paquetes diseminados, para evitar enviarlos una segunda vez. Una manera de lograr esta meta es hacer que el enrutador de origen ponga un número de secuencia en cada paquete que recibe de sus *hosts*. Cada enrutador necesita entonces una lista por cada enrutador de origen que indique los números de secuencia originados en ese enrutador que ya ha visto. Si un paquete de entrada está en la lista, no se disemina.

Para evitar que la lista crezca sin límites, cada lista debe incluir un contador, *k*, que indique que todos los números de secuencia hasta *k* ya han sido vistos. Al llegar un paquete, es fácil comprobar si el paquete es un duplicado; de ser así, se descarta. Es más, no se necesita la lista completa por debajo de *k*, pues *k* la resume efectivamente.

Una variación de la inundación, un poco más práctica, es la **inundación selectiva**. En este algoritmo, los enrutadores no envían cada paquete de entrada por todas las líneas, sino sólo por aquellas que van aproximadamente en la dirección correcta. Generalmente no tiene mucho caso enviar un paquete dirigido al oeste a través de una línea dirigida al este, a menos que la topología sea extremadamente peculiar.

La inundación no es práctica en la mayoría de las aplicaciones, pero tiene algunos usos. Por ejemplo, en aplicaciones militares, donde pueden volar en pedazos grandes cantidades de enrutadores en cualquier momento, es altamente deseable la tremenda robustez de la inundación. En las aplicaciones de bases de datos distribuidas a veces es necesario actualizar concurrentemente todas las bases de datos, en cuyo caso puede ser útil la inundación. Un tercer posible uso de la inundación es como métrica contra la que pueden compararse otros algoritmos de enrutamiento. La inundación siempre escoge la trayectoria más corta posible, porque escoge en paralelo todas las trayectorias posibles. En consecuencia, ningún otro algoritmo puede producir un retardo más corto (si ignoramos la carga extra generada por el proceso de inundación mismo).

```
#define MAX_NODES 1024                  /* número máximo de nodos */
#define INFINITY 1000000000             /* un número mayor que cualquier trayectoria máxima */
int n, dist[MAX_NODES][MAX_NODES];      /* dist[i][j] es la distancia entre i y j */

void shortest_path(int s, int t, int path[ ])
{ struct state {                        /* la trayectoria con la que se está trabajando */
    int predecessor;                    /* nodo previo */
    int length;                         /* longitud del origen a este nodo */
    enum {permanent, tentative} label;  /* estado de la etiqueta */
  } state[MAX_NODES];

  int i, k, min;
  struct state *
        p;
  for (p = &state[0]; p < &state[n]; p++) {     /* estado de inicialización */
    p->predecessor = -1;
    p->length = INFINITY;
    p->label = tentative;
  }
  state[t].length = 0; state[t].label = permanent;
  k = t;                                /* k es el nodo de trabajo inicial */
  do {                                  /* ¿hay una trayectoria mejor desde k? */
    for (i = 0; i < n; i++)             /* este grafo tiene n nodos */
        if (dist[k][i] != 0 && state[i].label == tentative) {
            if (state[k].length + dist[k][i] < state[i].length) {
                state[i].predecessor = k;
                state[i].length = state[k].length + dist[k][i];
            }
        }

    /* Encuentra el nodo etiquetado tentativamente con la etiqueta menor. */
    k = 0; min = INFINITY;
    for (i = 0; i < n; i++)
        if (state[i].label == tentative && state[i].length < min) {
            min = state[i].length;
            k = i;
        }
    state[k].label = permanent;
  } while (k != s);

  /* Copia la trayectoria en el arreglo de salida. */
  i = 0; k = s;
  do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}
```

Figura 5-7. Algoritmo de Dijkstra para calcular la trayectoria más corta a través de un grafo.

5.2.4. Enrutamiento basado en flujo

Los algoritmos estudiados hasta ahora sólo toman en cuenta la topología; no consideran la carga. Si, por ejemplo, siempre hay una gran cantidad de tráfico entre A y B en la figura 5-6, entonces podría ser mejor enrutar el tráfico de A a C a través de $AGEFC$, aun cuando esta trayectoria es mucho más larga que ABC. En esta sección veremos un algoritmo estático que usa tanto la topología como la carga para el enrutamiento. Su nombre es **enrutamiento basado en flujo**.

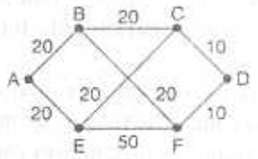
En algunas redes, la tasa media de flujo de datos entre cada par de nodos es relativamente estable y predecible. Por ejemplo, en una red corporativa de una cadena de tiendas de menudeo, cada tienda puede enviar pedidos, informes de ventas, actualizaciones de inventario y otros tipos de mensajes bien definidos a sitios conocidos siguiendo un patrón predefinido, por lo que el volumen total de tráfico varía poco de día a día. En las condiciones en que se conoce por adelantado el tráfico promedio de i a j y es razonablemente constante en el tiempo, es posible analizar matemáticamente los flujos para optimar el enrutamiento.

La idea en que se basa el análisis es que, para una línea dada, si se conocen la capacidad y el flujo promedio es posible calcular el retardo promedio de los paquetes en esa línea a partir de la teoría de colas. De los retardos promedio de todas las líneas, es directo el cálculo de un promedio ponderado por el flujo para obtener el retardo de paquete medio de la subred completa. El problema de enrutamiento se reduce entonces a encontrar el algoritmo de enrutamiento que produzca el retardo promedio mínimo para la subred. Véase la figura 5-8.

Para usar esta técnica, debe conocerse por adelantado cierta información. Primero, debe conocerse la topología de la subred. Segundo, debe estar dada la matriz de tráfico, F_{ij}. Tercero, debe estar disponible la matriz de capacidad, C_{ij}, donde se especifica la capacidad de cada línea en bps. Por último, debe escogerse un algoritmo tentativo de enrutamiento.

Como ejemplo de este método, considere la subred dúplex integral de la figura 5-8(a). Los pesos de los arcos dan las capacidades, C_{ij}, en cada dirección, medidas en kbps. La matriz de la figura 5-8(b) tiene una entrada por cada par origen-destino. La entrada par al origen i al destino j muestra la ruta a usar para el tráfico i-j y también la cantidad de paquetes/seg a enviar del origen i al destino j. Por ejemplo, de B a D van 3 paquetes/seg, y usan la ruta BFD para llegar ahí. Nótese que ya se ha aplicado un algoritmo de enrutamiento para obtener las rutas mostradas en la matriz.

Dada esta información, es directo el cálculo del total de la línea i, λ_i. Por ejemplo, el tráfico B-D contribuye 3 paquetes/seg a la línea BF, y también 3 paquetes/seg a la línea FD. De la misma manera, el tráfico A-D contribuye 1 paquete/seg a cada una de las tres líneas. En la columna λ_i de la figura 5-9 se muestra el tráfico total de cada línea dirigida al este. En este ejemplo, todo el tráfico es simétrico, es decir, el tráfico XY es idéntico al tráfico YX, para todas las X y Y. En las redes reales, esta condición no siempre se mantiene. En la figura también se muestra el número medio de paquetes/seg de cada línea, μC_i, suponiendo un tamaño medio de paquete de $1/\mu = 800$ bits.



(a)

Destino

Origen	A	B	C	D	E	F
A		9 AB	4 ABC	1 ABFD	7 AE	4 AEF
B	9 BA		8 BC	3 BFD	2 BFE	4 BF
C	4 CBA	8 CB		3 CD	3 CE	2 CEF
D	1 DFBA	3 DFB	3 DC		3 DCE	4 DF
E	7 EA	2 EFB	3 EC	3 ECD		5 EF
F	4 FEA	4 FB	2 FEC	4 FD	5 FE	

(b)

Figura 5-8. (a) Subred con capacidades de línea mostradas en kbps. (b) El tráfico en paquetes/seg y la matriz de enrutamiento.

La penúltima columna de la figura 5-9 indica el retardo medio por línea, derivado de la fórmula de la teoría de colas

$$T = \frac{1}{\mu C - \lambda}$$

donde $1/\mu$ es el tamaño medio del paquete en bits, C es la capacidad en bps y λ es el flujo medio en paquetes/seg. Por ejemplo, con una capacidad $\mu C = 25$ paquetes/seg y un flujo real de $\lambda = 14$ paquetes/seg, el retardo medio es de 91 mseg. Nótese que, con $\lambda = 0$, el retardo medio aún es de 40 mseg, porque la capacidad es de 25 paquetes/seg. En otras palabras, el "retardo" incluye tanto encolamiento como tiempo de servicio.

Para calcular el tiempo de retardo medio de la subred completa, tomamos la suma ponderada de cada una de las ocho líneas, siendo el peso la fracción del total de tráfico que usa esa línea. En este ejemplo, la media resulta ser de 86 mseg.

Para evaluar un algoritmo de enrutamiento diferente, podemos repetir el proceso completo, sólo que con diferentes flujos, para obtener un retardo promedio nuevo. Si nos restringimos a algoritmos de enrutamiento de una trayectoria, como hemos hecho hasta ahora, hay un número finito de maneras de enrutar paquetes de cada de origen a cada destino. Siempre es posible escribir un programa que sencillamente las pruebe todas, una tras otra, y encontrar la que tiene el retardo medio menor. Dado que este cálculo puede hacerse por adelantado fuera de línea, el que pueda consumir tiempo no es necesariamente un problema serio. Éste es entonces el mejor algoritmo de enrutamiento. Bertsekas y Gallager (1992) explican detalladamente el enrutamiento basado en flujo.

l	Línea	λ_l (paq./seg)	C_l (kbps)	μC_l (paq./seg)	T_l (mseg)	Peso
1	AB	14	20	25	91	0.171
2	BC	12	20	25	77	0.146
3	CD	6	10	12.5	154	0.073
4	AE	11	20	25	71	0.134
5	EF	13	50	62.5	20	0.159
6	FD	8	10	12.5	222	0.098
7	BF	10	20	25	67	0.122
8	EC	8	20	25	59	0.098

Figura 5-9. Análisis de la subred de la figura 5-8 usando un tamaño medio de paquete de 800 bits. El tráfico en reversa (*BA*, *CB*, etc.) es igual al tráfico directo.

5.2.5. Enrutamiento por vector de distancia

Las computadoras modernas generalmente usan algoritmos de enrutamiento dinámico en lugar de los estáticos antes descritos. En particular, dos algoritmos dinámicos, el enrutamiento por vector de distancia y el enrutamiento por estado de enlace, son los más comunes. En esta sección veremos el primer algoritmo. En la siguiente estudiaremos el segundo.

Los algoritmos de **enrutamiento por vector de distancia** operan haciendo que cada enrutador mantenga una tabla (por ejemplo, un vector) que da la mejor distancia conocida a cada destino y la línea a usar para llegar ahí. Estas tablas se actualizan intercambiando información con los vecinos.

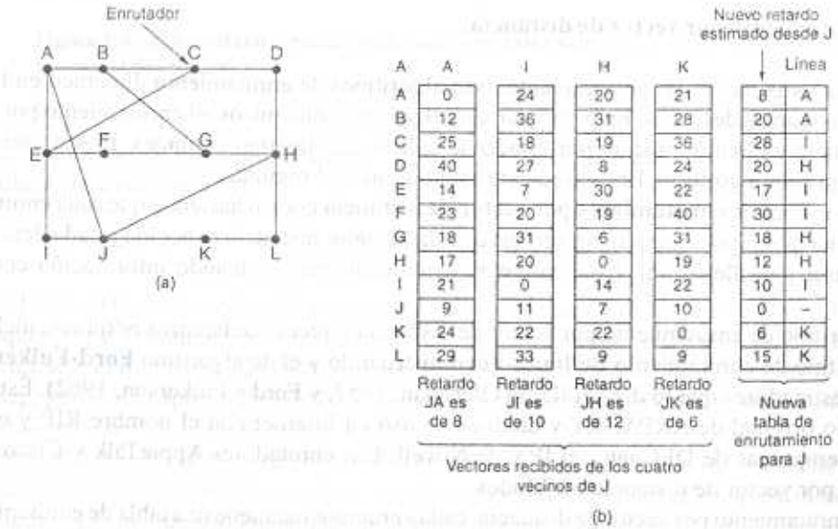
El algoritmo de enrutamiento por vector de distancia a veces recibe otros nombres, incluido el de algoritmo de enrutamiento **Bellman-Ford** distribuido y el de algoritmo **Ford-Fulkerson**, por los investigadores que lo desarrollaron (Bellman, 1957, y Ford y Fulkerson, 1962). Éste fue el algoritmo original de ARPANET y también se usó en Internet con el nombre RIP y en las versiones tempranas de DECnet y el IPX de Novell. Los enrutadores AppleTalk y Cisco usan protocolos por vector de distancia mejorados.

En el enrutamiento por vector de distancia, cada enrutador mantiene una tabla de enrutamiento indizada por, y conteniendo un registro de, cada enrutador de la subred. Esta entrada comprende dos partes: la línea preferida de salida hacia ese destino y una estimación del tiempo o distancia a ese destino. La métrica usada podría ser la cantidad de escalas, el retardo de tiempo en milisegundos, el número total de paquetes encolados por la trayectoria, o algo parecido.

Se supone que el enrutador conoce la "distancia" a cada uno de sus vecinos. Si la métrica es de escalas, la distancia simplemente es una escala. Si la métrica es la longitud de la cola, el enrutador simplemente examina cada cola. Si la métrica es el retardo, el enrutador puede medirlo directamente con paquetes especiales de ECO que el receptor simplemente marca con la hora y envía de regreso tan rápido como puede.

Por ejemplo, supóngase que se usa como métrica el retardo y que el enrutador conoce el retardo a cada uno de sus vecinos. Una vez cada T mseg, cada enrutador envía a todos sus vecinos una lista de sus retardos estimados a cada destino. También recibe una lista parecida de cada vecino. Imagine que una de estas tablas acaba de llegar del vecino X, siendo X_i la estimación de X respecto al tiempo que le toma llegar al enrutador i. Si el enrutador sabe que el retardo a X es de m mseg, también sabe que puede alcanzar el enrutador i a través de X en $X_i + m$ mseg vía X. Efectuando este cálculo para cada vecino, un enrutador puede encontrar la estimación que parezca ser la mejor y usar esa estimación y la línea correspondiente en su nueva tabla de enrutamiento. Nótese que la vieja tabla de enrutamiento no se usa en este cálculo.

Este proceso de actualización se ilustra en la figura 5-10. En la parte (a) se muestra una subred. En las primeras cuatro columnas de la parte (b) aparecen los vectores de retardo recibidos de los vecinos del enrutador J. A indica tener un retardo de 12 mseg a B, un retardo de 25 mseg a C, un retardo de 40 mseg a D, etc. Supóngase que J ha medido o estimado el retardo a sus miembros, A, I, H y K en 8, 10, 12 y 6 mseg, respectivamente.



	A	I	H	K	Nuevo retardo estimado desde J	Línea
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	–
K	24	22	22	0	6	K
L	29	33	9	9	15	K

Figura 5-10. (a) Subred. (b) Entrada de A, I, H, K, y la nueva tabla de enrutamiento de J.

Considere la manera en que J calcula su nueva ruta al enrutador G. Sabe que puede llegar a A en 8 mseg, y A indica ser capaz de llegar a G en 18 mseg, por lo que J sabe que puede contar con un retardo de 26 mseg a G si reenvía a través de A los paquetes destinados a G. Del mismo modo, J calcula el retardo a G a través de I, H y K en 41 (31 + 10), 18 (6 + 12) y 37 (31 + 6) mseg, respectivamente. El mejor de estos valores es 18, por lo que escribe una entrada en su tabla de enrutamiento indicando que el retardo a G es de 18 mseg, y que la ruta a usar es vía H. Se lleva

a cabo el mismo cálculo para los demás destinos, y la nueva tabla de enrutamiento se muestra en la última columna de la figura.

El problema del conteo a infinito

El enrutamiento por vector de distancia funciona en teoría, pero tiene un problema serio en la práctica: aunque converge en la respuesta correcta, puede hacerlo lentamente. En particular, reacciona con rapidez a las buenas noticias, pero con lentitud ante las malas. Considere un enrutador cuya mejor ruta al destino X es larga. Si en el siguiente intercambio el vecino A informa repentinamente un retardo corto a X, el enrutador simplemente se conmuta a modo de usar la línea a A para enviar tráfico hasta X. En un intercambio de vectores, se procesan las buenas noticias.

Para ver la rapidez de propagación de las buenas noticias, considere la subred de cinco nodos (lineal) de la figura 5-11, en la que la métrica de retardo es el número de escalas. Supóngase que A está desactivado inicialmente y que los otros enrutadores lo saben. En otras palabras, habrán registrado como infinito el retardo a A.

A	B	C	D	E	
∞	∞	∞	∞	∞	Inicialmente
1	∞	∞	∞	∞	Tras 1 intercambio
1	2	∞	∞	∞	Tras 2 intercambios
1	2	3	∞	∞	Tras 3 intercambios
1	2	3	4	∞	Tras 4 intercambios

(a)

A	B	C	D	E	
1	2	3	4		Inicialmente
3	2	3	4		Tras 1 intercambio
3	4	3	4		Tras 2 intercambios
5	4	5	4		Tras 3 intercambios
5	6	5	6		Tras 4 intercambios
7	6	7	6		Tras 5 intercambios
7	8	7	8		Tras 6 intercambios
	⋮				
∞	∞	∞	∞		

(b)

Figura 5-11. El problema de conteo a infinito.

Al activarse A, los demás enrutadores saben de él gracias a los intercambios de vectores. Por sencillez, supondremos que hay un gong gigantesco en algún lado, golpeado periódicamente para iniciar simultáneamente un intercambio de vectores entre todos los enrutadores. En el momento del primer intercambio, B se entera de que su vecino de la izquierda tiene un retardo de 0 hacia A. B crea entonces una entrada en su tabla de enrutamiento, indicando que A está a una escala de distancia hacia la izquierda. Los demás enrutadores aún piensan que A está desactivado. Las entradas de la tabla de enrutamiento de A en este punto se muestran en la segunda fila de la figura 5-11(a). Durante el siguiente intercambio, C se entera de que B tiene una trayectoria a A de longitud 1, por lo que actualiza su tabla de enrutamiento para indicar una trayectoria de longitud 2, pero D y E no se enteran de las buenas nuevas sino hasta después. Como es evidente, las buenas noticias se difunden a razón de una escala por intercambio. En una subred cuya

trayectoria mayor tiene una longitud de N escalas, en un lapso de N intercambios todo mundo sabrá sobre las líneas y enrutadores recientemente revividos.

Ahora consideremos la situación de la figura 5-11(b), en la que todas las líneas y enrutadores están activos inicialmente. Los enrutadores B, C, D y E tienen distancias a A de 1, 2, 3 y 4, respectivamente. De pronto, A se desactiva, o bien se corta la línea entre A y B, que de hecho es la misma cosa desde el punto de vista de B.

En el primer intercambio de paquetes, B no escucha nada de A. Afortunadamente, C dice: "No te preocupes. Tengo una trayectoria a A de longitud 2". B no sabe que la trayectoria de C pasa a través de B mismo. Hasta donde B sabe, C puede tener 10 líneas de salida, todas con trayectorias independientes a A de longitud 2. Como resultado, B ahora piensa que puede llegar a A por medio de C, con una longitud de trayectoria de 3. D y E no actualizan sus entradas para A en el primer intercambio.

En el segundo intercambio, C nota que cada uno de sus vecinos indica tener una trayectoria a A de longitud 3. C escoge una de ellas al azar y hace que su nueva distancia a A sea de 4, como se muestra en el tercer renglón de la figura 5-11(b). Los intercambios subsecuentes producen la historia mostrada en el resto de la figura 5-11(b).

A partir de esta figura, debe quedar clara la razón por la que las malas noticias viajan con lentitud: ningún enrutador tiene jamás un valor mayor en más de una unidad que el mínimo de todos sus vecinos. Gradualmente, todos los enrutadores elevan sus cuentas hacia el infinito, pero el número de intercambios requerido depende del valor numérico usado para el infinito. Por esta razón, es prudente hacer infinito igual a la trayectoria más larga, más 1. Si la métrica es el retardo de tiempo, no hay un límite superior bien definido, por lo que se necesita un valor alto para evitar tratar a una trayectoria con un retardo grande como si estuviera desactivada. Este problema se conoce como el problema de **conteo a infinito**, lo cual no es del todo sorprendente.

Recorte por horizonte dividido

En la literatura se han propuesto muchas soluciones específicas al problema del conteo a infinito, cada una más complicada y menos útil que la anterior. Describiremos una de ellas e indicaremos también por qué falla. El algoritmo de **horizonte dividido** funciona de la misma manera que el enrutamiento por vector de distancia, excepto que la distancia a X no se informa en la línea por la que se envían paquetes para X (en realidad, se informa como infinita). En el estado inicial de la figura 5-11(b), por ejemplo, C le dice a D la verdad sobre la distancia a A, pero C le dice a B que la distancia a A es infinita. De la misma manera, D le dice la verdad a E, pero le miente a C.

Veamos ahora lo que pasa cuando A se desactiva. En el primer intercambio, B descubre que la línea directa desapareció, y C está informando también una distancia infinita a A. Dado que ninguno de sus vecinos puede llegar a A, B establece también su distancia como infinita. En el siguiente intercambio, C escucha que A es inalcanzable desde sus dos vecinos, por lo que también marca a A como inalcanzable. Usando el horizonte dividido, las malas noticias se propagan a razón de una escala por intercambio. Esta velocidad es mucho mejor que sin el horizonte dividido.

La verdadera mala noticia es que el horizonte dividido, aunque se utiliza ampliamente, a veces falla. Considere, por ejemplo, la subred de cuatro nodos de la figura 5-12. Inicialmente, tanto A como B tienen una distancia a D de 2, y C tiene una distancia de 1 al mismo lugar.

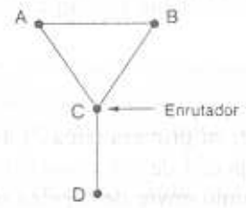


Figura 5-12. Ejemplo de dónde falla el horizonte dividido.

Ahora suponga que se desactiva la línea CD. Usando el horizonte dividido, tanto A como B le indican a C que no pueden llegar a D. Por tanto, C concluye de inmediato que D es inalcanzable y lo informa tanto a A como a B. Desafortunadamente A escucha que B tiene una trayectoria de longitud 2 a D, por lo que supone que puede llegar a D a través de B en tres escalas. De manera parecida, B concluye que puede llegar a D en tres escalas a través de A. En el siguiente intercambio, cada uno establece en 4 su distancia a D. Ambos cuentan gradualmente a infinito, que es precisamente el comportamiento que estamos tratando de evitar.

5.2.6. Enrutamiento por estado de enlace

El enrutamiento por vector de distancia se usó en ARPANET hasta 1979, cuando fue reemplazado por el enrutamiento por estado de enlace. Dos problemas principales causaron su defunción. Primero, dado que la métrica de retardo era la longitud de la cola, no tomaba en cuenta el ancho de banda al escoger rutas. Inicialmente, todas las líneas eran de 56 kbps, por lo que el ancho de banda no era importante, pero una vez que se modernizaron algunas líneas a 230 kbps y otras a 1.544 Mbps, el no tomar en cuenta el ancho de banda se volvió un problema importante. Por supuesto, habría sido posible cambiar la métrica de retardo para considerar también el ancho de banda, pero existía un segundo problema, que el algoritmo con frecuencia tardaba demasiado en convergir, aun con trucos como el horizonte dividido. Por estas razones, el algoritmo fue reemplazado por uno completamente nuevo, llamado **enrutamiento por estado de enlace**. Hoy en día se usan ampliamente variantes del enrutamiento por estado de enlace.

El concepto en que se basa el enrutamiento por estado de enlace es sencillo y puede postularse en cinco partes. Cada enrutador debe

1. Descubrir a sus vecinos y conocer sus direcciones de red.
2. Medir el retardo o costo para cada uno de sus vecinos.
3. Construir un paquete que indique todo lo que acaba de aprender.
4. Enviar este paquete a todos los demás enrutadores.
5. Calcular la trayectoria más corta a todos los demás enrutadores.

De hecho, la topología total y todos los retardos se miden experimentalmente y se distribuyen a cada enrutador. Entonces puede usarse el algoritmo de Dijkstra para encontrar la trayectoria más corta a todos los demás enrutadores. A continuación veremos con mayor detalle estos cinco pasos.

Conocimiento de los vecinos

Al ponerse en operación un enrutador, su primera tarea es averiguar quiénes son sus vecinos; esto lo logra enviando un paquete especial de HOLA (*HELLO*) por cada línea punto a punto. Se espera que el enrutador del otro extremo envíe de regreso una respuesta indicando quién es. Estos nombres deben ser globalmente únicos puesto que, cuando un enrutador distante escucha después que tres enrutadores están conectados a *F*, es indispensable que pueda determinar si los tres se refieren al mismo *F*.

Al conectarse dos o más enrutadores mediante una LAN, la situación es ligeramente más complicada. En la figura 5-13(a) se ilustra una LAN a la que están conectados directamente tres enrutadores, *A*, *C* y *F*. Cada uno de estos enrutadores está conectado a uno o más enrutadores adicionales, como se muestra.

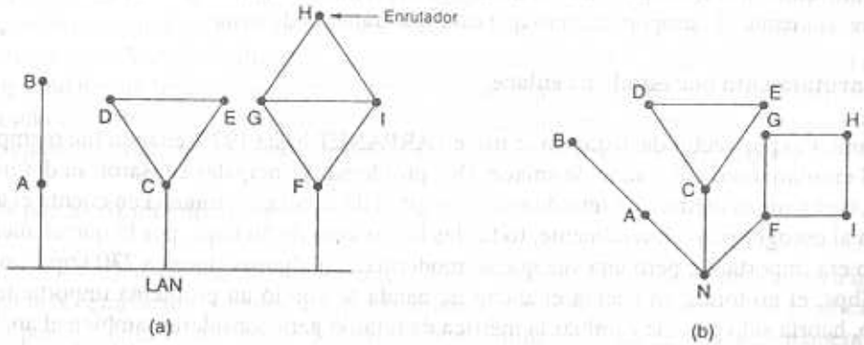


Figura 5-13. (a) Nueve enrutadores y una LAN. (b) Modelo de grafo de (a).

Una manera de modelar la LAN es considerarla como otro nodo, como se muestra en la figura 5-13(b). Aquí hemos introducido un nodo artificial nuevo, *N*, al que están conectados *A*, *C* y *F*. El hecho de que sea posible ir de *A* a *C* a través de la LAN se representa aquí mediante la trayectoria *ANC*.

Medición del costo de la línea

El algoritmo de enrutamiento por estado de enlace requiere que cada enrutador sepa, o cuando menos tenga una idea razonable, del retardo a cada uno de sus vecinos. La manera más directa de determinar este retardo es enviar un paquete especial ECO (*ECHO*) a través de la línea, el cual debe enviar de regreso inmediatamente el otro lado. Si mide el tiempo de ida y vuelta y lo divide entre dos, el enrutador transmisor puede tener una idea razonable del retardo. Para obtener mejores resultados aún, la prueba puede llevarse a cabo varias veces y usarse el promedio.

Un asunto interesante es si se debe tomar en cuenta la carga al medir el retardo. Para considerar la carga, el temporizador de viaje redondo debe iniciarse al encolar el paquete de ECO. Si se desea ignorar la carga, el temporizador debe iniciarse cuando el paquete de ECO llega al frente de la cola.

Pueden citarse argumentos a favor en ambos sentidos. La inclusión de los retardos inducidos por el tráfico en las mediciones implica que, cuando un enrutador puede escoger entre dos líneas con el mismo ancho de banda, una con carga alta continua y otra sin ella, considerará como trayectoria más corta la ruta a través de la línea sin carga. Esta selección resultará en un mejor desempeño.

Desafortunadamente, también hay un argumento en contra de la inclusión de la carga en el cálculo del retardo. Considere la subred de la figura 5-14, dividida en dos partes, este y oeste, conectadas por dos líneas, *CF* y *EI*. Suponga que la mayor parte del tráfico entre el este y el oeste usa la línea *CF* y, como resultado, esta línea tiene tráfico alto con retardos grandes. La inclusión del retardo por encolamiento en el cálculo de la trayectoria más corta hará más atractiva a *EI*. Una vez instaladas las nuevas tablas de enrutamiento, la mayor parte del tráfico este-oeste pasará ahora por *EI*, carga extrando esta línea. En consecuencia, en la siguiente actualización, *CF* aparecerá como la trayectoria más corta. Como resultado, las tablas de enrutamiento pueden oscilar sin control, conduciendo a un enrutamiento errático y muchos problemas potenciales. Si se ignora la carga y sólo se considera el ancho de banda, no ocurre este problema. De manera alterna, puede distribuirse la carga entre ambas líneas, pero esta solución no aprovecha al máximo la mejor trayectoria.

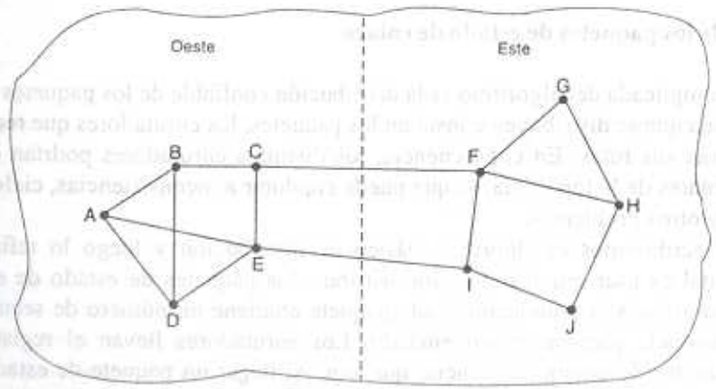


Figura 5-14. Subred en la que las partes este y oeste están conectadas por dos líneas.

Construcción de los paquetes de estado de enlace

Una vez que se ha recabado la información necesaria para el intercambio, el siguiente paso es que cada enrutador construya un paquete con todos los datos. El paquete comienza con la identidad del transmisor, seguida de un número de secuencia, una edad (que se describirá después) y una lista de vecinos. Para cada vecino, se cita el retardo a ese vecino. En la figura 5-15(a) se da un ejemplo de subred, mostrándose el retardo en las líneas. Los paquetes de estado de enlace de los seis enrutadores se muestran en la figura 5-15(b).

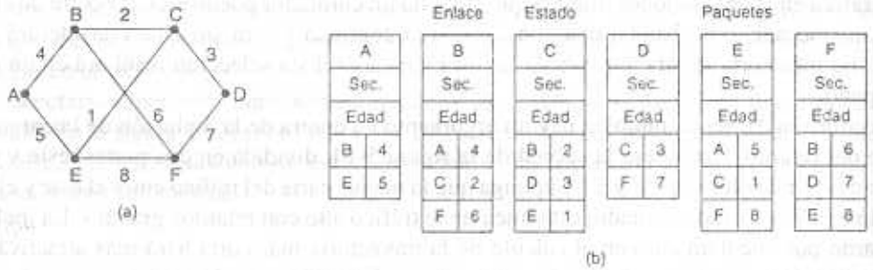


Figura 5-15. (a) Subred. (b) Paquetes de estado de enlace para esta subred.

Es fácil construir los paquetes de estado de enlace. La parte difícil es determinar cuándo construirlos. Una posibilidad es construirlos periódicamente, es decir, a intervalos regulares. Otra posibilidad es al ocurrir un evento significativo, como la caída o reactivación de una línea o de un vecino, o el cambio apreciable de sus propiedades.

Distribución de los paquetes de estado de enlace

La parte más complicada del algoritmo es la distribución confiable de los paquetes de estado de enlace. A medida que se distribuyen e instalan los paquetes, los enrutadores que reciban los primeros cambiarán sus rutas. En consecuencia, los distintos enrutadores podrían estar usando versiones diferentes de la topología, lo que puede conducir a inconsistencias, ciclos, máquinas inalcanzables y otros problemas.

Primero describiremos el algoritmo básico de distribución y luego lo refinaremos. La idea fundamental es usar inundación para distribuir los paquetes de estado de enlace. A fin de mantener controlada la inundación, cada paquete contiene un número de secuencia que se incrementa con cada paquete nuevo enviado. Los enrutadores llevan el registro de todos los pares (enrutador de origen, secuencia) que ven. Al llegar un paquete de estado de enlace, se revisa contra la lista de paquetes ya vistos. Si es nuevo, se reenvía a través de todas las líneas, excepto aquella por la que llegó. Si es un duplicado, se descarta. Si llega un paquete con

determinar este retardo es enviar un paquete especial ECO (*ECHO*) a través de la línea, el cual debe enviar de regreso inmediatamente el otro lado. Si mide el tiempo de ida y vuelta y lo divide entre dos, el enrutador transmisor puede tener una idea razonable del retardo. Para obtener mejores resultados aún, la prueba puede llevarse a cabo varias veces y usarse el promedio.

Un asunto interesante es si se debe tomar en cuenta la carga al medir el retardo. Para considerar la carga, el temporizador de viaje redondo debe iniciarse al encolar el paquete de ECO. Si se desea ignorar la carga, el temporizador debe iniciarse cuando el paquete de ECO llega al frente de la cola.

Pueden citarse argumentos a favor en ambos sentidos. La inclusión de los retardos inducidos por el tráfico en las mediciones implica que, cuando un enrutador puede escoger entre dos líneas con el mismo ancho de banda, una con carga alta continua y otra sin ella, considerará como trayectoria más corta la ruta a través de la línea sin carga. Esta selección resultará en un mejor desempeño.

Desafortunadamente, también hay un argumento en contra de la inclusión de la carga en el cálculo del retardo. Considere la subred de la figura 5-14, dividida en dos partes, este y oeste, conectadas por dos líneas, *CF* y *EI*. Suponga que la mayor parte del tráfico entre el este y el oeste usa la línea *CF* y, como resultado, esta línea tiene tráfico alto con retardos grandes. La inclusión del retardo por encolamiento en el cálculo de la trayectoria más corta hará más atractiva a *EI*. Una vez instaladas las nuevas tablas de enrutamiento, la mayor parte del tráfico este-oeste pasará ahora por *EI*, carga extrando esta línea. En consecuencia, en la siguiente actualización, *CF* aparecerá como la trayectoria más corta. Como resultado, las tablas de enrutamiento pueden oscilar sin control, conduciendo a un enrutamiento errático y muchos problemas potenciales. Si se ignora la carga y sólo se considera el ancho de banda, no ocurre este problema. De manera alterna, puede distribuirse la carga entre ambas líneas, pero esta solución no aprovecha al máximo la mejor trayectoria.

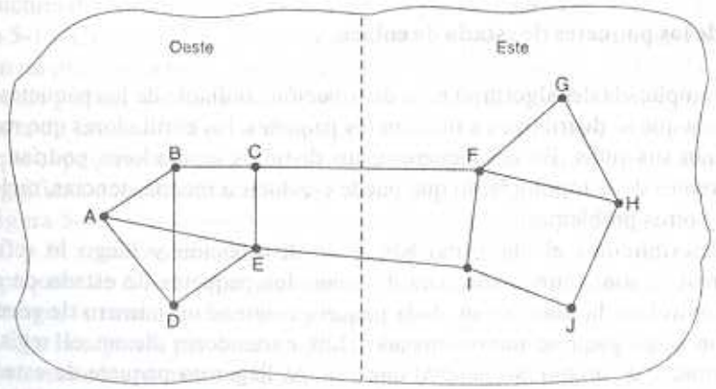
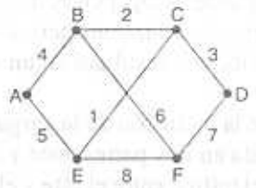


Figura 5-14. Subred en la que las partes este y oeste están conectadas por dos líneas.

Construcción de los paquetes de estado de enlace

Una vez que se ha recabado la información necesaria para el intercambio, el siguiente paso es que cada enrutador construya un paquete con todos los datos. El paquete comienza con la identidad del transmisor, seguida de un número de secuencia, una edad (que se describirá después) y una lista de vecinos. Para cada vecino, se cita el retardo a ese vecino. En la figura 5-15(a) se da un ejemplo de subred, mostrándose el retardo en las líneas. Los paquetes de estado de enlace de los seis enrutadores se muestran en la figura 5-15(b).



(a)

Enlace Estado Paquetes

A		B		C		D		E		F	
Sec.		Sec.		Sec.		Sec.		Sec.		Sec.	
Edad		Edad		Edad		Edad		Edad		Edad	
B	4	A	4	B	2	C	3	A	5	B	6
E	5	C	2	D	3	F	7	C	1	D	7
		F	6	E	1			F	8	E	8

(b)

Figura 5-15. (a) Subred. (b) Paquetes de estado de enlace para esta subred.

Es fácil construir los paquetes de estado de enlace. La parte difícil es determinar cuándo construirlos. Una posibilidad es construirlos periódicamente, es decir, a intervalos regulares. Otra posibilidad es al ocurrir un evento significativo, como la caída o reactivación de una línea o de un vecino, o el cambio apreciable de sus propiedades.

Distribución de los paquetes de estado de enlace

La parte más complicada del algoritmo es la distribución confiable de los paquetes de estado de enlace. A medida que se distribuyen e instalan los paquetes, los enrutadores que reciban los primeros cambiarán sus rutas. En consecuencia, los distintos enrutadores podrían estar usando versiones diferentes de la topología, lo que puede conducir a inconsistencias, ciclos, máquinas inalcanzables y otros problemas.

Primero describiremos el algoritmo básico de distribución y luego lo refinaremos. La idea fundamental es usar inundación para distribuir los paquetes de estado de enlace. A fin de mantener controlada la inundación, cada paquete contiene un número de secuencia que se incrementa con cada paquete nuevo enviado. Los enrutadores llevan el registro de todos los pares (enrutador de origen, secuencia) que ven. Al llegar un paquete de estado de enlace, se revisa contra la lista de paquetes ya vistos. Si es nuevo, se reenvía a través de todas las líneas, excepto aquella por la que llegó. Si es un duplicado, se descarta. Si llega un paquete con

número de secuencia menor que el mayor visto hasta el momento, se rechaza como obsoleto.

Este algoritmo tiene algunos problemas, pero son manejables. Primero, si los números de secuencia vuelven a comenzar, reinará la confusión. La solución aquí es usar un número de secuencia de 32 bits. Con un paquete de estado de enlace por segundo, el tiempo para volver a empezar será de 137 años, por lo que puede ignorarse esta posibilidad.

Segundo, si llega a caerse un enrutador, perderá el registro de su número de secuencia. Si comienza nuevamente en 0, se rechazará como duplicado el siguiente paquete.

Tercero, si llega a corromperse un número de secuencia y se recibe 65,540 en lugar de 4 (un error de 1 bit), los paquetes 5 a 65,540 serán rechazados como obsoletos, dado que se piensa que el número de secuencia actual es 65,540.

La solución a todos estos problemas es incluir la edad de cada paquete después del número de secuencia y disminuirla una vez cada segundo. Al llegar la edad a cero, se descarta la información de ese enrutador. Normalmente, entra un paquete nuevo cada 10 minutos, digamos, por lo que la información de los enrutadores sólo expira cuando está caído el enrutador (o se pierden seis paquetes consecutivos, evento poco probable). El campo de edad también es disminuido por cada enrutador durante el proceso inicial de inundación para asegurar que no pueda perderse ningún paquete y sobrevivir durante un periodo de tiempo indefinido (se descarta un paquete cuya edad es cero).

Algunos refinamientos de este algoritmo lo hacen más robusto. Al llegar un paquete de estado de enlace a un enrutador para ser inundado, no se encola para transmisión inmediata. En cambio, entra en un área de retención donde espera un tiempo corto. Si antes de transmitirlo entra otro paquete de estado de enlace de la misma fuente, se comparan sus números de secuencia. Si son iguales, se descarta el duplicado. Si son diferentes, se deshecha el más viejo. Como protección contra los errores en las líneas enrutador-enrutador, todos los paquetes de estado de enlace requieren reconocimiento. Al desactivarse una línea, se examina el área de retención en orden por turno circular para seleccionar un paquete o reconocimiento a enviar.

La estructura de datos usada por el enrutador *B* para la subred de la figura 5-15(a) se describe en la figura 5-16. Cada renglón aquí corresponde a un paquete de estado de enlace recién llegado, pero aún no procesado por completo. La tabla registra dónde se originó el paquete, su número de secuencia y edad, y los datos. Además, hay indicadores de transmisión y reconocimiento para cada una de las tres líneas de *B* (a *A*, *C* y *F*, respectivamente). Los indicadores de envío significan que el paquete debe enviarse a través de la línea indicada. Los indicadores de reconocimiento significan que deben reconocerse ahí.

En la figura 5-16, el paquete de estado de enlace de *A* llegó directamente, por lo que debe enviarse a *C* y *F*, y reconocerse ante *A*, como lo muestran los bits de indicación. De la misma manera, el paquete de *F* tiene que reenviarse a *A* y a *C*, y reconocerse ante *F*.

Sin embargo, la situación del tercer paquete, de *E*, es diferente; llegó dos veces, la primera a través de *EAB* y la segunda por medio de *EFB*. En consecuencia, este paquete tiene que enviarse sólo a *C*, pero reconocerse tanto ante *A* como ante *F*, como lo indican los bits.

Si llega un duplicado mientras el original aún está en el *buffer*, los bits tienen que cambiar. Por ejemplo, si llega una copia del estado de *C* desde *F* antes de reenviarse la cuarta entrada de

			Indicadores envío			Indicadores ACK			
Origen	Sec.	Edad	A	C	F	A	C	F	Datos
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

Figura 5-16. *Buffer* de paquetes para el enrutador *B* de la figura 5-15.

la tabla, cambiarán los seis bits a 100011 para indicar que el paquete debe ser reconocido ante *F*, pero no enviarse ahí.

Cálculo de las nuevas rutas

Una vez que un enrutador ha acumulado un grupo completo de paquetes de estado de enlace, puede construir el grafo de la subred completa porque todos los enlaces están representados. De hecho, cada enlace se representa dos veces, una para cada dirección. Los dos valores pueden promediarse o usarse por separado.

Ahora puede ejecutarse localmente el algoritmo de Dijkstra para construir la trayectoria más corta posible a todos los destinos. Los resultados de este algoritmo pueden instalarse en las tablas de enrutamiento, y reiniciarse la operación normal.

Para una subred con n enrutadores, cada uno de los cuales tiene k vecinos, la memoria requerida para almacenar los datos de entrada es proporcional a kn. En las subredes grandes éste puede ser un problema. También puede serlo el tiempo de cómputo. Sin embargo, en muchas situaciones prácticas, el enrutamiento por estado de enlace funciona bien.

Sin embargo, problemas con el *hardware* o el *software* pueden causar estragos con este algoritmo (lo mismo que con otros). Por ejemplo, si un enrutador indica tener una línea que no tiene, u olvida una línea que sí tiene, el grafo de la subred será incorrecta. Si un enrutador deja de reenviar paquetes, o los corrompe al hacerlo, surgirán problemas. Por último, si al enrutador se le acaba la memoria o se ejecuta mal el algoritmo de cálculo de enrutamiento, ocurrirán cosas malas. A medida que la subred crece al orden de decenas o cientos de miles de nodos, la probabilidad de falla ocasional de un enrutador deja de ser insignificante. Lo importante es tratar de limitar el daño cuando ocurra lo inevitable. Perlman (1988) estudia detalladamente estos problemas y sus soluciones.

El enrutamiento por estado de enlace se usa ampliamente en las redes actuales, por lo que son pertinentes unas pocas palabras sobre algunos protocolos que lo usan. El protocolo OSPF, que se emplea cada vez con mayor frecuencia en Internet, usa un algoritmo de estado de enlace. Describiremos el OSPF en la sección 5.5.5.

Otro protocolo de estado de enlace importante es el **IS-IS** (*Intermediate System-Intermediate System*, **sistema intermedio-sistema intermedio**), diseñado por DECnet y adoptado por la ISO para uso con su protocolo de capa de red sin conexiones, CLNP. Desde entonces se ha modificado para manejar también otros protocolos, siendo el más notable el IP. El IS-IS se usa en varios *backbone* de Internet (incluido el viejo *backbone* NSFNET) y en muchos sistemas celulares digitales, como CDPD. El NetWare de Novell usa una variante menor del IS-IS (NLSP) para el enrutamiento de paquetes IPX.

Básicamente, IS-IS distribuye una imagen de la topología de enrutadores, a partir de la cual se calculan las trayectorias más cortas. Cada enrutador anuncia, en su información de estado de enlace, las direcciones de capa de red que puede alcanzar directamente. Estas direcciones pueden ser IP, IPX, AppleTalk o cualquier otra dirección. El IS-IS inclusive puede manejar varios protocolos de red al mismo tiempo.

Muchas de las innovaciones diseñadas para IS-IS fueron adoptadas por el OSPF (OSPF se diseñó varios años después que IS-IS). Éstas incluyen un método de autoestabilización de las actualizaciones de estado de enlace por inundación, el concepto de un enrutador designado en una LAN y el método de cálculo y manejo de la división de trayectorias y métricas múltiples. En consecuencia, hay muy poca diferencia entre el IS-IS y el OSPF. La diferencia más importante es que el IS-IS está codificado de tal manera que es fácil y natural llevar simultáneamente información sobre varios protocolos de capa de red, característica que no tiene el OSPF. Esta ventaja es especialmente valiosa en ambientes multiprotocolo grandes.

5.2.7. Enrutamiento jerárquico

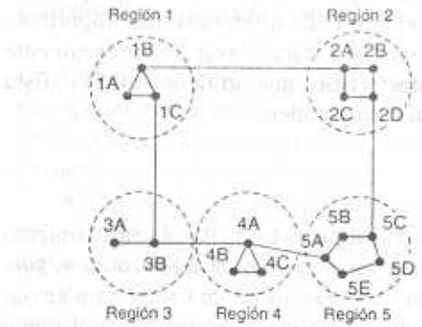
A medida que crecen en tamaño las redes, crecen proporcionalmente las tablas de enrutamiento del enrutador. Las tablas que siempre crecen no sólo consumen memoria del enrutador, sino que también se necesita más tiempo CPU para examinarlas y más ancho de banda para enviar informes de estado entre enrutadores. En cierto momento, la red puede crecer hasta el punto en que ya no es factible que cada enrutador tenga una entrada para cada uno de los demás enrutadores, por lo que el enrutamiento tendrá que hacerse jerárquicamente, como ocurre en la red telefónica.

Al usarse el enrutamiento jerárquico, los enrutadores se dividen en lo que llamaremos **regiones**, donde cada enrutador conoce todos los detalles de la manera de enrutar paquetes a destinos dentro de su propia región, pero no sabe nada de la estructura interna de las otras regiones. Al interconectar diferentes redes, es natural considerar cada una como región independiente, a fin de liberar a los enrutadores de una red de la necesidad de conocer la estructura topológica de las demás.

En las redes enormes puede ser insuficiente una jerarquía de dos niveles; puede ser necesario agrupar las regiones en cúmulos, los cúmulos en zonas, las zonas en grupos, etc., hasta que se nos agoten los nombres para los agregados. Como ejemplo de jerarquía multinivel, considere una posible forma de enrutar un paquete de Berkeley, California a Malindi, Kenya. El enrutador

de Berkeley conocería la topología detallada de California, pero podría enviar todo el tráfico exterior al enrutador de Los Ángeles. El enrutador de Los Ángeles podría enrutar el tráfico a otros enrutadores del país, pero enviaría el tráfico internacional a Nueva York. El enrutador de Nueva York tendría la programación para dirigir todo el tráfico al enrutador del país de destino encargado del manejo de tráfico internacional, digamos en Nairobi. Por último, el paquete encontraría su camino por el árbol de Kenya hasta llegar a Malindi.

En la figura 5-17 se da un ejemplo cuantitativo de enrutamiento en una jerarquía de dos niveles con cinco regiones. La tabla de enrutamiento completa para el enrutador 1*A* tiene 17 entradas, como se muestra en la figura 5-17(b). Si el enrutamiento es jerárquico, como en la figura 5-17(c), hay entradas para todos los enrutadores locales, igual que antes, pero las demás regiones se han condensado en un solo enrutador, por lo que todo el tráfico para la región 2 va a través de la línea 1*B*-2*A*, pero el resto del tráfico remoto viaja por la línea 1*C*-3*B*. El enrutamiento jerárquico redujo la tabla de 17 entradas a 7. A medida que crece la razón entre la cantidad de regiones y el número de enrutadores por región, aumentan los ahorros de espacio de tabla.



(a)

Tabla completa para 1A

Dest.	Línea	Escalas
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Tabla jerárquica para 1A

Dest.	Línea	Escalas
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

Figura 5-17. Enrutamiento jerárquico.

Desafortunadamente, estas ganancias de espacio no son gratuitas. Se paga un precio, y este precio adopta la forma de una longitud de trayectoria mayor. Por ejemplo, la mejor ruta de 1*A* a 5*C* es a través de la región 2 pero, con el enrutamiento jerárquico, todo el tráfico a la región 5 pasa por la región 3, porque eso es mejor para la mayoría de los destinos de la región 5.

Al volverse muy grande una sola red, surge una pregunta interesante: ¿cuántos niveles debe tener la jerarquía? Por ejemplo, considere una subred con 720 enrutadores. Si no hay jerarquía, cada enrutador necesita 720 tablas de enrutamiento. Si partimos la subred en 24 regiones de 30 enrutadores cada una, cada enrutador necesitará 30 entradas locales más 23 entradas remotas, para un total de 53 entradas. Si escogemos una jerarquía de tres niveles, con ocho cúmulos, cada uno de los cuales contiene 9 regiones de 10 enrutadores, cada enrutador necesita 10 entradas para los enrutadores locales, 8 entradas para el enrutamiento a otras regiones dentro de su propio cúmulo y 7 entradas para cúmulos distantes, para un total de 25 entradas. Kamoun y Kleinrock (1979) descubrieron que el número óptimo de niveles para una subred de enrutadores es de ln *N*, requiriéndose un total de *e* ln *N* entradas por enrutador. También han demostrado que el aumento en la longitud media efectiva de trayectoria causada por el enrutamiento jerárquico es lo bastante pequeña como para ser generalmente aceptable.

5.2.8. Enrutamiento para *hosts* móviles

Hoy día, millones de personas tienen computadoras portátiles, y generalmente quieren leer su correo electrónico y acceder a sus sistemas de archivos normales desde cualquier lugar del mundo. Estos *hosts* móviles generan una nueva complicación: para enrutar un paquete a un *host* móvil, la red primero tiene que encontrarlo. El tema de la incorporación de *hosts* móviles en una red es muy joven, pero en esta sección plantearemos algunos de los problemas relacionados y sugeriremos una posible solución.

El modelo del mundo que usan generalmente los diseñadores de red se muestra en la figura 5-18. Aquí tenemos una WAN que consiste en enrutadores y *hosts*. Conectadas a la WAN hay varias LAN, MAN y células inalámbricas del tipo estudiado en el capítulo 2.

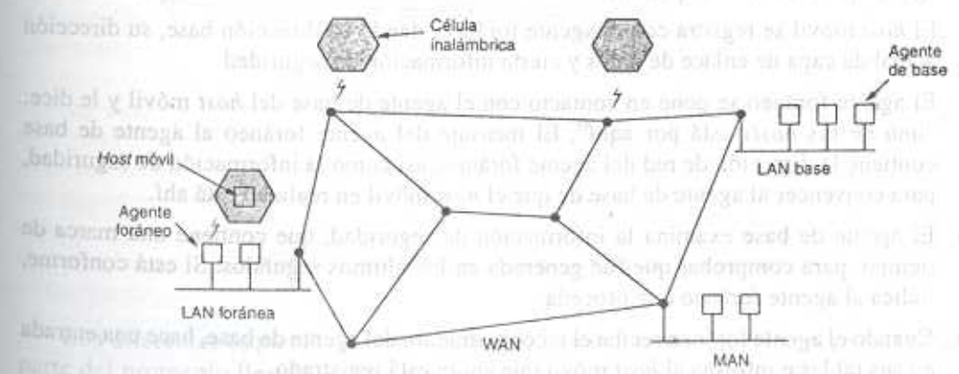


Figura 5-18. WAN a la que están conectadas LAN, MAN y células inalámbricas.

Se dice que son estacionarios los usuarios que nunca se mueven; se conectan a la red mediante alambres de cobre o fibra óptica. En contraste podemos distinguir otros dos tipos de

de Berkeley
exterior al en
otros enrutad
Nueva York
encargado d
encontraría s

En la fig
niveles con
entradas, co
figura 5-17(
regiones se l
través de la l
jerárquico n
regiones y e

Re

1A

3

F

Des
precio a
5C es a
pasa por

usuarios. Los usuarios migratorios básicamente son usuarios estacionarios que se mueve lugar fijo a otro de tiempo en tiempo, pero que usan la red sólo cuando están conectados mente a ella. Los usuarios errantes hacen su cómputo en movimiento, y quieren mante conexiones mientras se mueven. Usaremos el término **usuarios móviles** para indicar cua de las dos últimas categorías, es decir, todos los usuarios que están lejos de casa.

Se supone que todos los usuarios tienen una **localidad base** que nunca cambia. Los u también tienen una dirección de base permanente, que puede servir para determinar su loc base, de manera análoga a como el número telefónico 1-212-5551212 indica Estados (código de país 1) y Manhattan (212). La meta de enrutamiento en los sistemas con u móviles es posibilitar el envío de paquetes a usuarios móviles usando su dirección base, que los paquetes lleguen eficientemente a ellos en cualquier lugar en el que puedan es difícil, por supuesto, es encontrarlos.

En el modelo de la figura 5-18, el mundo se divide (geográficamente) en unidades peq Llamémoslas áreas, siendo un área típicamente una LAN o una célula inalámbrica. Cad tiene uno o más **agentes foráneos**, que llevan el registro de todos los usuarios que visitan Además, cada área tiene un **agente de base**, que lleva el registro de todos los usuarios m cuya base está en el área, pero que actualmente están visitando otra área.

Al entrar un usuario nuevo en un área, ya sea al conectarse a ella (por ejemplo, conect a la LAN), o simplemente al entrar en la célula, su computadora debe registrarse con el foráneo de ese lugar. El procedimiento de registro funciona típicamente de esta manera:

1. Periódicamente, cada agente foráneo difunde un paquete que anuncia su exister dirección. Un *host* móvil recién llegado puede esperar uno de estos mensajes, pero llega ninguno con suficiente rapidez, el *host* móvil puede difundir un paquete que "¿hay agentes foráneos por ahí?"
2. El *host* móvil se registra con el agente foráneo, dando su dirección base, su dire actual de capa de enlace de datos y cierta información de seguridad.
3. El agente foráneo se pone en contacto con el agente de base del *host* móvil y le "uno de tus *hosts* está por aquí". El mensaje del agente foráneo al agente de contiene la dirección de red del agente foráneo, así como la información de segur para convencer al agente de base de que el *host* móvil en realidad está ahí.
4. El agente de base examina la información de seguridad, que contiene una marc tiempo, para comprobar que fue generada en los últimos segundos. Si está confo indica al agente foráneo que proceda.
5. Cuando el agente foráneo recibe el reconocimiento del agente de base, hace una en en sus tablas e informa al *host* móvil que ahora está registrado.

Idealmente, al salir un usuario de un área, también eso debe anunciarse para permitir q borre el registro pero, cuando terminan, muchos usuarios apagan abruptamente sus computad

Al enviarse un paquete a un usuario móvil, se enruta a la LAN base del usuario, ya qu es lo indicado en la dirección, como se ilustra en el paso 1 de la figura 5-19. Los paq

dos al usuario móvil por su LAN base son interceptados por el agente de base, el cual ces busca la localidad nueva (temporal) del usuario móvil y encuentra la dirección del e foráneo que maneja al usuario móvil. El agente de base entonces hace dos cosas. Primero, sula el paquete en el campo de carga útil de un paquete exterior y envía este último al e foráneo (paso 2 de la figura 5-19). Este mecanismo se llama proceso de túnel y lo nos con mayor detalle después. Tras obtener el paquete encapsulado, el agente foráneo eve el paquete original del campo de carga útil y lo envía al usuario móvil como marco de de datos.

egundo, el agente de base indica al transmisor que en lo futuro envíe paquetes al *host* móvil sulándolos en la carga útil de paquetes explícitamente dirigidos al agente foráneo, en lugar mplemente enviarlos a la dirección base del usuario móvil (paso 3). Los paquetes subsi- tes ahora pueden enrutarse directamente al usuario por medio del agente foráneo (paso 4), iendo la localidad base por completo.

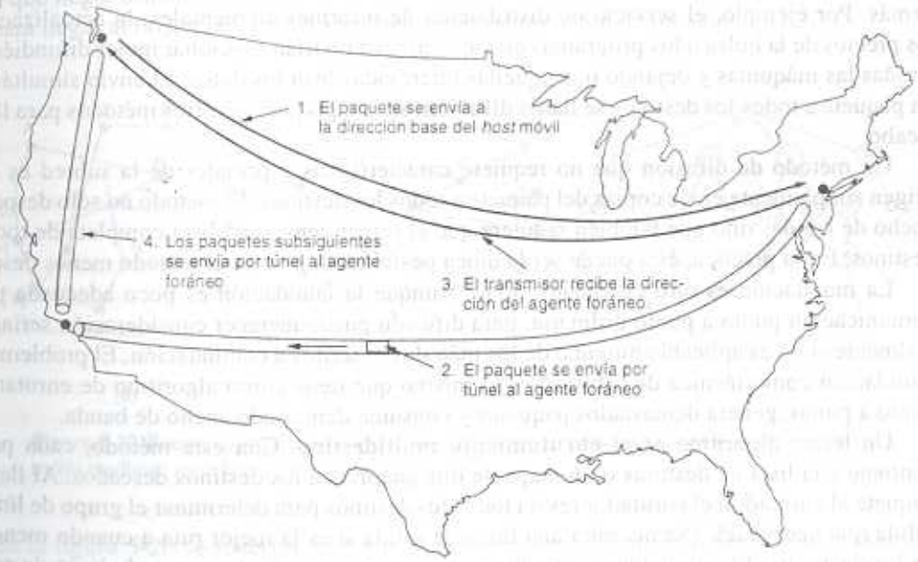


Figura 5-19. Enrutamiento de paquetes para usuarios móviles.

Los diferentes esquemas propuestos difieren en varios sentidos. Primero está el asunto de la te del protocolo llevada a cabo por los enrutadores y la llevada a cabo por los *hosts* y, en el mo caso, la capa que se encarga de ella en los *hosts*. Segundo, en unos cuantos esquemas, los utadores a lo largo del camino registran las direcciones en mapas, para poder interceptarlas y rigir el tráfico aún antes de que llegue a la dirección base. Tercero, en algunos esquemas, a visitante recibe una dirección temporal única; en otros, la dirección temporal se refiere a un nte que maneja el tráfico de todos los visitantes.

usuarios. Los usuarios migratorios básicamente son usuarios estacionarios que se mueven de un lugar fijo a otro de tiempo en tiempo, pero que usan la red sólo cuando están conectados físicamente a ella. Los usuarios errantes hacen su cómputo en movimiento, y quieren mantener sus conexiones mientras se mueven. Usaremos el término **usuarios móviles** para indicar cualquiera de las dos últimas categorías, es decir, todos los usuarios que están lejos de casa.

Se supone que todos los usuarios tienen una **localidad base** que nunca cambia. Los usuarios también tienen una dirección de base permanente, que puede servir para determinar su localidad base, de manera análoga a como el número telefónico 1-212-5551212 indica Estados Unidos (código de país 1) y Manhattan (212). La meta de enrutamiento en los sistemas con usuarios móviles es posibilitar el envío de paquetes a usuarios móviles usando su dirección base, y hacer que los paquetes lleguen eficientemente a ellos en cualquier lugar en el que puedan estar. Lo difícil, por supuesto, es encontrarlos.

En el modelo de la figura 5-18, el mundo se divide (geográficamente) en unidades pequeñas. Llamémoslas áreas, siendo un área típicamente una LAN o una célula inalámbrica. Cada área tiene uno o más **agentes foráneos**, que llevan el registro de todos los usuarios que visitan el área. Además, cada área tiene un **agente de base**, que lleva el registro de todos los usuarios móviles cuya base está en el área, pero que actualmente están visitando otra área.

Al entrar un usuario nuevo en un área, ya sea al conectarse a ella (por ejemplo, conectándose a la LAN), o simplemente al entrar en la célula, su computadora debe registrarse con el agente foráneo de ese lugar. El procedimiento de registro funciona típicamente de esta manera:

1. Periódicamente, cada agente foráneo difunde un paquete que anuncia su existencia y dirección. Un *host* móvil recién llegado puede esperar uno de estos mensajes, pero si no llega ninguno con suficiente rapidez, el *host* móvil puede difundir un paquete que diga: "¿hay agentes foráneos por ahí?"
2. El *host* móvil se registra con el agente foráneo, dando su dirección base, su dirección actual de capa de enlace de datos y cierta información de seguridad.
3. El agente foráneo se pone en contacto con el agente de base del *host* móvil y le dice: "uno de tus *hosts* está por aquí". El mensaje del agente foráneo al agente de base contiene la dirección de red del agente foráneo, así como la información de seguridad, para convencer al agente de base de que el *host* móvil en realidad está ahí.
4. El agente de base examina la información de seguridad, que contiene una marca de tiempo, para comprobar que fue generada en los últimos segundos. Si está conforme, indica al agente foráneo que proceda.
5. Cuando el agente foráneo recibe el reconocimiento del agente de base, hace una entrada en sus tablas e informa al *host* móvil que ahora está registrado.

Idealmente, al salir un usuario de un área, también eso debe anunciarse para permitir que se borre el registro pero, cuando terminan, muchos usuarios apagan abruptamente sus computadoras.

Al enviarse un paquete a un usuario móvil, se enruta a la LAN base del usuario, ya que eso es lo indicado en la dirección, como se ilustra en el paso 1 de la figura 5-19. Los paquetes

enviados al usuario móvil por su LAN base son interceptados por el agente de base, el cual entonces busca la localidad nueva (temporal) del usuario móvil y encuentra la dirección del agente foráneo que maneja al usuario móvil. El agente de base entonces hace dos cosas. Primero, encapsula el paquete en el campo de carga útil de un paquete exterior y envía este último al agente foráneo (paso 2 de la figura 5-19). Este mecanismo se llama proceso de túnel y lo veremos con mayor detalle después. Tras obtener el paquete encapsulado, el agente foráneo remueve el paquete original del campo de carga útil y lo envía al usuario móvil como marco de enlace de datos.

Segundo, el agente de base indica al transmisor que en lo futuro envíe paquetes al *host* móvil encapsulándolos en la carga útil de paquetes explícitamente dirigidos al agente foráneo, en lugar de simplemente enviarlos a la dirección base del usuario móvil (paso 3). Los paquetes subsiguientes ahora pueden enrutarse directamente al usuario por medio del agente foráneo (paso 4), omitiendo la localidad base por completo.

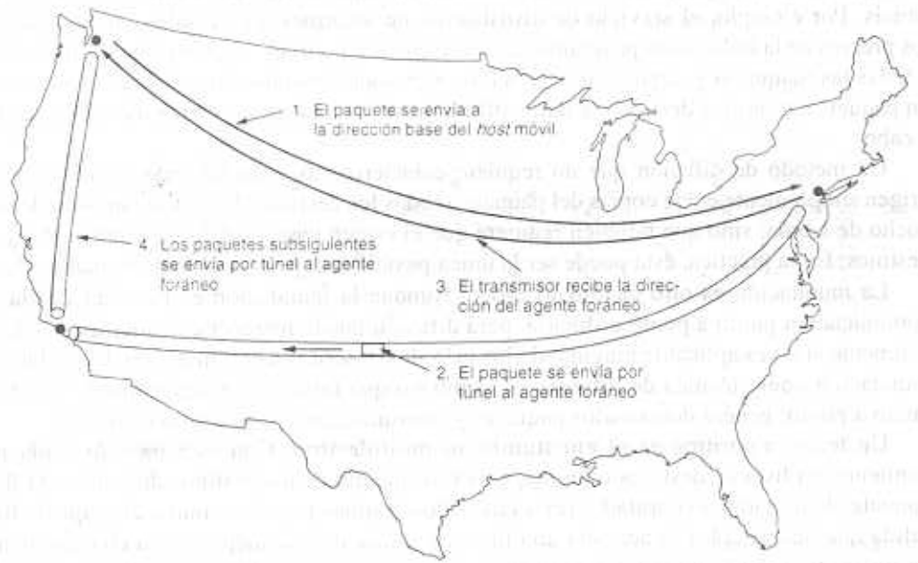


Figura 5-19. Enrutamiento de paquetes para usuarios móviles.

Los diferentes esquemas propuestos difieren en varios sentidos. Primero está el asunto de la parte del protocolo llevada a cabo por los enrutadores y la llevada a cabo por los *hosts* y, en el último caso, la capa que se encarga de ella en los *hosts*. Segundo, en unos cuantos esquemas, los enrutadores a lo largo del camino registran las direcciones en mapas, para poder interceptarlas y redirigir el tráfico aún antes de que llegue a la dirección base. Tercero, en algunos esquemas, cada visitante recibe una dirección temporal única; en otros, la dirección temporal se refiere a un agente que maneja el tráfico de todos los visitantes.

Cuarto, los esquemas difieren en la manera en que logran realmente que los paquetes dirigidos a un destino lleguen a uno diferente. Una posibilidad es cambiar la dirección de destino y simplemente retransmitir el paquete modificado. Como alternativa, el paquete completo, con dirección base y todo, puede encapsularse en la carga útil de otro paquete enviado a la dirección temporal. Por último, los esquemas difieren en sus aspectos de seguridad. Por lo general, cuando un *host* o enrutador recibe un mensaje del tipo "a partir de este momento, envíame por favor todo el correo de Genoveva", puede tener dudas acerca de con quién está hablando y de si se trata de una buena idea. Se han analizado y comparado varios protocolos de *host* móvil en (Ioannidis y Maguire, 1993; Myles y Skellern, 1993; Perkins, 1993; Teraoka *et al.*, 1993, y Wada *et al.*, 1993).

5.2.9. Enrutamiento por difusión

En algunas aplicaciones, los *hosts* necesitan enviar mensajes a varios otros *hosts* o a todos los demás. Por ejemplo, el servicio de distribución de informes ambientales, la actualización de los precios de la bolsa o los programas de radio en vivo podrían funcionar mejor difundiéndolos a todas las máquinas y dejando que aquellas interesadas lean los datos. El envío simultáneo de un paquete a todos los destinos se llama **difusión**; se han propuesto varios métodos para llevarla a cabo.

Un método de difusión que no requiere características especiales de la subred es que el origen simplemente envíe copias del paquete a todos los destinos. El método no sólo desperdicia ancho de banda, sino que también requiere que el origen tenga una lista completa de todos los destinos. En la práctica, ésta puede ser la única posibilidad, pero es el método menos deseable.

La inundación es otro candidato obvio. Aunque la inundación es poco adecuada para la comunicación punto a punto ordinaria, para difusión puede merecer consideración seria, especialmente si no es aplicable ninguno de los métodos descritos a continuación. El problema de la inundación como técnica de difusión es el mismo que tiene como algoritmo de enrutamiento punto a punto: genera demasiados paquetes y consume demasiado ancho de banda.

Un tercer algoritmo es el **enrutamiento multidestino**. Con este método, cada paquete contiene una lista de destinos o un mapa de bits que indica los destinos deseados. Al llegar un paquete al enrutador, el enrutador revisa todos los destinos para determinar el grupo de líneas de salida que necesitará. (Se necesita una línea de salida si es la mejor ruta a cuando menos uno de los destinos.) El enrutador genera una copia nueva del paquete para cada línea de salida a usar, e incluye en cada paquete sólo aquellos destinos que usan la línea. En efecto, el grupo de destinos se divide entre las líneas de salida. Tras una cantidad suficiente de escalas, cada paquete llevará sólo un destino, así que puede tratarse como un paquete normal. El enrutamiento multidestino es como los paquetes con direccionamiento individual, excepto que, cuando varios paquetes deben seguir la misma ruta, uno de ellos paga la tarifa completa y los demás viajan gratis.

Un cuarto algoritmo de difusión usa explícitamente el árbol de descenso para el enrutador que inicia la difusión, o cualquier otro árbol de extensión adecuado. El **árbol de extensión** es un subgrupo de la subred que incluye todos los enrutadores pero no contiene ciclos. Si cada enrutador sabe cuáles de sus líneas pertenecen al árbol de extensión, puede copiar un paquete de

entrada difundido en todas las líneas del árbol de extensión, excepto en aquella por la que llegó. Este método hace un uso excelente del ancho de banda, generando la cantidad mínima de paquetes necesarios para llevar a cabo el trabajo. El único problema es que cada enrutador debe tener conocimiento de algún árbol de extensión para que pueda funcionar. Algunas veces está disponible esta información (por ejemplo, con el enrutamiento por estado de enlace), pero a veces no (por ejemplo, con el enrutamiento por vector de distancia).

Nuestro último algoritmo de difusión es un intento de aproximar el comportamiento del anterior, aun cuando los enrutadores no saben nada en lo absoluto sobre árboles de extensión. La idea es excepcionalmente sencilla una vez planteada. Cuando llega un paquete difundido a un enrutador, éste lo revisa para ver si llegó por la línea normalmente usada para enviar paquetes *al* origen de la difusión. De ser así, hay excelentes posibilidades de que el paquete difundido haya seguido la mejor ruta desde el enrutador y, por tanto, sea la primera copia en llegar al enrutador. Siendo éste el caso, el enrutador reenvía copias del paquete por todas las líneas, excepto aquella por la que llegó. Sin embargo, si el paquete difundido llegó por una línea diferente de la preferida para llegar al origen, se descarta el paquete como probable duplicado.

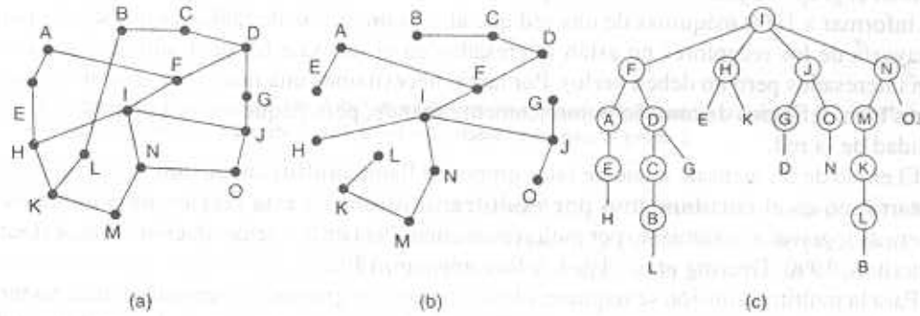


Figura 5-20. Reenvío por trayectoria invertida. (a) Subred. (b) Árbol de extensión. (c) Árbol construido mediante reenvío por trayectoria invertida.

En la figura 5-20 se muestra un ejemplo del algoritmo, llamado **reenvío por trayectoria invertida**. En la parte (a) se muestra una subred, en la parte (b) se muestra un árbol de descenso para el enrutador *I* de esa subred, y en la parte (c) se muestra el funcionamiento del algoritmo de trayectoria invertida. En la primera escala, *I* envía paquetes a *F*, *H*, *J* y *N*, como lo indica el segundo renglón del árbol. Cada uno de estos paquetes llega a *I* por la trayectoria preferida (suponiendo que la trayectoria preferida pasa a través del árbol de descenso), como lo indica el círculo alrededor de la letra. En la segunda escala, se generan ocho paquetes, dos por cada uno de los enrutadores que recibieron un paquete en la primera escala. Como resultado, los ocho llegan a enrutadores no visitados previamente, y todos menos uno llegan a través de la línea preferida. De los nueve paquetes generados en la tercera escala, sólo dos llegan por la trayectoria preferida (a *C* y *L*), así que sólo estos generan paquetes posteriores. Tras cinco escalas y 24

paquetes, termina la difusión, en comparación con cuatro escalas y 14 paquetes si se hubiera seguido exactamente el árbol de descenso.

La ventaja principal del reenvío por trayectoria invertida es que es razonablemente eficiente y fácil de poner en práctica. No requiere que los enrutadores conozcan los árboles de extensión ni tiene la carga extra de una lista de destinos o de un mapa de bits en cada paquete de difusión, como los tiene el direccionamiento multidestino. Tampoco requiere mecanismos especiales para detener el proceso, como en el caso de la inundación (ya sea un contador de escalas en cada paquete y un conocimiento previo del diámetro de la subred, o una lista de paquetes ya vistos por origen).

5.2.10. Enrutamiento por multitransmisión

En algunas aplicaciones, procesos muy separados trabajan juntos en grupos; por ejemplo, un grupo de procesos que implementan una base de datos distribuida. Con frecuencia es necesario que un proceso envíe un mensaje a todos los demás miembros del grupo. Si el grupo es pequeño, se puede simplemente transmitir a todos los demás miembros un mensaje punto a punto. Si el grupo es grande, esta estrategia es cara. A veces puede usarse la difusión, pero su uso para informar a 1000 máquinas de una red que abarca un millón de nodos es ineficiente porque la mayoría de los receptores no están interesados en el mensaje (o, peor aún, definitivamente están interesados pero no deben verlo). Por tanto, necesitamos una manera de enviar mensajes a grupos bien definidos de tamaño numéricamente grande, pero pequeños en comparación con la totalidad de la red.

El envío de un mensaje a uno de tales grupos se llama **multitransmisión**, y su algoritmo de enrutamiento es el **enrutamiento por multitransmisión**. En esta sección describiremos una manera de lograr el enrutamiento por multitransmisión. Para información adicional, véase (Deering y Cheriton, 1990; Deering *et al.*, 1994, y Rajagopalan, 1992).

Para la multitransmisión se requiere administración de grupos. Se necesita alguna manera de crear y destruir grupos, y un mecanismo para que los procesos se unan a los grupos y salgan de ellos. La forma de realizar estas tareas no le concierne al algoritmo de enrutamiento. Lo que sí le concierne es que, cuando un proceso se una a un grupo, informe a su *host* del hecho. Es importante que los enrutadores sepan cuáles de sus *hosts* pertenecen a qué grupos. Los *hosts* deben informar a sus enrutadores de los cambios en los miembros del grupo, o los enrutadores deben enviar periódicamente la lista de sus *hosts*. De cualquier manera, los enrutadores aprenden qué *hosts* pertenecen a cuáles grupos. Los enrutadores les dicen a sus vecinos, de manera que la información se propaga a través de la subred.

Para hacer el enrutamiento de multitransmisión, cada enrutador calcula un árbol de extensión que cubre a todos los demás enrutadores de la subred. Por ejemplo, en la figura 5-21(a) tenemos una subred con dos grupos, 1 y 2. Algunos enrutadores están conectados a *hosts* que pertenecen a uno o ambos grupos, como se indica en la figura. En la figura 5-21(b) se muestra un árbol de extensión para el enrutador de la izquierda.

Al enviar un proceso un paquete multitransmisión a un grupo, el primer enrutador examina su árbol de extensión y lo recorta, removiendo todas las líneas que no conducen a *hosts* que son

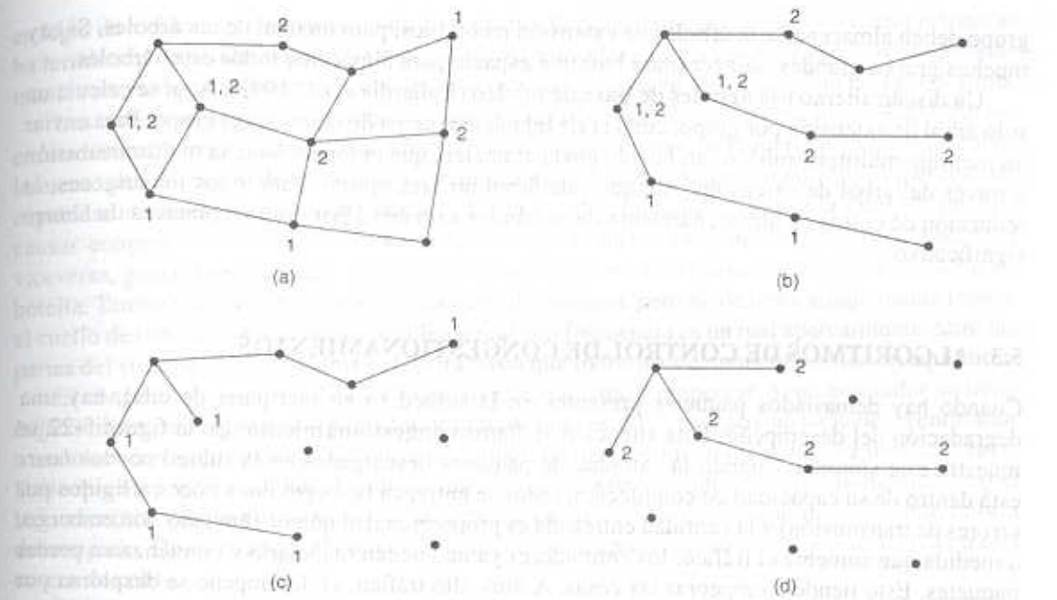


Figura 5-21. (a) Subred. (b) Árbol de extensión del enrutador del extremo izquierdo. (c) Árbol multitransmisión del grupo 1. (d) Árbol multitransmisión para el grupo 2.

miembros del grupo. En nuestro ejemplo de la figura 5-21, en la parte (c) se muestra el árbol de extensión recortado del grupo 1. De la misma manera, en la figura 5-21(d) se presenta el árbol de extensión recortado del grupo 2. Los paquetes multitransmisión se reenvían sólo a través del árbol de extensión apropiado.

Hay varias maneras de recortar el árbol de extensión. La más sencilla puede usarse si se maneja enrutamiento por estado de enlace, y cada enrutador está consciente de la topología completa de la subred, incluyendo qué *hosts* pertenecen a cuáles grupos. Entonces puede recortarse el árbol comenzando por el final de cada trayectoria y trabajando hacia la raíz, removiendo todos los enrutadores que no pertenecen al grupo en cuestión.

Con el enrutador por vector de distancia puede seguirse una estrategia de recorte diferente. El algoritmo básico es el reenvío por trayectoria invertida. Sin embargo, cuando un enrutador sin *hosts* interesados en un grupo en particular y sin conexiones con otros enrutadores recibe un mensaje de multitransmisión para ese grupo, responde con un mensaje de RECORTE (*PRUNE*), indicando al transmisor que no envíe más multitransmisiones para ese grupo. Cuando un enrutador que no tiene miembros del grupo entre sus propios *hosts* recibe uno de tales mensajes por todas sus líneas, también puede responder con un mensaje de RECORTE. De esta manera, la subred se recorta recursivamente.

Una desventaja potencial de este algoritmo es que no funciona bien en redes grandes. Supóngase que una red tiene n grupos, cada uno con un promedio de m miembros. Por cada

grupo deben almacenarse m árboles de extensión recortados, para un total de mn árboles. Si hay muchos grupos grandes, se necesitará bastante espacio para almacenar todos estos árboles.

Un diseño alterno usa **árboles de base de núcleo** (Ballardie *et al.*, 1993). Aquí se calcula un solo árbol de extensión por grupo, con la raíz (el núcleo) cerca de la mitad del grupo. Para enviar un mensaje multitransmisión, un *host* lo envía al núcleo, que entonces hace la multitransmisión a través del árbol de extensión. Aunque este árbol no será óptimo para todos los orígenes, la reducción de costos de almacenamiento de m árboles a un árbol por grupo representa un ahorro significativo.

5.3. ALGORITMOS DE CONTROL DE CONGESTIONAMIENTOS

Cuando hay demasiados paquetes presentes en la subred (o en una parte de ella), hay una degradación del desempeño. Esta situación se llama **congestionamiento**. En la figura 5-22 se muestra este síntoma. Cuando la cantidad de paquetes descargados en la subred por los *hosts* está dentro de su capacidad de conducción, todos se entregan (excepto unos pocos afligidos por errores de transmisión) y la cantidad entregada es proporcional al número enviado. Sin embargo, a medida que aumenta el tráfico, los enrutadores ya no pueden manejarlo y comienzan a perder paquetes. Esto tiende a empeorar las cosas. A muy alto tráfico, el desempeño se desploma por completo, y casi no hay entrega de paquetes.

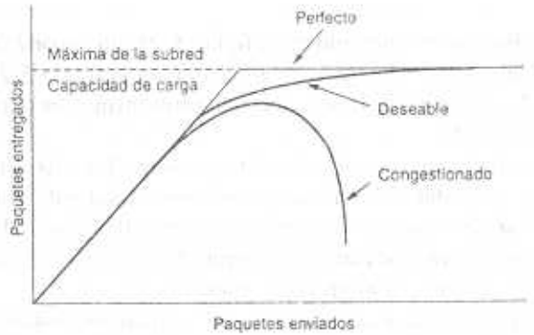


Figura 5-22. Cuando se genera demasiado tráfico, ocurren congestionamientos y se degrada marcadamente el desempeño.

El congestionamiento puede ocurrir por varias razones. Si repentinamente comienzan a llegar cadenas de paquetes por tres o cuatro líneas de entrada y todos necesitan la misma línea de salida, se generará una cola. Si no hay suficiente memoria para contenerlos a todos, se perderán paquetes. La adición de memoria puede ayudar hasta cierto punto, pero Nagle (1987) descubrió que si los enrutadores tienen una cantidad infinita de memoria, el congestionamiento empeora

en lugar de mejorar, ya que para cuando los paquetes llegan al principio de la cola su temporización ha terminado (repetidamente) y se han enviado duplicados. Todos estos paquetes serán debidamente reenviados al siguiente enrutador, aumentando la carga en todo el camino hasta el destino.

Los procesadores lentos también pueden causar congestionamientos. Si las CPU de los enrutadores son lentas para llevar a cabo las tareas de administración requeridas (*buffers* de encolamiento, actualización de tablas, etc.), pueden alargarse las colas, aun cuando haya un exceso de capacidad de línea. De la misma manera, las líneas de poco ancho de banda también pueden causar congestionamientos. La modernización de las líneas sin cambiar los procesadores, o viceversa, generalmente ayuda un poco, pero con frecuencia simplemente desplaza el cuello de botella. También la modernización de una parte del sistema, pero no de todo, simplemente mueve el cuello de botella a otra parte. El problema real con frecuencia es un mal apareamiento entre las partes del sistema. Este problema persistirá hasta que todos los componentes estén en equilibrio.

El congestionamiento tiende a alimentarse de sí mismo y empeorar. Si un enrutador no tiene *buffers* libres, debe ignorar los paquetes de entrada nuevos. Al descartarse un paquete, el enrutador transmisor (un vecino) puede terminar su temporización y retransmitirlo, tal vez muchas veces. Dado que no puede descartar el paquete mientras no sea reconocido, el congestionamiento en el lado receptor obliga al transmisor a abstenerse de liberar un *buffer* que normalmente habría desocupado. De este modo, el congestionamiento se acumula, como autos que se acercan a una caseta de cobro.

Vale la pena indicar explícitamente la diferencia entre el control de congestionamiento y el control de flujo, pues la relación es sutil. El control de congestionamiento tiene que ver con el aseguramiento de que la subred sea capaz de transportar el tráfico ofrecido. Es un asunto global, en el que interviene el comportamiento de todos los *hosts*, todos los enrutadores, el proceso de almacenamiento y reenvío dentro de los enrutadores y todos los demás factores que tienden a disminuir la capacidad de transporte de la subred.

El control de flujo, en contraste, se relaciona con el tráfico punto a punto entre un transmisor dado y un receptor dado; su tarea es asegurar que un transmisor rápido no pueda transmitir datos continuamente a una velocidad mayor que la que puede absorber el receptor. El control de flujo casi siempre comprende una retroalimentación directa del receptor al transmisor, para indicar al transmisor cómo van las cosas en el otro lado.

Para captar la diferencia entre estos dos conceptos, considere una red de fibra óptica con una capacidad de 1000 gigabits/seg en la que una supercomputadora está tratando de transferir un archivo a una computadora personal que opera a 1 Gbps. Aunque no hay congestionamiento (la red misma no está en problemas), se requiere control de flujo para obligar a la supercomputadora a detenerse con frecuencia para darle a la computadora personal un momento de respiro.

En el otro extremo, considere una red de almacenamiento y reenvío con líneas de 1 Mbps y 1000 computadoras grandes, la mitad de las cuales tratan de transferir archivos a 100 kbps a la otra mitad. Aquí el problema no es que los transmisores rápidos superen en poder a los receptores lentos, sino simplemente que el tráfico ofrecido total excede lo que la red puede manejar.

La razón por la que se confunden con frecuencia el control de congestionamiento y el control de flujo es que algunos algoritmos de control de congestionamiento operan enviando mensajes de regreso a varios orígenes, indicándoles que reduzcan su velocidad al meterse en

problemas la red. Por tanto, un *host* puede recibir un mensaje de "reducción de velocidad" tanto porque el receptor no puede manejar la carga como porque la red no la puede manejar. Regresaremos a este punto después.

Comenzaremos nuestro estudio del control de congestionamiento examinando un modelo general para manejarlo. Luego veremos enfoques generales para prevenirlo. Tras eso, veremos varios algoritmos dinámicos para manejarlo una vez que se ha establecido.

5.3.1. Principios generales del control de congestionamiento

Muchos problemas de los sistemas complejos, como las redes de computadoras, pueden verse desde el punto de vista de la teoría de control. Este enfoque conduce a la división de todas las soluciones en dos grupos: de ciclo abierto y de ciclo cerrado. Esencialmente, las soluciones de ciclo abierto intentan resolver el problema mediante un buen diseño, para asegurarse de que no ocurra desde un principio. Una vez que el sistema está operando, no se hacen correcciones a medio camino.

Las herramientas para llevar a cabo el control de ciclo abierto incluyen decidir cuándo aceptar tráfico nuevo, decidir cuándo descartar paquetes, y cuáles, y tomar decisiones de calendarización en varios puntos de la red. Todas tienen en común el hecho de que toman decisiones independientemente del estado actual de la red.

En contraste, las soluciones de ciclo cerrado se basan en el concepto de un ciclo de realimentación. Este enfoque tiene tres partes cuando se aplica al control de congestionamiento:

1. Supervisar el sistema para detectar cuándo y dónde ocurren congestionamientos.
2. Pasar esta información a lugares en los que puedan llevarse a cabo acciones.
3. Ajustar la operación del sistema para corregir el problema.

Pueden usarse varias métricas para supervisar la subred en búsqueda de congestionamientos. Las principales entre éstas son el porcentaje de paquetes descartados debido a falta de espacio de *buffer*, la longitud promedio de las colas, la cantidad de paquetes para los cuales termina la temporización, por lo que se transmiten de nuevo, el retardo promedio de los paquetes y la desviación estándar del retardo de paquete. En todos los casos, un aumento en las cifras indica un aumento en el congestionamiento.

El segundo paso del ciclo de realimentación es la transferencia de información relativa al congestionamiento desde el punto en que se detecta hasta el punto en que puede hacerse algo al respecto. La manera obvia es que el enrutador que detecta el congestionamiento envíe un paquete al origen (u orígenes) del tráfico, anunciando el problema. Por supuesto, estos paquetes extra aumentan la carga precisamente en el momento en que no se necesita más carga, es decir, cuando la subred está congestionada.

Sin embargo, existen otras posibilidades. Por ejemplo, puede reservarse un bit o campo en cada paquete para que los enrutadores lo llenen cuando el congestionamiento rebasa algún umbral. Al detectar un enrutador este estado congestionado, llena el campo de todos los paquetes de salida, para avisar a los vecinos.

Otro enfoque más es hacer que los *hosts* o enrutadores envíen periódicamente paquetes de prueba para preguntar explícitamente sobre el congestionamiento. Esta información puede usarse entonces para enrutar el tráfico alrededor de las áreas con problemas. Algunas estaciones de radio tienen helicópteros que vuelan sobre la ciudad para informar del congestionamiento en las calles, con la esperanza de que los escuchas enrutarán sus paquetes (autos) fuera de las zonas conflictivas.

En todos los esquemas de realimentación, la esperanza es que el conocimiento sobre los congestionamientos hará que los *hosts* emprendan acciones adecuadas con miras a reducir el congestionamiento. Para operar correctamente, debe ajustarse con cuidado la escala de tiempo. Si cada vez que llegan dos paquetes seguidos el enrutador grita ALTO, y cada vez que el enrutador está inactivo durante 20 μseg grita SIGA, el sistema oscilará sin control y nunca convergirá. Por otra parte, si un enrutador espera 30 minutos para asegurarse antes de decir cualquier cosa, el mecanismo de control de congestionamientos reaccionará tan lentamente que no será de utilidad. Para funcionar bien se requiere un justo medio, pero encontrar la constante de tiempo correcta no es un asunto trivial.

Se conocen muchos algoritmos de control de congestionamiento. Con objeto de organizarlos lógicamente, Yang y Reddy (1995) ha desarrollado una taxonomía de los algoritmos de control de congestionamiento; comienzan por dividir los algoritmos de ciclo abierto en algoritmos que actúan en el origen y que actúan en el destino. Los algoritmos de ciclo cerrado también se dividen en dos subcategorías: realimentación explícita contra realimentación implícita. En los algoritmos de realimentación explícita, se envían paquetes de regreso desde el punto de congestionamiento para avisar al origen. En los algoritmos implícitos, el origen deduce la existencia de un congestionamiento haciendo observaciones locales, como el tiempo necesario para el regreso de los reconocimientos.

La presencia de congestionamientos significa que la carga es (temporalmente) mayor (en una parte del sistema) que la que pueden manejar los recursos. Vienen a la mente dos soluciones: aumentar los recursos o disminuir la carga. Por ejemplo, la subred puede comenzar a usar líneas telefónicas de discado para aumentar temporalmente el ancho de banda entre ciertos puntos. En los sistemas como SMDS (véase el capítulo 1), se puede pedir a la portadora ancho de banda adicional por un rato. En los sistemas satelitales, el aumento en la potencia de transmisión frecuentemente da un mayor ancho de banda. La división del tráfico entre varias rutas en lugar de usar siempre la mejor también puede aumentar efectivamente el ancho de banda. Por último, pueden ponerse en línea enrutadores de repuesto que normalmente sirven sólo como respaldos (para hacer al sistema tolerante a fallas), a fin de contar con mayor capacidad cuando aparece un congestionamiento severo.

Sin embargo, a veces no es posible aumentar la capacidad, o ya ha sido aumentada al máximo. Entonces, la única forma de reducir el congestionamiento es disminuir la carga. Existen varias maneras de reducir la carga, que incluye negar el servicio a algunos usuarios, degradar el servicio para algunos o todos los usuarios y obligar a los usuarios a programar sus solicitudes de una manera más predecible.

Algunos de estos métodos, que estudiaremos en breve, se aplican mejor a los circuitos virtuales. En las subredes que usan circuitos virtuales internamente, estos métodos pueden usarse en la capa de red. En las subredes de datagramas algunas veces pueden utilizarse en las conexiones

de capa de transporte. En este capítulo nos enfocaremos en su uso en la capa de red. En el siguiente, veremos lo que puede hacerse en la capa de transporte para manejar los congestionamientos.

5.3.2. Políticas de prevención de congestionamientos

Comencemos nuestro estudio de los métodos de control de congestionamientos estudiando los sistemas de ciclo abierto. Estos sistemas están diseñados para reducir al mínimo el congestionamiento desde el inicio, en lugar de permitir que ocurra y reaccionar después del hecho. Los métodos tratan de lograr su meta usando políticas adecuadas en varios niveles. En la figura 5-23 vemos diferentes políticas de enlace de datos, red y transporte que pueden afectar el congestionamiento (Jain, 1990).

Capa	Políticas
Transporte	• Política de retransmisión • Política de caché de paquetes fuera de orden • Política de acuses de recibo • Política de control de flujo • Determinación de terminaciones de temporización
Red	• Circuitos virtuales *vs.* datagramas en la subred • Política de encolamiento y servicio de paquetes • Política de descartado de paquetes • Política de enrutamiento • Gestión de tiempo de vida de paquetes
Enlace de datos	• Política de retransmisiones • Política de caché de paquetes fuera de orden • Política de acuses de recibo • Política de control de flujo

Figura 5-23. Políticas que afectan el congestionamiento.

Comencemos por la capa de enlace de datos y avancemos hacia arriba. La política de retransmisiones tiene que ver con la rapidez con que termina de temporizar un transmisor y con lo que transmite al ocurrir una terminación de temporizador. Un transmisor nervioso que a veces termina de temporizar demasiado pronto y retransmite todos los paquetes pendientes usando el protocolo de regresar *n* impondrá una carga más pesada al sistema que un transmisor calmado que usa repetición selectiva. Muy relacionada con esto está la política de *caché*. Si los receptores rutinariamente descartan todos los paquetes que llegan fuera de orden, estos paquetes tendrán que transmitirse nuevamente después, creando una carga extra.

La política de acuse de recibos también afecta el congestionamiento. Si cada paquete es reconocido de inmediato, los paquetes de acuse de recibo generan tráfico extra. Sin embargo, si se guardan los acuses de recibo para incorporarlos en el tráfico en reversa, pueden resultar terminaciones de temporización y retransmisiones extra. Un esquema de control de flujo estricto (por ejemplo, una ventana pequeña) reduce la tasa de datos y permite, por tanto, atacar los congestionamientos.

En la capa de red, la decisión entre circuitos virtuales y datagramas afecta el congestionamiento, ya que muchos algoritmos de control de congestionamiento sólo funcionan con subredes de circuitos virtuales. La política de encolamiento y servicio de paquetes se refiere a que los enrutadores tengan una cola por línea de entrada, una cola por línea de salida o ambas cosas. También se relaciona con el orden de proceso de los paquetes (por ejemplo, por turno circular o con base en prioridades). La política de descartado es la regla que indica qué paquete descartar cuando no hay espacio. Una buena política puede ayudar a aliviar el congestionamiento y una mala puede hacerlo peor.

El algoritmo de enrutamiento puede evitar el congestionamiento distribuyendo el tráfico entre todas las líneas, pero uno malo puede enviar demasiado tráfico por líneas ya congestionadas. Por último, la administración del tiempo de vida de los paquetes se encarga del tiempo que puede vivir un paquete antes de ser descartado. Si este tiempo es demasiado grande, los paquetes perdidos pueden bloquear la operación durante un buen rato, pero si es demasiado corto, los paquetes pueden expirar antes de alcanzar su destino, induciendo retransmisiones.

En la capa de transporte surgen los mismos problemas que en la capa de enlace de datos, pero además es más difícil la determinación del intervalo de expiración, porque el tiempo de tránsito a través de la red es menos predecible que el tiempo de tránsito por un alambre entre dos enrutadores. Si el intervalo es demasiado corto, se enviarán innecesariamente paquetes extra; si es demasiado largo, se reducirá el congestionamiento, pero el tiempo de respuesta sufrirá cada vez que se pierda un paquete.

5.3.3. Conformación de tráfico

Una de las causas principales del congestionamiento es que el tráfico con frecuencia es por ráfagas. Si pudiera hacerse que los *hosts* transmitieran a una tasa uniforme, serían menos comunes los congestionamientos. Otro método de ciclo abierto para ayudar a manejar el congestionamiento es obligar a la transmisión de los paquetes a una tasa más predecible. Este enfoque de administración de congestionamiento se usa ampliamente en las redes ATM y se llama **conformación de tráfico**.

La conformación de tráfico se ocupa de la regulación de la *tasa* promedio (y tendencia a ráfagas) de la transmisión de datos. En contraste, los protocolos de ventana corrediza que estudiamos antes limitan la cantidad de datos en tránsito al mismo tiempo, no la rapidez a la que se transmiten. Al establecerse un circuito virtual, el usuario y la subred (es decir, el cliente y la portadora) acuerdan cierto patrón de tráfico (es decir, forma) para ese circuito. Mientras el cliente cumpla con su parte del trato y sólo envíe paquetes según el contrato acordado, la portadora promete entregarlos todos a tiempo. La conformación de tráfico reduce el congestionamiento y ayuda por tanto a la portadora a cumplir sus promesas. Tales arreglos no son tan importantes para las transferencias de archivos, pero son de gran importancia para los datos en tiempo real, como en las conexiones de audio y vídeo, que no toleran bien los congestionamientos.

En efecto, con la conformación de tráfico, el cliente le dice a la portadora: "mi patrón de transmisiones se parecerá a esto. ¿Puedes manejarlo?" Si la portadora está de acuerdo, surge el asunto de que la portadora sepa si el cliente está ajustándose al acuerdo, y de lo que debe hacer

si no es así. La supervisión de un flujo de tráfico se llama **vigilancia de tráfico**. El acuerdo de una conformación de tráfico y su vigilancia posterior son más sencillos en las subredes de circuitos virtuales que en las subredes de datagramas. Sin embargo, aun en las subredes de datagramas pueden aplicarse los mismos conceptos a las conexiones de capa de transporte.

Algoritmo de cubeta con goteo

Imagínese una cubeta con un pequeño agujero en el fondo, como se ilustra en la figura 5-24(a). Sin importar la rapidez con que entra agua en la cubeta, el flujo de salida tiene una tasa constante, ρ, cuando hay agua en la cubeta, y una tasa de cero cuando la cubeta está vacía. También, una vez que se llena la cubeta, cualquier agua adicional que entra se derrama por los costados y se pierde (es decir, no aparece en el flujo por debajo del agujero).

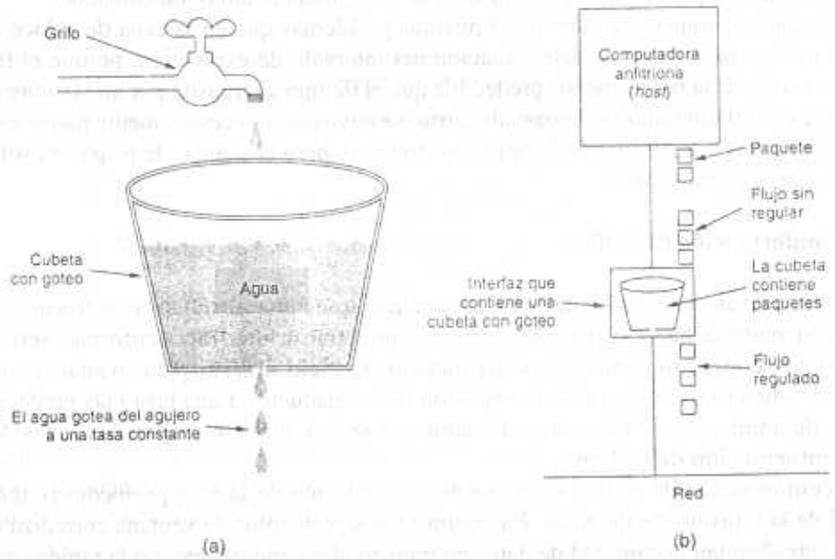


Figura 5-24. (a) Una cubeta con goteo, llena de agua. (b) Cubeta con goteo, llena de paquetes.

Puede aplicarse el mismo concepto a los paquetes, como se muestra en la figura 5-24(b). Conceptualmente, cada *host* está conectado a la red mediante una interfaz que contiene una cubeta con goteo, es decir, una cola interna finita. Si llega un paquete a la cola cuando está llena, se descarta el paquete. En otras palabras, si uno o más procesos del *host* tratan de enviar paquetes cuando ya está en cola la cantidad máxima, se descarta el paquete sin más. Este arreglo puede incorporarse en el *hardware* de la interfaz, o simularse a través del sistema operativo del *host*. El esquema fue propuesto inicialmente por Turner (1986), y se llama **algoritmo de cubeta con goteo** (*leaky bucket algorithm*). De hecho, no es otra cosa que un sistema de encolamiento de un solo servidor con un tiempo de servicio constante.

El *host* puede poner en la red un paquete por pulso de reloj. Nuevamente, esto puede obligarse desde la tarjeta de interfaz o desde el sistema operativo. Este mecanismo convierte un flujo desigual de paquetes de los procesos de usuario dentro del *host* en un flujo continuo de paquetes hacia la red, moderando las ráfagas y reduciendo en buena medida las posibilidades de congestionamiento.

Cuando los paquetes son todos del mismo tamaño (por ejemplo, células ATM), este algoritmo puede usarse como se describe. Sin embargo, al usarse paquetes de tamaño variable, con frecuencia es mejor permitir un número fijo de bytes por pulso, en lugar de un solo paquete. Por tanto, si la regla es de 1024 bytes por pulso, pueden recibirse por pulso un solo paquete de 1024 bytes, dos paquetes de 512 bytes, cuatro paquetes de 256 bytes, etc. Si el conteo de bytes residuales es demasiado bajo, el siguiente paquete debe esperar hasta el siguiente pulso.

La implementación del algoritmo de cubeta con goteo es fácil. La cubeta con goteo consiste en una cola finita. Al llegar un paquete, si hay espacio en la cola, se agrega a ella; de otro modo se descarta. En cada pulso de reloj se transmite un paquete (a menos que la cola esté vacía).

La cubeta con goteo que usa conteo de bits se implementa casi de la misma manera. En cada pulso se inicia un contador en n. Si el primer paquete de la cola tiene menos bytes que el valor actual del contador, se transmite y se disminuye el contador en esa cantidad de bytes. Pueden enviarse paquetes adicionales en tanto el contador sea lo suficientemente grande. Al caer el contador por debajo de la longitud del siguiente paquete de la cola, se detiene la transmisión hasta el siguiente pulso, en cuyo momento se sobrescribe el conteo de bytes residuales, perdiéndose.

Como ejemplo de cubeta con goteo, imagine que una computadora puede producir datos a razón de 25 millones de bytes/seg (200 Mbps) y que la red también opera a esta velocidad. Sin embargo, los enrutadores pueden manejar esta tasa de datos sólo durante intervalos cortos. Durante intervalos grandes funcionan mejor con tasas que no exceden 2 millones de bytes/seg. Ahora suponga que los datos llegan en ráfagas de un millón de bytes, con una ráfaga de 40 mseg cada segundo. Para reducir la tasa promedio a 2 MB/seg, podemos usar una cubeta con goteo de ρ = 2 MB/seg y capacidad C de 1 MB. Esto significa que las ráfagas de hasta 1 MB pueden manejarse sin pérdidas de datos, ya que se distribuyen a través de 500 mseg, sin importar la velocidad a la que lleguen.

En la figura 5-25(a) vemos la entrada de la cubeta con goteo operando a 25 MB/seg durante 40 mseg. En la figura 5-25(b) vemos la salida drenándose a una velocidad uniforme de 2 MB/seg durante 500 mseg.

Algoritmo de cubeta con ficha

El algoritmo de cubeta con goteo impone un patrón de salida rígido a la tasa promedio, sin importar la cantidad de ráfagas que tenga el tráfico. En muchas aplicaciones es mejor permitir que la salida se acelere un poco al llegar ráfagas grandes, por lo que se necesita un algoritmo más flexible, de preferencia uno que nunca pierda datos. El algoritmo de cubeta con ficha (**token bucket algorithm**) es uno de tales algoritmos. En este algoritmo, la cubeta con goteo contiene

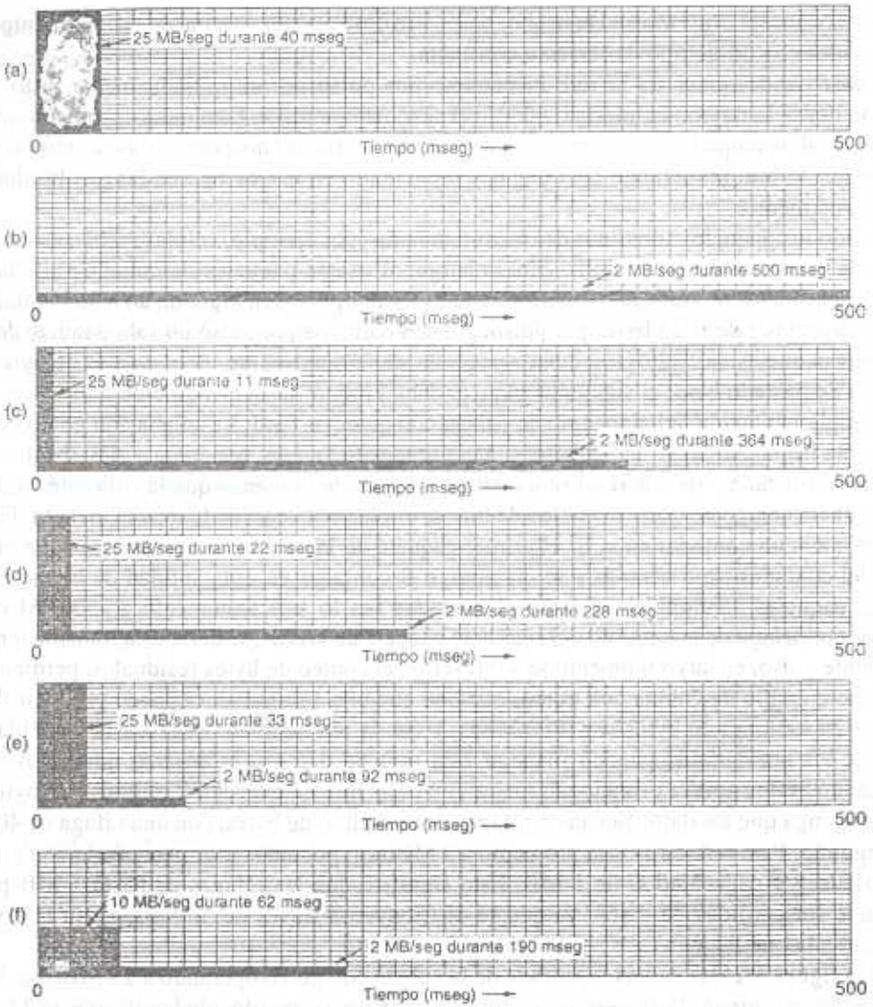


Figura 5-25. (a) Entrada a una cubeta con goteo. (b) Salida de una cubeta con goteo. (c) - (e) Salida de una cubeta con fichas con capacidades de 250kB, 500kB y 750kB. (f) Salida de una cubeta con fichas de 500kB que alimenta a una cubeta con goteo de 10 MB/seg.

fichas, generadas por un reloj a razón de una ficha cada ΔT seg. En la figura 5-26(a) vemos una cubeta que contiene tres fichas, con cinco paquetes esperando ser transmitidos. Para transmitir un paquete, el transmisor debe capturar y destruir una ficha. En la figura 5-26(b) vemos que han pasado tres de los cinco paquetes, pero los otros dos están atorados, esperando la generación de dos o más fichas.

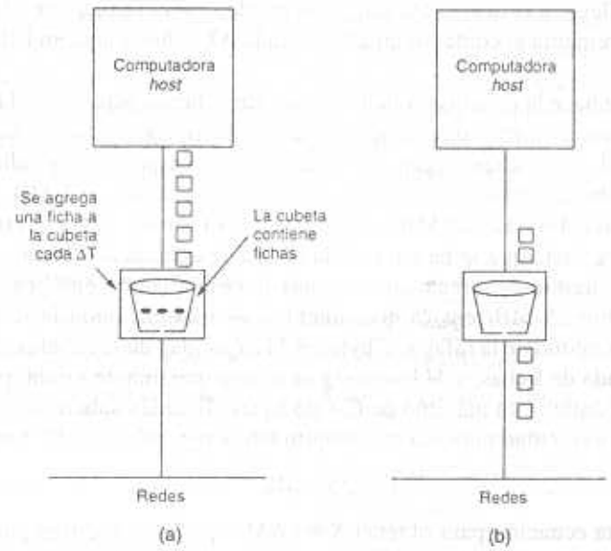


Figura 5-26. Algoritmo de cubeta con fichas. (a) Antes. (b) Después.

El algoritmo de cubeta con ficha ofrece una forma diferente de conformación de tráfico que el algoritmo de cubeta con goteo. Este último no permite que los *hosts* inactivos acumulen permisos para enviar posteriormente ráfagas grandes. El algoritmo de cubeta con ficha sí permite el ahorro, hasta el tamaño máximo de la cubeta, n. Esta propiedad significa que pueden enviarse a la vez ráfagas de hasta n paquetes, permitiendo cierta irregularidad en la corriente de salida y dando una respuesta más rápida a las ráfagas de entrada repentinas.

Otra diferencia entre los dos algoritmos es que el algoritmo de cubeta con ficha descarta las fichas cuando la cubeta se llena, pero nunca descarta los paquetes. En contraste, el algoritmo de cubeta con goteo descarta los paquetes cuando se llena la cubeta.

Aquí también es posible una variación menor, en la que cada ficha representa el derecho de transmisión, no de un paquete, sino de k bytes. Sólo puede transmitirse un paquete si hay suficientes fichas disponibles para cubrir su longitud en bytes. Las fichas fraccionarias se guardan para uso futuro.

Los algoritmos de cubeta con goteo y cubeta con ficha pueden servir también para regular el tráfico entre los enrutadores, de la misma manera en que se usan para regular la salida de un *host*, como en nuestros ejemplos. Sin embargo, una diferencia clara es que una cubeta con ficha que regula a un *host* puede hacer que el *host* detenga el envío cuando las reglas dicen que debe hacerlo. Indicar a un enrutador que detenga la transmisión mientras sigue recibiendo entradas puede resultar en la pérdida de datos.

La implementación del algoritmo básico de cubeta con ficha simplemente es una variable que cuenta fichas. El contador se incrementa en uno cada ΔT y disminuye en uno cada vez que se

envía un paquete. Al llegar a cero el contador, ya no pueden enviarse paquetes. En la variante de conteo de bits, se incrementa el contador en k bytes cada ΔT y disminuye en la longitud de cada paquete enviado.

En esencia, lo que hace la cubeta con ficha es permitir ráfagas, pero limitadas a una longitud máxima regulada. Véase un ejemplo en la figura 5-25(c). Aquí tenemos una cubeta de fichas con 250 KB de capacidad. Las fichas llegan con una tasa tal que permite la salida a 2 MB/seg. Suponiendo que la cubeta con ficha está llena cuando llega la ráfaga de 1 MB, la cubeta puede drenarse a la velocidad máxima de 25 MB/seg durante unos 11 mseg. Entonces tiene que desacelerarse hasta 2 MB/seg hasta que se ha enviado la ráfaga de entrada completa.

El cálculo de la longitud de ráfaga con tasa máxima es un tanto complicado. No es sólo la división de 1 MB entre 25 MB/seg, ya que, mientras se está enviando la ráfaga, llegan más fichas. Si S seg es la longitud de la ráfaga, C bytes es la capacidad de la cubeta con ficha, ρ bytes/seg es la tasa de llegada de fichas, y M bytes/seg es la tasa máxima de salida, podemos ver que una ráfaga de salida contiene un máximo de $C + \rho S$ bytes. También sabemos que la cantidad de bytes en una ráfaga a velocidad máxima con longitud de S segundos es MS. Por tanto, tenemos

$$C + \rho S = MS$$

Podemos resolver esta ecuación para obtener $S = C/(M - \rho)$. Para nuestros parámetros de C = 250 KB, M = 25 MB/seg y ρ = 2 MB/seg, tenemos un tiempo de ráfaga de unos 11 mseg. En la figura 5-25(d) y en la figura 5-25(e) se muestra la cubeta con ficha para capacidades de 500 KB y 750 KB, respectivamente.

Un problema potencial con el algoritmo de cubeta con ficha es que permite ráfagas largas, aunque puede regularse el intervalo máximo de ráfaga mediante una selección cuidadosa de ρ y M. Con frecuencia es deseable reducir la tasa pico, pero sin regresar al bajo valor de la cubeta con goteo original.

Una manera de lograr tráfico más uniforme es poner una cubeta con goteo después de la cubeta con ficha. La tasa de la cubeta con goteo deberá ser mayor que la ρ de la cubeta con ficha, pero menor que la tasa máxima de la red. En la figura 5-25(f) se muestra la salida de una cubeta con ficha de 500 KB seguida de una cubeta con goteo de 10 MB/seg.

La vigilancia de estos esquemas puede ser un tanto complicada. En esencia, la red tiene que simular el algoritmo y asegurarse que no se envíen más paquetes o bytes de lo permitido. Los paquetes de más se descartan o se degradan, como estudiaremos después.

5.3.4. Especificaciones de flujo

La conformación de tráfico es más efectiva cuando el transmisor, el receptor y la subred están de acuerdo. Para ponerse de acuerdo, es necesario especificar el patrón de tráfico de una manera precisa. Tal arreglo se llama **especificación de flujo**, y consiste en una estructura de datos que describe tanto el patrón de tráfico interpolado como la calidad del servicio deseado por las aplicaciones. Una especificación se puede aplicar tanto a los paquetes enviados por un circuito virtual como a una secuencia de datagramas enviados entre un origen y un destino (o aun a varios destinos).

En esta sección describiremos una especificación de flujo diseñada por Partridge (1992), la cual se presenta en la figura 5-27. La idea es que, antes de establecer una conexión, o antes del envío de una secuencia de datagramas, el origen presente la especificación de flujo a la subred, para su aprobación. La subred puede aceptarla, rechazarla, o presentar una contrapropuesta ("no puedo darte un retardo promedio de 100 mseg; ¿te sirve uno de 150 mseg?"). Una vez que el transmisor y la subred han llegado a un acuerdo, el transmisor puede preguntar al receptor si también está de acuerdo.

Características de la entrada	Servicio deseado
Tamaño máximo de paquetes (bytes)	Sensibilidad a pérdidas (bytes)
Tasa de cubeta con fichas (bytes/seg)	Intervalo de pérdidas (µseg)
Tamaño de la cubeta con fichas (bytes)	Sensibilidad a pérdidas por ráfaga (paquetes)
Tasa de transmisión máxima (bytes/seg)	Retardo mínimo notado (µseg)
	Variación máxima del retardo (µseg)
	Calidad de la garantía

Figura 5-27. Ejemplo de especificación de flujo.

Examinemos ahora los parámetros de nuestra especificación de flujo de ejemplo, comenzando por la especificación de tráfico. El *tamaño máximo de paquete* indica el tamaño que pueden tener los paquetes. Los siguientes dos parámetros suponen implícitamente que el tráfico se conformará mediante el algoritmo de cubeta con fichas trabajando con bytes; indican la cantidad de bytes que se colocan en la cubeta con fichas por segundo, así como el tamaño de la cubeta. Si la tasa es de r bytes/seg y el tamaño de la cubeta es de b bytes, entonces, en un intervalo de tiempo arbitrario Δt, la cantidad máxima de bytes que pueden enviarse es de $b + r\Delta t$. Aquí, el primer término representa el contenido máximo de la cubeta al inicio del intervalo y el segundo representa los créditos nuevos que han entrado durante el intervalo. La *tasa de transmisión máxima* es la tasa más alta que puede producir el *host* en cualquier condición, y especifica implícitamente el intervalo de tiempo en el que podría vaciarse la cubeta con fichas.

En la segunda columna se especifica lo que la aplicación pide a la subred. El primer y segundo parámetros representan el numerador y el denominador de una fracción que da la relación de pérdidas máxima aceptable (por ejemplo, 1 byte por hora). Alternamente, estos parámetros pueden indicar que el flujo es insensible a la pérdida de paquetes. La *sensibilidad a pérdidas en ráfaga* indica la cantidad de paquetes perdidos consecutivos que puede tolerarse.

Los siguientes dos parámetros de servicio tienen que ver con el retardo. El *retardo mínimo notado* indica el tiempo que puede retardarse un paquete sin que la aplicación se dé cuenta. En una transferencia de archivo, podría ser de un segundo, pero en un flujo de audio el límite podría ser de 3 mseg. La *variación máxima del retardo* intenta cuantificar el hecho de que algunas aplicaciones son insensibles al retardo mismo, pero son altamente sensibles a la **fluctuación**, es decir, a la cantidad de variación en el tiempo de tránsito de los paquetes de lado a lado. Este

parámetro es el doble de microsegundos que el retardo de un paquete puede variar respecto del promedio. Por tanto, un valor de 2000 significa que el paquete podrá llegar hasta 1 mseg antes o después, pero no más.

Por último, la *calidad de la garantía* indica si la aplicación realmente necesita lo que está pidiendo o no. Por un lado, las características de pérdida y retardo podrían ser metas ideales, pero no hay problema si no se logran; por otro, podrían ser tan importantes que, de no cumplirse, la aplicación simplemente termina. También son posibles posiciones intermedias.

Aunque hemos visto a la especificación de flujo como una solicitud de la aplicación a la subred, también puede ser un valor devuelto que indica lo que puede hacer la subred. Por tanto, puede usarse para una negociación prolongada del nivel de servicio.

Un problema inherente a cualquier especificación de flujo es que la aplicación podría no saber realmente lo que quiere. Por ejemplo, un programa de aplicación que se ejecuta en Nueva York podría conformarse con un retardo de 200 mseg a Sydney, pero estar muy descontenta con el mismo retardo de 200 mseg a Boston. Aquí, el "servicio mínimo" claramente es una función de lo que se cree que es posible.

5.3.5. Control de congestionamientos en las subredes de circuitos virtuales

Los métodos de control del congestionamiento antes descritos son básicamente de ciclo abierto: tratan de evitar que ocurran los congestionamientos, en lugar de manejarlos una vez ocurridos. En esta sección describiremos algunos enfoques para el control dinámico de congestionamientos en las redes de circuitos virtuales. En las siguientes dos veremos técnicas que pueden usarse en cualquier subred.

Una de las técnicas que se usa ampliamente para evitar que empeoren de los congestionamientos que ya han comenzado es el **control de admisión**. La idea es sencilla: una vez que se ha detectado el congestionamiento, no se establecen circuitos virtuales nuevos hasta que ha desaparecido el problema. Por tanto, fallan los intentos por establecer conexiones nuevas de capa de transporte. Permitir el acceso a más gente simplemente empeoraría las cosas. Aunque este enfoque es burdo, su implementación es sencilla. En el sistema telefónico, al carga extrarse un conmutador también se pone en práctica el control de admisión, al no darse tonos de marca.

Un enfoque alterno es permitir el establecimiento de nuevos circuitos virtuales, pero enrutando cuidadosamente los circuitos nuevos por otras rutas que no tienen problemas. Por ejemplo, considere la subred de la figura 5-28(a), en la cual están congestionados dos enrutadores, según se indica.

Supóngase que un *host* conectado al enrutador *A* quiere establecer una conexión con un *host* conectado al enrutador *B*. Normalmente, esta conexión pasaría a través de uno de los enrutadores congestionados. Para evitar esta situación, podemos redibujar la subred como se muestra en la figura 5-28(b), omitiendo los enrutadores congestionados y todas sus líneas. La línea de trazo interrumpido muestra una ruta posible para el circuito virtual que evita los enrutadores congestionados.

Otra estrategia que tiene que ver con los circuitos virtuales es negociar un acuerdo entre el *host* y la subred al establecerse un circuito virtual. Este arreglo normalmente especifica el volumen y la forma del tráfico, la calidad de servicio requerida y otros parámetros. Para cumplir

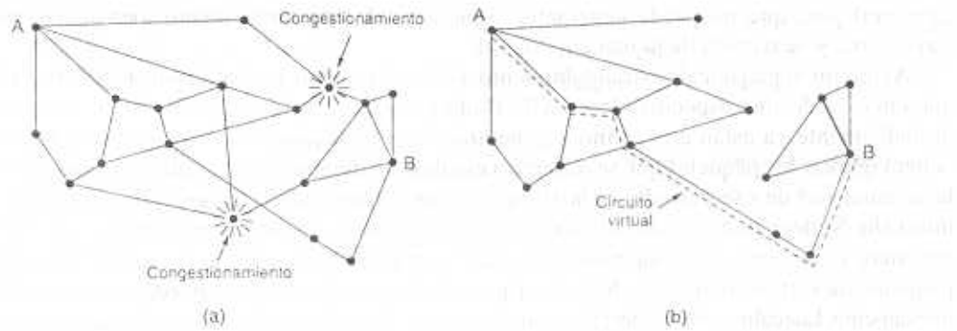


Figura 5-28. (a) Subred congestionada. (b) Subred redibujada que elimina el congestionamiento, y circuito virtual de *A* a *B*.

con su parte del acuerdo, la subred típicamente reservará servicios a lo largo de la trayectoria al establecerse el circuito. Estos recursos pueden incluir espacio en tablas y en *buffer* en los enrutadores y ancho de banda en las líneas. De esta manera, es poco probable que ocurran congestionamientos en los circuitos virtuales nuevos, porque está garantizada la disponibilidad de todos los recursos necesarios.

Este tipo de reservaciones pueden llevarse a cabo todo el tiempo como procedimiento operativo estándar, o sólo cuando está congestionada la subred. Una desventaja de hacerlo todo el tiempo es que se tiende a desperdiciar recursos. Si seis circuitos virtuales que podrían usar 1 Mbps pasan por la misma línea física de 6 Mbps, la línea tiene que marcarse como llena, aunque pocas veces ocurra que los seis circuitos virtuales transmitan al mismo tiempo. En consecuencia, el precio del control de congestionamiento es un ancho de banda no usado.

5.3.6. Paquetes de estrangulamiento

Veamos ahora un enfoque que puede usarse tanto en circuitos virtuales como en subredes de datagramas. Cada enrutador puede supervisar fácilmente el uso de sus líneas de salida y de otros recursos. Por ejemplo, puede asociar cada línea a una variable real, u, cuyo valor, entre 0.0 y 1.0, refleja el uso reciente de esa línea. Para tener una buena estimación de u, puede tomarse periódicamente una muestra del uso instantáneo de la línea, f (0 o 1), y actualizar u periódicamente de acuerdo con

$$u_{\text{nvo}} = au_{\text{ant}} + (1 - a)f$$

donde la constante a determina la rapidez con que el enrutador olvida la historia reciente.

Siempre que u rebasa el umbral, la línea de salida entra en un estado de "advertencia". Cada paquete nuevo que llega se revisa para ver si su línea de salida está en el estado de advertencia. De ser así, el enrutador envía un **paquete de estrangulamiento** de regreso al *host* de origen, dándole el destino encontrado en el paquete. El paquete original se marca (se enciende un bit de

cabecera) para que no pueda generar más paquetes de estrangulamiento más adelante en la trayectoria, y se reenvía de la manera normal.

Al recibir el paquete de estrangulamiento, el *host* de origen, está obligado a reducir el tráfico que envía al destino especificado en *X*%. Dado que otros paquetes destinados al mismo lugar probablemente ya están en camino y generarán nuevos paquetes de estrangulamiento, el *host* deberá ignorar los paquetes que se refieran a ese destino durante un intervalo de tiempo fijo. Tras la terminación de ese periodo, el *host* espera más paquetes de estrangulamiento durante otro intervalo. Si llega uno, es que la línea aún está congestionada, por lo que el *host* reduce el flujo aún más y comienza a ignorar nuevamente los paquetes de estrangulamiento. Si no llegan paquetes de estrangulamiento durante el periodo de escucha, el *host* puede aumentar el flujo nuevamente. La realimentación implícita en este protocolo puede ayudar a evitar el congestionamiento sin reducir ningún flujo a menos que ocurran problemas.

El *host* puede reducir el tráfico ajustando sus parámetros de política; por ejemplo, el tamaño de la ventana o la tasa de salida de la cubeta con goteo. Por lo común, el primer paquete de estrangulamiento hace que la tasa de datos se reduzca al 0.50 de su valor previo, el siguiente causa una reducción a 0.25, etc. Los aumentos se llevan a cabo en incrementos menores, para evitar que ocurra nuevamente el congestionamiento.

Se han propuesto diversas variaciones de este algoritmo de control de congestionamiento. Por una parte, los enrutadores pueden mantener varios umbrales. Dependiendo del umbral cruzado, el paquete de estrangulamiento puede contener una advertencia leve, una advertencia severa o un ultimátum.

Otra variación es el manejo de longitudes de cola o de uso de los *buffers*, en lugar del uso de línea, como señal de alarma. Por supuesto, puede usarse el mismo ponderado exponencial con esta métrica que con *u*.

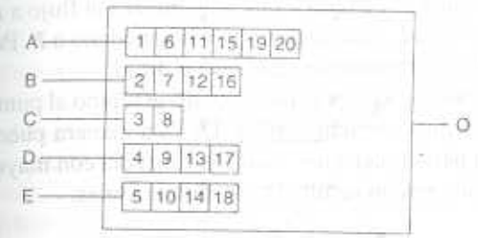
Encolamiento justo ponderado

Un problema con el uso de paquetes de estrangulamiento es que la acción a tomar por los *hosts* de origen es voluntaria. Supóngase que un enrutador está siendo abrumado por paquetes de cuatro orígenes, y que envía paquetes de estrangulamiento a todos ellos. Uno de ellos se refrena, como se espera, pero los otros tres siguen enviando a toda velocidad. El resultado es que el *host* honesto recibe una porción aún menor del ancho de banda del que tenía antes.

Para superar este problema, y por tanto hacer más atractivo el acatamiento, Nagle (1987) propuso un algoritmo de **encolamiento justo**. La esencia del algoritmo es que los enrutadores tienen varias colas para cada línea de salida, una para cada origen. Cuando una línea está ociosa, el enrutador examina las colas por turno, tomando el primer paquete de la siguiente cola. De esta manera, con *n hosts* compitiendo por una línea de salida dada, cada *host* envía uno de cada *n* paquetes. El envío de más paquetes no mejorará esta fracción. Algunos conmutadores ATM usan este algoritmo.

Aunque ayuda, el algoritmo tiene un problema: da más ancho de banda a los *hosts* que usan paquetes grandes que a los que usan paquetes pequeños. Demers *et al.* (1990) han sugerido una mejora en la que la búsqueda por turno se hace de manera que simula una ronda byte por byte, en

lugar de una ronda paquete por paquete. En efecto, se examinan repetidamente las colas, byte por byte, hasta que se encuentra el pulso de reloj en el que termina cada paquete. Se ordenan los paquetes en orden de terminación y se envían en ese orden. El algoritmo se ilustra en la figura 5-29.



(a)

Paquete	Tiempo de terminación
C	8
B	16
D	17
E	18
A	20

(b)

Figura 5-29. (a) Un enrutador con cinco paquetes encolados para la línea *O*. (b) Tiempos de terminación de los cinco paquetes.

En la figura 5-29(a) vemos paquetes de longitudes de 2 a 6 bytes. En el pulso (virtual) de reloj 1 se envía el primer byte del paquete por la línea *A*; luego sale el primer byte del paquete por la línea *B*, etc. El primer paquete que termina es el *C*, tras 8 pulsos. El orden se da en la figura 5-29(b). En ausencia de paquetes nuevos, los paquetes se enviarán en el orden listado, de *C* a *A*.

Uno de los problemas de este algoritmo es que da a todos los *hosts* la misma prioridad. En muchas situaciones, es deseable dar al servidor de archivos y a otros servidores más ancho de banda que a los clientes, de modo que puedan transmitir dos o más bytes por pulso. Este algoritmo modificado se llama **encolamiento justo ponderado**, y es de amplio uso. A veces el peso es igual a la cantidad de circuitos virtuales o de flujos que salen de una máquina, por lo que cada proceso recibe el mismo ancho de banda. En (Shreedhar y Varghese, 1995) se estudia una puesta en práctica eficiente del algoritmo.

Paquetes de estrangulamiento de escala por escala

A altas velocidades y distancias grandes, el envío de un paquete de estrangulamiento a los *hosts* de origen no funciona bien, porque la reacción es muy lenta. Considere por ejemplo a un *host* en San Francisco (enrutador *A* de la figura 5-30) que está enviando tráfico a un *host* en Nueva York (enrutador *D* de la figura 5-30) a 155 Mbps. Si al *host* de Nueva York se le comienza a terminar el espacio de *buffer*, le llevará unos 30 mseg a un paquete de estrangulamiento regresar a San Francisco para indicarle que disminuya su velocidad. La propagación de paquetes de estrangulamiento se muestra en el segundo, tercer y cuarto pasos de la figura 5-30(a). En esos 30 mseg se habrán enviado otros 4.6 megabits (por ejemplo, en 10,000 células ATM). Aun si el *host* de San Francisco se desactiva de inmediato, los 4.6 megabits en el conducto continuarán fluyendo, y habrá que encargarse de ellos. Sólo hasta el séptimo diagrama de la figura 5-30(a) el enrutador de Nueva York notará un flujo más lento.

Un enfoque alterno es hacer que el paquete de estrangulamiento tenga efecto en cada escala que haga, como se muestra en la secuencia de la figura 5-30(b). Aquí, tan pronto como el paquete de estrangulamiento llega a *F*, se obliga a *F* a reducir el flujo a *D*. Hacerlo requerirá que *F* destine más *buffers* al flujo, ya que el origen aún está transmitiendo a toda máquina, pero da a *D* un alivio inmediato, como en un mensaje comercial de un remedio contra el dolor de cabeza. En el siguiente paso, el paquete de estrangulamiento llega a *E*, e indica a éste que reduzca el flujo a *F*. Esta acción le impone una mayor carga a los *buffers* de *E*, pero le da un alivio inmediato a *F*. Por último, el paquete de estrangulamiento llega a *A* y efectivamente se reduce el flujo.

El efecto neto de este esquema de escala por escala es proporcionar un alivio rápido al punto de congestionamiento, a expensas de usar más *buffers* corriente arriba. De esta manera puede cortarse el congestionamiento en la raíz, pero sin perder paquetes. La idea se estudia con mayor detalle en (Mishra y Kanakia, 1992) donde también se dan resultados de simulaciones.

5.3.7. Desprendimiento de carga

Cuando ninguno de los métodos anteriores elimina el congestionamiento, los enrutadores pueden sacar la artillería pesada: el desprendimiento de carga. El **desprendimiento de carga** es una manera rebuscada de decir que, al inundarse los enrutadores de paquetes que no pueden manejar, simplemente los tiran. El término viene del mundo de la generación de energía eléctrica, donde se refiere a la práctica de instalaciones que intencionalmente producen apagones en ciertas áreas para salvar a la red completa de venirse abajo en días de verano calurosos en los que la demanda de energía eléctrica excede por mucho el suministro.

Un enrutador abrumado por paquetes puede simplemente escoger paquetes al azar para desprenderse de ellos, pero normalmente puede hacer algo mejor. El paquete a descartar puede depender de las aplicaciones que se están ejecutando. En la transferencia de archivos vale más un paquete viejo que uno nuevo, pues el deshacerse del paquete 6 y mantener los paquetes 7 a 10 causará un hueco en el receptor que podría obligar a la retransmisión de los paquetes 6 a 10 (si el receptor descarta rutinariamente los paquetes fuera de orden). En un archivo de 12 paquetes, deshacerse del paquete 6 podría requerir la retransmisión de los paquetes 7 a 12, y deshacerse del 10 podría requerir la retransmisión sólo del 10 al 12. En contraste, en multimedia es más importante un paquete nuevo que uno viejo. La política anterior (más viejo es mejor que más nuevo, se llama con frecuencia **vino**, y la última (más nuevo es mejor que más viejo) se llama con frecuencia **leche**.

Un paso por encima de esto en cuanto a inteligencia requiere la cooperación de los transmisores. En muchas aplicaciones, algunos paquetes son más importantes que otros. Por ejemplo, ciertos algoritmos de compresión de vídeo transmiten periódicamente un marco entero y sus marcos subsiguientes como diferencias respecto al último marco completo. En este caso, es preferible desprenderse de un paquete que es parte de una diferencia desprenderse de uno que es parte de un marco completo. Como otro ejemplo, considere la transmisión de un documento que contiene texto ASCII e imágenes. La pérdida de una línea de píxeles de una imagen es mucho menos dañina que la pérdida de una línea de texto legible.

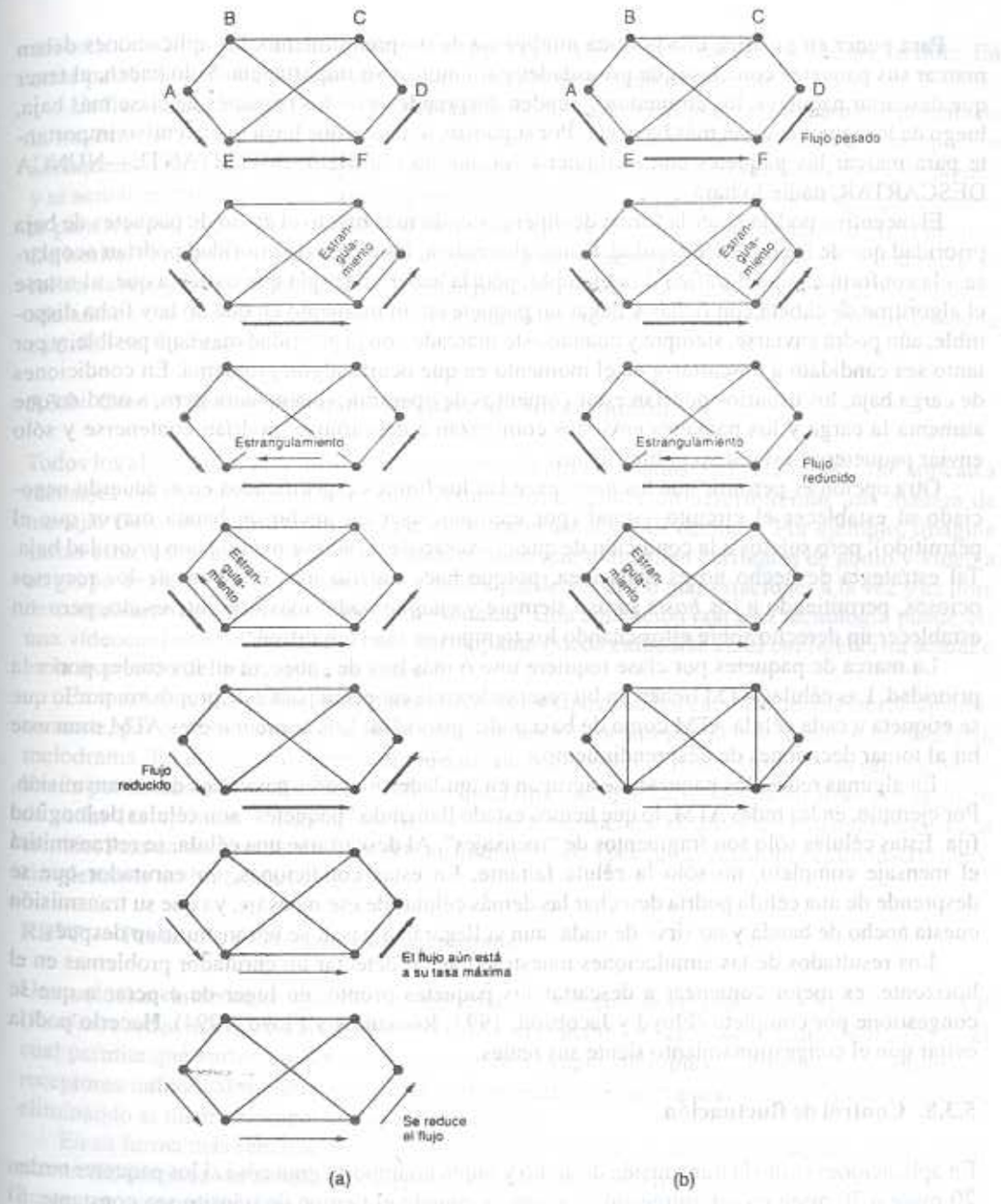


Figura 5-30. (a) Paquete de estrangulamiento que afecta sólo al origen. (b) Paquete de estrangulamiento que afecta cada escala por la que pasa.

Para poner en práctica una política inteligente de desprendimiento, las aplicaciones deben marcar sus paquetes con clases de prioridades para indicar su importancia. Si lo hacen, al tener que descartar paquetes, los enrutadores pueden desprenderse de los paquetes de clase más baja, luego de la siguiente clase más baja, etc. Por supuesto, a menos que haya un incentivo importante para marcar los paquetes con cualquier cosa que no sea MUY IMPORTANTE—NUNCA DESCARTAR, nadie lo hará.

El incentivo podría tener la forma de dinero, siendo más barato el envío de paquetes de baja prioridad que de los de alta prioridad. Como alternativa, las clases de prioridad podrían acoplarse a la conformación del tráfico. Por ejemplo, podría haber una regla que indicara que, al usarse el algoritmo de cubeta con fichas y llegar un paquete en un momento en que no hay ficha disponible, aún podrá enviarse, siempre y cuando esté marcado con la prioridad más baja posible, y por tanto sea candidato a descartarse en el momento en que ocurra algún problema. En condiciones de carga baja, los usuarios podrían estar contentos de operar de esta manera pero, a medida que aumenta la carga y los paquetes enviados comienzan a descartarse, podrían contenerse y sólo enviar paquetes al estar disponibles fichas.

Otra opción es permitir que los *hosts* excedan los límites especificados en el acuerdo negociado al establecer el circuito virtual (por ejemplo, usar un ancho de banda mayor que el permitido), pero sujetos a la condición de que el exceso de tráfico se marque con prioridad baja. Tal estrategia de hecho no es mala idea, porque hace un uso más eficiente de los recursos ociosos, permitiendo a los *hosts* su uso siempre y cuando nadie más esté interesado, pero sin establecer un derecho sobre ellos cuando los tiempos se vuelven difíciles.

La marca de paquetes por clase requiere uno o más bits de cabecera en los cuales poner la prioridad. Las células ATM tienen un bit reservado en la cabecera para este propósito, por lo que se etiqueta a cada célula ATM como de baja o alta prioridad. Los conmutadores ATM usan este bit al tomar decisiones de desprendimiento.

En algunas redes, los paquetes se agrupan en unidades mayores para fines de retransmisión. Por ejemplo, en las redes ATM, lo que hemos estado llamando "paquetes" son células de longitud fija. Estas células sólo son fragmentos de "mensajes". Al descartarse una célula, se retransmitirá el mensaje completo, no sólo la célula faltante. En estas condiciones, un enrutador que se desprende de una célula podría desechar las demás células de ese mensaje, ya que su transmisión cuesta ancho de banda y no sirve de nada: aun si llegaran a pasar, se retransmitirían después.

Los resultados de las simulaciones muestran que, al detectar un enrutador problemas en el horizonte, es mejor comenzar a descartar los paquetes pronto, en lugar de esperar a que se congestione por completo (Floyd y Jacobson, 1993; Romanow y Floyd, 1994). Hacerlo podría evitar que el congestionamiento siente sus reales.

5.3.8. Control de fluctuación

En aplicaciones como la transmisión de audio y vídeo no importa gran cosa si los paquetes tardan 20 mseg o 30 mseg en ser entregados, siempre y cuando el tiempo de tránsito sea constante. Si unos paquetes se tardan 20 mseg y otros 30 mseg, se obtendrá una calidad desigual del sonido o la imagen. Por tanto, el arreglo podría ser que el 99% de los paquetes se entreguen con un retardo

del orden de 24.5 mseg a 25.5 mseg. Por supuesto, el valor medio escogido debe ser factible. En otras palabras, debe calcularse una cantidad media de congestionamiento.

La fluctuación puede limitarse calculando el tiempo de tránsito esperado para cada escala de la trayectoria. Al llegar un paquete a un enrutador, el enrutador lo examina para saber qué tan adelantado o retrasado está respecto lo programado. Esta información se almacena en el paquete y se actualiza en cada escala. Si el paquete está adelantado, se retiene durante el tiempo suficiente para regresarlo a lo programado; si está retrasado, el enrutador trata de sacarlo rápidamente. De hecho, el algoritmo para determinar cuál de varios paquetes que compiten por una línea de salida debe seguir siempre puede escoger el paquete más retrasado. De esta manera, los paquetes adelantados se frenan y los paquetes retrasados se aceleran, reduciendo en ambos casos la cantidad de fluctuación.

5.3.9. Control de congestionamiento para multitransmisión

Todos los algoritmos de control de congestionamiento estudiados hasta el momento se aplican a mensajes de un solo origen a un solo destino. En esta sección describiremos una manera de manejar flujos de multitransmisión de varios orígenes a varios destinos. Por ejemplo, imagine varias estaciones de televisión de circuito cerrado que transmiten corrientes de audio y vídeo a un grupo de receptores, cada uno de los cuales puede ver una o más estaciones a la vez y es libre de conmutarse de estación a estación a voluntad. Una aplicación con esta tecnología puede ser una vídeoconferencia, en la cual cada participante puede enfocarse en el conferencista actual o en la expresión del jefe, según se desee.

En muchas aplicaciones multitransmisión, los grupos pueden cambiar de miembros dinámicamente, por ejemplo, cuando la gente entra en una vídeoconferencia o se aburre y cambia a un melodrama. En estas condiciones, el enfoque de hacer que los transmisores reserven ancho de banda por adelantado no funciona bien, ya que requeriría que cada transmisor llevara el registro de todas las entradas y salidas de su auditorio y regenerara el árbol de extensión con cada cambio. Para un sistema diseñado para transmitir televisión por cable, con millones de suscriptores, simplemente no funcionaría.

RSVP — Protocolo de reservación de recursos

Una solución interesante que puede controlar un entorno semejante es el protocolo RSVP (*Resource reSerVation Protocol*, **protocolo de reservación de recursos**) (Zhang *et al.*, 1993), el cual permite que varios transmisores transmitan a grupos múltiples de receptores, permite que receptores individuales conmuten canales libremente y perfecciona el uso del ancho de banda, eliminando al mismo tiempo el congestionamiento.

En su forma más sencilla, el protocolo usa enrutamiento de multitransmisión con árboles de extensión, como se vio antes. A cada grupo se le asigna un grupo de direcciones. Para enviar a un grupo, un transmisor pone la dirección del grupo en sus paquetes. El algoritmo estándar de multitransmisión construye entonces un árbol de extensión que cubre a todos los miembros del grupo. El algoritmo de enrutamiento no es parte del RSVP. La única diferencia con la multitransmisión

normal es un poco de información extra multitransmitida al grupo periódicamente para indicarle a los enrutadores a lo largo del árbol que mantengan ciertas estructuras de datos en sus memorias.

Como ejemplo, considere la red de la figura 5-31(a). Los *hosts* 1 y 2 son transmisores multitransmisión, y los *hosts* 3, 4 y 5 son receptores multitransmisión. En este ejemplo, los transmisores y los receptores son distintos pero, en general, los dos grupos pueden traslaparse. Los árboles de multitransmisión de los *hosts* 1 y 2 se muestran en la figura 5-31(b) y 5-31(c), respectivamente.

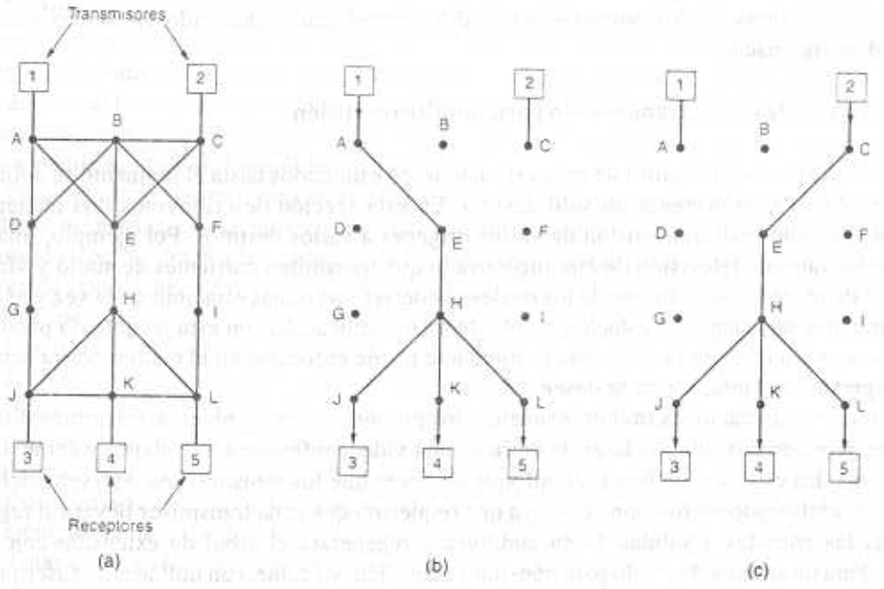


Figura 5-31. (a) Red. (b) Árbol de extensión de multitransmisión para el *host* 1. (c) Árbol de extensión de multitransmisión para el *host* 2.

Para obtener mejor recepción y eliminar el congestionamiento, cualquiera de los receptores de un grupo puede enviar un mensaje de reservación por el árbol al transmisor. El mensaje se propaga usando el algoritmo de reenvío por trayectoria invertida estudiado antes. En cada escala, el enrutador nota la reservación y reserva el ancho de banda necesario; si no hay suficiente ancho de banda disponible, informa de una falla. En el momento que el mensaje llega de regreso al origen, se ha reservado el ancho de banda desde el transmisor hasta el receptor que hace la solicitud de reservación a lo largo del árbol de extensión.

Un ejemplo de tales reservaciones se muestra en la figura 5-32(a). Aquí el *host* 3 ha solicitado un canal al *host* 1. Una vez establecido el canal, los paquetes pueden fluir de 1 a 3 sin congestionamientos. Ahora considere lo que sucede si el *host* 3 a continuación reserva un canal

al otro transmisor, el *host* 2, para que el usuario pueda ver dos programas de televisión a la vez. Se reserva una segunda trayectoria, como se muestra en la figura 5-32(b). Nótese que se requieren dos canales individuales del *host* 3 al enrutador *E*, porque se están transmitiendo dos corrientes independientes.

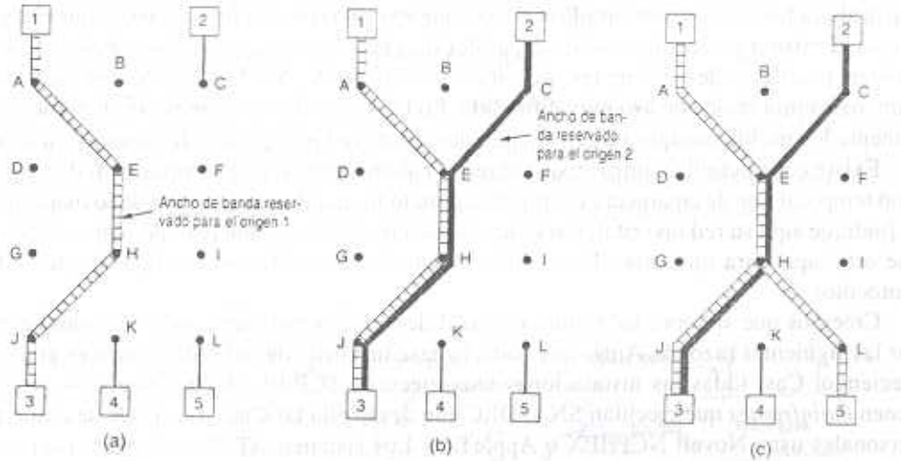


Figura 5-32. (a) El *host* 3 solicita un canal al *host* 1. (b) El *host* 3 solicita entonces un segundo canal al *host* 2. (c) El *host* 5 solicita un canal al *host* 1.

Por último, en la figura 5-32(c), el *host* 5 decide observar el programa transmitido por el *host* 1 y también hace una reservación. Primero se reserva ancho de banda dedicado hasta el enrutador *H*. Sin embargo, este enrutador ve que ya tiene una alimentación del *host* 1 por lo que, si ya se ha reservado el ancho de banda necesario, no necesita reservar nuevamente. Nótese que los *hosts* 3 y 5 podrían haber solicitado diferentes cantidades de ancho de banda (por ejemplo, el 3 tiene una televisión de blanco y negro, por lo que no quiere la información de color), así que la capacidad reservada debe ser lo bastante grande para satisfacer al receptor más voraz.

Al hacer una reservación, un receptor puede especificar (opcionalmente) uno o más orígenes de los que quiere recibir. También puede especificar si estas selecciones quedarán fijas durante toda la reservación, o si el receptor quiere mantener abierta la opción de cambiar los orígenes después. Los enrutadores usan esta información para optimar la planeación del ancho de banda. En particular, sólo se establece que dos receptores van a compartir una trayectoria si ambos están de acuerdo en no cambiar los orígenes posteriormente.

La razón de esta estrategia en el caso totalmente dinámico es que el ancho de banda reservado está desacoplado de la selección del origen. Una vez que un receptor ha reservado ancho de banda, puede conmutarse a otro origen y conservar la parte de la trayectoria existente que es válida para el nuevo origen. Si, por ejemplo, el *host* 2 está transmitiendo varias corrientes de

vídeo, el *host* 3 puede conmutarse entre ellas a voluntad sin cambiar su reservación: a los enrutadores no les importa el programa que está viendo el receptor.

5.4. INTERREDES

Hasta ahora hemos supuesto implícitamente que hay una sola red homogénea y que cada máquina usa el mismo protocolo en cada capa. Por desgracia, este supuesto es demasiado optimista. Existen muchas redes diferentes, que incluyen las LAN, MAN y WAN. En cada capa hay numerosos protocolos de uso muy difundido. En las siguientes secciones estudiaremos cuidadosamente los problemas que surgen cuando dos o más redes se juntan, formando una **interred**.

Existe considerable controversia sobre si la abundancia actual de tipos de red es una condición temporal que desaparecerá tan pronto como todo mundo se dé cuenta de lo maravillosa que es [indique aquí su red favorita], o si es una característica inevitable pero permanente del mundo, que está aquí para quedarse. Tener diferentes redes invariablemente implica tener diferentes protocolos.

Creemos que siempre habrá una variedad de redes (y por tanto, de protocolos) diferentes, por las siguientes razones. Antes que nada, la base instalada de redes diferentes es grande y está creciendo. Casi todas las instalaciones UNIX ejecutan TCP/IP. Muchos negocios grandes aún tienen *mainframes* que ejecutan SNA. DEC aún desarrolla DECnet. Las LAN de computadoras personales usan Novell NCP/IPX o AppleTalk. Los sistemas ATM comienzan a ser comunes. Por último, en las redes satelitales, celulares e infrarrojas se usan con frecuencia protocolos especializados. Esta tendencia continuará por años, debido a la gran cantidad de redes existentes y porque no todos los fabricantes perciben que es de interés suyo el que sus clientes puedan migrar fácilmente al sistema de otro fabricante.

Segundo, a medida que las computadoras y las redes se vuelven más baratas, el lugar de la toma de decisiones se desplaza hacia abajo. Muchas compañías tienen políticas en el sentido de que las compras de más de un millón de dólares tienen que ser aprobadas por la gerencia general, las compras de más de 100,000 dólares tienen que ser aprobadas por la gerencia media, pero las compras por debajo de 100,000 dólares pueden ser hechas por los jefes de los departamentos sin aprobación de los superiores. Esto puede conducir fácilmente a la instalación de Ethernet en el departamento de contabilidad, *token bus* en el departamento de ingeniería y *token ring* en el departamento de personal.

Tercero, las distintas redes (por ejemplo, ATM e inalámbricas) tienen tecnologías radicalmente diferentes, por lo que no debe sorprendernos que, a medida que hay avances nuevos en *hardware*, también se cree *software* nuevo adaptado al nuevo *hardware*. Por ejemplo, la casa típica ahora es como la oficina típica de hace 10 años: está llena de computadoras que no se hablan entre ellas. En el futuro, podría ser común que el teléfono, la televisión y otros aparatos estuvieran todos en red, para su control remoto. Esta nueva tecnología sin duda generará nuevos protocolos.

Como ejemplo de la interacción entre redes diferentes, considere el siguiente caso. En la mayoría de las universidades, los departamentos de informática e ingeniería electrónica tienen sus propias LAN, con frecuencia diferentes. Además, el centro de cómputo de la universidad

muchas veces tiene una *mainframe* y una supercomputadora, la primera para el cuerpo docente de humanidades que no desea meterse en el asunto del mantenimiento de computadoras, y la segunda para los físicos que quieren triturar números. Como consecuencia de estas diferentes redes e instalaciones, son fáciles de imaginar las siguientes situaciones:

1. LAN-LAN: Un informático descargando archivos para ingeniería.
2. LAN-WAN: Un informático enviando correo a un físico lejano.
3. WAN-WAN: Dos poetas intercambiando sonetos.
4. LAN-WAN-LAN: Ingenieros de diferentes universidades comunicándose.

En la figura 5-33 se ilustran estos cuatro tipos de conexiones como líneas punteadas. En cada caso, es necesario introducir una "caja negra" en las uniones entre dos redes, para manejar las conversiones necesarias a medida que los paquetes se mueven de una red a otra.

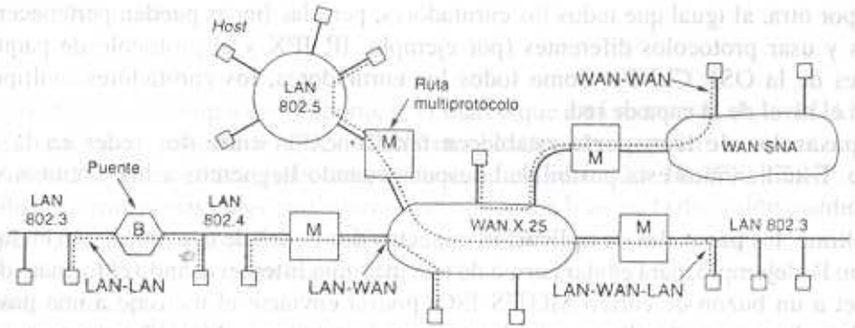


Figura 5-33. Interconexión de redes.

El nombre usado para la caja negra que conecta las dos redes depende de la capa que se encarga del trabajo. A continuación se indican algunos nombres comunes (aunque no hay mucho acuerdo en cuanto a la terminología de esta área).

Capa 1: Los repetidores copian bits individuales entre segmentos de cable.

Capa 2: Los puentes almacenan y reenvían marcos de enlace de datos entre las LAN.

Capa 3: Los enrutadores multiprotocolo reenvían paquetes entre redes disímiles.

Capa 4: Las pasarelas de transporte conectan corrientes de bytes en la capa de transporte.

Arriba de 4: Las pasarelas de aplicación permiten el trabajo conjunto por encima de la capa 4.

Por conveniencia, a veces usaremos el término "pasarela" (*gateway*) para indicar cualquier dispositivo que conecta dos o más redes disímiles.

Los **repetidores** son dispositivos de bajo nivel que simplemente amplifican o regeneran señales débiles. Se necesitan para proporcionar corriente que impulse la señal en cables largos. En el 802.3, por ejemplo, las propiedades de temporización del protocolo MAC (el valor de τ escogido) permiten cables de hasta 2.5 km, pero los *chips* del transceptor sólo pueden proporcionar potencia suficiente para llegar a 500 metros. La solución es usar repetidores para extender la longitud del cable donde se desea.

A diferencia de los repetidores, que copian los bits a medida que llegan, los **puentes** son dispositivos que almacenan y reenvían. Los puentes aceptan un marco entero y lo pasan a la capa de enlace de datos donde se verifica la suma de comprobación. Entonces el marco se envía a la capa física para reenviarlo por otra red. Los puentes pueden hacer cambios menores al marco antes de reenviarlo, como agregar o quitar algunos campos de la cabecera del marco. Puesto que los puentes son dispositivos de enlace de datos, no tienen tratos con las cabeceras de la capa 3 y superiores, y no pueden hacer cambios ni tomar decisiones que dependan de ellas.

Conceptualmente, los **enrutadores multiprotocolo** son parecidos a los puentes, excepto que se encuentran en la capa de red; simplemente toman paquetes de entrada de una línea y los reenvían por otra, al igual que todos los enrutadores, pero las líneas pueden pertenecer a redes diferentes y usar protocolos diferentes (por ejemplo, IP, IPX y el protocolo de paquetes sin conexiones de la OSI, CLNP). Como todos los enrutadores, los enrutadores multiprotocolo operan en el nivel de la capa de red.

Las **pasarelas de transporte** establecen una conexión entre dos redes en la capa de transporte. Estudiaremos esta posibilidad después cuando lleguemos a los circuitos virtuales concatenados.

Por último, las **pasarelas de aplicación** conectan dos partes de una aplicación en la capa de aplicación. Por ejemplo, para enviar correo de una máquina Internet usando el formato de correo de Internet a un buzón de correo MOTIS ISO, podría enviarse el mensaje a una pasarela de correo. Ésta desempacaría el mensaje, lo convertiría a formato MOTIS y lo reenviaría a la segunda red usando los protocolos de red y de transporte ahí empleados.

Cuando una pasarela está entre dos WAN operadas por organizaciones diferentes, posiblemente en países diferentes, la operación conjunta con una máquina clase estación de trabajo puede conducir a muchos desacuerdos y discusiones. Para eliminar estos problemas, puede usarse un enfoque un tanto diferente. La pasarela se divide por la mitad y se conectan ambas partes mediante un alambre. Cada una de las mitades se llama **media pasarela**, y cada una pertenece a, y es operada por, uno de los operadores de la red. El problema general del uso de pasarelas se reduce entonces a lograr un acuerdo sobre un protocolo común a usar en el alambre, uno que sea neutral y que no favorezca a ninguna parte. En la figura 5-34 se muestra tanto una pasarela completa como una media pasarela. Cualquiera de los dos tipos puede usarse en cualquiera de las capas (por ejemplo, también existen los medios puentes).

Dicho todo esto, la situación es más turbia en la práctica de lo que es en teoría. Muchos dispositivos comerciales combinan la funcionalidad de puente y de enrutador. La propiedad clave de un puente puro es que examina las cabeceras de los marcos de capa de enlace de datos y no inspecciona ni modifica los paquetes de capa de red que forman parte de los marcos. Un

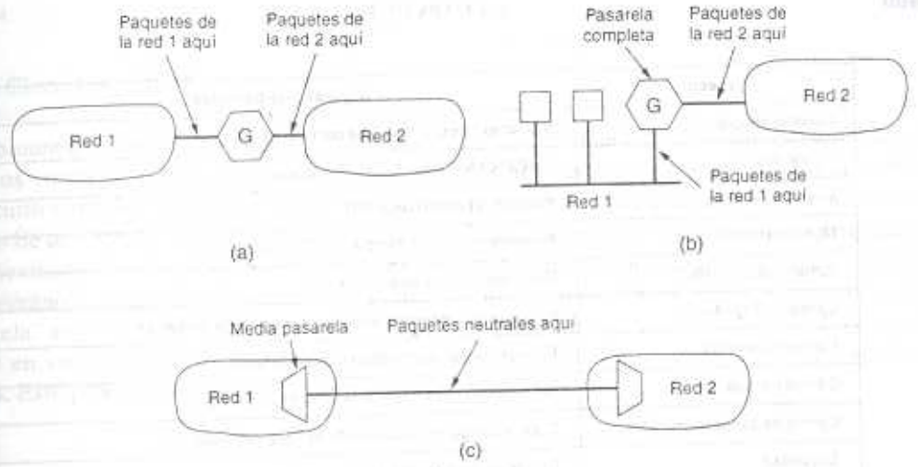


Figura 5-34. (a) Pasarela completa entre dos WAN. (b) Compasarela completa entre una LAN y una WAN. (c) Dos medias pasarelas.

puente no distingue, ni tampoco le importa, si el marco que está reenviado de una LAN 802.x a una 802.y contiene un paquete IP, IPX o CLNP en el campo de carga útil.

Un enrutador, en contraste, sabe muy bien si es un enrutador IP, IPX, CLNP o los tres combinados; examina estas cabeceras y toma decisiones con base en la dirección encontrada ahí. Por otra parte, cuando un enrutador puro entrega un paquete a la capa de enlace de datos, no sabe ni le importa si será llevado por un marco Ethernet o un marco *token ring*. Ésa es la función de la capa de enlace de datos.

La confusión en la industria viene de dos orígenes. Primero, desde el punto de vista funcional, los puentes y los enrutadores no son tan diferentes: aceptan las PDU (*Protocol Data Units*, unidades de datos de protocolo) de entrada, examinan algunos campos de cabecera, y toman decisiones sobre a dónde deben enviar las PDU, con base en información de las cabeceras y las tablas internas.

Segundo, muchos productos comerciales se venden con la etiqueta equivocada, o combinan la funcionalidad tanto de puentes como de enrutadores. Por ejemplo, los puentes de enrutamiento desde el origen no son realmente puentes, ya que requieren una capa de protocolo por encima de la capa de enlace de datos para hacer su trabajo. Si desea un análisis muy ilustrativo de los puentes y los enrutadores, vea el capítulo 12 de (Perlman, 1992).

5.4.1. Diferencias entre las redes

Las redes pueden diferir de muchas maneras. En la figura 5-35 listamos algunas de las diferencias que ocurren en la capa de red. La conciliación de estas diferencias es lo que hace más difícil el proceso de interconexión de redes que la operación con una sola red.

Aspecto	Algunas posibilidades
Servicio ofrecido	Sin conexiones, orientado a conexiones
Protocolos	IP, IPX, CLNP, AppleTalk, DECnet, etc.
Direccionamiento	Plano (802) o jerárquico (IP)
Multitransmisión	Presente o ausente (también multitransmisión)
Tamaño de paquete	Cada red tiene su propio máximo
Calidad del servicio	Puede estar presente o ausente; muchos tipos diferentes
Manejo de errores	Entrega confiable, ordenada y desordenada
Control de flujo	Ventana corrediza, control de tasa, otros o ninguno
Control de congestionamientos	Cubeta con goteo, paquetes de estrangulamiento, etc.
Seguridad	Reglas de confidencialidad, cifrado, etc.
Parámetros	Diferentes terminaciones de temporización, especificaciones de flujo, etc.
Contabilidad	Por tiempo de conexión, por paquete, por byte, o sin ella

Figura 5-35. Algunas de las muchas maneras en que pueden diferir las redes.

Cuando los paquetes enviados por un origen en una red deben transitar a través de una o más redes foráneas antes de llegar a la red de destino (que también puede ser diferente de la red de origen), pueden ocurrir muchos problemas en las interfaces entre las redes. Para comenzar, cuando deben transitar paquetes de una red orientada a conexión a una sin conexiones, deben reordenarse, algo que el transmisor no espera y que el receptor no está preparado para manejar. Con frecuencia se necesitarán conversiones de protocolo, que pueden ser difíciles si la funcionalidad requerida no puede expresarse. También se necesitarán conversiones de direcciones, lo que podría requerir algún tipo de sistema de directorio. El paso de paquetes multitransmisión a través de una red que no reconoce la multitransmisión requiere la generación de paquetes individuales para cada destino.

Los diferentes tamaños máximos de paquete usados por las diferentes redes son un dolor de cabeza importante. ¿Cómo se pasa un paquete de 8000 bytes a través de una red cuyo tamaño máximo de paquete es de 1500 bytes? Es importante la diferencia en la calidad de servicio cuando un paquete que tiene restricciones de tiempo real pasa a través de una red que no ofrece garantías de tiempo real.

El control de errores, de flujo y de congestionamientos suelen ser diferentes entre las diferentes redes. Si tanto el origen como el destino esperan la entrega de paquetes en secuencia y sin errores, pero una red intermedia simplemente descarta paquetes cuando huele un congestionamiento en el horizonte, o los paquetes pueden vagar sin sentido durante un rato y emerger repentinamente para ser entregados, muchas aplicaciones fallarán. Existen diferentes mecanismos de seguridad, ajustes de parámetros y reglas de contabilidad, e incluso leyes de confidencialidad internacionales, que pueden causar problemas.

5.4.2. Circuitos virtuales concatenados

Son comunes dos estilos de interred: la concatenación orientada a conexión de subredes de circuitos virtuales, y los datagramas estilo Internet. Ahora estudiaremos ambos. En el modelo de circuito virtual concatenado, que se muestra en la figura 5-36, se establece una conexión con un *host* de una red distante de un modo parecido a la manera en que se establecen normalmente las conexiones. La subred ve que el destino es remoto y construye un circuito virtual al enrutador más cercano a la red de destino; luego construye un circuito virtual de ese enrutador a una "pasarela" externa (enrutador multiprotocolo). Esta pasarela registra la existencia del circuito virtual en sus tablas y procede a construir otro circuito virtual a un enrutador de la siguiente subred. Este proceso continúa hasta llegar al *host* de destino.

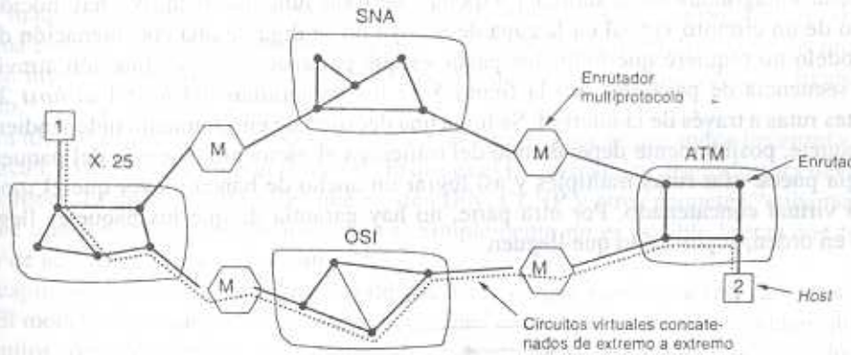


Figura 5-36. Interredes que usan circuitos virtuales concatenados.

Una vez que comienzan a fluir paquetes de datos por la trayectoria, cada pasarela releva los paquetes de entrada, haciendo las conversiones de formatos de paquete y números de circuito virtual según sea necesario. Obviamente, todos los paquetes de datos deben atravesar la misma secuencia de pasarelas y, por tanto, llegar en orden.

La característica esencial de este enfoque es que se establece una secuencia de circuitos virtuales desde el origen, a través de una o más pasarelas, hasta el destino. Cada pasarela mantiene tablas que indican los circuitos virtuales que pasan a través suyo, a dónde se deben enrutar y el número de circuito virtual nuevo.

Aunque en la figura 5-36 se muestra la conexión hecha con una pasarela completa, también podría hacerse con medias pasarelas.

Este esquema funciona mejor cuando todas las redes tienen aproximadamente las mismas propiedades. Por ejemplo, si todas ellas garantizan la entrega confiable de paquetes de capa de red, entonces, salvo una caída a lo largo de la ruta, el flujo del origen al destino también será confiable. De igual modo, si ninguna de ellas garantiza la entrega confiable, entonces la con-

catenación de los circuitos virtuales tampoco será confiable. Por otra parte, si la máquina de origen está en una red que garantiza la entrega confiable, pero una de las redes intermedias puede perder paquetes, la concatenación habrá cambiado fundamentalmente la naturaleza del servicio.

Los circuitos virtuales concatenados también son comunes en la capa de transporte. En particular, es posible construir un conducto de bits usando, digamos, OSI, que termine en una pasarela, y luego tener una conexión TCP entre esa pasarela y la siguiente. De este modo, puede construirse un circuito virtual de extremo a extremo abarcando diferentes redes y protocolos.

5.4.3. Interredes sin conexiones

El modelo alterno de interred es el modelo de datagramas, mostrado en la figura 5-37. En este modelo, el único servicio que ofrece la capa de red a la capa de transporte es la capacidad de inyectar datagramas en la subred y esperar que todo funcione bien. No hay noción en lo absoluto de un circuito virtual en la capa de red, ya no se diga de una concatenación de ellos. Este modelo no requiere que todos los paquetes que pertenecen a una conexión atraviesen la misma secuencia de pasarelas. En la figura 5-37 los datagramas del *host* 1 al *host* 2 toman diferentes rutas a través de la interred. Se toma una decisión de enrutamiento independiente para cada paquete, posiblemente dependiendo del tráfico en el momento de envío del paquete. Esta estrategia puede usar rutas múltiples y así lograr un ancho de banda mayor que el modelo de circuito virtual concatenado. Por otra parte, no hay garantía de que los paquetes llegarán al destino en orden, suponiendo que lleguen.

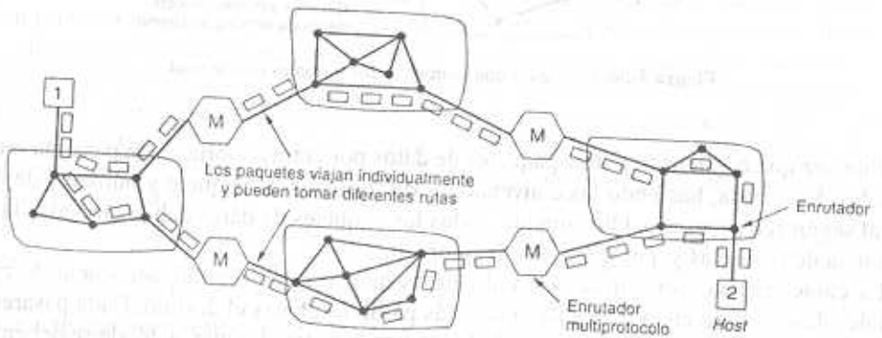


Figura 5-37. Interred sin conexiones.

El modelo de la figura 5-37 no es tan sencillo como parece. Por una parte, si cada red tiene su propio protocolo de capa de red, no es posible que un paquete de una red transite por otra. Podríamos imaginar a los enrutadores tratando de traducir de un formato a otro, pero a menos que los dos formatos sean parientes cercanos con los mismos campos de información, tales

conversiones siempre serán incompletas, y frecuentemente destinadas al fracaso. Por esta razón, pocas veces se intentan las conversiones.

Un segundo problema, más serio, es el direccionamiento. Imagínese un caso sencillo: un *host* de Internet está tratando de enviar un paquete IP a un *host* en un *host* OSI adyacente. El protocolo de datagramas de OSI, CLNP, se basó en el IP y es lo bastante parecido como para que una conversión pudiera funcionar bien. El problema es que todos los paquetes IP llevan la dirección Internet de 32 bits del *host* de destino en un campo de cabecera. Los *hosts* OSI no tienen direcciones de Internet de 32 bits; usan direcciones decimales parecidas a los números telefónicos.

Para hacer posible la conversión entre formatos en el enrutador multiprotocolo, alguien tendría que asignar una dirección de Internet de 32 bits a cada *host* OSI. Llevado al límite, este enfoque significaría la asignación de una dirección de Internet a cada máquina del mundo a la que pudiera querer hablar un *host* de Internet. También significaría asignar una dirección OSI a cada máquina del mundo a la que pudiera querer hablar una *host* OSI. Ocurre el mismo problema con todos los demás espacios de direcciones (SNA, AppleTalk, etc.). Los problemas aquí son insuperables. Además, alguien tendría que mantener una base de datos ligando todo con todo.

Otra idea es diseñar un paquete universal de "interred" y hacer que todos los enrutadores lo reconozcan. Este enfoque es, precisamente, lo que es el IP: un paquete diseñado para llevarse por muchas redes. El único problema es que existen IPX, CLNP y otros paquetes "universales", lo que hace que todos sean menos universales. Simplemente no es posible lograr que todos se pongan de acuerdo en un solo formato.

Recapitulemos ahora brevemente las dos maneras en que puede atacarse la interacción de redes. El modelo de circuito virtual concatenado tiene en esencia las mismas ventajas que el uso de circuitos virtuales en una sola subred: pueden reservarse *buffers* por adelantado, puede garantizarse la secuencia, pueden usarse cabeceras cortas y pueden evitarse los problemas causados por paquetes duplicados retrasados.

El modelo también tiene las mismas desventajas: el espacio de tablas requerido en los enrutadores para cada conexión abierta, la falta de enrutamiento alterno para evitar áreas congestionadas y la vulnerabilidad a fallas de los enrutadores en la trayectoria; también tiene la desventaja de ser difícil, si no imposible, su implementación si una de las redes que intervienen es una red no confiable de datagramas.

Las propiedades del enfoque por datagramas para interconectar redes son las mismas que las de las subredes de datagramas: un mayor potencial de congestionamiento, pero también mayor potencial de adaptarse a él, la robustez ante fallas de los enrutadores y la necesidad de cabeceras más grandes. En una interred son posibles varios algoritmos de enrutamiento adaptable, igual que en una sola red de datagramas.

Una ventaja principal del enfoque por datagramas para la interconexión de redes es que puede usarse en subredes que no usan circuitos virtuales. Muchas LAN, redes móviles (por ejemplo, flotas aéreas y navales) e incluso algunas WAN caen en esta categoría. Cuando una interred incluye una de éstas, surgen serios problemas si la estrategia de interredes se basa en circuitos virtuales.

5.4.4. Proceso de túnel

El manejo del caso general de lograr la interacción de dos redes diferentes es extremadamente difícil. Sin embargo, hay un caso especial común que puede manejarse. Este caso es cuando el *host* de origen y el de destino están en la misma clase de red, pero hay una red diferente en medio. Como ejemplo, piense en un banco internacional con una Ethernet basada en TCP/IP en París, una Ethernet basada en TCP/IP en Londres y una WAN PTT en medio, como se muestra en la figura 5-38.

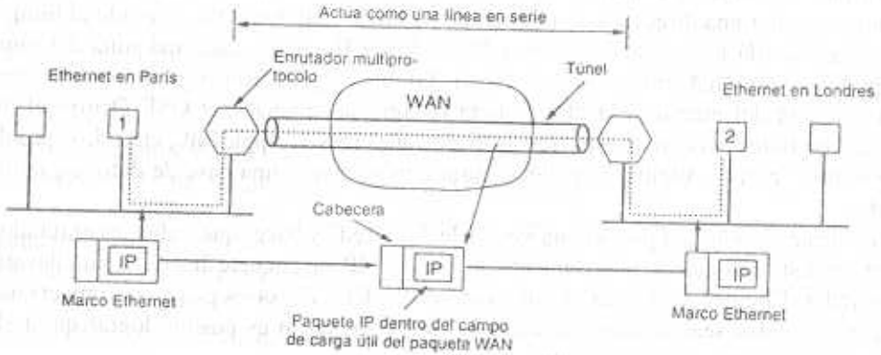


Figura 5-38. Proceso en túnel de un paquete de París a Londres.

La solución a este problema es una técnica llamada **proceso de túnel**. Para enviar un paquete IP al *host* 2, el *host* 1 construye el paquete que contiene la dirección IP del *host* 2, lo inserta en un marco Ethernet dirigido al enrutador multiprotocolo de París, y lo pone en el Ethernet. Cuando el enrutador multiprotocolo recibe el marco, retira el paquete IP, lo inserta en el campo de carga útil del paquete de capa de red de la WAN, y dirige este último a la dirección de la WAN del enrutador multiprotocolo de Londres. Al llegar ahí, el enrutador de Londres retira el paquete IP y lo envía al *host* 2 en un marco Ethernet.

La WAN pude visualizarse como un gran túnel que se extiende de un enrutador multiprotocolo al otro. El paquete IP simplemente viaja de un extremo del túnel al otro, bien acomodado en una caja bonita. No tiene que preocuparse por lidiar con la WAN. Tampoco tienen que hacerlo los *hosts* de cualquiera de los Ethernet. Sólo el enrutador multiprotocolo tiene que entender los paquetes IP y WAN. De hecho, la distancia completa entre la mitad de un enrutador multiprotocolo y la mitad del otro actúa como una línea en serie.

El proceso de túnel puede aclararse mediante una analogía. Considere una persona que maneja su auto de París a Londres. En Francia, el auto se mueve con su propia energía, pero al llegar al Canal de la Mancha, se carga en un tren de alta velocidad y se transporta a Inglaterra a través del Chunnel (los autos no pueden conducirse a través del Chunnel). En efecto, el auto se

transporta como carga, como se muestra en la figura 5-39. En el otro extremo, se libera el auto en las carreteras inglesas y nuevamente continúa moviéndose con sus propios medios. El proceso de túnel de paquetes a través de una red foránea funciona de la misma manera.

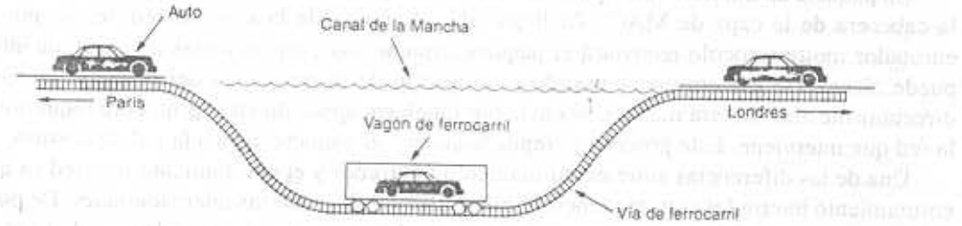


Figura 5-39. Paso por túnel de un auto de Francia a Inglaterra.

5.4.5. Enrutamiento por interred

El enrutamiento a través de una interred es parecido al enrutamiento en una sola subred, pero con algunas complicaciones adicionales. Considere, por ejemplo, la interred de la figura 5-40(a) en la que cinco redes están conectadas mediante seis enrutadores multiprotocolo. Un modelo de grafo de esta situación se complica por el hecho de que cada enrutador multiprotocolo puede acceder (es decir, enviar paquetes) directamente a todos los demás enrutadores conectados a cualquier red a la que esté conectado. Por ejemplo, *B* en la figura 5-40(a) puede acceder directamente a *A* y a *C* a través de la red 2, y también a *D* a través de la red 3. Esto conduce al grafo de la figura 5-40(b).

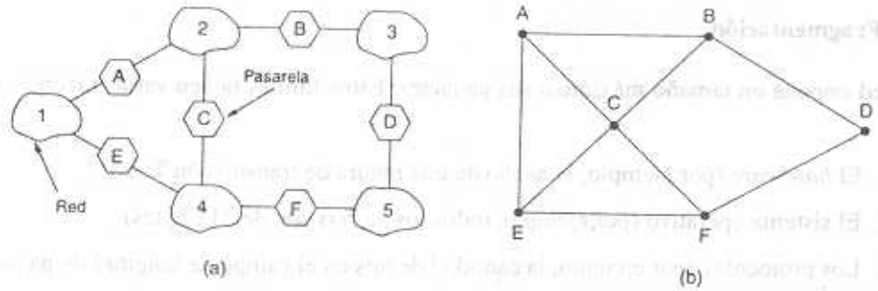


Figura 5-40. (a) Interred. (b) Grafo de la interred.

Una vez construido el grafo, pueden aplicarse algoritmos de enrutamiento conocidos, como el algoritmo de vector de distancia y el de estado de enlace, al grupo de enrutadores multiprotocolo. Esto da un algoritmo de enrutamiento de dos niveles: en cada red se usa un **protocolo de pasarela interior**, pero entre las redes se usa un **protocolo de pasarela exterior** ("pasarela es

un término antiguo para "enrutador"). De hecho, ya que cada red es independiente, pueden usar algoritmos diferentes. Dado que cada red de una interred es independiente de las demás, con frecuencia se denomina **sistema autónomo** (AS, *Autonomous System*).

Un paquete de interred típico parte de su LAN dirigido al enrutador multiprotocolo local (en la cabecera de la capa de MAC). Al llegar ahí, el código de la capa de red decide por cuál enrutador multiprotocolo reenviará el paquete, usando sus propias tablas de enrutamiento. Si puede alcanzarse ese enrutador usando el protocolo de la red nativa del paquete, se reenvía directamente ahí. De otra manera, se envía por túnel, encapsulado en el protocolo requerido por la red que interviene. Este proceso se repite hasta que el paquete llega a la red de destino.

Una de las diferencias entre el enrutamiento interredes y el enrutamiento intrared es que el enrutamiento interredes con frecuencia requiere el cruce de fronteras internacionales. De pronto, entran en escena varias leyes, como las estrictas leyes suecas de confidencialidad sobre la exportación de datos personales de ciudadanos suecos. Otro ejemplo es la ley canadiense que indica que el tráfico de datos que se origina en Canadá y llega a un destino en Canadá no puede dejar el país. Esto significa que el tráfico de Windsor, Ontario a Vancouver no puede enrutarse a través de Detroit, Estados Unidos, que está cerca.

Otra diferencia entre el enrutamiento interior y el exterior es el costo. Dentro de una sola red, normalmente se aplica un solo algoritmo de cargo. Sin embargo, redes diferentes pueden estar bajo administraciones diferentes, y una ruta puede ser menos cara que otra. Del mismo modo, la calidad de servicio ofrecida por diferentes redes puede ser distinta, y ésta puede ser una razón para escoger una ruta y no otra.

En una interred grande, la selección de la mejor ruta puede ser una operación muy tardada. Estrin *et al.* (1992) han propuesto manejar este problema precalculando rutas para pares (origen, destino) comunes y almacenándolas en una base de datos que se consulta en el momento de seleccionarse la ruta.

5.4.6. Fragmentación

Cada red impone un tamaño máximo a sus paquetes. Estos límites tienen varias razones, entre ellas:

1. El *hardware* (por ejemplo, el ancho de una ranura de transmisión TDM).
2. El sistema operativo (por ejemplo, todos los *buffers* son de 512 bytes).
3. Los protocolos (por ejemplo, la cantidad de bits en el campo de longitud de paquete).
4. El cumplimiento de algún estándar (inter)nacional.
5. El deseo de reducir hasta cierto nivel las retransmisiones inducidas por errores.
6. El deseo de evitar que un paquete ocupe el canal demasiado tiempo.

El resultado de estos factores es que los diseñadores de redes no están en libertad de escoger cualquier tamaño máximo de paquete que deseen. Las cargas útiles máximas van desde 48 bytes

(células ATM) hasta 65,515 bytes (paquetes IP), aunque el tamaño de la carga útil en las capas superiores con frecuencia es más grande.

Surge un problema obvio cuando un paquete grande quiere viajar a través de una red cuyo tamaño máximo de paquete es demasiado pequeño. Una solución es asegurar que no ocurra el problema. En otras palabras, la interred debe usar un algoritmo de enrutamiento que evite el envío de paquetes a través de redes que no pueden manejarlos. Sin embargo, ésta solución en realidad no es una solución. ¿Qué ocurre si el paquete original es demasiado grande para ser manejado por la red de destino? El algoritmo de enrutador no puede pasar por alto el destino.

Básicamente, la única solución al problema es permitir que las pasarelas dividan los paquetes en **fragmentos**, enviando cada paquete como paquete de interred individual. Sin embargo, como lo sabe cualquier padre de un niño pequeño, la conversión de un objeto grande en fragmentos pequeños es significativamente más fácil que el proceso inverso. (Los físicos inclusive le han dado un nombre a este efecto: segunda ley de la termodinámica.) Las redes de conmutación de paquetes también tienen problemas al unir nuevamente los fragmentos.

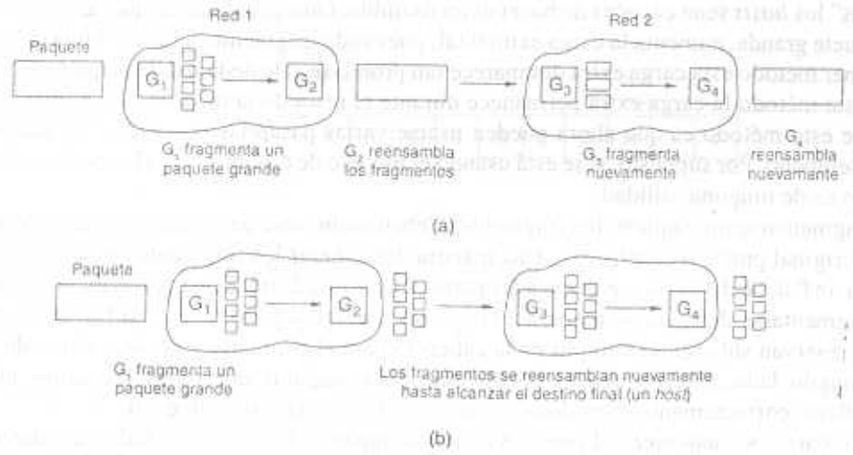


Figura 5-41. (a) Fragmentación transparente. (b) Fragmentación no transparente.

Existen dos estrategias opuestas para recombinar los fragmentos y recuperar el paquete original. La primera es hacer transparente la fragmentación causada por una red de "paquete pequeño" a las demás redes subsiguientes por las que debe pasar el paquete en el camino a su destino final. Esta opción se muestra en la figura 5-41(a). Al llegar un paquete de tamaño excesivo a la pasarela, ésta lo divide en fragmentos. Todos los fragmentos se dirigen a la misma pasarela de salida, donde se recombinan las piezas. De esta manera se ha hecho transparente el paso a través de la red de paquete pequeño. Las redes subsiguientes ni siquiera se enteran de que ha ocurrido una fragmentación. Las redes ATM, por ejemplo, tienen *hardware* especial para

proporcionar fragmentación transparente de paquetes en células y luego reensamblar las células en paquetes. En el mundo ATM, a la fragmentación se le llama segmentación; el concepto es el mismo, pero algunos de los detalles son diferentes.

La fragmentación transparente es sencilla, pero tiene algunos problemas. Por una parte, la pasarela de salida debe saber cuándo ha recibido todas las piezas, por lo que debe incluirse un campo de conteo o un bit de "fin de paquete" en cada paquete. Por otra parte, todos los paquetes deben salir por la misma pasarela. Al no permitir que algunos fragmentos sigan una ruta al destino final, y otros fragmentos una ruta distinta, puede bajar un poco el desempeño. Un último problema es la carga extra requerida para reensamblar y volver a fragmentar repetidamente un paquete grande que pasa a través de una serie de redes de paquete pequeño.

La otra estrategia de fragmentación es abstenerse de recombinar los fragmentos en las pasarelas intermedias. Una vez que se ha fragmentado un paquete, cada fragmento se trata como si fuera un paquete original. Todos los fragmentos pasan a través de la pasarela (o pasarelas) de salida, como se muestra en la figura 5-41(b). La recombinación ocurre sólo en el *host* de destino.

La fragmentación no transparente también tiene algunos problemas. Por ejemplo, requiere que "todos" los *hosts* sean capaces de hacer el reensamble. Otro problema es que, al fragmentarse un paquete grande, aumenta la carga extra total, pues cada fragmento debe tener una cabecera. En el primer método esta carga extra desaparece tan pronto se sale de la red de paquete pequeño, pero en este método la carga extra permanece durante el resto de la travesía. Sin embargo, una ventaja de este método es que ahora pueden usarse varias pasarelas de salida, lográndose un mejor desempeño. Por supuesto, si se está usando el modelo de circuito virtual concatenado, esta ventaja no es de ninguna utilidad.

Al fragmentarse un paquete, los fragmentos deben numerarse de tal manera que la corriente de datos original pueda reconstruirse. Una manera de numerar los fragmentos es usar un árbol. Si el paquete 0 debe dividirse, se llama a las partes 0.0, 0.1, 0.2, etc. Si estos fragmentos mismos deben fragmentarse después, se numera a las piezas 0.0.0, 0.0.1, 0.0.2, . . . , 0.1.0, 0.1.1, 0.1.2, etc. Si se reservan suficientes campos en la cabecera para el peor caso y no se generan duplicados en ningún lado, este esquema es suficiente para asegurar que todas las partes puedan reensamblarse correctamente en el destino, sin importar en qué orden lleguen.

Sin embargo, si cualquier red pierde o descarta paquetes, hay la necesidad de retransmisiones de extremo a extremo, con efectos poco afortunados para el sistema de numeración. Supóngase que se fragmenta inicialmente un paquete de 1024 bits en cuatro fragmentos del mismo tamaño, 0.0, 0.1, 0.2 y 0.3. Se pierde el fragmento 0.1, pero las otras partes llegan al destino. En algún momento termina la temporización del origen y se retransmite el paquete original, pero esta vez la ruta tomada pasa a través de una red con un límite de 512 bits, por lo que se generan dos fragmentos. Al llegar el nuevo fragmento 0.1 al destino, el receptor pensará que ya se han recibido las cuatro partes y reconstruirá incorrectamente el paquete.

Un sistema de numeración completamente diferente, y mejor, es que el protocolo de interred defina un tamaño de fragmento elemental lo bastante pequeño como para que el fragmento elemental pueda pasar a través de todas las redes. Al fragmentarse un paquete, todas las partes

son iguales al tamaño de fragmento elemental, excepto la última, que puede ser más corta. Un paquete de interred puede contener varios fragmentos, por razones de eficiencia. La cabecera de interred debe proporcionar el número de paquete original y el número del (primer) fragmento elemental contenido en el paquete. Como siempre, también debe haber un bit que indique que el último fragmento elemental contenido en el paquete de interred es el último del paquete original.

Este enfoque requiere dos campos de secuencia en la cabecera de interred: el número original de paquete y el número de fragmento. Evidentemente hay una concesión entre el tamaño del fragmento elemental y la cantidad de bits en el número de fragmento. Dado que se supone que el tamaño del fragmento elemental es aceptable para todas las redes, la fragmentación subsiguiente de un paquete de interred que contiene varios fragmentos no produce problemas. El límite último aquí es hacer que el fragmento elemental sea un solo bit o byte, siendo entonces el número de fragmento el desplazamiento del bit o byte en el paquete original, como se muestra en la figura 5-42.

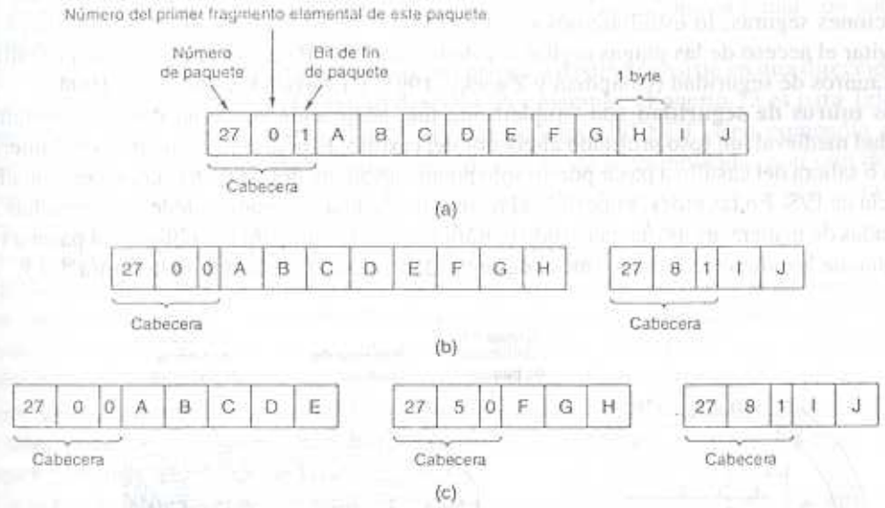


Figura 5-42. Fragmentación cuando el tamaño de datos elemental es de 1 byte. (a) Paquete original, que contiene 10 bytes de datos. (b) Fragmentos tras pasar a través de una red con un tamaño máximo de paquete de 8 bytes. (c) Fragmentos tras pasar a través de una pasarela de tamaño 5.

Algunos protocolos de interred llevan este método aún más lejos y consideran a toda la transmisión a través de un circuito virtual como un paquete gigantesco, por lo que cada fragmento contiene el número absoluto de byte del primer byte del fragmento. En (Kent y Mogul, 1987) se estudian algunos otros temas relacionados con la fragmentación.

5.4.7. Muros de seguridad

La capacidad de conectar una computadora, donde sea, a otra computadora, también donde sea, es una bendición y una maldición. Para los individuos en casa, vagar por Internet es muy divertido. Para los administradores de seguridad de las corporaciones, es una pesadilla. La mayor parte de las compañías tiene grandes cantidades de información confidencial en línea (secretos industriales, planes de desarrollo de productos, estrategias de mercadeo, análisis financieros, etc.). La divulgación de esta información a un competidor podría tener consecuencias desastrosas.

Además del peligro de la fuga de información, también hay el peligro de la entrada de información no deseada. En particular, los virus, gusanos y otras plagas digitales (Kaufman *et al.*, 1995) pueden romper la seguridad, destruir información valiosa y desperdiciar grandes cantidades de tiempo del administrador tratando de poner en orden el desastre que dejan. Con frecuencia estas plagas son importadas por empleados descuidados que quieren jugar un nuevo y apasionante juego.

En consecuencia, se requieren mecanismos para mantener adentro los bits "buenos" y fuera los bits "malos". Un método es el cifrado. Este enfoque protege los datos en tránsito entre instalaciones seguras; lo estudiaremos en el capítulo 7. Sin embargo, el cifrado no hace nada para evitar el acceso de las plagas digitales y de los *hackers*. Para lograr esta meta, necesitamos ver los muros de seguridad (Chapman y Zwicky, 1995, y Cheswick y Bellovin, 1994).

Los **muros de seguridad** son simplemente una adaptación moderna del viejo sistema de seguridad medieval: un foso profundo alrededor del castillo. Este diseño obligaba a cualquiera que entrara o saliera del castillo a pasar por un solo puente levadizo, donde podía ser inspeccionado por la policía de E/S. En las redes, es posible el mismo truco: una compañía puede tener muchas LAN conectadas de manera arbitraria, pero todo el tráfico de o a la compañía es obligado a pasar a través de un puente levadizo electrónico (muro de seguridad), como se muestra en la figura 5-43.

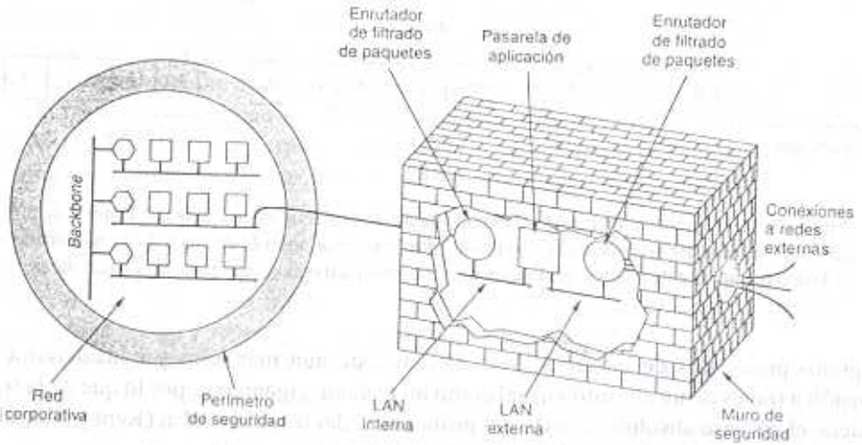


Figura 5-43. Muro de seguridad que consiste en dos filtros de paquetes y una pasarela de aplicación.

El muro de seguridad en esta configuración tiene dos componentes: dos enrutadores que filtran paquetes y una pasarela de aplicación. Existen configuraciones más sencillas, pero la ventaja de este diseño es que cada paquete debe transitar por dos filtros y una pasarela de aplicación para entrar o salir. No existe otra ruta. Los lectores que piensen que un solo punto de revisión es suficiente con toda seguridad no han tomado recientemente un vuelo internacional en una línea aérea comercial.

Cada **filtro de paquetes** es un enrutador estándar equipado con cierta funcionalidad extra, la cual permite la inspección de cada paquete de entrada o salida. Los paquetes que satisfacen algún criterio se reenvían normalmente. Los que fallan la prueba se descartan.

En la figura 5-43, lo más común es que el filtro de paquetes de la LAN interior revise los paquetes de salida, y el de la LAN exterior revise los paquetes de entrada. Los paquetes que saltan la primera valla pasan a la pasarela de aplicación para otra revisión. El objetivo de poner los dos filtros de paquetes en LAN diferentes es asegurar que ningún paquete entre o salga sin tener que pasar a través de la pasarela de aplicación; no hay ningún camino alrededor de ella.

Los filtros de paquetes comúnmente son manejados por tablas configuradas por el administrador del sistema. En estas tablas se listan los orígenes y los destinos aceptables, los orígenes y los destinos bloqueados, y las reglas predeterminadas sobre las acciones a tomar con paquetes que llegan de, o salen a, otras máquinas.

En el caso común de una instalación UNIX, un origen o destino consiste en una dirección IP y un puerto. Los puertos indican el servicio deseado. Por ejemplo, el puerto 23 es para Telnet, el puerto 79 es para Finger y el puerto 119 es para noticias USENET. Una compañía podría bloquear los paquetes de entrada de todas las direcciones de IP combinadas con uno de estos puertos. De esta manera, nadie fuera de la compañía podría establecer una sesión a través de Telnet, o buscar personas usando el *daemon* Finger. Es más, la compañía evitaría que sus usuarios pasaran todo el día leyendo las noticias de USENET.

El bloqueo de paquetes de salida es más difícil porque, aunque la mayoría de las instalaciones se apegan a las convenciones estándar de nombres de puertos, no están obligadas a hacerlo. Además, en algunos servicios importantes, como FTP (*File Transfer Protocol*, protocolo de transferencia de archivos), los números de puerto se asignan dinámicamente. Además, aunque el bloqueo de conexiones TCP es difícil, el bloqueo de paquetes UDP es aún más difícil porque se sabe muy poco de antemano sobre lo que harán. Muchos filtros de paquetes simplemente excluyen por completo el tráfico UDP.

La segunda mitad del mecanismo de muro de seguridad es la **pasarela de aplicación**. En lugar de simplemente ver paquetes en bruto, la pasarela opera en el nivel de aplicación. Por ejemplo, puede establecerse una pasarela de correo para examinar cada mensaje que entra o sale. Para cada uno, la pasarela decide si debe transmitirlo o descartarlo con base en los campos de cabecera, tamaño del mensaje e inclusive el contenido (por ejemplo, en una instalación militar, la presencia de palabras como "nuclear" o "bomba" podrían causar una acción especial).

Las instalaciones están en libertad de establecer una o más pasarelas de aplicación para aplicaciones específicas, pero no es poco común que las organizaciones suspicaces permitan el correo electrónico de entrada y de salida, y tal vez el uso de la *World Wide Web*, pero prohíban cualquier

otra cosa como demasiado peligrosa. En combinación con el cifrado y el filtrado de paquetes, este arreglo ofrece una cantidad limitada de seguridad al precio de algunos inconvenientes.

Una nota final se refiere a la comunicación inalámbrica y a los muros de seguridad. Es fácil diseñar un sistema que, desde el punto de vista lógico, sea completamente seguro pero que, en la práctica, tenga tantas fugas como una coladera. Esta situación puede ocurrir si algunas de las máquinas son inalámbricas y usan radiocomunicación, que pasa por encima del muro de seguridad en ambas direcciones.

5.5. LA CAPA DE RED EN INTERNET

En la capa de red, la Internet puede verse como un conjunto de subredes, o **sistemas autónomos** (AS, *Autonomous System*) interconectados. No hay una estructura real, pero existen varios *backbone* principales. Éstos se construyen a partir de líneas de alto ancho de banda y enrutadores rápidos. Conectadas a los *backbone* hay redes regionales (de nivel medio), y conectadas a estas redes regionales están las LAN de muchas universidades, compañías y proveedores de servicio Internet. En la figura 5-44 se presenta un dibujo de esta organización cuasijerárquica.

El pegamento que mantiene unida la Internet es el protocolo de capa de red, **IP** (*Internet Protocol*, **protocolo Internet**). A diferencia de la mayoría de los protocolos de capa de red anteriores, éste se diseñó desde el principio con la interconexión de redes en mente. Una buena manera de visualizar la capa de red es la siguiente. Su trabajo es proporcionar un medio de mejor

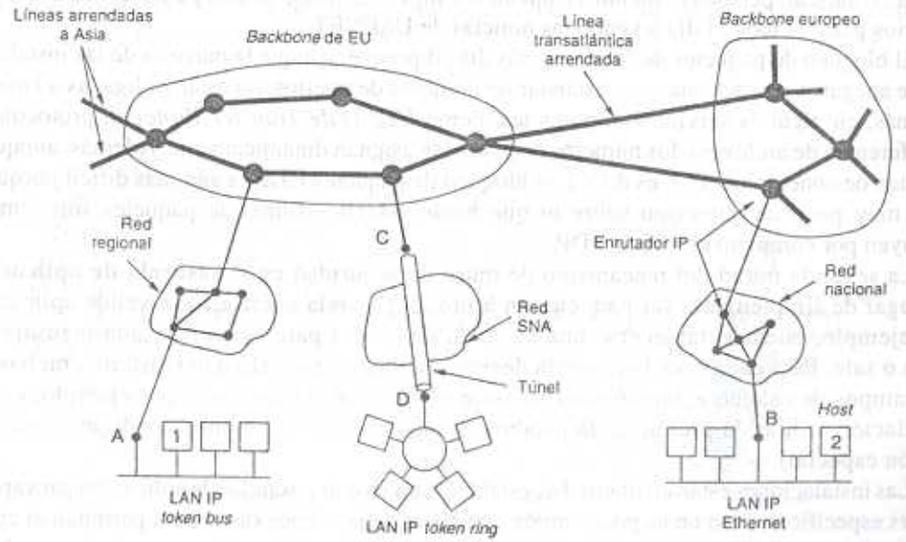


Figura 5-44. La Internet es un conjunto interconectado de muchas redes.

esfuerzo para el transporte de datagramas del origen al destino, sin importar si estas máquinas están en la misma red, o si hay otras redes entre ellas.

La comunicación en Internet funciona como sigue. La capa de transporte toma corrientes de datos y las divide en datagramas. En teoría, los datagramas pueden ser de hasta 64 Kbytes cada uno, pero en la práctica por lo general son de unos 1500 bytes. Cada datagrama se transmite a través de Internet, posiblemente fragmentándose en unidades más pequeñas en el camino. Cuando todas las piezas llegan finalmente a la máquina de destino, son reensambladas por la capa de red, dejando el datagrama original. Este datagrama entonces es entregado a la capa de transporte, que lo introduce en la corriente de entrada del proceso receptor.

5.5.1. El protocolo IP

Un lugar adecuado para comenzar nuestro estudio de la capa de red de Internet es el formato de los datagramas de IP mismos. Un datagrama IP consiste en una parte de cabecera y una parte de texto. La cabecera tiene una parte fija de 20 bytes y una parte opcional de longitud variable. El formato de la cabecera se muestra en la figura 5-45. Se transmite en orden *big endian*:* de izquierda a derecha, comenzando por el bit de orden mayor del campo de *versión*. (SPARC es *big endian*; Pentium es *little endian*.) En las máquinas *little endian*, se requiere conversión por *software* tanto para la transmisión como para la recepción.

El campo de *versión* lleva el registro de la versión del protocolo al que pertenece el datagrama. Al incluir la versión en cada datagrama es posible hacer que la transición entre versiones se lleve

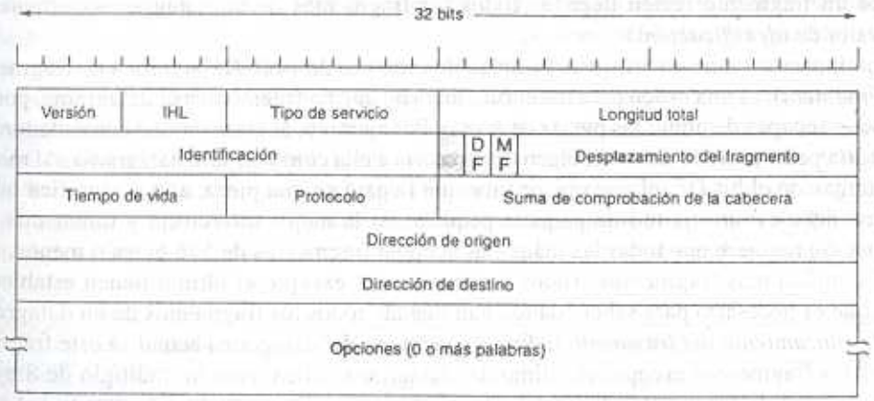


Figura 5-45. La cabecera de IP (protocolo de Internet).

* En *Los viajes de Gulliver*, de Jonathan Swift, las naciones de Lilliput y Blefescu estaban en guerra a causa de sus diferencias en cuanto a la forma correcta de comer huevos: rompiéndolos por el extremo angosto (*little end*) o por el más ancho (*big end*). En computación se aplican los términos *little endian* y *big endian* a tecnologías similares pero incompatibles que, a pesar de que no existe una razón de peso para preferir una a la otra, son defendidas con vehemencia por sus respectivos partidarios. (*N. del supervisor.*)

meses, o inclusive años, ejecutando algunas máquinas la versión vieja y otras la versión nueva.

Dado que la longitud de la cabecera no es constante, se incluye un campo en la cabecera, *IHL*, para indicar la longitud en palabras de 32 bits. El valor mínimo es de 5, cifra que aplica cuando no hay opciones. El valor máximo de este campo de 4 bits es de 15, lo que limita la cabecera a 60 bytes y, por tanto, el campo de opciones a 40 bytes. Para algunas opciones, por ejemplo para una que registre la ruta que ha seguido un paquete, 40 bytes es muy poco, lo que hace inútil esta opción.

El campo de *tipo de servicio* permite al *host* indicar a la subred el tipo de servicio que quiere. Son posibles varias combinaciones de confiabilidad y velocidad. Para voz digitalizada, la entrega rápida le gana a la entrega precisa. Para la transferencia de archivos, es más importante la transmisión libre de errores que la transmisión rápida.

El campo mismo contiene (de izquierda a derecha) un campo de *precedencia*; tres indicadores, *D*, *T* y *R*; y dos bits no usados. El campo de *precedencia* es una prioridad, de 0 (normal) a 7 (paquete de control de red). Los tres bits indicadores permiten al *host* especificar lo que le interesa más del grupo {retardo (*delay*), rendimiento (*throughput*), confiabilidad (*reliability*)}. En teoría, estos campos permiten a los enrutadores tomar decisiones entre, por ejemplo, un enlace satelital de alto rendimiento y alto retardo o una línea arrendada con bajo rendimiento y poco retardo. En la práctica, los enrutadores actuales ignoran por completo el campo de *tipo de servicio*.

La *longitud total* incluye todo el datagrama: tanto la cabecera como los datos. La longitud máxima es de 65,535 bytes. Actualmente este límite es tolerable, pero con las redes futuras de gigabits se requerirán datagramas más grandes.

El campo de *identificación* es necesario para que el *host* de destino determine a qué datagrama pertenece un fragmento recién llegado. Todos los fragmentos de un datagrama contienen el mismo valor de *identificación*.

A continuación viene un bit sin uso y luego dos campos de 1 bit. *DF* significa no fragmentar (*Don't Fragment*); es una orden para los enrutadores de que no fragmenten el datagrama, porque el destino es incapaz de juntar las piezas de nuevo. Por ejemplo, al arrancar una computadora, su ROM podría pedir el envío de una imagen de memoria a ella como un solo datagrama. Al marcar el datagrama con el bit *DF*, el transmisor sabe que llegará en una pieza, aún si significa que el datagrama debe evitar una red de paquete pequeño en la mejor trayectoria y tomar una ruta subóptima. Se requiere que todas las máquinas acepten fragmentos de 576 bytes o menos.

MF significa más fragmentos. Todos los fragmentos excepto el último tienen establecido este bit, que es necesario para saber cuándo han llegado todos los fragmentos de un datagrama.

El *desplazamiento del fragmento* indica en qué parte del datagrama actual va este fragmento. Todos los fragmentos excepto el último del datagrama deben tener un múltiplo de 8 bytes, que es la unidad de fragmento elemental. Dado que se proporcionan 13 bits, puede haber un máximo de 8192 fragmentos por datagrama, dando una longitud máxima de datagrama de 65,536 bytes, uno más que el campo de *longitud total*.

El campo de *tiempo de vida* es un contador que sirve para limitar la vida de un paquete. Se supone que este contador cuenta el tiempo en segundos, permitiendo una vida máxima de 255 seg; debe disminuirse en cada salto y se supone que disminuye muchas veces al encolarse durante un tiempo grande en un enrutador. En la práctica, simplemente cuenta los saltos. Cuando

el contador llega a cero, el paquete se descarta y se envía de regreso un paquete de aviso al *host* de origen. Esta característica evita que los datagramas vaguen eternamente, algo que de otra manera podría ocurrir si se llegan a corromper las tablas de enrutamiento.

Una vez que la capa de red ha ensamblado un datagrama completo, necesita saber qué hacer con él. El campo de *protocolo* indica la capa de transporte a la que debe entregarse. TCP es una posibilidad, pero también está UDP y algunos más. La numeración de los protocolos es global en toda la Internet, y se define en el RFC 1700.

La *suma de comprobación de la cabecera* verifica solamente la cabecera. Tal suma de comprobación es útil para la detección de errores generados por palabras de memoria erróneas en un enrutador. El algoritmo es sumar todas las medias palabras de 16 bits a medida que llegan, usando aritmética de complemento a uno, y luego obtener el complemento a uno del resultado. Para los fines de este algoritmo, se supone que la *suma de comprobación de la cabecera* es cero cuando llega. Este algoritmo es más robusto que una suma normal. Nótese que la *suma de comprobación de la cabecera* debe recalcularse en cada salto, pues cuando menos uno de los campos siempre cambia (el campo de *tiempo de vida*), pero pueden usarse trucos para acelerar el cálculo.

La *dirección de origen* y la *dirección de destino* indican el número de red y el número de *host*. Estudiaremos las direcciones de Internet en la siguiente sección. El campo de *opciones* se diseñó para proporcionar un recurso que permitiera que las versiones subsiguientes del protocolo incluyeran información no presente en el diseño original, para permitir a los experimentadores probar ideas nuevas y para evitar la asignación de bits de cabecera a información pocas veces necesaria. Las opciones son de longitud variable. Cada una empieza con un código de 1 byte que identifica la opción. Algunas opciones vienen seguidas de un campo de longitud de la opción de 1 byte, y luego de uno o más bytes de datos. El campo de *opciones* se rellena para completar múltiplos de cuatro bytes. Actualmente hay cinco opciones definidas, las que se listan en la figura 5-46, pero no todos los enrutadores reconocen a todas.

La opción de *seguridad* indica qué tan secreta es la información. En teoría, un enrutador militar puede usar este campo para especificar que no se enrute a través de ciertos países que los militares consideren "malos". En la práctica, todos los enrutadores lo ignoran, por lo que su única función real es la de ayudar a los espías a encontrar la información importante con mayor facilidad.

La opción de *enrutamiento estricto desde el origen* da la trayectoria completa desde el origen hasta el destino como secuencia de direcciones IP. Se requiere que el datagrama siga esa

Opción	Descripción
Seguridad	Especifica qué tan secreto es el datagrama
Enrutamiento estricto desde el origen	Indica la trayectoria completa a seguir
Enrutamiento libre desde el origen	Da una lista de los enrutadores que no deben evitarse
Registrar ruta	Hace que cada enrutador agregue su dirección de IP
Marca de tiempo	Hace que cada enrutador agregue su dirección y su marca de tiempo

Figura 5-46. Opciones del IP.

ruta exacta. Esta opción se usa sobre todo cuando los administradores de sistemas envíen paquetes de emergencia porque las tablas de enrutamiento se han corrompido, o para hacer mediciones de tiempo.

La opción de *enrutamiento libre desde el origen* requiere que el paquete pase por los enrutadores indicados en la lista, y en el orden especificado, pero se le permite pasar a través de otros enrutadores en el camino. Normalmente, esta opción sólo indicará algunos enrutadores, para obligar a una trayectoria en particular. Por ejemplo, si se desea obligar a un paquete de Londres a Sydney a ir hacia el oeste en lugar de hacia el este, esta opción podría especificar enrutadores en Nueva York, Los Ángeles y Honolulú. Esta opción es de mucha utilidad cuando las consideraciones políticas o económicas dictan pasar a través de, o evitar, ciertos países.

La opción de *registrar ruta* indica a los enrutadores a lo largo de la trayectoria que agreguen su dirección de IP al campo de opción. Esto permite a los administradores del sistema buscar fallas en los algoritmos de enrutamiento ("¿por qué todos los paquetes de Houston a Dallas pasan por Tokio primero?"). Al establecerse inicialmente ARPANET, ningún paquete pasaba nunca por más de nueve enrutadores, por lo que 40 bytes de opciones eran más que suficientes. Como se mencionó antes, ahora esto es demasiado poco.

Por último, la opción de *marca de tiempo* es como la opción de *registrar ruta*, excepto que además de registrar su dirección IP de 32 bits, cada enrutador también registra una marca de tiempo de 32 bits. Esta opción también es principalmente para búsqueda de fallas en los algoritmos de enrutamiento.

5.5.2. Direcciones IP

Cada *host* y enrutador de Internet tiene una dirección de IP, que codifica su número de red y su número de *host*. La combinación es única: no hay dos máquinas que tengan la misma dirección de IP. Todas las direcciones de IP son de 32 bits de longitud y se usan en los campos de *dirección de origen* y *dirección de destino* de los paquetes IP. Los formatos usados para las direcciones IP se muestran en la figura 5-47. Aquellas máquinas conectadas a varias redes tienen direcciones de IP diferentes en cada red.

Los formatos de clase A, B, C y D permiten hasta 126 redes con 16 millones de *hosts* cada una, 16,382 redes con hasta 64K *hosts*, 2 millones de redes (por ejemplo, LAN) de hasta 254 *hosts* cada una, y multitransmisión, en la cual se dirige un datagrama a múltiples *hosts*. Las direcciones que comienzan con 11110 se reservan para uso futuro. Hay decenas de miles de redes conectadas ahora a Internet, y la cifra se duplica cada año. Los números de red los asigna el **NIC** (*Network Information Center*, **centro de información de redes**) para evitar conflictos.

Las direcciones de red, que son números de 32 bits, generalmente se escriben en **notación decimal con puntos**. En este formato, cada uno de los 4 bytes se escribe en decimal, de 0 a 255. Por ejemplo, la dirección hexadecimal C0290614 se escribe como 192.41.6.20. La dirección de IP menor es 0.0.0.0 y la mayor 255.255.255.255.

Los valores 0 y −1 tienen significado especial, como se muestra en la figura 5-48. El valor 0 significa esta red o este *host*. El valor −1 se usa como dirección de difusión para indicar todos los *hosts* de la red indicada.

Figura 5-47. Formatos de dirección IP.

La dirección de IP 0.0.0.0 es usada por los *hosts* cuando están siendo arrancados, pero no se usa después. Las direcciones de IP con 0 como número de red se refieren a la red actual. Estas direcciones permiten que las máquinas se refieran a su propia red sin saber su número (pero tienen que saber su clase para saber cuántos 0 hay que incluir). La dirección que consiste solamente en unos permite la difusión en la red local, por lo común una LAN. Las direcciones con un número de red propio y solamente unos en el campo de *host* permite que las máquinas envíen paquetes de difusión a LAN distantes desde cualquier parte de Internet. Por último, todas las direcciones de la forma 127.xx.yy.zz se reservan para pruebas de realimentación. Los paquetes enviados a esa dirección no se colocan en el alambre; se procesan localmente y se tratan como paquetes de entrada. Esto permite que los paquetes se envíen a la red local sin que el transmisor conozca su número. Esta característica también se usa para la detección de fallas en el *software* de la red.

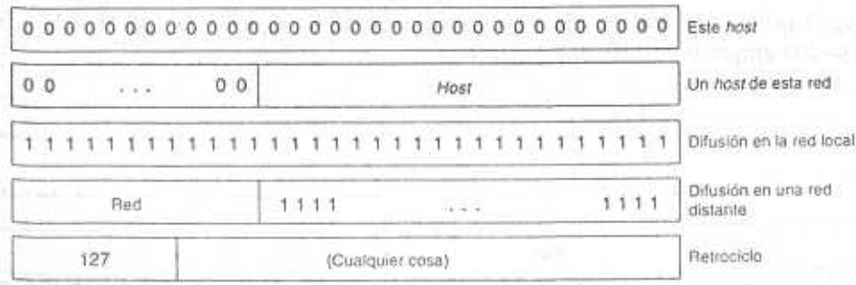


Figura 5-48. Direcciones especiales de IP.

5.5.3. Subredes

Como hemos visto, todos los *host* de una red deben tener el mismo número de red. Esta propiedad del direccionamiento IP puede causar problemas a medida que crecen las redes. Por ejemplo, considere una compañía que comienza con una LAN clase C en Internet. A medida que pasa el tiempo, podría adquirir más de 254 máquinas, y por tanto necesitar una segunda dirección clase C. De la misma manera, podría adquirir una segunda LAN de un tipo diferente y querer una dirección de IP diferente para ella (las LAN podrían estar unidas mediante un puente para formar una sola red IP, pero los puentes tienen sus propios problemas). Finalmente, podría terminar con muchas LAN, cada una con su propio enrutador y su propio número de red clase C.

Al crecer la cantidad de redes locales distintas, su administración puede volverse un dolor de cabeza grave. Cada vez que se instala una red nueva, el administrador del sistema tiene que comunicarse con el NIC para obtener un número de red nuevo. Después, este número debe anunciarse mundialmente. Además, mover una máquina de una LAN a otra requiere el cambio de su dirección de IP, lo que a su vez puede significar la modificación de sus archivos de configuración y también el anuncio de la nueva dirección de IP a todo el mundo. Si a otra máquina se le da la dirección de IP recién liberada, esa máquina recibirá el correo electrónico y otros datos dirigidos a la máquina original hasta que la dirección se haya propagado por todo el mundo.

La solución a este problema es permitir la división de una red en varias partes para uso interno, pero aún actuar como una sola red ante el mundo exterior. En la literatura sobre Internet, a estas partes se les llama **subredes**. Como mencionamos en el capítulo 1, este uso entra en conflicto con la "subred" cuyo significado es el grupo de todos los enrutadores y líneas de comunicación de una red. Esperamos que el contexto deje en claro el significado de que se trata. En esta sección usaremos la nueva definición. Si nuestra compañía en crecimiento tenía inicialmente una dirección clase B en lugar de clase C, puede comenzar simplemente por numerar los *hosts* de 1 a 254. Al llegar la segunda LAN podría decidir, por ejemplo, dividir el número de *host* de 16 bits en un número de subred de 6 bits y un número de *host* de 10 bits, como se muestra en la figura 5-49. Esta división permite 62 LAN (se reservan el 0 y el −1) cada una con hasta 1022 *hosts*.

Fuera de la red, la subred no es visible, por lo que la asignación de una subred nueva no requiere comunicación con el NIC ni la modificación de bases de datos externas. En este ejemplo, la primera subred podría usar direcciones de IP a partir de 130.50.4.1, la segunda subred podría empezar en 130.50.8.1, etcétera.

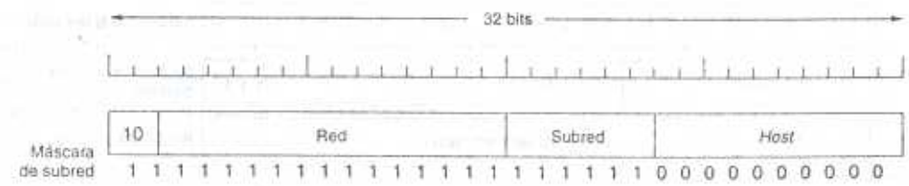


Figura 5-49. Una de las formas de generar una subred clase B.

Para ver el funcionamiento de las subredes, es necesario explicar la manera en que se procesan los paquetes IP en un enrutador. Cada enrutador tiene una tabla en la que se lista cierto número de direcciones IP (red, 0) y cierto número de direcciones IP (esta red, *host*). El primer tipo indica cómo llegar a redes distantes. El segundo tipo indica cómo llegar a redes locales. Asociada a cada tabla está la interfaz de red a usar para llegar al destino y cierta información adicional.

Al llegar un paquete de IP, se busca su dirección de destino en la tabla de enrutamiento. Si el paquete es para una red distante, se reenvía al siguiente enrutador de la interfaz dada en la tabla; si es para un *host* local (por ejemplo, en la LAN del enrutador), se envía directamente al destino. Si la red no está en la tabla, el paquete se reenvía a un enrutador predeterminado con tablas más extensas. Este algoritmo significa que cada enrutador sólo tiene que llevar el registro de otras redes y *hosts* locales, no de pares red-*host*, reduciendo en gran medida el tamaño de la tabla de enrutamiento.

Al introducirse subredes, se cambian las tablas de enrutamiento, agregando entradas con forma de (esta red, subred, 0) y (esta red, esta subred, *host*). Por tanto, un enrutador de la subred *k* sabe cómo llegar a todas las demás subredes y también cómo llegar a todos los *hosts* de la subred *k*; no tiene que saber los detalles sobre los *hosts* de otras subredes. De hecho, todo lo que se necesita es hacer que cada enrutador haga un AND booleano con la **máscara de subred** (véase la figura 5-49) para deshacerse del número de *host* y buscar la dirección resultante en sus tablas (tras determinar la clase de red de la que se trata). Por ejemplo, a un paquete dirigido a 130.50.15.6 que llega a un enrutador de la subred 5 se le hace un AND con la máscara de subred de la figura 5-49 para dar la dirección 130.50.12.0. Esta dirección se busca en las tablas de enrutamiento para averiguar la manera de llegar a los *hosts* de la subred 3. Así, el enrutador de la subred 5 se ahorra el trabajo de mantener un registro de las direcciones de enlace de datos de *hosts* que no pertenecen a la subred 5. El proceso de subred reduce así el espacio de tabla de enrutamiento creando una jerarquía de tres niveles.

5.5.4. Protocolos de control de Internet

Además del IP, que se usa para la transferencia de datos, Internet tiene varios protocolos de control que se usan en la capa de red, incluidos ICMP, ARP, RARP y BOOTP. En esta sección veremos cada uno de ellos.

Protocolo de control de mensajes de Internet

La operación de Internet es supervisada cuidadosamente por los enrutadores. Al ocurrir algo inesperado, el ICMP (*Internet Control Message Protocol*, **protocolo de control de mensajes de Internet**), que también se usa para probar Internet, informa del suceso. Se ha definido una docena de tipos de mensajes de ICMP; los más importantes se listan en la figura 5-50. Cada tipo de mensaje de ICMP se encapsula en un paquete IP.

El mensaje DESTINO INALCANZABLE (*DESTINATION UNREACHABLE*) se usa cuando la subred o un enrutador no pueden ubicar el destino, o un paquete con el bit *DF* no puede entregarse porque está en el camino una red de "paquete pequeño".

El mensaje de *tiempo excedido* (*TIME EXCEEDED*) se envía cuando un paquete se descarta debido a que su contador llega a cero. Este suceso es un síntoma de que los paquetes están en

Tipo de mensaje	Descripción
Destino inalcanzable	No pudo entregarse el paquete
Tiempo excedido	Campo de tiempo de vida llegó a cero
Problema de parámetro	Campo de cabecera no válido
Supresión de origen	Paquete de estrangulamiento
Reenvío	Enseña geografía a un enrutador
Solicitud de eco	Pregunta a una máquina si está viva
Respuesta de eco	Sí, estoy viva
Solicitud de marca de tiempo	Igual que la solicitud de eco, pero con marca de tiempo
Respuesta de marca de tiempo	Igual que la respuesta de eco, pero con marca de tiempo

Figura 5-50. Los principales tipos de mensajes ICMP.

ciclo, de que hay un congestionamiento enorme, o de que los valores de temporización son demasiado bajos.

El mensaje de PROBLEMA DE PARÁMETRO (*PARAMETER PROBLEM*) indica que se ha detectado un valor ilegal en un campo de cabecera. Este problema indica una falla en el *software* de IP del *host*, o posiblemente en el *software* de un enrutador transitado.

El mensaje SUPRESIÓN DE ORIGEN (*SOURCE QUENCH*) se usaba antes para controlar a los *hosts* que enviaban demasiados paquetes. Al recibir un *host* este mensaje, se esperaba que se refrenara. Este mensaje se usa poco en la actualidad porque, al ocurrir congestionamientos, estos paquetes tienden a echarle más leña al fuego. El control de congestionamiento de Internet se hace ahora en gran medida en la capa de transporte, y se estudiará en el capítulo 6.

El mensaje de REDIRECCIONAMIENTO (*REDIRECT*) se usa cuando un enrutador se da cuenta de que un paquete parece estar mal enrutado el enrutador lo usa para indicar al *host* transmisor el posible error.

Los mensajes de SOLICITUD DE ECO (*ECHO REQUEST*) y RESPUESTA DE ECHO (*ECHO REPLAY*) sirven para ver si un destino dado es alcanzable y está vivo. Al recibir el mensaje de ECO, se espera que el destino devuelva un mensaje de RESPUESTA DE ECO. Los mensajes SOLICITUD DE MARCA DE TIEMPO (*TIMESTAMP REQUEST*) y RESPUESTA DE MARCA DE TIEMPO (*TIMESTAMP REPLY*) son parecidos, excepto que el tiempo de llegada del mensaje y el tiempo de partida de la respuesta se registran en la respuesta. Este recurso se emplea para medir el desempeño de la red.

Además de estos mensajes, hay cuatro más que tienen que ver con el direccionamiento de Internet, para permitir que los *hosts* descubran sus números de red y para manejar el caso de varias LAN que comparten una sola dirección IP. El ICMP se define en el RFC 792.

Protocolo de resolución de direcciones

Aunque cada máquina de Internet tiene una (o más) direcciones de IP, éstas no pueden usarse para enviar paquetes porque el *hardware* de la capa de enlace de datos no entiende las direcciones de Internet. Hoy día, la mayoría de los *hosts* están conectados a alguna LAN mediante una tarjeta de interfaz que sólo entiende direcciones de LAN. Por ejemplo, todas las tarjetas Ethernet que se han fabricado vienen equipadas con una dirección de Ethernet de 48 bits. Los fabricantes de tarjetas Ethernet solicitan un bloque de direcciones de una autoridad central para asegurarse de que dos tarjetas no puedan tener la misma dirección (para evitar conflictos en caso de que las dos tarjetas aparezcan en la misma LAN). Las tarjetas envían y reciben marcos con base en direcciones de Ethernet de 48 bits. No saben nada sobre las direcciones de IP de 32 bits.

Ahora surge la pregunta: ¿cómo se proyectan las direcciones de IP en las direcciones de capa de enlace de datos, como las de Ethernet? Para explicar el funcionamiento de esto, usemos el ejemplo de la figura 5-51, en la que se ilustra una universidad pequeña con varias redes clase C. Aquí tenemos dos Ethernet, uno en el departamento de informática, con una dirección IP de 192.31.65.0, y otra en el de ingeniería electrónica, con dirección de IP de 192.31.63.0. Éstas se conectan mediante un anillo FDDI del campus con una dirección IP de 192.31.60.0. Cada máquina de un Ethernet tiene una dirección de Ethernet única, etiquetada *E1* a *E6*, y cada máquina del anillo FDDI tiene una dirección FDDI, etiquetada *F1* a *F3*.

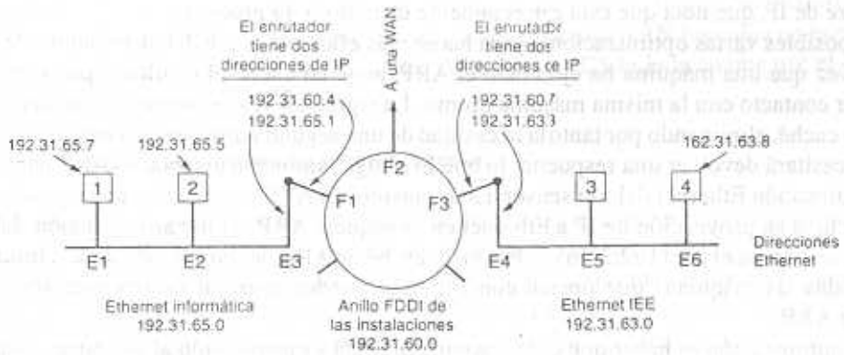


Figura 5-51. Tres redes clase C interconectadas: dos Ethernets y un anillo FDDI.

Comencemos por ver la manera en que un usuario del *host* 1 envía un paquete a un usuario del *host* 2. Supongamos que el transmisor sabe el nombre del receptor pretendido, posiblemente algo como *maria@eagle.cs.uni.edu*. El primer paso es encontrar la dirección IP del *host* 2, conocido como *eagle.cs.uni.edu*. Esta búsqueda la lleva a cabo el sistema de nombres de dominio (DNS, *Domain Name System*) que estudiaremos en el capítulo 7. Por lo pronto, simplemente supondremos que el DNS devuelve la dirección IP del *host* 2 (192.31.65.5).

El *software* de capa superior del *host* 1 construye ahora un paquete con 192.31.65.5 en el campo de *dirección de destino* y lo entrega al *software* de IP para su transmisión. El *software* de IP puede ver la dirección y saber si el destino está en su propia red, pero necesita una manera de encontrar la dirección Ethernet del destino. Una solución es tener un archivo de configuración en algún lugar del sistema que proyecte las direcciones IP en las direcciones Ethernet. Esta

solución ciertamente es posible pero, en organizaciones con miles de máquinas, el mantenimiento de estos archivos es una tarea susceptible a errores y con alto consumo de tiempo.

Una mejor solución es que el *host* 1 envíe un paquete de difusión por el Ethernet preguntando: "¿quién es el dueño de la dirección IP 192.31.65.5?" La difusión llegará a cada máquina del Ethernet 192.31.65.0, y cada una revisará su propia dirección IP. Sólo el *host* 2 responderá con su dirección Ethernet (*E2*). De esta manera, el *host* 1 aprende que la dirección de IP 192.31.65.5 está en el *host* con la dirección Ethernet *E2*. El protocolo para preguntar esto y obtener la respuesta se llama **ARP** (*Address Resolution Protocol*, **protocolo de resolución de direcciones**). Casi todas las máquinas de Internet lo ejecutan. Este protocolo se define en el RFC 826.

La ventaja del ARP sobre los archivos de configuración es la sencillez. El administrador del sistema no necesita hacer mucho, excepto asignar a cada máquina una dirección IP y decidir sobre las máscaras de subred. El ARP hace lo demás.

En este punto, el *software* de IP del *host* 1 construye un marco Ethernet dirigido a *E2*, pone el paquete IP (dirigido a 192.31.65.5) en el campo de carga útil, y lo arroja al Ethernet. La tarjeta de Ethernet del *host* 2 detecta este marco, lo reconoce como un marco para él mismo, lo recoge y causa una interrupción. El operador de Ethernet extrae el paquete IP de la carga útil y lo pasa al *software* de IP, que nota que está correctamente dirigido, y lo procesa.

Son posibles varias optimizaciones para hacer más eficiente el ARP. Por principio de cuentas, una vez que una máquina ha ejecutado el ARP, pone en caché el resultado por si necesita establecer contacto con la misma máquina pronto. La siguiente vez encontrará la proyección en su propio caché, eliminando por tanto la necesidad de una segunda difusión. En muchos casos, el *host* 2 necesitará devolver una respuesta, lo que lo obliga también a ejecutar el ARP para determinar la dirección Ethernet del transmisor. Esta transmisión ARP puede evitarse haciendo que el *host* 1 incluya su proyección de IP a Ethernet en el paquete ARP. Al llegar la difusión del ARP al *host* 2, se ingresa el par (192.31.65.7, E1) en el caché de ARP del *host* 2 para su uso futuro. De hecho, todas las máquinas que operan con Ethernet pueden ingresar esta proyección en sus cachés de ARP.

Otra optimización es hacer que cada máquina difunda su proyección al arranque. Esta difusión generalmente se hace en la forma de un ARP que busca su propia dirección de IP. No debe haber una respuesta, pero un efecto secundario de la difusión es crear una entrada en el caché de ARP de todos. Si llega una respuesta, es que dos máquinas tienen asignada la misma dirección de IP. La nueva debe informar de ello al administrador del sistema y no debe arrancar.

Para permitir el cambio de las proyecciones, por ejemplo, cuando se descompone una tarjeta Ethernet y se reemplaza por una nueva (y por tanto con una dirección Ethernet nueva), las entradas en el caché de ARP deben terminar su temporización la transcurrir algunos minutos.

Ahora veamos la figura 5-51 nuevamente, sólo que esta vez el *host* 1 quiere enviar un paquete al *host* 6 (192.31.63.8). El ARP fallará, porque el *host* 4 no verá la difusión (los enrutadores no reenvían las difusiones de nivel Ethernet). Hay dos soluciones. Primero, el enrutador CS podría configurarse para responder a las solicitudes de ARP para la red 192.31.63.0 (y posiblemente para otras redes locales). En este caso, el *host* 1 creará en el caché de ARP la entrada (192.31.63.8, E3) y felizmente enviará todo el tráfico para el *host* 4 al enrutador local. Esta

solución se llama **ARP apoderado** (*proxy* ARP). La segunda solución es hacer que el *host* 1 vea de inmediato que el destino está en una red remota y simplemente envíe todo ese tráfico a la dirección Ethernet predeterminada que maneja todo el tráfico remoto, en este caso *E3*. Esta solución no requiere hacer que el enrutador CS sepa a cuáles redes remotas sirve.

De cualquier manera, lo que ocurre es que el *host* 1 empaca el paquete IP en el campo de carga útil de un marco Ethernet dirigido a *E3*. Cuando el enrutador CS recibe el marco Ethernet, retira el paquete IP del campo de carga y busca la dirección IP en sus tablas de enrutamiento, descubriendo que los paquetes para la red 192.31.63.0 deben ir al enrutador 192.31.60.7. Si el enrutador no sabe ya la dirección FDDI de 192.31.60.7, difunde un paquete ARP por el anillo y se entera de que la dirección de su anillo es *F3*. El enrutador inserta entonces el paquete en el campo de carga útil de un marco FDDI dirigido a *F3* y lo pone en el anillo.

En el enrutador EE, el operador de FDDI retira el paquete del campo de carga útil y lo entrega al *software* de IP, el cual descubre que necesita enviar el paquete al 192.31.63.8. Si esta dirección no está en su caché de ARP, difunde una solicitud ARP por el Ethernet EE y se entera de que la dirección de destino es *E6*, por lo que construye un marco Ethernet dirigido a *E6*, pone el paquete en el campo de carga útil, y lo envía a través del Ethernet. Al llegar el marco Ethernet al *host* 4, el paquete se extrae del marco y se entrega al *software* de IP para su proceso.

Una transmisión del *host* 1 a una red distante a través de una WAN funciona en esencia de la misma manera, excepto que esta vez las tablas del enrutador CS le indican que use el enrutador WAN cuya dirección FDDI es *F2*.

Protocolo de resolución de direcciones en reversa

El ARP resuelve el problema de encontrar la dirección de Ethernet correspondiente a una dirección IP dada. A veces tiene que resolverse el problema inverso: dada una dirección Ethernet, ¿cuál es la dirección IP correspondiente? En particular, este problema ocurre al iniciarse una estación de trabajo sin disco. Tal máquina normalmente recibirá la imagen binaria de su sistema operativo de un servidor de archivos remoto, pero ¿cómo conoce su dirección de IP?

La solución es usar el **RARP** (*Reverse Address Resolution Protocol*, **protocolo de resolución de direcciones en reversa**, definido en el RFC 903). Este protocolo permite que una estación de trabajo recién iniciada difunda su dirección Ethernet y diga, "mi dirección Ethernet de 48 bits es 14.04.05.18.01.25. ¿Sabe alguien por ahí mi dirección de IP?" El servidor RARP ve esta solicitud, busca la dirección de Ethernet en sus archivos de configuración y envía de regreso la dirección de IP correspondiente.

El uso del RARP es preferible a integrar una dirección de IP en la imagen de memoria, pues permite usar la misma imagen para todas las máquinas. Si la dirección de IP estuviera incluido en la imagen, cada estación de trabajo necesitaría su propia imagen.

Una desventaja del RARP es que usa una dirección de destino que contiene únicamente unos (difusión limitada) para llegar al servidor RARP. Sin embargo, tales difusiones no son reenviadas por los enrutadores, por lo que se requiere un servidor RARP en cada red. Para superar este problema, se ha inventado un protocolo alterno de arranque llamado **BOOTP** (véanse los RFC

951, 1048 y 1084). A diferencia del RARP, BOOTP usa mensajes UDP, los cuales se reenvían a través de los enrutadores. Este protocolo también proporciona información adicional a una estación de trabajo sin disco, incluida la dirección IP del servidor de archivos que contiene la imagen de memoria, la dirección IP del enrutador predeterminado y la máscara de subred a usar. El BOOTP se describe en el RFC 951.

5.5.5. Protocolo de enrutamiento de pasarela interior: OSPF

Como mencionamos antes, la Internet se compone de una gran cantidad de sistemas autónomos (AS). Cada AS es operado por una organización diferente y puede usar internamente su propio algoritmo de enrutamiento. Por ejemplo, las redes internas de las compañías *X*, *Y* y *Z* generalmente se verían como tres AS si las tres estuvieran en Internet. Las tres pueden usar algoritmos de enrutamiento diferentes internamente. No obstante, la existencia de estándares, aun para enrutadores internos, simplifica la implementación en las líneas divisorias entre los AS y permite la reutilización de código. En esta sección estudiaremos el enrutamiento dentro de un AS; en el siguiente, veremos el enrutamiento entre varias AS. El algoritmo de enrutamiento interno de un AS se llama **protocolo de pasarela interior**; al algoritmo de enrutamiento entre varias AS se le llama **protocolo de pasarela exterior**.

El protocolo de pasarela interior original de Internet fue un protocolo de vector de distancia (RIP) basado en el algoritmo Bellman-Ford. Este protocolo funcionó bien en sistemas pequeños, pero menos bien a medida que los AS se volvieron más grandes; también padecía el problema de conteo a infinito y generalmente de una convergencia lenta, por lo que se reemplazó en mayo de 1979 por un protocolo de estado de enlace. En 1988, la *Internet Engineering Task Force* (grupo de trabajo de ingeniería de Internet) comenzó a trabajar en un sucesor. Ese sucesor, llamado **OSPF** (*Open Shortest Path First*, **abrir primero la trayectoria más corta**), se convirtió en estándar en 1990. Muchos fabricantes de enrutadores ya lo manejan, y se volverá el principal protocolo de pasarela interior en el futuro cercano. A continuación daremos una introducción al funcionamiento del OSPF. Para conocer la historia completa, véase el RFC 1247.

Dada la amplia experiencia con otros protocolos de enrutamiento, el grupo que diseñó el nuevo protocolo tenía una larga lista de requisitos a cumplir. Primero, el algoritmo tenía que publicarse como literatura abierta, de ahí la "O" (de *Open*) en OSPF. No serviría una solución patentada, que fuera propiedad de una compañía. Segundo, el nuevo protocolo tenía que reconocer una variedad de métricas de distancia, incluidas distancia física, retardo y otras. Tercero, tenía que ser un algoritmo dinámico, uno que se adaptara a los cambios de topología rápida y automáticamente.

Cuarto, algo nuevo para el OSPF, tenía que reconocer el enrutamiento basado en el tipo de servicio. El nuevo protocolo tenía que ser capaz de enrutar el tráfico de tiempo real de una manera y otros tipos de tráfico de otra manera. El protocolo IP tiene el campo de *tipo de servicio*, pero ningún protocolo de enrutamiento existente lo usaba.

Quinto, y relacionado con lo anterior, el nuevo protocolo tenía que efectuar equilibrio de cargas, dividiendo la carga entre varias líneas. La mayoría de los protocolos previos enviaban todos los paquetes a través de la mejor ruta. La segunda mejor ruta no se usaba en absoluto. En muchos casos, la división de la carga a través de varias líneas produce un mejor desempeño.

Sexto, se requería el reconocimiento de sistemas jerárquicos. En 1988, la Internet se había hecho tan grande que no se podía esperar que un solo enrutador conociera la topología completa. El nuevo protocolo de enrutamiento tenía que diseñarse de modo que ningún enrutador tuviera que conocerla.

Séptimo, se requería un mínimo de seguridad para evitar que los estudiantes ociosos burlaran a los enrutadores enviándoles información de enrutamiento falsa. Por último, se requería un mecanismo para manejar los enrutadores que se conectaran a Internet a través de un túnel. Los protocolos previos no manejaban bien esto.

El OSPF reconoce tres tipos de conexiones y redes:

1. Líneas punto a punto entre dos enrutadores (exactamente).
2. Redes multiacceso con difusión (por ejemplo, la mayoría de las LAN).
3. Redes multiacceso sin difusión (por ejemplo, la mayoría de las WAN de conmutación de paquetes).

Una red **multiacceso** es una que puede tener varios enrutadores, cada uno de los cuales puede comunicarse directamente con todos los demás. Todas las LAN y WAN tienen esta propiedad. En la figura 5-52(a) se muestra un AS que contiene los tres tipos de redes. Nótese que los *hosts* generalmente no desempeñan ningún papel en el OSPF.

El OSPF funciona haciendo una abstracción del conjunto de redes, enrutadores y líneas en un grafo dirigido en el que a cada arco se le asigna un costo (distancia, retardo, etc.). Entonces se calcula la trayectoria más corta con base en los pesos de los arcos. Una conexión en serie entre dos enrutadores se representa mediante un par de arcos, uno en cada dirección. Sus pesos pueden ser diferentes. Una red multiacceso se representa mediante un nodo para la red misma más un nodo para cada enrutador. Los arcos del nodo de red a los enrutadores tienen un peso de 0 y se omiten del grafo.

En la figura 5-52(b) se muestra la representación gráfica de la red de la figura 5-52(a). Lo que hace fundamentalmente el OSPF es representar la red como un grafo de este tipo y luego calcular la trayectoria más corta de un enrutador a todos los demás.

Muchas de las AS de Internet son grandes y nada fáciles de manejar. El OSPF permite su división en áreas numeradas, donde un área es una red o un grupo de redes contiguas. Las áreas no se traslapan, pero no necesitan ser exhaustivas, es decir, algunos enrutadores podrían no pertenecer a ningún área. Un área es una generalización de una subred. Fuera de un área, su topología y detalles no son visibles.

Cada AS tiene un área de *backbone*, llamada área 0. Todas las áreas se conectan al *backbone*, posiblemente mediante túneles, por lo que hay la posibilidad de ir de cualquier área del AS a cualquier otra a través del *backbone*. Un túnel se representa en el grafo como un arco y tiene un costo. Cada enrutador conectado a dos o más áreas es parte del *backbone*. Al igual que en otras áreas, la topología del *backbone* no es visible desde fuera del *backbone*.

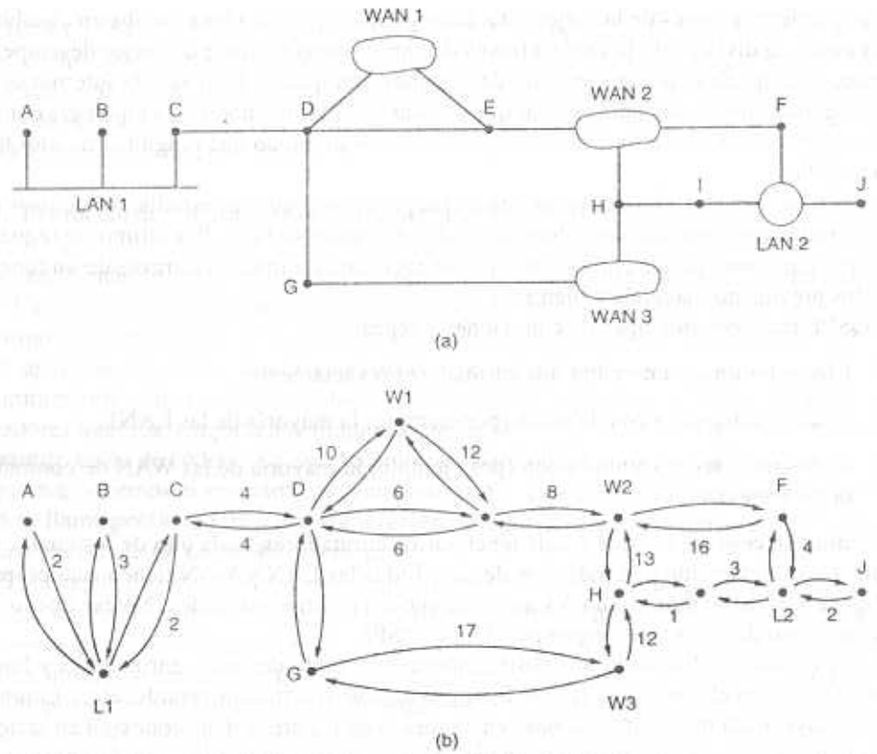


Figura 5-52. (a) Sistema autónomo. (b) Representación con grafos de (a).

Dentro de un área, cada enrutador tiene la misma base de datos de estado de enlace y ejecuta el mismo algoritmo de trayectoria más corta; su tarea principal es calcular la trayectoria más corta de sí mismo a todos los demás enrutadores del área, incluido el enrutador que está conectado al *backbone*, de los cuales debe haber cuando menos uno. Un enrutador que se conecta a dos áreas necesita las bases de datos de ambas áreas y debe ejecutar para cada una por separado el algoritmo de trayectoria más corta.

La manera en que el OSPF maneja el enrutamiento de tipo de servicio es teniendo varios grafos, uno etiquetado con los costos cuando la métrica es el retardo, otro etiquetado con los costos cuando la métrica es el rendimiento, y uno más etiquetado con los costos cuando la métrica es la confiabilidad. Aunque esto triplica el cálculo necesario, permite rutas separadas para optimar el retardo, el rendimiento y la confiabilidad.

Durante la operación normal, pueden necesitarse tres tipos de rutas: intraárea, interárea e interAS. Las rutas intraárea son la más fáciles, dado que el enrutador de origen ya conoce la trayectoria más corta al enrutador de destino. El enrutamiento interárea siempre procede en tres

pasos: va del origen al *backbone*, pasa a través del *backbone* al área de destino y va al destino. Este algoritmo obliga a una configuración en estrella en el OSPF, siendo el *backbone* el centro y las demás áreas los rayos. Los paquetes se enrutan del origen al destino "como vienen". No se encapsulan ni se envían por túnel, a menos que vayan a un área cuya única conexión al *backbone* sea un túnel. En la figura 5-53 se muestra parte de la Internet con AS y áreas.

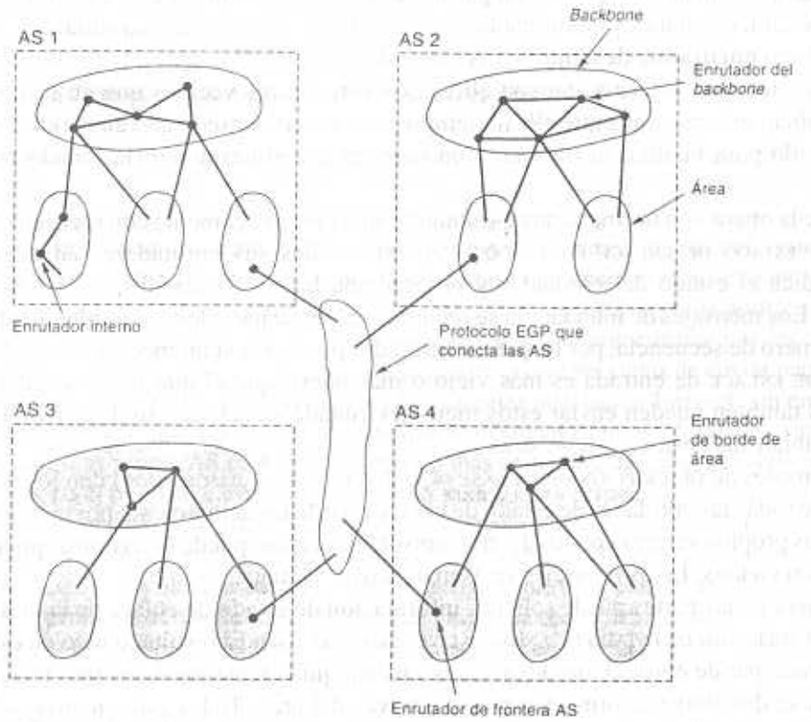


Figura 5-53. Relación entre los AS, los *backbone* y las áreas en el OSPF.

El OSPF distingue cuatro clases de enrutadores:

1. Enrutadores internos que están contenidos en una sola área.
2. Enrutadores de borde de área que conectan dos o más áreas.
3. Enrutadores de *backbone* que están en el *backbone*.
4. Enrutadores de frontera de AS que hablan con los enrutadores de otras AS.

Se permite que estas clases se traslapen. Por ejemplo, todos los enrutadores de borde son automáticamente parte del *backbone*. Además, un enrutador que está en el *backbone* pero no es

parte de ninguna otra área también es un enrutador interno. En la figura 5-53 se ilustran ejemplos de las cuatro clases de enrutadores.

Al arrancar un enrutador, envía mensajes de HOLA (*HELLO*) por todas sus líneas punto a punto y los multitransmite por las LAN al grupo que consiste en todos los demás enrutadores. En las WAN, el enrutador requiere cierta información de configuración para saber con quién tiene que hacer contacto. A partir de las respuestas, cada enrutador aprende quiénes son sus vecinos.

El OSPF funciona intercambiando información entre enrutadores **adyacentes**, que no es lo mismo que entre enrutadores vecinos. En particular, es ineficiente hacer que todos los enrutadores de una LAN hablen con todos los enrutadores de otra LAN. Para evitar esta situación, se elige un enrutador como **enrutador designado**, el cual se dice que es adyacente a todos los demás enrutadores, e intercambia información con ellos. Los enrutadores vecinos que no son adyacentes no intercambian información entre ellos. Siempre se mantiene actualizado un enrutador designado de respaldo para facilitar la transición en caso de que el enrutador designado primario se caiga.

Durante la operación normal, cada enrutador inunda periódicamente con mensajes de ACTUALIZACIÓN DE ESTADO DE ENLACE (*LINK STATE UPDATE*) a todos sus enrutadores adyacentes. Este mensaje indica el estado del enrutador y proporciona los costos usados en la base de datos topológica. Los mensajes de inundación se reconocen a fin de hacerlos confiables. Cada mensaje tiene un número de secuencia, por lo que un enrutador puede ver si un mensaje de ACTUALIZACIÓN DE ESTADO DE ENLACE de entrada es más viejo o más nuevo que el que tiene actualmente. Los enrutadores también pueden enviar estos mensajes cuando se activa o se desactiva una línea y cuando cambian sus costos.

Los mensajes de DESCRIPCIÓN DE LA BASE DE DATOS (*DATABASE DESCRIPTION*) dan los números de secuencia de todas las entidades de estado de enlace guardadas actualmente por el transmisor. Al comparar sus propios valores con los del transmisor, el receptor puede determinar quién tiene los valores más recientes. Estos mensajes se usan al activar la línea.

Cualquiera de las partes puede solicitar información de estado de enlace de la otra mediante mensajes de SOLICITUD DE ESTADO DE ENLACE (*LINK STATE REQUEST*). El resultado neto de este algoritmo es que cada par de enrutadores adyacentes verifica quién tiene los datos más recientes, y de esta manera se distribuye información nueva a través del área. Todos estos mensajes se envían como paquetes IP en bruto. Los cinco tipos de mensaje se resumen en la figura 5-54.

Tipo de mensaje	Descripción
Hola	Sirve para descubrir quiénes son los vecinos
Actualización de estado de enlace	Proporciona los costos del transmisor a sus vecinos
Reconocimiento de estado de enlace	Reconocimiento de la actualización de estado de enlace
Descripción de la base de datos	Anuncia las actualizaciones que tiene el transmisor
Solicitud de estado de enlace	Solicita información del compañero

Figura 5-54. Los cinco tipos de mensajes OSPF.

Por último, podemos juntar todas las piezas. Mediante inundación, cada enrutador informa a los demás enrutadores de su área acerca de sus vecinos y sus costos. Esta información permite a cada enrutador construir el grafo de su(s) área(s) y calcular la trayectoria más corta. El área de *backbone* también hace esto. Además, los enrutadores de *backbone* aceptan información de los enrutadores de borde de área a fin de calcular la mejor ruta de cada enrutador de *backbone* a los demás enrutadores. Esta información se propaga de regreso a los enrutadores de borde de área, quienes la divulgan en sus áreas. Usando esta información, un enrutador a punto de enviar un paquete interárea puede seleccionar el mejor enrutador de salida al *backbone*.

5.5.6. Protocolo de enrutamiento de pasarela exterior: BGP

Dentro de un solo AS, el protocolo de enrutamiento recomendado en Internet es el OSPF (aunque ciertamente no es el único en uso). Entre los AS, se usa un protocolo diferente, el **BGP** (*Border Gateway Protocol*, **protocolo de pasarela exterior**). Se requiere un protocolo diferente entre las AS porque las metas de un protocolo de pasarela interior y un protocolo de pasarela exterior no son iguales. Todo lo que tiene que hacer un protocolo de pasarela interior es mover paquetes con la mayor eficiencia posible del origen al destino; no necesita preocuparse por la política.

Los enrutadores de protocolo de pasarela exterior tienen que preocuparse por los asuntos políticos, y mucho. Por ejemplo, un AS corporativo podría querer ser capaz de enviar paquetes a cualquier instalación Internet y recibir paquetes de cualquier instalación Internet; sin embargo, tal vez no quiera transportar paquetes en tránsito que se originan en un AS foráneo y van a otro AS foráneo, aun si su propio AS está en la trayectoria más corta entre los dos AS foráneos ("ése es su problema, no el nuestro"). Por otra parte, el AS podría estar dispuesto a transportar tráfico en tránsito de sus vecinos, o inclusive de otros AS que pagaran por su servicio. Por ejemplo, las compañías telefónicas podrían estar felices de funcionar como portadoras para sus clientes, pero no para otros. Los protocolos de pasarela exterior en general, y en particular el BGP, se han diseñado para permitir muchos tipos de políticas de enrutamiento aplicables al tráfico interAS.

Las políticas típicas comprenden consideraciones políticas, de seguridad o económicas. Algunos ejemplos de restricciones de enrutamiento son:

1. Prohibición del tráfico en tránsito a través de ciertas AS.
2. Nunca poner a Irak en una ruta que comience en el Pentágono.
3. No utilizar a Estados Unidos para llegar de la Columbia Británica a Ontario.
4. Sólo transitar por Albania si no hay una alternativa hacia el destino.
5. El tráfico que comience o termine en IBM® no debe transitar por Microsoft®.

Las políticas se configuran manualmente en cada enrutador BGP. No son parte del protocolo mismo.

Desde el punto de vista de un enrutador BGP, el mundo consiste en otros enrutadores BGP y en las líneas que los conectan. Se consideran conectados dos enrutadores BGP si comparten una red común. Dado el interés especial del BGP en el tráfico en tránsito, las redes se agrupan en

una de tres categorías. La primera categoría es la de las **redes de punta**, que sólo tienen una conexión al grafo BGP; no se pueden usar para tráfico en tránsito porque no hay nadie del otro lado. Después vienen las **redes multiconectadas**. Éstas podrían usarse para el tráfico en tránsito, excepto que se niegan. Por último, están las **redes de tránsito**, como los *backbones*, que están dispuestas a manejar los paquetes de terceros, posiblemente con algunas restricciones.

Los pares de enrutadores BGP se comunican entre ellos estableciendo conexiones TCP. Este tipo de operación proporciona comunicación confiable y esconde todos los detalles de la red por la que se pasa.

El BGP fundamentalmente es un protocolo de vector de distancia, pero muy diferente de casi todos los demás, como el RIP. En lugar de mantener sólo el costo a cada destino, cada enrutador BGP lleva el registro de la trayectoria seguida. Del mismo modo, en lugar de dar periódicamente a cada vecino sus costos estimados a todos los destinos posibles, cada enrutador BGP le dice a sus vecinos la trayectoria exacta que está usando.

Como ejemplo, considere los enrutadores BGP mostrados en la figura 5-55(a). En particular, considere la tabla de enrutamiento de *F*. Suponga que se usa la trayectoria *FGCD* para llegar a *D*. Cuando los vecinos dan su información de rutas, proporcionan sus trayectorias completas, como se muestra en la figura 5-55(b) (por sencillez, sólo se muestra el destino *D*).

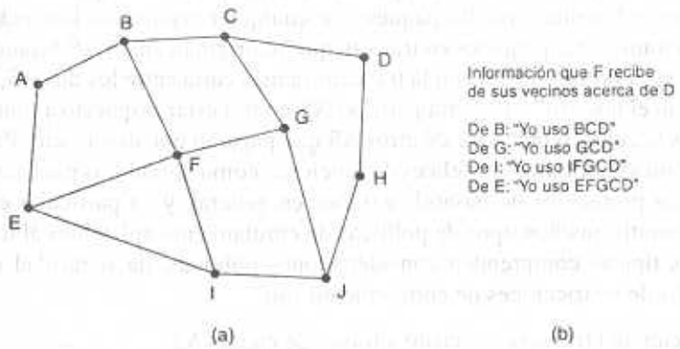


Figura 5-55. (a) Grupo de enrutadores BGP. (b) Información enviada a *F*.

Una vez que llegan todas las trayectorias de los vecinos, *F* las examina para ver cuál es la mejor. Pronto descarta las trayectorias de *I* y *E*, pues éstas pasan a través de *F* mismo. La decisión entonces está entre usar *B* o *G*. Cada enrutador BGP contiene un módulo que examina las rutas a un destino dado y las pondera, devolviendo un número para la "distancia" a ese destino por cada ruta. Cualquier ruta que viole una restricción por política automáticamente recibe una ponderación infinita. El enrutador entonces toma la ruta con la distancia más corta. La función de ponderación no es parte del protocolo BGP y puede ser cualquier función que quieran los administradores del sistema.

El BGP soluciona fácilmente el problema de conteo a infinito que es la plaga de otros algoritmos de enrutamiento por vector de distancia. Por ejemplo, supóngase que se cae *G* o se desactiva la línea *FG*. Entonces *F* recibe rutas de sus tres vecinos restantes. Estas rutas son *BCD*, *IFGCD* y *EFGCD*. Puede verse de inmediato que las dos últimas rutas no tienen sentido, pues pasan a través del mismo *F*, por lo que se escoge *FBCD* como ruta nueva. Otros algoritmos de vector de distancia con frecuencia toman las decisiones equivocadas porque no pueden saber cuáles de sus vecinos tienen rutas independientes a los destinos, y cuáles no. La definición actual del BGP está en el RFC 1654. Puede encontrarse información adicional útil en el RFC 1268.

5.5.7. Multidifusión Internet

La comunicación IP normal es entre un transmisor y un receptor. Sin embargo, en algunas aplicaciones es útil que un proceso pueda transmitir a una cantidad grande de receptores de manera simultánea. Como ejemplos están la actualización con réplica, las bases de datos distribuidas, la transmisión de cotizaciones de acciones a varias casas de bolsa y el manejo de conferencias telefónicas digitales (es decir, multiparte).

El IP maneja la multitransmisión, usando direcciones clase D. Cada dirección clase D identifica a un grupo de *hosts*. Hay 28 bits disponibles para la identificación de grupos, por lo que pueden existir 250 millones de grupos al mismo tiempo. Cuando un proceso envía un paquete a una dirección clase D, se hace un mejor esfuerzo para entregarlo a todos los miembros del grupo dirigido, pero no se dan garantías. Algunos miembros podrían no recibir el paquete.

Se reconocen dos tipos de dirección de grupo: las direcciones permanentes y las temporales. Un grupo permanente siempre está ahí y no tiene que configurarse. Cada grupo permanente tiene una dirección permanente de grupo. Algunos ejemplos de direcciones permanentes de grupo son:

224.0.0.1 Todos los sistemas de una LAN
224.0.0.2 Todos los enrutadores de una LAN
224.0.0.5 Todos los enrutadores OSPF de una LAN
224.0.0.6 Todos los enrutadores OSPF designados de una LAN

Deben crearse grupos temporales antes de poder usar estas direcciones. Un proceso puede pedir a su *host* que se una a un grupo específico; también puede pedirle que se retire del grupo. Cuando el último proceso de un *host* deja un grupo, ese grupo ya no está presente en el *host*. Cada *host* lleva el registro de los grupos a los que pertenecen sus procesos.

La multitransmisión se pone en práctica mediante enrutadores multitransmisión especiales, que pueden o no estar en los mismos puntos que los enrutadores estándar. Aproximadamente una vez cada minuto, todos los enrutadores multitransmisión envían una multitransmisión de *hardware* (por ejemplo, de capa de enlace de datos) a los *hosts* en su LAN (dirección 224.0.0.1) solicitándoles que informen los grupos a los que pertenecen sus procesos. Cada *host* envía de regreso respuestas a todas las direcciones clase D en las que está interesado.

Estos paquetes de solicitud y respuesta usan un protocolo llamado **IGMP** (*Internet Group Management Protocol*, **protocolo de administración de grupos de Internet**), que es vagamente

análogo al ICMP. Tiene sólo dos tipos de paquetes, solicitud y respuesta, teniendo cada uno un formato fijo y sencillo que contiene cierta información de control en la primera palabra del campo de carga útil y una dirección clase D en la segunda palabra; se describe en el RFC 1112.

El enrutamiento multitransmisión se hace usando árboles de extensión. Cada enrutador multitransmisión intercambia información con sus vecinos usando un protocolo de vector de distancia modificado a fin de que cada uno construya un árbol de extensión por grupo, cubriendo a todos los miembros del grupo. Se usan varias optimizaciones para podar el árbol y eliminar enrutadores y redes no interesadas en grupos en particular. El protocolo hace uso intensivo del proceso en túnel para evitar dar molestias a nodos que no están en un árbol de extensión.

5.5.8. IP móvil

Muchos usuarios de Internet tienen computadoras portátiles y quieren mantenerse conectados a Internet al visitar una instalación Internet lejana, y aun en el camino intermedio. Desafortunadamente, el sistema de direccionamiento IP hace que el trabajo lejos de casa sea más fácil de plantear que de hacer. En esta sección examinaremos el problema y la solución. En (Johnson, 1995) se da una descripción más detallada.

El verdadero villano es el esquema de direccionamiento mismo. Cada dirección IP contiene tres campos: la clase, el número de red y el número de *host*. Por ejemplo, considere la máquina con dirección IP 160.80.40.20. El 160.80 da la clase (B) y el número de red (8272); el 40.20 es el número de *host* (10260). Los enrutadores de todo el mundo tienen tablas de enrutamiento que indican la línea a usar para llegar a la red 160.80. Cuando llega un paquete con una dirección IP de destino de la forma 160.80.xxx.yyy, sale por esa línea.

Si de pronto, la máquina con esa dirección se lleva a algún lugar lejano, los paquetes para ella se seguirán enviando a su LAN (o enrutador) base. El dueño ya no podrá recibir correo electrónico, etc. Darle a la máquina una nueva dirección IP correspondiente a su nueva ubicación no es muy atractivo, pues habría que informar a una gran cantidad de gente, programas y bases de datos sobre el cambio.

Otro enfoque es hacer que los enrutadores usen direcciones IP completas para el enrutamiento, en lugar de sólo la clase y la red. Sin embargo, esta estrategia requeriría que cada enrutador tuviera tablas de millones de entradas, con un costo astronómico para Internet.

Cuando la gente comenzó a exigir la posibilidad de tener *hosts* móviles, el IETF estableció un grupo de trabajo para encontrar una solución. El grupo de trabajo pronto formuló varias metas consideradas deseables en cualquier solución. Las principales fueron:

1. Todo *host* móvil debe ser capaz de usar su dirección IP base en cualquier lugar.
2. No se permiten cambios al *software* de los *hosts* fijos.
3. No se permiten cambios al *software* del enrutador ni a sus tablas.
4. La mayoría de los paquetes para los *hosts* móviles no deben desviarse en el camino.
5. No debe incurrirse en carga extra cuando un *host* móvil está en su base.

La solución escogida fue la que se describe en la sección 5.2.8. En síntesis, cada instalación que quiera permitir que sus usuarios vaguen debe crear un agente de base. Cada instalación que permita visitantes tiene que crear un agente foráneo. Al aparecer un *host* móvil en una instalación foránea, se pone en contacto con el *host* foráneo y se registra. El *host* foráneo entonces se comunica con el agente de base del usuario y le da una **dirección de encargado** (*care-off address*), normalmente la misma dirección IP del agente foráneo.

Al llegar un paquete a la LAN base del usuario, llega por un enrutador conectado a la LAN. El enrutador entonces trata de localizar al *host* de la manera normal, difundiendo un paquete ARP que pregunte, por ejemplo: "¿cuál es la dirección Ethernet de 160.80.40.20?" El agente de base responde a esta solicitud dando su propia dirección Ethernet. El enrutador entonces envía los paquetes para 160.80.40.20 al agente de base. Éste a su vez, los envía en túnel a la dirección de encargado, encapsulándolos en el campo de carga útil de un paquete IP dirigido al agente foráneo. El agente foráneo entonces los desencapsula y los entrega a la dirección de enlace de datos del *host* móvil. Además, el agente de base entrega la dirección de encargado al transmisor, para que los paquetes futuros puedan enviarse en túnel directamente al agente foráneo. Esta solución cumple con todos los requisitos indicados antes.

Probablemente vale la pena mencionar un pequeño detalle. En el momento de moverse el *host* móvil, el enrutador probablemente tiene en caché sus direcciones Ethernet (que pronto dejarán de ser válidas). Para reemplazar esa dirección de Ethernet por la del agente de base, se usa un truco llamado **ARP gratuito** (*gratuitious ARP*). Éste es un mensaje especial al enrutador, no solicitado, que causa que reemplace una entrada específica de caché, en este caso la del *host* móvil a punto de irse. Al regresar el *host* móvil después, se usa el mismo truco para actualizar el caché del enrutador.

Nada del diseño impide que un *host* móvil sea su propio agente foráneo, pero este enfoque sólo funciona si el *host* móvil (en su capacidad como agente foráneo) está conectado lógicamente a Internet en su instalación actual. También, debe poder adquirir una dirección IP de encargado (temporal) para usarla. Esa dirección IP debe pertenecer a la LAN a la que está conectado actualmente.

La solución IETF para *hosts* móviles resuelve otros problemas no mencionados hasta ahora. Por ejemplo, ¿cómo localizar a los agentes? La solución es que cada agente difunda periódicamente sus direcciones y el tipo de servicios que está dispuesto a proporcionar (por ejemplo, base, foráneo o ambos). Al llegar un *host* móvil a algún lado, simplemente puede esperar estas difusiones, llamadas **anuncios** (*advertisements*). Como alternativa, puede difundir un paquete anunciando su llegada y esperar que el agente foráneo local responda.

Otro problema que tenía que resolverse es lo que había que hacer con los *hosts* móviles mal educados que salen sin decir adiós. La solución es hacer que el registro sea válido sólo durante un intervalo de tiempo fijo; si no se renueva periódicamente, termina su temporización, y así el agente foráneo puede limpiar sus tablas.

Un tema más es el de la seguridad. Cuando un agente de base recibe un mensaje solicitándole que redirija todos los paquetes de Nora a alguna dirección IP, no debe acceder a menos que esté convencido de que Nora es el origen de la solicitud, y no alguien que está tratando de hacerse pasar por ella. Se usan protocolos cifrados de verificación de autenticidad para este fin. Estudiaremos tales protocolos en el capítulo 7.

Un punto final cubierto por el grupo de trabajo se relaciona con los niveles de movilidad. Imagine un avión con un Ethernet a bordo usado por las computadoras de navegación y aeronáutica. En este Ethernet hay un enrutador estándar que habla con la Internet alambrada de tierra mediante un enlace de radio. Un buen día, a algún astuto ejecutivo de mercadeo se le ocurre instalar conectores Ethernet en los descansabrazos para que los pasajeros con computadoras móviles también se puedan conectar.

Ahora tenemos dos niveles de movilidad: las propias computadoras del avión, que son estacionarias respecto del Ethernet, y las computadoras de los pasajeros, que son móviles respecto de él. Además, el enrutador de a bordo es móvil respecto de los enrutadores de tierra. La movilidad respecto de un sistema que en sí mismo es móvil puede manejarse usando túneles recursivos.

5.5.9. CIDR — Enrutamiento interdominios sin clases

El IP ya ha estado en uso intensivo por más de una década; ha funcionado extremadamente bien, como lo demuestra el crecimiento exponencial de la Internet. Desgraciadamente, el IP se está convirtiendo con rapidez en una víctima de su propia popularidad: se le están acabando las direcciones. Este desastre inminente ha propiciado una gran cantidad de controversias y debates en la comunidad de Internet sobre lo que debe hacerse al respecto. En esta sección describiremos tanto el problema como varias soluciones propuestas. En (Huitema, 1996) puede encontrarse una descripción más completa.

En 1987 unos pocos visionarios predijeron que algún día la Internet podría crecer hasta 100,000 redes. La mayoría de los expertos minimizaron el problema aduciendo que ocurriría en muchas décadas, si es que llegaba a ocurrir. En 1996 se conectó la red 100,000. El problema, en pocas palabras, es que la Internet se está quedando rápidamente sin direcciones de IP. En teoría, existen cerca de dos mil millones de direcciones, pero la práctica de organizar el espacio de direcciones por clases (véase la figura 5-47) desperdicia millones de ellas. En particular, el verdadero villano es la red clase B. Para la mayoría de las organizaciones, una red clase A, con 16 millones de direcciones, es demasiado grande, y una red clase C, de 256 direcciones, es demasiado pequeña. Una red clase B, con 65,536, es la adecuada. En el folclor de Internet, esta situación se conoce como el **problema de los tres osos** (como en *Ricitos de Oro y los tres osos*).

En realidad, una dirección clase B es demasiado grande para la mayor parte de las organizaciones. Hay estudios que demuestran que más de la mitad de todas las redes clase B tienen menos de 50 *hosts*. Una red clase C habría bastado, pero sin duda todas las organizaciones que solicitaron una dirección clase B pensaron que un día el campo de *hosts* de 8 bits les quedaría pequeño. En retrospectiva, podría haber sido mejor que las redes clase C usaran 10 bits en lugar de 8 para el número de *host*, permitiendo 1022 *hosts* por red. De haber sido éste el caso, la mayor parte de las organizaciones probablemente se habría conformado con una red clase C, y habría habido medio millón de ellas (en vez de sólo 16,384 redes clase B).

Sin embargo, entonces habría surgido más rápidamente otro problema: la explosión de tablas de enrutamiento. Desde el punto de vista de los enrutadores, el espacio de direcciones IP es una jerarquía de dos niveles, con números de red y números de *host*. Los enrutadores no

tienen que saber de todos los *hosts*, pero sí tienen que saber de todas las redes. Si se usaran medio millón de redes clase C, todos los enrutadores de Internet necesitarían una tabla con medio millón de entradas, una por red, indicando la línea a usar para llegar a esa red, así como otra información.

Actualmente, el almacenamiento de medio millón de entradas tal vez es posible, aunque caro en los enrutadores críticos que guardan las tablas en la RAM estática de las tarjetas de E/S. Un problema más serio es que la complejidad de diferentes algoritmos relacionados con el mantenimiento de tablas crece con una tasa mayor que la lineal. Peor aún, mucho del *software* y *firmware* existente de los enrutadores se diseñó en un momento en que la Internet tenía 1000 redes conectadas, y tener 10,000 redes parecía estar a décadas de distancia. Las decisiones de diseño tomadas entonces ya no tienen nada de óptimas.

Además, diversos algoritmos de enrutamiento requieren que cada enrutador transmita sus tablas periódicamente. Cuanto más grandes sean las tablas, más probabilidad habrá de que algunas partes se pierdan en el camino, dando pie a datos incompletos en el otro lado y posiblemente a inestabilidades de enrutamiento.

El problema de las tablas de enrutamiento podría haberse resuelto usando una jerarquía más. Por ejemplo, podría haber servido hacer que cada dirección de IP tuviera un campo de país, estado, ciudad, red y *host*. Entonces cada enrutador sólo necesitaría saber cómo llegar a los países, a los estados o provincias de su propio país, a las ciudades de su estado o provincia y a las redes de su ciudad. Por desgracia, esta solución requeriría bastante más de 32 bits para las direcciones de IP y usaría ineficientemente las direcciones (Liechtenstein tendría tantos bits como Estados Unidos).

En resumen, la mayoría de las soluciones resuelven un problema pero crean uno nuevo. Una solución que ahora se está poniendo en práctica y que dará a Internet un respiro es el **CIDR** (*Classless InterDomain Routing*, **enrutamiento interdominios sin clases**). El concepto básico del CIDR, que se describe en el RFC 1519, es repartir las redes clase C restantes, de las que hay casi dos millones, en bloques de tamaño variable. Si una instalación necesita, digamos, 2000 direcciones, se le da un bloque de 2048 direcciones (ocho redes clase C contiguas), y no una dirección clase B completa. Del mismo modo, una instalación que necesita 8000 direcciones recibe 8192 direcciones (32 redes contiguas clase C).

Además de usar como unidades bloques de redes contiguas clase C, las reglas de asignación para las direcciones clase C también se cambiaron en el RFC 1519. El mundo se dividió en cuatro zonas, y a cada una se le dio una parte del espacio de direcciones clase C. La asignación fue la siguiente:

Las direcciones 194.0.0.0 a 195.255.255.255 son para Europa
Las direcciones 198.0.0.0 a 199.255.255.255 son para Norteamérica
Las direcciones 200.0.0.0 a 201.255.255.255 son para Centro y Sudamérica
Las direcciones 202.0.0.0 a 203.255.255.255 son para Asia y el Pacífico

De este modo, cada región recibió unos 32 millones de direcciones para repartir, con otros 320 millones de direcciones clase C, de la 204.0.0.0 a 223.255.255.255, guardadas como reserva

para el futuro. La ventaja de esta asignación es que ahora cualquier enrutador fuera de Europa que recibe un paquete dirigido a 194.xx.yy.zz o a 195.xx.yy.zz simplemente puede enviarlo a su pasarela europea estándar. En efecto, ahora se han comprimido 32 millones de direcciones en una sola entrada de tabla de enrutamiento. Lo mismo ocurre con las demás regiones.

Por supuesto, una vez que llega un paquete 194.xx.yy.zz a Europa, se requieren tablas de enrutamiento más detalladas. Una posibilidad es tener 131,072 entradas para las redes 194.0.0.xx a 195.255.255.xx, pero ésta es precisamente la explosión de tablas de enrutamiento que estamos tratando de evitar. En vez de ello, cada entrada de tabla de enrutamiento se extiende con una máscara de 32 bits. Al llegar un paquete, se extrae primero su dirección de destino. Luego (conceptualmente) se examina la tabla de enrutamiento entrada por entrada, enmascarando la dirección de destino y comparándola con las entradas de la tabla en busca de una igual.

Para hacer más claro este proceso de comparación, consideremos un ejemplo. Supóngase que la Universidad de Cambridge necesita 2048 direcciones y se le asignan las direcciones 194.24.0.0 a 194.24.7.255, junto con la máscara 255.255.248.0. En seguida, la Universidad de Oxford solicita 4096 direcciones. Puesto que un bloque de 4096 direcciones debe caer en un límite de 4096 bytes, no pueden asignarse las direcciones que comienzan en 194.8.0.0. En cambio, Oxford recibe 194.24.16.0 a 194.24.31.255, junto con la máscara 255.255.240.0. Ahora la Universidad de Edimburgo solicita 1024 direcciones, y se le asignan las direcciones 194.24.8.0 a 194.24.11.255 y la máscara 255.255.252.0.

Las tablas de enrutamiento de toda Europa se actualizan con tres entradas, que contienen una dirección base y una máscara. Estas entradas (en binario) son:

Dirección	Máscara
11000010 00011000 00000000 00000000	11111111 11111111 11111000 00000000
11000010 00011000 00010000 00000000	11111111 11111111 11110000 00000000
11000010 00011000 00001000 00000000	11111111 11111111 11111100 00000000

Ahora considere lo que ocurre cuando llega un paquete dirigido a 194.24.17.4, que en binario es

11000010 00011000 00010001 00000100

Primero se le hace un AND booleano con la máscara de Cambridge para obtener

11000010 00011000 00010000 00000000

Este valor no es igual a la dirección base de Cambridge, por lo que vuelve a hacerse un AND con la máscara de Oxford para obtener

11000010 00011000 00010000 00000000

Este valor sí es igual a la máscara de Oxford, por lo que el paquete se envía al enrutador de Oxford. En la práctica, las entradas de enrutador no se prueban secuencialmente; se usan trucos de indización para acelerar la búsqueda. También, es posible que dos entradas coincidan después del AND, en cuyo caso gana la que tenga una máscara con más bits 1. Por último, la misma idea puede aplicarse a todas las direcciones, no sólo a las nuevas direcciones clase C, por lo que, con el CIDR, las viejas redes clase A, B y C no se usan más para enrutamiento. Ésta es la razón por la que al CIDR se le llama enrutamiento sin clases. El CIDR se describe con mayor detalle en (Ford *et al.*, 1993, y Huitema, 1995).

5.5.10. IPv6

Si bien el CIDR puede alargar unos pocos años más de tiempo, todo mundo se da cuenta que los días del IP en su forma actual (IPv4) están contados. Además de estos problemas técnicos, hay otro asunto acechando. Hasta hace poco, la Internet ha sido utilizada en gran medida por universidades, industrias de alta tecnología y el gobierno (especialmente el Departamento de Defensa de Estados Unidos). Con la explosión del interés por la Internet que comenzó a mediados de los años 90, es muy posible que en el próximo milenio será usada por un grupo mucho más grande de gente, especialmente gente con necesidades diferentes. Por una parte, millones de personas con computadoras portátiles inalámbricas podrían usarla para mantenerse en contacto con sus bases. Por otra, con la inminente convergencia de las industrias de la computación, las comunicaciones y el entretenimiento, tal vez no pase mucho tiempo antes de que todos los televisores del mundo estén en un nodo Internet, produciendo mil millones de máquinas utilizadas para recibir vídeo a solicitud. En estas circunstancias, se hizo evidente que el IP tenía que evolucionar y volverse más flexible.

Al ver en el horizonte estos problemas, el IETF comenzó a trabajar en 1990 en una versión nueva del IP, una que nunca se quedaría sin direcciones, resolvería varios otros problemas y sería más flexible y eficiente también. Sus metas principales eran:

1. Manejar miles de millones de *hosts*, aún con asignación de espacio de direcciones ineficiente.
2. Reducir el tamaño de las tablas de enrutamiento.
3. Simplificar el protocolo, para permitir a los enrutadores el procesamiento más rápido de los paquetes.
4. Proporcionar mayor seguridad (verificación de autenticidad y confidencialidad) que el IP actual.
5. Prestar mayor atención al tipo de servicio, especialmente con datos en tiempo real.
6. Ayudar a la multitransmisión permitiendo la especificación de alcances.
7. Posibilitar que un *host* sea móvil sin cambiar su dirección.
8. Permitir que el protocolo evolucione.
9. Permitir que el protocolo viejo y el nuevo coexistan por años.

Para encontrar un protocolo que cumpliera con todos estos requisitos, la IETF hizo una convocatoria solicitando propuestas y estudios en el RFC 1550. Se recibieron 21 respuestas, no todas propuestas completas. En diciembre de 1992 había siete propuestas serias en la mesa; iban desde hacer cambios menores al IP hasta desecharlo y reemplazarlo por un protocolo completamente diferente.

Una propuesta fue ejecutar TCP sobre CLNP, que, con sus direcciones de 160 bits habría proporcionado suficiente espacio de direcciones para siempre y habría unificado dos protocolos de capa de red principales. Sin embargo, muchos pensaron que esto habría sido una aceptación de que algunas cosas en el mundo OSI en realidad estaban bien hechas, algo considerado políticamente incorrecto en los círculos de Internet. El CLNP se creó tomando como modelo al IP, por lo que los dos no son realmente tan diferentes. De hecho, el protocolo escogido es más diferente del IP que CLNP. Otra cosa en contra el CLNP fue su pobre manejo de tipos de servicio, algo requerido para transmitir multimedia eficientemente.

Tres de las mejores propuestas se publicaron en *IEEE Network* (Deering, 1993; Francis, 1993, y Katz y Ford, 1993). Tras muchos análisis, revisiones e intrigas, se seleccionó una versión modificada de la combinación de las propuestas de Deering y Francis, llamada ahora **SIPP** (*Simple Internet Protocol Plus*, **protocolo simple de Internet mejorado**), y se le dio la designación **IPv6** (el IPv5 ya estaba en uso con un protocolo experimental de flujos en tiempo real).

El IPv6 cumple los objetivos bastante bien: mantiene las buenas características del IP, descarta y reduce las malas, y agrega nuevas donde se necesitan. En general, IPv6 no es compatible con IPv4, pero es compatible con todos los demás protocolos Internet, incluidos TCP, UDP, ICMP, IGMP, OSPF, BGP y DNS, a veces requiriendo pequeñas modificaciones (principalmente para manejar direcciones más grandes). Las características principales del IPv6 se analizan a continuación. Puede encontrarse mayor información en los RFC 1883 a 1887.

Por principio, y lo más importante, el IPv6 tiene direcciones más grandes que el Ipv4; son de 16 bytes de longitud, lo que resuelve el problema que se buscaba resolver: proporcionar una cantidad prácticamente ilimitada de direcciones Internet. Hablaremos más sobre las direcciones un poco más adelante.

La segunda mejora principal del IPv6 es la simplificación de la cabecera, que contiene sólo 7 campos (contra 13 en el IPv4). Este cambio permite a los enrutadores procesar con mayor rapidez los paquetes y mejorar, por tanto, el rendimiento. También estudiaremos pronto la cabecera.

La tercera mejora importante fue el mejor apoyo de las opciones. Este cambio fue esencial con la nueva cabecera, pues campos que antes eran obligatorios ahora son opcionales. Además, es diferente la manera de representar las opciones, haciendo más sencillo que los enrutadores hagan caso omiso de opciones no dirigidas a ellos. Esta característica mejora el tiempo de procesamiento de los paquetes.

Una cuarta área en la que el IPv6 representa un avance importante es la seguridad. La IETF tenía infinidad de historias sobre preadolescentes precoces que usaban sus computadoras personales para meterse en bancos e instalaciones militares por todas partes de Internet. Se tenía la fuerte sensación de que había que hacerse algo para mejorar la seguridad. Las verificaciones de autenticidad y la confidencialidad son características clave del IP nuevo.

Por último, se tenía que prestar mayor atención al tipo de servicio que en el pasado. El IPv4 de hecho tiene un campo de 8 bits dedicado a este asunto, pero con el crecimiento esperado del tráfico multimedia en el futuro, se requiere mucho más.

La cabecera principal del IPv6

La cabecera del IPv6 se muestra en la figura 5-56. El campo de *versión* siempre es de 6 para el IPv6 (y de 4 para el IPv4). Durante el periodo de transición del IPv4 al IPv6, que probablemente se llevará una década, los enrutadores podrán examinar este campo para saber el tipo de paquete que tienen. Como nota al margen, esta prueba ocupa algunas instrucciones en la trayectoria crítica, por lo que muchas implementaciones probablemente la evitarán usando algún campo de la cabecera de enlace de datos para distinguir los paquetes IPv4 de los IPv6. De esta manera, los paquetes pueden pasarse directamente al manejador correcto de la capa de red. Sin embargo, hacer que la capa de enlace de datos esté consciente de los tipos de los paquetes de red viola por completo el principio de diseño de que ninguna capa debe estar enterada del significado de los bits entregados por la capa superior a ella. Los debates entre los bandos de "hacerlo bien" y "hacerlo rápido" sin duda serán largos y acalorados.

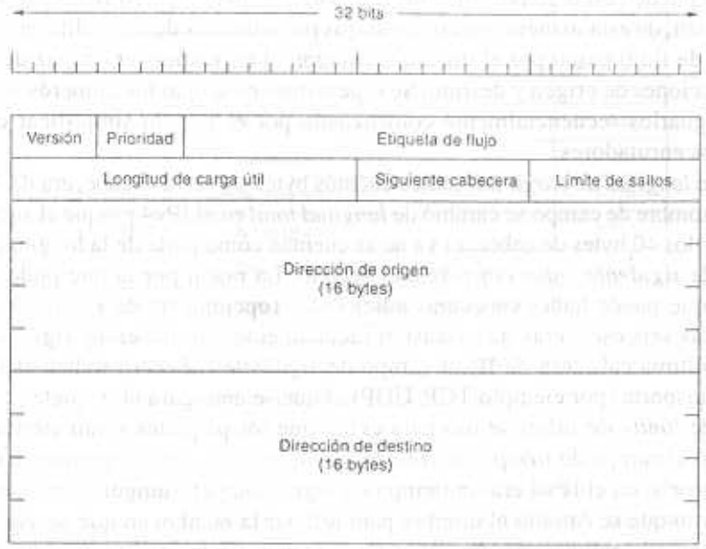


Figura 5-56. Cabecera fija del IPv6 (obligatoria).

El campo de *prioridad* se usa para distinguir entre paquetes a cuyas de orígenes se les puede controlar el flujo y aquellos a los que no. Los valores 0 a 7 son para transmisiones capaces de reducir su velocidad en caso de un congestionamiento. Los valores 8 a 15 son para tráfico de tiempo real cuya tasa de envío es constante, aun si todos los paquetes se están perdiendo. El audio y el vídeo caen en esta última categoría. Esta distinción permite a los enrutadores manejar mejor los paquetes en caso de congestionamientos. Dentro de cada grupo, los paquetes de

número más bajo son menos importantes que los paquetes de número alto. El estándar IPv6 sugiere, por ejemplo, usar 1 para noticias, 4 para FTP y 6 para conexiones Telnet, pues el retardo de un paquete de noticias durante algunos segundos no es perceptible, pero el retardo de un paquete Telnet ciertamente lo es.

El campo de *etiqueta de flujo* aún es experimental, pero se usará para permitir a un origen y a un destino establecer una seudoconexión con propiedades y requisitos particulares. Por ejemplo, una cadena de paquetes de un proceso en cierto *host* de origen dirigido a cierto proceso en cierto *host* de destino puede tener requisitos de retardo muy estrictos, y por tanto necesitar un ancho de banda reservado. El flujo puede establecerse por adelantado, dándole un identificador. Cuando aparece un paquete con una *etiqueta de flujo* diferente de cero, todos los enrutadores pueden buscarla en sus tablas internas para ver el tipo de tratamiento especial que requiere. En efecto, los flujos son un intento de tener lo mejor de ambos mundos: la flexibilidad de una subred de datagramas y las garantías de una subred de circuitos virtuales.

Cada flujo está designado por la dirección de origen, la dirección de destino y el número de flujo, por lo que pueden estar activos muchos flujos al mismo tiempo entre un par dado de direcciones IP. También, de esta manera, aun si dos flujos provenientes de *hosts* diferentes pero con el mismo número de flujo pasan por el mismo enrutador, el enrutador será capaz de distinguirlos usando las direcciones de origen y destino. Se espera que se escojan los números de flujo al azar, en lugar de asignarlos secuencialmente comenzando por el 1, para simplificar el proceso de dispersión en los enrutadores.

El campo de *longitud de carga útil* indica cuántos bytes siguen a la cabecera de 40 bytes de la figura 5-56. El nombre de campo se cambió de *longitud total* en el IPv4 porque el significado cambió ligeramente: los 40 bytes de cabecera ya no se cuentan como parte de la longitud, como antes.

El campo de *siguiente cabecera* revela el secreto. La razón por la que pudo simplificarse la cabecera es que puede haber cabeceras adicionales (opcionales) de extensión. Este campo indica cuál de las seis cabeceras de extensión (actualmente), de haberlas, sigue a ésta. Si esta cabecera es la última cabecera de IP, el campo de *siguiente cabecera* indica el manejador de protocolo de transporte (por ejemplo TCP, UDP) al que se entregará el paquete.

El campo de *límite de saltos* se usa para evitar que los paquetes vivan eternamente. En la práctica es igual al campo de *tiempo de vida* del IPv4, es decir, un campo que se disminuye en cada salto. En teoría, en el IPv4 era un tiempo en segundos, pero ningún enrutador lo usaba de esa manera, por lo que se cambió el nombre para reflejar la manera en que se usa realmente.

Luego vienen los campos de *dirección de origen* y *dirección de destino*. La propuesta original de Deering, el SIP, usaba direcciones de 8 bytes, pero durante el periodo de revisión muchas personas sintieron que, con direcciones de 8 bytes, en algunas décadas el IPv6 se quedaría sin direcciones, y que con direcciones de 16 bytes nunca se acabarían. Otros argumentaban que 16 bytes era demasiado, mientras que unos más estaban a favor de usar direcciones de 20 bytes para hacerlas compatibles con el protocolo de datagramas de OSI. Otra facción quería direcciones de tamaño variable. Tras mucho debate, se decidió que la mejor media sería una dirección de 16 bytes de longitud fija.

El espacio de direcciones del IPv6 se divide como se muestra en la figura 5-57. Las direcciones que comienzan con 80 ceros se reservan para direcciones IPv4. Se reconocen dos variantes,

Prefijo (binario)	Uso	Fracción
0000 0000	Reservado (incluye IPv4)	1/256
0000 0001	No asignado	1/256
0000 001	Direcciones OSI NSAP	1/128
0000 010	Direcciones IPX de Novell NetWare	1/128
0000 011	No asignado	1/128
0000 1	No asignado	1/32
0001	No asignado	1/16
001	No asignado	1/8
010	Direcciones basadas en proveedor	1/8
011	No asignado	1/8
100	Direcciones basadas en geografía	1/8
101	No asignado	1/8
110	No asignado	1/8
1110	No asignado	1/16
1111 0	No asignado	1/32
1111 10	No asignado	1/64
1111 110	No asignado	1/128
1111 1110 0	No asignado	1/512
1111 1110 10	Direcciones de enlace de uso local	1/1024
1111 1110 11	Direcciones de instalación de uso local	1/1024
1111 1111	Multitransmisión	1/256

Figura 5-57. Direcciones IPv6.

distinguidas por los siguientes 16 bits. Estas variantes se relacionan con la manera en que se enviarán en túnel los paquetes IPv6 a través de la infraestructura IPv4 existente.

El uso de prefijos distintos para las direcciones basadas en el proveedor y las basadas en la geografía es una media entre dos diferentes visiones del futuro de Internet. Las direcciones basadas en el proveedor tienen sentido si se piensa que en el futuro habrá algunas compañías que proporcionarán servicio Internet a sus clientes, lo que es parecido al servicio telefónico que prestan actualmente AT&T, MCI, Sprint, British Telecom y otras. Cada una de estas compañías tendrá una fracción del espacio de direcciones. Los primeros 5 bits que siguen al prefijo 010 sirven para indicar el registro donde se debe buscar al proveedor. Actualmente operan tres registros, para Norteamérica, Europa y Asia. Pueden agregarse hasta 29 registros nuevos posteriormente.

Cada registro está en libertad de dividir los 15 bytes restantes como crea conveniente. Se espera que la mayoría de ellos usarán un número de proveedor de 3 bytes, dando unos 16

millones de proveedores, a fin de permitir que una compañía grande actúe como su propio proveedor. Otra posibilidad es el uso de 1 byte para indicar proveedores nacionales y dejarlos efectuar asignaciones posteriores. De esta manera, pueden introducirse niveles adicionales de jerarquía, según sea necesario.

El modelo geográfico es el mismo que el actual de Internet, en el que los proveedores no desempeñan un papel importante. De esta manera, el IPv6 puede manejar ambos tipos de direcciones.

Las direcciones locales de enlace y de instalación sólo tienen significado local; pueden reutilizarse sin conflictos en cada organización, pero no pueden propagarse fuera de los límites de la organización, lo que las hace útiles para las organizaciones que actualmente usan muros de seguridad para protegerse del resto de Internet.

Las direcciones multitransmisión tienen un campo indicador de 4 bits y un campo de alcance de 4 bits a continuación del prefijo, y luego un identificador de grupo de 112 bits. Uno de los bits indicadores hace la distinción entre grupos permanentes y transitorios. El campo de alcance permite que una multitransmisión se limite al enlace, instalación u organización actuales, o al planeta. Estos cuatro alcances se distribuyen entre los 16 valores para permitir la adición posterior de nuevos alcances. Por ejemplo, el alcance planetario es 14, por lo que el código 15 está disponible para permitir la expansión futura de Internet a otros planetas, sistemas solares y galaxias.

Además de manejar direcciones unitransmisión estándar (punto a punto) y multitransmisión, el IPv6 también maneja un nuevo tipo de direccionamiento: transmisión a cualquiera. La **transmisión a cualquiera** (*anycasting*) es como la multitransmisión, en el sentido de que el destino es un grupo de direcciones, pero en lugar de tratar de entregar el paquete a todos, intenta entregarlo a uno solo, generalmente el más cercano. Por ejemplo, al ponerse en contacto con un grupo de servidores de archivos cooperativos, un cliente puede usar la transmisión a cualquiera para llegar al más cercano, sin tener que saber quién es. La transmisión a cualquiera usa direcciones normales de unitransmisión. Es responsabilidad del sistema de enrutamiento escoger al afortunado *host* que recibirá el paquete.

Se ha desarrollado una nueva notación para escribir direcciones de 16 bytes: se escriben como ocho grupos de cuatro dígitos hexadecimales, separados los grupos por dos puntos, como sigue:

8000:0000:0000:0000:0123:4567:89AB:CDEF

Ya que muchas direcciones tendrán muchos ceros en ellas, se han autorizado tres optimizaciones. Primero, los ceros a la izquierda de un grupo pueden omitirse, por lo que 0123 puede escribirse como 123. Segundo, pueden reemplazarse uno o más grupos de 16 ceros por un par de signos de dos puntos. Por tanto, la dirección anterior se vuelve ahora

8000::0123:4567:89AB:CDEF

Por último, las direcciones IPv4 pueden escribirse como un par de signos de dos puntos y un número decimal anterior separado por puntos, como por ejemplo

::192.31.20.46

Tal vez no sea necesario ser tan explícitos al respecto, pero hay muchas direcciones de 16 bytes. Específicamente, hay 2^{128} de ellas, lo que aproximadamente es 3×10^{38}. Si la Tierra completa, incluidos los océanos, estuviera cubierta de computadoras, el IPv6 permitiría 7×10^{23} direcciones de IP por metro cuadrado. Los estudiantes de química notarán que este número es mayor que el número de Avogadro. Aunque no fue la intención darle a cada molécula de la superficie terrestre su propia dirección de IP, no estamos lejos de ello.

En la práctica, el espacio de direcciones no se usará eficientemente, igual que el espacio de números telefónicos (el código de área de Manhattan, 212, está prácticamente lleno, pero el de Wyoming, 307, está casi vacío). En el RFC 1715, Huitema calculó que, usando la asignación de números telefónicos como guía, hasta en la situación más pesimista habrá más de 1000 direcciones IP por metro cuadrado de la superficie terrestre (tierra y agua). En cualquier situación probable, habrá billones de ellas por metro cuadrado. En pocas palabras, parece poco probable que se acabarán en el futuro previsible. También vale la pena indicar que hasta ahora sólo se ha asignado el 28% del espacio de direcciones. El otro 72% está disponible para propósitos futuros que aún no se han pensado.

Es instructivo comparar la cabecera de IPv4 (figura 5-45) con la de IPv6 (figura 5-56) para ver lo que se ha dejado fuera del IPv6. El campo *IHL* se fue porque la cabecera de IPv6 tiene una longitud fija. El campo de *protocolo* (*Protocol*) se retiró porque el campo de *siguiente cabecera* (*next header*) indica lo que sigue a la última cabecera de IP (por ejemplo, un segmento UDP o TCP).

Se retiraron todos los campos relacionados con la fragmentación, puesto que el IPv6 tiene un enfoque distinto hacia la fragmentación. Para empezar, todos los *hosts* y enrutadores que se ajustan al IPv6 deben reconocer paquetes de 576 bytes. Esta regla hace que sea menos posible que ocurra la fragmentación. Además, cuando un *host* envía un paquete de IPv6 demasiado grande, en lugar de fragmentarlo, el enrutador que es incapaz de reenviarlo devuelve un mensaje de error. Este mensaje indica al *host* que divida todos los paquetes futuros a ese destino. Es mucho más eficiente hacer que el *host* envíe paquetes del tamaño correcto desde el principio que hacer que los enrutadores los fragmenten sobre la marcha.

Por último, el campo de suma de comprobación (*checksum*) desaparece, porque su cálculo reduce en gran medida el desempeño. Con las redes confiables de hoy, además del hecho de que la capa de enlace de datos y las capas de transporte normalmente tienen sus propias sumas de comprobación, el provecho de otra suma de comprobación no valía el costo de desempeño que generaba. Al removerse estas características ha quedado un protocolo de capa de red compacto y sólido. Por tanto, la meta del IPv6 (un protocolo rápido y flexible con bastante espacio de direcciones) se ha cumplido con este diseño.

Cabeceras de extensión

Con todo, algunos de los campos faltantes en ocasiones son necesarios, por lo que el IPv6 introdujo el concepto de **cabecera de extensión** opcional. Estas cabeceras pueden usarse para proporcionar información extra, pero codificada de una manera eficiente. Hay seis tipos de cabeceras de extensión definidas actualmente, que se listan en la figura 5-58. Todas son opciona-

les, pero si hay más de una, deben aparecer justo después de la cabecera fija, y de preferencia en el orden listado.

Cabecera de extensión	Descripción
Opciones salto por salto	Información diversa para los enrutadores
Enrutamiento	Ruta total o parcial a seguir
Fragmentación	Manejo de fragmentos de datagramas
Verificación de autenticidad	Comprobación de la identidad del transmisor
Carga útil cifrada de seguridad	Información sobre el contenido cifrado
Opciones de destino	Información adicional para el destino

Figura 5-58. Cabeceras de extensión del IPv6.

Algunas de las cabeceras tienen un formato fijo; otras contienen un número variable de campos de longitud variable. En éstos, cada elemento está codificado como una tupla (tipo, longitud, valor). El *tipo* es un campo de 1 byte que indica la opción de la que se trata. Los valores de *tipo* se han escogido de modo que los dos primeros bits indican a los enrutadores que no saben cómo procesar la opción lo que tienen que hacer. Las posibilidades son: saltar la opción, descartar el paquete, descartar el paquete y enviar de regreso un paquete ICMP, y lo mismo que lo anterior, pero no enviar paquetes ICMP a direcciones de multitransmisión (para evitar que un paquete de multitransmisión malo genere millones de informes ICMP).

La *longitud* también es un campo de 1 byte, e indica la longitud del valor (0 a 255 bytes). El *valor* es cualquier información requerida, de hasta 255 bytes.

La cabecera de salto por salto se usa para información que deben examinar todos los enrutadores a lo largo de la trayectoria. Hasta ahora, se ha definido una opción: manejo de datagramas de más de 64K. El formato de esta cabecera se muestra en la figura 5-59.

Siguiente cabecera	0	194	0
Longitud de la carga útil grande			

Figura 5-59. La cabecera de extensión salto por salto para datagramas grandes (jumbogramas).

Como con todas las cabeceras de extensión, ésta comienza con 1 byte que indica el tipo de cabecera que sigue. A este byte le sigue uno que indica la longitud de la cabecera salto por salto en bytes, excluyendo los primeros 8 bytes, que son obligatorios. Los 2 bytes siguientes indican que esta opción define el tamaño del datagrama (código 194) como número de 4 bytes. Los últimos 4 bytes indican el tamaño del datagrama. No se permiten los tamaños menores que 65,536, que darán como resultado el que el primer enrutador descarte el paquete y devuelva un mensaje

ICMP de error. Los datagramas que usan esta cabecera de extensión se llaman **jumbogramas**. El uso de jumbogramas es importante para las aplicaciones de supercomputadoras que deben transferir con eficiencia gigabytes de datos a través de Internet.

La cabecera de enrutamiento lista uno o más enrutadores que deben visitarse en el camino al destino. Hay la posibilidad tanto de enrutamiento estricto (se suministra la trayectoria completa) como libre (sólo se proporcionan enrutadores seleccionados), pero están combinados. El formato de la cabecera de enrutamiento se muestra en la figura 5-60.

Siguiente cabecera	0	Cantidad de direcciones	Siguiente dirección
	Mapa de bits		
Direcciones 1 – 24			

Figura 5-60. La cabecera de extensión para enrutamiento.

Los primeros 4 bytes de la cabecera de extensión de enrutamiento contienen cuatro enteros de 1 byte: el tipo de la siguiente cabecera, el tipo de enrutamiento (actualmente 0), la cantidad de direcciones presentes en esta cabecera (1 a 24) y el índice de la siguiente dirección a visitar. Este último campo comienza en 0 y aumenta con cada dirección que se visita.

Después viene un byte reservado seguido de un mapa de bits con bits para cada una de las 24 direcciones IPv6 potenciales que le siguen. Estos bits indican si debe visitarse cada dirección directamente después de la anterior (enrutamiento estricto desde el origen) o si pueden intervenir otros enrutadores intermedios (enrutamiento libre desde de origen).

La cabecera de fragmentación maneja la fragmentación de una manera parecida a la del IPv4. La cabecera contiene el identificador del datagrama, el número de fragmento y un bit que indica si seguirán más fragmentos. En el IPv6, a diferencia del IPv4, sólo el *host* de origen puede fragmentar un paquete. Los enrutadores a lo largo del camino no pueden hacerlo. Aunque este cambio es un rompimiento filosófico con el pasado, simplifica el trabajo del enrutador y acelera el enrutamiento. Como se mencionó antes, si un enrutador confronta un paquete demasiado grande, lo descarta y devuelve un paquete ICMP al origen. Esta información permite que el *host* de origen fragmente el paquete en pedazos más pequeños usando esta cabecera y lo intente de nuevo.

La cabecera de verificación de autenticidad proporciona un mecanismo mediante el cual el receptor de un paquete puede estar seguro de quién lo envió. Con el IPv4 no existe tal garantía. La carga cifrada de seguridad posibilita el cifrado del contenido de un paquete de modo que sólo el receptor pretendido pueda leerlo. Estas cabeceras usan técnicas de cifrado para lograr su cometido. Daremos una descripción breve a continuación, pero los lectores no familiarizados con el cifrado moderno podrían no entender la descripción ahora, y deberán regresar tras leer el capítulo 7 (en particular, la sección 7.1), que trata los protocolos de cifrado.

Cuando un transmisor y un receptor desean comunicarse con seguridad, primero deben ponerse de acuerdo en una o más claves secretas que sólo ellos conocen. La manera en que lo

hacen no es asunto del IPv6. A cada una de estas claves se le asigna un número de clave único de 32 bits. Los números de clave son globales, por lo que si Alicia está usando la clave 4 para hablar con Roberto, no puede usar una clave 4 para hablar con Carolina. Asociados a cada número de clave hay otros parámetros, como tiempo de vida de la clave y otros.

Para enviar un mensaje con verificación de autenticidad, el transmisor primero construye un paquete que consiste en todas las cabeceras IP y la carga útil, y luego reemplaza los campos que cambian en el camino (por ejemplo, el *límite de saltos*) por ceros. Se rellena el paquete con ceros hasta un múltiplo de 16 bytes. Del mismo modo, la clave secreta a usar se rellena con ceros hasta un múltiplo de 16 bytes. Ahora se calcula una suma de comprobación cifrada con base en la concatenación de la clave secreta rellenada, el paquete rellenado y nuevamente la clave secreta rellenada. Los usuarios pueden definir su propio algoritmo de suma de comprobación cifrada, pero se recomienda a los usuarios con poca habilidad en el cifrado usar el algoritmo predeterminado, el MD5.

Ahora llegamos al papel de la cabecera de verificación de autenticidad. Básicamente, ésta contiene tres partes. La primera consiste en 4 bytes que contienen el siguiente número de cabecera, la longitud de la cabecera de verificación de autenticidad, y 16 bits cero. Después viene el número de clave de 32 bits. Por último, se incluye la suma de comprobación MD5 (u otra).

El receptor entonces usa el número de clave para encontrar la clave secreta. La versión rellenada de ella se agrega al principio y al final de la carga útil rellenada, los campos de cabecera variables se llenan con ceros, y se calcula la suma de comprobación; si está de acuerdo con la suma incluida en la cabecera de verificación de autenticidad, el receptor puede estar seguro de que el paquete llegó del transmisor con quien comparte la clave secreta, y también puede estar seguro de que el paquete no fue alterado en el camino. Las propiedades del MD5 hacen imposible, desde el punto de vista de cómputo, que un intruso falsifique la identidad del transmisor o modifique el paquete de alguna manera que pueda escapar a la detección.

Es importante apuntar que la carga útil de un paquete con verificación de autenticidad se envía sin cifrado. Cualquier enrutador a lo largo del camino puede leer lo que dice. En muchas aplicaciones el secreto no es realmente importante, sólo la relación de autenticidad. Por ejemplo, si un usuario indica a su banco que pague su cuenta telefónica, probablemente no habrá necesidad real del secreto, pero hay una necesidad muy real de que el banco tenga la seguridad completa de quién envió el paquete que contiene la orden de pago.

Para paquetes que deben enviarse secretamente, se usa la cabecera de extensión de carga útil cifrada de seguridad. Ésta comienza con un número de clave de 32 bits, seguido de la carga cifrada. El algoritmo de cifrado es responsabilidad del transmisor y del receptor, pero por omisión es DES en modo de encadenamiento de bloque cifrado (CBC). Al usarse DES-CBC, el campo de carga útil comienza por el vector de inicialización (un múltiplo de 4 bytes), luego la carga útil, y luego el relleno a un múltiplo de 8 bytes. Si se desean tanto cifrado como verificación de autenticidad, se requieren ambas cabeceras.

La intención de la cabecera de opciones de destino es incluir campos que sólo deben interpretarse en el *host* de destino. En la versión inicial de IPv6, las únicas opciones definidas son las opciones nulas para rellenar esta cabecera a un múltiplo de 8 bytes, por lo que no se usará inicialmente; se incluyó para asegurar que el nuevo *software* de enrutamiento y de los *hosts* puedan manejarla, en caso de que a alguien se le ocurra una opción de destino algún día.

Controversias

Dados el proceso de diseño abierto y las fuertes opiniones de muchas de las personas participantes, no debería sorprender que muchas decisiones tomadas para el IPv6 fueran tema de fuertes controversias. Resumiremos a continuación algunas de ellas. Véanse en (Huitema, 1996) todos los detalles escabrosos.

Ya hemos mencionado el argumento sobre la longitud de las direcciones. El resultado fue una concesión: las direcciones de 16 bytes de longitud fija.

Surgió otra pelea sobre la longitud del campo de *límite de saltos*. Una parte sentía que la limitación de la cantidad máxima de saltos a 255 (implícita al usar un campo de 8 bits) fue un error grave. A fin de cuentas, hoy son comunes las trayectorias de 32 saltos, y en 10 años podrán ser comunes trayectorias mucho más grandes. Esta gente argumentaba que el uso de una dirección enorme era ir demasiado lejos, pero que el uso de una cuenta de saltos pequeña era tener una visión miope. Desde su punto de vista, el peor pecado que puede cometer un informático es proporcionar insuficientes bits en algún lugar.

La respuesta fue que se pueden hacer argumentos para aumentar todos los campos, lo que llevaría a una cabecera inflada. También, la función del campo de límite de saltos es evitar que los paquetes vaguen durante demasiado tiempo, y 65,535 saltos son demasiados. Por último, a medida que crezca Internet, se construirán más y más enlaces de larga distancia, posibilitando la ida de un país a otro en media docena de saltos, cuando mucho. Si se requieren más de 125 saltos para llegar del origen y el destino a sus pasarelas internacionales, algo está mal con las columnas vertebrales nacionales. Los de 8 bits ganaron esta partida.

Otra papa caliente fue el tamaño máximo de paquete. La comunidad de las supercomputadoras quería paquetes de más de 64 kB. Cuando una supercomputadora comienza a transferir, el asunto realmente va en serio, y no quiere que se le interrumpa cada 64 kB. El argumento en contra de los paquetes grandes es que, si un paquete de 1 MB llega a una línea T1 de 1.5 Mbps, el paquete bloqueará la línea durante más de 5 segundos, produciendo un retardo muy notorio para los usuarios interactivos que comparten la línea. Se llegó a una concesión: los paquetes normales se limitan a 64 kB, pero puede usarse la cabecera de extensión de salto por salto para permitir los jumbogramas.

Un tercer tema candente fue la desaparición de la suma de comprobación del IPv5. Para algunas personas esto representa algo parecido a quitarle los frenos a un automóvil. Hacerlo ciertamente aligera al automóvil, y por tanto puede ir más rápido pero, de ocurrir un evento inesperado, tendremos problemas.

El argumento en contra de las sumas de comprobación fue que cualquier aplicación a la que de verdad le importa la integridad de sus datos de todos modos tiene que tener una suma de comprobación en la capa de transporte, por lo que tener otra en el IP (además de la suma de comprobación de la capa de enlace de datos) es un exceso. Además, la experiencia mostraba que el cálculo de una suma de comprobación en el IP era un gasto importante en el IPv4. El bando en contra de la suma de comprobación ganó ésta, y el IPv6 no tiene una suma de comprobación.

Los *hosts* móviles también fueron tema de contienda. Si una computadora portátil vuela al otro lado del mundo, ¿puede continuar operando en el destino con la misma dirección IPv6, o

tiene que usar un esquema con agentes foráneos y agentes de base? Los *hosts* móviles también generan asimetrías en el sistema de enrutamiento. Es posible el caso en que una computadora móvil pequeña pueda escuchar fácilmente la señal emitida por un enrutador estacionario grande, pero que el enrutador estacionario no pueda escuchar la débil señal emitida por el *host* móvil. En consecuencia, algunas personas querían incluir reconocimiento explícito de *hosts* móviles en el IPv6. Este esfuerzo falló cuando no se pudo generar consenso para ninguna propuesta específica.

Probablemente la batalla principal fue sobre la seguridad. Todos estaban de acuerdo en que se necesitaba. La guerra fue sobre dónde y cuándo. Primero dónde. El argumento en favor de ponerla en la capa de red es que entonces se vuelve un servicio estándar que todas las aplicaciones pueden usar sin ninguna planeación adelantada. El argumento en contra es que las aplicaciones realmente seguras por lo general no quieren nada menos que el cifrado de terminal a terminal, donde la aplicación de origen hace el cifrado y la aplicación de destino lo deshace. Con cualquier cosa menos, el usuario está a merced de implementaciones de capa de red con fallas potenciales, sobre las que no tiene control. La respuesta a este argumento es que tales aplicaciones simplemente pueden abstenerse de usar las características de seguridad del IP y encargarse ellas mismas del asunto. La réplica a esto es que la gente que no confía en que la red lo haga bien no quiere pagar el precio de implementaciones de IP lentas y estorbosas que tengan esta capacidad, aun si está inhabilitada.

Otro aspecto sobre dónde poner la seguridad se relaciona con que muchos países (pero no todos) tienen leyes de exportación estrictas en lo referente a cifrado. Algunos, particularmente Francia e Irak, también restringen mucho su uso doméstico, de manera que la gente no pueda ocultar secretos de la policía. Como resultado, cualquier puesta en práctica de IP que use un sistema de cifrado lo bastante robusto como para tener algún valor no podría exportarse de los Estados Unidos (y de muchos otros países) a clientes mundiales. La mayoría de los proveedores de computadoras se oponen enérgicamente a tener que mantener dos juegos de *software*, uno para uso doméstico y otro para exportación.

Una solución potencial es que todos los proveedores muden sus talleres de cifrado a un país que no regule el cifrado, como Finlandia o Suiza. Se podría diseñar y fabricar *software* de cifrado sólido ahí y embarcarlo legalmente a todos los países, excepto Francia e Irak. El problema con este enfoque es que el diseño de parte del *software* de enrutamiento en un país y parte en otro puede llevar a problemas de integración.

La última controversia relacionada con la seguridad tiene que ver con la selección de los algoritmos predeterminados que todas las implementaciones deben reconocer. Si bien que se pensaba que el MD5 era relativamente seguro, los avances recientes en cifrado pueden debilitarlo. Ningún criptógrafo serio cree que el DES es seguro ante ataques de grandes gobiernos, pero probablemente es lo bastante bueno como para frustrar incluso al preadolescente más precoz, cuando menos por ahora. El arreglo fue entonces integrar seguridad en el IPv6, usar un algoritmo de suma de comprobación de primer nivel para una buena verificación de autenticidad y un algoritmo debilucho para el secreto, pero dar al usuario la opción de reemplazar estos algoritmos por los suyos propios.

Un punto donde no hubo controversia es que nadie espera que la Internet IPv4 se apague un domingo por la mañana y reinicie como Internet IPv6 la mañana del lunes. En cambio, se

convertirán "islas" aisladas a IPv6, comunicándose inicialmente a través de túneles. A medida que crezcan las islas IPv6, se integrarán a islas más grandes. Tarde o temprano todas las islas se integrarán, y la Internet habrá sido convertida por completo. Dada la inversión masiva en los enrutadores IPv4 actualmente instalados, el proceso de conversión probablemente tardará una década. Por esta razón, se ha puesto un enorme esfuerzo en asegurar que esta transición sea lo menos dolorosa posible.

5.6. LA CAPA DE RED EN LAS REDES ATM

Las capas del modelo ATM (véase la figura 1-30) no se proyectan en las capas OSI muy bien que digamos, lo que conduce a ambigüedades. La capa de enlace de datos OSI se encarga del protocolo de enmarcado y del de transferencia entre dos máquinas que están en el mismo alambre (o fibra) físico. Los protocolos de capa de enlace de datos son protocolos de un solo salto. No se ocupan de conexiones terminal a terminal, puesto que la conmutación y el enrutamiento no ocurren en la capa de enlace de datos. Sobre esto no hay duda.

La capa más baja que va del origen al destino, y por tanto comprende enrutamiento y conmutación (es decir, es multisalto), es la capa de red. La capa ATM se encarga de mover células del origen al destino, y en definitiva se relaciona con algoritmos y protocolos de enrutamiento en los conmutadores ATM. Esta capa también maneja las direcciones globales. Con tal funcionalidad, la capa ATM desempeña el trabajo esperado de la capa de red. La capa ATM no garantiza una confiabilidad del 100%, pero no se requiere un protocolo de capa de red.

También, la capa ATM se asemeja a la capa 3 del X.25, que nadie discute que sea un protocolo de capa de red. Dependiendo de estados de bits, el protocolo de capa de red X.25 puede o no ser confiable, pero la mayoría de las implementaciones lo tratan como no confiable. Dado que la capa ATM tiene la funcionalidad esperada de la capa de red, no tiene la funcionalidad esperada de la capa de enlace de datos, y es bastante parecida a otros protocolos existentes de capa de red, en este capítulo estudiaremos la capa ATM.

La confusión surge porque mucha gente de la comunidad ATM considera que la capa ATM es una capa de enlace de datos, e inclusive, al hacer la emulación de una LAN, una capa física. Mucha gente de la comunidad Internet también la considera como una capa de enlace de datos, porque quieren poner IP encima de ella, y el hacer de la capa ATM una capa de enlace de datos se adecua bien a este concepto. (Aunque, de acuerdo con este razonamiento, para la comunidad Internet *todas* las redes operan en la capa de enlace de datos, sin importar sus características físicas.)

El único problema es que la capa ATM no tiene las características de un protocolo de capa de enlace de datos: un protocolo de un solo salto usado por máquinas en las terminales opuestas de un alambre, como los protocolos 1 a 6 del capítulo 3. La capa ATM tiene las características de un protocolo de capa de red: circuitos virtuales terminal a terminal, conmutación y enrutamiento.

Esto recuerda al autor una vieja adivinanza:

Pregunta: ¿Cuántas piernas tendría una mula si llamáramos pierna a la cola?
Respuesta: Cuatro. El *llamar* pierna a la cola no la *convierte* en una pierna.

Dicho esto, al lector se le previene de la controversia que hay aquí, combinada con una buena dosis de puro sentimiento.

La capa ATM se orienta a conexiones, tanto en términos del servicio que ofrece como de la manera en que opera internamente. El elemento básico de la capa ATM es el circuito virtual (llamado con frecuencia **canal virtual**). Un circuito virtual normalmente es una conexión de un origen a un destino, aunque también se permiten conexiones multitransmisión. Los circuitos virtuales son unidireccionales, pero puede crearse un par de circuitos al mismo tiempo. Ambas partes del par se direccionan al mismo identificador, de modo que un circuito virtual de hecho es dúplex integral. Sin embargo, la capacidad del canal y otras propiedades pueden ser diferentes en las dos direcciones, e inclusive pueden ser de cero en una de ellas.

La capa ATM es inusual para un protocolo orientado a conexiones en el sentido de que no proporciona acuses de recibo. La razón de este diseño es que la ATM se diseñó para usarse en redes de fibra óptica, que son altamente confiables. Se consideró adecuado dejar el control de errores a las capas superiores. A fin de cuentas, el envío de acuses de recibo en la capa de enlace de datos o en la de red sólo es una optimización. Siempre es suficiente que la capa de transporte envíe un mensaje y luego envíe nuevamente el mensaje completo si no se tiene un acuse de recibo a tiempo.

Es más, las redes ATM se usan con frecuencia para tráfico en tiempo real, como audio y vídeo. Para este tipo de tráfico, la retransmisión ocasional de una célula mala es peor que simplemente ignorarla.

A pesar de su falta de acuses de recibo, la capa ATM sí proporciona una garantía firme: las células que se envíen por un circuito virtual nunca llegarán fuera de orden. Se permite a la subred ATM descartar células si ocurren congestionamientos, pero en ninguna circunstancia puede reordenar las células enviadas por un solo circuito virtual. Sin embargo, no se dan garantías sobre el orden si un *host* envía células por *diferentes* circuitos virtuales. Por ejemplo, si un *host* envía una célula por el circuito virtual 10 y luego envía otra por el circuito virtual 20 al mismo destino, la segunda célula podría llegar primero. Si las dos células se hubieran enviado por el mismo circuito virtual, la primera enviada siempre sería la primera en llegar.

La capa ATM reconoce una jerarquía de conexión de dos niveles que es visible a la capa de transporte. A lo largo de cualquier trayectoria de transmisión de un origen dado a un destino dado, un grupo de circuitos virtuales puede agruparse en lo que se llama una **trayectoria virtual**, como se presenta en la figura 5-61. Conceptualmente, una trayectoria virtual es como un haz de pares trenzados de cobre: al reenrutarlo, todos los pares (circuitos virtuales) se reenrutan juntos. Consideraremos en detalle las implicaciones de este diseño de dos niveles más adelante.

5.6.1. Formatos de célula

En la capa ATM se distinguen dos interfaces: la **UNI** (*User-Network Interface*, **interfaz usuario-red**) y la **NNI** (*Network-Network Interface*, **interfaz red-red**). La primera define el límite entre un *host* y una red ATM (en muchos casos, entre el cliente y la portadora). La última se aplica a la línea entre dos conmutadores ATM (el término ATM para los enrutadores).

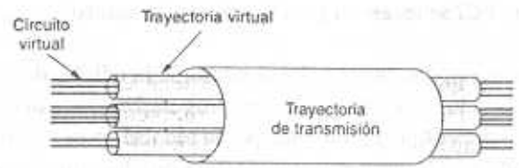


Figura 5-61. Una trayectoria de transmisión puede contener varias trayectorias virtuales, cada una de las cuales puede contener varios circuitos virtuales.

En ambos casos, las células consisten en una cabecera de 5 bytes seguida de una carga útil de 48 bytes, pero las dos cabeceras son ligeramente diferentes. Las cabeceras, como las definieron en el foro ATM, se ilustran en la figura 5-62. Las células se transmiten comenzando por el byte más a la izquierda, y por el bit más a la izquierda de cada byte.

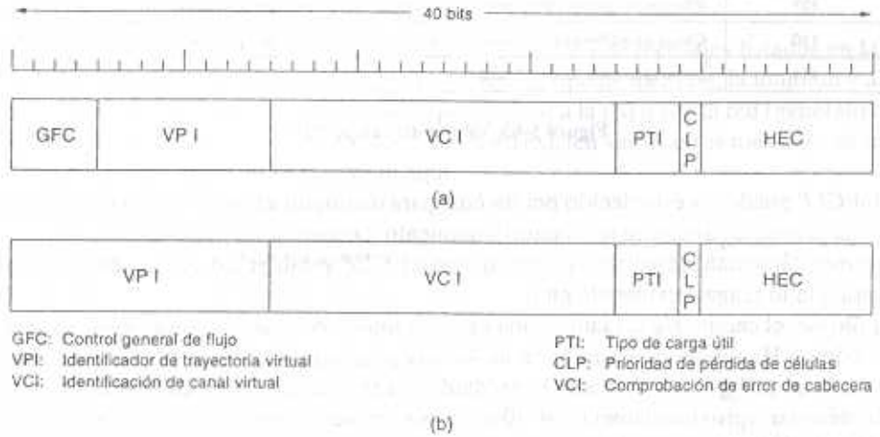


Figura 5-62. (a) La cabecera de capa ATM en la UNI. (b) La cabecera de capa ATM en la NNI.

El campo *GFC* está presente sólo en las células entre un *host* y la red; es sobrescrito por el primer conmutador al que llega, por lo que no tiene un significado de terminal a terminal, y no se entrega al destino. Originalmente se pensó que este campo tendría alguna utilidad para el control de flujo o de prioridad entre los *hosts* y las redes, pero no hay valores definidos para él, y la red lo ignora. La mejor manera de pensar en él es como una falla del estándar.

El campo *VPI* es un entero pequeño que selecciona una trayectoria virtual en particular (véase la figura 5-61). De la misma manera, el campo *VCI* selecciona un circuito virtual en particular en la trayectoria virtual seleccionada. Dado que el campo *VPI* tiene 8 bits (en la UNI) y el campo *VCI* tiene 16 bits, en teoría un *host* puede tener hasta 256 haces de VC, conteniendo cada uno hasta 65,536 circuitos virtuales. En realidad, la cantidad disponible de ambos es un

poco menor, pues algunos *VCI* se reservan para funciones de control, como el establecimiento de circuitos virtuales.

El campo *PTI* define el tipo de carga útil que contiene la célula, de acuerdo con los valores dados en la figura 5-63. Aquí los tipos de célula son proporcionados por el usuario, pero la información de congestionamientos es proporcionada por la red. En otras palabras, una célula enviada con *PTI* 000 podría llegar con 010 para avisar al destino que hay problemas en el camino.

Tipo de carga	Significado
000	Célula de datos de usuario, sin congestionamientos, célula tipo 0
001	Célula de datos de usuario, sin congestionamientos, célula tipo 1
010	Célula de datos de usuario, hubo congestionamiento, célula tipo 0
011	Célula de datos de usuario, hubo congestionamiento, célula tipo 1
100	Información de mantenimiento entre conmutadores adyacentes
101	Información de mantenimiento entre conmutadores de origen y destino
110	Célula de administración de recursos (usada para el control de congestionamientos ABR)
111	Reservado para función futura

Figura 5-63. Valores del campo *PTI*.

El bit *CLP* puede ser establecido por un *host* para distinguir entre el tráfico de alta prioridad y el de baja prioridad. Si ocurre un congestionamiento y deben descartarse células, los conmutadores primero intentan descartar las que tienen el *CLP* establecido en 1 antes de descartar cualquiera que lo tenga establecido en 0.

Por último, el campo *HEC* es una suma de comprobación de la cabecera; no verifica la carga útil. Un código Hamming de un número de 40 bits sólo requiere 5 bits, por lo que, con 8 bits, puede usarse un código más refinado. El escogido puede corregir todos los errores de un solo bit y puede detectar aproximadamente el 90% de los errores multibit. Numerosos estudios han mostrado que la gran mayoría de los errores de los enlaces ópticos son errores de un solo bit.

A continuación de la cabecera vienen 48 bytes de carga útil. Sin embargo, no todos los 48 bytes están disponibles para el usuario, pues algunos de los protocolos AAL ponen sus cabeceras y sus terminaciones dentro de la carga.

El formato NNI es igual al formato UNI, excepto que el campo *GFC* no está presente y esos 4 bits se usan para hacer que el campo *VPI* sea de 12 bits en lugar de 8.

5.6.2. Establecimiento de la conexión

ATM maneja circuitos virtuales tanto permanentes como conmutados. Los primeros siempre están presentes y pueden usarse a voluntad, como en las líneas arrendadas. Los segundos tienen que establecerse cada vez que se usan, como al hacer una llamada telefónica. En esta sección describiremos la manera en que se establecen los circuitos virtuales conmutados.

Técnicamente, el establecimiento de la conexión no es parte de la capa ATM, sino que es manejado por el plano de control (véase la figura 1-30) usando un protocolo ITU muy complicado llamado **Q.2931** (Stiller, 1995). No obstante, el lugar lógico para manejar el establecimiento de una conexión de capa de red es en la capa de red, y otros protocolos de capa de red parecidos se encargan del establecimiento de la conexión aquí, por lo que ahora lo estudiaremos.

Hay varias maneras de establecer una conexión. La normal es adquirir primero un circuito virtual para señalización, y usarlo. Para establecer tal circuito, células que contienen una solicitud se envían por la trayectoria virtual 0, circuito virtual 5. Si hay éxito, se abre un circuito virtual nuevo por el que pueden enviarse y recibirse solicitudes y respuestas de establecimiento de conexión.

La razón de este procedimiento de establecimiento de dos pasos es que de esta manera el ancho de banda reservado para el circuito virtual 5 (que apenas se usa) puede mantenerse extremadamente bajo. También, se proporciona una manera alterna de establecer circuitos virtuales. Algunas portadoras pueden permitir que los usuarios tengan trayectorias virtuales permanentes entre destinos predefinidos, o permitirles establecerlos dinámicamente. Una vez que un *host* tiene una trayectoria virtual a algún otro *host*, puede asignar él mismo circuitos virtuales, sin implicar a los conmutadores.

El establecimiento de un circuito virtual usa los seis tipos de mensajes listados en la figura 5-64. Cada mensaje ocupa una o más células y contiene el tipo de mensaje, la longitud y algunos parámetros. Los mensajes pueden ser enviados por un *host* a la red o por la red (generalmente en respuesta a un mensaje de otro *host*) a un *host*. También existen varios otros mensajes de informe de estado y de errores, pero no se muestran aquí.

Mensaje	Significado al ser enviado por un *host*	Significado al ser enviado por una red
ESTABLECER	Favor de establecer un circuito	Llamada entrante
LLAMADA EN PROCESO	Vi la llamada entrante	Se intentará su solicitud de llamada
CONEXIÓN	Acepto la llamada entrante	Se aceptó su solicitud de llamada
CONEXIÓN RECONOCIDA	Gracias por aceptar	Gracias por hacer la llamada
LIBERACIÓN	Favor de terminar la llamada	La otra parte ya no quiere hacer más
LIBERACIÓN COMPLETA	Reconocimiento de LIBERACIÓN	Reconocimiento de LIBERACIÓN

Figura 5-64. Mensajes usados para establecer y liberar conexiones.

El procedimiento normal para establecer una llamada es que un *host* envíe un mensaje de ESTABLECER (*SETUP*) por un circuito virtual especial. La red entonces responde con LLAMADA EN PROCESO (*CALL PROCEEDING*)para reconocer la recepción de la solicitud. A medida que el mensaje de ESTABLECER se propaga hacia el destino, es reconocido en cada salto por un mensaje de LLAMADA EN PROCESO.

Cuando el mensaje de ESTABLECER finalmente llega a su destino, el *host* de destino puede responder con CONEXIÓN (*CONNECT*) para aceptar la llamada. La red envía entonces un mensaje de CONEXIÓN RECONOCIDA (*CONNECT ACK*) para indicar que se ha recibido el mensaje de CONEXIÓN. A

medida que el mensaje de conexión se propaga de regreso al originador, cada conmutador que lo recibe lo reconoce con un mensaje de CONEXIÓN RECONOCIDA. Esta secuencia de eventos se presenta en la figura 5-65(a).

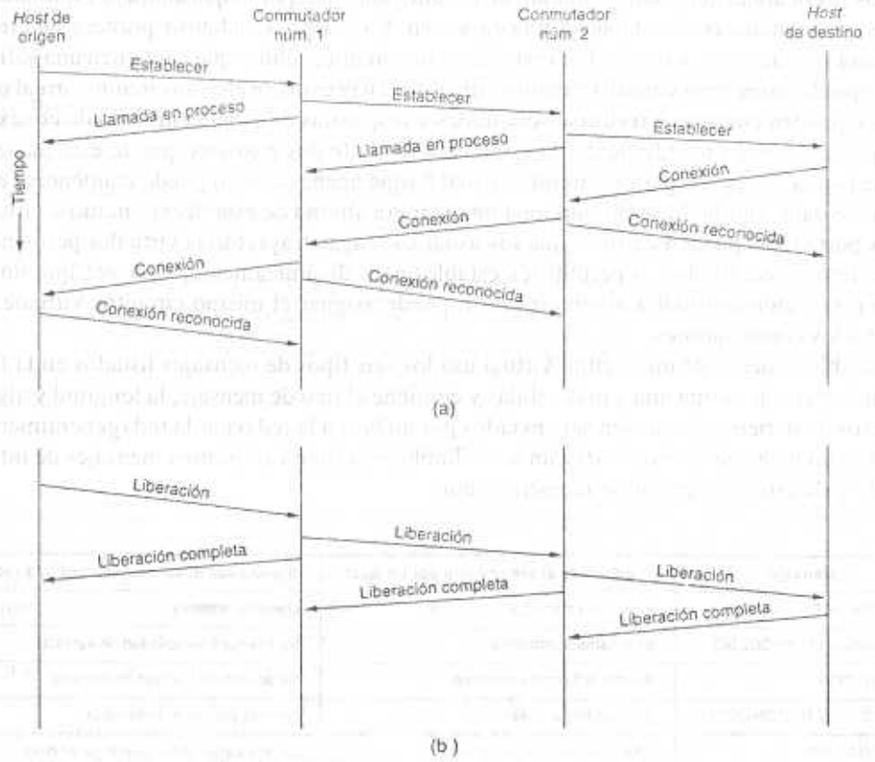


Figura 5-65. (a) Establecimiento de una conexión en una red ATM. (b) Liberación de la conexión.

La secuencia para liberar un circuito virtual es sencilla. El *host* que desea colgar simplemente envía un mensaje de LIBERACIÓN (*RELEASE*), que se propaga al otro extremo y causa que el circuito se libere. En cada salto a lo largo del camino se reconoce el mensaje, como se muestra en la figura 5-65(b).

Las redes ATM permiten el establecimiento de canales multitransmisión. Un canal multitransmisión tiene un transmisor y más de un receptor; se construye estableciendo una conexión con uno de los destinos de la manera normal. Entonces se envía el mensaje AGREGAR PARTE (*ADD PARTY*) para sumar un segundo destino al circuito virtual devuelto por la llamada previa. Pueden enviarse posteriormente más mensajes AGREGAR PARTE para aumentar el tamaño del grupo de multitransmisión.

A fin de establecer una conexión con un destino, es necesario especificar el destino, incluyendo su dirección en el mensaje de ESTABLECER. Las direcciones ATM tienen tres formas. La primera es de 20 bytes de longitud y se basa en las direcciones OSI. El primer byte indica en cuál de tres formatos está la dirección. En el primer formato, los bytes 2 y 3 especifican un país, y el byte 4 da el formato del resto de la dirección, que contiene una autoridad de 3 bytes, un dominio de 2 bytes, un área de 2 bytes y una dirección de 6 bytes, más algunos otros elementos. En el segundo formato, los bytes 2 y 3 designan a una organización internacional en lugar de un país. El resto de la dirección es igual que en el formato 1. De manera alterna, también se permite una forma más vieja de direccionamiento (CCITT E.164) que usa números telefónicos ISDN decimales de 15 dígitos.

5.6.3. Enrutamiento y conmutación

Al establecerse un circuito virtual, el mensaje de ESTABLECER sigue su camino a través de la red del origen al destino. El algoritmo de enrutamiento determina la trayectoria tomada por este mensaje y, por tanto, el circuito virtual. El estándar ATM no especifica ningún algoritmo de enrutamiento en particular, por lo que la portadora está en libertad de escoger entre los algoritmos estudiados antes en este capítulo, o usar otro diferente.

La experiencia con las redes previas orientadas a conexiones, como X.25, ha demostrado que puede gastarse una cantidad considerable de capacidad de cómputo de los conmutadores en la determinación de la manera de convertir la información del circuito virtual de cada célula en la selección de la línea de salida. Los diseñadores de ATM quisieron evitar esto, por lo que se ha diseñado la capa ATM para hacer posible un enrutamiento eficiente. En particular, la idea fue enrutar en el campo *VPI*, pero no en el campo *VCI*, excepto en el salto final en cada dirección, cuando las células se envían entre un conmutador y un *host*. Entre dos conmutadores sólo debía usarse la trayectoria virtual.

El uso de sólo los *VPI* entre los conmutadores del interior tiene varias ventajas. Para comenzar, una vez que se ha establecido una trayectoria virtual entre un origen y un destino, cualquier circuito virtual adicional a lo largo de esa trayectoria simplemente puede seguir la trayectoria existente. No se tienen que tomar nuevas decisiones de enrutamiento. Es como si ya se hubiera tendido un haz de pares trenzados entre el origen y el destino. El establecimiento de una conexión nueva simplemente requiere la asignación de uno de los pares sin uso.

Segundo, el enrutamiento de células individuales es más fácil cuando todos los circuitos virtuales de una trayectoria dada están siempre en el mismo haz. La decisión de enrutamiento sólo implica examinar un número de 12 bits, no un número de 12 bits y otro de 16 bits. Describiremos más adelante la manera en que ocurre la conmutación de células pero, aun sin entrar en detalles, debe ser claro que es factible la indización en una tabla de 2^{12} entradas, pero que en una tabla de 2^{28} no lo es.

Tercero, basar todo el enrutamiento en trayectorias virtuales facilita la conmutación de un grupo completo de circuitos virtuales. Considere por ejemplo la hipotética columna vertebral ATM de Estados Unidos ilustrada en la figura 5-66. Normalmente los circuitos virtuales de Nueva York a San Francisco pasan a través de Omaha y Denver. Sin embargo, suponga que

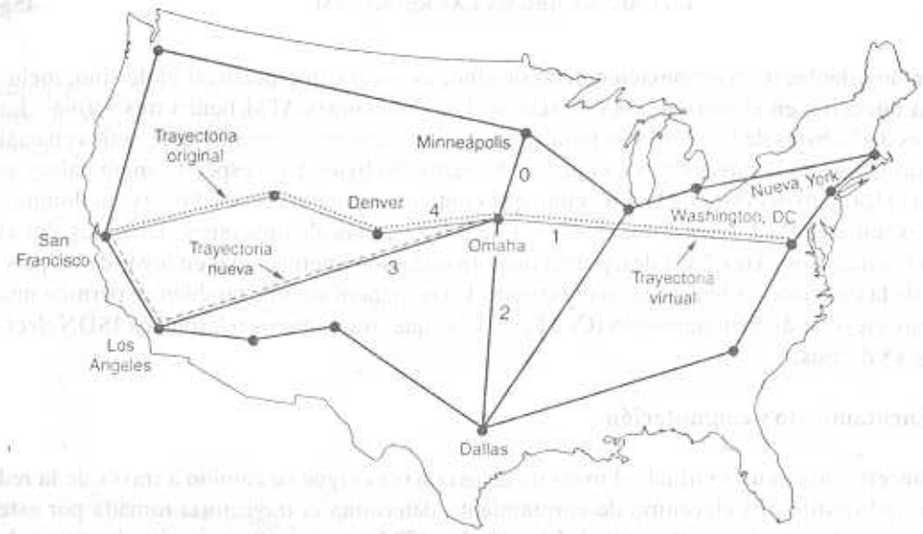


Figura 5-66. El reenrutamiento de una trayectoria virtual también reenruta todos sus circuitos virtuales.

ocurre una perturbación en la línea Omaha-Denver. Al reenrutar la trayectoria virtual Omaha-Denver a Los Ángeles y luego a San Francisco, pueden conmutarse todos los circuitos virtuales (potencialmente 65,535) en una sola operación, en lugar de miles de operaciones potenciales.

Por último, las trayectorias virtuales facilitan a las portadoras ofrecer grupos cerrados de usuarios (redes privadas) a los clientes corporativos. Una compañía puede establecer una red de trayectorias virtuales permanentes entre sus diferentes oficinas, y luego asignar circuitos virtuales entre estas trayectorias a solicitud. No pueden entrar llamadas en la red privada desde fuera, y ninguna llamada puede salir de la red privada, excepto a través de pasarelas especiales. A muchas compañías les gusta este tipo de seguridad.

Aún está por verse si los conmutadores realmente usarán el campo *VPI* para enrutamiento, como se planeó, o si usarán la combinación de los campos *VPI* y *VCI* (negando entonces todas las ventajas que acabamos de estudiar). La experiencia inicial en el campo no es alentadora.

Veamos ahora cómo se pueden enrutar células en un conmutador interno (uno que está conectado solamente a otros conmutadores, y no a ningún *host*). Para hacer concreto el planteamiento, considérese el conmutador de Omaha de la figura 5-66. Para cada una de sus cinco líneas de entrada tiene una tabla, *vpi_table*, indizada por los *VPI* de entrada, que indica cuál de las cinco líneas de salida debe usar y qué *VPI* debe poner en las células de salida. Supongamos que las cinco líneas tienen la numeración de 0 a 4, en dirección de las manecillas del reloj, comenzando por Minneápolis. Para cada línea de salida, el conmutador mantiene un mapa de bits que indica los *VPI* usados actualmente en esa línea.

Al activarse el conmutador, todas las entradas de todas las estructuras *vpi_table* se marcan como desocupadas. De la misma manera, todos los mapas de bits se marcan para indicar que

Origen	Línea de entrada	VPI de entrada	Destino	Línea de salida	VPI de salida	Trayectoria
Nueva York	1	1	San Francisco	4	1	Nueva
Nueva York	1	2	Denver	4	2	Nueva
Los Ángeles	3	1	Minneápolis	0	1	Nueva
Washington, DC	1	3	Los Ángeles	3	2	Nueva
Nueva York	1	1	San Francisco	4	1	Anterior
San Francisco	4	3	Washington, DC	1	4	Nueva
Washington, DC	1	5	San Francisco	4	4	Nueva
Nueva York	1	2	Denver	4	2	Anterior
San Francisco	4	5	Minneápolis	0	2	Nueva
Nueva York	1	1	San Francisco	4	1	Anterior

Figura 5-67. Algunas rutas a través del conmutador de Omaha de la figura 5-66.

están disponibles todos los *VPI* (excepto los reservados). Ahora supóngase que llegan llamadas, como se muestra en la figura 5-67.

A medida que se establecen las trayectorias virtuales (y circuitos virtuales), se hacen entradas en las tablas. Supondremos que los circuitos virtuales son dúplex integral, de modo que cada uno que se establece resulta en dos entradas, una para el tráfico directo del origen y otra para el tráfico en reversa del destino.

Las tablas que corresponden a las rutas de la figura 5-67 se muestran en la figura 5-68. Por ejemplo, la primera llamada genera la entrada (4, 1) para *VPI* 1 de la tabla de Washington, DC, porque se refiere a células que entran por la línea 1, con *VPI* 1 y con destino a San Francisco. Sin embargo, también se hace una entrada en la tabla de Denver para *VPI* 1 mostrando que las células que vienen de Denver con *VPI* 1 deben salir por la línea 1 con *VPI* 1. Estas son células que viajan en el otro sentido (de San Francisco a Nueva York) por esta trayectoria virtual. Nótese que en algunos casos dos o tres circuitos virtuales comparten una trayectoria común. No se necesitan entradas nuevas en las tablas para los circuitos virtuales adicionales que conecten un origen y un destino que ya tienen una trayectoria asignada.

Ahora podemos explicar la manera en que las células se procesan dentro de un conmutador. Supóngase que llega una célula por la línea 1 (Washington, DC) con *VPI* 3. El *hardware* o el *software* del conmutador usa el 3 como índice en la tabla de la línea 1 y ve que la célula debe salir por la línea 3 (Los Ángeles) con *VPI* 2; sobrescribe el campo *VPI* con un 2 y pone el número de la línea de salida, 3, en algún lugar de la célula, por ejemplo, en el campo *HEC* dado que, de todas maneras, éste tendrá que recalcularse más adelante.

Ahora la pregunta es cómo llevar la célula de su *buffer* de entrada actual a la línea 3. Sin embargo, este asunto (el enrutamiento dentro de un conmutador) se estudió detalladamente en el capítulo 2, y vimos la manera de hacerlo con conmutadores por *knockout* y Batcher-banianos.

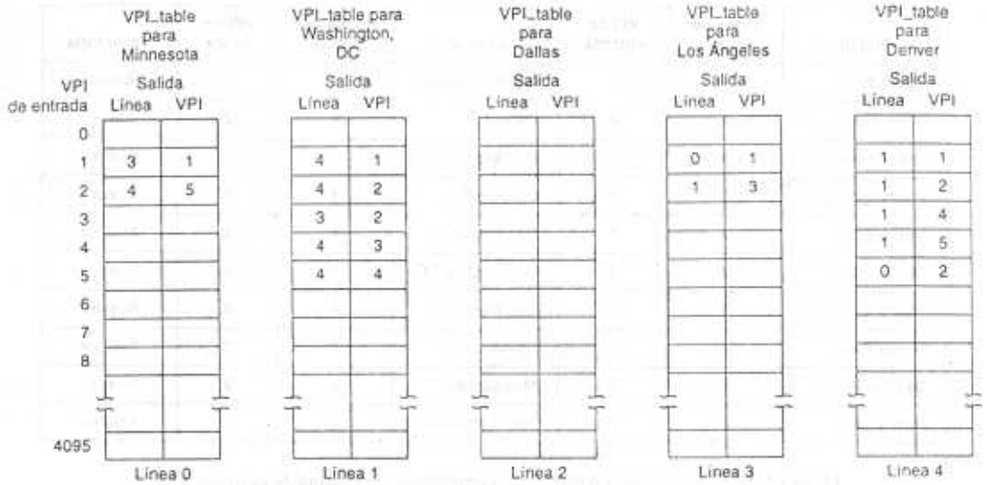


Figura 5-68. Entradas en las tablas para las rutas de la figura 5-67.

En este punto es fácil ver la manera en que puede reenrutarse un haz entero de circuitos virtuales, como se hace en la figura 5-66. Al cambiar la entrada de *VPI* 1 de la tabla de Washington, DC de (4, 1) a (3, 3), las células de Nueva York para San Francisco se desviarán a Los Ángeles. Por supuesto, hay que notificar este evento al conmutador de Los Ángeles, por lo que el conmutador tiene que generar y enviar un mensaje de ESTABLECER a Los Ángeles para establecer la trayectoria nueva con *VPI* 3. Una vez establecida esta trayectoria, todos los circuitos virtuales de Nueva York a San Francisco se redirigen a través de Los Ángeles, aun si hay miles de ellos. Si no existieran las trayectorias virtuales, cada circuito virtual tendría su propia entrada en la tabla y se reenrutaría por separado.

Vale la pena apuntar explícitamente que el análisis anterior se refiere al ATM en las WAN. En una LAN el asunto es mucho más sencillo. Por ejemplo, puede usarse una sola trayectoria virtual para todos los circuitos virtuales.

5.6.4. Categorías de servicios

Después de un buen tiempo de prueba y error, ya para la especificación 4.0 del ATM estaban claros los tipos de tráfico que llevaban las redes ATM y los tipos de servicios requeridos por el cliente. En consecuencia, el estándar se modificó para listar explícitamente las categorías de servicio comúnmente usadas, a fin de permitir a los proveedores de equipo optimizar sus tarjetas adaptadoras y los conmutadores para algunas o todas estas categorías. Las categorías de servicios seleccionadas como importantes se listan en la figura 5-69.

La clase **CBR** (*Constant Bit Rate*, **tasa de bits constante**) pretende simular un alambre de cobre o una fibra óptica (sólo que a un costo mucho mayor). Los bits se ponen en un extremo y

Clase	Descripción	Ejemplo
CBR	Tasa de bits constante	Circuito T1
RT-VBR	Tasa de bits variable: tiempo real	Videoconferencias en tiempo real
NRT-VBR	Tasa de bits variable: tiempo no real	Correo electrónico multimedia
ABR	Tasa de bits disponible	Consultas al Web
UBR	Tasa de bits no especificada	Transferencia de archivos en segundo plano

Figura 5-69. Categorías de servicios ATM.

salen por el otro. No hay comprobación de errores, control de flujo ni ningún otro proceso. No obstante, esta clase es esencial para hacer una transición suave entre el sistema telefónico actual y los sistemas B-ISDN del futuro, ya que los canales PCM de grado de voz, los circuitos T1 y la mayor parte del resto del sistema telefónico usa la transmisión de bits síncrona de tasa constante. Con la clase CBR puede transportarse directamente todo este tráfico a través de un sistema ATM. La CBR también es adecuada para todas las demás cadenas interactivas (es decir, en tiempo real) de audio y vídeo.

La siguiente clase, la **VBR** (*Variable Bit Rate*, **tasa variable de bits**), se divide en dos subclases, la de tiempo real (RT-VBR) y la de tiempo no real (NRT-VBR), respectivamente. La RT-VBR es para servicios que tienen tasas de bits variables en combinación con requisitos muy estrictos de tiempo real, como el vídeo comprimido interactivo (por ejemplo, videoconferencias). Debido a la manera en que funcionan el MPEG y otros esquemas de compresión, con un marco base completo seguido de una serie de diferencias entre el marco actual y el marco base, la tasa de transmisión varía intensamente en el tiempo (Pancha y El Zarki, 1994). A pesar de esta variación, es esencial que la red ATM no genere ninguna fluctuación en el patrón de llegada de las células, ya que esto causaría que la imagen apareciese inestable. En otras palabras, tanto el retardo medio de las células como la variación del retardo de las células deben estar cuidadosamente controlados. Por otra parte, aquí es tolerable un bit o célula perdido de vez en cuando y lo mejor es ignorarlo.

La otra subclase VBR es para tráfico en el que la entrega a tiempo es importante pero la aplicación puede tolerar una cierta cantidad de fluctuación. Por ejemplo, el correo electrónico multimedia normalmente se almacena temporalmente en el disco local del receptor antes de presentarse en pantalla, por lo que cualquier variación en los tiempos de entrega de las células se eliminaría antes de visualizarse el correo electrónico.

La categoría de servicio **ABR** (*Available Bit Rate*, **tasa de datos disponible**) se diseñó para trabajo en ráfagas cuya gama de ancho de banda se conoce aproximadamente. Un ejemplo típico podría ser su uso en una compañía que conecta sus oficinas mediante un conjunto de líneas arrendadas. Normalmente, la compañía puede seleccionar entre instalar suficiente capacidad para manejar la carga pico, lo que significa que algunas líneas están ociosas parte del día, o instalar la suficiente capacidad para la carga mínima, lo que conduce a congestionamientos durante la parte más atareada del día.

El uso del servicio ABR evita tener que comprometerse con un ancho de banda fijo. Con el ABR es posible decir, por ejemplo, que la capacidad entre dos puntos siempre será de 5 Mbps, pero puede tener picos de hasta 10 Mbps. El sistema garantizará entonces 5 Mbps todo el tiempo, y hará lo posible para proporcionar 10 Mbps cuando sea necesario, pero sin prometer nada.

La ABR es la única categoría de servicio en la que la red proporciona tasa de retroalimentación al transmisor, solicitándole que disminuya la velocidad al ocurrir congestionamientos. Suponiendo que el transmisor respeta tales solicitudes, se espera que la pérdida de células del tráfico ABR sea baja. Viajar por ABR es un poco como volar por lista de espera: si hay asientos sobrantes (sobrecapacidad), los pasajeros en lista de espera son transportados sin demora. Si hay capacidad insuficiente, tienen que esperar (a menos que esté disponible un poco del ancho de banda mínimo).

Por último llegamos a la UBR (*Unspecified Bit Rate*, **tasa de datos no especificada**), que no hace promesas y no realimenta información sobre los congestionamientos. Esta categoría se adapta bien al envío de paquetes IP, puesto que el IP tampoco hace promesas respecto a la entrega. Se aceptan todas las células UBR y, si sobra capacidad, también se entregan. Si ocurren congestionamientos, se descartan las células UBR sin realimentación al transmisor y sin esperar que el transmisor reduzca su velocidad.

Continuando con nuestra analogía de la lista de espera, con el UBR todos los pasajeros en lista de espera llegan a abordar, pero si a medio camino el piloto ve que se agota el combustible, los pasajeros en lista de espera son arrojados sin miramientos por la salida de emergencia. Para hacer atractiva la UBR, las portadoras tienden a hacerla más barata que las otras clases. Para aplicaciones que no tienen restricciones de entrega y quieren llevar ellas mismas su propio control de errores y de flujo, la UBR es una elección razonable. La transferencia de archivos, el correo electrónico y las noticias del USENET son candidatos potenciales para el servicio UBR, puesto que ninguna de estas aplicaciones tiene características de tiempo real.

Las propiedades de las diferentes categorías de servicio se resumen en la figura 5-70.

Característica de servicio	CBR	RT-VBR	NRT-VBR	ABR	UBR
Garantía de ancho de banda	Sí	Sí	Sí	Opcional	No
Adecuado para tráfico en tiempo real	Sí	Sí	No	No	No
Adecuado para tráfico en ráfagas	No	No	Sí	Sí	Sí
Realimentación sobre congestionamientos	No	No	No	Sí	No

Figura 5-70. Características de las categorías de servicio ATM.

5.6.5. Calidad del servicio

La calidad del servicio es un asunto importante para las redes ATM, en parte porque se usan para tráfico en tiempo real, como audio y vídeo. Al establecerse un circuito virtual, tanto la capa de transporte (por lo común un proceso de la máquina *host*, el "cliente") como la capa de red

ATM (por ejemplo, un operador de red, la "portadora") deben ponerse de acuerdo en un contrato que defina el servicio. En el caso de una red pública, este contrato debe tener implicaciones legales. Por ejemplo, si la portadora está de acuerdo en no perder más de una célula por cada mil millones y pierde dos células por cada mil millones, los abogados del cliente pueden alterarse y comenzar a gritar "¡incumplimiento de contrato!"

El contrato entre el cliente y la red tiene tres partes:

1. El tráfico que se generará.
2. El servicio acordado.
3. Los requisitos de cumplimiento.

Vale la pena apuntar que el contrato puede ser diferente para cada sentido. En una aplicación de vídeo a solicitud, el ancho de banda necesario para el control remoto del usuario al servidor de vídeo podría ser de 1200 bps. En el otro sentido podría ser de 5 Mbps. Cabe señalar que, si el cliente y la portadora no pueden acordar los términos, o si la portadora es incapaz de proporcionar el servicio deseado, no se establecerá el circuito virtual.

La primera parte del contrato es el **descriptor de tráfico** que caracteriza la carga que se generará. La segunda parte del contrato especifica la calidad del servicio deseada por el cliente y aceptada por la portadora. Tanto la carga como el servicio deben formularse en términos de cantidades medibles, para que el cumplimiento pueda determinarse objetivamente. Decir simplemente "carga moderada" o "buen servicio" no es suficiente.

Para hacer posible tener contratos de tráfico concretos, el estándar ATM define una serie de parámetros de **QoS** (*Quality of Service*, **calidad de servicio**) cuyos valores pueden negociar el cliente y la portadora. Para cada parámetro de calidad de servicio se especifica el desempeño de peor caso y se requiere que la portadora lo cumpla o lo mejore. En algunos casos, el parámetro es un mínimo; en otros es un máximo. Nuevamente, la calidad del servicio se especifica por separado para cada sentido; algunas de las más importantes se listan en la figura 5-71, pero no todas aplican a todas las categorías de servicio.

Los primeros tres parámetros especifican la rapidez a la que quiere transmitir el usuario. La **PCR** (*Peak Cell Rate*, **tasa de células pico**) es la rapidez máxima con que el transmisor planea enviar células. Este parámetro puede ser menor que lo permitido por el ancho de banda de la línea. Si el transmisor planea sacar células cada 4 µseg, su *PCR* es de 250,000 células/seg, aun si el tiempo real de transmisión de células puede ser de 2.7 µseg.

La **SCR** (*Sustained Cell Rate*, **tasa de células sostenida**) es la tasa esperada o requerida de células promediada en un intervalo de tiempo grande. Para el tráfico CBR, la *SCR* será igual a la *PCR*, pero para las demás categorías de servicio será sustancialmente menor. La razón *PCR/SCR* es una medida de las ráfagas del tráfico.

La **MCR** (*Minimum Cell Rate*, **tasa de células mínima**) es la cantidad mínima de células/seg que el cliente considera aceptable. Si la portadora es incapaz de garantizar esta cantidad de ancho de banda, debe rechazar la conexión. Cuando se solicita el servicio ABR, el ancho de banda real debe estar entre la *MCR* y la *PCR*, pero puede variar dinámicamente durante el

Parámetro	Siglas	Significado
Tasa de células pico	PCR	Tasa máxima a la que se enviarán células
Tasa de células sostenida	SCR	Tasa de células promedio a largo plazo
Tasa de células mínima	MCR	La tasa de células mínima aceptable
Tolerancia de variación de retardo de celulas	CDVT	La fluctuación de células máxima aceptable
Tasa de pérdidas de células	CLR	Fracción de las células perdidas o entregadas demasiado tarde
Retardo de transferencia de células	CTD	El tiempo que se lleva la entrega (medio y máximo)
Variación de retardo de células	CDV	La variación en los tiempos de entrega de células
Tasa de errores de células	CER	Fracción de las células entregadas sin error
Proporción de bloques de células con errores severos	SECBR	Fracción de los bloques alterados
Tasa de mala inserción de células	CMR	Fracción de las células entregadas a un destino equivocado

Figura 5-71. Algunos de los parámetros de calidad de servicio.

tiempo de vida de la conexión. Si el cliente y la portadora acuerdan establecer la *MCR* en 0, entonces el servicio ABR se vuelve similar al servicio UBR.

La **CVDT** (*Cell Variation Delay Tolerance*, **tolerancia de variación de retardo de células**) indica la cantidad de variación que habrá en los tiempos de transmisión de las células; se especifica independientemente para la *PCR* y la *SCR*. Con un origen perfecto que opera a la *PCR*, cada célula aparecerá *exactamente* 1/*PCR* después de la previa. Ninguna célula llegará temprano y ninguna llegará tarde, ni siquiera un picosegundo. Para un origen real que opera a la *PCR*, habrá un poco de variación en los tiempos de transmisión de las células. La pregunta es: ¿cuánta variación es aceptable? ¿Puede llegar una célula 1 nseg antes? ¿Qué tal 30 segundos? La *CDVT* controla la cantidad de variabilidad aceptable usando un algoritmo de cubeta con goteo que describiremos pronto.

Los siguientes tres parámetros describen características de la red y se miden en el receptor. Los tres son negociables. La **CLR** (*Cell Loss Ratio*, **tasa de pérdida de células**) es autoexplicativa. Mide la fracción de las células transmitidas que no se entregan en absoluto o que se entregan demasiado tarde, siendo inservibles por lo mismo (por ejemplo, en el tráfico en tiempo real). El **CTD** (*Cell Transfer Delay*, **retardo de transferencia de células**) es el tiempo promedio de tránsito del origen al destino. La **CDV** (*Cell Delay Variation*, **variación de retardo de células**) mide la uniformidad con que se entregan las células.

El modelo de *CTD* y *CDV* se muestra en la figura 5-72. Aquí vemos la probabilidad de que una célula tarde un tiempo t en llegar, en función de t. Para un origen, destino y ruta a través de los conmutadores intermedios dados, siempre hay un retardo mínimo debido al tiempo de propagación y de conmutación. Sin embargo, no todas las células llegan en el tiempo mínimo; la función de densidad de probabilidad por lo general tiene una cola grande. Escogiendo un valor de *CTD*, el cliente y la portadora de hecho acuerdan la tardanza con la que puede entregarse una célula y aún considerarla como correctamente entregada. Normalmente la *CDV* se escogerá de manera que α, la fracción de células rechazadas por llegar demasiado tarde, sea del orden

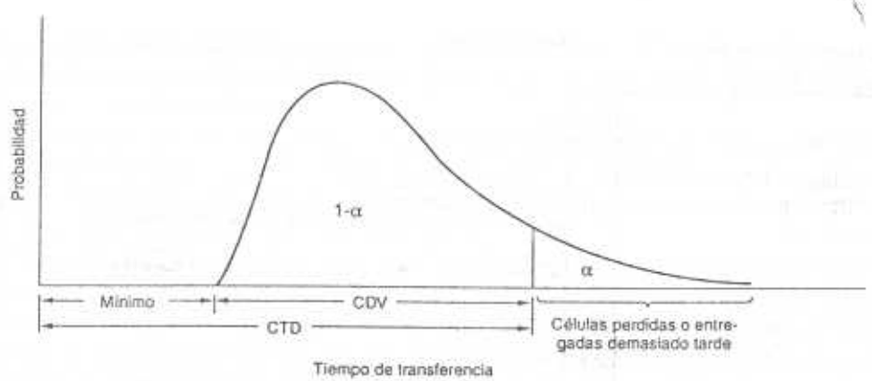


Figura 5-72. La función de densidad de probabilidad para los tiempos de llegada de las células.

de 10^{-10} o menos. La *CDV* mide la distribución de tiempos de entrega. Para el tráfico en tiempo real, este parámetro con frecuencia es más importante que el *CTD*.

Los últimos tres parámetros QoS especifican características de la red, y generalmente no son negociables. La **CER** (*Cell Error Ratio*, **razón de errores de células**) es la fracción de las células que se entregan con uno o más bits equivocados. La **SECBR** (*Severely-Errored Cell Block Ratio*, **tasa de bloques de células con errores severos**) es la fracción de bloques de N células en los que M o más células contienen errores. Por último, la **CMR** (*Cell Misinsertion Rate*, **tasa de mala inserción de células**) es la cantidad de células/seg que se entregan al destino equivocado debido a un error no detectado en la cabecera.

La tercera parte del contrato de tráfico indica lo que constituye el cumplimiento de las reglas. Si el cliente envía una célula demasiado pronto, ¿invalida esto el contrato? Si la portadora no alcanza a cumplir uno de los objetivos de calidad durante un periodo de 1 mseg, ¿puede interponer una demanda el cliente? De hecho, esta parte del contrato se negocia entre las partes y dice qué tan estrictamente se harán cumplir las primeras dos partes.

Los modelos de calidad de servicio del ATM y de Internet son algo diferentes, lo que tiene un impacto en sus respectivas puestas en práctica. El modelo ATM se basa estrictamente en las conexiones, mientras que el modelo de Internet usa datagramas más flujos (por ejemplo, RSVP). En (Crowcroft *et al.*, 1995) se hace una comparación entre estos dos modelos.

5.6.6. Conformación y vigilancia de tráfico

El mecanismo para usar y hacer cumplir los parámetros de calidad de servicio se basa (en parte) en un algoritmo específico, el algoritmo genérico de tasa de células (**GCRA, Generic Cell Rate Algorithm**), y se ilustra en la figura 5-73. Este algoritmo funciona revisando cada célula para ver si se ajusta a los parámetros de su circuito virtual.

El GCRA tiene dos parámetros. Estos especifican la tasa de llegada máxima permitida (*PCR*) y la cantidad de variación de ésta que puede tolerarse (*CDVT*). El recíproco de la *PCR*,

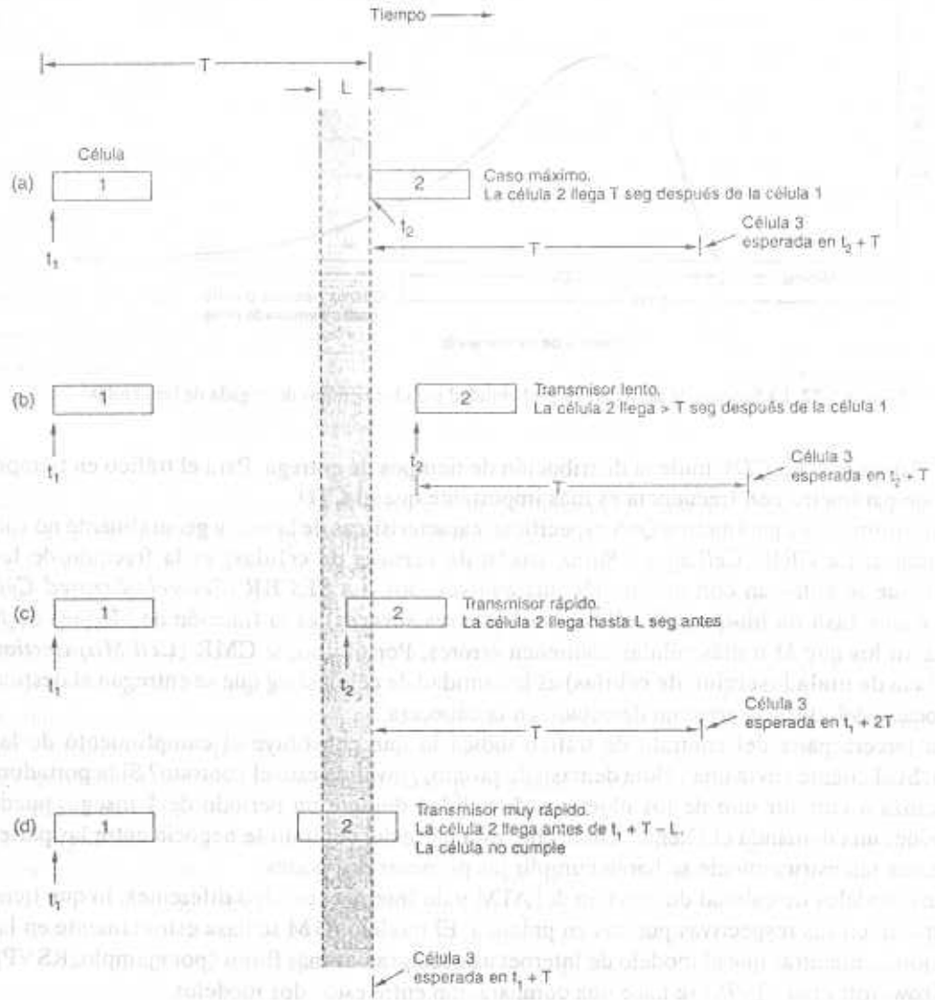


Figura 5-73. El algoritmo genérico de tasa de células.

$T = 1/PCR$, es el tiempo mínimo entre arribo de células, como se muestra en la figura 5-37(a). Si el cliente está de acuerdo en no enviar más de 100,000 células/seg, entonces $T = 10$ µseg. En el caso máximo, llega una célula puntualmente cada 10 µseg. Para evitar los números muy pequeños, trabajaremos con microsegundos, pero dado que todos los parámetros son números reales, la unidad de tiempo no importa.

Siempre se permite a un transmisor espaciar células consecutivas a intervalos mayores que T, como se muestra en la figura 5-73(b). Cualquier célula que llegue más de T µseg después de la previa cumple con lo establecido.

El problema surge cuando los transmisores que tienden a salir en falso, como en la figura 5-73(c) y (d). Si una célula llega un poco antes (a $t_1 + T - L$, o después), cumple, pero se espera la siguiente célula a $t_1 + 2T$, (no a $t_2 + T$), para evitar que el transmisor envíe cada célula L µseg antes, y por tanto aumente la tasa de células pico.

Si llega una célula más de L mseg antes, se declara como incumplidora. El tratamiento de las células incumplidoras es responsabilidad de la portadora. Algunas portadoras simplemente las descartan; otras las guardan, pero establecen el bit *CLP* para marcarlas como de baja prioridad, a fin de permitir a los conmutadores que descarten las células incumplidoras antes en el caso de un congestionamiento. El uso del bit *CLP* también puede ser diferente según las diferentes categorías de servicio de la figura 5-69.

Consideremos ahora lo que ocurre si un transmisor trata de hacer un poco de trampa, como se muestra en la figura 5-74(a). En lugar de esperar hasta el momento T para enviar la célula 2, el

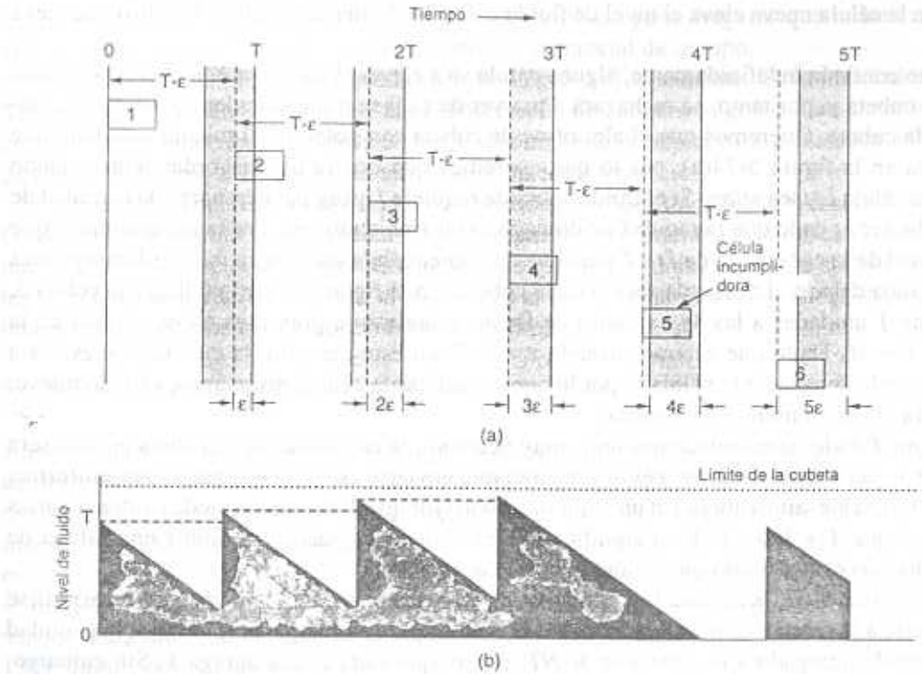


Figura 5-74. (a) Transmisor tratando de hacer trampa. (b) Mismo patrón de llegada de células, pero visto ahora como una cubeta con goteo.

transmisor la envía un poco antes, a $T - \varepsilon$, donde, digamos, $\varepsilon = 0.3L$. Esta célula se acepta sin problemas.

Ahora el transmisor envía la célula 3, nuevamente a $T - \varepsilon$ después de la célula previa, es decir, a $T - 2\varepsilon$. Nuevamente se acepta la célula. Sin embargo, cada célula posterior se acerca más y más al límite fatal $T - L$. En este caso, la célula 5 llega a $T - 4\varepsilon$ ($T - 1.2L$), que es demasiado pronto, por lo que se declara incumplidora y es descartada por la interfaz de la red.

Visto en estos términos, el algoritmo GCRA es un **algoritmo de programación virtual**; sin embargo, visto de otra manera, equivale a un algoritmo de cubeta con goteo, como se observa en la figura 5-74(b). Imagine que cada célula cumplidora que llega vierte T unidades de fluido en una cubeta con goteo. La cubeta gotea el fluido a razón de 1 unidad/mseg, por lo que tras T µseg ha salido todo. Si las células llegan precisamente cada T µseg, cada una que llegue encontrará recién vaciada la cubeta y la rellenará con T unidades de fluido. Por tanto, el nivel de fluido aumenta a T cuando llega una célula, y se reduce linealmente hasta llegar a cero. Esta situación se ilustra en la figura 5-74(b) entre 0 y T.

Dado que el fluido se drena linealmente con el tiempo, en un tiempo t tras la llegada de una célula, la cantidad que queda de su fluido es de $T - t$. En el momento que llega la célula 2, en $T - \varepsilon$, aún hay e unidades de fluido en la cubeta. La llegada de la célula nueva eleva este valor a $T + \varepsilon$. Del mismo modo, en el momento que llega la célula 3, quedan 2ε unidades en la cubeta, por lo que la célula nueva eleva el nivel de fluido a $T + 2\varepsilon$. Al llegar la célula 4, el nivel se eleva a $T + 3\varepsilon$.

Si esto continúa indefinidamente, alguna célula va a elevar el nivel por encima de la capacidad de la cubeta y, por tanto, se rechazará. Para ver de cuál se trata, calculemos ahora la capacidad de la cubeta. Queremos que el algoritmo de cubeta con goteo dé el mismo resultado que se observa en la figura 5-74(a), por lo que queremos que ocurra un desbordamiento cuando llegue una célula L µseg antes. Si el fluido sobrante requiere L µseg para drenarse, la cantidad de fluido debe ser L, dado que la rapidez de drenado es de 1 unidad/µseg. Por tanto, queremos que la capacidad de la cubeta sea de $L + T$ para que cualquier célula que llegue más de L mseg antes sea rechazada debido al desbordamiento de la cubeta. En la figura 5-74(b), al llegar la célula 5, la suma de T unidades a las 4ε unidades de fluido actualmente presentes eleva el nivel de la cubeta a $T + 4\varepsilon$. Dado que estamos usando $\varepsilon = 0.3L$ en este ejemplo, la cubeta se elevaría a $T + 1.2L$ por la llegada de la célula 5, por lo que se rechaza la célula, no se agrega fluido nuevo, y la cubeta en algún momento se vacía.

Para un T dado, si establecemos un L muy pequeño, la capacidad de la cubeta apenas será mayor de T, por lo que todas las células tendrán que enviarse con un espaciado muy uniforme. Sin embargo, si elevamos ahora L a un valor mucho mayor que T, la cubeta puede contener varias células, porque $T + L \gg T$. Esto significa que el transmisor puede transmitir una ráfaga de células una tras otra a la tasa pico y aun lograr su aceptación.

Podemos calcular fácilmente la cantidad de células cumplidoras, N, que pueden transmitirse una tras otra a la tasa de células pico ($PCR = 1/T$). Durante una ráfaga de N células, la cantidad total de fluido agregado a la cubeta es de NT, puesto que cada célula agrega T. Sin embargo, durante la ráfaga máxima, el fluido se drena de la cubeta a una razón de 1 unidad por intervalo de tiempo. Digamos que el tiempo de transmisión de célula es de δ unidades de tiempo. Nótese que

$\delta \leq T$ porque es perfectamente posible que un transmisor en una línea de 155.52 Mbps acuerde enviar no más de 100,000 células/seg, en cuyo caso $\delta = 2.73$ µseg y $T = 10$ µseg. Durante la ráfaga de N células, la cantidad de drenado es de $(N - 1)\delta$ porque el drenado no comienza hasta haberse completado la transmisión de la primera célula.

A partir de estas observaciones, establecemos que el incremento neto del fluido de la cubeta durante la ráfaga máxima es de $NT - (N - 1)\delta$. La capacidad de la cubeta es de $T + L$. Igualando estas dos cantidades obtenemos

$$NT - (N - 1)\delta = T + L$$

Si despejamos N de esta ecuación obtenemos

$$N = 1 + \frac{L}{T - \delta}$$

Sin embargo, si este número no es un entero, debe redondearse al entero menor más cercano para evitar el desbordamiento de la cubeta. Por ejemplo, con $PCR = 100{,}000$ células/seg, $\delta = 2.73$ µseg y $L = 50$ µseg, pueden enviarse siete células una tras otra a razón de 155.52 Mbps sin llenar la cubeta. Una octava célula sería incumplidora.

El GCRA normalmente se especifica dando los parámetros T y L. T simplemente es el recíproco de PCR; L es $CDVT$. El GCRA también se usa para asegurar que la tasa media de células no exceda la SCR durante ningún periodo sustancial de tiempo.

En este ejemplo, supusimos que las células llegan uniformemente. En la realidad no es así. Sin embargo, también puede usarse aquí el algoritmo de cubeta con goteo. Con cada llegada de célula, se hace una comprobación para ver si hay lugar en la cubeta para T unidades adicionales de fluido. Si lo hay, la célula es cumplidora; de otra manera no.

Además de proporcionar una regla respecto a las células cumplidoras y las que no lo son, el GCRA también estructura el tráfico a modo de eliminar una parte de las ráfagas. A menor $CDVT$, mayor efecto de emparejamiento, pero también mayor oportunidad de que las células se descarten como no cumplidoras. Algunas implementaciones combinan la cubeta con goteo GCRA con una cubeta con fichas para proporcionar mayor emparejamiento.

5.6.7. Control de congestionamientos

Aun con la conformación de tráfico, las redes ATM no cumplen automáticamente los requisitos de desempeño establecidos en el contrato de tráfico. Por ejemplo, el congestionamiento en los conmutadores intermedios siempre es un problema potencial, especialmente cuando circulan más de 350,000 células/seg por cada línea, y un conmutador puede tener 100 líneas. En consecuencia, se ha invertido una buena cantidad de esfuerzo para estudiar el asunto del desempeño y el congestionamiento de las redes ATM. En esta sección veremos algunos de los enfoques usados. Para información adicional, véase (Eckberg, 1992; Eckberg *et al.*, 1991; Hong y Suda, 1991; Jain, 1995, y Newman, 1994).

Las redes ATM deben manejar tanto el congestionamiento a largo plazo, causado por la generación de más tráfico del que puede manejar el sistema, como el congestionamiento a corto

plazo, causado por las ráfagas del tráfico. Por ello, se usan varias estrategias diferentes al mismo tiempo. Las más importantes de éstas pertenecen a tres categorías:

1. Control de admisión.
2. Reservación de recursos.
3. Control de congestionamiento basado en la tasa.

Ahora estudiaremos cada una de estas estrategias.

Control de admisión

En las redes de baja velocidad, generalmente es adecuado esperar a que ocurra un congestionamiento y reaccionar a él indicando al origen de los paquetes que reduzca su velocidad. En las redes de alta velocidad, este enfoque con frecuencia funciona mal, pues en el intervalo entre el envío de la notificación y la llegada de la notificación al origen pueden llegar miles de paquetes adicionales.

Además, muchas redes ATM tienen orígenes de tráfico en tiempo real que producen datos con una tasa intrínseca. Indicar a una de origen que reduzca su velocidad tal vez no funcione (imagine un teléfono digital nuevo con una luz roja: al señalarse un congestionamiento, la luz roja se enciende y se requiere que el que habla lo haga 25% más despacio).

En consecuencia, las redes ATM hacen hincapié en evitar que ocurra el congestionamiento. Sin embargo, para el tráfico CBR, VBR y UBR no hay ningún control de congestionamiento en absoluto, por lo que un poco de prevención vale más que una buena dosis de medicina. Una herramienta principal para evitar el congestionamiento es el control de admisión. Cuando un *host* quiere un circuito virtual nuevo, debe describir el tráfico que va a generar y el servicio esperado. La red puede entonces verificar si es posible manejar esta conexión sin afectar adversamente las conexiones existentes. Tendrán que examinarse varias rutas potenciales para encontrar una que pueda llevar a cabo la tarea. Si no puede localizarse ninguna ruta, se rechaza la llamada.

El rechazo de admisión debe hacerse de manera justa. ¿Es justo que un zángano que cambia continuamente el canal de su televisor pueda dejar fuera a 100 individuos ocupados que tratan de leer su correo electrónico? Si no se aplican controles, una cantidad pequeña de usuarios con gran ancho de banda puede afectar severamente a muchos usuarios con bajo ancho de banda. Para evitar esto, los usuarios deben dividirse en clases con base en el uso. La probabilidad de negación de servicio debe ser aproximadamente igual para todas las clases (posiblemente dando a cada clase su propio grupo de recursos).

Reservación de recursos

Estrechamente relacionada con el control de admisión está la técnica de reservación de recursos por adelantado, generalmente en el momento de establecimiento de la llamada. Dado que el descriptor de tráfico indica la tasa de células pico, la red tiene la posibilidad de reservar suficiente ancho de banda a lo largo de la trayectoria para manejar esa tasa. El ancho de banda puede reservarse haciendo que el mensaje de ESTABLECER reserve ancho de banda a lo largo de cada

línea que atraviesa, asegurándose por supuesto de que el ancho de banda total reservado a través de una línea sea menor que la capacidad de esa línea. Si el mensaje de ESTABLECER llega a una línea llena, debe dar marcha atrás y buscar una trayectoria alterna.

El descriptor de tráfico puede contener no sólo el ancho de banda pico, sino también el ancho de banda medio. Si, por ejemplo, un *host* quiere un ancho de banda pico de 100,000 células/seg, pero un ancho de banda medio de sólo 20,000 células/seg, en principio pueden multiplexarse cinco de tales circuitos en la misma troncal física. El problema es que las cinco conexiones podrían estar ociosas durante media hora, y luego comenzar a trabajar a la tasa pico, causando pérdidas masivas de células. Dado que el tráfico VBR puede multiplexarse estadísticamente, pueden ocurrir problemas con esta categoría de servicio. Se están estudiando posibles soluciones.

Control de congestionamiento basado en la tasa

Con el tráfico CBR y VBR, generalmente no es posible que el transmisor disminuya su velocidad, aun ante un congestionamiento, debido a la naturaleza de tiempo real o tiempo semirreal inherente del origen de información. Con la UBR a nadie le importa; si hay demasiadas células, las sobrantes se descartan.

Sin embargo, con el tráfico ABR es posible y razonable que la red señale a uno o más transmisores y les solicite disminuir su velocidad temporalmente hasta que la red se pueda recuperar. Es del interés de los transmisores el cumplimiento, ya que la red siempre puede castigarlos deshaciéndose de su exceso de células.

La manera de detectar, señalar y controlar el congestionamiento del tráfico ABR fue un tema importante durante el desarrollo del estándar ATM, con argumentos intensos a favor de varias soluciones propuestas. Ahora veremos brevemente algunas de las soluciones rechazadas, antes de examinar el ganador.

En una propuesta, cuando un transmisor deseaba enviar una ráfaga de datos, primero tenía que enviar una célula especial para reservar el ancho de banda necesario. Tras el regreso del acuse de recibo, podía comenzar la ráfaga. Aquí la ventaja es que el congestionamiento nunca ocurre, pues el ancho de banda requerido siempre está ahí cuando se necesita. El foro ATM rechazó esta solución debido al gran retardo potencial antes de poder transmitir.

Una segunda propuesta tenía conmutadores que enviaban de regreso células de estrangulamiento cada vez que comenzaba un congestionamiento. Al recibirse una de tales células, se esperaba que el transmisor redujera a la mitad su tasa de transmisión de células. Se propusieron varios esquemas para volver a elevar la tasa después, al despejarse el congestionamiento. Este esquema se rechazó porque las células de estrangulamiento podrían perderse en el congestionamiento, y porque el esquema parecía injusto para los usuarios pequeños. Por ejemplo, considérese un conmutador que recibe corrientes de 100 Mbps de cada uno de cinco usuarios, y una corriente de 100 kbps de otro usuario. Muchos miembros del comité sintieron que era inadecuado indicar al usuario de 100 kbps que cediera 50 kbps porque él causaba demasiado congestionamiento.

Una tercera propuesta aprovechaba el hecho de que los límites de los paquetes se marcan mediante un bit en la última célula. La idea era descartar células para aliviar el congestionamiento,

pero hacer esto de manera altamente selectiva. El conmutador debía barrer la corriente de células de entrada para encontrar el final de un paquete, y luego descartar todas las células del siguiente paquete. Por supuesto, este paquete podría transmitirse después, pero descartar todas las células k de un paquete a fin de cuentas conduce a la retransmisión de un solo paquete, lo que es mucho mejor que descartar k células al azar, asunto que puede dar pie a k retransmisiones de paquetes. Este esquema se rechazó por consideraciones de justicia, puesto que la siguiente marca de fin de paquete encontrada podría no pertenecer al transmisor causante de la carga extra del conmutador. Además el esquema no necesitaba estandarizarse. Cualquier proveedor de conmutadores estaría en libertad de escoger las células a descartar al ocurrir un congestionamiento.

Tras mucho debate, la batalla se enfocó en dos contendientes, una solución basada en crédito (Kung y Morris, 1995) y una basada en tasa (Bonomi y Fendick, 1995). La solución basada en crédito esencialmente era un protocolo dinámico de ventana corrediza. Requería que cada conmutador mantuviera, por cada circuito virtual, un crédito (de hecho, la cantidad de *buffers* reservados para ese circuito). Mientras cada célula transmitida tuviera un *buffer* esperándola, nunca podría surgir un congestionamiento.

El argumento en contra vino de los proveedores de conmutadores. No querían encargarse de la contabilidad para llevar el registro de los créditos y no querían reservar demasiados *buffers* por adelantado. La cantidad de carga extra y desperdicio requeridos se consideró muy grande, por lo que se adoptó el esquema de control de congestionamiento basado en tasa, que funciona como sigue.

El modelo básico es que, tras cada k células de datos, cada transmisor envía una célula especial **RM** (*Resource Management*, **administración de recursos**). Esta célula viaja por la misma trayectoria que las células de datos, pero recibe un tratamiento especial de los conmutadores a lo largo del camino. Al llegar al destino, la célula se examina, actualiza y envía de regreso al transmisor. La trayectoria completa de las células RM se muestra en la figura 5-75.

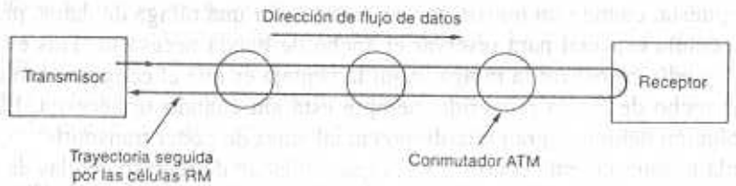


Figura 5-75. Trayectoria seguida por las células RM.

Además, se proporcionan otros dos mecanismos de control de congestionamientos. Primero, los conmutadores carga extrados pueden generar espontáneamente células RM y enviarlas de regreso al transmisor. Segundo, los conmutadores carga extrados pueden establecer el bit *PTI* central de las células de datos que viajan del transmisor al receptor. Sin embargo, ninguno de estos métodos es completamente confiable, pues estas células pueden perderse en el congestionamiento sin que nadie se dé cuenta. En cambio, una célula RM perdida será notada

por el transmisor cuando no regrese en el intervalo de tiempo esperado. Como nota al margen, el bit *CLP* no se usa para el control de congestionamientos.

El control de congestionamientos ABR se basa en la idea de que cada transmisor tiene una tasa actual, la **ACR** (*Actual Cell Rate*, **tasa de células real**), que está entre la *MCR* y la *PCR*. Al ocurrir un congestionamiento, la *ACR* se reduce (pero no por debajo de la *MCR*). Si no hay congestionamientos, la *ACR* se aumenta (pero no por encima de la *PCR*). Cada célula RM enviada contiene la tasa a la cual le gustaría transmitir al transmisor, posiblemente a la *PCR*, posiblemente a una menor. Este valor se llama **ER** (*Explicit Rate*, **tasa explícita**). A medida que la célula RM pasa a través de varios conmutadores en su camino al receptor, aquellos que están congestionados pueden reducir la *ER*. Ningún conmutador la puede aumentar. La reducción puede ocurrir tanto en sentido directo como en el inverso. Cuando el transmisor recibe la célula RM, puede ver la tasa mínima aceptable según todos los conmutadores a lo largo de la trayectoria. Entonces puede ajustar la *ACR*, de ser necesario, de acuerdo con lo que el conmutador más lento puede manejar.

El mecanismo de congestionamiento que usa el bit *PTI* central se integra en las células RM al hacer que el receptor incluya este bit (tomado de la última célula de datos) en cada célula RM enviada de regreso. El bit no puede tomarse de la célula RM misma porque todas las células RM tienen establecido este bit todo el tiempo, como se muestra en la figura 5-63.

La capa ATM es bastante complicada. En este capítulo sólo hemos destacado una parte de los temas. Para información adicional, véase (De Prycker, 1993; McDysan y Spohn, 1995; Minoli y Vitella, 1994, y La Porta *et al.*, 1994). Sin embargo, se advierte al lector que todas estas referencias estudian el estándar ATM 3, no el ATM 4, que no se terminó sino hasta 1996.

5.6.8. LAN ATM

A medida que se hace más obvio que la meta original del ITU de reemplazar la red telefónica pública conmutada por una red ATM se llevará mucho tiempo, la atención se está centrando en el uso de la tecnología ATM para conectar entre sí las LAN existentes. En este enfoque, una red ATM puede funcionar como LAN, conectando *hosts* individuales, o como puente, conectando varias LAN. Aunque ambos conceptos son interesantes, generan algunos problemas complicados que estudiaremos a continuación. Puede encontrarse información adicional acerca de las LAN ATM en (Chao *et al.*, 1994; Newman, 1994, y Truong *et al.*, 1995).

El problema principal que debe resolverse es cómo proporcionar un servicio de LAN sin conexiones a través de una red ATM orientada a conexiones. Una posible solución es introducir un servidor sin conexiones en la red. Cada *host* establece inicialmente una conexión con este servidor, y envía a él todos los paquetes para que los reenvíe. Aunque es sencilla, esta solución no aprovecha todo el ancho de banda de la red ATM, y el servidor sin conexiones fácilmente puede volverse un cuello de botella.

Un enfoque alterno, propuesto por el foro ATM, se muestra en la figura 5-76. Aquí cada *host* tiene circuitos virtuales ATM (potenciales) a todos los demás *hosts*. Estos circuitos virtuales pueden establecerse y liberarse dinámicamente según se necesite, o pueden ser circuitos virtuales permanentes. Para enviar un marco, el *host* de origen encapsula primero el paquete en el campo

de carga útil de un mensaje AAL ATM, y lo envía al destino, de la misma manera en que se transmiten marcos a través de Ethernet, *token ring* y otras LAN.

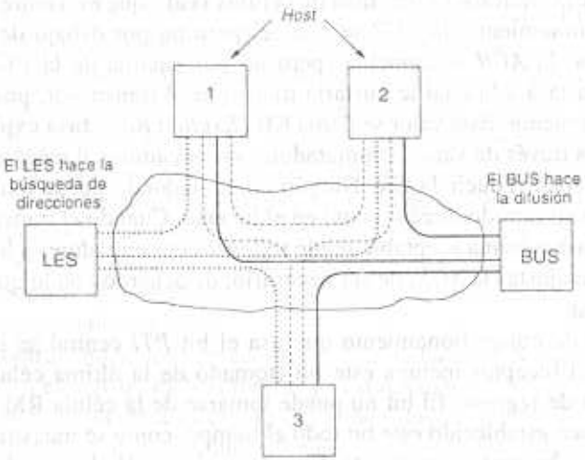


Figura 5-76. Simulación ATM de una LAN.

El problema principal generado por este esquema es cómo determinar los circuitos virtuales a los que pertenecen las direcciones de IP (u otras direcciones de capa de red). En una LAN 802 este problema se resuelve mediante el protocolo ARP, en el que un *host* puede difundir una solicitud como: "¿quién tiene la dirección de IP 192.31.20.47?" El *host* que usa esa dirección devuelve entonces una respuesta punto a punto, que se almacena en caché para su uso futuro.

Con una LAN ATM esta solución no funciona, pues las LAN ATM no manejan la difusión. Este problema se resuelve introduciendo un servidor nuevo, el **LES** (*LAN Emulation Server*, **servidor de simulación de LAN**). Para buscar una dirección de capa de red (por ejemplo, una dirección IP), un *host* envía un paquete (por ejemplo, una solicitud ARP) al LES, que entonces busca la dirección ATM correspondiente y la devuelve a la máquina que la solicita. Esta dirección entonces puede usarse para enviar paquetes encapsulados al destino.

Sin embargo, esta solución sólo resuelve el problema de localización del *host*. Algunos programas usan la difusión o la multitransmisión como parte esencial de la aplicación. Para estas aplicaciones, se introduce el **BUS** (*Broadcast/Unknown Server*, **servidor de difusión/desconocido**), que tiene conexiones a todos los *hosts* y puede simular la difusión enviando un paquete a todos ellos, uno por uno. Los *hosts* también pueden acelerar la entrega de un paquete a un *host* desconocido enviando el paquete al BUS para difundirlo y luego (en paralelo) buscar la dirección (para uso futuro) usando el LES.

Un modelo semejante a éste ha sido adoptado por el IETF como la manera oficial en que Internet usa una red ATM para transportar paquetes IP. En este modelo, el servidor LES se llama

servidor **ATMARP**, pero la funcionalidad es esencialmente la misma. La difusión y multitransmisión no se contemplan en la propuesta IETF. El modelo se describe en el RFC 1483 y el RFC 1577. Otra buena de origen de información es (Comer, 1995).

En el método del IETF puede agruparse un conjunto de *hosts* ATM para formar una **subred IP lógica** (LIS, *logical IP subnet*). Cada LIS tiene su propio servidor ATMARP. En efecto, una LIS actúa como una subred LAN virtual. Los *hosts* de la misma LIS pueden intercambiar directamente paquetes IP, pero los *hosts* de LIS diferentes deben pasar a través de un enrutador. La razón de tener las LIS es que cada *host* de una LIS debe (potencialmente) tener circuitos virtuales abiertos hacia todos los demás *hosts* de su LIS. Al restringir la cantidad de *hosts* por LIS, se puede reducir la cantidad de circuitos virtuales abiertos a un número manejable.

Otro uso de las redes ATM es como puentes para conectar varias LAN existentes. En esta configuración, sólo una máquina de cada LAN necesita una conexión ATM. Al igual que todos los puentes transparentes, el puente ATM debe escuchar en modo promiscuo a todas las LAN a las que está conectado, reenviando los marcos a donde se necesiten. Dado que los puentes sólo usan direcciones de MAC (no de IP), los puentes ATM deben construir un árbol de extensión, al igual que los puentes 802.

En resumen, si bien la simulación LAN del ATM es un concepto interesante, hay cuestionamientos serios sobre su desempeño y precio, y ciertamente enfrentan una fuerte competencia de las LAN y los puentes existentes, bien establecidos y optimizados. El que las LAN ATM reemplacen en algún momento a las LAN y puentes 802 está por verse.

5.7. RESUMEN

La capa de red proporciona servicios a la capa de transporte; puede basarse tanto en circuitos virtuales como en datagramas. En ambos casos, la tarea principal de esta capa es enrutar paquetes del origen al destino. En las subredes de circuitos virtuales se toma una decisión de enrutamiento al establecerse el circuito virtual; en las subredes de datagramas, se hace con cada paquete.

Se usan muchos algoritmos de enrutamiento en las redes de computadoras. Los algoritmos estáticos incluyen el enrutamiento por trayectoria más corta, la inundación y el enrutamiento basado en flujo. Los algoritmos dinámicos incluyen el enrutamiento por vector de distancia y el enrutamiento por estado de enlace. La mayoría de las redes usan alguno de éstos. Otros temas importantes relativos al enrutamiento son el enrutamiento jerárquico, el enrutamiento para *hosts* móviles, el enrutamiento por difusión y el enrutamiento por multitransmisión.

Las subredes pueden congestionarse, aumentando el retardo y reduciendo el rendimiento de los paquetes. Los diseñadores de redes intentan evitar los congestionamientos mediante un diseño adecuado. Las técnicas incluyen conformación de tráfico, especificaciones de flujo y reservación de ancho de banda. Si ocurre un congestionamiento, habrá que encargarse de él. Pueden enviarse paquetes de estrangulamiento de regreso, deshecharse parte de la carga, y aplicarse otros métodos.

Las redes difieren de varias maneras, por lo que cuando se conectan redes múltiples pueden ocurrir problemas. A veces los problemas pueden superarse enviando los paquetes en túnel a

través de una red hostil, pero si las redes de origen y destino son diferentes, este enfoque falla. Cuando las diferentes redes tienen diferentes tamaños máximos de paquete, puede requerirse una fragmentación.

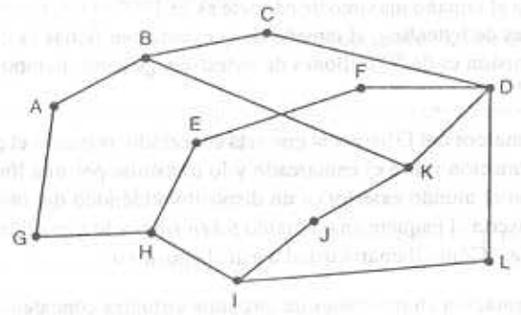
La Internet posee una copiosa variedad de protocolos relacionados con la capa de red. Éstos incluyen el protocolo de capa de transporte de datos, IP, pero también los protocolos de control ICMP, ARP y RARP, y los protocolos de enrutamiento OSPF y BGP. La Internet se está quedando sin direcciones de IP, por lo que se ha desarrollado una versión nueva del IP, el IPv6.

A diferencia de la Internet basada en datagramas, las redes ATM usan circuitos virtuales. Éstos deben establecerse antes de poder transmitir, y liberarse después de la transmisión. La calidad del servicio y el control de congestionamientos son asuntos de gran importancia en las redes ATM.

PROBLEMAS

1. Indique dos aplicaciones de ejemplo para las cuales es adecuado un servicio orientado a conexiones. Luego dé dos ejemplos en los que el servicio sin conexiones es lo mejor.
2. ¿Hay circunstancias en las cuales un servicio de circuito virtual entregará (o cuando menos debería entregar) paquetes en desorden? Explique.
3. Las subredes de datagramas enrutan cada paquete como unidad separada, independiente de las demás. Las subredes de circuitos virtuales no tienen que hacer esto, ya que cada paquete de datos sigue una ruta predeterminada. ¿Significa esto que las subredes de circuitos virtuales no necesitan la capacidad de enrutar paquetes aislados de un origen arbitrario a un destino arbitrario? Explique su respuesta.
4. Dé tres ejemplos de parámetros de protocolo que podrían negociarse al establecer una conexión.
5. Considere el siguiente problema de diseño que concierne la puesta en práctica del servicio de circuitos virtuales. Si los circuitos virtuales son internos a la subred, cada paquete de datos debe tener una cabecera de 3 bytes, y cada enrutador debe destinar hasta 8 bytes de almacenamiento para la identificación de circuitos. Si se usan internamente datagramas, se requieren cabeceras de 15 bytes, pero no se requiere espacio de tabla en los enrutadores. La capacidad de transmisión cuesta 1 centavo por cada 10^6 bytes, por salto. Puede comprarse memoria de enrutamiento por 1 centavo por byte y se deprecia en dos años (sólo horas de trabajo). Estadísticamente, la sesión promedio dura 1000 seg, tiempo durante el cual se transmiten 200 paquetes. El paquete medio requiere cuatro saltos. ¿Qué implementación es más económica, y por cuánto?
6. Suponiendo que todos los enrutadores y *hosts* están trabajando de manera adecuada y que el *software* de ambos está libre de errores, ¿hay alguna posibilidad, por pequeña que sea, de que un paquete sea entregado al destino equivocado?
7. Dé una heurística sencilla para encontrar dos trayectorias a través de una red de un origen dado a un destino dado que pueda sobrevivir a la pérdida de cualquier línea de comunicación (suponiendo que existen dos de tales trayectorias). Los enrutadores se consideran lo bastante confiables, por lo que no es necesario preocuparse por la posibilidad de caída de los enrutadores.

8. Considere la subred de la figura 5-15(a). Se usa enrutamiento por vector de distancia y acaban de llegar los siguientes vectores al enrutador *C*: de *B*: (5, 0, 8, 12, 6, 2); de *D*: (16, 12, 6, 0, 9, 10), y de *E*: (7, 6, 3, 9, 0, 4). Los retardos medidos a *B*, *D* y *E* son 6, 3 y 5, respectivamente. ¿Cuál es la nueva tabla de enrutamiento de *C*? Indique tanto la línea de salida a usar como el retardo esperado.
9. Si se registran los retardos como números de 8 bits en una red de 50 enrutadores, y se intercambian vectores de retardo dos veces por segundo, ¿qué ancho de banda por línea dúplex integral es consumido por el algoritmo de enrutamiento distribuido? Suponga que cada enrutador tiene tres líneas a los demás enrutadores.
10. En la figura 5-16 el OR booleano de los dos grupos de bits ACF es de 111 en cada columna. ¿Es éste un mero accidente, o es cierto en todas las circunstancias?
11. Con enrutamiento jerárquico de 4800 enrutadores, ¿qué tamaños de región y de *cluster* deben escogerse para reducir al mínimo el tamaño de la tabla de enrutamiento de una jerarquía de tres capas?
12. En el texto se indicó que, cuando un *host* móvil no está en casa, los paquetes enviados a su LAN base son interceptados por su agente de base. En una red IP de una LAN 802.3, ¿cómo logra esta intercepción el agente de base?
13. Viendo la subred de la figura 5-5, ¿cuántos paquetes se generan por una difusión de *B*, usando
    (a) reenvío por trayectoria invertida?
    (b) árbol de descenso?
14. Calcule un árbol de extensión multitransmisión para el enrutador *C* de la siguiente subred para un grupo con miembros en los enrutadores *A*, *B*, *C*, *D*, *E*, *F*, *I* y *K*.



15. Como posible mecanismo de control de congestionamientos en una subred que usa circuitos virtuales internamente, un enrutador podría abstenerse de reconocer un paquete recibido hasta que (1) sabe que su última transmisión por el circuito virtual se recibió con éxito y que (2) tiene un *buffer* libre. Por sencillez, supóngase que los enrutadores usan un protocolo de parada y espera y que cada circuito virtual tiene un *buffer* dedicado a él para cada sentido del tráfico. Si se requieren *T* seg para transmitir un paquete (de datos o de acuse de recibo) y hay *n* enrutadores en la trayectoria, ¿cuál es la velocidad con que se entregan paquetes al *host* de destino? Supóngase que los errores de transmisión son poco frecuentes y que la conexión *host*-enrutador es infinitamente rápida.

16. Una subred de datagramas permite que los enrutadores puedan deshacerse de paquetes cuando lo necesiten. La probabilidad de que un enrutador descarte un paquete es de p. Considere el caso de un *host* de origen conectado al enrutador de origen, que está conectado al enrutador de destino, y por él al *host* de destino. Si cualquiera de los enrutadores descarta un paquete, el *host* de origen tarde o temprano termina la temporización e intenta de nuevo. Si tanto las líneas *host*-enrutador como enrutador-enrutador se cuentan como saltos, ¿cuál es la media de
(a) saltos que hace un paquete por transmisión?
(b) transmisiones que hace un paquete?
(c) saltos requeridos por paquete recibido?

17. Cite una razón por la que el algoritmo de cubeta con goteo debe tener sólo un paquete por intervalo, independientemente del tamaño del paquete.

18. La variante de conteo de bytes del algoritmo de cubeta con goteo se usa en cierto sistema. La regla es que pueden enviarse por intervalo un paquete de 1024 bytes, dos paquetes de 512 bytes, etc. Indique una restricción seria de este sistema que no se menciona en el texto.

19. Una red ATM usa un esquema de cubeta con fichas para la conformación de tráfico. Se pone una ficha nueva en la cubeta cada 5 µseg. ¿Cuál es la tasa de datos neta máxima sustentable (excluyendo los bits de cabecera)?

20. Una computadora de una red de 6 Mbps se regula mediante una cubeta con fichas. La cubeta con fichas se llena a razón de 1 Mbps. Inicialmente está llena a su capacidad máxima de 8 megabits. ¿Durante cuánto tiempo puede la computadora transmitir a 6 Mbps?

21. En la figura 5-27 se muestran cuatro características de entrada para una especificación de flujo propuesta. Imagine que el tamaño máximo de paquete es de 1000 bytes, la velocidad de la cubeta con fichas es de 10 millones de bytes/seg, el tamaño de la cubeta con fichas es de 1 millón de bytes y la tasa máxima de transmisión es de 50 millones de bytes/seg. ¿Cuánto tiempo puede durar una ráfaga de máxima velocidad?

22. Un dispositivo acepta marcos del Ethernet al que está conectado; remueve el paquete que hay en cada marco, le agrega información sobre el enmarcado y lo transmite por una línea telefónica arrendada (su única conexión con el mundo exterior) a un dispositivo idéntico del otro lado. Este dispositivo retira el enmarcado, inserta el paquete en un marco *token ring*, y lo transmite a un *host* local a través de una LAN *token ring*. ¿Cómo llamaría usted a este dispositivo?

23. ¿Se requiere la fragmentación en interredes de circuitos virtuales concatenados, o sólo en los sistemas de datagramas?

24. El proceso en túnel a través de una red de circuitos virtuales concatenados es directo: el enrutador multiprotocolo de un extremo simplemente establece un circuito virtual con el otro extremo y pasa paquetes a través de él. ¿Puede usarse también el proceso en túnel en las subredes de datagramas? De ser así, ¿cómo?

25. Un datagrama IP que usa la opción de *enrutamiento estricto desde el origen* tiene que fragmentarse. ¿Piensa que la opción se copia en cada fragmento, o basta con simplemente ponerlo en el primer fragmento? Explique su respuesta.

26. Supóngase que, en lugar de usar 16 bits para la parte de red de una dirección clase B, se hubieran usado 20 bits. ¿Cuántas redes clase B habría?

27. Convierta la dirección de IP cuya representación hexadecimal es C22F1582 a notación decimal con puntos.

28. Una red clase B de Internet tiene una máscara de subred de 255.255.240.0. ¿Cuál es la cantidad máxima de *hosts* por subred?

29. Usted acaba de explicarle el protocolo ARP a un amigo. Al terminar, él dice: "Ya entiendo. El ARP proporciona un servicio a la capa de red, por lo que es parte de la capa de enlace de datos". ¿Qué le contesta usted?

30. El ARP y el RARP proyectan direcciones de un espacio al otro; en este sentido, son parecidos. Sin embargo, las implementaciones de estos protocolos son fundamentalmente diferentes. ¿En qué difieren principalmente?

31. Describa una manera de reensamblar fragmentos de IP en el destino.

32. La mayoría de los algoritmos de reensamble de datagramas IP tiene un temporizador para evitar que un fragmento perdido bloquee los *buffers* de reensamble por siempre. Suponga que un datagrama se divide en cuatro fragmentos. Llegan los primeros tres fragmentos, pero el último se retarda. Tarde o temprano termina el temporizador y los tres fragmentos en la memoria del receptor se descartan. Un poco después llega el último fragmento. ¿Qué debe hacerse con él?

33. La mayoría de los protocolos usa el número de saltos como la métrica que debe reducirse al mínimo al hacer cálculos de enrutamiento. En las redes ATM, la cantidad de saltos no es demasiado importante. ¿Por qué no? *Sugerencia:* repase el capítulo 2, para ver la manera en que funcionan los conmutadores ATM. ¿Usan almacenamiento y reenvío?

34. Tanto en IP como en ATM, la suma de comprobación cubre sólo la cabecera y no los datos. ¿Por qué supone que se escogió este diseño?

35. Una persona que vive en Boston viaja a Minneápolis, llevando consigo su computadora portátil. Para su sorpresa, la LAN de su destino en Minneápolis es una LAN inalámbrica IP, por lo que no tiene que conectarse. ¿Aún es necesario pasar por el sistema completo de agentes de base y agentes foráneos para que lleguen correctamente el correo electrónico y otros tipos de tráfico?

36. El IPv6 usa direcciones de 16 bytes. Si se asigna un bloque de 1 millón de direcciones cada picosegundo, ¿cuánto durarán las direcciones?

37. El campo de *protocolo* usado en la cabecera IPv4 no existe en la cabecera fija IPv6. ¿Por qué no?

38. Al introducirse el protocolo IPv6, ¿tiene que cambiarse el protocolo ARP? De ser así, ¿cuáles son los cambios conceptuales o técnicos?

39. En el capítulo 1 clasificamos las interacciones entre la red y los *hosts* usando cuatro clases de primitivas: *solicitud, indicación, respuesta* y *confirmación*. Clasifique los mensajes ESTABLECER y CONEXIÓN de la figura 5-65 según estas categorías.

40. Se está estableciendo un circuito virtual en una red ATM. Entre los *hosts* de origen y destino hay tres conmutadores ATM. ¿Cuántos mensajes (incluidos acuses de recibo) se enviarán para establecer el circuito?

41. La lógica usada para construir la tabla de la figura 5-67 es sencilla: siempre se asigna el *VPI* más bajo no utilizado a una conexión. Si se solicita un circuito virtual nuevo entre Nueva York y Denver, ¿qué *VPI* se le asignará?

42. En la figura 5-73(c), si una célula llega demasiado pronto, la siguiente aún debe llegar en $t_1 + 2T$. Supóngase que la regla fuera diferente; es decir, que la siguiente célula se espera en $t_2 + T$ y que el transmisor aprovechó al máximo esta regla. ¿Qué tasa máxima de células podría lograrse entonces? Para $T = 10$ µseg y $L = 2$ µseg, indique la tasa máxima de células original y nueva, respectivamente.

43. ¿Cuál es la longitud máxima de ráfaga en una conexión ATM ABR de 155.52 Mbps cuyo valor *PCR* es de 200,000 y cuyo valor L es de 25 µseg?

44. Escriba un programa para simular enrutamiento usando inundación. Cada paquete debe contener un contador que disminuye con cada salto. Al llegar el contador a cero, el paquete se descarta. El tiempo es discreto, manejando cada línea un paquete por intervalo de tiempo. Haga tres versiones del programa: todas las líneas inundadas, todas las líneas inundadas, excepto la línea de entrada, y sólo las mejores k líneas (seleccionadas estadísticamente) inundadas. Compare la inundación con el enrutamiento determinista ($k = 1$) en términos de retardo y ancho de banda usado.

45. Escriba un programa que simule una red de cómputo que usa tiempo discreto. El primer paquete de cada cola de enrutador hace un salto por intervalo de tiempo. Cada enrutador tiene una cantidad finita de *buffers*. Si llega un paquete y no hay lugar para él, se descarta y no se retransmite. En cambio, hay un protocolo terminal a terminal, completo con temporizadores y paquetes de reconocimiento, que tarde o temprano regenera el paquete del enrutador de origen. Dibuje en un grafo el rendimiento de la red en función del intervalo de terminación de temporización, parametrizado por tasa de errores.

6

LA CAPA DE TRANSPORTE

La capa de transporte no es sólo otra capa. Es el corazón de la jerarquía completa de protocolos. La tarea de esta capa es proporcionar un transporte de datos confiable y económico de la máquina de origen a la máquina de destino, independientemente de la red o redes físicas en uso. Sin la capa de transporte, el concepto total de los protocolos en capas tendría poco sentido. En este capítulo estudiaremos en detalle la capa de transporte, incluidos sus servicios, diseño, protocolos y desempeño.

6.1. EL SERVICIO DE TRANSPORTE

En las siguientes secciones haremos una introducción al servicio de transporte. Veremos el tipo de servicio proporcionado a la capa de aplicación (o de sesión, si existe) y especialmente la manera en que puede caracterizarse la calidad del servicio. Después veremos cómo las aplicaciones acceden al servicio de transporte, es decir, cómo es la interfaz.

6.1.1. Servicios proporcionados a las capas superiores

La meta fundamental de la capa de transporte es proporcionar un servicio eficiente, confiable y económico a sus usuarios, que normalmente son procesos de la capa de aplicación. Para lograr este objetivo, la capa de transporte hace uso de los servicios proporcionados por la capa de red.

41. La lógica usada para construir la tabla de la figura 5-67 es sencilla: siempre se asigna el *VPI* más bajo no utilizado a una conexión. Si se solicita un circuito virtual nuevo entre Nueva York y Denver, ¿qué *VPI* se le asignará?

42. En la figura 5-73(c), si una célula llega demasiado pronto, la siguiente aún debe llegar en $t_1 + 2T$. Supóngase que la regla fuera diferente; es decir, que la siguiente célula se espera en $t_2 + T$ y que el transmisor aprovechó al máximo esta regla. ¿Qué tasa máxima de células podría lograrse entonces? Para $T = 10$ µseg y $L = 2$ µseg, indique la tasa máxima de células original y nueva, respectivamente.

43. ¿Cuál es la longitud máxima de ráfaga en una conexión ATM ABR de 155.52 Mbps cuyo valor *PCR* es de 200,000 y cuyo valor L es de 25 µseg?

44. Escriba un programa para simular enrutamiento usando inundación. Cada paquete debe contener un contador que disminuye con cada salto. Al llegar el contador a cero, el paquete se descarta. El tiempo es discreto, manejando cada línea un paquete por intervalo de tiempo. Haga tres versiones del programa: todas las líneas inundadas, todas las líneas inundadas, excepto la línea de entrada, y sólo las mejores k líneas (seleccionadas estadísticamente) inundadas. Compare la inundación con el enrutamiento determinista ($k = 1$) en términos de retardo y ancho de banda usado.

45. Escriba un programa que simule una red de cómputo que usa tiempo discreto. El primer paquete de cada cola de enrutador hace un salto por intervalo de tiempo. Cada enrutador tiene una cantidad finita de *buffers*. Si llega un paquete y no hay lugar para él, se descarta y no se retransmite. En cambio, hay un protocolo terminal a terminal, completo con temporizadores y paquetes de reconocimiento, que tarde o temprano regenera el paquete del enrutador de origen. Dibuje en un grafo el rendimiento de la red en función del intervalo de terminación de temporización, parametrizado por tasa de errores.

6

LA CAPA DE TRANSPORTE

La capa de transporte no es sólo otra capa. Es el corazón de la jerarquía completa de protocolos. La tarea de esta capa es proporcionar un transporte de datos confiable y económico de la máquina de origen a la máquina de destino, independientemente de la red o redes físicas en uso. Sin la capa de transporte, el concepto total de los protocolos en capas tendría poco sentido. En este capítulo estudiaremos en detalle la capa de transporte, incluidos sus servicios, diseño, protocolos y desempeño.

6.1. EL SERVICIO DE TRANSPORTE

En las siguientes secciones haremos una introducción al servicio de transporte. Veremos el tipo de servicio proporcionado a la capa de aplicación (o de sesión, si existe) y especialmente la manera en que puede caracterizarse la calidad del servicio. Después veremos cómo las aplicaciones acceden al servicio de transporte, es decir, cómo es la interfaz.

6.1.1. Servicios proporcionados a las capas superiores

La meta fundamental de la capa de transporte es proporcionar un servicio eficiente, confiable y económico a sus usuarios, que normalmente son procesos de la capa de aplicación. Para lograr este objetivo, la capa de transporte hace uso de los servicios proporcionados por la capa de red.

El *hardware* o el *software* de la capa de transporte que se encarga del trabajo se llama **entidad de transporte**. La entidad de transporte puede estar en el núcleo (*kernel*) del sistema operativo, en un proceso de usuario independiente, en un paquete de biblioteca que forma parte de las aplicaciones de la red o en la tarjeta de interfaz de la red. En algunos casos, la portadora puede proporcionar servicio de transporte confiable, en cuyo caso la entidad de transporte reside en máquinas especiales de interfaz en la orilla de la subred a la que se conectan los *hosts*. La relación (lógica) entre las capas de red, transporte y aplicación se ilustra en la figura 6-1.

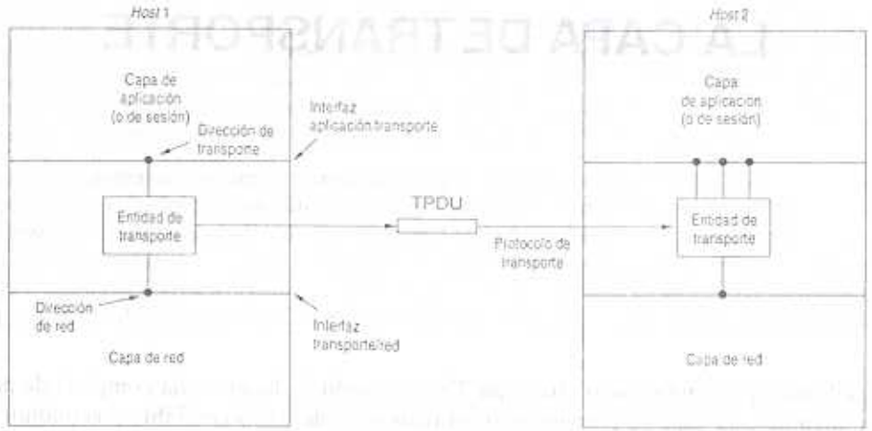


Figura 6-1. Las capas de red, transporte y aplicación.

Así como hay dos tipos de servicio de red, orientado a conexiones y sin conexiones, también hay dos tipos de servicio de transporte. El servicio de transporte orientado a conexiones es parecido al servicio de red orientado a conexiones en muchos sentidos. En ambos casos, las conexiones tienen tres fases: establecimiento, transferencia de datos y liberación. El direccionamiento y el control de flujo también son semejantes en ambas capas. Además, el servicio de transporte sin conexiones es muy parecido al servicio de red sin conexiones.

La pregunta obvia es entonces: ¿si el servicio de la capa de transporte es tan parecido al de la capa de red, por qué hay dos capas diferentes? ¿Por qué no es suficiente una sola capa? La respuesta es sutil, pero crucial, y nos remite a la figura 1-16. En esta figura podemos ver que la capa de red es parte de la subred de comunicación y es operada por la portadora (cuando menos en las WAN). ¿Qué sucede si la capa de red ofrece un servicio orientado a conexiones pero no es confiable? ¿Qué tal si esa capa pierde paquetes con frecuencia? ¿Qué ocurre si los enrutadores se vienen abajo de cuando en cuando?

Problemas, eso es lo que ocurre. Los usuarios no tienen control sobre la subred, por lo que no pueden resolver los problemas de un mal servicio usando mejores enrutadores o incluyendo

una porción mayor del manejo de errores en la capa de enlace de datos. La única posibilidad es poner otra capa encima de la capa de red que mejore la calidad del servicio. Si a la mitad de una transmisión larga se informa a una entidad de transporte que su conexión de red ha sido terminada abruptamente, sin indicación de lo sucedido a los datos actualmente en tránsito, la entidad puede establecer una nueva conexión de red con la entidad de transporte remota. Usando esta nueva conexión de red, la entidad puede enviar una solicitud a su igual preguntando cuáles datos llegaron y cuáles no, y reiniciar desde la interrupción.

Esencialmente, la existencia de la capa de transporte hace posible que el servicio de transporte sea más confiable que el servicio de red subyacente. La capa de transporte puede detectar y compensar paquetes perdidos y datos alterados. Es más, las primitivas del servicio de transporte pueden diseñarse de modo que sean independientes de las primitivas del servicio de red, que pueden variar considerablemente entre las redes (por ejemplo, el servicio LAN sin conexiones puede ser bastante diferente del servicio WAN orientado a conexiones).

Gracias a la capa de transporte, es posible escribir programas de aplicación usando un grupo estándar de primitivas, y hacer que estos programas trabajen en una variedad amplia de redes, sin tener que preocuparse de lidiar con interfaces de subred diferentes y transmisiones no confiables. Si ninguna red real tuviera fallas, y si todas tuvieran las mismas primitivas de servicio, la capa de transporte probablemente sería innecesaria. Sin embargo, en el mundo real esta capa cumple la función clave de aislar las capas superiores respecto de la tecnología, el diseño y las imperfecciones de la subred.

Por esta razón, mucha gente distingue entre las capas 1 a 4, por una parte, y la(s) capa(s) por encima de la 4, por la otra. Las cuatro capas inferiores pueden verse como el **proveedor del servicio de transporte**, y la(s) capa(s) superiores son el **usuario del servicio de transporte**. Esta distinción entre proveedor y usuario tiene un impacto considerable en el diseño de las capas y pone a la capa de transporte en una posición clave, ya que forma el límite principal entre el proveedor y el usuario del servicio confiable de transmisión de datos.

6.1.2. Calidad del servicio

Otra manera de ver la capa de transporte es considerar que su función primaria es mejorar la QoS (*Quality of Service*, **calidad del servicio**) proporcionada por la capa de red. Si el servicio de red es impecable, la capa de transporte tiene una tarea fácil. Si, por otra parte, el servicio de red es malo, la capa de transporte tiene que construir un puente sobre el abismo que separa lo que quieren los usuarios de transporte y lo que la capa de red proporciona.

Aunque a primera vista la calidad del servicio podría parecer un concepto vago (lograr que todos se pongan de acuerdo en lo que constituye un "buen" servicio es un ejercicio nada trivial), la QoS puede caracterizarse por varios parámetros específicos, como vimos en el capítulo 5. El servicio de transporte puede permitir que el usuario especifique valores preferidos, aceptables y mínimos para varios parámetros de servicio en el momento de establecerse una conexión. Algunos de los parámetros también se aplican al transporte sin conexiones. Es responsabilidad de la capa de transporte examinar estos parámetros y, dependiendo de los tipos de servicio de red disponibles, determinar si puede proporcionar el servicio requerido. En el resto de esta sección

estudiaremos algunos parámetros QoS, que se presentan resumidos en la figura 6-2. Nótese que pocas redes y protocolos proporcionan todos estos parámetros. Muchos simplemente hacen su mejor esfuerzo para reducir la tasa de errores residual, y hasta ahí. Otros tienen arquitecturas de QoS complejas (Campbell *et al.*, 1994).

Retardo de establecimiento de conexión
Probabilidad de falla de establecimiento de conexión
Rendimiento
Retardo de tránsito
Tasa de errores residual
Protección
Prioridad
Tenacidad

Figura 6-2. Parámetros de calidad de servicio típicos de la capa de transporte.

El *retardo de establecimiento de la conexión* es el tiempo que transcurre entre la solicitud de una conexión de transporte y la confirmación del usuario del servicio de transporte; incluye el retardo de proceso de la entidad de transporte remota. Al igual que con todos los parámetros que miden un retardo, a menor retardo, mejor servicio.

La *probabilidad de falla de establecimiento de la conexión* es la posibilidad de que una conexión no se establezca en el lapso de máximo retardo de establecimiento; por ejemplo, debido a congestionamiento de la red, falta de espacio de tablas en algún lado u otros problemas internos.

El parámetro de *rendimiento* mide la cantidad de bytes de datos de usuario transferidos por segundo, medido durante algún intervalo de tiempo. El rendimiento se mide por separado en cada sentido.

El *retardo de tránsito* mide el tiempo entre el envío de un mensaje por el usuario de transporte de la máquina de origen y su recepción por el usuario de transporte de la máquina de destino. Al igual que con el rendimiento, cada sentido se maneja por separado.

La *tasa de error residual* mide la cantidad de mensajes perdidos o alterados como una fracción del total enviado. En teoría, la tasa de error residual debe ser cero, ya que es responsabilidad de la capa de transporte ocultar todos los errores de la capa de red. En la práctica, este parámetro puede tener algún valor finito pequeño.

El parámetro de *protección* provee un mecanismo para que el usuario de transporte especifique su interés en que la capa de transporte proporcione protección de lectura y modificación de los datos transmitidos contra terceros no autorizados (interceptación).

El parámetro de *prioridad* proporciona un mecanismo para que el usuario indique que algunas de sus conexiones son más importantes que otras, y que, en el caso de un congestionamiento, se asegure de que las conexiones de alta prioridad sean atendidas antes que las de baja prioridad.

Por último, el parámetro de *tenacidad* da la probabilidad de que la capa de transporte por sí misma termine instantáneamente una conexión debido a problemas internos o congestionamiento.

Los parámetros de QoS los especifica el usuario de transporte cuando se solicita una conexión. Pueden darse tanto el valor deseado como el mínimo. En algunos casos, al ver los parámetros de QoS, la capa de transporte puede darse cuenta de inmediato de que algunos de ellos son inalcanzables, en cuyo caso indica al solicitante que falló el intento de conexión, sin molestarse en comunicarse con el destino. El informe de fallas especifica la razón de la falla.

En otros casos, la capa de transporte sabe que no puede lograr la meta deseada (por ejemplo, rendimiento de 600 Mbps), pero puede lograr una tasa menor, aunque aún aceptable (por ejemplo, 150 Mbps). Entonces envía la tasa menor y la tasa mínima aceptable a la máquina remota, solicitando establecer una conexión. Si la máquina remota no puede manejar el valor propuesto, pero puede manejar un valor por encima del mínimo, puede hacer una contraoferta. Si no puede manejar ningún valor superior al mínimo, rechaza el intento de conexión. Por último, se informa al usuario de transporte originador si se estableció o se rechazó la conexión y, si se estableció, los valores de los parámetros acordados.

Este proceso se llama **negociación abierta**. Una vez que se han negociado las opciones, permanecen así durante el resto de la vida de la conexión. Para evitar que los clientes se vuelvan demasiado codiciosos, muchas portadoras suelen cobrar más por una mejor calidad de servicio.

6.1.3. Primitivas de servicio de transporte

Las primitivas del servicio de transporte permiten a los usuarios de transporte (por ejemplo, programas de aplicación) el acceso al servicio de transporte. Cada servicio tiene sus propias primitivas de acceso. En esta sección examinaremos primero un servicio de transporte sencillo e hipotético y luego veremos un ejemplo real.

El servicio de transporte es parecido al servicio de red, pero hay algunas diferencias importantes. La principal es que la intención del servicio de red es modelar el servicio ofrecido por las redes reales, con todo y verrugas. Las redes reales pueden perder paquetes, por lo que el servicio de red generalmente no es confiable.

El servicio de transporte orientado a conexiones, en cambio, sí es confiable. Claro que las redes reales no están libres de errores, pero ése es precisamente el propósito de la capa de transporte: proporcionar un servicio confiable por una red no confiable.

Como ejemplo, considere dos procesos conectados mediante tubos en UNIX. Ambos consideran que la conexión entre ellas es perfecta. No quieren saber de acuses de recibo, paquetes perdidos, congestionamientos ni nada por el estilo. Lo que quieren es una conexión 100% confiable. El proceso *A* pone datos en un extremo del tubo y el proceso *B* lo saca por el otro. De esto se trata el servicio de transporte orientado a conexiones: de esconder las imperfecciones del

Primitiva	TPDU enviada	Significado
LISTEN (ESCUCHAR)	(ninguna)	Se bloquea hasta que algún proceso intenta el contacto
CONNECT (CONECTAR)	SOLICITUD DE CONEXIÓN	Intenta activamente establecer una conexión
SEND (ENVIAR)	DATOS	Envía información
RECEIVE (RECIBIR)	(ninguna)	Se bloquea hasta que llega una TPDU de DATOS
DISCONNECT (DESCONECTAR)	SOLICITUD DE DESCONEXIÓN	Este lado quiere liberar la conexión

Figura 6-3. Primitivas de un servicio de transporte sencillo.

servicio de red para que los procesos del usuario simplemente puedan suponer la existencia de una corriente de bits libre de errores.

Como nota al margen, la capa de transporte puede proporcionar también un servicio no confiable (de datagramas), pero hay relativamente poco que decir al respecto, por lo que en este capítulo nos concentraremos en el servicio de transporte orientado a conexiones.

Una segunda diferencia entre el servicio de red y el de transporte es a quién están dirigidos los servicios. El servicio de red lo usan sólo las entidades de transporte. Pocos usuarios escriben sus propias entidades de transporte y, por tanto, pocos usuarios o programas llegan a ver internamente el servicio de red. En contraste, muchos programas (y por tanto programadores) ven las primitivas de transporte. En consecuencia, el servicio de transporte debe ser cómodo y fácil de usar.

Para tener una idea de lo que podría ser el servicio de transporte, considere las cinco primitivas listadas en la figura 6-3. Esta interfaz de transporte ciertamente es sencilla, pero da la esencia de lo que debe hacer una interfaz de transporte orientada a conexiones: permite que los programas de aplicación establezcan, usen y liberen conexiones, lo que basta para muchas aplicaciones.

Para ver cómo podrían usarse estas primitivas, considere una aplicación con un servidor y cierta cantidad de clientes remotos. Para comenzar, el servidor ejecuta una primitiva LISTEN (escuchar), normalmente llamando a un procedimiento de biblioteca que hace una llamada de sistema para bloquear al servidor hasta la aparición de un cliente. Cuando un cliente quiere hablar con el servidor, ejecuta una primitiva CONNECT (conectarse). La entidad de transporte ejecuta esta primitiva bloqueando al que llama y enviando un paquete al servidor. Encapsulado en la carga útil de este paquete hay un mensaje de capa de transporte para la entidad de transporte del servidor.

Es pertinente aquí una nota rápida sobre la terminología. A falta de un mejor término, usaremos bajo protesta las siglas poco elegantes de **TPDU** (*Transport Protocol Data Unit*, **unidad de datos del protocolo de transporte**) para referirnos a los mensajes enviados de una entidad de transporte a otra. Por tanto, las TPDU (intercambiadas por la capa de transporte) están contenidas en paquetes (intercambiados por la capa de red). A su vez, los paquetes están contenidos en marcos (intercambiados por la capa de enlace de datos). Al llegar un marco, la capa de enlace de datos procesa la cabecera del marco y pasa el contenido del campo de carga útil del

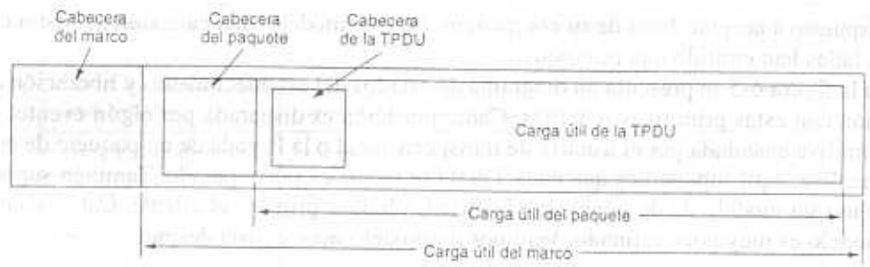


Figura 6-4. Anidamiento de las TPDU, los paquetes y los marcos.

marco a la entidad de red. La entidad de red procesa la cabecera del paquete y pasa el contenido de la carga útil del paquete a la entidad de transporte. Este anidamiento se ilustra en la figura 6-4.

Regresando a nuestro ejemplo de cliente-servidor, la llamada CONNECT del cliente causa el envío de una TPDU CONNECTION REQUEST (solicitud de conexión) al servidor. Al llegar ésta, la entidad de transporte la revisa para comprobar que el servidor está bloqueado en LISTEN (es decir, está interesado en manejar solicitudes). Entonces desbloquea el servidor y envía una TPDU CONNECTION ACCEPTED (conexión aceptada) de regreso al cliente. Al llegar esta TPDU, el cliente se desbloquea y se establece la conexión.

Ahora pueden intercambiarse datos usando las primitivas SEND (enviar) y RECEIVE (recibir). En la forma más simple, cualquiera de las dos partes puede emitir una RECEIVE (bloqueadora) para esperar que la otra parte emita una SEND. Al llegar la TPDU, el receptor se desbloquea y ya puede procesar la TPDU y enviar una respuesta. Mientras ambos lados puedan llevar el control de quién tiene el turno para transmitir, este esquema funcionará bien.

Nótese que, en la capa de red, incluso un intercambio de datos unidireccional es más complicado que en la capa de transporte. Cada paquete de datos enviado también será reconocido (tarde o temprano). Los paquetes que llevan una TPDU de control también son reconocidos, implícita o explícitamente. Estos reconocimientos son manejados por las entidades de transporte usando el protocolo de capa de red, y no son visibles para los usuarios de transporte. De la misma manera, las entidades de transporte necesitarán preocuparse por los temporizadores y las retransmisiones. Nada de esta maquinaria es vista por los usuarios de transporte. Para ellos, una conexión es un tubo de bits confiable: un usuario mete bits en él y mágicamente aparecen en el otro lado. Esta capacidad de esconder la complejidad es la razón por la cual los protocolos en capas son herramientas tan poderosas.

Cuando ya no se necesita una conexión, debe liberarse para desocupar espacio de tablas en las dos entidades de transporte. La desconexión tiene dos variantes: asimétrica y simétrica. En la variante asimétrica, cualquiera de los dos usuarios de transporte puede emitir una primitiva DISCONNECT (desconectar), que resulta en el envío de una TPDU DISCONNECT a la entidad de transporte remota. A su llegada, se libera la conexión.

En la variante simétrica, cada sentido se cierra por separado, independientemente del otro. Al emitir una DISCONNECT un lado, quiere decir que ya no tiene más datos por enviar, pero aún

Primitiva	TPDU enviada	Significado
LISTEN (ESCUCHAR)	(ninguna)	Se bloquea hasta que algún proceso intenta el contacto
CONNECT (CONECTAR)	SOLICITUD DE CONEXIÓN	Intenta activamente establecer una conexión
SEND (ENVIAR)	DATOS	Envía información
RECEIVE (RECIBIR)	(ninguna)	Se bloquea hasta que llega una TPDU de DATOS
DISCONNECT (DESCONECTAR)	SOLICITUD DE DESCONEXIÓN	Este lado quiere liberar la conexión

Figura 6-3. Primitivas de un servicio de transporte sencillo.

servicio de red para que los procesos del usuario simplemente puedan suponer la existencia de una corriente de bits libre de errores.

Como nota al margen, la capa de transporte puede proporcionar también un servicio no confiable (de datagramas), pero hay relativamente poco que decir al respecto, por lo que en este capítulo nos concentraremos en el servicio de transporte orientado a conexiones.

Una segunda diferencia entre el servicio de red y el de transporte es a quién están dirigidos los servicios. El servicio de red lo usan sólo las entidades de transporte. Pocos usuarios escriben sus propias entidades de transporte y, por tanto, pocos usuarios o programas llegan a ver internamente el servicio de red. En contraste, muchos programas (y por tanto programadores) ven las primitivas de transporte. En consecuencia, el servicio de transporte debe ser cómodo y fácil de usar.

Para tener una idea de lo que podría ser el servicio de transporte, considere las cinco primitivas listadas en la figura 6-3. Esta interfaz de transporte ciertamente es sencilla, pero da la esencia de lo que debe hacer una interfaz de transporte orientada a conexiones: permite que los programas de aplicación establezcan, usen y liberen conexiones, lo que basta para muchas aplicaciones.

Para ver cómo podrían usarse estas primitivas, considere una aplicación con un servidor y cierta cantidad de clientes remotos. Para comenzar, el servidor ejecuta una primitiva LISTEN (escuchar), normalmente llamando a un procedimiento de biblioteca que hace una llamada de sistema para bloquear al servidor hasta la aparición de un cliente. Cuando un cliente quiere hablar con el servidor, ejecuta una primitiva CONNECT (conectarse). La entidad de transporte ejecuta esta primitiva bloqueando al que llama y enviando un paquete al servidor. Encapsulado en la carga útil de este paquete hay un mensaje de capa de transporte para la entidad de transporte del servidor.

Es pertinente aquí una nota rápida sobre la terminología. A falta de un mejor término, usaremos bajo protesta las siglas poco elegantes de **TPDU** (*Transport Protocol Data Unit*, **unidad de datos del protocolo de transporte**) para referirnos a los mensajes enviados de una entidad de transporte a otra. Por tanto, las TPDU (intercambiadas por la capa de transporte) están contenidas en paquetes (intercambiados por la capa de red). A su vez, los paquetes están contenidos en marcos (intercambiados por la capa de enlace de datos). Al llegar un marco, la capa de enlace de datos procesa la cabecera del marco y pasa el contenido del campo de carga útil del

Figura 6-4. Anidamiento de las TPDU, los paquetes y los marcos.

marco a la entidad de red. La entidad de red procesa la cabecera del paquete y pasa el contenido de la carga útil del paquete a la entidad de transporte. Este anidamiento se ilustra en la figura 6-4.

Regresando a nuestro ejemplo de cliente-servidor, la llamada CONNECT del cliente causa el envío de una TPDU CONNECTION REQUEST (solicitud de conexión) al servidor. Al llegar ésta, la entidad de transporte la revisa para comprobar que el servidor está bloqueado en LISTEN (es decir, está interesado en manejar solicitudes). Entonces desbloquea el servidor y envía una TPDU CONNECTION ACCEPTED (conexión aceptada) de regreso al cliente. Al llegar esta TPDU, el cliente se desbloquea y se establece la conexión.

Ahora pueden intercambiarse datos usando las primitivas SEND (enviar) y RECEIVE (recibir). En la forma más simple, cualquiera de las dos partes puede emitir una RECEIVE (bloqueadora) para esperar que la otra parte emita una SEND. Al llegar la TPDU, el receptor se desbloquea y ya puede procesar la TPDU y enviar una respuesta. Mientras ambos lados puedan llevar el control de quién tiene el turno para transmitir, este esquema funcionará bien.

Nótese que, en la capa de red, incluso un intercambio de datos unidireccional es más complicado que en la capa de transporte. Cada paquete de datos enviado también será reconocido (tarde o temprano). Los paquetes que llevan una TPDU de control también son reconocidos, implícita o explícitamente. Estos reconocimientos son manejados por las entidades de transporte usando el protocolo de capa de red, y no son visibles para los usuarios de transporte. De la misma manera, las entidades de transporte necesitarán preocuparse por los temporizadores y las retransmisiones. Nada de esta maquinaria es vista por los usuarios de transporte. Para ellos, una conexión es un tubo de bits confiable: un usuario mete bits en él y mágicamente aparecen en el otro lado. Esta capacidad de esconder la complejidad es la razón por la cual los protocolos en capas son herramientas tan poderosas.

Cuando ya no se necesita una conexión, debe liberarse para desocupar espacio de tablas en las dos entidades de transporte. La desconexión tiene dos variantes: asimétrica y simétrica. En la variante asimétrica, cualquiera de los dos usuarios de transporte puede emitir una primitiva DISCONNECT (desconectar), que resulta en el envío de una TPDU DISCONNECT a la entidad de transporte remota. A su llegada, se libera la conexión.

En la variante simétrica, cada sentido se cierra por separado, independientemente del otro. Al emitir una DISCONNECT un lado, quiere decir que ya no tiene más datos por enviar, pero aún

está dispuesto a aceptar datos de su compañero. En este modelo, una conexión se libera cuando ambos lados han emitido una DISCONNECT.

En la figura 6-5 se presenta un diagrama de estados del establecimiento y liberación de una conexión con estas primitivas sencillas. Cada transición es disparada por algún evento, ya sea una primitiva ejecutada por el usuario de transporte local o la llegada de un paquete de entrada. Por sencillez, aquí suponemos que cada TPDU se reconoce por separado. También suponemos que se usa un modelo de desconexión simétrica, yéndose primero el cliente. Cabe señalar que este modelo es muy poco refinado. Veremos un modelo más realista después.

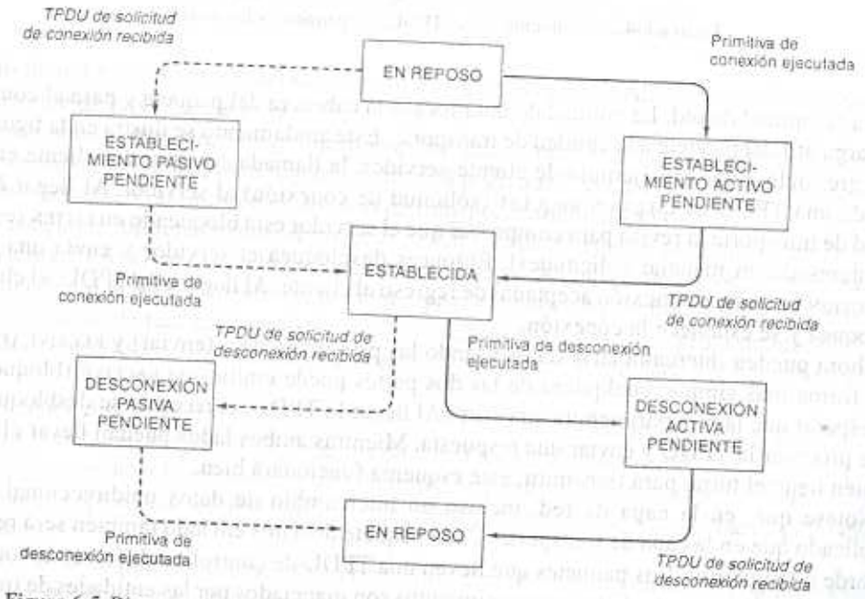


Figura 6-5. Diagrama de estados de un esquema sencillo de manejo de conexiones. Las transiciones rotuladas con cursivas son causadas por llegadas de paquetes. Las líneas continuas muestran la secuencia de estados del cliente. Las líneas punteadas muestran la secuencia de estados del servidor.

Sockets de Berkeley

Inspeccionemos brevemente otro grupo de primitivas de transporte, las primitivas de socket usadas en el UNIX de Berkeley para el TCP. Estas primitivas se listan en la figura 6-6. En términos generales, las primitivas siguen el modelo de nuestro primer ejemplo, pero ofrecen más características y flexibilidad. No veremos aquí las TPDU correspondientes. Ese análisis tendrá que esperar hasta que estudiemos el TCP posteriormente en este capítulo.

Las primeras cuatro primitivas de la lista son ejecutadas en ese orden por los servidores. La primitiva SOCKET crea un nuevo punto final y le asigna espacio de tablas dentro de la entidad de

Primitiva	Significado
SOCKET (ENCHUFAR)	Crea un nuevo punto terminal de comunicación
BIND (LIGAR)	Conecta una dirección local a un socket
LISTEN (ESCUCHAR)	Anuncia la disposición a aceptar conexiones; indica tamaño de cola
ACCEPT (ACEPTAR)	Bloquea al invocador hasta la llegada de un intento de conexión
CONNECT (CONECTAR)	Intenta activamente establecer una conexión
SEND (ENVIAR)	Envía datos a través de la conexión
RECEIVE (RECIBIR)	Recibe datos de la conexión
CLOSE (CERRAR)	Libera la conexión

Figura 6-6. Primitivas de socket para TCP.

transporte. Los parámetros de la llamada especifican el formato de direccionamiento a usar, el tipo de servicio deseado (por ejemplo, corriente de bytes confiable), y el protocolo. Una llamada SOCKET con éxito devuelve un descriptor de archivo ordinario para usarse con las siguientes llamadas, de la misma manera que lo hace una llamada OPEN.

Los sockets recién creados no tienen direcciones. Éstas se asignan usando la primitiva BIND (ligar). Una vez que un servidor ha ligado una dirección a un socket, los clientes remotos pueden conectarse a él. La razón para no hacer que la llamada SOCKET cree directamente una dirección es que a algunos procesos les importan sus direcciones (por ejemplo, han estado usando su misma dirección durante años y todos la conocen), mientras que a otros no les importa.

A continuación viene la llamada LISTEN (escuchar), que asigna espacio para poner en cola las llamadas entrantes por si varios clientes intentan conectarse al mismo tiempo. En contraste con LISTEN en nuestro primer ejemplo, en el modelo de sockets LISTEN no es una llamada bloqueadora.

Para bloquearse en espera de una conexión entrante, el servidor ejecuta una primitiva ACCEPT (aceptar). Cuando llega una TPDU solicitando una conexión, la entidad de transporte crea un socket nuevo con las mismas propiedades que el original y devuelve un descriptor de archivo para él. El servidor entonces puede ramificar (*fork*) un proceso o hilo para manejar la conexión en el socket nuevo y regresar a esperar la siguiente conexión en el socket original.

Ahora veamos el lado del cliente. Aquí también debe crearse un socket primero usando la primitiva SOCKET, pero no se requiere BIND, puesto que la dirección usada no le importa al servidor. La primitiva CONNECT (conectar) bloquea al invocador y comienza activamente el proceso de conexión. Al completarse éste (es decir, cuando se recibe la TPDU adecuada del servidor) el proceso cliente se desbloquea y se establece la conexión. Ambos lados pueden usar ahora SEND y RECEIVE para transmitir y recibir datos a través de la conexión dúplex integral.

La liberación de las conexiones a los sockets es simétrica. Cuando ambos lados han ejecutado una primitiva CLOSE, se libera la conexión.

está dispuesto a aceptar datos de su compañero. En este modelo, una conexión se libera cuando ambos lados han emitido una DISCONNECT.

En la figura 6-5 se presenta un diagrama de estados del establecimiento y liberación de una conexión con estas primitivas sencillas. Cada transición es disparada por algún evento, ya sea una primitiva ejecutada por el usuario de transporte local o la llegada de un paquete de entrada. Por sencillez, aquí suponemos que cada TPDU se reconoce por separado. También suponemos que se usa un modelo de desconexión simétrica, yéndose primero el cliente. Cabe señalar que este modelo es muy poco refinado. Veremos un modelo más realista después.

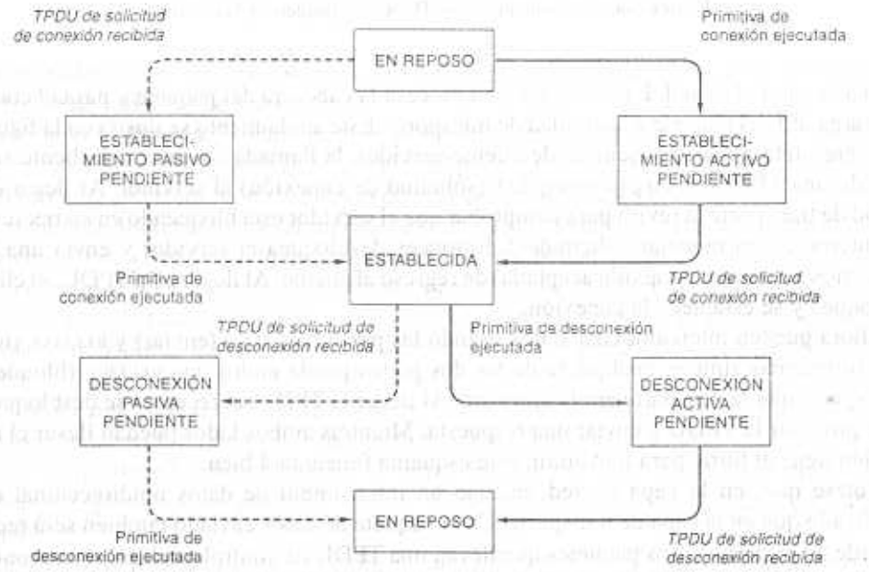


Figura 6-5. Diagrama de estados de un esquema sencillo de manejo de conexiones. Las transiciones rotuladas con cursivas son causadas por llegadas de paquetes. Las líneas continuas muestran la secuencia de estados del cliente. Las líneas punteadas muestran la secuencia de estados del servidor.

Sockets de Berkeley

Inspeccionemos brevemente otro grupo de primitivas de transporte, las primitivas de socket usadas en el UNIX de Berkeley para el TCP. Estas primitivas se listan en la figura 6-6. En términos generales, las primitivas siguen el modelo de nuestro primer ejemplo, pero ofrecen más características y flexibilidad. No veremos aquí las TPDU correspondientes. Ese análisis tendrá que esperar hasta que estudiemos el TCP posteriormente en este capítulo.

Las primeras cuatro primitivas de la lista son ejecutadas en ese orden por los servidores. La primitiva SOCKET crea un nuevo punto final y le asigna espacio de tablas dentro de la entidad de

Primitiva	Significado
SOCKET (ENCHUFAR)	Crea un nuevo punto terminal de comunicación
BIND (LIGAR)	Conecta una dirección local a un socket
LISTEN (ESCUCHAR)	Anuncia la disposición a aceptar conexiones; indica tamaño de cola
ACCEPT (ACEPTAR)	Bloquea al invocador hasta la llegada de un intento de conexión
CONNECT (CONECTAR)	Intenta activamente establecer una conexión
SEND (ENVIAR)	Envía datos a través de la conexión
RECEIVE (RECIBIR)	Recibe datos de la conexión
CLOSE (CERRAR)	Libera la conexión

Figura 6-6. Primitivas de socket para TCP.

transporte. Los parámetros de la llamada especifican el formato de direccionamiento a usar, el tipo de servicio deseado (por ejemplo, corriente de bytes confiable), y el protocolo. Una llamada SOCKET con éxito devuelve un descriptor de archivo ordinario para usarse con las siguientes llamadas, de la misma manera que lo hace una llamada OPEN.

Los sockets recién creados no tienen direcciones. Éstas se asignan usando la primitiva BIND (ligar). Una vez que un servidor ha ligado una dirección a un socket, los clientes remotos pueden conectarse a él. La razón para no hacer que la llamada SOCKET cree directamente una dirección es que a algunos procesos les importan sus direcciones (por ejemplo, han estado usando su misma dirección durante años y todos la conocen), mientras que a otros no les importa.

A continuación viene la llamada LISTEN (escuchar), que asigna espacio para poner en cola las llamadas entrantes por si varios clientes intentan conectarse al mismo tiempo. En contraste con LISTEN en nuestro primer ejemplo, en el modelo de sockets LISTEN no es una llamada bloqueadora.

Para bloquearse en espera de una conexión entrante, el servidor ejecuta una primitiva ACCEPT (aceptar). Cuando llega una TPDU solicitando una conexión, la entidad de transporte crea un socket nuevo con las mismas propiedades que el original y devuelve un descriptor de archivo para él. El servidor entonces puede ramificar (*fork*) un proceso o hilo para manejar la conexión en el socket nuevo y regresar a esperar la siguiente conexión en el socket original.

Ahora veamos el lado del cliente. Aquí también debe crearse un socket primero usando la primitiva SOCKET, pero no se requiere BIND, puesto que la dirección usada no le importa al servidor. La primitiva CONNECT (conectar) bloquea al invocador y comienza activamente el proceso de conexión. Al completarse éste (es decir, cuando se recibe la TPDU adecuada del servidor) el proceso cliente se desbloquea y se establece la conexión. Ambos lados pueden usar ahora SEND y RECEIVE para transmitir y recibir datos a través de la conexión dúplex integral.

La liberación de las conexiones a los sockets es simétrica. Cuando ambos lados han ejecutado una primitiva CLOSE, se libera la conexión.

6.2. ELEMENTOS DE LOS PROTOCOLOS DE TRANSPORTE

El servicio de transporte se implementa mediante un **protocolo de transporte** entre las dos entidades de transporte. En ciertos aspectos, los protocolos de transporte se parecen a los protocolos de enlace de datos que estudiamos detalladamente en el capítulo 3. Ambos se encargan del control de errores, la secuencia y el control de flujo, entre otros asuntos.

Sin embargo, existen diferencias significativas entre los dos. Estas diferencias se deben a diferencias importantes entre los entornos en que operan ambos protocolos, como se muestra en la figura 6-7. En la capa de enlace de datos, dos enrutadores se comunican directamente mediante un canal físico mientras que, en la capa de transporte, este canal físico es reemplazado por la subred completa. Esta diferencia tiene muchas implicaciones importantes para los protocolos.

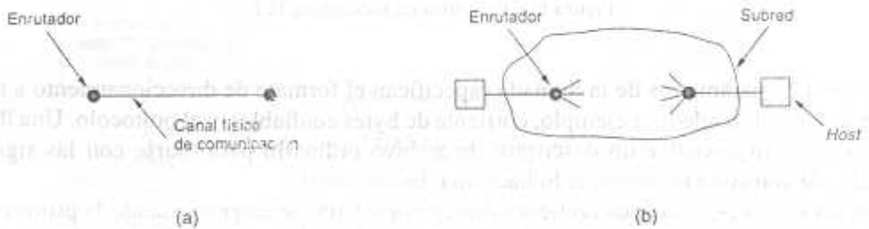


Figura 6-7. (a) Entorno de la capa de enlace de datos. (b) Entorno de la capa de transporte.

Por una parte, en la capa de enlace de datos no es necesario que un enrutador especifique el enrutador con el que quiere hablar; cada línea de salida especifica de manera única un enrutador en particular. En la capa de transporte, se requiere el direccionamiento explícito de los destinos.

Por otro lado, el proceso de establecimiento de una conexión a través del alambre de la figura 6-7(a) es sencillo: la otra terminal siempre está ahí (a menos que se haya caído, en cuyo caso no está ahí). Sea como sea, no hay demasiado que hacer. En la capa de transporte, el establecimiento inicial de la conexión es más complicado, como veremos.

Otra diferencia, muy irritante, entre la capa de enlace de datos y la capa de transporte es la existencia potencial de capacidad de almacenamiento en la subred. Al enviar un enrutador un marco, éste puede llegar o perderse, pero no puede rebotar de un lado a otro durante un rato, esconderse en un rincón alejado del mundo y emerger repentinamente en algún momento inoportuno 30 segundos después. Si la subred usa datagramas y enrutamiento adaptable internamente, hay una probabilidad no despreciable de que un paquete pueda almacenarse durante varios segundos y entregarse después. Las consecuencias de esta capacidad de almacenamiento de paquetes en la subred pueden ser desastrosas y requerir el uso de protocolos especiales.

Una última diferencia entre las capas de enlace de datos y de transporte es una de cantidad, más bien que de tipo. Se requieren *buffers* y control de flujo en ambas capas, pero la presencia de una cantidad de conexiones grande y dinámicamente variable en la capa de transporte puede

requerir un enfoque distinto del que se usa en la capa de enlace de datos. En el capítulo 3 vimos que algunos de los protocolos asignan una cantidad fija de *buffers* a cada línea de modo que, al llegar un marco, siempre haya un *buffer* disponible. En la capa de transporte, la gran cantidad de conexiones que deben manejarse hace menos atractiva la idea de dedicar muchos *buffers* a cada una. En las siguientes secciones examinaremos todos estos importantes temas, además de otros.

6.2.1. Direccionamiento

Cuando un proceso de aplicación desea establecer una conexión con un proceso de aplicación remoto, debe especificar a cuál debe conectarse. (El transporte sin conexiones tiene el mismo problema: ¿a quién debe enviarse cada mensaje?) El método que normalmente se emplea es definir direcciones de transporte en las que los procesos pueden estar a la escucha de solicitudes de conexión. En Internet, estos puntos terminales son pares (dirección IP, puerto local). En las redes ATM, son pares AAL-SAP. Usaremos el término neutral **TSAP** (*Transport Service Access Point*, **punto de acceso al servicio de transporte**). Los puntos terminales análogos de la capa de red (por ejemplo, direcciones de capa de red) se llaman entonces **NSAP** (*Network Service Access Point*, **puntos de acceso al servicio de red**). Las direcciones IP son ejemplos de NSAP.

En la figura 6-8 se ilustra la relación entre el NSAP, el TSAP, la conexión de red y la conexión de transporte de una subred orientada a conexiones (por ejemplo, ATM). Nótese que una entidad de transporte normalmente reconoce varios TSAP. En algunas redes también existen múltiples NSAP, pero en otras cada máquina sólo tiene un NSAP (por ejemplo, una dirección IP). Una posible situación de conexión para una conexión de transporte a través de una capa de red orientada a conexiones es la siguiente.

1. Un proceso servidor de hora del día del *host* 2 se conecta al TSAP 122 para esperar una llamada entrante. La manera en que un proceso se conecta a un TSAP está fuera del modelo de red y depende por entero del sistema operativo local. Podría, por ejemplo, usarse una llamada como nuestra LISTEN.
2. Un proceso de aplicación del *host* 1 quiere averiguar la hora del día, por lo que emite una solicitud CONNECT especificando el TSAP 6 como el origen y el TSAP 122 como destino.
3. La entidad de transporte del *host* 1 selecciona una dirección de red en su máquina (si tiene más de una) y establece una conexión de red entre ellas. (En una subred sin conexiones, no se establecería esta conexión de capa de red.) Usando esta conexión de capa de red, la entidad de transporte del *host* 1 puede hablar con la entidad de transporte del *host* 2.
4. Lo primero que dice la entidad de transporte de 1 a su igual de 2 es: "Buenos días. Me gustaría establecer una conexión de transporte entre mi TSAP 6 y tu TSAP 122. ¿Qué dices?"

5. La entidad de transporte de 2 pregunta entonces al servidor de hora del día en el TSAP 122 si está dispuesto a aceptar una conexión nueva. Si está de acuerdo, se establece la conexión de transporte.

Nótese que la conexión de transporte va de TSAP a TSAP, mientras que la conexión de red sólo cubre una parte del camino, de NSAP a NSAP.

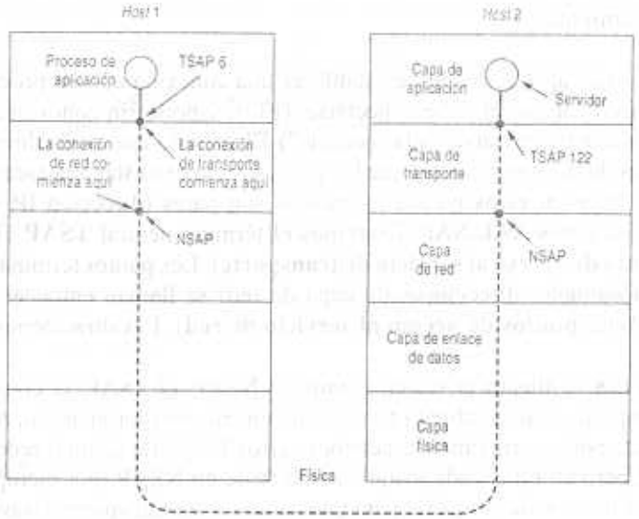


Figura 6-8. Los TSAP, NSAP y conexiones.

El panorama que hemos bosquejado está muy bien, excepto que hemos escondido un pequeño problema: ¿cómo sabe el proceso de usuario del *host* 1 que el servidor de hora del día (*time-of-day server*) está conectado al TSAP 122? Una posibilidad es que el servidor de hora del día se ha estado conectando al TSAP 122 durante años, y gradualmente todos los usuarios de la red han aprendido esto. En este modelo, los servicios tienen direcciones TSAP estables, que pueden imprimirse en papel y darse a los nuevos usuarios al unirse a la red.

Aunque las direcciones TSAP estables podrían funcionar bien con una cantidad pequeña de servicios clave que nunca cambian, en general los procesos de usuario frecuentemente quieren hablar con otros procesos de usuario que sólo existen durante un tiempo corto y no tienen una dirección TSAP conocida por adelantado. Es más, si puede haber muchos procesos de servidor, la mayoría de los cuales se usan pocas veces, es un desperdicio tenerlos activos a todos, escuchando en una dirección TSAP estable todo el día. En pocas palabras, se requiere un esquema mejor.

Uno de tales esquemas, empleado por los *hosts* UNIX de Internet, se muestra en la figura 6-9 de una manera simplificada; se conoce como **protocolo inicial de conexión**. En lugar de que cada servidor concebible escuche en un TSAP bien conocido, cada máquina que desea ofrecer

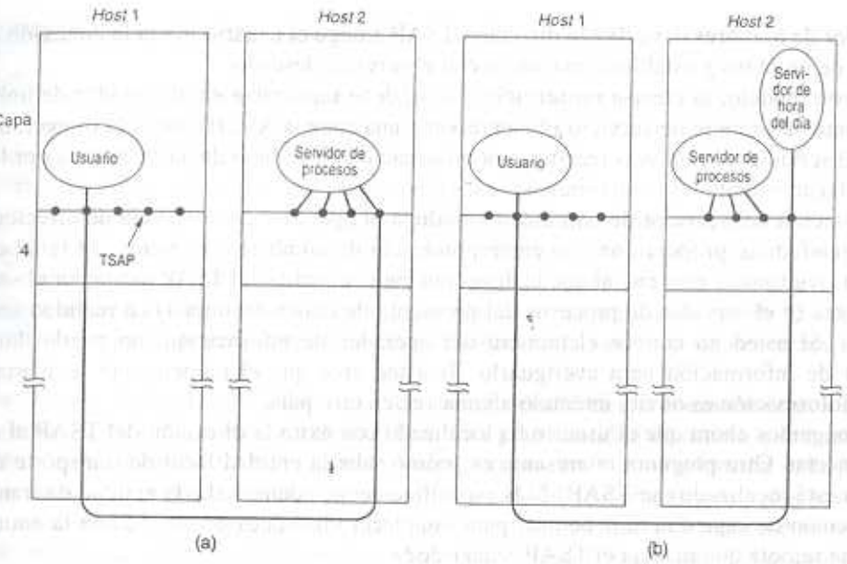


Figura 6-9. Manera en que un proceso de usuario del *host* 1 establece una conexión con un servidor de hora del día del *host* 2.

servicio a usuarios remotos tiene un **servidor de procesos** especial que actúa como apoderado (*proxy*) de los servidores de menor uso y escucha en un grupo de puertos al mismo tiempo, esperando una solicitud de conexión TCP. Los usuarios potenciales de un servicio comienzan por emitir una solicitud CONNECT, especificando la dirección TSAP (puerto TCP) del servicio que desean. Si no hay ningún servidor esperándolos, consiguen una conexión al servidor de procesos, como se muestra en la figura 6-9(a).

Tras obtener la solicitud entrante, el servidor de procesos genera el servidor solicitado, permitiéndole heredar la conexión con el usuario existente. El nuevo servidor entonces hace el trabajo requerido, mientras que el servidor de procesos retorna a escuchar solicitudes nuevas, como se muestra en la figura 6-9(b).

Aunque el protocolo de conexión inicial funciona bien para aquellos servidores que pueden crearse cuando se necesitan, hay muchas situaciones en las que los servicios existen independientemente del servidor de procesos. Por ejemplo, un servidor de archivos necesita operar un *hardware* especial (una máquina con un disco) y no puede simplemente crearse sobre la marcha cuando alguien quiere hablar con él.

Para manejar esta situación, se usa con frecuencia un esquema alterno. En este modelo, existe un proceso especial llamado **servidor de nombres**, o a veces **servidor de directorio**. Para encontrar la dirección TSAP correspondiente a un nombre de servicio dado, como "hora del día", el usuario establece una conexión con el servidor de nombres (que escucha en un TSAP bien conocido). Entonces el usuario envía un mensaje especificando el nombre del servicio, y

el servidor de nombres devuelve la dirección TSAP. Luego el usuario libera la conexión con el servidor de nombres y establece una nueva con el servicio deseado.

En este modelo, al crearse un servicio nuevo, debe registrarse en el servidor de nombres, dando tanto su nombre de servicio (generalmente una cadena ASCII) como la dirección de su TSAP. El servidor de nombres registra esta información en su base de datos interna por lo que, cuando llegan solicitudes posteriores, sabe las respuestas.

La función del servidor de nombres es análoga al operador de asistencia de directorio del sistema telefónico: proporciona una correspondencia de nombres a números. Al igual que en el sistema telefónico, es esencial que la dirección bien conocida del TSAP usada por el servidor de nombres (o el servidor de procesos del protocolo de conexión inicial) en realidad sea bien conocida. Si usted no conoce el número del operador de información, no puede llamar al operador de información para averiguarlo. Si usted cree que el número que se marca para obtener información es obvio, inténtelo alguna vez en otro país.

Supongamos ahora que el usuario ha localizado con éxito la dirección del TSAP al que se debe conectar. Otra pregunta interesante es ¿cómo sabe la entidad local de transporte en qué máquina está localizado ese TSAP? Más específicamente, ¿cómo sabe la entidad de transporte qué dirección de capa de red debe usar para establecer una conexión de red con la entidad de transporte remota que maneja el TSAP requerido?

La respuesta depende de la estructura de las direcciones TSAP. Una estructura posible es que las direcciones TSAP sean **direcciones jerárquicas**. Con direcciones jerárquicas, la dirección consiste en una secuencia de campos usados para dividir en porciones disconexas el espacio de direcciones. Por ejemplo, una dirección TSAP verdaderamente universal podría tener la siguiente estructura:

dirección = <galaxia> <estrella> <planeta> <país> <red> <*host*> <puerto>

Con este esquema es directa la localización de un TSAP en cualquier parte del universo conocido. De manera equivalente, si una dirección TSAP es una concatenación de una dirección NSAP y un puerto (un identificador local que especifica uno de los TSAP locales), entonces, cuando se le da una dirección TSAP a una entidad de transporte para conectarse a ella, usa la dirección NSAP contenida en la dirección TSAP para alcanzar la entidad de transporte remota adecuada.

Como ejemplo sencillo de dirección jerárquica, considere el número telefónico 19076543210. Este número puede analizarse como 1-907-654-3210, donde 1 es un código de país (Estados Unidos + Canadá), 907 es un código de área (Alaska), 654 es una oficina terminal de Alaska, y 3210 es uno de los "puertos" (líneas de suscriptor) de esa oficina terminal.

La alternativa al espacio jerárquico de direcciones es el **espacio plano de direcciones**. Si las direcciones TSAP no son jerárquicas, se requiere un segundo nivel de transformación para localizar la máquina adecuada. Tendría que haber un servidor de nombres que tomara direcciones de puerto como entrada y devolviera direcciones de red como salida. De manera alterna, en algunas situaciones (por ejemplo, en una LAN), es posible difundir una solicitud pidiendo a la máquina de destino que se identifique enviando un paquete.

6.2.2. Establecimiento de una conexión

El establecimiento de una conexión suena fácil, pero en realidad es sorprendentemente complicado. A primera vista, parecería suficiente con que una entidad de transporte enviara una TPDU CONNECTION REQUEST al destino y esperara una respuesta CONNECTION ACCEPTED. El problema ocurre cuando la red puede perder, almacenar o duplicar paquetes.

Imagine una subred que está tan congestionada que los reconocimientos casi nunca regresan a tiempo, y cada paquete termina de temporizar y se retransmite dos o tres veces. Suponga que la subred usa datagramas internamente, y que cada paquete sigue una ruta diferente. Algunos de los paquetes podrían atorarse en un congestionamiento de tráfico dentro de la subred y tardar mucho tiempo en llegar, es decir, se almacenarían en la subred y reaparecerían mucho después.

La peor pesadilla posible es la que sigue. Un usuario establece una conexión con un banco, envía mensajes indicando al banco que transfiera una gran cantidad de dinero a la cuenta de una persona no del todo confiable, y entonces libera la conexión. Por mala fortuna, cada paquete de la transacción se duplica y almacena en la subred. Tras liberar la conexión, todos los paquetes salen de la subred y llegan al destino en orden, solicitando al banco que establezca una conexión nueva, transfiera el dinero (nuevamente) y libere la conexión. El banco no tiene manera de saber que son duplicados; debe suponer que ésta es una segunda transacción independiente, y transfiere nuevamente el dinero. Durante el resto de esta sección estudiaremos el problema de los duplicados retardados, haciendo hincapié en los algoritmos para establecer conexiones de una manera confiable, de modo que pesadillas como la anterior no puedan ocurrir.

El meollo del problema es la existencia de duplicados retrasados. Esto puede atacarse de varias maneras, ninguna de las cuales es muy satisfactoria. Una es usar direcciones de transporte desechables. En este enfoque, cada vez que se requiere una dirección de transporte, se genera una nueva. Al liberarse la conexión, se descarta la dirección. Esta estrategia imposibilita el modelo de servidor de procesos de la figura 6-9.

Otra posibilidad es dar a cada conexión un identificador de conexión (es decir, un número de secuencia que se incremente con cada conexión establecida), seleccionado por la parte iniciadora, y ponerlo en cada TPDU, incluida la que solicita la conexión.

Tras la liberación de una conexión, cada entidad de transporte podría actualizar una tabla que liste conexiones obsoletas como pares (entidad de transporte igual, identificador de conexión). Cada vez que entrara una solicitud de conexión, podría cotejarse con la tabla para ver si pertenece a una conexión previamente liberada.

Por desgracia, este esquema tiene una falla básica: requiere que cada entidad de transporte mantenga una cierta cantidad de información histórica durante un tiempo indefinido. Si se cae una máquina, perdiéndose su memoria, ya no sabrá qué identificadores de conexión usó.

Más bien, necesitamos un enfoque diferente. En lugar de permitir que los paquetes vivan eternamente en la subred, debemos diseñar un mecanismo para matar a los paquetes viejos que

aún andan vagando por ahí. Si podemos asegurar que ningún paquete viva más allá de cierto tiempo conocido, el problema se vuelve algo más manejable.

El tiempo de vida de un paquete puede restringirse a un máximo conocido usando una de las siguientes técnicas:

1. Diseño de subred restringido.
2. Contador de saltos en cada paquete.
3. Marca de tiempo en cada paquete.

El primer método incluye cualquier método que evite que los paquetes hagan ciclos, combinado con una manera de limitar el retardo por congestionamientos a través de la trayectoria más larga posible (ahora conocida). El segundo método consiste en incrementar el conteo de saltos cada vez que se reenvía el paquete. El protocolo de enlace de datos simplemente descarta cualquier paquete cuyo contador de saltos ha excedido cierto valor. El tercer método requiere que cada paquete lleve la hora en la que fue creado, acordando los enrutadores descartar cualquier paquete que haya rebasado cierto tiempo predeterminado. Este último método requiere que los relojes de los enrutadores estén sincronizados, lo que no es una tarea fácil a menos que se logre la sincronización externamente a la red, por ejemplo escuchando la WWV o alguna otra estación de radio que difunda la hora exacta periódicamente.

En la práctica, necesitaremos garantizar no sólo que el paquete está muerto, sino también que todos los acuses de recibo están muertos, por lo que ahora introduciremos T, que es un múltiplo pequeño del tiempo de vida de paquete máximo verdadero. El múltiplo depende del protocolo, y simplemente tiene el efecto de hacer más grande a T. Si esperamos un tiempo T tras el envío de un paquete, podemos estar seguros de que todos los rastros suyos ya han desaparecido, y que ni él ni sus acuses de recibo aparecerán repentinamente de la nada para complicar el asunto.

Teniendo limitados los tiempos de vida de los paquetes, es posible proponer una manera a prueba de errores de establecer conexiones seguras. El método descrito a continuación se debe a Tomlinson (1975); resuelve el problema pero presenta algunas peculiaridades propias. El método fue refinado por Sunshine y Dalal (1978). En la práctica se usan ampliamente variantes suyas.

Para resolver el problema de una máquina que pierde toda la memoria acerca de su estado tras una caída, Tomlinson propuso equipar cada *host* con un reloj de hora del día. Los relojes de los diferentes *hosts* no necesitan estar sincronizados. Se supone que cada reloj tiene la forma de un contador binario que se incrementa a intervalos uniformes. Además, la cantidad de bits del contador debe ser igual o mayor que la cantidad de bits en los números de secuencia. Por último, y lo más importante, se supone que el reloj continúa operando aun ante la caída de un *host*.

La idea básica es asegurar que nunca estén pendientes dos TPDU de número idéntico al mismo tiempo. Cuando se establece una conexión, los k bits de orden menor del reloj se usan como número inicial de secuencia (también k bits). Por tanto, y a diferencia de los protocolos del capítulo 3, cada conexión comienza a numerar sus TPDU con un número de secuencia diferente. El espacio de secuencia también debe ser lo bastante grande para que, al dar la vuelta los

números de secuencia, las TPDU viejas con el mismo número de secuencia hayan desaparecido hace mucho tiempo. Esta relación lineal entre tiempo y números secuenciales iniciales se muestra en la figura 6-10.

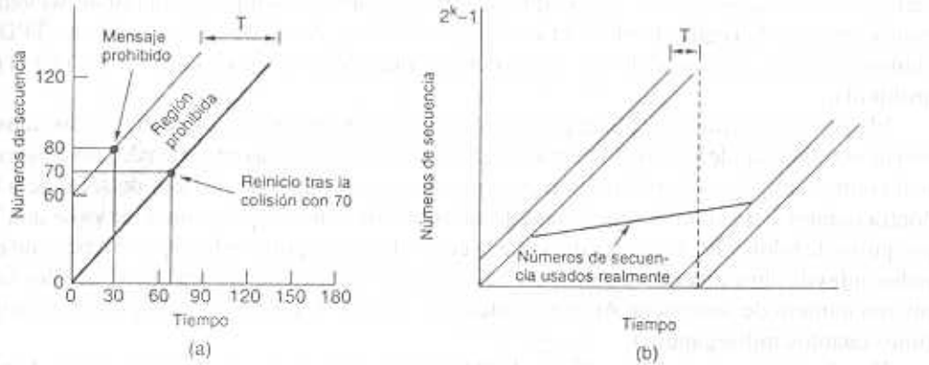


Figura 6-10. (a) Las TPDU no pueden entrar en la zona prohibida. (b) El problema de la resincronización.

Una vez que ambas entidades de transporte han acordado el número de secuencia inicial, puede usarse cualquier protocolo de ventana corrediza para el control de flujo de datos. En realidad, la curva inicial de números de secuencia (indicada por la línea gruesa) no es realmente lineal, sino una escalera, ya que el reloj avanza en pasos discretos. Por sencillez, ignoraremos este detalle.

Ocurre un problema cuando se cae un *host*. Al regresar nuevamente, sus entidades de transporte no saben dónde estaban en el espacio de secuencias. Una solución es requerir que las entidades de transporte estén inactivas durante T segundos tras una recuperación para permitir que todas las TPDU viejas mueran. Sin embargo, en una interred compleja, T puede ser bastante grande, por lo que no es atractiva esta estrategia.

Para evitar requerir T seg de tiempo muerto tras una caída, es necesario introducir una nueva restricción en el uso de números de secuencia. Podemos ver claramente la necesidad de esta restricción mediante un ejemplo. Sea T, el tiempo máximo de vida de un paquete, 60 seg. El pulso del reloj es de uno por segundo. Como se muestra en la figura 6-10, el número de secuencia inicial de una conexión abierta en el momento x será x. Imagine que, en $t = 30$ seg, una TPDU de datos ordinaria enviada a través de la conexión 5, previamente abierta, recibe el número de secuencia 80. Llamemos a esta TPDU X. De inmediato tras el envío de la TPDU X, el *host* se cae y reinicia pronto. En $t = 60$, el *host* comienza a reabrir las conexiones 0 a 4. En $t = 70$, reabre la conexión 5, usando el número de secuencia inicial 70, como se requiere. Durante los siguientes 15 segundos envía las TPDU de datos 70 a 80. Por tanto, en $t = 85$ se ha inyectado una TPDU nueva con número de secuencia 80 y conexión 5 en la subred. Por desgracia, la TPDU X aún

existe. Si ésta llegara al receptor antes de la nueva TPDU 80, la TPDU *X* sería aceptada y se rechazaría la TPDU 80 correcta como duplicado.

Para evitar tales problemas, debemos evitar la asignación de números de secuencia nuevos (es decir, asignados a TPDU nuevas) durante un tiempo *T* antes de su uso potencial como números iniciales de secuencia. Las combinaciones ilegales de tiempo y número de secuencia se muestran como la **región prohibida** en la figura 6-10(a). Antes de enviar cualquier TPDU por alguna conexión, la entidad de transporte debe leer el reloj y comprobar que no está en la región prohibida.

El protocolo puede meterse en problemas de dos maneras. Si un *host* envía demasiados datos con demasiada rapidez a través de una conexión recién abierta, la curva de número de secuencia real contra tiempo puede subir con mayor rapidez que la curva de número de secuencia inicial contra tiempo. Esto significa que la tasa de datos máxima en cualquier conexión es de una TPDU por pulso de reloj, y también significa que la entidad de transporte debe esperar hasta que el reloj pulse antes de abrir una nueva conexión tras un reinicio por caída, no sea que se use dos veces el mismo número de secuencia. Ambos puntos son argumentos a favor de un pulso de reloj corto (unos cuantos milisegundos).

Desafortunadamente, el ingreso en la región prohibida desde abajo al enviar con demasiada rapidez no es la única manera de meterse en problemas. Por la figura 6-10(b) debe quedar claro que con cualquier tasa de datos menor que la tasa del reloj, la curva de números de secuencia reales usados contra tiempo tarde o temprano entrará en la región prohibida por la izquierda. Cuanto mayor sea la pendiente de la curva de números de secuencia reales, mayor será el retardo de este evento. Como indicamos antes, justo antes de enviar cada TPDU, la entidad de transporte debe comprobar que no está a punto de entrar en la región prohibida; de ser así, debe retardar la TPDU durante *T* seg o resincronizar los números de secuencia.

El método basado en reloj resuelve el problema del duplicado retrasado de las TPDU de datos, pero para que este método resulte de utilidad, debe establecerse primero una conexión. Dado que las TPDU de control también pueden retrasarse, hay el problema potencial de lograr que ambos lados acuerden en número de secuencia inicial. Supóngase, por ejemplo, que se establecen conexiones haciendo que el *host* 1 envíe una TPDU CONNECTION REQUEST con el número de secuencia inicial y el número de puerto de destino a un igual remoto, el *host* 2. El receptor, el *host* 2, reconoce entonces esta solicitud enviando de regreso una TPDU CONNECTION ACCEPTED. Si la TPDU CONNECTION REQUEST se pierde, pero aparece con retardo una CONNECTION REQUEST duplicada en el *host* 2, se establecerá incorrectamente la conexión.

Para resolver este problema, Tomlinson (1975) desarrolló el **protocolo de acuerdo de tres vías** (*three-way handshake*). Este protocolo de establecimiento no requiere que ambos lados comiencen a transmitir con el mismo número de secuencia, por lo que puede usarse con otros métodos de sincronización distintos del método de reloj global. El procedimiento normal de establecimiento al iniciar el *host* 1 se muestra en la figura 6-11(a). El *host* 1 escoge un número de secuencia, *x*, y envía al *host* 2 una TPDU CONNECTION REQUEST que lo contiene. El *host* 2 responde con una TPDU CONNECTION ACCEPTED reconociendo *x* y anunciando su propio número de secuencia inicial, *y*. Por último, el *host* 1 reconoce la selección de un número de secuencia inicial del *host* 2 en la primera TPDU de datos que envía.

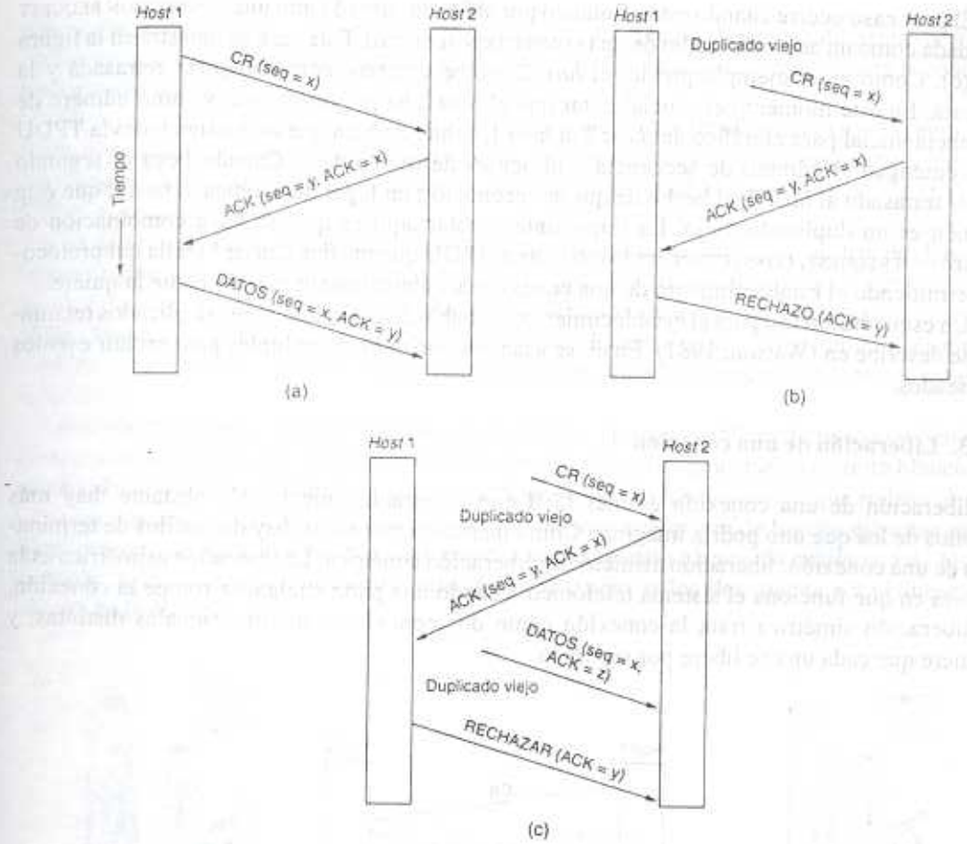


Figura 6-11. Tres situaciones de protocolo para establecer una conexión usando un protocolo de acuerdo de tres vías (*three-way handshake*). CR y ACK significan SOLICITUD DE CONEXIÓN y CONEXIÓN ACEPTADA, respectivamente. (a) Operación normal. (b) SOLICITUD DE CONEXIÓN duplicada vieja que aparece de la nada. (c) SOLICITUD DE CONEXIÓN duplicada y ACK duplicada.

Ahora veamos la manera en que funciona el protocolo de acuerdo de tres vías en presencia de una TPDU de control duplicada con retraso. En la figura 6-12(b), la primera TPDU es una CONNECTION REQUEST duplicada con retraso de una conexión vieja. Esta TPDU llega al *host* 2 sin el conocimiento del *host* 1. El *host* 2 reacciona a esta TPDU enviando al *host* 1 una TPDU CONECTION ACCEPTED, solicitando de hecho la comprobación de que el *host* 1 en verdad trató de establecer una nueva conexión. Al rechazar el *host* 1 el intento de establecimiento del *host* 2, éste se da cuenta de que fue engañado por un duplicado con retardo y abandona la conexión. De esta manera, un duplicado con retardo no causa daño.

El peor caso ocurre cuando están flotando por ahí en la subred tanto una CONNECTION REQUEST retardada como un acuse de recibo de una CONNECTION ACCEPTED. Este caso se muestra en la figura 6-11(c). Como en el ejemplo previo, el *host* 2 recibe una CONNECTION REQUEST retrasada y la contesta. En este momento es crucial notar que el *host* 2 ha propuesto usar y como número de secuencia inicial para el tráfico del *host* 2 al *host* 1, sabiendo bien que no existen todavía TPDU que contengan el número de secuencia *y* ni acuses de recibo de *y*. Cuando llega el segundo TPDU retrasado al *host* 2, el hecho de que se reconoció *z* en lugar de *y* indica al *host* 2 que éste también es un duplicado viejo. Lo importante a notar aquí es que no haya combinación de CONNECTION REQUEST, CONNECTION ACCEPTED u otras TPDU que puedan causar la falla del protocolo, permitiendo el establecimiento de una conexión accidentalmente cuando nadie la quiere.

Un esquema alterno para el establecimiento confiable de conexiones ante duplicados retardados se describe en (Watson, 1981). En él, se usan temporizadores múltiples para excluir eventos indeseados.

6.2.3. Liberación de una conexión

La liberación de una conexión es más fácil que su establecimiento. No obstante, hay más escollos de los que uno podría imaginar. Como mencionamos antes, hay dos estilos de terminación de una conexión: liberación asimétrica y liberación simétrica. La liberación asimétrica es la manera en que funciona el sistema telefónico: cuando una parte cuelga, se rompe la conexión. La liberación simétrica trata la conexión como dos conexiones unidireccionales distintas, y requiere que cada una se libere por separado.

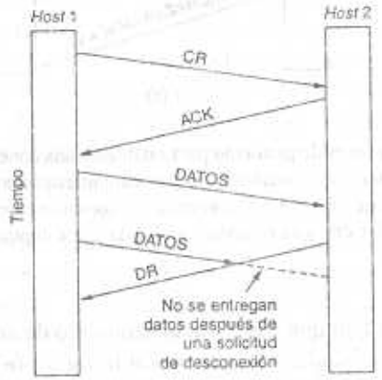


Figura 6-12. Desconexión abrupta con pérdida de datos.

La liberación asimétrica es abrupta y puede resultar en la pérdida de datos. Considere la situación de la figura 6-12. Tras establecerse la conexión, el *host* 1 envía una TPDU que llega

adecuadamente al *host* 2. Entonces el *host* 1 envía otra TPDU. Desgraciadamente, el *host* 2 emite una DISCONNECT antes de llegar la segunda TPDU. El resultado es que se libera la conexión y se pierden datos.

Es obvio que se requiere un protocolo de liberación más refinado para evitar la pérdida de datos. Una posibilidad es usar la liberación simétrica, en la que cada dirección se libera independientemente de la otra. Aquí, un *host* puede continuar enviando datos aun tras haber enviado una TPDU DISCONNECT.

La liberación simétrica es ideal cuando cada proceso tiene una cantidad fija de datos por enviar y sabe con certidumbre cuándo los ha enviado. En otras situaciones, la determinación de si se ha efectuado o no todo el trabajo y si debe terminarse o no la conexión no es tan obvia. Podríamos pensar en un protocolo en el que el *host* 1 diga: "Ya terminé. ¿Terminaste también?" Si el *host* 2 responde: "Ya terminé también. Adiós", la conexión puede liberarse con seguridad.

Por desgracia, este protocolo no siempre funciona. Hay un problema famoso que tiene que ver con este asunto; se llama **problema de los dos ejércitos**. Imagine que un ejército blanco está acampado en un valle, como se muestra en la figura 6-13. En los dos cerros que rodean al valle hay ejércitos azules. El ejército blanco es más grande que cualquiera de los dos ejércitos azules por separado, pero juntos éstos son más grandes que el ejército blanco. Si cualquiera de los dos ejércitos azules ataca por su cuenta, será derrotado, pero si los dos atacan simultáneamente obtendrán la victoria.

Figura 6-13. Problema de los dos ejércitos.

Los ejércitos azules quieren sincronizar sus ataques. Sin embargo, su único medio de comunicación es el envío de mensajeros a pie a través del valle, donde podrían ser capturados, perdiéndose el mensaje (es decir, tienen que usar un canal de comunicación no confiable). La pregunta es: ¿Existe un protocolo que permita que los ejércitos azules ganen?

Supóngase que el comandante del ejército azul núm. 1 envía un mensaje que dice: "Propongo que ataquemos al amanecer del 29 de marzo. ¿Qué les parece?" Ahora supongamos que llega

el mensaje y que el comandante del ejército azul núm. 2 está de acuerdo, y que su respuesta llega con seguridad al ejército azul núm. 1. ¿Ocurrirá el ataque? Probablemente no, porque el comandante núm. 2 no sabe si su respuesta llegó. Si no llegó, el ejército azul núm. 1 no atacará, y sería tonto de su parte emprender el ataque.

Mejoremos ahora el protocolo haciéndolo un protocolo de acuerdo de tres vías (*three-way handshake*). El iniciador de la propuesta original debe dar un acuse de recibo de la respuesta. Suponiendo que no se pierden mensajes, el ejército azul núm. 2 recibirá el acuse de recibo, pero ahora el que dudará será el comandante del ejército azul núm. 1. A fin de cuentas, no sabe si ha llegado su acuse de recibo y, si no llegó, sabe que el ejército azul núm. 2 no atacará. Podríamos probar ahora un protocolo de acuerdo de cuatro vías, pero tampoco ayudaría.

De hecho, puede demostrarse que no existe un protocolo que funcione. Supóngase que existiera algún protocolo. O el último mensaje del protocolo es esencial, o no lo es. Si no lo es, hay que eliminarlo (así como los demás mensajes no esenciales) hasta que quede un protocolo en el que todos los mensajes sean esenciales. ¿Qué ocurre si el mensaje final no pasa? Acabamos de decir que es esencial, por lo que, si se pierde, el ataque no ocurrirá. Dado que el transmisor del mensaje final nunca puede estar seguro de su llegada, no se arriesgará a atacar. Peor aún, el otro ejército azul sabe esto, por lo que no atacará tampoco.

Para ver la aplicación del problema de los dos ejércitos a la liberación de conexiones, simplemente sustituya "atacar" por "desconectar". Si ninguna de las partes está preparada para desconectarse hasta estar convencida de que la otra está preparada para desconectarse también, nunca ocurrirá la desconexión.

En la práctica, generalmente estamos más dispuestos a correr riesgos al liberar conexiones que al atacar ejércitos blancos, por lo que la situación no es del todo desesperada. En la figura 6-14 se ilustran cuatro situaciones de liberación usando un protocolo de acuerdo de tres vías. Aunque este protocolo no es infalible, generalmente es adecuado.

En la figura 6-14(a) vemos el caso normal en el que uno de los usuarios envía una TPDU DR (solicitud de desconexión, DISCONNECTION REQUEST) a fin de iniciar la liberación de una conexión. Al llegar, el receptor devuelve también una TPDU de DR e inicia un temporizador, para el caso de que se pierda su DR.

Al llegar esta DR, el transmisor original envía de regreso una TPDU de ACK y libera la conexión. La liberación de una conexión significa que la entidad de transporte remueve la información sobre la conexión de su tabla de conexiones abiertas y avisa de alguna manera al dueño de la conexión (el usuario de transporte). Esta acción es diferente de la emisión de una primitiva DISCONNECT por parte de un usuario de transporte.

Si se pierde la TPDU ACK, como se muestra en la figura 6-14(b), la situación se salva por el temporizador. Si expira el temporizador, se libera de todos modos la conexión.

Ahora consideremos el caso de la pérdida del segundo DR. El usuario que inicia la desconexión no recibirá la respuesta esperada, terminará de temporizar y comenzará de nuevo. En la figura 6-14(c) vemos la manera en que funciona esto, suponiendo que la segunda vez no se pierden algunas TPDU y que todas las TPDU se entregan correctamente y a tiempo.

Nuestra última situación, la figura 6-14(d), es la misma que la figura 6-14(c), excepto que ahora suponemos que todos los intentos repetidos de retransmitir la DR también fallan debido a la

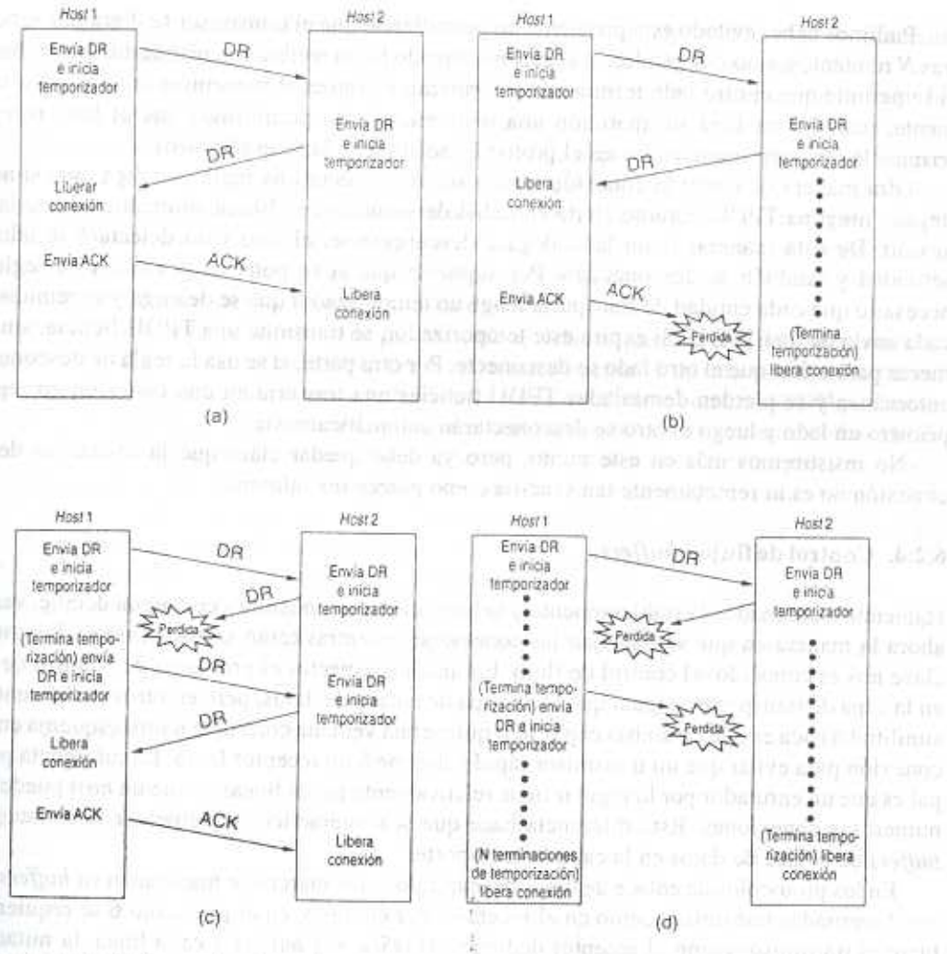


Figura 6-14. Cuatro situaciones de protocolo de liberación de una conexión. (a) Caso normal del protocolo de acuerdo de tres vías (*three-way handshake*). (b) Pérdida del último ACK. (c) Respuesta perdida. (d) Respuesta perdida y pérdida de las DR subsecuentes.

pérdida de algunas TPDU. Tras *N* reintentos, el transmisor simplemente se da por vencido y libera la conexión. Mientras tanto, el receptor termina de temporizar y también se sale.

Aunque este protocolo generalmente es suficiente, en teoría puede fallar si se pierden el DR inicial y *N* retransmisiones. El transmisor se dará por vencido y liberará la conexión, pero el otro lado no sabrá nada sobre los intentos de desconexión y seguirá plenamente activo. Esta situación resulta en una conexión medio abierta.

Pudimos haber evitado este problema no permitiendo que el transmisor se diera por vencido tras *N* reintentos, sino obligándolo a seguir insistiendo hasta recibir una respuesta. Sin embargo, si se permite que el otro lado termine de temporizar, entonces el transmisor continuará eternamente, pues nunca hará su aparición una respuesta. Si no permitimos que el lado receptor termine la temporización, entonces el protocolo se atora en la figura 6-14(b).

Otra manera de cortar la conexión medio abierta es tener una regla que diga que, si no ha llegado ninguna TPDU durante cierta cantidad de segundos, se libera automáticamente la conexión. De esta manera, si un lado llega a desconectarse, el otro lado detectará la falta de actividad y también se desconectará. Por supuesto que si se pone en práctica esta regla, es necesario que cada entidad de transporte tenga un temporizador que se detenga y se reinicie con cada envío de una TPDU. Si expira este temporizador, se transmite una TPDU ficticia, simplemente para evitar que el otro lado se desconecte. Por otra parte, si se usa la regla de desconexión automática y se pierden demasiadas TPDU ficticias una tras otra en una conexión en reposo, primero un lado y luego el otro se desconectarán automáticamente.

No insistiremos más en este punto, pero ya debe quedar claro que la liberación de una conexión no es ni remotamente tan sencilla como parece inicialmente.

6.2.4. Control de flujo y *buffers*

Habiendo examinado el establecimiento y la liberación de conexiones con algún detalle, veamos ahora la manera en que se manejan las conexiones mientras están en uso. Uno de los asuntos clave nos es conocido: el control de flujo. En muchos aspectos el problema del control de flujo en la capa de transporte es igual que en la capa de enlace de datos, pero en otros es diferente. La similitud básica es que en ambas capas se requiere una ventana corrediza u otro esquema en cada conexión para evitar que un transmisor rápido abrume a un receptor lento. La diferencia principal es que un enrutador por lo regular tiene relativamente pocas líneas, y que un *host* puede tener numerosas conexiones. Esta diferencia hace que sea impráctico implementar la estrategia de *buffers* de enlace de datos en la capa de transporte.

En los protocolos de enlace de datos del capítulo 3, los marcos se manejaban en *buffers* tanto en el enrutador transmisor como en el receptor. Por ejemplo, en el protocolo 6 se requiere que tanto el transmisor como el receptor dediquen *MaxSeq* + 1 *buffers* a cada línea, la mitad para entradas y la mitad para salidas. En un *host* con un máximo de, digamos, 64 conexiones y un número de secuencia de 4 bits, este protocolo requerirá 1024 *buffers*.

En la capa de enlace de datos, el lado transmisor debe poner en *buffer* los marcos de salida porque cabe la posibilidad de que tengan que retransmitirse. Si la subred provee un servicio de datagramas, la entidad de transporte transmisora también debe manejar *buffers*, por la misma razón. Si el receptor sabe que el transmisor pone en *buffer* todas las TPDU hasta que se reconocen, el receptor podría o no dedicar *buffers* específicos a conexiones específicas, según considere necesario. Por ejemplo, el receptor podría mantener un solo grupo de *buffers* compartido por todas las conexiones. Cuando entra una TPDU, se hace un intento por adquirir dinámicamente un *buffer* nuevo. Si hay uno disponible, se acepta la TPDU; de otro modo, se descarta. Dado que el transmisor está preparado para retransmitir las TPDU perdidas por la subred, no hay nada de

malo en hacer que el receptor se deshaga de las TPDU, aunque se desperdicien algunos recursos. El transmisor simplemente sigue intentando hasta que recibe un reconocimiento.

En resumen, si el servicio de red no es confiable, el transmisor debe poner en *buffer* todas las TPDU enviadas, igual que la capa de enlace de datos. Sin embargo, con un servicio de red confiable son posibles otras componendas. En particular, si el transmisor sabe que el receptor siempre tiene espacio de *buffer*, no necesita retener copias de las TPDU que envía. Sin embargo, si el receptor no puede garantizar que se aceptará cada TPDU de entrada, el transmisor tendrá que usar *buffers* de todas maneras. En el último caso, el transmisor no puede confiar en el acuse de recibo de la capa de red porque éste sólo significa que ha llegado la TPDU, no que ha sido aceptada. Regresaremos después a este importante punto.

Aun si el receptor ha acordado manejar los *buffers*, todavía queda la cuestión del tamaño de los *buffers*. Si la mayoría de las TPDU tiene aproximadamente el mismo tamaño, es natural organizar los *buffers* como una reserva de *buffers* de tamaño idéntico, con una TPDU por *buffer*, como en la figura 6-15(a). Sin embargo, si hay una variación grande en el tamaño de las TPDU, de unos cuantos caracteres ingresados en una terminal hasta miles de caracteres provenientes de transferencias de archivos, el fondo de *buffers* de tamaño fijo presenta problemas. Si el tamaño de los *buffers* se escoge igual a la TPDU más grande, se desperdiciará espacio cada vez que llegue una TPDU corta. Si el tamaño de *buffer* se escoge menor que el tamaño máximo de TPDU, se requerirán varios *buffers* para las TPDU grandes, con la complejidad asociada.

Otra forma de enfrentar el problema del tamaño de los *buffers* es el uso de *buffers* de tamaño variable, como en la figura 6-15(b). La ventaja aquí es un mejor uso de la memoria, al costo de

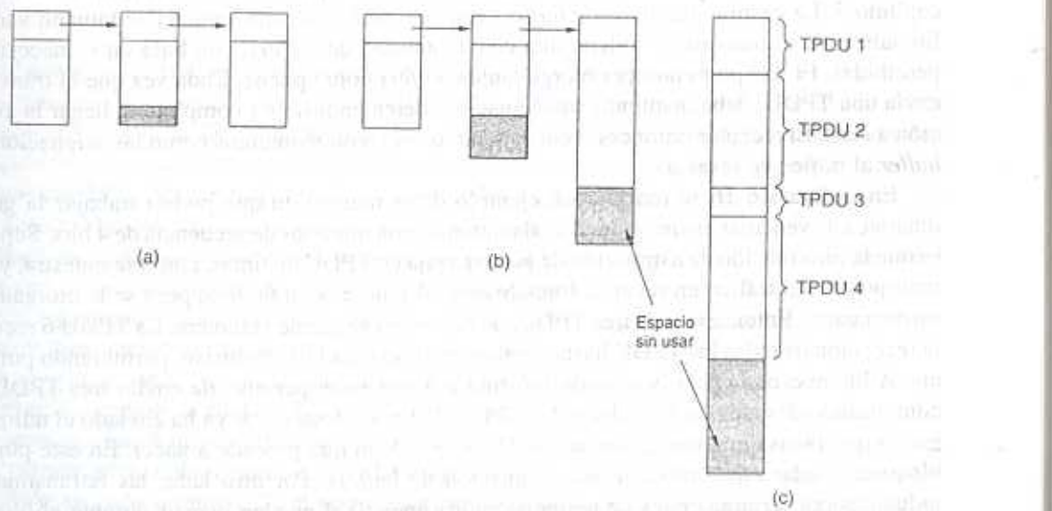


Figura 6-15. (a) *Buffers* encadenados de tamaño fijo. (b) *Buffers* encadenados de tamaño variable. (c) Un gran *buffer* circular por conexión.

una gestión de *buffers* más complicada. Una tercera posibilidad es dedicar un solo *buffer* circular grande por conexión, como en la figura 6-15(c). Este sistema también hace buen uso de la memoria cuando todas las conexiones tienen una carga alta, pero es deficiente si algunas conexiones son de poca carga.

La media óptima entre los *buffers* en el origen y los *buffers* en el destino depende del tipo de tráfico transportado por la conexión. Con un tráfico de bajo ancho de banda con ráfagas, como el producido por una terminal interactiva, es mejor no dedicarle *buffers*, sino adquirirlos de manera dinámica en ambos extremos. Dado que el transmisor no puede estar seguro de que el receptor será capaz de adquirir un *buffer*, el transmisor debe retener una copia de la TPDU hasta recibir su acuse de recibo. Por otra parte, con la transferencia de archivos y otro tráfico de alto ancho de banda, es mejor si el receptor dedica una ventana completa de *buffers*, para permitir el flujo de datos a máxima velocidad. Por tanto, para un tráfico en ráfagas de bajo ancho de banda, es mejor mantener *buffers* en el transmisor; para alto ancho de banda, es mejor hacerlo en el receptor.

A medida que se abren y cierran conexiones, y a medida que cambia el patrón de tráfico, el transmisor y el receptor necesitan ajustar dinámicamente sus asignaciones de *buffers*. En consecuencia, el protocolo de transporte debe permitir a un *host* transmisor solicitar espacio de *buffer* en el otro extremo. Los *buffers* podrían repartirse por conexión o, en conjunto, para todas las conexiones en operación entre los dos *hosts*. Como alternativa, el receptor, sabiendo su situación de *buffers* (pero sin saber el tráfico generado) podría indicar al transmisor "te he reservado *X* *buffers*". Si aumentase la cantidad de conexiones abiertas, podría ser necesario reducir una asignación, por lo que el protocolo debe contemplar esta posibilidad.

Una manera razonablemente general de manejar la asignación dinámica de *buffers* es desacoplarlos de los acuses de recibo, en contraste con los protocolos de ventana corrediza del capítulo 3. La gestión dinámica de *buffers* implica, de hecho, una ventana de tamaño variable. Inicialmente, el transmisor solicita una cierta cantidad de *buffers*, con base en su necesidades percibidas. El receptor entonces otorga tantos *buffers* como puede. Cada vez que el transmisor envía una TPDU, debe disminuir su asignación, deteniéndose por completo al llegar la asignación a cero. El receptor entonces incorpora tanto los reconocimientos como las asignaciones de *buffer* al tráfico en reversa.

En la figura 6-16 se muestra un ejemplo de la manera en que podría trabajar la gestión dinámica de ventanas en una subred de datagramas con números de secuencia de 4 bits. Supóngase que la información de asignación de *buffers* viaja en TPDU distintas, como se muestra, y no se incorpora en el tráfico en reversa. Inicialmente, *A* quiere ocho *buffers*, pero se le otorgan solamente cuatro. Entonces envía tres TPDU, de las cuales se pierde la tercera. La TPDU 6 reconoce la recepción de todas las TPDU hasta el número de secuencia 1, inclusive, permitiendo por tanto que *A* libere esos *buffers*, y además informa a *A* que tiene permiso de enviar tres TPDU más comenzando después de 1 (es decir, las TPDU 2, 3 y 4). *A* sabe que ya ha enviado el número 2, por lo que piensa que puede enviar las TPDU 3 y 4, lo que procede a hacer. En este punto se bloquea y debe esperar una nueva asignación de *buffers*. Por otro lado, las retransmisiones inducidas por terminaciones de temporización (línea 9) sí pueden ocurrir durante el bloqueo, pues usan *buffers* ya asignados. En la línea 10, *B* reconoce la recepción de todas la TPDU hasta la 4, inclusive, pero se niega a permitir que *A* continúe. Tales situaciones son imposibles con los

protocolos de ventana fija del capítulo 3. La siguiente TPDU de *B* a *A* asigna otro *buffer* y permite a *A* continuar.

	A	Mensaje	B	Comentarios
1	→	<solicito 8 *buffers*>	→	A quiere 8 *buffers*
2	←	<ack = 15, buf = 4>	←	B solo otorga los mensajes 0 a 3
3	→	<seq = 0, data = m0>	→	A tiene 3 *buffers* libres
4	→	<seq = 1, data = m1>	→	A tiene 2 *buffers* libres
5	→	<seq = 2, data = m2>	...	Mensaje perdido, pero A piensa que le queda 1
6	←	<ack = 1, buf = 3>	←	B reconoce 0 y 1, permite 2-4
7	→	<seq = 3, data = m3>	→	A tiene un *buffer* libre
8	→	<seq = 4, data = m4>	→	A tiene 0 *buffers* libres, y debe detenerse
9	→	<seq = 2, data = m2>	→	A termina de temporizar y retransmite
10	←	<ack = 4, buf = 0>	←	Todo reconocido, pero A bloqueado aún
11	←	<ack = 4, buf = 1>	←	A puede enviar 5 ahora
12	←	<ack = 4, buf = 2>	←	B encontró un *buffer* nuevo en algún lado
13	→	<seq = 5, data = m5>	→	A tiene 1 *buffer* libre
14	→	<seq = 6, data = m6>	→	A está bloqueado nuevamente
15	←	<ack = 6, buf = 0>	←	A aún está bloqueado
16	...	<ack = 6, buf = 4>	←	Interbloqueo potencial

Figura 6-16. Asignación dinámica de buffers. Las flechas muestran la dirección de la transmisión. Los puntos suspensivos (...) indican una TPDU perdida.

Pueden surgir problemas potenciales con los esquemas de asignación de *buffers* de este tipo en las redes de datagramas si hay pérdidas de TPDU. Observe la línea 16. *B* ha asignado ahora más *buffers* a *A*, pero la TPDU de asignación se perdió. Dado que las TPDU de control no están en secuencia, *A* se encuentra estancado. Para evitar esta situación, cada *host* debe enviar periódicamente una TPDU de control con el acuse de recibo y estado de *buffers* de cada conexión. De esta manera, el estancamiento se romperá tarde o temprano.

Hasta ahora hemos supuesto tácitamente que el único límite impuesto a la tasa de datos del transmisor es la cantidad de espacio de *buffer* disponible en el receptor. A medida que siga cayendo significativamente el precio de la memoria, podría llegar a ser factible equipar a los *hosts* con tanta memoria que la falta de *buffers* dejaría de ser un problema.

Si el espacio de *buffer* ya no limita el flujo máximo, aparecerá otro cuello de botella: la capacidad de carga de la subred. Si enrutadores adyacentes pueden intercambiar cuando mucho *x* marcos/seg y hay *k* trayectorias sin traslapo entre un par de *hosts*, no hay manera de que esos *hosts* puedan intercambiar más de *kx* TPDU/seg, sin importar la cantidad de espacio de *buffer* disponible en cada terminal. Si el transmisor presiona demasiado (es decir, envía más de *kx* TPDU/seg), la subred se congestionará, pues será incapaz de entregar las TPDU a la velocidad con que llegan.

Lo que se necesita es un mecanismo basado en la capacidad de carga de la subred en lugar de la capacidad de *buffer* del receptor. Es claro que el mecanismo de control de flujo debe aplicarse al transmisor para evitar que estén pendientes demasiadas TPDU sin acuse de recibo al mismo tiempo. Belsnes (1975) propuso el uso de un esquema de flujo de ventana corrediza en el que el transmisor ajusta dinámicamente el tamaño de la ventana para igualarla a la capacidad de carga de la red. Si la red puede manejar c TPDU/seg y el tiempo de ciclo (incluidos transmisión, propagación, encolamiento, proceso en el receptor y devolución del acuse de recibo) es de r, entonces la ventana del transmisor debe ser cr. Con una ventana de este tamaño, el transmisor normalmente opera con la tubería llena. Cualquier pequeña disminución en el desempeño de la red causará que se bloquee.

A fin de ajustar periódicamente el tamaño de la ventana, el transmisor podría vigilar ambos parámetros y calcular después el tamaño de ventana deseado. La capacidad de carga puede determinarse con sólo contar la cantidad de TPDU con acuse de recibo durante algún periodo de tiempo y dividirla entre el periodo de tiempo. Durante la medición, el transmisor debe enviar a la mayor velocidad posible, para asegurarse que sea la capacidad de carga de la red, y no la baja tasa de entrada, el factor limitante de la tasa de acuses de recibo. El tiempo requerido para el acuse de recibo de una TPDU transmitida puede medirse exactamente y mantenerse una media de operación. Dado que la capacidad de la red depende de la cantidad de tráfico en ella, debe ajustarse el tamaño de la ventana con frecuencia para responder a los cambios en la capacidad de carga. Como veremos después, la Internet usa un esquema parecido.

6.2.5. Multiplexión

La multiplexión de varias conversaciones en conexiones, circuitos virtuales y enlaces físicos desempeña un papel importante en diferentes capas de la arquitectura de la red. En la capa de transporte puede surgir la necesidad de multiplexión por varias razones. Por ejemplo, en las redes que usan circuitos virtuales en la subred, cada conexión abierta consume espacio de tabla en los enrutadores durante la duración completa de la conexión. Si también se dedican *buffers* al circuito virtual en cada enrutador, un usuario que haya dejado establecida una sesión entre una terminal y una máquina remota durante el descanso para el café estará consumiendo recursos caros. Aunque esta implementación de la conmutación de paquetes echa por tierra una de las razones principales para tener conmutación de paquetes (facturar al usuario con base en la cantidad de datos enviados, no el tiempo de conexión), muchas portadoras han escogido este enfoque porque se parece mucho al modelo de conmutación de circuitos al que se han acostumbrado por décadas.

La consecuencia de una estructura de precios que penaliza fuertemente a las instalaciones que tienen muchos circuitos virtuales abiertos durante largos periodos de tiempo es que se vuelve atractiva la multiplexión de diferentes conexiones de transporte en la misma conexión de red. Esta forma de multiplexión, llamada **multiplexión ascendente**, se muestra en la figura 6-17(a). En esta figura, cuatro conexiones de transporte diferentes usan la misma conexión de red (por ejemplo, un circuito virtual ATM) al *host* remoto. Cuando el tiempo de conexión es un

componente importante de la facturación de la portadora, es responsabilidad de la capa de transporte agrupar las conexiones de transporte de acuerdo con su destino y hacer corresponder cada grupo con la mínima cantidad de conexiones de red. Si se hacen corresponder demasiadas conexiones de transporte con una sola conexión de red, el rendimiento será bajo, porque la ventana generalmente estará llena y los usuarios tendrán que esperar su turno para enviar un mensaje. Si se hacen corresponder muy pocas conexiones con una conexión de red, el servicio será caro. Al usarse multiplexión ascendente con ATM, tenemos la situación irónica (¿trágica?) de tener que identificar la conexión usando un campo de la cabecera de transporte, aunque ATM proporciona expresamente para ese propósito más de 4000 números de circuito virtual por trayectoria virtual.

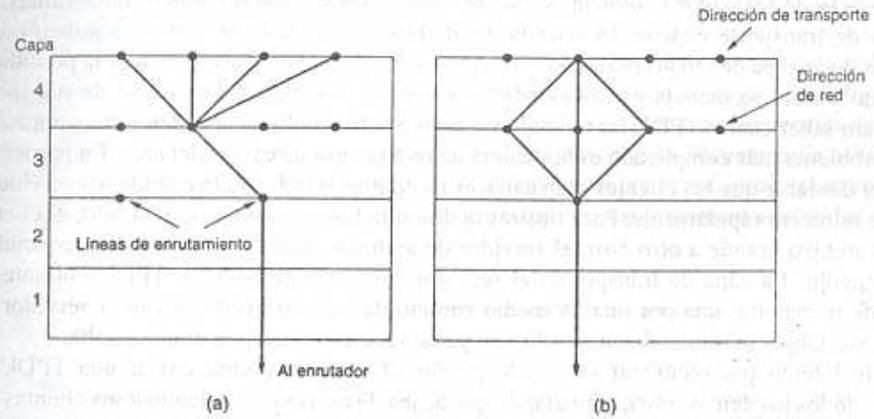


Figura 6-17. (a) Multiplexión ascendente. (b) Multiplexión descendente.

La multiplexión también puede ser útil en la capa de transporte por otra razón, relacionada con las decisiones técnicas de la portadora, en lugar de sus decisiones de precio. Supóngase, por ejemplo, que cierto usuario importante necesita una conexión de alto ancho de banda de tarde en tarde. Si la subred obliga a un control de flujo de ventana corrediza con un número de secuencia de n bits, el usuario deberá detener el envío tan pronto estén pendientes $2^n - 1$ paquetes y deberá esperar que los paquetes se propaguen al *host* remoto y se reconozcan. Si la conexión física es vía satélite, el usuario está limitado de hecho a $2^n - 1$ paquetes cada 540 mseg. Con, por ejemplo, $n = 8$ y paquetes de 128 bytes, el ancho de banda útil es de 484 kbps, aunque el ancho de banda del canal físico sea más de 100 veces mayor.

Una posible solución es hacer que la capa de transporte abra varias conexiones de red y distribuya el tráfico entre ellas por turno circular, como se indica en la figura 6-17(b). Este *modus operandi* se llama **multiplexión descendente**. Con k conexiones de red abiertas, el ancho

de banda efectivo se aumenta en un factor de k. Con 4095 circuitos virtuales, paquetes de 128 bytes y un número de secuencia de 8 bits, en teoría es posible lograr tasas de datos de más de 1.6 Gbps. Por supuesto que este desempeño puede lograrse sólo si la línea de salida puede manejar 1.6 Gbps, puesto que los 4095 circuitos virtuales aún se envían a través de una línea física, cuando menos en la figura 6-17(b). Si hay disponibles varias líneas de salida, puede usarse también la multiplexión descendente para aumentar aún más el desempeño.

6.2.6. Recuperación de caídas

Si los *hosts* y los enrutadores están sujetos a caídas, la recuperación de éstas se vuelve un tema importante. Si la entidad de transporte está por entero dentro de los *hosts*, la recuperación de caídas de red y de enrutador es directa. Si la capa de red proporciona servicio de datagramas, las entidades de transporte esperan la pérdida de algunas TPDU todo el tiempo, y saben cómo manejarla. Si la capa de red proporciona servicio orientado a conexiones, entonces la pérdida de un circuito virtual se maneja estableciendo uno nuevo y sondeando la entidad de transporte remota para saber cuáles TPDU ha recibido y cuáles no. Estas últimas pueden retransmitirse.

Un problema más complicado es la manera de recuperarse de caídas del *host*. En particular, podría ser deseable que los clientes sean capaces de continuar trabajando cuando los servidores caen y se reinician rápidamente. Para ilustrar la dificultad, supongamos que un *host*, el cliente, envía un archivo grande a otro *host*, el servidor de archivos, usando un protocolo de parada y espera sencillo. La capa de transporte del servidor simplemente pasa las TPDU entrantes al usuario de transporte, una por una. A medio camino de la transmisión se cae el servidor. Al regresar, sus tablas se reinicializan, por lo que ya no sabe con precisión dónde estaba.

En un intento por recuperar su estado previo, el servidor podría enviar una TPDU de difusión a todos los demás *hosts*, anunciando que acaba de caerse y solicitando a sus clientes que informen sobre el estado de todas las conexiones abiertas. Cada cliente puede estar en uno de dos estados: una TPDU pendiente, *S1*, o ninguna TPDU pendiente, *S0*. Con base en esta información de estado, el cliente debe decidir si retransmitirá o no la TPDU más reciente.

A primera vista parecería obvio: el cliente debe retransmitir sólo si tiene una TPDU pendiente no reconocida (es decir, si está en el estado *S1*) cuando se entera de la caída. Sin embargo, una inspección más cercana revela dificultades con este enfoque ingenuo. Considere, por ejemplo, la situación en la que la entidad de transporte del servidor envía primero un reconocimiento y luego ejecuta la actualización del proceso de aplicación. La escritura de una TPDU en esta corriente de salida y el envío de un reconocimiento son dos eventos indivisibles diferentes que no pueden hacerse simultáneamente. Si ocurre una caída tras el envío del acuse de recibo pero antes de hacer la escritura, el cliente recibirá el acuse de recibo y estará, por tanto, en el estado *S0* al llegar el anuncio de recuperación de la caída. Entonces el cliente no retransmitirá, pensando (incorrectamente) que la TPDU llegó. Esta decisión del cliente conduce a una TPDU faltante.

En este punto el lector podría pensar: "Ese problema se resuelve fácilmente. Todo lo que hay que hacer es reprogramar la entidad de transporte para que primero haga la escritura y luego

envíe el acuse de recibo". Piénselo mejor. Imagine que se ha hecho la escritura pero que la caída ocurre antes de poder enviar el acuse de recibo. El cliente estará en el estado *S1*, y por tanto retransmitirá, conduciendo a una TPDU duplicada no detectada en la corriente de salida al proceso de aplicación del servidor.

Sin importar cómo se programen el transmisor y el receptor, siempre habrá situaciones en las que el protocolo no podrá recuperarse correctamente. El servidor puede programarse de una de dos maneras: mandar acuse de recibo primero o escribir primero. El cliente puede programarse de cuatro maneras: siempre retransmitir la última TPDU, nunca retransmitir la última TPDU, retransmitir sólo en el estado *S0* o retransmitir sólo en el estado *S1*. Esto da ocho combinaciones pero, como veremos, para cada combinación existe algún grupo de eventos que hacen que falle el protocolo.

Son posibles tres eventos en el servidor: el envío de un acuse de recibo (*A*), la escritura al proceso de salida (*W*) y una caída (*C*). Los tres eventos pueden ocurrir en seis órdenes diferentes: *AC(W)*, *AWC*, *C(AW)*, *C(WA)*, *WAC* y *WC(A)*, donde los paréntesis se usan para indicar que ni *A* ni *W* pueden seguir a *C* (es decir, una vez que el servidor se cae, se cayó). En la figura 6-18 se muestran las ocho combinaciones de la estrategia cliente-servidor y las secuencias de eventos válidas para cada una. Nótese que para cada estrategia hay alguna secuencia de eventos que causa la falla del protocolo. Por ejemplo, si el cliente siempre retransmite, el evento *AWC* generará un duplicado no detectado, aun si los otros dos eventos funcionan adecuadamente.

Estrategia usada por el *host* transmisor

Estrategia usada por el *host* transmisor	Primero ACK, luego escritura			Primero escribe, luego ACK		
	AC(W)	AWC	C(AW)	C(WA)	WAC	WC(A)
Siempre retransmitir	BIEN	DUP.	BIEN	BIEN	DUP.	DUP.
Nunca retransmitir	PERD.	BIEN	PERD.	PERD.	BIEN	BIEN
Retransmitir con S0	BIEN	DUP.	PERD.	PERD.	DUP.	BIEN
Retransmitir con S1	PERD.	BIEN	BIEN	BIEN	BIEN	DUP.

BIEN = El protocolo funciona correctamente.
DUP. = El protocolo genera un mensaje duplicado
PERD. = El protocolo pierde un mensaje

Figura 6-18. Diferentes combinaciones de estrategias cliente y servidor.

Hacer más elaborado el protocolo no sirve de nada. Aunque el cliente y el servidor intercambien varias TPDU antes de que el servidor intente escribir, para que el cliente sepa exactamente lo que está a punto de ocurrir, el cliente no tiene manera de saber si ha ocurrido una caída justo antes o justo después de la escritura. La conclusión es inevitable: según nuestra regla básica de que no

debe haber eventos simultáneos, la caída de un *host* y su recuperación no pueden hacerse transparentes a las capas superiores.

En términos más generales, este resultado puede replantearse como que la recuperación de una caída de capa N sólo puede hacerla la capa $N + 1$, y sólo si la capa superior retiene suficiente información de estado. Como se mencionó antes, la capa de transporte puede recuperarse de fallas de la capa de red, siempre y cuando cada extremo de una conexión lleve el registro de dónde está.

Este problema nos lleva al tema de qué significa en realidad un acuse de recibo de extremo a extremo. En principio, el protocolo de transporte es de extremo a extremo, y no está encadenado como las capas inferiores. Ahora considere el caso de un usuario que introduce solicitudes de transacciones con una base de datos remota. Suponga que la entidad de transporte remota está programada para pasar primero las TPDU a la siguiente capa superior y luego emitir los acuses de recibo. Aun en este caso, la recepción de un acuse de recibo en la máquina del usuario no significa necesariamente que el *host* remoto se quedó encendido suficiente tiempo para actualizar la base de datos. Probablemente es imposible lograr un verdadero acuse de recibo de extremo a extremo, cuya recepción significa que el trabajo se ha hecho, y cuya falta significa que no. Este punto lo analizan con mayor detalle Saltzer *et al.* (1984).

6.3. UN PROTOCOLO DE TRANSPORTE SENCILLO

Para hacer más concretas las ideas estudiadas hasta ahora, en esta sección examinaremos detalladamente una capa de transporte de ejemplo. El ejemplo se ha seleccionado con cuidado para que sea razonablemente realista, pero lo bastante sencillo para que sea fácil de entender. Las primitivas de servicio abstractas que usaremos son las primitivas orientadas a conexiones de la figura 6-3.

6.3.1. Las primitivas de servicio de ejemplo

Nuestro primer problema es cómo expresar de manera concreta estas primitivas de transporte. CONNECT (conexión) es fácil: sencillamente tenemos un procedimiento de biblioteca *connect* que puede llamarse con los parámetros adecuados necesarios para establecer una conexión. Los parámetros son los TSAP locales y remotos. Durante la llamada, el indicador se bloquea (es decir, se suspende) mientras la entidad de transporte intenta establecer la conexión. Si la conexión tiene éxito, se desbloquea el solicitante y puede comenzar la transmisión de datos.

Cuando un proceso quiere poder aceptar llamadas entrantes, llama a *listen* (escuchar), especificando un TSAP en particular en el que quiere escuchar. El proceso entonces se bloquea hasta que algún proceso remoto intenta establecer una conexión con su TSAP.

Nótese que este modelo es altamente asimétrico. Un lado es pasivo, ejecutando un *listen* y esperando hasta que ocurre algo. El otro lado es activo e inicia la conexión. Surge una pregunta interesante sobre lo que debe hacerse si el lado activo comienza primero. Una estrategia es hacer que falle el intento de conexión si no hay un escucha en el TSAP remoto. Otra estrategia

es hacer que el iniciador se bloquee (posiblemente para siempre) hasta que aparezca un escucha.

Un término medio, usado en nuestro ejemplo, es mantener la solicitud de conexión del lado receptor durante cierto intervalo de tiempo. Si un proceso de ese *host* llama a *listen* antes de que termine la temporización, se establece la conexión; de otro modo se rechaza y se desbloquea al solicitante, devolviéndole un error.

Para liberar una conexión, usaremos un procedimiento *disconnect* (desconectar). Cuando ambos lados se han desconectado, se libera la conexión. En otras palabras, estamos usando un modelo de desconexión simétrico.

La transmisión de datos tiene precisamente el mismo problema que el establecimiento de una conexión: el envío es activo, pero la recepción es pasiva. Usaremos la misma solución para la transmisión de datos que para el establecimiento de la conexión, una llamada activa *send* (envío) que transmite datos, y una llamada pasiva de *receive* (recibir) que de bloquea hasta que llega una TPDU.

Nuestra definición concreta de servicio consiste por tanto en cinco primitivas: CONNECT, LISTEN, DISCONNECT, SEND y RECEIVE. Cada primitiva corresponde exactamente a un procedimiento de biblioteca que ejecuta la primitiva. Los parámetros de las primitivas de servicio y los procedimientos de biblioteca son los siguientes:

```
connum = LISTEN(local)
connum = CONNECT(local, remote)
status = SEND(connum, buffer, bytes)
status = RECEIVE(connum, buffer, bytes)
status = DISCONNECT(connum)
```

La primitiva LISTEN anuncia la disposición del que llama a aceptar solicitudes de conexión dirigidas a los TSAP indicados. El usuario de la primitiva se bloquea hasta que se hace un intento de conexión a él. No hay expiración de la temporización.

La primitiva CONNECT toma dos parámetros, un TSAP local (es decir, dirección de transporte), *local*, y un TSAP remoto, *remote*, e intenta establecer una conexión de transporte entre los dos; si tiene éxito, devuelve en *connum* un número no negativo que sirve para identificar la conexión en llamadas subsiguientes; si falla, la razón del fracaso se pone en *connum* como número negativo. En nuestro modelo sencillo, cada TSAP puede participar sólo en una conexión de transporte, por lo que una razón posible de falla es que una de las direcciones de transporte ya esté en uso. Algunas otras razones son: *host* remoto caído, dirección local ilegal y dirección remota ilegal.

La primitiva SEND transmite el contenido del *buffer* como mensaje por la conexión de transporte indicada, posiblemente en varias unidades si es demasiado grande. Los errores posibles, devueltos en *status*, son falta de conexión, dirección de *buffer* ilegal y cuenta negativa.

La primitiva RECEIVE indica el deseo del solicitante de aceptar datos. El tamaño del mensaje de entrada se pone en *bytes*. Si el proceso remoto ha liberado la conexión o la dirección de *buffer* es ilegal (por ejemplo, fuera del programa de usuario), se asigna a *status* a un código de error que indica la naturaleza del problema.

La primitiva DISCONNECT termina una conexión de transporte; el parámetro *connum* indica cuál. Los errores posibles son que *connum* pertenezca a otro proceso, o que *connum* no sea un identificador de conexión válido. El código de error (o 0 si todo funcionó bien) se devuelve en *status*.

6.3.2. La entidad de transporte de ejemplo

Antes de ver el código de la entidad de transporte de ejemplo, tenga bien presente que este ejemplo es análogo a los primeros ejemplos presentados en el capítulo 3: es para propósitos pedagógicos, no una propuesta seria. En aras de la sencillez, se han omitido aquí muchos de los detalles técnicos (como comprobación extensa de errores) necesarios para un sistema de producción.

La capa de transporte hace uso de las primitivas de servicio de red para enviar y recibir las TPDU. Para este ejemplo, necesitamos escoger las primitivas de servicio de red a usar. Una selección podría haber sido el servicio de datagramas no confiable. No lo hemos seleccionado para mantener sencillo el ejemplo. Con el servicio de datagramas no confiable, el código de transporte habría sido grande y complejo, encargándose principalmente de paquetes perdidos y retrasados. Es más, muchas de estas ideas ya se analizaron con detalle en el capítulo 3.

En cambio, hemos optado por usar un servicio de red confiable orientado a conexiones. De esta manera, podremos enfocarnos a los asuntos de transporte que no ocurren en las capas inferiores. Éstos incluyen el establecimiento de conexiones, la liberación de conexiones y el manejo de crédito, entre otros. Un servicio sencillo de transporte construido encima de una red ATM podría verse así.

En general, la entidad de transporte puede ser parte del sistema operativo del *host* o puede ser un paquete de rutinas de biblioteca operando en el espacio de direcciones del usuario; también puede estar contenida en un *chip* coprocesador o en una tarjeta de red conectada al *backplane* del *host*. Por sencillez, nuestro ejemplo se ha programado como si fuera un paquete de biblioteca, pero los cambios necesarios para hacerlo parte del sistema operativo son mínimos (principalmente la manera de acceder a los *buffers*).

Vale la pena señalar, sin embargo, que en este ejemplo la "entidad de transporte" no es en realidad una entidad independiente, sino parte del proceso de usuario. En particular, cuando el usuario ejecuta una primitiva que se bloquea, como LISTEN, toda la entidad de transporte se bloquea también. Aunque este diseño es adecuado para un *host* con un solo proceso de usuario, en un *host* con varios usuarios sería más natural hacer que la entidad de transporte sea un proceso aparte, diferente de todos los procesos de usuario.

La interfaz con la capa de red se establece mediante los procedimientos *to_net* y *from_net* (no se muestran). Cada uno tiene seis parámetros. Primero viene la identificación de la conexión, que se relaciona uno a uno con los circuitos virtuales de la red. A continuación vienen los bits *Q* y *M* que, al estar establecidos en 1, indican "mensaje de control" y "continúan más datos de este mensaje en el siguiente paquete", respectivamente. Tras esto tenemos el tipo de paquete, seleccionado entre los seis tipos de paquete listados en la figura 6-19. Por último, tenemos un apuntador a los datos mismos y un entero que indica el número de bytes de datos.

Paquete de red	Significado
CALL REQUEST (solicitud de llamada)	Se envía para establecer una conexión
CALL ACCEPTED (llamada aceptada)	Respuesta a CALL REQUEST
CLEAR REQUEST (solicitud de desconexión)	Se envía para liberar una conexión
CLEAR CONFIRMATION (confirmación de desconexión)	Respuesta a CLEAR REQUEST
DATA (datos)	Sirve para transportar datos
CREDIT (crédito)	Paquete de control para manejar la ventana

Figura 6-19. Paquetes de capa de red usados en nuestro ejemplo.

En las llamadas a *to_net*, la entidad de transporte llena todos los parámetros para que los lea la capa de red; en las llamadas de *from_net*, la capa de red desmembra un paquete de entrada antes de pasarlo a la entidad de transporte. Al pasar información como parámetros de procedimiento en lugar de pasar el paquete de salida o entrada mismo, la capa de transporte queda protegida de los detalles del protocolo de capa de red. Si la entidad de transporte intentara enviar un paquete estando llena la ventana corrediza del circuito virtual subyacente, se suspende en *to_net* hasta haber espacio en la ventana. Este mecanismo es transparente para la entidad de transporte y lo controla la capa de red usando comandos como *enable_transport_layer* y *disable_transport_layer* análogos a los descritos en los protocolos del capítulo 3. La gestión de la ventana de la capa de paquetes también la realiza la capa de red.

Además de este mecanismo de suspensión transparente, hay procedimientos *sleep* y *wakeup* (dormir y despertar), que no se muestran, invocados por la entidad de transporte. El procedimiento *sleep* se invoca cuando la entidad de transporte está bloqueada lógicamente en espera de un evento externo, generalmente la llegada de un paquete. Tras haber llamado a *sleep*, la entidad de transporte (y por supuesto el proceso de usuario) deja de ejecutarse.

El código de la entidad de transporte se muestra en la figura 6-20. Cada conexión está siempre en uno de siete estados, a saber:

1. IDLE (inactivo) — No se ha establecido todavía una conexión.
2. WAITING (en espera) — Se ha ejecutado CONNECT y enviado CALL REQUEST.
3. QUEUED (en cola) — Ha llegado una CALL REQUEST; aún no hay LISTEN.
4. ESTABLISHED (establecida) — Se ha establecido la conexión.
5. SENDING (enviando) — El usuario está esperando permiso de enviar un paquete.
6. RECEIVING (recibiendo) — Se ha hecho un RECEIVE.
7. DISCONNECTING (desconexión) — Se ha hecho localmente un DISCONNECT.

Las transiciones entre estados pueden ocurrir al suceder cualquiera de los siguientes eventos: se ejecuta una primitiva, llega un paquete o termina el temporizador.

```
#define MAX_CONN 32                    /* cantidad máxima de conexiones simultáneas */
#define MAX_MSG_SIZE 8192              /* tamaño máximo de mensaje, en bytes */
#define MAX_PKT_SIZE 512               /* paquete más grande en bytes */
#define TIMEOUT 20
#define CRED 1
#define OK 0

#define ERR_FULL -1
#define ERR_REJECT -2
#define ERR_CLOSED -3
#define LOW_ERR -3

typedef int transport_address;
typedef enum {CALL_REQ,CALL_ACC,CLEAR_REQ,CLEAR_CONF,DATA_PKT,CREDIT} pkt_type;
typedef enum {IDLE,WAITING,QUEUED,ESTABLISHED,SENDING,RECEIVING,DISCONN} cstate;

/* Variables globales. */
transport_address listen_address;      /* dirección local en la que se está escuchando */
int listen_conn;                       /* identificación de conexión para escucha */
unsigned char data[MAX_PKT_SIZE];      /* área de trabajo para paquetes de datos */

struct conn {
  transport_address local_address, remote_address;
  cstate state;                        /* estado de esta conexión */
  unsigned char *user_buf_addr;        /* apuntador al buffer de recepción */
  int byte_count;                      /* conteo de envío/recepción */
  int clr_req_received;                /* establecido al recibir paquete CLEAR_REQ */
  int timer;                           /* sirve para terminar temporización de paquetes CALL_REQ */
  int credits;                         /* cantidad de mensajes que se puede enviar */
} conn[MAX_CONN];

void sleep(void);                      /* prototipos */
void wakeup(void);
void to_net(int cid, int q, int m, pkt_type pt, usigned char *p, int bytes);
void from_net(int *cid, int *q, int *m, pkt_type *pt, unsigned char *p, int *bytes);

int listen(transport_address t)
{ /* El usuario quiere escuchar si hay una conexión. Ve si CALL_REQ ya llegó. */
  int i = 1, found = 0;

  for (i = 1; i <= MAX_CONN; i++)      /* busca en la tabla CALL_REQ */
     if (conn[i].state == QUEUED && conn[i].local_address == t) {
          found = i;
          break;
     }

  if (found == 0) {
     /* No hay CALL_REQ esperando. Se duerme hasta que llega o termina la temporización. */
     listen_address = t; sleep(); i = listen_conn ;
  }
  conn[i].state = ESTABLISHED;         /* se establece la conexión */
  conn[i].timer = 0;                   /* no se usa temporizador */
  listen_conn = 0;                     /* se supone que 0 es una dirección no válida */
  to_net(i, 0, 0, CALL_ACC, data, 0);  /* indica a la red que acepte una conexión */
  return(i);                           /* devuelve identificador de conexión */
}

int connect(transport_address l, transport_address r)
{ /* El usuario quiere conectarse a un proceso remoto; envía paquete CALL_REQ. */
  int i;
  struct conn *cptr;

  data[0] = r;  data[1] = l;           /* el paquete CALL_REQ necesita éstos */
  i = MAX_CONN;                        /* busca hacia atrás en la tabla */
  while (conn[i].state != IDLE && i > 1) i = i - 1;
  if (conn[i].state == IDLE) {
     /* Crea una entrada en la tabla indicando que se ha enviado CALL_REQ. */
     cptr = &conn[i];
     cptr->local_address = l; cptr->remote_address = r;
     cptr->state = WAITING; cptr->clr_req_received = 0;
     cptr->credits = 0; cptr->timer = 0;
     to_net(i, 0, 0, CALL_REQ, data, 2);
     sleep();                          /* espera CALL_ACC o CLEAR_REQ */
     if (cptr->state == ESTABLISHED) return(i);
     if (cptr->clr_req_received) {
          /* El otro lado rechazó la llamada. */
          cptr->state = IDLE;          /* Regresa al estado de reposo */
          to_net(i, 0, 0, CLEAR_CONF, data, 0);
          return(ERR_REJECT);
     }
  } else return(ERR_FULL);             /* rechaza conexión: no hay espacio de tabla */
}

int send(int cid, unsigned char bufptr[], int bytes)
{ /* El usuario quiere enviar un mensaje. */
  int i, count, m;
  struct conn *cptr = &conn[cid];

  /* Entra en el estado de transmisión. */
  cptr->state = SENDING;
  cptr->byte_count = 0;                /* # bytes enviados hasta ahora en este mensaje */
  if (cptr->clr_req_received == 0 && cptr->credits == 0) sleep();
  if (cptr->clr_req_received == 0) {
     /* Crédito disponible; divide mensaje en paquetes de ser necesario. */
     do {
          if (bytes - cptr->byte_count > MAX_PKT_SIZE) {/* mensaje multipaquete */
               count = MAX_PKT_SIZE; m = 1;  /* más paquetes después */
          } else {                          /* mensaje de un solo paquete */
               count = bytes - cptr->byte_count; m = 0;  /* último paquete de este mensaje */
          }
          for (i = 0; i < count; i++) data [i] = bufptr[cptr->byte_count + i];
          to_net(cid, 0, m, DATA_PKT, data, count);  /* envía un paquete */
          cptr->byte_count = cptr->byte_count + count;  /* incrementa bytes enviados hasta ahora */
     } while (cptr->byte_count < bytes);     /* repite ciclo hasta enviar todo el mensaje */
```

```
        cptr->credits--;                    /* cada mensaje consume un crédito */
        cptr->state = ESTABLISHED;
        return(OK);
    } else {
        cptr->state = ESTABLISHED;
        return(ERR_CLOSED);                 /* falló transmisión: el igual quiere desconectarse */
    }
}

int receive(int cid, unsigned char bufptr[], int *bytes)
{ /* Usuario preparado para recibir un mensaje. */
  struct conn *cptr = &conn[cid];

  if (cptr->clr_req_received == 0) {
      /* Conexión aún establecida; trata de recibir. */
      cptr->state = RECEIVING;
      cptr->user_buf_addr = bufptr;
      cptr->byte_count = 0;
      data[0] = CRED;
      data[1] = 1;
      to_net(cid, 1, 0, CREDIT, data 2);    /* envía crédito */
      sleep( );                             /* se bloquea esperando datos */
      *bytes = cptr->byte_count;
  }
  cptr->state = ESTABLISHED;
  return(cptr->clr_req_received ? ERR_CLOSED : OK);
}

int disconnect(int cid)
{ /* El usuario quiere liberar una conexión. */
  struct conn  *cptr = &conn[cid];

  if (cptr->clr_req_received) {             /* el otro lado inició la terminación */
      cptr->state = IDLE;                   /* ahora se libera la conexión */
      to_net(cid, 0, 0, CLEAR_CONF, data, 0);
  } else {                                  /* iniciamos la terminación */
      cptr-> state = DISCONN;               /* no liberada hasta que el otro lado esté de acuerdo */
      to_net(cid, 0, 0, CLEAR_REQ, data, 0);
  }
  return(OK);
}

void packet_arrival(void)
{ /* Ha llegado un paquete, obtenerlo y procesarlo. */
  int cid;                                  /* conexión por la que llegó el paquete */
  int count, i, q, m;
  pkt_type ptype;   /* CALL_REQ, CALL_ACC, CLEAR_REQ, CLEAR_CONF, DATA_PKT, CREDIT */
  unsigned char data[MAX_PKT_SIZE];         /* parte de datos del paquete de entrada */
  struct conn *cptr;

  from_net(&cid, &q, &m, &ptype, data, &count);   /* obténlo */
  cptr = &conn[cid];
```



```
  switch (ptype) {
    case CALL_REQ:                          /* usuario remoto quiere establecer una conexión */
        cptr->local_address = data[0];  cptr->remote_address = data[1];
        if (cptr->local_address == listen_address) {
            listen_conn = cid;  cptr->state = ESTABLISHED;  wakeup();
        } else {
            cptr->state = QUEUED;  cptr->timer = TIMEOUT;
        }
        cptr->clr_req_received = 0;  cptr->credits = 0;

    case CALL_ACC:                          /* usuario remoto ha aceptado nuestra CALL_REQ */
        cptr->state = ESTABLISHED;
        wakeup();

    case CLEAR_REQ:                         /* el usuario remoto quiere desconectarse o rechazar llamada */
        cptr->clr_req_received = 1;
        if (cptr->state == DISCONN) cptr->state = IDLE;   /* restaura colisión */
        if (cptr->state == WAITING || cptr->state == RECEIVING || cptr->state == SENDING) wakeup();

    case CLEAR_CONF:                        /* usuario remoto está de acuerdo en desconectar */
        cptr->state = IDLE;

    case CREDIT:                            /* usuario remoto espera datos */
        cptr->credits += data[1];
        if (cptr->state == SENDING) wakeup();

    case DATA_PKT:                          /* usuario remoto envió datos */
        for (i = 0; i < count; i++) cptr->user_buf_addr[cptr->byte_count + i] = data[i];
        cptr->byte_count += count;
        if (m == 0) wakeup();
  }
}

void clock(void)
{ /* Pulsó el reloj, ver si terminó la temporización de solicitudes de conexión en cola. */
  int i;
  struct conn *cptr;

  for (i = 1; i <= MAX_CONN; i++) {
      cptr = &conn[i];
      if (cptr->timer > 0) {                /* temporizador estaba operando */
          cptr->timer--;
          if (cptr->timer == 0) {           /* temporizador expiró */
              cptr->state = IDLE;
              to_net(i, 0, 0, CLEAR_REQ, data 0);
          }
      }
  }
}
```

Figura 6-20. Entidad de transporte de ejemplo.

Los procedimientos mostrados en la figura 6-20 son de dos tipos. Casi todos pueden invocarse directamente desde los programas de usuario. Sin embargo, *packet_arrival* y *clock* son diferentes; se disparan espontáneamente por eventos externos: la llegada de un paquete y los pulsos del reloj, respectivamente. De hecho, son rutinas de interrupción. Supondremos que no se invocan mientras está ejecutándose un procedimiento de entidad de transporte; sólo pueden invocarse cuando el proceso de usuario está dormido o ejecutándose fuera de la entidad de transporte. Esta propiedad es crucial para el funcionamiento correcto de la entidad de transporte.

La existencia del bit Q (calificador) en la cabecera del paquete nos permite evitar la carga extra de una cabecera de protocolo de transporte. Los mensajes de datos ordinarios se envían como paquetes de datos con $Q = 0$. Los mensajes de control de protocolo de transporte, de los cuales sólo hay uno (CREDIT) en nuestro ejemplo, se envían como paquetes de datos con $Q = 1$. Estos mensajes de control son detectados y procesados por la entidad de transporte receptora.

La principal estructura de datos usada por la entidad de transporte es el arreglo *conn*, que tiene un registro por cada conexión potencial. El registro mantiene el estado de la conexión, incluidas las direcciones de transporte en ambos extremos, la cantidad de mensajes enviados y recibidos por la conexión, el estado actual, el apuntador al *buffer* de usuario, la cantidad de bytes de los mensajes actuales enviados o recibidos hasta el momento, un bit que indica si el usuario remoto ha emitido o no un DISCONNECT, un temporizador y un contador de permisos que sirve para habilitar el envío de mensajes. No todos estos campos se usan en nuestro ejemplo sencillo, pero una entidad de transporte completa los necesitaría todos, y tal vez más. Se supone que cada entrada de *conn* está inicialmente en el estado inactivo (*IDLE*).

Cuando el usuario llama a CONNECT, la capa de red recibe instrucciones para enviar un paquete CALL REQUEST (solicitud de llamada) a la máquina remota, y el usuario se pone a dormir. Cuando llega el paquete CALL REQUEST al otro lado, la entidad de transporte se interrumpe para ejecutar *packet_arrival* a fin de comprobar que el usuario local está escuchando en la dirección especificada. De ser así, se devuelve un paquete CALL ACCEPTED (llamada aceptada) y se despierta al usuario remoto; de no ser así, se encola la CALL REQUEST durante *TIMEOUT* pulsos de reloj. Si se hace un LISTEN en este periodo, se establece la conexión; de otro modo, termina la temporización y se rechaza la solicitud con un paquete CLEAR REQUEST (borrar solicitud). Este mecanismo es necesario para evitar que el iniciador se bloquee eternamente en el caso de que un proceso remoto no quiera conectarse con él.

Aunque hemos eliminado la cabecera del protocolo de transporte, aún necesitamos una manera de llevar el control de cuál paquete pertenece a cuál conexión de transporte, ya que pueden existir varias conexiones al mismo tiempo. El enfoque más sencillo es usar el número de circuito virtual de la capa de red también como número de conexión de transporte. Es más, puede usarse también el número de circuito virtual como índice del arreglo *conn*. Al llegar un paquete por el circuito virtual k de la capa de red, pertenecerá a la conexión de transporte k, cuyo estado está en el registro *conn*[k]. En las conexiones iniciadas en un *host*, la entidad de transporte originadora selecciona el número de conexión. En las llamadas entrantes, la capa de red hace la selección, escogiendo cualquier número de circuito virtual no usado.

Para evitar tener que proporcionar y administrar *buffers* en la entidad de transporte, se usa aquí un mecanismo de control de flujo diferente de la ventana corrediza tradicional. Cuando un

usuario llama a RECEIVE, se envía un **mensaje de crédito** especial a la entidad de transporte de la máquina transmisora y se registra en el arreglo *conn*. Al llamar a SEND, la entidad de transporte revisa si ha llegado un crédito por la conexión especificada. De ser así, el mensaje se envía (en paquetes múltiples de ser necesario) y se disminuye el crédito; en caso contrario, la entidad de transporte se pone a dormir hasta la llegada de un crédito. Este mecanismo garantiza que no se enviará nunca un mensaje a menos que el otro lado ya haya hecho un RECEIVE. Como resultado, siempre que llega un mensaje hay la garantía de un *buffer* disponible en el cual puede ponerse. El esquema puede generalizarse fácilmente para permitir que los receptores proporcionen múltiples *buffers* y soliciten múltiples mensajes.

Debe tenerse presente la sencillez de la figura 6-20. Una entidad de transporte realista normalmente comprobaría la validez de todos los parámetros proporcionados por el usuario, manejaría la recuperación de caídas de la capa de red, se encargaría de colisiones de llamadas, y apoyaría un servicio de red más general, incluidas funciones como interrupciones, datagramas y versiones no bloqueadoras de las primitivas SEND y RECEIVE.

6.3.3. El ejemplo como máquina de estados finitos

Escribir una entidad de transporte es un trabajo difícil y exigente, especialmente para los protocolos más realistas. A fin de reducir la posibilidad de cometer errores, podemos representar el estado del protocolo como máquina de estados finitos.

Ya hemos visto que nuestro protocolo de ejemplo tiene siete estados por conexión. También es posible aislar 12 eventos que pueden ocurrir para pasar una conexión de un estado a otro. Cinco de estos eventos son las cinco primitivas de servicio. Otros seis son las llegadas de los seis tipos de paquete legales. El último es la expiración del temporizador. En la figura 6-21 se muestran las acciones principales del protocolo en forma de matriz. Las columnas son los estados y las filas son los 12 eventos.

Cada entrada de la matriz (es decir, la máquina de estados finitos) de la figura 6-21 tiene hasta tres campos: un predicado, una acción y un estado nuevo. El predicado indica en qué condiciones se emprende la acción. Por ejemplo, en la entrada de la esquina superior izquierda, si se ejecuta una LISTEN y no hay más espacio de tabla (predicado *P1*) falla la LISTEN y no cambia el estado. Por otra parte, si ya ha llegado un paquete CALL REQUEST para la dirección de transporte en la que se está escuchando (predicado *P2*), se establece de inmediato la conexión. Otra posibilidad es que *P2* sea falso, es decir, que no ha entrado ningún CALL REQUEST, en cuyo caso la conexión permanece en el estado *IDLE*, esperando un paquete CALL REQUEST.

Es importante indicar que la selección de estados a usar en la matriz no es fijada completamente por el protocolo mismo. En este ejemplo, no hay ningún estado *LISTENING* (escuchando), lo que sería algo razonable después de una LISTEN. No hay estado *LISTENING* porque un estado se asocia a un registro de conexión, y no se crea un registro de conexión mediante LISTEN. ¿Por qué no? Porque hemos decidido usar los números de circuito virtual de la capa de red como identificadores de la conexión, y para una LISTEN el número de circuito virtual es escogido por la capa de red al llegar el paquete CALL REQUEST.

		Estado					
		En reposo	Espera	Encolado	Establecido	Transmitiendo	Recibiendo	Desconexión
Primitivas	LISTEN	P1: -/Reposo P2: A1/Estab P2: A2/Reposo		-/Estab				
	CONNECT	P1: -/Reposo P1: A3/Espera						
	DISCONNECT				P4: A5/Reposo P4: A6/Desc			
	SEND				P5: A7/Estab P5: A8/Envío			
	RECEIVE				A9/Recibe			
Paquetes de entrada	Call_req	P3: A1/Estab P3: A4/Encolado						
	Call_acc		-/Estab					
	Clear_req		-/Reposo		A10/Estab	A10/Estab	A10/Estab	-/Reposo
	Clear_conf							-/Reposo
	DataPkt						A12/Estab	
	Credit				A11/Estab	A7/Estab		
Reloj	Timeout			-/Reposo				

Predicados	Acciones	
P1: Tabla de conexiones llena	A1: Envía Call_acc	A7: Envía mensaje
P2: Call_req pendiente	A2: Espera Call_req	A8: Espera crédito
P3: LISTEN pendiente	A3: Envía Call_req	A9: Envía crédito
P4: Clear_req pendiente	A4: Inicia temporizador	A10: Establece indicador Clr_req_received
P5: Crédito disponible	A5: Envía Clear_conf	A11: Registra crédito
	A6: Envía Clear_req	A12: Acepta mensaje

Figura 6-21. El protocolo de ejemplo como máquina de estados finitos. Cada entrada tiene un predicado opcional, una acción opcional y el estado nuevo. La tilde indica que no se realizó ninguna acción importante. Una barra sobre un predicado indica la negación del predicado. Las entradas en blanco corresponden a eventos imposibles o no válidos.

Las acciones *A1* a *A12* son las acciones principales, como el envío de paquetes y el inicio de temporizadores. No se listan todas las acciones menores, como la preparación inicial de los campos de un registro de conexión. Si una acción comprende despertar un proceso dormido, las

acciones que siguen a su despertar también cuentan. Por ejemplo, si entra un paquete CALL REQUEST y un proceso estaba dormido esperándolo, la transmisión del paquete CALL ACCEPT que sigue al despertar cuenta como parte de la acción de CALL REQUEST. Tras la ejecución de cada acción, la conexión puede pasar a un estado nuevo, como se muestra en la figura 6-21.

La ventaja de representar el protocolo como una matriz es triple. Primero, de esta manera es mucho más fácil que el programador revise sistemáticamente cada combinación de estado y evento para ver si se requiere una acción. En las implementaciones de producción se usarían algunas de las combinaciones para manejo de errores. En la figura 6-21 no se hace distinción entre situaciones imposibles e ilegales. Por ejemplo, si una conexión está en el estado *waiting*, el evento DISCONNECT es imposible porque el usuario está bloqueado y no puede ejecutar primitivas. Por otra parte, en el estado *sending*, no se esperan paquetes de datos porque no se ha emitido crédito. La llegada de un paquete de datos es un error del protocolo.

La segunda ventaja de la representación en matriz del protocolo es en su implementación. Podemos imaginar un arreglo de dos dimensiones en el que el elemento $a[i][j]$ es un apuntador o índice al procedimiento que maneja el suceso del evento i cuando se está en el estado j. Una posible puesta en práctica es escribir la entidad de transporte como ciclo corto, esperando un evento en la parte superior del ciclo. Al ocurrir un evento, se localiza la conexión pertinente y se extrae su estado. Sabiendo ahora el evento y el estado, la entidad de transporte simplemente los indiza en el arreglo a e invoca el procedimiento adecuado. Este enfoque produce un diseño mucho más regular y sistemático que nuestra entidad de transporte.

La tercera ventaja del enfoque de máquina de estados finitos se aprecia en la descripción del protocolo. En algunos documentos de estándares, se dan los protocolos como máquinas de estados finitos del tipo de la figura 6-21. Pasar de este tipo de descripción a una entidad de transporte operante es mucho más fácil si la entidad de transporte también es operada por una máquina de estados finitos basada en la del estándar.

La desventaja principal del enfoque de máquina de estados finitos es que puede ser más difícil de entender que el ejemplo de programación directa que usamos inicialmente. Sin embargo, puede resolverse parcialmente este problema dibujando la máquina de estados finitos en forma de grafo, como se hace en la figura 6-22.

6.4. LOS PROTOCOLOS DE TRANSPORTE DE INTERNET (TCP Y UDP)

La Internet tiene dos protocolos principales en la capa de transporte, un protocolo orientado a conexiones y uno sin conexiones. En las siguientes secciones estudiaremos ambos. El protocolo orientado a conexiones es el TCP; el protocolo sin conexiones es el UDP. Dado que el UDP es básicamente el IP con la adición de una cabecera corta, nos enfocaremos al TCP.

El TCP (*Transmission Control Protocol*, **protocolo de control de transmisión**) se diseñó específicamente para proporcionar una corriente de bytes confiable a través de una interred no confiable. Una interred es diferente de una sola red porque las distintas partes pueden tener

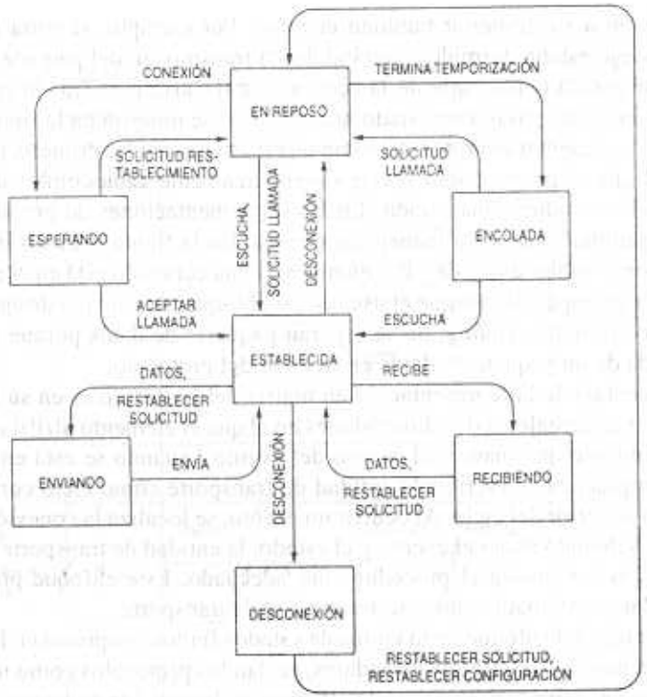


Figura 6-22. El protocolo de ejemplo en forma gráfica. Por sencillez, se han omitido las transiciones que dejan sin cambio el estado de las conexiones.

topologías, anchos de banda, retardos, tamaños de paquete y otros parámetros con grandes diferencias. Se diseñó el TCP para adaptarse dinámicamente a las propiedades de la interred y para ser robusto ante muchos tipos de fallas.

El TCP se definió formalmente en el RFC 793. A medida que pasó el tiempo, se detectaron varios errores e inconsistencias, y se cambiaron los requisitos de algunas áreas. Estas clarificaciones y algunas correcciones de fallas se detallan en el RFC 1122. En el RFC 1323 se presentan las extensiones.

Cada máquina que reconoce el TCP tiene una entidad de transporte TCP, ya sea un proceso de usuario o una parte del núcleo que maneja las corrientes TCP y tiene interfaz con la capa IP. Una entidad TCP acepta corrientes de datos de usuario de los procesos locales, los divide en partes que no exceden 64K bytes (en la práctica, generalmente de unos 1500 bytes), y envía cada parte como datagrama IP independiente. Cuando llegan a una máquina datagramas IP que contienen datos TCP, son entregados a la entidad TCP, que reconstruye las corrientes originales de bytes. Por sencillez, a veces sólo usaremos "TCP" para indicar la entidad de transporte TCP (una parte del *software*) o el protocolo TCP (un grupo de reglas). Por el contexto quedará claro a

qué nos estamos refiriendo. Por ejemplo, en "el usuario proporciona al TCP los datos", claramente nos estamos refiriendo a la entidad de transporte TCP.

La capa IP no ofrece ninguna garantía de que los datagramas se entregarán adecuadamente, por lo que es responsabilidad del TCP terminar de temporizar y retransmitirlos según se necesite. Los datagramas que sí llegan pueden hacerlo desordenadamente; también es responsabilidad del TCP reensamblarlos en mensajes con la secuencia adecuada. En pocas palabras, el TCP debe proveer la confiabilidad que la mayoría de los usuarios quiere y que el IP no proporciona.

6.4.1. Modelo de servicio TCP

El servicio TCP se obtiene haciendo que tanto el transmisor como el receptor creen puntos terminales, llamados sockets, como se explicó en la sección 6.1.3. Cada socket tiene un número (dirección) de socket que consiste en la dirección IP del *host* y en un número de 16 bits local a ese *host*, llamado **puerto**. Puerto es el nombre en TCP de un TSAP. Para obtener el servicio TCP, debe establecerse explícitamente una conexión entre un socket de la máquina transmisora y un socket de la máquina receptora. Las llamadas de socket se listan en la figura 6-6.

Puede usarse un socket para varias conexiones al mismo tiempo. En otras palabras, dos o más conexiones pueden terminar en el mismo socket. Las conexiones se identifican mediante los identificadores de ambas terminales, es decir (*socket1, socket2*). No se usan números de circuito virtual ni ningún otro identificador.

Los números de puerto por debajo del 256 se llaman **puertos bien conocidos** (*well-known ports*) y se reservan para servicios estándar. Por ejemplo, cualquier proceso que desee establecer una conexión a un *host* para transferir un archivo usando FTP puede conectarse al puerto 21 del *host* de destino para comunicarse con su *daemon* (proceso reentrante) de FTP. De la misma manera, para establecer una sesión interactiva remota usando TELNET, se usa el puerto 23. La lista de puertos bien conocidos se da en el RFC 1700.

Todas las conexiones TCP son dúplex integral y punto a punto. Dúplex integral significa que el tráfico puede ir en ambos sentidos al mismo tiempo. Punto a punto significa que cada conexión tiene exactamente dos puntos terminales. El TCP no reconoce la multitransmisión ni la difusión.

Una conexión TCP es una corriente de bytes, no una corriente de mensajes. Los límites de mensaje no se conservan de extremo a extremo. Por ejemplo, si el proceso transmisor hace cuatro escrituras de 512 bytes en una corriente TCP, estos datos pueden entregarse al proceso receptor como cuatro bloques de 512 bytes, dos bloques de 1024 bytes, un bloque de 2048 bytes (véase la figura 6-23), o de algún otro modo. No hay manera de que el receptor detecte las unidades en las que se escribieron los datos.

Los archivos de UNIX tienen también esta propiedad. El lector de un archivo no puede saber si el archivo se escribió bloque por bloque, byte por byte, o todo de golpe. Como sucede con los archivos UNIX, el *software* del TCP no tiene idea de lo que significan los bytes y no tiene interés en averiguarlo. Un byte simplemente es un byte.

Cuando una aplicación pasa datos al TCP, el TCP puede enviarlos de inmediato o guardarlos en un *buffer* (a fin de reunir una cantidad mayor de información para enviarla junta), a discreción

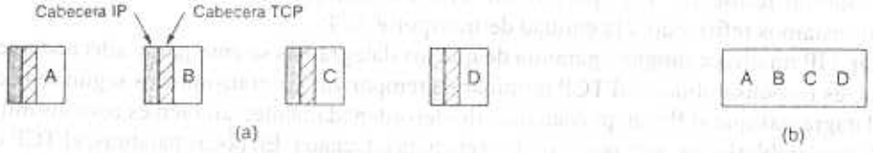


Figura 6-23. (a) Cuatro segmentos de 512 bytes enviados como datagramas IP independientes. (b) Los 2048 bytes de datos entregados a la aplicación con una sola llamada READ.

propia. Sin embargo, a veces la aplicación simplemente quiere que los datos se envíen de inmediato. Por ejemplo, supóngase que un usuario está en sesión con una máquina remota. Tras la terminación de una línea de comandos y el retorno de carro, es esencial que la línea se envíe a la máquina remota de inmediato y no se guarde en *buffer* hasta la entrada de la siguiente línea. Para obligar la salida de datos, las aplicaciones pueden usar la bandera (*flag*) PUSH que ordena al TCP no retrasar la transmisión.

Algunas de las primeras aplicaciones usaron la bandera PUSH como una especie de marcador para delinear los límites de los mensajes. Aunque este truco a veces funciona, en ocasiones falla porque no todas las implementaciones de TCP pasan la bandera PUSH a la aplicación del lado receptor. Es más, si llegan otros PUSH adicionales antes de la transmisión del primero (por ejemplo, por estar ocupada la línea de salida), el TCP está en libertad de juntar todos los datos que entran por PUSH en un solo datagrama IP, sin separación entre las diferentes partes.

Una última característica del servicio TCP que vale la pena mencionar son los **datos urgentes**. Cuando un usuario interactivo pulsa la tecla DEL o CTRL-C para interrupir un cómputo remoto ya iniciado, la aplicación transmisora pone cierta información de control en la corriente de datos y se la da al TCP junto con la bandera URGENT. Este evento hace que el TCP deje de acumular datos y transmita de inmediato todo lo que tiene para esa conexión.

Al recibirse los datos urgentes en el destino, se interrumpe la aplicación receptora (por ejemplo, recibe una señal en términos de UNIX), para que la aplicación pueda detener lo que estaba haciendo y leer la corriente de datos hasta encontrar los datos urgentes. Se marca el fin de los datos urgentes, por lo que la aplicación sabe cuándo han terminado. El comienzo de los datos urgentes no se marca; es responsabilidad de la aplicación averiguarlo. Este esquema básicamente proporciona un mecanismo burdo de señalamiento, y deja todo lo demás en manos de la aplicación.

6.4.2. El protocolo TCP

En esta sección daremos un repaso general del protocolo TCP; en la siguiente veremos la cabecera del protocolo, campo por campo. Cada byte de una conexión TCP tiene su propio número de secuencia de 32 bits. En un *host* que opera a toda velocidad en una LAN de 10 Mbps,

en teoría los números de secuencia podrían volver a comenzar en una hora, pero en la práctica tarda mucho más tiempo. Se usan los números de secuencia tanto para los acuses de recibo como para el mecanismo de ventana, que utilizan campos de cabecera de 32 bits distintos.

La entidad TCP transmisora y la receptora intercambian datos en forma de segmentos. Un **segmento** consiste en una cabecera TCP fija de 20 bytes (más una parte opcional) seguida de cero o más bytes de datos. El *software* de TCP decide el tamaño de los segmentos; puede acumular datos de varias escrituras para formar un segmento, o dividir los datos de una escritura en varios segmentos. Hay dos límites que restringen el tamaño de segmento. Primero, cada segmento, incluida la cabecera TCP, debe caber en la carga útil de 65,535 bytes del IP. Segundo, cada red tiene una **unidad máxima de transferencia**, o MTU (*maximum transfer unit*), y cada segmento debe caber en la MTU. En la práctica, las MTU generalmente son de unos cuantos miles de bytes y por tanto definen el límite superior del tamaño de segmento. Si un segmento pasa a través de una serie de redes sin fragmentarse y luego se topa con una cuya MTU es menor que el segmento, el enrutador de la frontera fragmenta el segmento en dos o más segmentos más pequeños.

Un segmento demasiado grande para transitar por una red puede dividirse en varios segmentos mediante un enrutador. Cada segmento nuevo recibe sus propias cabeceras TCP e IP, por lo que la fragmentación en los enrutadores aumenta la carga extra total (puesto que cada segmento adicional agrega 40 bytes de información de cabecera).

El protocolo básico usado por las entidades TCP es el protocolo de ventana corrediza. Cuando un transmisor envía un segmento, también inicia un temporizador. Cuando llega el segmento al destino, la entidad TCP receptora devuelve un segmento (con datos, si existen, de otro modo sin ellos) que contiene un número de acuse de recibo igual al siguiente número de secuencia que espera recibir. Si el temporizador del transmisor expira antes de la recepción del acuse de recibo, el transmisor envía de nuevo el segmento.

Aunque este protocolo suena sencillo, tiene muchos vericuetos que cubriremos a continuación. Por ejemplo, dado que los segmentos pueden fragmentarse, es posible que llegue una parte del segmento transmitido y que la entidad TCP receptora envíe un acuse de recibo, pero la otra parte se pierda. También pueden llegar segmentos fuera de orden, por lo que los bytes 3072-4095 podrían llegar pero no enviarse acuse de recibo porque los bytes 2048-3071 no han aparecido aún. También pueden retardarse segmentos en tránsito durante tanto tiempo que el transmisor termina de temporizar y retransmite nuevamente. Si un segmento retransmitido toma una ruta distinta a la del original y se fragmenta de manera diferente, pueden llegar esporádicamente partes tanto del original como del duplicado, requiriéndose una administración cuidadosa para lograr una corriente de bits confiable. Por último, siendo tantas las redes que constituyen la Internet, es posible que un segmento pueda toparse ocasionalmente con una red congestionada (o rota) en alguna parte de su trayectoria.

El TCP debe estar preparado para manejar y resolver estos problemas de una manera eficiente. Se ha invertido una cantidad considerable de esfuerzo en la optimización del desempeño de las corrientes TCP, incluso ante problemas de red. A continuación se estudiarán varios de los algoritmos usados por muchas implementaciones de TCP.

6.4.3. La cabecera de segmento TCP

En la figura 6-24 se muestra la distribución de un segmento TCP. Cada segmento comienza con una cabecera de formato fijo de 20 bytes. La cabecera fija puede ir seguida de opciones de cabecera. Tras las opciones, si las hay, pueden continuar hasta 65,535 – 20 – 20 = 65,515 bytes de datos, donde los primeros 20 se refieren a la cabecera IP y los segundos a la cabecera TCP. Los segmentos sin datos son legales y se usan por lo común para acuses de recibo y mensajes de control.

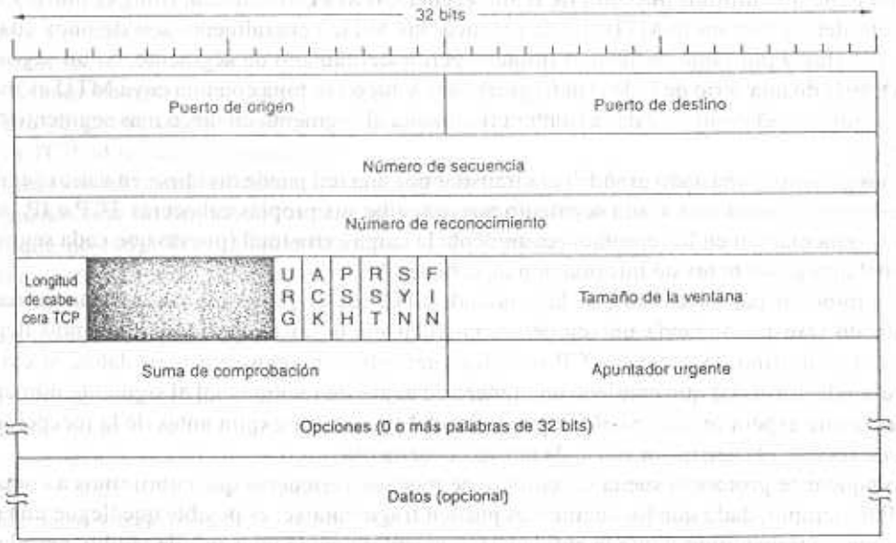


Figura 6-24. Cabecera TCP.

Realicemos la disección de la cabecera TCP campo por campo. Los campos de *puerto de origen* y *puerto de destino* identifican los puntos terminales locales de la conexión. Cada *host* puede decidir por sí mismo la manera de asignar sus propios puertos comenzando por el 256. La dirección de un puerto más la dirección IP de su *host* forman un TSAP único de 48 bits. Los números de socket de origen y de destino en conjunto identifican la conexión.

Los campos de *número de secuencia* y *número de acuse de recibo* desempeñan sus funciones normales. Nótese que el segundo especifica el siguiente byte esperado, no el último byte correctamente recibido. Ambos tienen 32 bits de longitud porque cada byte de datos está numerado en una corriente TCP.

La *longitud de cabecera TCP* indica la cantidad de palabras de 32 bits contenidas en la cabecera TCP. Esta información es necesaria porque el campo de *opciones* es de longitud

variable, por lo que la cabecera también. Técnicamente, este campo en realidad indica el comienzo de los datos en el segmento, medido en palabras de 32 bits, pero ese número es simplemente la longitud de la cabecera en palabras, por lo que el efecto es el mismo.

A continuación viene un campo de 6 bits que no se usan. El que este campo haya sobrevivido intacto durante más de una década es testimonio de lo bien pensado que está el TCP. Protocolos menos bien hechos lo habrían necesitado para corregir errores del diseño original.

Ahora vienen seis banderas de 1 bit. *URG* se establece en 1 si está en uso el *apuntador urgente*. El *apuntador urgente* sirve para indicar un desplazamiento en bytes a partir del número actual de secuencia en el que se encuentran datos urgentes. Este recurso sustituye los mensajes de interrupción. Como se mencionó antes, este recurso es un mecanismo rudimentario para permitir que el transmisor envíe una señal al receptor sin implicar al TCP en la razón de la interrupción.

El bit *ACK* se establece en 1 para indicar que el *número de acuse de recibo* es válido. Si el *ACK* es 0, el segmento no contiene un acuse de recibo, por lo que se ignora el campo de *número de acuse de recibo*.

El bit *PSH* indica datos empujados (con PUSH). Por este medio se solicita atentamente al receptor entregar los datos a la aplicación a su llegada y no ponerlos en *buffer* hasta la recepción de un *buffer* completo (lo que podría hacer en otras circunstancias por razones de eficiencia).

Se usa el bit *RST* para restablecer una conexión que se ha confundido debido a una caída de *host* u otra razón; también sirve para rechazar un segmento no válido o un intento de abrir una conexión. Por lo general, si usted recibe un segmento con el bit *RST* encendido, tiene un problema entre manos.

El bit *SYN* se usa para establecer conexiones. La solicitud de conexión tiene *SYN* = 1 y *ACK* = 0 para indicar que el campo de acuse de recibo incorporado no está en uso. La respuesta de conexión sí lleva un reconocimiento, por lo que tiene *SYN* = 1 y *ACK* = 1. En esencia, el bit *SYN* se usa para denotar CONNECTION REQUEST y CONNECTION ACCEPTED, usándose el bit *ACK* para distinguir entre ambas posibilidades.

El bit *FIN* se usa para liberar una conexión; especifica que el transmisor no tiene más datos que transmitir. Sin embargo, tras cerrar una conexión, un proceso puede continuar recibiendo datos indefinidamente. Ambos segmentos, *SYN* y *FIN*, tienen números de secuencia y por tanto tienen la garantía de procesarse en el orden correcto.

El control de flujo en el TCP se maneja usando una ventana corrediza de tamaño variable. El campo de *ventana* indica la cantidad de bytes que pueden enviarse comenzando por el byte que ya se ha enviado acuse de recibo. Es válido un campo de *ventana* de 0, e indica que se han recibido los bytes hasta *número de acuse de recibo* – 1, inclusive, pero que el receptor actualmente necesita un descanso y quisiera no recibir más datos por el momento, gracias. El permiso para enviar puede otorgarse después enviando un segmento con el mismo *número de acuse de recibo* y un campo de *ventana* distinto de cero.

También se proporciona una *suma de comprobación* para confiabilidad extrema. Es una suma de comprobación de la cabecera, los datos y la pseudocabecera conceptual mostrada en la figura 6-25. Al realizar este cálculo, se establece el campo de *suma de comprobación* del TCP en cero, y se rellena el campo de datos con un byte cero adicional si la longitud es un número non. El algoritmo de suma de comprobación simplemente suma todas las palabras de 16 bits en

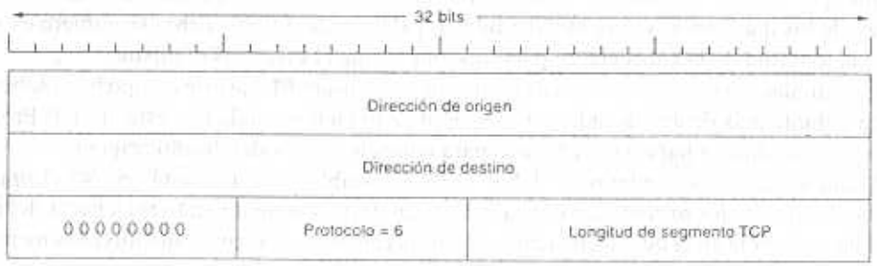


Figura 6-25. Pseudocabecera incluida en la suma de comprobación del TCP.

complemento a 1 y luego obtiene el complemento a 1 de la suma. Como consecuencia, al realizar el cálculo el receptor con el segmento completo, incluido el campo de *suma de comprobación*, el resultado debe ser 0.

La pseudocabecera contiene las direcciones IP de 32 bits de las máquinas de origen y de destino, el número de protocolo de TCP (6), y la cuenta de bytes del segmento TCP (incluida la cabecera). La inclusión de la pseudocabecera en el cálculo de la suma de comprobación TCP ayuda a detectar paquetes mal entregados, pero hacerlo viola la jerarquía de protocolos puesto que las direcciones de IP que contiene pertenecen a la capa IP, no a la capa TCP.

El campo de *opciones* se diseñó para contar con una manera de agregar características extra no cubiertas por la cabecera normal. La opción más importante es la que permite que cada *host* especifique la carga útil TCP máxima que está dispuesto a aceptar. El uso de segmentos grandes es más eficiente que el de segmentos pequeños, puesto que la cabecera de 20 bytes puede entonces amortizarse entre más datos, pero los *hosts* pequeños tal vez no puedan manejar segmentos muy grandes. Durante el establecimiento de la conexión, cada lado puede anunciar su máximo y ver el de su compañero. El más pequeño de los dos números es el ganador. Si un *host* no usa esta opción, predetermina una carga útil de 536 bytes. Se requiere que todos los *hosts* Internet acepten segmentos TCP de 536 + 20 = 556 bytes.

En las líneas con alto ancho de banda, alto retardo o ambas cosas, la ventana de 64 KB con frecuencia es un problema. En una línea T3 (44.736 Mbps) se requieren sólo 12 mseg para enviar una ventana completa de 64 KB. Si el retardo de propagación de ida y vuelta es de 50 mseg (típico de una fibra transcontinental), el transmisor estará inactivo 3/4 del tiempo en espera de acuses de recibo. En una conexión satelital la situación es peor aún. Un tamaño de ventana más grande permitirá al transmisor continuar enviando datos, pero como el campo de *tamaño de ventana* es de 16 bits, es imposible expresar tal tamaño. En el RFC 1323 se propuso una opción de *escala de ventana* que permite al transmisor y al receptor negociar un factor de escala de ventana. Este número permite que ambos lados desplacen el campo de tamaño de ventana hasta 16 bits a la izquierda, permitiendo por tanto un máximo de 2^{32} bytes. La mayoría de las implementaciones actuales de TCP manejan esta opción.

Otra opción propuesta en el RFC 1106 y ahora de uso difundido es el empleo de la repetición selectiva en lugar del protocolo de regresar *n* (*go back n*). Si el receptor recibe un segmento malo

y luego una gran cantidad de segmentos buenos, el protocolo TCP normal en algún momento terminará de temporizar y retransmitirá todos los segmentos sin acuse de recibo, incluidos los que se recibieron correctamente. El RFC 1106 introdujo los NAK, para permitir que el receptor solicite un segmento (o segmentos) específico. Tras recibirlo, puede enviar un acuse de recibo de todos los datos que tiene en *buffer*, reduciendo de esta manera la cantidad de datos retransmitidos.

6.4.4. Gestión de una conexión TCP

En el TCP se establecen las conexiones usando el protocolo de acuerdo de tres vías (*three-way handshake*) estudiado en la sección 6.2.2. Para establecer una conexión, un lado, digamos el servidor, espera pasivamente una conexión entrante ejecutando las primitivas LISTEN y ACCEPT y especificando cierto origen o bien nadie en particular.

El otro lado, digamos el cliente, ejecuta una primitiva CONNECT especificando la dirección y el puerto IP con el que se desea conectar, el tamaño máximo de segmento TCP que está dispuesto a aceptar y opcionalmente algunos datos de usuario (por ejemplo, una contraseña). La primitiva CONNECT envía un segmento TCP con el bit *SYN* encendido y el bit *ACK* apagado, y espera una respuesta.

Al llegar el segmento al destino, la entidad TCP ahí revisa si hay un proceso que haya ejecutado un LISTEN en el puerto indicado en el campo de *puerto de destino*. Si no lo hay, envía una contestación con el bit *RST* encendido para rechazar la conexión.

Si algún proceso está escuchando en el puerto, ese proceso recibe el segmento TCP entrante y puede entonces aceptar o rechazar la conexión; si la acepta, se devuelve un segmento de acuse de recibo. La secuencia de segmentos TCP enviados en el caso normal se muestra en la figura 6-26(a).

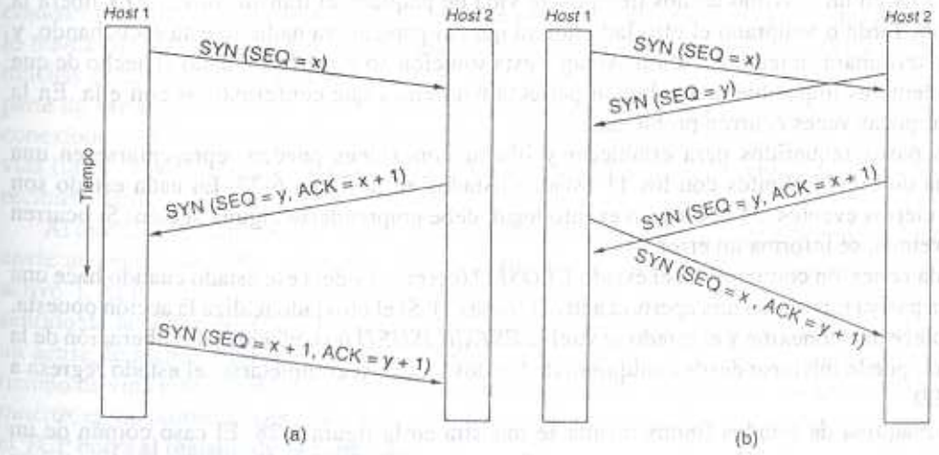


Figura 6-26. (a) Establecimiento de una conexión TCP en el caso normal. (b) Colisión de llamadas.

Nótese que un segmento *SYN* consume 1 byte de espacio de secuencia, por lo que puede reconocerse sin ambigüedades.

En el caso en que dos *hosts* intentan simultáneamente establecer una conexión entre los mismos dos sockets, la secuencia de eventos es la que se ilustra en la figura 6-26(b). El resultado de estos eventos es que sólo se establece una conexión, no dos, pues las conexiones se identifican por sus puntos terminales. Si el primer establecimiento resulta en una conexión identificada por (x, y), al igual que el segundo, sólo se hace una entrada de tabla, es decir, de (x, y).

El número de secuencia inicial de una conexión no es 0 por las razones que señalamos antes. Se usa un esquema basado en reloj, con un pulso de reloj cada 4 μseg. Por seguridad adicional, al caerse un *host*, no podrá reiniciarse durante el tiempo máximo de paquete (120 seg) para asegurar que no haya paquetes de conexiones previas vagando por Internet.

Aunque las conexiones TCP son dúplex integral, para entender la manera en que se liberan las conexiones es mejor visualizarlas como un par de conexiones símplex. Cada conexión símplex se libera independientemente de su igual. Para liberar una conexión, cualquiera de las partes puede enviar un segmento TCP con el bit *FIN* establecido, lo que significa que no tiene más datos por transmitir. Al reconocerse el *FIN*, ese sentido se apaga. Sin embargo, puede continuar un flujo de datos indefinido en el otro sentido. Cuando ambos sentidos se han apagado, se libera la conexión. Normalmente se requieren cuatro segmentos TCP para liberar una conexión, un *FIN* y un *ACK* para cada sentido. Sin embargo, es posible que el primer *ACK* y el segundo *FIN* estén contenidos en el mismo segmento, reduciendo la cuenta total a tres.

Al igual que con las llamadas telefónicas en las que ambas partes dicen adiós y cuelgan el teléfono simultáneamente, ambos extremos de una conexión TCP pueden enviar segmentos *FIN* al mismo tiempo. Ambos se reconocen de la manera normal, y se apaga la conexión. De hecho, en esencia no hay diferencia entre la liberación secuencial o simultánea por parte de los *hosts*.

Para evitar el problema de los dos ejércitos, se usan temporizadores. Si no llega una respuesta a un *FIN* en un máximo de dos tiempos de vida de paquete, el transmisor del *FIN* libera la conexión. Tarde o temprano el otro lado notará que, al parecer, ya nadie lo está escuchando, y también terminará su temporización. Aunque esta solución no es perfecta, dado el hecho de que teóricamente es imposible una solución perfecta tendremos que conformarnos con ella. En la práctica, pocas veces ocurren problemas.

Los pasos requeridos para establecer y liberar conexiones pueden representarse en una máquina de estados finitos con los 11 estados listados en la figura 6-27. En cada estado son legales ciertos eventos. Al ocurrir un evento legal, debe emprenderse alguna acción. Si ocurren otros eventos, se informa un error.

Cada conexión comienza en el estado *CLOSED* (cerrado) y deja ese estado cuando hace una apertura pasiva (LISTEN), o una apertura activa (CONNECT). Si el otro lado realiza la acción opuesta, se establece una conexión y el estado se vuelve *ESTABLISHED* (establecido). La liberación de la conexión puede iniciarse desde cualquiera de los dos lados. Al completarse, el estado regresa a *CLOSED*.

La máquina de estados finitos misma se muestra en la figura 6-28. El caso común de un cliente que se conecta activamente a un servidor pasivo se indica con líneas gruesas (continuas

Estado	Descripción
CLOSED	No hay conexión activa ni pendiente
LISTEN	El servidor espera una llamada
SYN RCVD	Llegó solicitud de conexión; espera ACK
SYN SENT	La aplicación comenzó a abrir una conexión
ESTABLISHED	Estado normal de transferencia de datos
FIN WAIT 1	La aplicación dijo que ya terminó
FIN WAIT 2	El otro lado acordó liberar
TIMED WAIT	Espera que todos los paquetes mueran
CLOSING	Ambos lados intentaron cerrar simultáneamente
CLOSE WAIT	El otro lado inició una liberación
LAST ACK	Espera que todos los paquetes mueran

Figura 6-27. Estados usados en la máquina de estados finitos de administración de conexiones TCP.

para el cliente, punteadas para el servidor). Las líneas delgadas son secuencias de eventos poco comunes. Cada línea de la figura 6-28 se marca mediante un par *evento/acción*. El evento puede ser una llamada de sistema iniciada por el usuario (CONNECT, LISTEN, SEND o CLOSE), la llegada de un segmento (*SYN, FIN, ACK* o *RST*) o, en un caso, una terminación de temporización del doble del tiempo de vida máximo del paquete. La acción es el envío de un segmento de control (*SYN, FIN* o *RST*), o nada, indicado por —. Los comentarios aparecen entre paréntesis.

El diagrama puede entenderse mejor siguiendo primero la trayectoria de un cliente (la línea continua gruesa) y luego la de un servidor (línea punteada gruesa). Al emitir una solicitud CONNECT una aplicación de la máquina cliente, la entidad TCP local crea un registro de conexión, lo marca para indicar que está en el estado *SYN SENT*, y envía un segmento *SYN*. Nótese que muchas conexiones pueden estar abiertas (o en proceso de apertura) al mismo tiempo como parte de varias aplicaciones, por lo que el estado es por conexión y se asienta en el registro de conexiones. Al llegar el *SYN+ACK*, el TCP envía el *ACK* final del protocolo de acuerdo de tres vías (*three-way handshake*) y se conmuta al estado *ESTABLISHED*. Ahora pueden enviarse y recibirse datos.

Al terminar una aplicación, ejecuta una primitiva CLOSE, que causa que la entidad TCP local envíe un segmento *FIN* y espere el *ACK* correspondiente (recuadro punteado rotulado "cierre activo"). Al llegar el *ACK*, se hace una transición al estado *FIN WAIT 2*, y ya está cerrado un sentido de la conexión. Cuando también cierra el otro lado, llega un *FIN*, para el cual se envía un acuse de recibo. Ahora ambos lados están cerrados, pero el TCP espera un tiempo igual al tiempo de vida máximo del paquete para garantizar que todos los paquetes de la conexión hayan muerto, como protección en caso de la pérdida de un acuse de recibo. Al expirar el temporizador, el TCP borra el registro de la conexión.

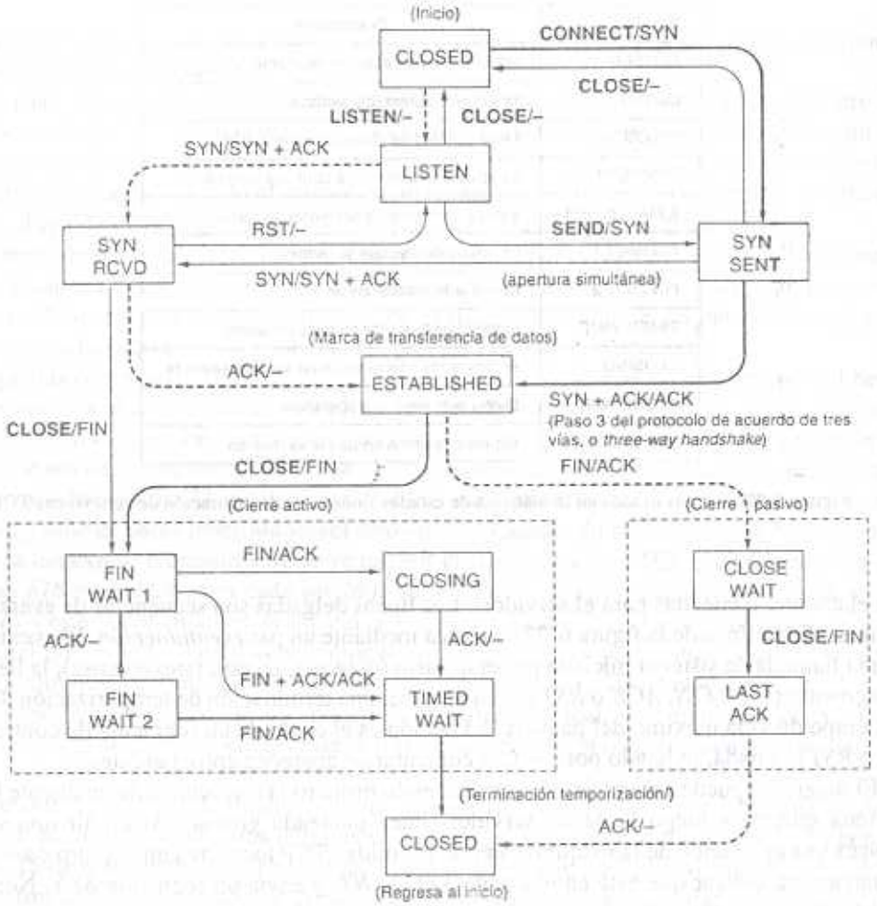


Figura 6-28. Máquina de estados finitos de administración de conexiones TCP. La línea continua gruesa es la trayectoria normal de un cliente. La línea punteada gruesa es la trayectoria normal de un servidor. Las líneas delgadas son eventos poco comunes.

Examinemos ahora la administración de la conexión desde el punto de vista del servidor. El servidor hace un LISTEN y se detiene a esperar la aparición de alguien. Al llegar un *SYN*, se envía un acuse de recibo y el servidor pasa al estado *SYN RCVD*. Cuando llega el acuse de recibo del *SYN* del servidor, el acuerdo de tres vías (*three-way handshake*) se ha completado y el servidor regresa al estado *ESTABLISHED*. Ahora puede ocurrir la transferencia de datos.

Cuando el cliente ha tenido suficiente, hace un CLOSE, que causa la llegada de un *FIN* al servidor (recuadro punteado rotulado "cierre pasivo"). Entonces se envía una señal al servidor.

Cuando éste también hace un CLOSE, se envía un *FIN* al cliente. Al llegar el acuse de recibo del cliente, el servidor libera la conexión y elimina el registro de conexión.

6.4.5. Política de transmisión del TCP

La administración de ventanas en el TCP no está atada directamente a los acuses de recibo como en la mayoría de los protocolos de enlace de datos. Por ejemplo, supóngase que el receptor tiene un *buffer* de 4096 bytes, como se muestra en la figura 6-29. Si el transmisor envía un segmento de 2048 bytes que se recibe correctamente, el receptor enviará el acuse de recibo del segmento. Sin embargo, dado que ahora sólo tiene 2048 de espacio de *buffer* (hasta que la aplicación retire algunos datos del *buffer*), anunciará una ventana de 2048 comenzando con el siguiente byte esperado.

Ahora el transmisor envía otros 2048 bytes, para los cuales el receptor envía el acuse de recibo, pero la ventana anunciada es de 0. El transmisor debe detenerse hasta que el proceso

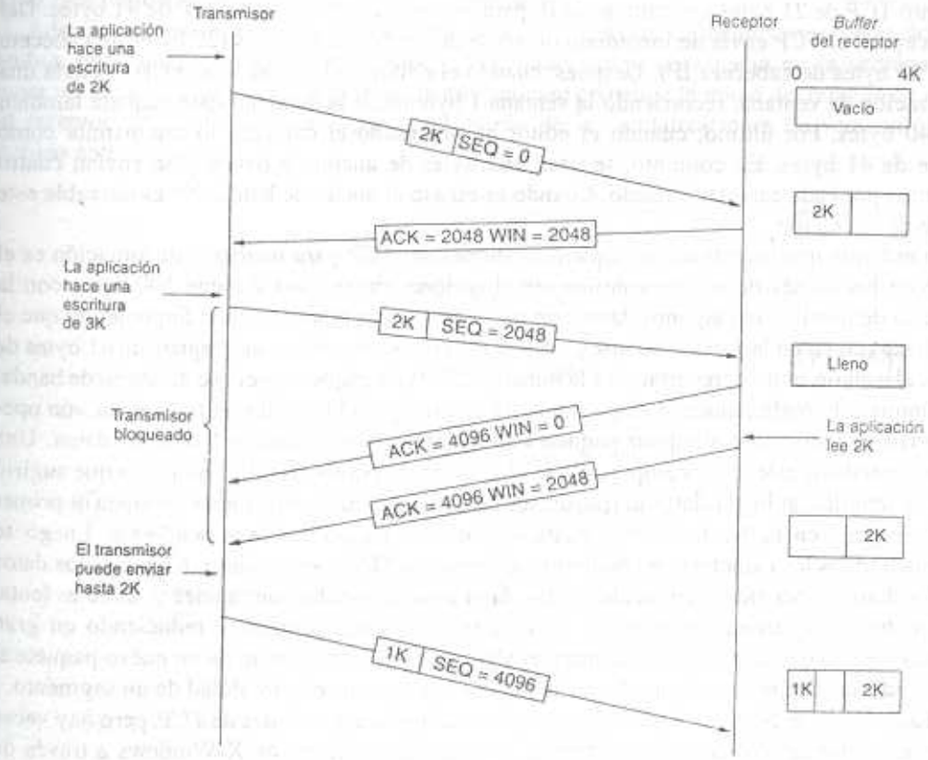


Figura 6-29. Administración de ventanas en TCP.

de aplicación del *host* receptor retire algunos datos del *buffer*, en cuyo momento el TCP puede anunciar una ventana más grande.

Cuando la ventana es de 0, el transmisor normalmente no puede enviar segmentos, salvo dos excepciones. Primero, pueden enviarse datos urgentes (por ejemplo, para permitir que el usuario mate el proceso en ejecución en la máquina remota). Segundo, el transmisor puede enviar un segmento de 1 byte para hacer que el receptor reanuncie el siguiente byte esperado y el tamaño de la ventana. El estándar TCP proporciona explícitamente esta opción para evitar un interbloqueo si llega a perderse un anuncio de ventana.

No se requiere que los transmisores envíen datos tan pronto como llegan de la aplicación. Tampoco se requiere que los receptores envíen acuses de recibo tan pronto como sea posible. Por ejemplo, en la figura 6-29, cuando llegaron los primeros 2 KB de datos, el TCP, sabiendo que tenía disponible una ventana de 4 KB, habría actuado perfectamente bien si simplemente pone en *buffer* los datos hasta la llegada de otros 2 KB, para poder transmitir un segmento con una carga útil de 4 KB. Esta libertad puede explotarse para mejorar el desempeño.

Considere una conexión TELNET a un editor interactivo que reacciona con cada pulso de tecla. En el peor caso, al llegar un carácter a la entidad TCP transmisora, el TCP crea un segmento TCP de 21 bytes que entrega al IP para su envío como datagrama IP de 41 bytes. Del lado receptor, el TCP envía de inmediato un acuse de recibo de 40 bytes (20 bytes de cabecera TCP y 20 bytes de cabecera IP). Después, cuando el editor ha leído el byte, el TCP envía una actualización de ventana, recorriendo la ventana 1 byte hacia la derecha. Este paquete también es de 40 bytes. Por último, cuando el editor ha procesado el carácter, lo retransmite como paquete de 41 bytes. En conjunto, se usan 162 bytes de ancho de banda y se envían cuatro segmentos por cada carácter pulsado. Cuando es escaso el ancho de banda, no es deseable este método de operación.

Un enfoque que usan muchas implementaciones del TCP para mejorar esta situación es el retardo de los acuses de recibo y de las actualizaciones de ventana durante 500 mseg con la esperanza de que lleguen algunos datos con los cuales viajar gratuitamente. Suponiendo que el editor hace eco en un lapso de 500 mseg, sólo se necesita enviar ahora un paquete de 41 bytes de regreso al usuario remoto, recortando a la mitad la cuenta de paquetes y el uso de ancho de banda.

Aunque esta regla reduce la carga impuesta a la red por el receptor, el transmisor aún opera de manera ineficiente al enviar paquetes de 41 bytes que contienen 1 byte de datos. Una manera de reducir este uso es empleando el **algoritmo de Nagle** (Nagle, 1984). Lo que sugirió Nagle es sencillo: al llegar datos al transmisor un byte a la vez, simplemente se envía el primer byte y se pone en *buffer* los demás hasta el acuse de recibo del byte pendiente. Luego se transmiten todos los caracteres en *buffer* en un segmento TCP y se comienzan a poner los datos en *buffer* hasta haber sido reconocidos todos. Si el usuario escribe con rapidez y la red es lenta, puede entrar una cantidad importante de caracteres en cada segmento, reduciendo en gran medida el ancho de banda usado. Además, el algoritmo permite el envío de un nuevo paquete si han entrado suficientes datos para llenar la mitad de la ventana o la totalidad de un segmento.

El algoritmo de Nagle se usa ampliamente en las implementaciones de TCP, pero hay veces en que es mejor inhabilitarlo. En particular, al operar una aplicación X-Windows a través de Internet, los movimientos del ratón tienen que enviarse a la computadora remota. Su acumulación para enviarlos en ráfagas hace que el movimiento del cursor sea errático, lo que no complace mucho a los usuarios.

Otro problema que puede arruinar el desempeño del TCP es el **síndrome de ventana tonta** (*silly window syndrome*) (Clark, 1982). Este problema ocurre cuando se pasan datos a la entidad transmisora en bloques grandes, pero una aplicación interactiva del lado receptor lee datos a razón de 1 byte a la vez. Para ver el problema, observe la figura 6-30. Inicialmente, el *buffer* TCP del lado receptor está lleno y el transmisor lo sabe (es decir, tiene un tamaño de ventana de 0). Entonces la aplicación interactiva lee un carácter de la corriente TCP. Esta acción hace feliz al TCP receptor, por lo que envía una actualización de ventana al transmisor indicando que está bien que envíe 1 byte. El transmisor accede y envía 1 byte. El *buffer* ahora está lleno, por lo que el receptor reconoce el segmento de 1 byte pero establece la ventana en 0. Este comportamiento puede continuar indefinidamente.

La solución de Clark es evitar que el receptor envíe una actualización de ventana para 1 byte. En cambio, se le obliga a esperar hasta tener disponible una cantidad decente de espacio, y luego lo anuncia. Específicamente, el receptor no debe enviar una actualización de ventana hasta que pueda manejar el tamaño máximo de segmento que anunció al establecerse la conexión, o que su *buffer* esté vacío a la mitad, lo que sea más pequeño.

Además, el transmisor también puede ayudar al no enviar segmentos muy pequeños. En cambio, debe intentar esperar hasta haber acumulado suficiente espacio en la ventana para enviar un segmento completo, o cuando menos uno que contenga la mitad del tamaño de *buffer* del receptor (que debe estimar a partir del patrón de las actualizaciones de ventana que ha recibido anteriormente).

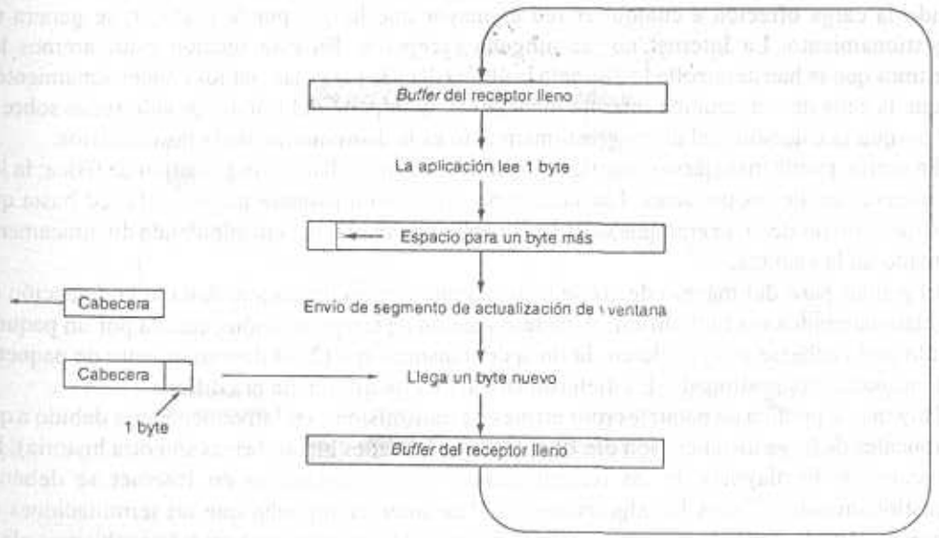


Figura 6-30. Síndrome de ventana tonta.

El algoritmo de Nagle y la solución de Clark al síndrome de ventana tonta son complementarios. Nagle trataba de resolver el problema causado por la entrega de datos al TCP desde la aplicación transmisora un byte a la vez. Clark trataba de resolver el problema de que la aplicación receptora toma los datos del TCP un byte a la vez. Ambas soluciones son válidas y pueden operar juntas. La meta es que el transmisor no envíe segmentos pequeños y que el receptor no los pida.

El TCP receptor también puede hacer más para mejorar el desempeño que simplemente actualizar ventanas en unidades grandes. Al igual que el TCP transmisor, tiene la capacidad de poner datos en *buffer*, por lo que puede bloquear una solicitud READ de la aplicación hasta poder proporcionar un bloque grande de datos. Hacer esto reduce la cantidad de llamadas al TCP y, por tanto, la carga extra. Por supuesto, también aumenta el tiempo de respuesta, pero en las aplicaciones no interactivas, como la transferencia de archivos, la eficiencia puede tener mayor peso que el tiempo de respuesta a las solicitudes individuales.

Otro problema del receptor es qué debe hacer con los segmentos fuera de orden. Pueden conservarse o descartarse, al albedrío del receptor. Por supuesto, los acuses de recibo pueden enviarse sólo después de haber recibido todos los datos hasta el byte reconocido. Si el receptor recibe los segmentos 0, 1, 2, 4, 5, 6 y 7, puede enviar un acuse de recibo de todos los bytes hasta el último byte del segmento 2, inclusive. Al terminar el transmisor la temporización, retransmitirá el segmento 3. Si el receptor tiene en *buffer* los segmentos 4 a 7, al recibir el segmento 3 puede enviar un acuse de recibo de todos los bytes hasta el final del segmento 7.

6.4.6. Control de congestionamientos con TCP

Cuando la carga ofrecida a cualquier red es mayor que la que puede manejar, se genera un congestionamiento. La Internet no es ninguna excepción. En esta sección estudiaremos los algoritmos que se han desarrollado durante la última década para manejar los congestionamientos. Aunque la capa de red también intenta manejarlos, gran parte del trabajo pesado recae sobre el TCP porque la solución real al congestionamiento es la disminución de la tasa de datos.

En teoría, puede manejarse el congestionamiento aprovechando un principio de física: la ley de conservación de los paquetes. La idea es no inyectar un paquete nuevo en la red hasta que salga uno viejo (es decir, se entregue). El TCP intenta lograr esta meta manipulando dinámicamente el tamaño de la ventana.

El primer paso del manejo de congestionamientos es su detección. Antaño la detección de congestionamientos era muy difícil. Una terminación de temporización causada por un paquete perdido pudo deberse a (1) ruido en la línea de transmisión o (2) el descartamiento de paquetes en el enrutador congestionado. La determinación de esta diferencia era difícil.

Hoy día, la pérdida de paquetes por errores de transmisión es relativamente rara debido a que las troncales de larga distancia son de fibra (aunque las redes inalámbricas son otra historia). En consecuencia, la mayoría de las terminaciones de temporización en Internet se deben a congestionamientos. Todos los algoritmos TCP de Internet suponen que las terminaciones de temporización son causadas por congestionamientos y las revisan en busca de problemas, de la misma manera que los mineros observan a sus canarios.

Antes de analizar la manera en que el TCP reacciona al congestionamiento, describiremos primero lo que hace para evitar que ocurra. Al establecerse una conexión, se tiene que seleccionar un tamaño de ventana adecuado. El receptor puede especificar una ventana con base en su tamaño de *buffer*. Si el transmisor se ajusta a su tamaño de ventana, no ocurrirán problemas por desbordamiento de *buffers* en la terminal receptora, pero aún pueden ocurrir debido a congestionamientos internos en la red.

En la figura 6-31 ilustramos este problema hidráulicamente. En la figura 6-31(a) vemos un tubo grueso que conduce a un receptor de poca capacidad. Mientras el transmisor no envíe más agua de la que puede contener la cubeta, no se perderá agua. En la figura 6-31(b), el factor limitante no es la capacidad de la cubeta, sino la capacidad de conducción interna de la red. Si entra demasiada agua a alta velocidad, esta retrocederá, perdiéndose algo (en este caso, por el desbordamiento del embudo).

La solución de Internet es aceptar que existen dos problemas potenciales (capacidad de la red y capacidad del receptor) y manejarlos por separado. Para ello, cada transmisor mantiene dos ventanas: la ventana que ha otorgado el receptor y una segunda ventana, la **ventana de congestionamiento**; cada una refleja la cantidad de bytes que puede enviar el transmisor. La cantidad de bytes que pueden enviarse es la cifra menor de las dos ventanas. Por tanto, la ventana efectiva es el mínimo de lo que el transmisor piensa que es correcto y lo que el receptor piensa

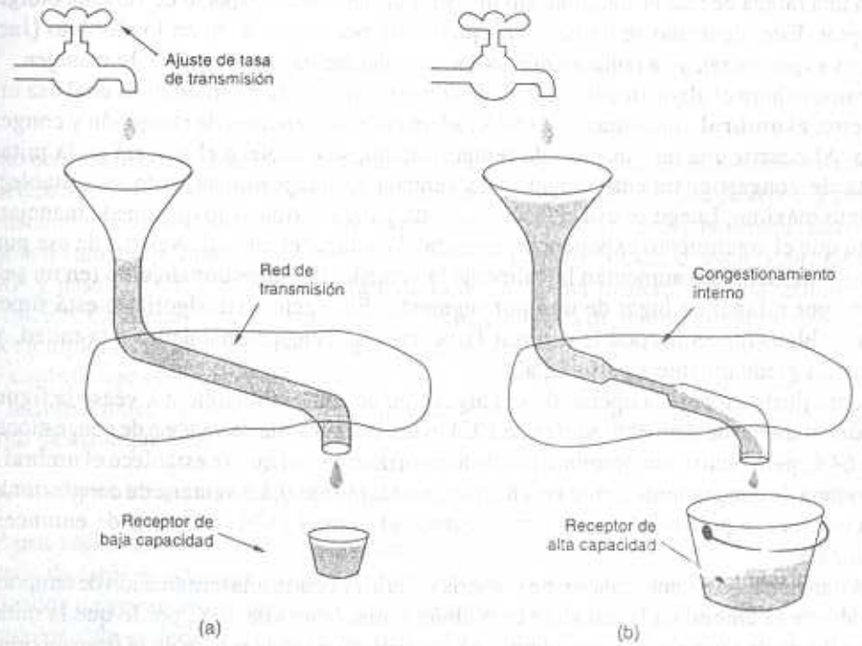


Figura 6-31. (a) Red rápida alimentando un receptor de baja capacidad. (b) Red lenta alimentando un receptor de alta capacidad.

que está bien. Si el receptor dice "envía 8K" pero el transmisor sabe que las ráfagas de más de 4K saturan la red, envía 4K. Por otra parte, si el receptor dice "envía 8K" y el transmisor sabe que las ráfagas de hasta 32K pueden llegar sin problemas, envía los 8K solicitados.

Al establecerse una conexión, el transmisor asigna a la ventana de congestionamiento el tamaño de segmento máximo usado por la conexión; entonces envía un segmento máximo. Si se recibe el acuse de recibo de este segmento antes de terminar el temporizador, el transmisor agrega el equivalente en bytes de un segmento a la ventana de congestionamiento para hacerla de dos segmentos de tamaño máximo, y envía dos segmentos. A medida que se reconoce cada uno de estos segmentos, se aumenta el tamaño de la ventana de congestionamiento en un segmento máximo. Cuando la ventana de congestionamiento es de n segmentos, si de todos los n se reciben acuses de recibo a tiempo, se aumenta el tamaño de la ventana de congestionamiento en la cuenta de bytes correspondiente a n segmentos. De hecho, cada ráfaga reconocida con éxito duplica la ventana de congestionamiento.

La ventana de congestionamiento sigue creciendo exponencialmente hasta ocurrir una terminación de temporización o alcanzar el tamaño de la ventana receptora. La idea es que, si las ráfagas de 1024, 2048 y 4096 bytes funcionan bien, pero una ráfaga de 8192 produce una terminación de temporización, la ventana de congestionamiento debe establecerse en 4096 para evitar el congestionamiento. Mientras el tamaño de la ventana de congestionamiento permanezca en 4096, no se enviará una ráfaga de mayor longitud, sin importar la cantidad de espacio de ventana otorgada por el receptor. Este algoritmo se llama **arranque lento**, pero no es lento en lo absoluto (Jacobson, 1988); es exponencial, y se requiere que todas las implementaciones de TCP lo manejen.

Veamos ahora el algoritmo de control de congestionamiento de Internet, el cual usa un tercer parámetro, el **umbral**, inicialmente de 64K, además de las ventanas de recepción y congestionamiento. Al ocurrir una terminación de temporización, se establece el umbral en la mitad de la ventana de congestionamiento actual, y la ventana de congestionamiento se restablece a un segmento máximo. Luego se usa el arranque lento para determinar lo que puede manejar la red, excepto que el crecimiento exponencial termina al alcanzar el umbral. A partir de ese punto, las transmisiones exitosas aumentan linealmente la ventana de congestionamiento (en un segmento máximo por ráfaga) en lugar de uno por segmento. En efecto, este algoritmo está suponiendo que probablemente es aceptable recortar la ventana de congestionamiento a la mitad, y luego aumentarla gradualmente a partir de ahí.

Como ilustración de la operación del algoritmo de congestionamiento, véase la figura 6-32. El tamaño máximo de segmento aquí es de 1024 bytes. Inicialmente, la ventana de congestionamiento era de 64K, pero ocurre una terminación de temporización, así que se establece el umbral en 32K y la ventana de congestionamiento en 1K, para la transmisión 0. La ventana de congestionamiento entonces crece exponencialmente hasta alcanzar el umbral (32K). A partir de entonces, crece linealmente.

La transmisión 13 tiene mala suerte (debería saberlo) y ocurre una terminación de temporización. Se establece el umbral en la mitad de la ventana actual (ahora de 40K, por lo que la mitad es de 20K), e inicia de nuevo el arranque lento. Al llegar los acuses de recibo de la transmisión 18, los primeros cuatro incrementan la ventana de congestionamiento en un segmento máximo, pero después de eso el crecimiento se vuelve lineal nuevamente.

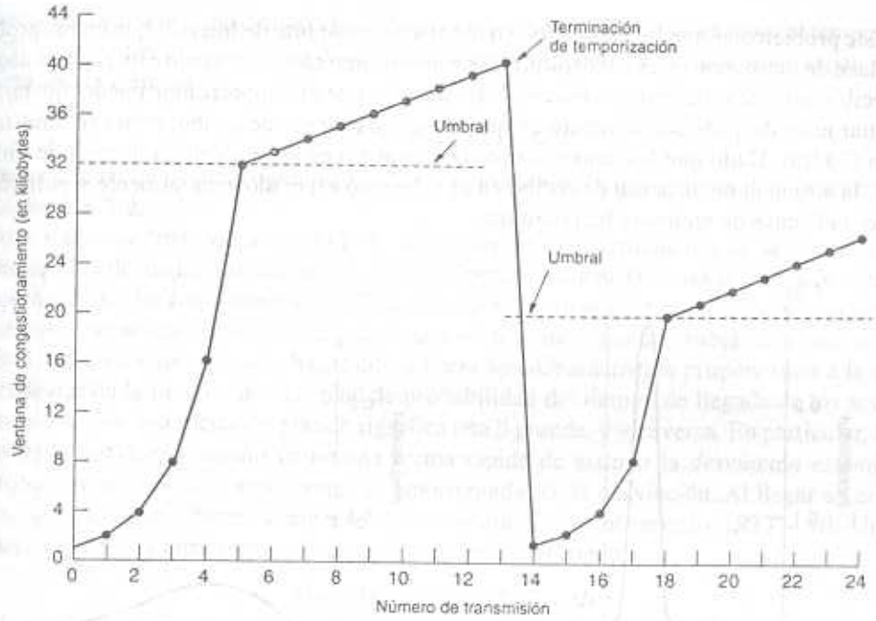


Figura 6-32. Ejemplo del algoritmo de congestionamiento de Internet.

Si no ocurren más terminaciones de temporización, la ventana de congestionamiento continuará creciendo hasta el tamaño de la ventana del receptor. En ese punto, dejará de crecer y permanecerá constante mientras no ocurran más terminaciones de temporización y la ventana del receptor no cambie de tamaño. Como nota al margen, si llega un paquete SOURCE QUENCH (suprimir origen) de ICMP y pasa al TCP, este evento será tratado de la misma manera que una terminación de temporización. Continúa el trabajo para mejorar los mecanismos de control de congestionamientos.

Por ejemplo, Brakmo *et al.* (1994) ha informado una mejora del rendimiento del TCP del 40 al 70% controlando con mayor precisión el reloj, prediciendo los congestionamientos antes de ocurrir terminaciones de temporización y usando este sistema de pronto aviso para mejorar el algoritmo de arranque lento.

6.4.7. Gestión de temporizadores del TCP

El TCP usa varios temporizadores (al menos conceptualmente) para hacer su trabajo. El más importante de éstos es el **temporizador de retransmisión**. Al enviarse un segmento, se inicia un temporizador de retransmisiones. Si el acuse de recibo del segmento llega antes de expirar el temporizador, éste se detiene. Si, por otra parte, el temporizador termina antes de llegar el acuse de recibo, se retransmite el segmento (y se inicia nuevamente el temporizador). Surge entonces la pregunta: ¿qué tan grande debe ser el intervalo de terminación del temporizador?

Este problema es mucho más difícil en la capa de transporte de Internet que en los protocolos de enlace de datos generales del capítulo 3. En este último caso, el retardo esperado es altamente predecible (es decir, tiene una variación baja), por lo que el temporizador puede ajustarse para terminar justo después del momento en que se espera el acuse de recibo, como se muestra en la figura 6-33(a). Dado que los acuses de recibo pocas veces se retardan en la capa de enlace de datos, la ausencia de un acuse de recibo en el momento esperado generalmente significa que el marco o el acuse de recibo se han perdido.

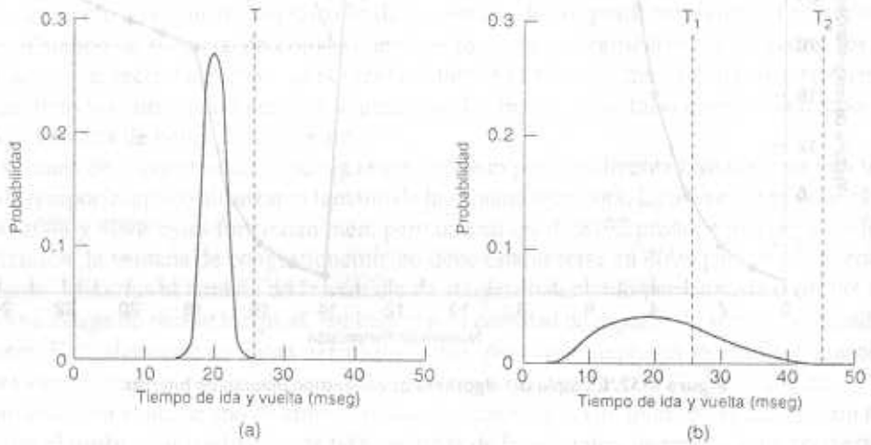


Figura 6-33. (a) Densidad de probabilidad de los tiempos de llegada de reconocimientos en la capa de enlace de datos. (b) Densidad de probabilidad de los tiempos de llegada de reconocimientos para el TCP.

El TCP enfrenta a un entorno radicalmente distinto. La función de densidad de probabilidad del tiempo que tarda en regresar un acuse de recibo TCP se parece más a la figura 6-33(b) que a la figura 6-33(a). Es complicada la determinación del tiempo de ida y vuelta al destino. Aun cuando se conoce, la selección del intervalo de terminación de temporización también es difícil. Si se hace demasiado corto, digamos T_1 en la figura 6-33(b), ocurrirán retransmisiones innecesarias, cargando la Internet con paquetes inútiles. Si se hace demasiado largo (T_2), el desempeño sufrirá debido al gran retardo de retransmisión de cada paquete perdido. Es más, la varianza y la media de la distribución de llegadas de acuses de recibo pueden variar con rapidez en unos cuantos segundos, a medida que se generan y se resuelven congestionamientos.

La solución es usar un algoritmo muy dinámico que ajuste constantemente el intervalo de terminación de temporización, con base en mediciones continuas del desempeño de la red. El algoritmo que generalmente usa el TCP lo debemos a Jacobson (1988) y funciona como sigue. Por cada conexión, el TCP mantiene una variable, *RTT* (*round-trip time*), que es la mejor estimación actual del tiempo de ida y vuelta al destino en cuestión. Al enviarse un segmento, se inicia un temporizador, tanto para ver el tiempo que tarda el acuse de recibo como para disparar una retransmisión si se tarda demasiado. Si llega el acuse de recibo antes de expirar el temporizador, el TCP mide el tiempo que tardó el acuse de recibo, digamos M. Entonces actualiza *RTT* de acuerdo con la fórmula

$$RTT = \alpha RTT + (1 - \alpha)M$$

donde α es un factor de amortiguamiento que determina el peso que se le da al valor anterior. Por lo común, $\alpha = 7/8$.

Aun dado un buen valor de *RTT*, la selección de una terminación de temporización de retransmisión adecuada es un asunto no trivial. Normalmente el TCP usa βRTT, pero el truco es seleccionar β. En las implementaciones iniciales, β siempre era 2, pero la experiencia demostró que un valor constante era inflexible puesto que no respondía cuando subía la variación.

En 1988, Jacobson propuso hacer que β fuera aproximadamente proporcional a la desviación estándar de la función de densidad de probabilidad del tiempo de llegada de los acuses de recibo, por lo que una variación grande significa una β grande, y viceversa. En particular, sugirió el uso de la *desviación media* como una forma rápida de estimar la *desviación estándar*. Su algoritmo requiere mantener otra variable amortiguada, D, la desviación. Al llegar un acuse de recibo, se calcula la diferencia entre el valor esperado y el observado, $|RTT - M|$. Un valor amortiguado de esta cifra se mantiene en D mediante la fórmula

$$D = \alpha D + (1 - \alpha)\,|RTT - M|$$

donde α puede ser o no el mismo valor usado para amortiguar *RTT*. Si bien D no es exactamente igual a la desviación estándar, es bastante buena, y Jacobson demostró la manera de calcularla usando sólo sumas, restas y desplazamientos de enteros, lo que es una gran ventaja. La mayoría de las implementaciones TCP usan ahora este algoritmo y establecen el intervalo de terminación de temporización en

$$\text{Terminación de temporización} = RTT + 4*D$$

La selección del factor 4 es un tanto arbitraria, pero tiene dos ventajas. Primero, puede hacerse la multiplicación por 4 con un solo desplazamiento. Segundo, reduce al mínimo las terminaciones de temporización y las retransmisiones innecesarias porque menos del 1% de todos los paquetes llegan más de cuatro desviaciones estándar tarde. (En realidad, Jacobson sugirió inicialmente que se usara 2, pero el trabajo posterior ha demostrado que 4 da un mejor desempeño.)

Un problema que ocurre con la estimación dinámica de *RTT* es qué se debe hacer cuando un segmento termina de temporizar y se envía de nuevo. Cuando llega el acuse de recibo, no es claro si éste se refiere a la primera transmisión o a una posterior. Si se adivina mal se puede contaminar seriamente la estimación de *RTT*. Phil Karn descubrió este problema de la manera difícil. Él es un radioaficionado interesado en la transmisión de paquetes TCP/IP a través de la radio amateur, un medio notoriamente poco confiable (en un buen día, pasarán la mitad de los paquetes). Karn hizo una propuesta sencilla: no actualizar el *RTT* con ninguno de los segmentos retransmitidos. En cambio, se duplica la terminación de temporización con cada falla hasta que los segmentos pasan a la primera. Este sistema se llama **algoritmo de Karn** y lo usan la mayoría de las implementaciones TCP.

El temporizador de retransmisiones no es el único usado por el TCP. El segundo temporizador es el **temporizador de persistencia**, diseñado para evitar el siguiente interbloqueo. El receptor envía un acuse de recibo con un tamaño de ventana de 0, indicando al transmisor que espere. Después, el receptor actualiza la ventana, pero se pierde el paquete con la actualización. Ahora, tanto el transmisor como el receptor están esperando que el otro haga algo. Cuando termina el temporizador de persistencia, el transmisor envía una prueba al receptor. La respuesta de la prueba indica el tamaño de la ventana. Si aún es de cero, se inicia el temporizador de persistencia nuevamente, repitiéndose el ciclo. Si es diferente de cero, pueden enviarse datos.

Un tercer temporizador usado en algunas implementaciones es el **temporizador de seguir con vida** (*keepalive timer*). Cuando una conexión ha estado ociosa durante demasiado tiempo, el temporizador de seguir con vida puede terminar, haciendo que un lado compruebe que el otro aún está ahí. Si no se recibe respuesta, se termina la conexión. Esta característica es motivo de controversias porque agrega carga extra y puede terminar una conexión por lo demás saludable debido a una partición temporal de la red.

El último temporizador usado en cada conexión TCP es el que se usa en el estado *TIMED WAIT* durante el cierre; opera durante el doble del tiempo máximo de vida de paquete para asegurar que, al cerrarse una conexión, todos los paquetes creados por ella hayan muerto.

6.4.8. UDP

El grupo de protocolos de Internet también maneja un protocolo de transporte sin conexiones, el **UDP** (*User Data Protocol*, **protocolo de datos de usuario**). El UDP ofrece a las aplicaciones un mecanismo para enviar datagramas IP en bruto encapsulados sin tener que establecer una conexión. Muchas aplicaciones cliente-servidor que tienen una solicitud y una respuesta usan el UDP en lugar de tomarse la molestia de establecer y luego liberar una conexión. El UDP se describe en el RFC 768. Un segmento UDP consiste en una cabecera de 8 bytes seguida de los datos. La cabecera se muestra en la figura 6-34. Los dos puertos sirven para lo mismo que en el TCP: para identificar los puntos terminales de las máquinas de origen y destino. El campo de *longitud UDP* incluye la cabecera de 8 bytes y los datos. La *suma de comprobación UDP* incluye la misma pseudocabecera de formato mostrada en la figura 6-25, la cabecera UDP y los datos UDP, rellenados a una cantidad par de bytes de ser necesario. Esta suma es opcional y se almacena como 0 si no se calcula (un cálculo real de 0 se almacena como una sucesión de unos, que es lo mismo en complemento a 1). Inutilizarla resulta absurdo, a menos que la calidad de los datos no importe (por ejemplo, voz digitalizada).

Figura 6-34. La cabecera UDP.

6.4.9. TCP y UDP inalámbricos

En teoría, los protocolos de transporte deben ser independientes de la tecnología de la capa de red subyacente. En particular, el TCP no debería preocuparse si el IP está operando por fibra o por radio. En la práctica sí importa, puesto que la mayoría de las implementaciones de TCP han sido optimizadas cuidadosamente con base en supuestos que se cumplen en las redes alámbricas, pero no en las inalámbricas. Ignorar las propiedades de la transmisión inalámbrica puede conducir a implementaciones del TCP correctas desde el punto de vista lógico pero con un desempeño horrendo.

El problema principal es el algoritmo de control de congestionamientos. Hoy día, casi todas las implementaciones de TCP suponen que las terminaciones de temporización ocurren por congestionamientos, no por paquetes perdidos. En consecuencia, al terminar un temporizador, el TCP disminuye su velocidad y envía con menor ímpetu (por ejemplo, el algoritmo de arranque lento de Jacobson). Lo que se pretende con este enfoque es reducir la carga de la red y aliviar así el congestionamiento.

Desafortunadamente, los enlaces de transmisión inalámbrica son muy poco confiables; pierden paquetes todo el tiempo. El enfoque adecuado para el manejo de paquetes perdidos es enviarlos nuevamente, tan pronto como sea posible. La reducción de la velocidad simplemente empeora las cosas. Si, digamos, se pierde el 20% de todos los paquetes, entonces cuando el transmisor envía 100 paquetes/seg, el rendimiento es de 80 paquetes/seg. Si el transmisor reduce su velocidad a 50 paquetes/seg, el rendimiento cae a 40 paquetes/seg.

En efecto, al perderse un paquete en una red alambrada, el transmisor debe reducir la velocidad. Cuando se pierde uno en una red inalámbrica, el transmisor debe acelerar. Cuando el transmisor no sabe de qué clase de red se trata, es difícil tomar la decisión correcta.

Con frecuencia, la trayectoria del transmisor al receptor no es homogénea. Los primeros 1000 km podrían ser a través de una red alambrada, pero el último km podría ser inalámbrico. Ahora es más difícil la decisión correcta en el caso de una terminación de temporización, ya que importa el lugar donde ocurrió el problema. Una solución propuesta por Bakne y Badrinath (1995), el **TCP indirecto**, es la división de la conexión TCP en dos conexiones distintas, como se muestra en la figura 6-35. La primera conexión va del transmisor a la estación base. La segunda va de la estación base al receptor. La estación base simplemente copia paquetes entre las conexiones en ambas direcciones.

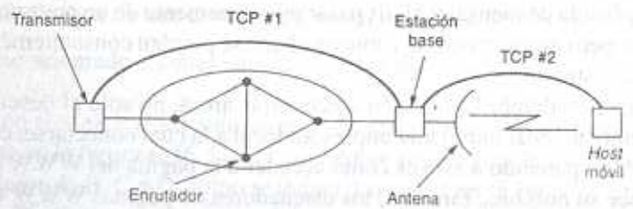


Figura 6-35. División de una conexión TCP en dos conexiones.

La ventaja de este esquema es que ahora ambas conexiones son homogéneas. Las terminaciones de temporización de la primera conexión pueden reducir la velocidad del transmisor, y las terminaciones de temporización de la segunda pueden acelerarla. La desventaja es que se viola por completo la semántica del TCP. Dado que cada parte de la conexión es una conexión TCP completa, la estación base reconoce cada segmento TCP de la manera normal, sólo que ahora la recepción de un reconocimiento en el transmisor no significa que el receptor recibió el segmento, sino únicamente que la estación base lo recibió.

Una solución diferente, debida a Balakrishnan *et al.* (1995), no divide la semántica del TCP; funciona haciendo varias modificaciones pequeñas al código de capa de red de la estación base. Uno de los cambios es la adición de un agente espía que observa y pone en caché los segmentos TCP que van al *host* móvil y los acuses de recibo que regresan de él. Cuando el agente espía ve un segmento TCP que sale al *host* móvil, pero no ve el regreso de un acuse de recibo antes de que su temporizador (relativamente corto) termine, simplemente retransmite ese segmento, sin indicar al origen que lo está haciendo.

El agente también genera una retransmisión cuando ve acuses de recibo duplicados del *host* móvil, lo que invariablemente significa que algo le ha fallado a este *host*. Los acuses de recibo duplicados se descartan en el lugar, para evitar que el origen los malinterprete como una señal de congestionamiento.

Sin embargo, una desventaja de esta transparencia es que, si el enlace inalámbrico tiene muchas pérdidas, el origen puede terminar de temporizar esperando un acuse de recibo e invocar el algoritmo de control de congestionamiento. Con el TCP indirecto, el algoritmo de control de congestionamiento nunca iniciará hasta que realmente haya un congestionamiento en la parte alambrada de la red.

El documento de Balakrishnan *et al.* también sugiere una solución al problema de segmentos perdidos que se originan en el *host* móvil. Al notar una estación base un hueco en los números de secuencia de entrada, genera una solicitud de repetición selectiva de los bytes faltantes usando una opción TCP. Gracias a estos dos mecanismos, el enlace inalámbrico se hace más confiable en ambas direcciones, sin que el origen lo sepa y sin cambiar la semántica del TCP.

Si bien el UDP no tiene los mismos problemas que el TCP, la comunicación inalámbrica también le produce dificultades. El problema principal es que los programas usan el UDP pensando que es altamente confiable; saben que no hay garantías, pero aun así esperan que sea casi perfecto. Un entorno inalámbrico estará muy lejos de serlo. En aquellos programas capaces de recuperarse de la pérdida de mensajes UDP, pasar repentinamente de un entorno en el que pueden perderse mensajes, pero rara vez ocurre, a uno en el que se pierden constantemente, puede dar pie a un desempeño desastroso.

La comunicación inalámbrica también afecta otras áreas, no sólo el desempeño. Por ejemplo, ¿cómo encuentra un *host* móvil una impresora local a la cual conectarse, en lugar de usar su impresora base? Algo parecido a esto es cómo acceder a la página del WWW de la célula local, aun si no se conoce su nombre. También, los diseñadores de páginas WWW tienden a suponer que hay mucho ancho de banda disponible. Un logotipo grande en cada página se vuelve contraproducente si su transmisión tarda 30 seg a 9600 bps cada vez que se hace referencia a la página, irritando sobremanera a los usuarios.

6.5. LOS PROTOCOLOS DE CAPA ATM AAL

En realidad no está claro si ATM tiene o no una capa de transporte. Por una parte, la capa ATM tiene la funcionalidad de una capa de red, y hay otra capa por encima de ella (AAL), lo que de alguna manera hace que AAL sea una capa de transporte. Algunos expertos están de acuerdo con este punto de vista (por ejemplo, De Prycker, 1993, página 112). Uno de los protocolos usados aquí (el AAL 5) es funcionalmente parecido al UDP, que sin duda es un protocolo de transporte.

Por otra parte, ninguno de los protocolos AAL proporciona una conexión confiable de extremo a extremo, como lo hace el TCP (aunque podrían con ciertos cambios menores). También, en la mayoría de las aplicaciones se usa otra capa de transporte encima de la AAL. En lugar de iniciar un debate, estudiaremos la capa AAL y sus protocolos en este capítulo sin sostener que es una verdadera capa de transporte.

La capa AAL de las redes ATM es radicalmente distinta del TCP, principalmente porque a los diseñadores les interesó principalmente transmitir corrientes de voz y vídeo, y aquí la entrega rápida es mucho más importante que la entrega correcta. Recuerde que la capa ATM simplemente envía células de 53 bytes en sucesión. No hay control de errores, no hay control de flujo y no hay ningún otro control. En consecuencia, no satisface los requisitos de la mayoría de las aplicaciones.

Para subsanar este problema, en la recomendación I.363 la ITU definió una capa de extremo a extremo por encima de la capa ATM. Esta capa, llamada AAL (*ATM Adaptation Layer*, **capa de adaptación de ATM**) tiene una historia tortuosa, llena de errores, correcciones y tareas inconclusas. En las siguientes secciones examinaremos esta capa y su diseño.

La meta de la AAL es proporcionar servicios útiles a programas de aplicación y protegerlos de la mecánica de dividir datos en células en el origen y reorganizarlos en el destino. Cuando la ITU comenzó a definir la AAL, se dio cuenta de que aplicaciones diferentes tienen requisitos distintos, por lo que organizó el espacio de servicio en tres ejes:

1. Servicio en tiempo real o servicio en tiempo no real.
2. Servicio de tasa de bits constante o servicio de tasa de bits variable.
3. Servicio orientado a conexiones o servicio sin conexiones.

En principio, con tres ejes y dos valores en cada eje, pueden definirse ocho servicios distintos, como se muestra en la figura 6-36. La ITU sintió que sólo cuatro de éstos eran de alguna utilidad, y los nombró clases A, B, C y D, como se indica. Los otros no se contemplaron. A partir del ATM 4.0, la figura 6-36 resulta un tanto obsoleta, por lo que se ha presentado aquí más bien como

antecedente para ayudar a entender la razón por la que los protocolos AAL se diseñaron de esta manera. En lugar de estas clases de servicio, la diferencia principal ahora es entre las clases de tráfico que estudiamos en el capítulo 5 (ABR, CBR, NRT-VBR, RT-VBR y UBR).

	A		B		C		D	
Sincronización	Tiempo real	Ninguna	Tiempo real	Ninguna	Tiempo real	Ninguna	Tiempo real	Ninguna
Tasa de bits	Constante		Variable		Constante		Variable	
Modo	Orientado a conexiones				Sin conexiones			

Figura 6-36. Clases originales de servicio reconocidas por la AAL (ahora obsoletas).

Para manejar estas cuatro clases de servicio, la ITU definió cuatro protocolos, AAL 1 a AAL 4. Sin embargo, luego se descubrió que los requisitos técnicos de las clases C y D eran tan parecidos que se combinaron el AAL 3 y el AAL 4 para formr el AAL 3/4. Entonces la industria de la computación, que estuvo dormida durante el cambio, se dio cuenta de que ninguna de esas clases era útil, y resolvió el problema definiendo otro protocolo, el AAL 5. Estudiaremos todos éstos un poco más adelante. También veremos un interesante protocolo de control usado en los sistemas ATM.

6.5.1. Estructura de la capa de adaptación de ATM

La capa de adaptación de ATM se divide en dos partes principales, una de las cuales se subdivide aún más, como se muestra en la figura 6-37.

La parte superior de la capa de adaptación de ATM se llama subcapa de convergencia. Su tarea es establecer la interfaz con la aplicación. Esta subcapa consiste en una subparte común a todas las aplicaciones (para un protocolo AAL dado) y una subparte específica para cada aplicación. Las funciones de cada una de estas partes dependen del protocolo pero pueden incluir enmarcado de mensajes y detección de errores.

Además, en el origen, la subcapa de convergencia se encarga de aceptar corrientes de bits o mensajes de longitud arbitraria de las aplicaciones y dividirlos en unidades de transmisión de 44 a 48 bytes. El tamaño exacto depende del protocolo, ya que algunos protocolos usan parte de la carga útil ATM de 48 bytes para sus propias cabeceras. En el destino, esta subcapa recompone los mensajes. En general, se conservan, cuando están presentes, los límites de los mensajes. En otras palabras, si el origen envía cuatro mensajes de 512 bytes, llegarán como cuatro mensajes de 512 bytes, no uno de 2048 bytes. En las corrientes de datos no existen límites, por lo que no se preservan.

La parte más baja de la AAL se llama subcapa SAR (*Segmentation And Reassembly*, **segmentación y recomposición**). Esta subcapa puede agregar cabeceras y apéndices a las unidades

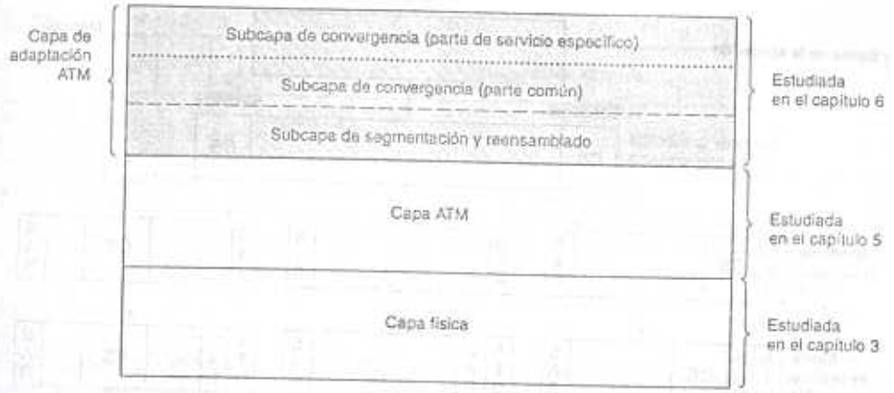


Figura 6-37. Modelo ATM que muestra la capa de adaptación ATM y sus subcapas.

de datos entregadas a ella por la subcapa de convergencia para formar cargas útiles de célula. Estas cargas se entregan a la capa ATM para su transmisión. En el destino, la subcapa SAR reensambla las células para obtener mensajes. La subcapa SAR básicamente se ocupa de las células y la subcapa de convergencia se ocupa de los mensajes.

El funcionamiento general de las subcapas de convergencia y SAR se muestra en la figura 6-38. Al llegar un mensaje al AAL desde la aplicación, la subcapa de convergencia puede darle una cabecera y apéndice a cada parte y pasarla a la capa ATM para su transmisión como célula independiente. Nótese que en la figura se muestra el caso más general porque algunos de los protocolos AAL tienen cabeceras y/o apéndice nulos.

La subcapa SAR también tiene algunas funciones adicionales para algunas clases de servicio (pero no todas). En particular, a veces maneja la detección de errores y la multiplexión. La subcapa SAR está presente en todas las clases de servicio, aunque el trabajo que realiza depende del protocolo específico.

La comunicación entre la aplicación y la subcapa AAL usa las primitivas *request* e *indication* OSI estándar que estudiamos en el capítulo 1. La comunicación entre las subcapas usa primitivas distintos.

6.5.2. AAL 1

AAL 1 es el protocolo usado para transmitir tráfico clase A, es decir, tráfico orientado a conexiones de tiempo real y con tasa de bits constante, como audio o vídeo sin compresión. Los bits son alimentados por la aplicación a una velocidad constante y deben entregarse en el otro lado a la misma velocidad constante, con retardo, fluctuación y carga extra mínimos. La entrada es una corriente de bits, sin límites de mensaje. Para este tráfico no se usan los protocolos de detección de errores como el de parada y espera porque los retardos que generan las terminaciones de temporización y las retransmisiones no son aceptables. Sin embargo, las células faltantes se

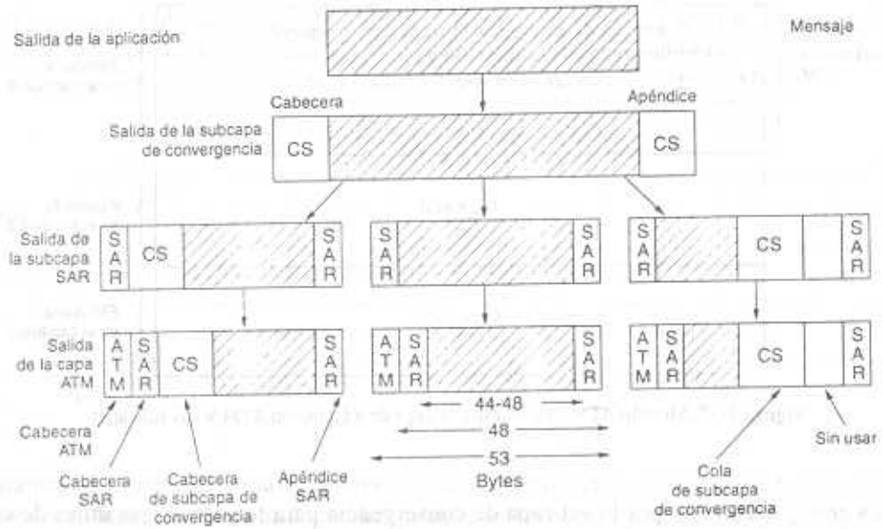


Figura 6-38. Las cabeceras y apéndices que pueden agregarse a un mensaje en una red ATM.

informan a la aplicación, que entonces, si lo desea, puede tomar sus propias medidas para recuperarlas.

AAL 1 tiene una subcapa de convergencia y una subcapa SAR. La primera detecta células perdidas y mal introducidas. (Una célula mal introducida es aquella que se entrega al destino equivocado como resultado de un error no detectado en el identificador de su circuito virtual o su trayectoria virtual.) Esta subcapa también amortigua el tráfico de entrada para proporcionar entrega de células a una tasa constante. Por último, la subcapa de convergencia divide los mensajes o la corriente de entrada en unidades de 46 o 47 bytes que se entregan a la subcapa SAR para su transmisión. En el otro extremo se extraen estas unidades y se reconstruye la entrada original. La subcapa de convergencia de AAL 1 no tiene ninguna cabecera de protocolo propia.

En contraste, la subcapa SAR de AAL 1 sí tiene un protocolo. Los formatos de sus células se muestran en la figura 6-39. Ambos formatos comienzan con una cabecera de 1 byte que contiene un número de secuencia de célula de 3 bits, *SN*, para detectar células perdidas o mal introducidas. A este campo le sigue un número de protección de secuencia (es decir, suma de comprobación) de 3 bits, el *SNP*, basado en el número de secuencia, para permitir la corrección de errores individuales y la detección de errores dobles en el campo de secuencia. Esta suma usa una comprobación de redundancia cíclica con el polinomio $x^3 + x + 1$. Un bit de paridad par que cubre el byte de cabecera reduce aún más la posibilidad de un número de secuencia equivocado. Las células AAL 1 no necesitan llevar 47 bytes. Por ejemplo, para transmitir voz digitalizada que llega a razón de 1 byte cada 125 µseg, el llenado de una célula con 47 bytes implicaría la recolección de muestras durante 5.875 mseg. Si este retardo antes de la transmisión es inacepta-

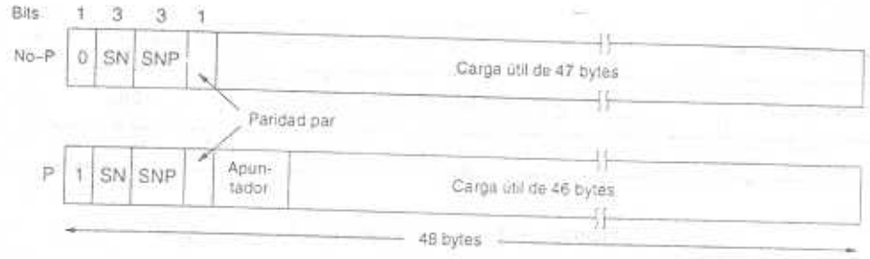


Figura 6-39. Formato de célula del AAL 1.

ble, pueden enviarse células parciales. En este caso, el número de bytes de datos reales por célula es igual para todas las células, y se acuerda por adelantado.

Las células *P* se usan cuando deben preservarse los límites de los mensajes. El campo de *apuntador* sirve para indicar el desfasamiento del comienzo del siguiente mensaje. Sólo las células con un número de secuencia par pueden ser células *P*, por lo que el apuntador está en el intervalo de 0 a 92, para que apunte dentro de la carga útil de su propia célula o de la que sigue. Nótese que este esquema permite que los mensajes tengan una cantidad arbitraria de bytes, por lo que pueden enviarse mensajes continuamente y no necesitan alinearse con los límites de la célula.

El bit de orden mayor del campo *apuntador* se reserva para uso futuro. El bit inicial de cabecera de todas las células de número impar forma una corriente de datos usada para la sincronización del reloj.

6.5.3. AAL 2

El AAL 1 se diseñó para corrientes de datos sencillas en tiempo real, orientadas a conexiones, y sin detección de errores, excepto por células faltantes o mal introducidas. Para el audio o vídeo puro sin compresión, o cualquier otra corriente de datos en la que unos cuantos bits alterados de vez en cuando no representan un problema, es adecuado el AAL 1.

En el audio o vídeo comprimido la tasa puede variar considerablemente con el tiempo. Por ejemplo, muchos esquemas de compresión transmiten un marco completo de vídeo y luego envían durante varios marcos sólo las diferencias entre los marcos subsiguientes y el último marco completo. Cuando la cámara es estacionaria y nada se mueve, los marcos de diferencia son pequeños, pero cuando la cámara está haciendo una panorámica rápida, son grandes. Además, los límites de los mensajes deben conservarse para que pueda reconocerse el comienzo del siguiente marco completo, aún en presencia de células perdidas o datos erróneos. Por estas razones se requiere un protocolo más complejo. Se diseñó para estos fines el AAL 2.

Como en AAL 1, la subcapa CS no tiene un protocolo, pero la subcapa SAR sí. El formato de célula SAR se muestra en la figura 6-40; tiene una cabecera de 1 byte y un apéndice de 2 bytes, dejando espacio para 45 bytes de datos por célula.

El campo *SN* (*Sequence Number*, número de secuencia) sirve para numerar las células en orden a fin de detectar células faltantes o mal introducidas. El campo *IT* (*Information Type*, tipo

Figura 6-40. Formato de célula AAL 2.

de información) indica que la célula es el comienzo, la mitad o el fin de un mensaje. El campo *LI* (*Length Indicator*, indicador de longitud) indica el tamaño de la carga en bytes (puede ser menor que 45 bytes). Por último, el campo *CRC* es una suma de comprobación de la célula completa que permite detectar errores.

Aunque puede parecer extraño, los tamaños de campo no se incluyen en el estándar. De acuerdo con un enterado, ya al final del proceso de estandarización el comité se dio cuenta de que el AAL 2 tenía tantos problemas que no debería usarse. Desafortunadamente, era demasiado tarde para detener el proceso de estandarización, y había una fecha que cumplir. En un último esfuerzo, el comité quitó todos los tamaños de campo para que pudiera publicarse el estándar formal a tiempo, pero de manera tal que nadie pudiera usarlo. Así es la vida en el mundo de los estándares.

6.5.4. AAL 3/4

Originalmente, la ITU tenía protocolos diferentes para las clases C y D, servicio orientado a conexiones y servicio sin conexiones para transporte de datos sensibles a pérdidas y errores, pero no dependientes del tiempo. Luego la ITU descubrió que no había necesidad real de dos protocolos, por lo que los combinó en uno solo, el AAL 3/4.

El AAL 3/4 puede operar de dos maneras: corriente o mensajes. En el modo de mensajes, cada llamada de la aplicación al AAL 3/4 inyecta un mensaje en la red. El mensaje se entrega como tal, es decir, se conservan los límites del mensaje. En el modo de corrientes no se conservan los límites. El análisis siguiente se concentrará en el modo de mensajes. En cada modo hay disponibles transporte confiable y no confiable (es decir, sin garantía).

Una característica del AAL 3/4 no presente en los otros protocolos es la multiplexión. Este aspecto del AAL 3/4 permite que viajen por el mismo circuito virtual múltiples sesiones (por ejemplo, sesiones interactivas remotas) de un solo *host*, y que se separen en el destino, como se ilustra en la figura 6-41.

La razón por la que es deseable esta facultad es que las portadoras con frecuencia cobran por cada establecimiento de conexión y por cada segundo que permanece abierta la conexión. Si un par de *hosts* tiene abiertas varias sesiones simultáneas, dar a cada una su propio circuito virtual será más caro que la multiplexión de todas ellas en el mismo circuito virtual. Si un circuito virtual tiene suficiente ancho de banda para manejar todo, no hay necesidad de más de uno.

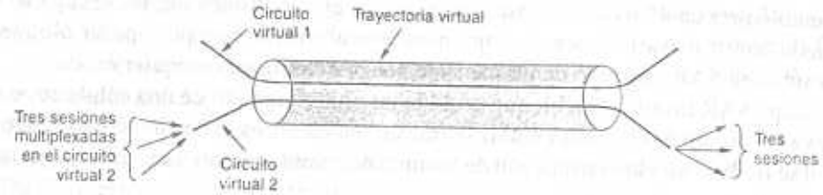


Figura 6-41. Multiplexión de varias sesiones en un circuito virtual.

Todas las sesiones que usan un solo circuito virtual reciben la misma calidad de servicio, ya que ésta se negocia para cada circuito virtual.

Esta cuestión es la verdadera razón por la que originalmente había formatos AAL 3 y AAL 4 separados: los estadounidenses querían multiplexión y los europeos no, así que cada grupo creó su propio estándar. Después, los europeos decidieron que ahorrar 10 bits de cabecera no valían el precio de que Estados Unidos y Europa no se pudieran comunicar. Por el mismo costo, se habrían podido empecinar en sus puntos de vista y tendríamos cuatro estándares AAL incompatibles (de los cuales uno está incompleto) en lugar de tres.

A diferencia del AAL 1 y el AAL 2, el AAL 3/4 tiene tanto un protocolo de subcapa de convergencia como uno de subcapa SAR. Los mensajes de hasta 65,535 bytes entran en la subcapa de convergencia desde la aplicación; primero se rellenan a un múltiplo de 4 bytes y luego se les agrega una cabecera y un apéndice, como se muestra en la figura 6-42.

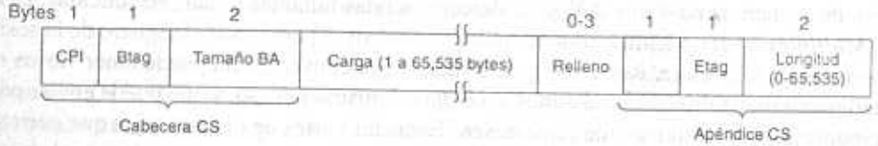


Figura 6-42. Formato de mensaje de subcapa de convergencia del AAL 3/4.

El campo *CPI* (*Common Part Indicator*, indicador de parte común) indica el tipo de mensaje y la unidad de conteo de los campos *tamaño BA* y *longitud*. Los campos *Btag* y *Etag* sirven para enmarcar los mensajes. Los dos bytes deben ser iguales y se incrementan en uno en cada mensaje nuevo enviado. Este mecanismo verifica si hay pérdida o mala introducción de células. El campo *tamaño BA* se usa para asignación de *buffers*; indica al receptor la cantidad de espacio de *buffer* a asignar para el mensaje antes de su llegada. El campo de *longitud* da nuevamente la longitud de la carga útil; en el modo de mensaje, debe ser igual al *tamaño BA*, pero en el modo de corriente puede ser diferente. El apéndice también contiene un byte no usado.

Una vez que la subcapa de convergencia ha construido el mensaje y le ha agregado una cabecera y una cola, como se muestra en la figura 6-42, pasa el mensaje a la subcapa SAR, que

lo divide en bloques de 44 bytes. Nótese que, para manejar multiplexión, la subcapa de convergencia puede construir varios mensajes internamente al mismo tiempo y pasar bloques de 44 bytes a la subcapa SAR, primero de un mensaje, luego de otro, en cualquier orden.

La subcapa SAR inserta cada bloque de 44 bytes en la carga útil de una célula cuyo formato se muestra en la figura 6-43. Estas células se transmiten entonces al destino para su reensamble, tras lo cual se lleva a cabo la verificación de la suma de comprobación y se realiza alguna acción, de ser necesario.

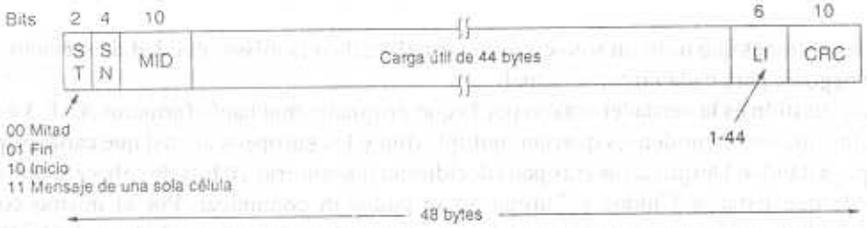


Figura 6-43. Formato de célula AAL 3/4.

Los campos de la célula AAL 3/4 son los siguientes. El campo *ST* (*Segment Type*, tipo de segmento) sirve para enmarcar los mensajes; indica si la célula inicia un mensaje, está a la mitad de él, es la última célula o si es un mensaje pequeño (es decir, de una sola célula). Sigue un número de secuencia de 4 bits, *SN*, para detectar células faltantes o mal introducidas. El campo *MID* (*Multiplexing ID*, identificador de multiplexión) sirve para llevar el registro de la sesión a la que pertenece cada célula. Recuerde que la subcapa de convergencia puede tener varios mensajes, pertenecientes a diferentes sesiones, en *buffer* al mismo tiempo, y que puede enviar partes de estos mensajes en cualquier orden que desee. Todas las partes de los mensajes que pertenecen a la sesión *i* llevan *i* en el campo *MID*, para poderlos reagrupar correctamente en el destino. El apéndice contiene la longitud de la carga útil y la suma de comprobación de la célula.

Nótese que el AAL 3/4 tiene carga extra de dos capas de protocolo: 8 bytes que se agregan a cada mensaje y 4 bytes que se agregan a cada célula. En suma, es un mecanismo de peso pesado, especialmente para los mensajes cortos.

6.5.5. AAL 5

Los protocolos AAL 1 a AAL 3/4 se diseñaron en gran medida para la industria de las telecomunicaciones y fueron estandarizados por el ITU sin mucha realimentación de la industria de la computación. Cuando la industria de la computación finalmente despertó y comenzó a entender las implicaciones de la figura 6-43, surgió una especie de pánico. La complejidad y la ineficiencia generadas por dos capas de protocolo, aunadas a la suma de comprobación extrañamente corta (sólo 10 bits), hizo que algunos investigadores inventaran un protocolo de adaptación nuevo. Lo

llamaron **SEAL** (*Simple Efficient Adaptation Layer*, **capa de adaptación sencilla y eficiente**), lo que sugiere qué pensaban estos diseñadores de los anteriores. Tras cierto debate, el foro ATM aceptó el SEAL y le asignaron el nombre AAL 5. Para mayor información sobre el AAL 5 y sus diferencias respecto al AAL 3/4, véase (Suzuki, 1994).

El AAL 5 ofrece varios tipos de servicio a sus aplicaciones. Una posibilidad es el servicio confiable (es decir, entrega garantizada con control de flujo para evitar desbordamientos). Otra posibilidad es el servicio no confiable (es decir, sin entrega garantizada), con opción de descartar o pasar a la aplicación (reportando como malas) las células con errores de suma de comprobación. Se apoyan tanto la unitransmisión como la multitransmisión, pero la multitransmisión no ofrece entrega garantizada.

Al igual que el AAL 3/4, el AAL 5 reconoce tanto el modo de mensajes como el modo de corriente. En el modo de mensajes, una aplicación puede pasar un datagrama de 1 a 65,535 bytes de longitud a la capa AAL para la entrega a su destino, ya sea con garantía o con el mejor esfuerzo. A su llegada a la subcapa de convergencia, se rellena un mensaje y se le agrega un apéndice, como se muestra en la figura 6-44. La cantidad de relleno (0 a 47 bytes) se selecciona para hacer que el mensaje completo, incluido el relleno y el apéndice, sea un múltiplo de 48 bytes. El AAL 5 no tiene una cabecera de subcapa de convergencia, sólo un apéndice de 8 bytes.

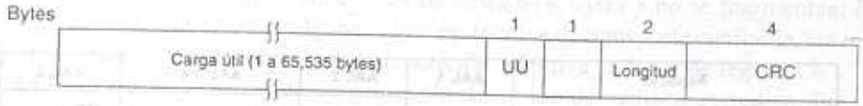


Figura 6-44. Formato de mensaje de subcapa de convergencia del AAL 5.

El campo *UU* (*User to User*, usuario a usuario) no lo usa la capa AAL misma, una capa superior puede usarlo para sus propios fines; por ejemplo, números de secuencia o multiplexión. La capa superior en cuestión puede ser la subparte específica para servicio de la subcapa de convergencia. El campo de *longitud* indica el tamaño de la carga útil real, en bytes, sin contar el relleno. Se usa un valor de 0 para abortar el mensaje actual a medio camino. El campo *CRC* es la suma de comprobación estándar de 32 bits del mensaje completo, incluido el relleno y el apéndice (con el campo *CRC* en 0). Se reserva un campo de 8 bits del apéndice para uso futuro.

El mensaje se transmite pasándolo a la subcapa SAR, que no agrega cabeceras ni apéndices; en cambio, divide el mensaje en unidades de 48 bytes y pasa cada una de ellas a la capa ATM para su transmisión. También le indica a la capa ATM que establezca un bit en el campo *PTI* de la última célula, de manera que se conserven los límites del mensaje. Puede argumentarse que ésta es una mezcla incorrecta de capas de protocolo, pues la capa AAL no debería estar usando bits de la cabecera de la capa ATM. Hacerlo viola el principio más básico de la ingeniería de protocolos y sugiere que las capas debieron diseñarse de manera diferente.

La ventaja principal del AAL 5 sobre el AAL 3/4 es una eficiencia mucho mayor. Si bien el AAL 3/4 agrega sólo 4 bytes por mensaje, también agrega 4 bytes por célula, reduciendo la

capacidad de carga a 44 bytes, una pérdida del 8% en mensajes grandes. El AAL 5 tiene un apéndice ligeramente más grande por mensaje (8 bytes), pero no tiene carga extra en cada célula. La falta de números de secuencia en las células se compensa con la suma de comprobación más grande, que puede detectar células perdidas, mal introducidas o faltantes sin necesidad de números de secuencia.

En la comunidad Internet, se espera que la manera normal de comunicarse con las redes ATM será el transporte de paquetes IP en el campo de carga útil del AAL 5. En el RFC 1483 y el RFC 1577 se estudian varios temas relacionados con este enfoque.

6.5.6. Comparación de los protocolos AAL

El lector tiene aquí derecho a pensar que los diferentes protocolos AAL parecen mal pensados e innecesariamente semejantes entre ellos. También es cuestionable el valor de tener subcapas de convergencia y SAR separadas, sobre todo porque el AAL 5 no tiene nada en la subcapa SAR. Una cabecera ATM ligeramente mejorada podría haber proporcionado secuencia, multiplexión y enmarcado más que adecuados.

Algunas de las diferencias entre los diferentes protocolos AAL se resumen en la figura 6-45. Éstas se relacionan con la eficiencia, el manejo de errores, la multiplexión y la relación entre las subcapas AAL.

Aspecto	AAL 1	AAL 2	AAL 3/4	AAL 5
Clase de servicio	A	B	C/D	C/D
Multiplexión	No	No	Sí	No
Delimitación de mensaje	Ninguna	Ninguna	Btag/Etag	Bit en PTI
Asignación de *buffers* por adelantado	No	No	Sí	No
Bytes de usuario disponibles	0	0	0	1
Relleno CS	0	0	Palabra de 32 bits	0-47 byts
Carga extra del protocolo CS (bytes)	0	0	8	8
Suma de comprobación CS	Ninguna	Ninguna	Ninguna	32 bits
Bytes de carga útil SAR	46-47	45	44	48
Carga extra de protocolo SAR (bytes)	1-2	3	4	0
Suma de comprobación SAR	Ninguna	Ninguna	10 bits	Ninguna

Figura 6-45. Algunas diferencias entre los diferentes protocolos AAL.

La impresión general que da el AAL es la de demasiadas variantes con demasiadas diferencias mínimas y un trabajo sin terminar. Las cuatro clases de servicio originales, A, B, C y D, de hecho han sido abandonadas. El AAL 1 probablemente no es necesario; el AAL 2 está incompleto; el AAL 3 y el AAL 4 nunca vieron la luz del día y el AAL 3/4 es ineficiente y tiene una suma de comprobación demasiado corta.

El futuro es del AAL 5, pero aun ahí hay lugar para mejoras. Los mensajes AAL 5 debieron tener un número de secuencia y un bit para distinguir los mensajes de datos y de control, para que pudiera usarse como protocolo de transporte confiable. Inclusive había espacio desocupado para ellos en el apéndice. Tal como está, para un transporte confiable se requiere además la carga adicional de una capa de transporte, cuando pudo haberse evitado. Si el comité AAL hubiera presentado su trabajo como proyecto escolar, el profesor probablemente se los habría devuelto con instrucciones de corregirlo y presentarlo nuevamente cuando lo concluyesen. Pueden encontrarse más críticas al ATM en (Sternbenz *et al.*, 1995).

6.5.7. SSCOP — protocolo de servicio específico orientado a conexiones

A pesar de todos estos diferentes protocolos AAL, ninguno de ellos proporciona conexiones de transporte confiables y sencillas punto a punto. En las aplicaciones en las que se requieren, existe otro protocolo AAL: el SSCOP (*Service Specific Connection Oriented Protocol*, **protocolo de servicio específico orientado a conexiones**). Sin embargo, el SSCOP sólo se usa para control, no para transmisión de datos.

Los usuarios del SSCOP envían mensajes, a cada uno de los cuales se asigna un número de secuencia de 24 bits. Los mensajes pueden ser de hasta 64K bytes y no se fragmentan; deben entregarse en orden. A diferencia de algunos otros protocolos de transporte confiable, los mensajes faltantes siempre se retransmiten usando repetición selectiva en lugar de regresar n.

El SSCOP es fundamentalmente un protocolo dinámico de ventana corrediza. Para cada conexión, el receptor mantiene una ventana de números de secuencia de mensaje que puede recibir, y un mapa de bits en el que marca los que ya tiene. Esta ventana puede cambiar de tamaño durante la operación del protocolo.

Lo que hace poco común al SSCOP es el manejo de los acuses de recibo: no hay incorporación. En su lugar, el transmisor sondea periódicamente al receptor para solicitarle que le envíe el mapa de bits que contiene el estado de la ventana. Con base en el resultado, el transmisor descarta los mensajes que han sido aceptados y actualiza la ventana. El SSCOP se describe con detalle en (Henderson, 1995).

6.6. DESEMPEÑO

Los asuntos relacionados con el desempeño son muy importantes en las redes de cómputo. Cuando hay cientos o miles de computadoras conectadas entre sí, son comunes las interacciones complejas, con consecuencias imprevisibles. Frecuentemente esta complejidad conduce a un desempeño pobre, sin que nadie sepa por qué. En las siguientes secciones examinaremos muchos temas relacionados con el desempeño de las redes para ver los tipos de problemas que existen y lo que se puede hacer para resolverlos.

Desafortunadamente, el entendimiento del desempeño de las redes es más un arte que una ciencia. Muy poca de la teoría tiene en realidad alguna utilidad en la práctica. Lo mejor que

podemos hacer es dar reglas empíricas derivadas de los tropiezos y ejemplos actuales tomados del mundo real. Intencionalmente hemos postergado este análisis hasta después de estudiar la capa de transporte en las redes TCP y ATM, a fin de poder señalar los lugares en los que se han hecho bien o mal las cosas.

La capa de transporte no es el único lugar en el que surgen asuntos relacionados con el desempeño. Vimos algunos de ellos en la capa de red, en el capítulo anterior. No obstante, la capa de red tiende a ocuparse principalmente del enrutamiento y el control de congestionamientos. Los puntos más amplios, orientados al sistema, tienden a relacionarse con el transporte, por lo que este capítulo es un lugar adecuado para examinarlos.

En las siguientes cinco secciones estudiaremos cinco aspectos del desempeño de las redes:

1. Problemas de desempeño.
2. Medición del desempeño de una red.
3. Diseño de sistemas con mejor desempeño.
4. Procesamiento de TPDU rápido.
5. Protocolos para redes futuras de alto desempeño.

Como comentario, necesitamos un nombre para las unidades intercambiadas por las unidades de transporte. El término de TCP, segmento, es confuso en el mejor de los casos, y nunca se usa fuera del mundo TCP en este contexto. Los términos correctos de ATM, CS-PDU, SAR-PDU y CPCS-PDU, son específicos de ATM. Los paquetes claramente se refieren a la capa de red y los mensajes pertenecen en la capa de aplicación. A falta de un término estándar, volveremos a llamar TPDU a las unidades intercambiadas por las entidades de transporte. Cuando queramos referirnos tanto a TPDU como a paquetes, usaremos paquete como término colectivo, como en "la CPU debe ser lo bastante rápida como para procesar los paquetes de entrada en tiempo real". Con esto nos referimos tanto al paquete de capa de red como a la TPDU encapsulada en él.

6.6.1. Problemas de desempeño en las redes de cómputo

Algunos problemas de desempeño, como el congestionamiento, son causados por sobrecargas temporales de los recursos. Si repentinamente llega más tráfico a un enrutador que el que puede manejar, se creará un congestionamiento y el desempeño bajará. Ya estudiamos el congestionamiento en detalle en el capítulo anterior.

El desempeño también se degrada cuando hay un desequilibrio estructural de los recursos. Por ejemplo, si una línea de comunicación de gigabits está conectada a una PC de bajo rendimiento, la pobre CPU no será capaz de procesar los paquetes de entrada a la velocidad suficiente, y se perderán algunos. Estos paquetes se retransmitirán tarde o temprano, agregando un retardo, desperdiciando ancho de banda y reduciendo en general el desempeño.

Las sobrecargas también pueden generarse sincrónicamente. Por ejemplo, si una TPDU contiene un parámetro erróneo (por ejemplo, el puerto o proceso al que está destinado), en muchos casos el receptor cortésmente enviará una notificación de error. Ahora considere lo que podría ocurrir si se difundiera una TPDU errónea a 10,000 máquinas: cada una podría devolver un mensaje de error. La **tormenta de difusión** resultante podría paralizar la red. El UDP adoleció de este problema hasta que se cambió el protocolo para hacer que los *hosts* evitaran responder a errores en las TPDU de UDP enviadas a direcciones de difusión.

Un segundo ejemplo de carga extra síncrona es lo que ocurre tras una falla del suministro eléctrico. Al regresar la energía, todas las máquinas saltan simultáneamente a sus ROM para reiniciarse. Una secuencia de rearranque podría requerir acudir primero a algún servidor (RARP) para conocer la identidad verdadera de la máquina, y luego a un servidor de archivos para obtener una copia del sistema operativo. Si cientos de máquinas hacen todo esto al mismo tiempo, el servidor probablemente se vendría abajo por la carga.

Aun en ausencia de sobrecargas síncronas y con suficientes recursos disponibles, puede haber un bajo desempeño debido a la falta de afinación del sistema. Por ejemplo, si una máquina tiene bastante potencia y memoria de CPU, pero no se ha asignado suficiente memoria como espacio de *buffer*, ocurrirán desbordamientos y se perderán varias TPDU. De la misma manera, si el algoritmo de calendarización no tiene una prioridad lo bastante alta como para procesar las TPDU de entrada, podrán perderse algunas.

Otro asunto relativo a la afinación es el ajuste correcto de las temporizaciones. Cuando se envía una TPDU, normalmente se utiliza un temporizador para protegerse contra pérdidas. Si se hace muy corta la temporización, ocurrirán retransmisiones innecesarias, congestionando los alambres; si se hace demasiado larga, ocurrirán retardos innecesarios tras la pérdida de una TPDU. Otros parámetros afinables incluyen el tiempo de espera para incorporar datos a paquetes antes de enviar acuses de recibo por separado, y la cantidad de retransmisiones antes de darse por vencido.

Las redes de gigabits traen consigo nuevos problemas de desempeño. Considere, por ejemplo, el envío de datos de San Diego a Boston cuando el *buffer* del receptor es de 64K bytes. Supóngase que el enlace es de 1 Gbps y que el retardo de la luz a través de la fibra es de 20 mseg. Inicialmente, en $t = 0$, el tubo está vacío, como se muestra en la figura 6-46(a). Apenas 500 μseg después [figura 6-46(b)] todas las TPDU están en la fibra. La TPDU a la cabeza ahora estará en el vecindario de Brawley, todavía en las profundidades del sur de California. Sin embargo, el transmisor debe detenerse hasta recibir una actualización de ventana.

Después de 20 mseg, la TPDU puntera llega a Boston, como se muestra en la figura 6-46(c), y se envía acuse de recibo. Por último, 40 mseg después de comenzar, llega el primer acuse de recibo al transmisor y puede transmitirse la segunda ráfaga. Dado que la línea de transmisión se usó durante 0.5 mseg de un total de 40, la eficiencia es de aproximadamente 1.25%. Esta situación es típica de la operación de protocolos anteriores sobre líneas de gigabits.

Una cantidad que conviene recordar durante el análisis del desempeño de redes es el **producto ancho de banda-retardo** que se obtiene al multiplicar el ancho de banda (en bits/seg) por el tiempo de retardo de ida y vuelta (en seg). El producto es la capacidad del tubo desde el transmisor al receptor y de regreso (en bits).

Para el ejemplo de la figura 6-46, el producto ancho de banda-retardo es de 40 millones de bits. En otras palabras, el transmisor tendría que enviar una ráfaga de 40 millones de bits para

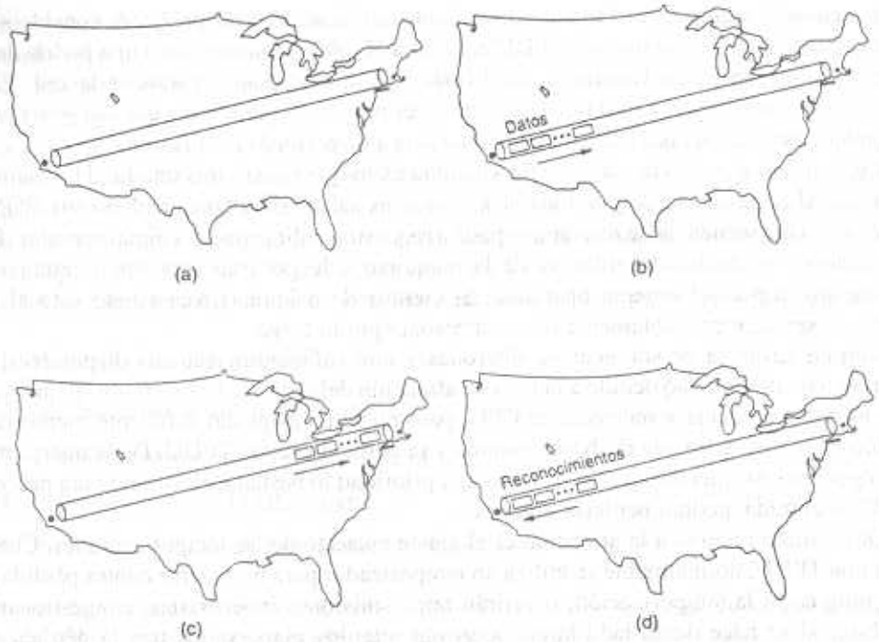


Figura 6-46. Estados de transmisión de un megabit de San Diego a Boston. (a) En $t = 0$. (b) Tras 500 µseg. (c) Tras 20 mseg. (d) Tras 40 mseg.

trabajar a toda velocidad hasta la llegada del primer acuse de recibo. Se requiere esta cantidad de bits para llenar el tubo (en ambas direcciones). Ésta es la razón por la que una ráfaga de medio millón de bits sólo logra una eficiencia del 1.25%: es sólo el 1.25% de la capacidad del tubo.

La conclusión a sacar aquí es que, para lograr un buen desempeño, la ventana del receptor debe tener cuando menos el tamaño del producto ancho de banda-retardo, y de preferencia ser un poco más grande, puesto que el receptor podría no responder instantáneamente. Para una línea transcontinental de gigabits se requieren cuando menos 5 megabytes para cada conexión.

Si la eficiencia es muy baja para el envío de un megabit, imagine lo que será al enviar unos cuantos cientos de bytes para una llamada a un procedimiento remoto. A menos que pueda encontrarse otro uso para la línea mientras el primer cliente espera su respuesta, una línea de gigabits no es mejor que una línea de megabits, sólo más cara.

Otro problema de desempeño que ocurre con las aplicaciones de tiempo crítico como audio y vídeo es la fluctuación. Tener un tiempo medio de transmisión corto no es suficiente. También se requiere una desviación estándar pequeña. El logro de un tiempo medio de transmisión corto con una desviación estándar pequeña requiere esfuerzos serios de ingeniería.

6.6.2. Medición del desempeño de las redes

Cuando una red tiene un desempeño pobre, sus usuarios frecuentemente se quejan con los operadores, exigiendo mejoras. Para mejorar el desempeño, los operadores deben primero determinar exactamente lo que ocurre. Para saberlo, los operadores deben efectuar mediciones. En esta sección veremos las mediciones de desempeño de las redes. El estudio siguiente se basa en el trabajo de Mogul (1993). Para un análisis más detallado del proceso de medición, véase (Jain, 1991, y Villamizan y Song, 1995).

El ciclo básico usado para mejorar el desempeño de las redes contiene los siguientes pasos:

1. Medir los parámetros pertinentes y el desempeño de la red.
2. Tratar de entender lo que ocurre.
3. Cambiar un parámetro.

Estos pasos se repiten hasta que el desempeño sea lo bastante bueno o que quede claro que se han hecho todas las mejoras posibles.

Las mediciones pueden hacerse de muchas maneras y en muchos lugares (tanto físicos como en la pila de protocolos). El tipo de medición más básico es arrancar un temporizador al iniciar una actividad y medir el tiempo que tarda la actividad. Por ejemplo, saber el tiempo que toma el acuse de recibo de una TPDU es una medición clave. Otras mediciones se hacen con contadores que registran la frecuencia con que ocurre un evento (por ejemplo, cantidad de TPDU perdidas). Por último, con frecuencia nos interesa saber la cantidad de algo, como el número de bytes procesados durante cierto intervalo de tiempo.

La medición del desempeño y los parámetros de una red tiene muchos escollos potenciales. A continuación describimos algunos de ellos. Cualquier intento sistemático de medir el desempeño de una red debe tener cuidado de evitarlos.

Asegúrese que el tamaño de la muestra es lo bastante grande

No mida el tiempo de envío de una TPDU, sino repita la medición, digamos, un millón de veces y obtenga el promedio. Una muestra grande reducirá la incertidumbre de la media y la desviación estándar medidas. Esta incertidumbre puede calcularse usando fórmulas estadísticas estándar.

Asegúrese de que las muestras son representativas

Idealmente, la secuencia total de un millón de mediciones debería repetirse a horas del día y de la semana diferentes para ver el efecto de diferentes cargas del sistema sobre la cantidad medida. Por ejemplo, las mediciones de congestionamientos sirven de poco si se toman en un momento en el que no hay congestionamientos. A veces los resultados pueden ser contraintuitivos inicial-

mente, como la presencia de congestionamiento intenso a las 10, 11, 13 y 14 horas, pero sin congestionamiento al mediodía (cuando todos los usuarios están en el refrigerio).

Tenga cuidado al usar relojes de intervalos grandes

Los relojes de computadora funcionan sumando uno a un contador a intervalos regulares. Por ejemplo, un temporizador de milisegundos suma uno al contador cada 1 mseg. El uso de tal temporizador para medir un evento que tarda menos de 1 mseg no es imposible, pero requiere cuidado.

Por ejemplo, para medir el tiempo de envío de una TPDU, el reloj del sistema (digamos, en milisegundos) debe leerse al entrar en el código de capa de transporte, y nuevamente al salir. Si el tiempo de envío real de la TPDU es de 300 μseg, la diferencia entre las dos lecturas será 0 o 1, ambas cifras equivocadas. Sin embargo, si la medición se repite un millón de veces y se suman todas las mediciones y se dividen entre un millón, el tiempo medio tendrá una exactitud del orden de menos de 1 μseg.

Asegúrese de que no ocurre nada inesperado durante sus pruebas

Realizar mediciones en un sistema universitario el día en que tiene que entregarse un importante proyecto de laboratorio puede dar resultados diferentes a los que se podrían obtener el día siguiente. Del mismo modo, si un investigador ha decidido difundir una vídeoconferencia por la red mientras usted hace sus pruebas, sus resultados podrían alterarse. Es mejor ejecutar las pruebas en un sistema inactivo y crear la carga completa usted mismo, pero aun este enfoque tiene algunos escollos. Usted podría pensar que nadie usará la red a las 3 A.M., pero esa podría ser precisamente la hora en la que el programa automático de respaldos comienza a copiar todos los discos a vídeocinta. Es más, puede haber un tráfico pesado hacia sus fantásticas páginas del *World Wide Web* desde husos horarios distantes.

El caché puede arruinar las mediciones

Si quiere medir los tiempos de transferencia de archivos, la manera obvia de hacerlo es abrir un archivo grande, leerlo todo, cerrarlo y ver el tiempo que tarda. Luego se repetiría la medición muchas veces más para obtener un buen promedio. El problema es que el sistema puede manejar el archivo en caché, por lo que sólo la primera medición realmente comprende tráfico de red. Las demás son sólo lecturas del caché local. Los resultados de tales mediciones son esencialmente inservibles (a menos que se quiera medir el desempeño del caché).

Con frecuencia puede evitarse el almacenamiento en caché simplemente desbordando el caché. Por ejemplo, si el caché es de 10 MB, el ciclo de prueba podría abrir, leer y cerrar dos archivos de 10 MB en cada vuelta, en un intento por obligar a que la tasa de encuentros del caché sea de 0. Aun así, se recomienda cuidado a menos que esté completamente seguro de que entiende el algoritmo de caché.

Los *buffers* pueden tener un efecto similar. Un programa de servicio de desempeño del TCP/IP bastante común ha llegado a informar que el UDP puede lograr un desempeño sustancialmente mayor al permitido por la línea física. ¿Por qué ocurre esto? Una llamada al UDP normalmente devuelve el control tan pronto como el núcleo ha aceptado el mensaje y lo ha agregado a la cola de transmisión. Si hay suficiente espacio de *buffer*, cronometrar 1000 llamadas UDP no implica que todos los datos se han enviado. La mayoría de ellos podría estar aún en el núcleo, pero el programa de servicio de desempeño piensa que se han transmitido todos.

Entienda lo que está midiendo

Al medir el tiempo de lectura de un archivo remoto, las mediciones dependen de la red, los sistemas operativos tanto en el cliente como en el servidor, las tarjetas de interfaz de *hardware* empleadas, sus controladores y otros factores. Si se tiene cuidado, finalmente se descubrirá el tiempo de transferencia de archivos para la configuración en uso. Si la meta es afinar esa configuración en particular, tales mediciones son adecuadas.

Sin embargo, si se están haciendo mediciones similares en tres sistemas diferentes a fin de seleccionar la tarjeta de interfaz de red a adquirir, sus resultados podrían desviarse por completo por el hecho de que uno de los controladores de la red realmente está mal y solamente está aprovechando el 10% del desempeño de la tarjeta.

Tenga cuidado con la extrapolación de los resultados

Suponga que hace mediciones con cargas de red simuladas que van desde 0 (en reposo) a 0.4 (40% de la capacidad), como lo indican los puntos de datos y la línea continua que los atraviesa en la figura 6-47. Puede ser tentador extrapolar linealmente, como lo indica la línea punteada. Sin embargo, muchos resultados de encolamiento comprenden un factor de $1/(1-\rho)$, donde ρ es la carga, por lo que los valores verdaderos pueden parecerse más a la línea punteada.

6.6.3. Diseño de sistemas para un mejor desempeño

La medición y los ajustes pueden con frecuencia mejorar considerablemente el desempeño, pero no pueden sustituir un buen diseño original. Una red mal diseñada puede mejorarse sólo hasta un límite. Más allá, tiene que rehacerse desde el principio.

En esta sección presentaremos algunas reglas empíricas basadas en la experiencia con muchas redes. Estas reglas se relacionan con el diseño del sistema, no sólo con el diseño de la red, ya que el *software* y el sistema operativo con frecuencia son más importantes que los enrutadores y las tarjetas de interfaz. La mayoría de estas ideas han sido del conocimiento común de los diseñadores de redes durante años y se han pasado verbalmente de generación en generación. Mogul fue el primero en postularlas explícitamente (1993); nuestro estudio sigue principalmente la secuencia del suyo. Otra fuente apropiada es (Metcalfe, 1993).

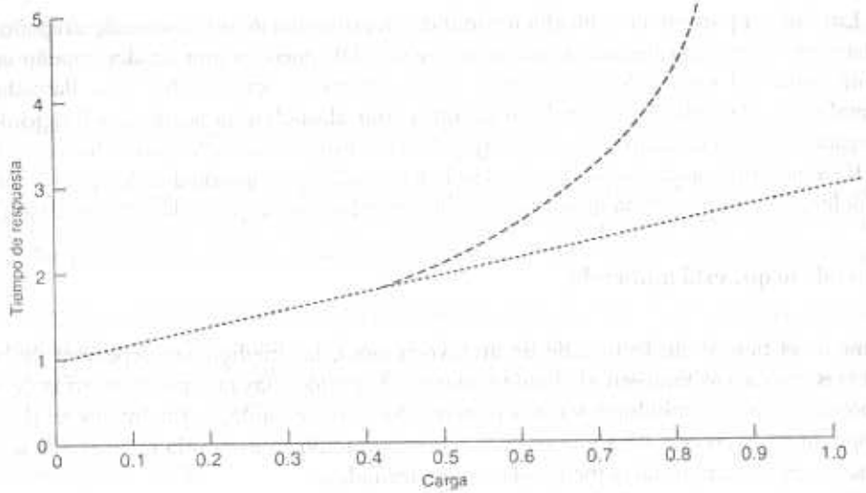


Figura 6-47. Respuesta en función de la carga.

Regla #1: La velocidad de la CPU es más importante que la velocidad de la red

Una amplia experiencia ha demostrado que en casi todas las redes la carga extra de los sistemas operativos y protocolos domina al tiempo real en el alambre. Por ejemplo, en teoría, el tiempo mínimo de RPC de un Ethernet es de 102 µseg, correspondiente a una solicitud mínima (64 bytes) seguida de una respuesta mínima (64 bytes). En la práctica, la reducción del tiempo RPC a 1500 µseg es un logro considerable (Van Renesse *et al.*, 1988). Nótese que 1500 µseg es 15 veces peor que el mínimo teórico. Casi toda la carga extra está en el *software*.

De la misma manera, el problema principal al operar a 1 Gbps es llevar los bits desde el *buffer* del usuario hasta la primera fibra a la velocidad suficiente y lograr que la CPU receptora los procese tan rápidamente como entran. En pocas palabras, si se duplica la velocidad de la CPU, con frecuencia casi se puede duplicar el rendimiento. La duplicación de la capacidad de la red en muchos casos no tiene efecto, ya que el cuello de botella generalmente está en los *hosts*.

Regla #2: Reducir el número de paquetes para reducir la carga extra de software

El procesamiento de una TPDU tiene cierta cantidad de carga extra por TPDU (por ejemplo, procesamiento de cabeceras) y cierta cantidad de procesamiento por byte (por ejemplo, procesar la suma de comprobación). Al enviar 1 millón de bytes, la carga extra por byte es la misma sin importar el tamaño de la TPDU. Sin embargo, el uso de TPDU de 128 bytes implica 32 veces más carga extra por TPDU que el uso de TPDU de 4K. Esta carga extra crece con rapidez.

Además de la carga extra de las TPDU, hay una carga extra en las capas inferiores que se debe considerar. Cada paquete que llega causa una interrupción. En un procesador RISC moderno, cada interrupción rompe la tubería de la CPU, interfiere con el caché, requiere un cambio del contexto en la administración de la memoria y obliga al almacenamiento de una cantidad de registros de CPU importante. Una reducción de n veces en las TPDU enviadas reduce la carga extra de la interrupción y de los paquetes en un factor de n.

Esta observación es un argumento a favor de la recolección de una cantidad importante de datos antes de su transmisión, a fin de reducir las interrupciones en el otro lado. El algoritmo de Nagle y la solución de Clark al síndrome de la ventana tonta (*silly window*) son intentos por lograr precisamente esto.

Regla #3: Reducir al mínimo las conmutaciones de contexto

Las conmutaciones de contexto (por ejemplo, del modo de núcleo al modo de usuario) son mortales; pueden tener los mismos inconvenientes que las interrupciones, siendo la peor una larga serie de fallas de caché iniciales. Las conmutaciones de contexto pueden reducirse haciendo que el procedimiento de biblioteca que envía los datos los guarde en un *buffer* interno hasta tener una buena cantidad de ellos. De la misma manera, en el lado receptor las TPDU de entrada pequeñas deben juntarse y pasarse al usuario como un bloque completo y no individualmente, para reducir al mínimo las conmutaciones de contexto.

En el mejor caso, un paquete de entrada causa una conmutación de contexto del usuario actual al núcleo, y luego una conmutación al proceso receptor para darle los nuevos datos. Desafortunadamente, en muchos sistemas operativos ocurren conmutaciones de contexto adicionales. Por ejemplo, si el administrador de la red ejecuta un proceso especial en el espacio de usuario, un paquete de llegada tenderá a causar una conmutación de contexto del usuario actual al núcleo, luego otra del núcleo al administrador de red, seguida de otra de regreso al núcleo y, por último, una del núcleo al proceso receptor. Esta secuencia se muestra en la figura 6-48. Todas estas conmutaciones de contexto en cada paquete desperdician mucho tiempo de CPU y tienen un efecto devastador sobre el desempeño de la red.

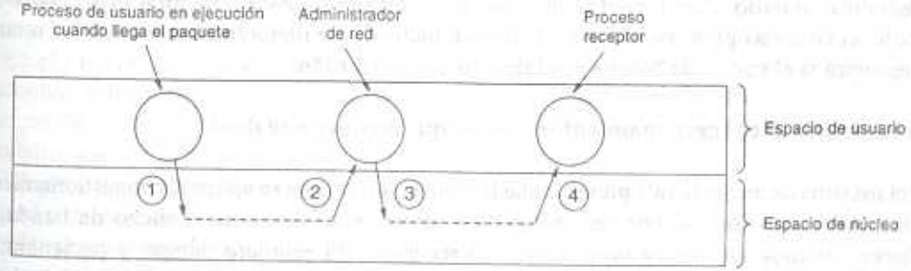


Figura 6-48. Cuatro conmutaciones de contexto para manejar un paquete con un administrador de red de espacio de usuario.

Regla #4: Reducir al mínimo las copias

Peores que las conmutaciones de contexto múltiples son las copias múltiples. No es inusitado que un paquete de entrada se copie tres o cuatro veces antes de entregarse la TPDU que contiene. Después de recibirse un paquete en la interfaz de la red en un *buffer* especial integrado a una tarjeta, es común que se copie en un *buffer* del núcleo. De ahí se copia en el *buffer* de capa de red, luego en el *buffer* de la capa de transporte y, por último, en el proceso de aplicación receptor.

Un sistema operativo ingenioso copiará una palabra a la vez, pero no es raro que se requieran unas cinco instrucciones por palabra (carga, almacenamiento, incremento de un registro de índice, prueba de fin de datos y ramificación condicional). En una máquina de 50 MIPS, tres copias de cada paquete a cinco instrucciones por palabra de 32 bits copiada requieren 75 nseg por byte de entrada. Tal máquina, por tanto, puede aceptar datos a una velocidad máxima de unos 107 Mbps. Si incluimos la carga extra del procesamiento de la cabecera, el manejo de interrupciones y las conmutaciones de contexto, podrían lograrse 50 Mbps, sin considerar el proceso real de los datos. Queda claro que el manejo de una línea de 1 Gbps está fuera de consideración.

De hecho, es probable que también una línea de 50 Mbps está fuera de consideración. En el cálculo anterior hemos supuesto que una máquina de 50 MIPS puede ejecutar 50 millones de instrucciones por segundo. En la realidad, las máquinas sólo pueden operar a tales velocidades si no hacen referencia a la memoria. Las operaciones de memoria con frecuencia son tres veces más lentas que las instrucciones registro a registro, por lo que la obtención de 16 Mbps de la línea de 1 Gbps podría considerarse muy buena. Nótese que la asistencia de *hardware* no ayuda aquí. El problema es que el sistema operativo está ejecutando demasiadas copias.

Regla #5: Puede comprarse más ancho de banda, pero no un retardo menor

Las tres reglas que siguen tienen que ver con la comunicación, más que con el proceso del protocolo. La primera regla indica que, si se desea más ancho de banda, se puede comprar. La instalación de una segunda fibra junto a la primera duplica el ancho de banda, pero no hace nada para reducir el retardo. Para que el retardo sea más corto es necesario mejorar el *software* del protocolo, el sistema operativo o la interfaz de red. Incluso si se mejoran las tres cosas, el retardo no se reducirá si el cuello de botella es el tiempo de transmisión.

Regla #6: Evitar el congestionamiento es mejor que recuperarse de él

La vieja máxima de que más vale prevenir que lamentar ciertamente se aplica al congestionamiento en redes. Al congestionarse una red, se pierden paquetes, se desperdicia ancho de banda, se introducen retardos inútiles, y otras cosas. La recuperación requiere tiempo y paciencia; es mejor que no sea necesaria. Evitar el congestionamiento es como recibir una vacuna: duele un poco en el momento, pero evita algo que sería mucho más doloroso.

Regla #7: Evitar terminaciones de temporización

Los temporizadores son necesarios en las redes, pero deben usarse con cuidado y deben reducirse al mínimo las terminaciones de temporización. Al expirar un temporizador, lo común es que se repita una acción. Si realmente es necesario repetir la acción, que así sea, pero su repetición innecesaria es un desperdicio.

La manera de evitar el trabajo extra es tener cuidado de que los intervalos de temporización sean más bien conservadores. Un temporizador que tarda demasiado en expirar agrega una pequeña cantidad de retardo extra a una conexión en el caso (improbable) de la pérdida de una TPDU. Un temporizador que termina cuando no debería consume tiempo de CPU valioso, desperdicia ancho de banda e impone una carga extra tal vez a docenas de enrutadores sin razón alguna.

6.6.4. Procesamiento rápido de las TPDU

La moraleja de la historia anterior es que el obstáculo principal en las redes rápidas es el *software* de los protocolos. En esta sección veremos algunas maneras de acelerar este *software*. Para mayor información, véase (Clark *et al.*, 1989; Edwards y Muir, 1995, y Chandranmenon y Varghese, 1995).

La carga extra de procesamiento de las TPDU tiene dos componentes: la carga extra por TPDU y la carga extra por byte. Ambas deben combatirse. La clave para el procesamiento rápido de las TPDU es separar el caso normal (transferencia normal de un solo sentido) y manejarlo como caso especial. Aunque se necesita una secuencia de TPDU especiales para entrar en el estado *ESTABLISHED*, una vez ahí el procesamiento de las TPDU es directo hasta que un lado cierra la conexión.

Comencemos por examinar el lado transmisor en el estado *ESTABLISHED* cuando hay datos por transmitir. Por claridad, supondremos que la entidad de transporte está en el núcleo, aunque los mismos conceptos se aplican si es un proceso de espacio de usuario o una biblioteca en el proceso transmisor. En la figura 6-49, el proceso transmisor está en el núcleo para ejecutar SEND. Lo primero que hace la entidad de transporte es probar si éste es el caso normal: el estado es *ESTABLISHED*, ningún lado está tratando de cerrar la conexión, se está enviando una TPDU normal (es decir, no fuera de banda) completa, y hay suficiente espacio de ventana disponible en el receptor. Si se cumplen todas la condiciones, no se requieren pruebas adicionales y puede seguirse la trayectoria rápida a través de la entidad de transporte transmisora.

En el caso normal, las cabeceras de las TPDU de datos consecutivas son casi iguales. Para aprovechar este hecho, se almacena una cabecera prototipo en la entidad de transporte. Al principio de la trayectoria rápida, la cabecera se copia lo más rápidamente posible en un *buffer* de trabajo, palabra por palabra. Los campos que cambian de una TPDU a otra se sobreescriben en el *buffer*. Con frecuencia, estos campos se deducen fácilmente de las variables de estado, como el siguiente número de secuencia. A continuación se pasan a la capa de red un apuntador a la cabecera completa de la TPDU más un apuntador a los datos de usuario. Aquí puede seguirse la misma estrategia (no se muestra en la figura 6-49). Por último, la capa de red entrega el paquete resultante a la capa de enlace de datos para su transmisión.

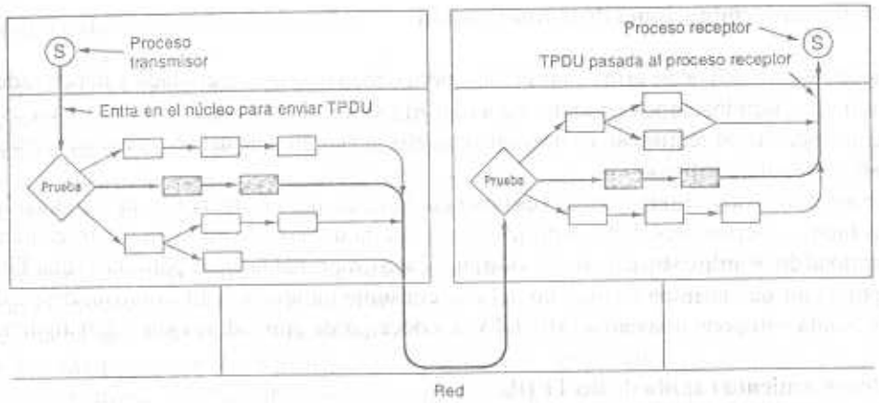


Figura 6-49. La trayectoria rápida del transmisor al receptor se indica con una línea gruesa. Los pasos de procesamiento de esta trayectoria se muestran sombreados.

Como ejemplo del funcionamiento de este principio en la práctica, consideremos el TCP/IP. En la figura 6-50(a) se muestra la cabecera TCP. Los campos que son iguales en varias TPDU consecutivas durante un flujo en un solo sentido aparecen sombreados. Todo lo que tiene que hacer la entidad de transporte transmisora es copiar las cinco palabras de la cabecera prototipo en el *buffer* de salida, actualizar el número de secuencia (copiándolo de una palabra en la memoria), calcular la suma de comprobación e incrementar el número de secuencia en la memoria. Entonces puede entregar la cabecera y los datos a un procedimiento IP especial para enviar una TPDU normal máxima. El IP entonces copia su cabecera prototipo de cinco palabras [véase la figura 6-50-(b)] en el *buffer*, llena el campo de *identificación* y calcula su suma de comprobación. El paquete ya está listo para transmitirse.

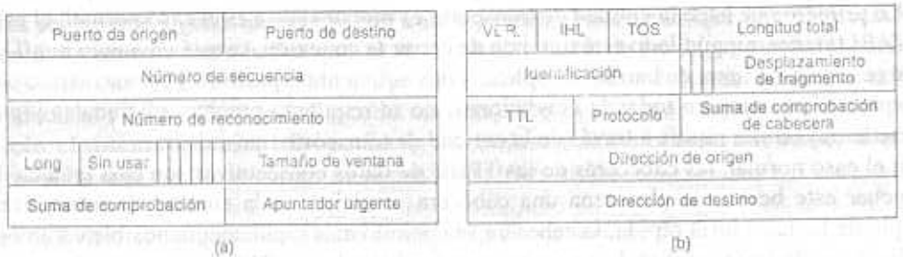


Figura 6-50. (a) Cabecera TCP. (b) Cabecera IP. En ambos casos, los campos sombreados se toman sin cambios del prototipo.

Veamos ahora al proceso de trayectoria rápida del lado receptor de la figura 6-49. El paso 1 es localizar el registro de conexión de la TPDU entrante. En el ATM, es fácil encontrar el registro

de conexión: el campo *VPI* puede usarse como índice de la tabla de trayectorias para encontrar la tabla de circuitos virtuales de esa trayectoria, y el *VCI* puede servir como índice para encontrar el registro de conexión. En el TCP, el registro de conexión puede almacenarse en una tabla de dispersión (*hash*) en la que alguna función sencilla de las dos direcciones IP y los dos puertos es la clave. Una vez localizado el registro de conexión, ambas direcciones y ambos puertos deben compararse para comprobar que se ha encontrado el registro correcto.

Una optimización que con frecuencia acelera la búsqueda del registro de conexión aún más es sencilla: mantener un apuntador al último registro usado e intentar ése primero. Clark *et al.* (1989) probó esto y observó una tasa de éxito mayor al 90%. Otras heurísticas de búsqueda se describen en (McKenney y Dove, 1992).

A continuación se revisa la TPDU para ver si es normal: el estado es *ESTABLISHED*, ninguno de los dos lados está tratando de cerrar la conexión, la TPDU es completa, no hay indicadores especiales encendidos y el número de secuencia es el esperado. Estas pruebas se llevan apenas unas cuantas instrucciones. Si se cumplen todas las condiciones, se invoca un procedimiento especial TCP de trayectoria rápida.

La trayectoria rápida actualiza el registro de la conexión y copia los datos en el espacio de usuario. Mientras copia, el procedimiento también calcula la suma de comprobación, eliminando una pasada extra por los datos. Si la suma de comprobación es correcta, se actualiza el registro de conexión y se devuelve un reconocimiento. El esquema general de hacer primero una comprobación rápida para ver si la cabecera es la esperada y tener un procedimiento especial para manejar ese caso se llama **predicción de cabecera**. Muchas implementaciones del TCP lo usan. Cuando esta optimización y todas las demás estudiadas en este capítulo se usan juntas, es posible conseguir que el TCP opere al 90% de la velocidad de una copia local de memoria a memoria, suponiendo que la red misma es lo bastante rápida.

Dos áreas en las que son posibles mejoras sustanciales del desempeño son el manejo de *buffers* y la administración de los temporizadores. El aspecto importante del manejo de *buffers* es evitar el copiado innecesario, como se explicó antes. La administración de los temporizadores es importante porque casi ninguno de los temporizadores expira; se ajustan para protegerse contra pérdidas de TPDU, pero la mayoría de las TPDU llegan correctamente, y sus reconocimientos también. Por tanto, es importante optimizar el manejo de los temporizadores para el caso en que los temporizadores casi nunca expiran.

Un esquema común consiste en usar una lista ligada de eventos de temporización ordenada por hora de expiración. La entrada inicial contiene un contador que indica la cantidad de pulsos de reloj que faltan para la expiración. Cada entrada subsecuente contiene un contador que indica el rezago en pulsos después de la entrada previa. Por tanto, si los temporizadores expiran a 3, 10 y 12 pulsos, respectivamente, los tres contadores son de 3, 7 y 2, respectivamente.

Después de cada pulso de reloj, se decrementa el contador de la cabecera inicial. Cuando llega a cero, su evento se procesa y el siguiente elemento de la lista es ahora el inicial; su contador no necesita cambiarse. En este esquema, la inserción y eliminación de temporizadores son operaciones caras, con tiempos de ejecución proporcionales a la longitud de la lista.

Puede usarse un enfoque más eficiente si el intervalo máximo del temporizador está limitado y se conoce por adelantado. Aquí puede utilizarse un arreglo llamado **rueda de temporización**,

como se muestra en la figura 6-51. Cada ranura corresponde a un pulso de reloj. El tiempo actual en la figura es $T = 4$. Los temporizadores se programan para expirar 3, 10 y 12 pulsos más adelante. Si se establece un temporizador nuevo para expirar en siete pulsos, simplemente se crea una entrada en la ranura 11. Del mismo modo, si el temporizador establecido para $T + 10$ tiene que cancelarse, debe examinarse la lista que comienza en la ranura 14 para eliminar la entrada pertinente. Nótese que el arreglo de la figura 6-51 no puede manejar temporizadores más allá de $T + 15$.

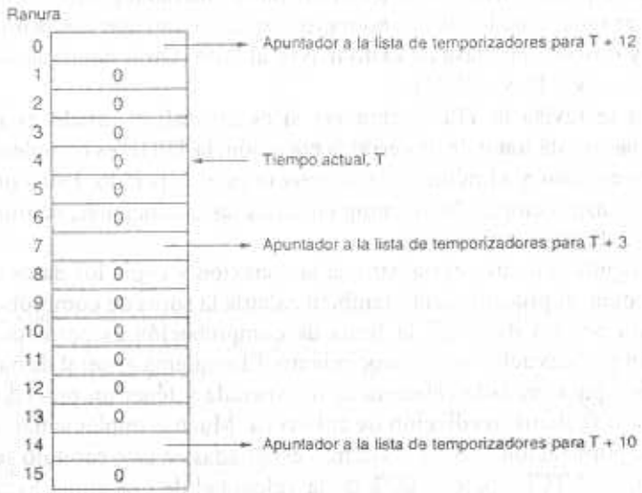


Figura 6-51. Rueda de temporización.

Cuando el reloj pulsa, el apuntador de tiempo actual avanza una ranura (circularmente). Si la entrada a la que ahora se apunta es diferente de cero, todos sus temporizadores se procesan. Se estudian muchas variaciones de la idea básica en (Varghese y Lauck, 1987).

6.6.5. Protocolos de redes de gigabits

Al inicio de la década de 1990 comenzaron a aparecer las redes de gigabits. La primera reacción de la gente fue usar en ellas los viejos protocolos, pero pronto surgieron varios problemas. En esta sección estudiaremos algunos de estos problemas y las medidas que están tomando los protocolos nuevos para resolverlos. Puede encontrarse más información en (Baransel *et al.*, 1995, y Partridge, 1994).

El primer problema es que muchos protocolos usan números de secuencia de 16 o 32 bits. Antaño, 2^{32} era una aproximación muy buena al infinito; hoy ya no lo es. Con una tasa de datos de 1 Gbps, se requieren unos 32 segundos para enviar 2^{32} bytes. Si los números de secuencia se refieren a bytes, como ocurre en el TCP, entonces un transmisor puede arrancar a toda velocidad trasmitiendo desde el byte 0 y, 32 segundos después, estar de nueva cuenta en el byte 0. Aun

suponiendo que se han recibido los acuses de recibo de todos los bytes, el transmisor no puede transmitir con seguridad datos nuevos desde 0 porque los paquetes viejos todavía podrían estar flotando en algún lado. En el Internet, por ejemplo, los paquetes pueden vivir 120 seg. Si se numeran los paquetes en lugar de los bytes, el problema es menos severo, a menos que los números de secuencia sean de 16 bits, en cuyo caso el problema es peor aún.

La causa de esta situación es que muchos diseñadores de protocolos simplemente supusieron, tácitamente, que el tiempo requerido para usar el espacio de secuencia completo excedería por mucho el tiempo de vida máximo de los paquetes. En consecuencia, no había necesidad de preocuparse por el problema de que duplicados viejos sobrevivieran aún al dar la vuelta los números de secuencia. A velocidades de gigabits, falla ese supuesto implícito.

Un segundo problema es que las velocidades de comunicación han mejorado con mucha mayor rapidez que las velocidades de cómputo. (Nota a los ingenieros en computación: ¡salgan a darles una paliza a los ingenieros en comunicaciones! Contamos con ustedes.) En los años 70, ARPANET operaba a 56 kbps y tenía computadoras que operaban a aproximadamente 1 MIPS. Los paquetes eran de 1008 bits, por lo que ARPANET era capaz de entregar unos 56 paquetes/seg. Con casi 18 mseg disponibles por paquete, un *host* podía darse el lujo de gastar 18,000 instrucciones en el procesamiento de un paquete. Claro que hacerlo requeriría todo el tiempo de CPU, pero podrían ejecutarse 9000 instrucciones por paquete y aun así tener la mitad del tiempo de CPU libre para llevar a cabo tareas reales.

Compare estas cantidades con las computadoras modernas de 100 MIPS que intercambian paquetes de 4 kB por una línea de gigabits. Los paquetes pueden entrar a una velocidad de más de 30,000 por segundo, por lo que el procesamiento de los paquetes debe completarse en 15 mseg si queremos reservar la mitad de la CPU para las aplicaciones. En 15 mseg, una computadora de 100 MIPS puede ejecutar 1500 instrucciones, apenas 1/6 de lo que tenían disponibles los *hosts* de ARPANET. Es más, las instrucciones RISC modernas hacen menos por instrucción de lo que hacían las viejas instrucciones CISC, por lo que el problema es aún peor que lo que parece. En conclusión, hay menos tiempo disponible para el procesamiento de los protocolos del que había antes, por lo que los protocolos deben volverse más sencillos.

Un tercer problema es que el protocolo de regresar n se desempeña mal en las líneas con un producto ancho de banda-retardo grande. Consideremos, por ejemplo, una línea de 4000 km que opera a 1 Gbps. El tiempo de transmisión de ida y vuelta es de 40 mseg, durante el cual un transmisor puede enviar 5 megabytes. Si se detecta un error, pasarán 40 mseg antes de que el transmisor se entere. Si se usa regresa n, el transmisor tendrá que retransmitir no sólo el paquete erróneo, sino también los 5 megabytes de paquetes que llegaron después. Evidentemente, éste es un desperdicio masivo de recursos.

Un cuarto problema es que las líneas de gigabits son fundamentalmente diferentes de las líneas de megabits en el sentido de que las líneas largas están limitadas por el retardo en lugar del ancho de banda. En la figura 6-52 mostramos el tiempo que tarda la transferencia de un archivo de 1 megabit a 4000 km con varias velocidades de transmisión. A velocidades de hasta 1 Mbps, el tiempo de transmisión está dominado por la velocidad con que pueden enviarse los bits. A 1 Gbps, el retardo de ida y vuelta de 40 mseg domina al 1 mseg que se tarda la inserción de los bits en la fibra. Aumentos mayores del ancho de banda apenas tienen algún efecto.

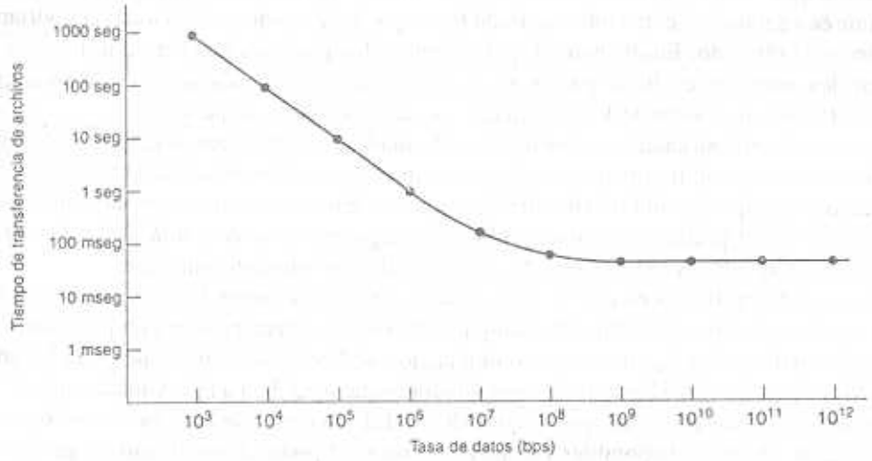


Figura 6-52. Tiempo de transferencia y reconocimiento de un archivo de 1 megabit a través de una línea de 4000 km.

La figura 6-52 tiene implicaciones desafortunadas para los protocolos de red; dice que los protocolos de parada y espera, como el RPC, tienen un límite superior de desempeño inherente. Este límite lo establece la velocidad de la luz. Ningún progreso tecnológico en la óptica mejorará la situación (aunque ayudarían nuevas leyes de la física).

Un quinto problema que vale la pena mencionar no es tecnológico ni de protocolo, como los otros, sino una consecuencia de las aplicaciones nuevas. En pocas palabras, en muchas aplicaciones de gigabits, como las de multimedia, la variación en los tiempos de llegada de los paquetes es tan importante como el retardo medio mismo. Una tasa de entrega lenta pero uniforme con frecuencia es preferible a una rápida pero variable.

Pasemos ahora de los problemas a las maneras de resolverlos. Primero haremos algunas observaciones generales, luego veremos los mecanismos de protocolo, la organización de los paquetes y el *software* de los protocolos.

El principio básico que deben aprender de memoria todos los diseñadores de redes de gigabits es:

Diseñar pensando en la velocidad, no en la optimización del ancho de banda.

Los protocolos viejos con frecuencia se diseñaban tratando de reducir al mínimo la cantidad de bits en el alambre, comúnmente usando campos pequeños y empacándolos en bytes y palabras. Hoy día hay más que suficiente ancho de banda. El procesamiento del protocolo es el problema, por lo que los protocolos deberían diseñarse para reducirlo al mínimo.

Una manera tentadora de acelerar el procedimiento es construir interfaces de red rápidas en *hardware*. Lo malo de esta estrategia es que, a menos que el protocolo sea excesivamente sencillo, "*hardware*" simplemente significa una tarjeta con una segunda CPU y su propio programa. Para evitar que el coprocesador de la red sea tan caro como la CPU principal, con frecuencia se usa un *chip* más lento. La consecuencia de este diseño es que una buena parte del tiempo la CPU principal (rápida) está esperando que una segunda CPU (lenta) haga el trabajo crítico. Es un mito pensar que la CPU principal tiene otras tareas que hacer mientras espera. Es más, cuando dos CPU de propósito general se comunican, pueden ocurrir condiciones de competencia, por lo que se requieren protocolos complejos entre los dos procesadores para sincronizarlos correctamente. Generalmente el mejor enfoque es hacer que los protocolos sean sencillos y dejar que la CPU principal haga el trabajo.

Veamos ahora el asunto de la realimentación en los protocolos de alta velocidad. Debido al ciclo de retardo grande (relativamente), debe evitarse la realimentación: la señalización del receptor al transmisor tarda demasiado. Un ejemplo de realimentación es el control de la tasa de transmisión mediante un protocolo de ventana corrediza. Para evitar los retardos (grandes) inherentes en el envío de actualizaciones de ventana del receptor al transmisor, es mejor usar un protocolo basado en la tasa. En tal protocolo, el transmisor puede enviar todo lo que quiera, siempre y cuando no envíe a mayor velocidad que cierta tasa acordada de antemano entre el transmisor y el receptor.

Un segundo ejemplo de realimentación es el algoritmo de arranque lento de Jacobson. Este algoritmo efectúa múltiples pruebas para saber qué tanto puede manejar la red. Con las redes de alta velocidad, efectuar media docena de pruebas pequeñas para ver la respuesta de la red es un desperdicio de ancho de banda enorme. Un esquema más eficiente es hacer que el transmisor, el receptor y la red reserven los recursos necesarios en el momento de establecer la conexión. La reservación de recursos por adelantado también tiene la ventaja de facilitar la reducción de la fluctuación. En pocas palabras, la elevación de las velocidades inevitablemente empuja el diseño hacia la operación orientada a conexiones, o a algo muy parecido.

La organización de los paquetes es una consideración importante en las redes de gigabits. La cabecera debería contener la menor cantidad de campos posible, a fin de reducir el tiempo de procesamiento. Estos campos deben ser lo bastante grandes como para cumplir su cometido, y estar alineados con los límites de palabra para facilitar su procesamiento. En este contexto, "bastante grande" significa que no ocurren problemas como números de secuencia que dan la vuelta mientras existen aún paquetes viejos, receptores incapaces de anunciar suficiente espacio de ventana porque el campo de ventana es demasiado pequeño, etcétera.

La cabecera y los datos deben tener sumas de comprobación aparte, por dos razones. Primera, hacer posible la obtención de la suma de comprobación de la cabecera, pero no de los datos. Segunda, comprobar que la cabecera está correcta antes de comenzar a copiar los datos en el espacio de usuario. Es deseable calcular la suma de comprobación de los datos al mismo tiempo que se copian los datos en el espacio de usuario, pero si el contenido de la cabecera está equivocado, el copiado podría hacerse a un proceso equivocado. Para evitar un copiado incorrecto pero permitir que se calcule la suma de comprobación de los datos durante el copiado, es esencial que las dos sumas de comprobación estén separadas.

El tamaño máximo de los datos debe ser lo bastante grande para permitir una operación eficiente inclusive ante retardos grandes. También, cuanto mayor es el tamaño del bloque de datos, menor es la fracción del ancho de banda total dedicada a las cabeceras.

Otra característica valiosa es la capacidad de enviar una cantidad normal de datos junto con la solicitud de conexión. Así, puede ahorrarse el tiempo de un viaje de ida y vuelta.

Por último, es importante decir algo sobre el *software* de los protocolos. Una idea clave es concentrarse en el caso exitoso. Muchos protocolos viejos tienden a hacer hincapié en lo que se debe hacer cuando algo falla (por ejemplo, cuando se pierde un paquete). Para hacer que los protocolos operen rápidamente, el diseñador debería enfocarse en reducir al mínimo el tiempo de procesamiento cuando todo funciona bien. Es secundaria la reducción al mínimo del tiempo de procesamiento cuando ocurren errores.

Un segundo asunto relacionado con el *software* es la minimización del tiempo de copiado. Como vimos antes, el copiado de datos con frecuencia es el origen principal de carga extra. Idealmente, el *hardware* debe poner en la memoria cada paquete de entrada como un bloque contiguo de datos. El *software* entonces debe copiar este paquete en el *buffer* del usuario con un solo copiado de bloque. Dependiendo del modo de funcionamiento del caché, inclusive puede ser deseable evitar un ciclo de copiado. En otras palabras, para copiar 1024 palabras, la manera más rápida podría ser la ejecución de 1024 instrucciones MOVE una tras otra (o 1024 pares cargar-almacenar). La rutina de copiado es tan crítica que debe desarrollarse cuidadosamente en código de ensamblador, a menos que haya una manera de lograr que el compilador produzca el código óptimo.

A finales de los años 80, hubo un interés efímero en los protocolos rápidos de propósito especial como NETBLT (Clark *et al.*, 1987), VTMP (Cheriton y Williamson, 1989) y XTP (Chesson, 1989). En (Doeringer *et al.*, 1990) se hace un estudio. Sin embargo, la tendencia ahora es hacia la simplificación de protocolos de propósito general para hacerlos también rápidos. ATM presenta muchas de las características que analizamos antes, al igual que IPv6.

6.7. RESUMEN

La capa de transporte es la clave para entender los protocolos en capas. Esta capa proporciona varios servicios, siendo el más importante una corriente de bytes punto a punto, confiable y orientada o conexiones desde el transmisor hasta el receptor. Se accede a ella a través de primitivas de servicio que permiten establecer, usar y liberar conexiones.

Los protocolos de transporte deben ser capaces de administrar conexiones a través de redes no confiables. El establecimiento de conexiones se complica por la existencia de paquetes duplicados retardados que pueden reaparecer en momentos inoportunos. Para manejarlos, se requieren protocolos de acuerdo de tres vías (*three-way handshakes*) para establecer las conexiones. La liberación de una conexión es más sencilla que su establecimiento, pero aun así es un asunto poco trivial debido al problema de los dos ejércitos.

Aun si la capa de red es completamente confiable, la capa de transporte tiene bastante trabajo, como vimos en nuestro ejemplo: debe manejar todas las primitivas de servicio, administrar conexiones y temporizadores y asignar y usar créditos.

El protocolo de transporte principal de Internet es el TCP. Éste usa una cabecera de 20 bytes en todos los segmentos. Los segmentos pueden ser fragmentados por los enrutadores de Internet, por lo que los *hosts* deben estar preparados para efectuar el reensamblado. Se ha trabajado mucho para optimizar el desempeño del TCP, usando los algoritmos de Nagle, Clark, Jacobson, Karn y otros.

El ATM tiene cuatro protocolos en la capa AAL. Los cuatro dividen los mensajes en células en el origen y reconstruyen los mensajes en el destino. Las subcapas CS y SAR agregan sus propias cabeceras y apéndices de varias maneras, dejando de 44 a 48 bytes de carga útil por célula.

El desempeño de las redes generalmente es dominado por la carga extra de procesamiento de los protocolos y las TPDU, y esta situación empeora a mayores velocidades. Los protocolos deberían diseñarse para reducir al mínimo la cantidad de TPDU, de conmutaciones de contexto y de veces que se copia cada TPDU. En las redes de gigabits se requieren protocolos sencillos que usen control de flujo por tasa, no por crédito.

PROBLEMAS

1. En nuestras primitivas de ejemplo de la figura 6-3, LISTEN es una llamada bloqueadora. ¿Es estrictamente necesario esto? De no serlo, explique cómo debe usarse una primitiva no bloqueadora. ¿Qué ventaja tendría esto respecto al esquema descrito en el texto?
2. En el modelo de la figura 6-5, se supone que la capa de red puede perder paquetes, y por tanto deben reconocerse individualmente. Supóngase que la capa de red es 100% confiable y que nunca pierde paquetes. ¿Qué cambios, si acaso, se necesitarían en la figura 6-5?
3. Imagine un problema generalizado de n ejércitos, en el que el acuerdo entre dos de los ejércitos es suficiente para la victoria. ¿Existe un protocolo que permita que ganen los azules?
4. Supóngase que el esquema operado por reloj para generar números de secuencia iniciales se usa con un contador de reloj de 15 bits de ancho. El reloj pulsa cada 100 mseg, y el tiempo de vida máximo de un paquete es de 60 seg. ¿Con qué frecuencia ocurre la resincronización
   (a) en el peor caso?
   (b) cuando los datos consumen 240 números de secuencia/min?
5. ¿Por qué tiene que ser el tiempo de vida máximo de paquete, T, lo bastante grande para asegurar que han desaparecido no sólo el paquete, sino también sus acuses de recibo?
6. Imagine que se usa un acuerdo de dos vías (*two-way handshake*) en lugar de uno de tres vías para establecer las conexiones. En otras palabras, no se requiere el tercer mensaje. ¿Son posibles ahora los interbloqueos? Dé un ejemplo o demuestre que no pueden existir.
7. Considere el problema de la recuperación después de una caída del *host* (es decir, la figura 6-18). Si el intervalo entre la escritura y el envío de un reconocimiento, o viceversa, puede hacerse relativamente pequeño, ¿cuáles son las dos mejores estrategias transmisor-receptor para reducir al mínimo la posibilidad de una falla del protocolo?
8. ¿Son posibles los interbloqueos con la entidad de transporte descrita en el texto?
9. Por curiosidad, el implementador de la entidad de transporte de la figura 6-20 ha decidido incluir contadores en el procedimiento *sleep* para obtener estadísticas sobre el arreglo *conn*. Entre éstas están la cantidad de conexiones en cada uno de los siete estados posibles, n_i ($i = 1, \ldots, 7$). Tras escribir un programa masivo en FORTRAN para analizar los datos, nuestro implementador descubrió que la relación $\sum n_i = MAX_CONN$ parece ser verdadera siempre. ¿Hay otras invariantes en las que intervengan sólo estas siete variables?

10. ¿Qué ocurre cuando el usuario de la entidad de transporte dada en la figura 6-20 envía un mensaje de longitud cero? Explique el significado de su respuesta.
11. Por cada evento que puede ocurrir en la entidad de transporte de la figura 6-20, indique si es legal o no cuando el usuario está durmiendo en el estado *sending*.
12. Explique las ventajas y desventajas de los créditos sobre los protocolos de ventana corrediza.
13. La fragmentación de datagramas y su reensamble son manejados por el IP y son invisibles para el TCP. ¿Significa esto que el TCP no tiene que preocuparse por los datos que llegan en el orden equivocado?
14. A un proceso del *host* 1 se le asignó el puerto p y a un proceso del *host* 2 se le asignó el puerto q. ¿Es posible que haya más de dos conexiones TCP entre estos dos puertos al mismo tiempo?
15. La carga máxima de un segmento TCP es de 65,515 bytes. ¿Por qué se escogió un número tan extraño?
16. Describa dos formas de entrar en el estado *SYN RCVD* de la figura 6-28.
17. Indique una desventaja potencial del uso del algoritmo de Nagle en una red muy congestionada.
18. Considere el efecto de usar arranque lento en una línea con un tiempo de ida y vuelta de 10 mseg y sin congestionamientos. La ventana receptora es de 24 kB y el tamaño máximo de segmento es de 2 kB. ¿Cuánto tiempo pasará antes de poder enviar la primera ventana completa?
19. Supóngase que la ventana de congestionamiento del TCP está ajustada a 18K bytes y que ocurre una terminación de temporización. ¿Qué tan grande será la ventana si las siguientes cuatro ráfagas de transmisión tienen éxito? Supóngase que el tamaño máximo de segmento es de 1 kB.
20. Si el tiempo de ida y vuelta del TCP, *RTT*, actualmente es de 30 mseg y los siguientes reconocimientos llegan después de 26, 32 y 24 mseg, respectivamente, ¿cuál es la nueva estimación de *RTT*? Use $\alpha = 0.9$.
21. Una máquina TCP envía ventanas de 65,535 bytes por un canal de 1 Gbps que tiene un retardo de 10 mseg en un solo sentido. ¿Cuál es el rendimiento máximo que se puede lograr? ¿Es eficiente la línea?
22. En una red que tiene un tamaño máximo de TPDU de 128 bytes, un tiempo de vida de TPDU máximo de 30 seg, y un número de secuencia de 8 bits, ¿cuál es la tasa máxima de datos por conexión?
23. ¿Por qué existe el UDP? ¿No habría bastado con permitir que los procesos de usuario enviaran paquetes IP en bruto?
24. Un grupo de N usuarios ubicados en el mismo edificio usan la misma computadora remota a través de una red ATM. El usuario medio genera L líneas de tráfico (entrada + salida) por hora, en promedio, siendo la longitud media de línea de P bytes, excluyendo las cabeceras ATM. La portadora de paquetes cobra C centavos por byte de datos de usuario transportado, más X centavos por hora, por cada circuito virtual ATM abierto. ¿En qué condiciones es efectivo desde el punto de vista del costo multiplexar las N conexiones de transporte en el mismo circuito virtual ATM, si tal multiplexión agrega 2 bytes de datos a cada paquete? Supóngase que un solo circuito virtual ATM tiene suficiente ancho de banda para todos los usuarios.

25. ¿Puede manejar el AAL 1 mensajes más cortos que 40 bytes usando el esquema con el campo de *apuntador*? Explique su respuesta.
26. Adivine los tamaños que tenían los campos del AAL 2 antes de que los quitaran del estándar.
27. El AAL 3/4 permite la multiplexión de varias sesiones en un solo circuito virtual. Dé un ejemplo de situación en la que eso no tiene ningún valor. Suponga que un circuito virtual tiene suficiente ancho de banda para llevar todo el tráfico. *Sugerencia*: Piense sobre trayectorias virtuales.
28. ¿Cuál es el tamaño de la carga útil del mensaje de mayor longitud que cabe en una sola célula AAL 3/4?
29. Cuando se envía un mensaje de 1024 bytes con el AAL 3/4, ¿qué eficiencia se obtiene? En otras palabras, ¿qué fracción de los bits transmitidos comprende bits de datos útiles? Repita el problema para el AAL 5.
30. Un dispositivo ATM transmite mensajes de una sola célula a 600 Mbps. Una de cada 100 células se altera completamente debido al ruido aleatorio. ¿Cuántos errores no detectados pueden esperarse por semana con la suma de comprobación de 32 bits del AAL 5?
31. Un cliente envía una solicitud de 128 bytes a un servidor situado a 100 km a través de una fibra óptica de 1 gigabit. ¿Cuál es la eficiencia de la línea durante la llamada al procedimiento remoto?
32. Considere nuevamente la situación del problema anterior. Calcule el tiempo de respuesta mínimo posible tanto para la línea dada de 1 Gbps como para una línea de 1 Mbps. ¿Cuáles son sus conclusiones?
33. Suponga que está midiendo el tiempo de recepción de una TPDU. Al ocurrir una interrupción, usted lee el reloj del sistema en milisegundos; al terminar de procesarse la TPDU, lee el reloj de nuevo. Se mide 0 mseg 270,000 veces y 1 mseg 730,000 veces. ¿Cuál es el tiempo de recepción de una TPDU?
34. Una CPU ejecuta instrucciones a razón de 100 MIPS. Los datos pueden copiarse 64 bits a la vez, costando seis instrucciones cada palabra copiada. Si un paquete de entrada tiene que copiarse dos veces, ¿puede este sistema manejar una línea de 1 Gbps? Por sencillez, considere que todas las instrucciones, inclusive aquellas que leen o escriben en la memoria, operan a la velocidad máxima de 100 MIPS.
35. Para encontrar una solución al problema de los números de secuencia que vuelven a comenzar mientras aún existen paquetes viejos, podrían usarse números de secuencia de 64 bits. Sin embargo, en teoría, una fibra óptica puede operar a 75 Tbps. ¿Qué tiempo de vida máximo de paquete se requiere para asegurar que las redes futuras de 75 Tbps no tenga este problema aun con números de secuencia de 64 bits? Suponga que cada byte tiene su propio número de secuencia, como ocurre en el TCP.
36. En el texto calculamos que una línea de gigabits entrega 30,000 paquetes/seg al *host*, lo que deja sólo 1500 instrucciones para procesar los paquetes si la CPU dedica la mitad del tiempo a las aplicaciones. Este cálculo supuso un paquete de 4 kB. Repita el cálculo para un paquete de tamaño ARPANET (128 bytes).
37. En una red de 1 Gbps que opera sobre 4000 km, el factor limitante es el retardo, no el ancho de banda. Considere una MAN en la que el origen y el destino están a una distancia promedio de 20 km. ¿A qué tasa de datos el retardo de ida y vuelta debido a la velocidad de la luz equivale al retardo de transmisión de un paquete de 1 kB?

38. Modifique el programa de la figura 6-20 para efectuar recuperación de errores. Agregue un tipo de paquete nuevo, *reset*, que puede llegar tras haberse abierto una conexión en ambos lados, sin que ninguno la haya cerrado después. Este evento, que ocurre simultáneamente en ambos lados de la conexión, implica que todos los paquetes que estaban en tránsito han sido entregados o destruidos, pero en ninguno de los dos casos continúan en la subred.

39. Escriba un programa que simule la administración de *buffers* en una entidad de transporte que usa una ventana corrediza para control de flujo en lugar del sistema de créditos de la figura 6-20. Permita que los procesos de capa superior abran conexiones, envíen datos y cierren conexiones al azar. Para mantener la sencillez, haga que todos los datos viajen de la máquina *A* a la máquina *B*, y ninguno en el otro sentido. Experimente con diferentes estrategias de asignación de *buffers* en *B*, como dedicar *buffers* a conexiones específicas en lugar de tener un fondo de *buffers* común, y mida el rendimiento total logrado con cada una.

7

LA CAPA DE APLICACIÓN

Habiendo concluido todos los preliminares, ahora llegamos a la capa de aplicación, donde pueden encontrarse todas las aplicaciones interesantes. Las capas por debajo de la de aplicación están ahí para proporcionar transporte confiable, pero no hacen ningún trabajo verdadero para los usuarios. En este capítulo estudiaremos algunas aplicaciones reales.

Sin embargo, aun en la capa de aplicación se necesitan protocolos de apoyo que permitan el funcionamiento de las aplicaciones reales; veremos tres de ellos antes de comenzar con las aplicaciones mismas. La primera área es la seguridad, que no es un solo protocolo, sino una gran cantidad de conceptos y protocolos que pueden usarse para asegurar la confidencialidad donde sea necesaria. La segunda es el DNS, que maneja los nombres en Internet. El tercer protocolo de apoyo sirve para la administración de la red. Después de esto, estudiaremos cuatro aplicaciones reales: el correo electrónico, USENET (noticias de red), *World Wide Web* y, por último, multimedia.

7.1. SEGURIDAD DE LA RED

Durante las primeras décadas de su existencia, las redes de computadoras fueron usadas principalmente por investigadores universitarios para el envío de correo electrónico, y por empleados corporativos para compartir impresoras. En estas condiciones, la seguridad no recibió mucha atención. Pero ahora, cuando millones de ciudadanos comunes usan redes para sus transacciones bancarias, compras y declaraciones de impuestos, la seguridad de las redes aparece en el hori-

38. Modifique el programa de la figura 6-20 para efectuar recuperación de errores. Agregue un tipo de paquete nuevo, *reset*, que puede llegar tras haberse abierto una conexión en ambos lados, sin que ninguno la haya cerrado después. Este evento, que ocurre simultáneamente en ambos lados de la conexión, implica que todos los paquetes que estaban en tránsito han sido entregados o destruidos, pero en ninguno de los dos casos continúan en la subred.

39. Escriba un programa que simule la administración de *buffers* en una entidad de transporte que usa una ventana corrediza para control de flujo en lugar del sistema de créditos de la figura 6-20. Permita que los procesos de capa superior abran conexiones, envíen datos y cierren conexiones al azar. Para mantener la sencillez, haga que todos los datos viajen de la máquina *A* a la máquina *B*, y ninguno en el otro sentido. Experimente con diferentes estrategias de asignación de *buffers* en *B*, como dedicar *buffers* a conexiones específicas en lugar de tener un fondo de *buffers* común, y mida el rendimiento total logrado con cada una.

7

LA CAPA DE APLICACIÓN

Habiendo concluido todos los preliminares, ahora llegamos a la capa de aplicación, donde pueden encontrarse todas las aplicaciones interesantes. Las capas por debajo de la de aplicación están ahí para proporcionar transporte confiable, pero no hacen ningún trabajo verdadero para los usuarios. En este capítulo estudiaremos algunas aplicaciones reales.

Sin embargo, aun en la capa de aplicación se necesitan protocolos de apoyo que permitan el funcionamiento de las aplicaciones reales; veremos tres de ellos antes de comenzar con las aplicaciones mismas. La primera área es la seguridad, que no es un solo protocolo, sino una gran cantidad de conceptos y protocolos que pueden usarse para asegurar la confidencialidad donde sea necesaria. La segunda es el DNS, que maneja los nombres en Internet. El tercer protocolo de apoyo sirve para la administración de la red. Después de esto, estudiaremos cuatro aplicaciones reales: el correo electrónico, USENET (noticias de red), *World Wide Web* y, por último, multimedia.

7.1. SEGURIDAD DE LA RED

Durante las primeras décadas de su existencia, las redes de computadoras fueron usadas principalmente por investigadores universitarios para el envío de correo electrónico, y por empleados corporativos para compartir impresoras. En estas condiciones, la seguridad no recibió mucha atención. Pero ahora, cuando millones de ciudadanos comunes usan redes para sus transacciones bancarias, compras y declaraciones de impuestos, la seguridad de las redes aparece en el hori-

zonte como un problema potencial de grandes proporciones. En las siguientes secciones estudiaremos la seguridad de las redes desde varios ángulos, señalaremos muchos peligros y estudiaremos varios algoritmos y protocolos para hacer más seguras las redes.

La seguridad es un tema amplio que cubre una multitud de pecados. En su forma más sencilla, la seguridad se ocupa de garantizar que los curiosos no puedan leer, o peor aún, modificar mensajes dirigidos a otros destinatarios; se preocupa por la gente que intenta acceder a servicios remotos no autorizados; también se ocupa de mecanismos para verificar que el mensaje supuestamente enviado por la autoridad fiscal que indica: "Pague el viernes o aténgase a las consecuencias" realmente viene de ella y no de la mafia. La seguridad también se ocupa del problema de la captura y reproducción de mensajes legítimos, y de la gente que intenta negar que envió ciertos mensajes.

La mayoría de los problemas de seguridad son causados intencionalmente por gente maliciosa que intenta ganar algo o hacerle daño a alguien. Algunos tipos de delincuentes más comunes se listan en la figura 7-1. Debe quedar claro por esta lista que hacer segura una red comprende mucho más que simplemente mantener los programas libres de errores; implica ser más listo que adversarios a menudo inteligentes, dedicados y a veces bien financiados. Debe quedar claro también que las medidas para detener a los adversarios casuales tendrán poco impacto sobre los serios.

Adversario	Meta
Estudiante	Divertirse husmeando el correo de la gente
Hacker	Probar el sistema de seguridad de alguien; robar datos
Representante de ventas	Indicar que representa a toda Europa, no sólo a Andorra
Hombre de negocios	Descubrir el plan estratégico de mercadeo de un competidor
Ex empleado	Vengar su despido
Contador	Estafar dinero de una compañía
Corredor de bolsa	Negar una promesa hecha a un cliente por correo electrónico
Timador	Robar números de tarjeta de crédito
Espía	Conocer la fuerza militar de un enemigo
Terrorista	Robar secretos de guerra bacteriológica

Figura 7-1. Algunas personas que causan problemas de seguridad, y por qué.

Los problemas de seguridad de las redes pueden dividirse en términos generales en cuatro áreas interrelacionadas: secreto, validación de identificación, no repudio y control de integridad. El secreto tiene que ver con mantener la información fuera de las manos de usuarios no autorizados. Esto es lo que normalmente viene a la mente cuando la gente piensa en la seguridad de las redes. La validación de identificación se encarga de determinar con quién se está hablando antes

de revelar información delicada o hacer un trato de negocios. El no repudio se encarga de las firmas: ¿cómo comprobar que su cliente realmente colocó una orden electrónica por 10 millones de marinolas con rosca a la izquierda a 89 centavos cada una cuando él luego alega que el precio era de 69 centavos? Por último, ¿cómo puede asegurarse de que un mensaje recibido realmente fue el enviado, y no algo que un adversario malicioso modificó en el camino o cocinó por su propia cuenta?

Todos estos temas (secreto, validación de identificación, no repudio y control de integridad) son pertinentes también en los sistemas tradicionales, pero con algunas diferencias importantes. El secreto y la integridad se logran usando correo registrado y poniendo bajo llave los documentos. El robo del tren del correo es más difícil de lo que era en los tiempos de Jesse James.

También, la gente puede por lo general distinguir entre un documento original en papel y una fotocopia, y con frecuencia esto es importante. Como prueba, haga una fotocopia de un cheque válido. Trate de cobrar el cheque original en su banco el lunes. Ahora trate de cobrar la fotocopia del cheque el martes. Observe la diferencia de comportamiento del banco. Con los cheques electrónicos, el original y la copia son idénticos. Es posible que los bancos tarden un poco en acostumbrarse a esto.

La gente valida la identificación de otra gente al reconocer sus caras, voces y letra. Las pruebas de firmas se manejan mediante firmas en papel, sellos, etc. Generalmente puede detectarse la alteración de documentos con el auxilio de expertos en escritura, papel y tinta. Ninguna de estas opciones está disponible electrónicamente. Es obvio que se requieren otras soluciones.

Antes de entrar en las soluciones mismas, vale la pena dedicar un instante a considerar el lugar que corresponde a la seguridad de las redes en la pila de protocolos. Probablemente no es un solo lugar. Cada capa tiene algo que contribuir. En la capa física podemos protegernos contra la intervención de las líneas encerrando las líneas de transmisión en tubos sellados que contienen gas argón a alta presión. Cualquier intento de hacer un agujero en el tubo liberará un poco de gas, reduciendo la presión y disparando una alarma. Algunos sistemas militares usan esta técnica.

En la capa de enlace de datos, los paquetes de una línea punto a punto pueden codificarse al dejar una máquina y descodificarse al llegar a otra. Los detalles pueden manejarse en la capa de enlace de datos, desconociendo las capas superiores lo que ocurre. Sin embargo, esta solución se viene abajo cuando los paquetes tienen que atravesar varios enrutadores, puesto que los paquetes tienen que descifrarse en cada enrutador, dejándolos vulnerables a ataques dentro del enrutador. Además, no se contempla que algunas sesiones estén protegidas (por ejemplo, aquellas que comprenden compras en línea mediante tarjeta de crédito) y otras no. No obstante, el cifrado de enlace (*link encryption*), como se llama a este método, puede agregarse fácilmente a cualquier red y con frecuencia es útil.

En la capa de red, pueden instalarse muros de seguridad para mantener adentro (o afuera) a los paquetes. Vimos los muros de seguridad en el capítulo 5. En la capa de transporte, pueden cifrarse conexiones enteras, terminal a terminal, es decir, proceso a proceso. Aunque estas soluciones ayudan en los asuntos relacionados con el secreto y mucha gente hace un gran esfuerzo para mejorarlas, ninguna de ellas resuelve los problemas de validación de identificación y no repudio de una manera suficientemente general. Para atacar estos problemas, las soluciones deben implementarse en la capa de aplicación, razón por la cual las estudiamos en este capítulo.

7.1.1. Cifrado tradicional

El cifrado tiene una larga y colorida historia. En esta sección simplemente delinearemos algunos de los puntos importantes, como antecedentes para lo que sigue. Si desea una historia completa, el libro de Kahn (1967) aún es lectura recomendada. Un estudio detallado de la tecnología de vanguardia actual puede encontrarse en (Kaufman *et al.*, 1995; Schneier, 1996, y Stinson, 1995).

Históricamente, cuatro grupos de gente han usado y contribuido al arte del cifrado: los militares, el cuerpo diplomático, los diaristas y los amantes. De éstos, los militares han tenido el papel más importante y han allanado el camino. Dentro de las organizaciones militares, los mensajes a cifrar tradicionalmente se han entregado a empleados mal pagados para su codificación y transmisión. El inmenso volumen de los mensajes ha impedido asignar esta labor a unos cuantos especialistas de élite.

Hasta la llegada de las computadoras, una de las restricciones principales del cifrado había sido la capacidad del empleado encargado de la codificación para realizar las transformaciones necesarias, frecuentemente en un campo de batalla con poco equipo. Una restricción adicional ha sido la dificultad de cambiar rápidamente de un método de cifrado a otro, puesto que esto significa el reentrenamiento de una gran cantidad de gente. Sin embargo, el peligro de que un empleado fuera capturado por el enemigo ha hecho indispensable la capacidad de cambiar el método de cifrado al instante, de ser necesario. De estos requisitos en conflicto se deriva el modelo de la figura 7-2.

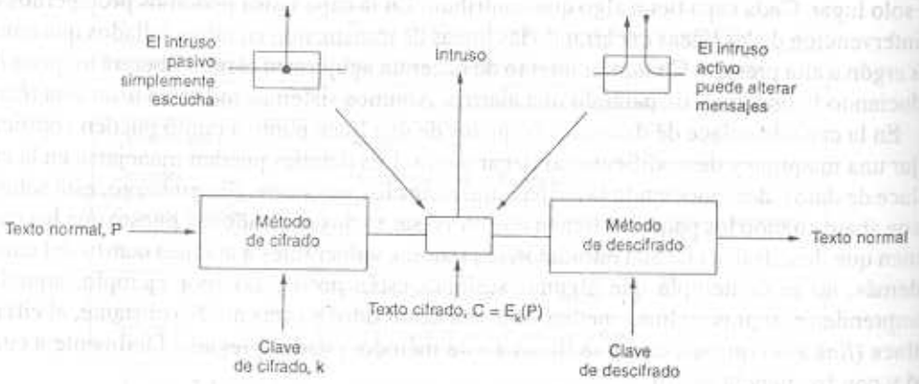


Figura 7-2. El modelo de cifrado.

Los mensajes a cifrar, conocidos como **texto normal** (*plain text*), se transforman mediante una función parametrizada por una **clave**. La salida del proceso de cifrado, conocida como **texto cifrado** (*ciphertext*), se transmite después, muchas veces mediante mensajero o radio. Suponemos que el enemigo, o **intruso**, escucha y copia con exactitud el texto cifrado completo. Sin

embargo, a diferencia del destinatario original, el intruso no conoce la clave de cifrado y no puede descifrar fácilmente el texto cifrado. A veces el intruso no sólo puede escuchar el canal de comunicación (intruso pasivo) sino que también puede registrar mensajes y reproducirlos después, inyectar sus propios mensajes y modificar los mensajes legítimos antes de que lleguen al destinatario (intruso activo). El arte de descifrar se llama **criptoanálisis**. El arte de diseñar cifradores (criptografía) y de descifrarlos (criptoanálisis) se conocen colectivamente como **criptología**.

A menudo resulta útil tener una notación para relacionar el texto normal, el texto cifrado y las claves. Usaremos $C = E_K(P)$ para indicar que el cifrado del texto normal P usando la clave K da el texto cifrado C. Del mismo modo, $P = D_K(C)$ representa el descifrado de C para obtener el texto normal nuevamente. Por tanto,

$$D_k(E_k(P)) = P$$

Esta notación sugiere que E y D son sólo funciones matemáticas, lo cual es cierto. El único truco es que ambas son funciones de dos parámetros, y hemos escrito uno de los parámetros (la clave) como subíndice, en lugar de como argumento, para distinguirlo del mensaje.

Una regla fundamental de la criptografía es que se debe suponer que el criptoanalista conoce el método general de cifrado usado. En otras palabras, el criptoanalista sabe cómo funciona el método de cifrado, E, de la figura 7-2. La cantidad de esfuerzo necesario para inventar, probar e instalar un método nuevo cada vez que el viejo está en peligro, o se piensa que lo está, siempre ha hecho impráctico mantenerlo en secreto, y el pensar que es secreto cuando no lo es hace más daño que bien.

Aquí es donde entra la clave. La clave consiste en una cadena corta (relativamente) que selecciona uno de muchos cifrados potenciales. En contraste con el método general, que tal vez se cambie cada cierto número de años, la clave puede cambiarse con la frecuencia requerida. Por tanto, nuestro modelo básico es un método general estable y conocido públicamente pero parametrizado por una clave secreta y fácilmente cambiable.

La naturaleza no secreta del algoritmo debe subrayarse. Al hacer público el algoritmo, el criptógrafo recibe asesoría gratuita de una gran cantidad de criptólogos académicos ansiosos por descifrar el código del sistema para que puedan publicar trabajos demostrando su inteligencia. Si muchos expertos han tratado de descifrar el algoritmo durante cinco años después de su publicación y nadie lo ha logrado, probablemente es bastante sólido.

El secreto real es la clave, y su longitud es un aspecto importante del diseño. Considere una cerradura de combinación. El principio general es que se introducen dígitos en secuencia. Todo el mundo lo sabe, pero la clave es secreta. Una longitud de clave de dos dígitos significa que hay 100 posibilidades. Una clave de tres dígitos de longitud significa 1000 posibilidades, y una clave de seis dígitos de longitud significa un millón. Cuanto más grande es la clave, mayor será el **factor de trabajo** que tendrá que enfrentar el criptoanalista. El factor de trabajo para descifrar el sistema mediante una búsqueda exhaustiva del espacio de claves crece exponencialmente con la longitud de la clave. El secreto radica en tener un algoritmo robusto (pero público) y una clave larga. Para evitar que su hermano menor lea su correo electrónico, las claves de 64 bits son suficientes. Para mantener a raya gobiernos poderosos, se requieren claves de hasta 256 bits.

Desde el punto de vista del criptoanalista, el problema del criptoanálisis tiene tres variaciones principales. Cuando él tiene cierta cantidad de texto cifrado pero no tiene texto común, enfrenta el problema de **sólo texto cifrado**. Los criptogramas que aparecen en la sección de acertijos de los diarios presentan este tipo de problema. Cuando se tiene un poco de texto cifrado y el texto normal correspondiente, el problema se conoce como **texto normal conocido** (*known plaintext*). Por último, cuando el criptoanalista tiene la capacidad de cifrar textos normales que él escoge, enfrenta el problema de **texto normal seleccionado** (*chosen plaintext*). Los criptogramas de los periódicos podrían descifrarse trivialmente si el criptoanalista pudiese hacer preguntas como: ¿cuál es el cifrado de ABCDE?

Los novatos en el negocio del cifrado con frecuencia suponen que, si un cifrado puede resistir un ataque de sólo texto cifrado, entonces es seguro. Este supuesto es muy ingenuo. En muchos casos, el criptoanalista puede hacer una buena estimación de partes del texto normal. Por ejemplo, lo primero que muchos sistemas de tiempo compartido dicen cuando se les llama es "INDIQUE SU CLAVE". Equipado con algunos pares concordantes de texto normal-texto cifrado, el trabajo del criptoanalista se vuelve mucho más fácil. Para lograr seguridad, el criptógrafo debe ser conservador y asegurarse que el sistema sea inviolable aun si su oponente puede cifrar cantidades arbitrarias de texto normal seleccionado.

Los modelos de cifrado históricamente se dividen en dos categorías: cifrados por sustitución y cifrados por transposición. Ahora reseñaremos cada uno de estos brevemente como antecedente para la criptografía moderna.

Cifrados por sustitución

En un **cifrado por sustitución**, cada letra o grupo de letras se reemplaza por otra letra o grupo de letras para disfrazarla. Uno de los cifrados más viejos conocidos es el **cifrado de César**, atribuido a Julio César. En este método, *a* se vuelve *D*, *b* se vuelve *E*, *c* se vuelve *F*, ... , y *z* se vuelve *C*. Por ejemplo, *ataque* se vuelve *DWDTXH*. En los ejemplos, el texto normal se presentará en minúsculas y el texto cifrado en mayúsculas.

Una pequeña generalización del cifrado de César permite que el alfabeto de texto cifrado se desplace k letras, en lugar de siempre 3. En este caso, k se convierte en una clave del método general de alfabetos desplazados circularmente. El cifrado de César posiblemente engañó a los cartagineses, pero no ha engañado a nadie desde entonces.

La siguiente mejora es hacer que cada uno de los símbolos del texto normal, digamos las 26 letras del abecedario (no incluimos la $\tilde{n}$) inglés, tengan una correspondencia con alguna otra letra. Por ejemplo,

texto normal:	a b c d e f g h i j k l m n o p q r s t u v w x y z
texto cifrado:	Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Este sistema general se llama **sustitución monoalfabética**, siendo la clave la cadena de 26 letras correspondiente al alfabeto completo. Para la clave anterior, el texto normal *ataque* se transformaría en el texto cifrado *QZQJXT*.

A primera vista, esto podría parecer un sistema seguro, porque, aunque el criptoanalista conoce el sistema general (sustitución letra por letra), no sabe cuál de las $26! \approx 4 \times 10^{26}$ claves posibles se está usando. En contraste con el cifrado de César, intentarlas todas no es un enfoque muy prometedor. Aun a 1 μseg por solución, una computadora tardaría 10^{13} años en intentar todas las claves.

No obstante, si se cuenta con una cantidad aún pequeña de texto cifrado, el cifrado puede descifrarse fácilmente. El ataque básico aprovecha las propiedades estadísticas de los lenguajes naturales. En inglés, por ejemplo (que es el idioma del autor de este libro), la *e* es la letra más común, seguida de *t, o, a, n, i*, etc. Las combinaciones de dos letras más comunes, o digramas, son *th, in, er, re* y *an*. Las combinaciones de tres letras más comunes, o trigramas, son *the, ing, and* e *ion*.

Un criptoanalista que intenta descifrar una codificación monoalfabética comenzaría por contar la frecuencia relativa de todas las letras del texto cifrado. Entonces podría asignar tentativamente la más común a la letra *e* y la siguiente más común a la letra *t*. Vería entonces los trigramas para encontrar uno común de la forma *tXe*, lo que sugerirá fuertemente que *X* es *h*. De la misma manera, si el patrón *thYt* ocurre con frecuencia, *Y* probablemente representa *a*. Con esta información se puede buscar un trigrama frecuente de la forma *aZW*, que con mucha probabilidad es *and*. Adivinando las palabras comunes, digramas y trigramas, y conociendo los patrones probables de las vocales y consonantes, el criptoanalista construye un texto normal tentativo, letra por letra.

Otro enfoque es adivinar una palabra o frase probable. Por ejemplo, considere el siguiente texto cifrado de una compañía contable (en bloques de cinco caracteres):

CTBMN BYCTC BTJDS QXBNS GSTJC BTSWX CTQTZ CQVUJ
QJSGS TJQZZ MNQJS VLNSX VSZJU JDSTS JQUUS JUBXJ
DSKSU JSNTK BGAQJ ZBGYQ TLCTZ BNYBN QJSW

Una palabra muy probable en un mensaje de una compañía contable de un país de habla inglesa es *financial*. Usando nuestro conocimiento de que *financial* tiene una letra repetida (*i*), con cuatro letras intermedias entre su aparición, buscamos letras repetidas en el texto cifrado con este espaciado. Encontramos 12 casos, en las posiciones 6, 15, 27, 31, 42, 48, 56, 66, 70, 71, 76 y 82. Sin embargo, sólo dos de éstos, el 31 y 42, tienen la siguiente letra (correspondiente a *n* en el texto normal) repetida en el lugar adecuado. De estos dos, sólo el 31 tiene también la *a* en la posición correcta, por lo que sabemos que *financial* comienza en la posición 30. A partir de aquí, la deducción de la clave es fácil usando las estadísticas de frecuencia del texto en inglés.

Cifrados por transposición

Los cifrados por sustitución conservan el orden de los símbolos de texto normal, pero los disfrazan. Los **cifrados por transposición**, en contraste, reordenan las letras pero no las disfrazan. En la figura 7-3 se presenta un cifrado de transposición común, la transposición columnar.

La clave del cifrado es una palabra o frase que no contiene letras repetidas. En este ejemplo, la clave es MEGABUCK. El propósito de la clave es numerar las columnas, estando la columna 1 bajo la letra clave más cercana al inicio del alfabeto, y así sucesivamente. El texto normal se escribe horizontalmente, en filas. El texto cifrado se lee por columnas, comenzando por la columna cuya letra clave es la más baja.

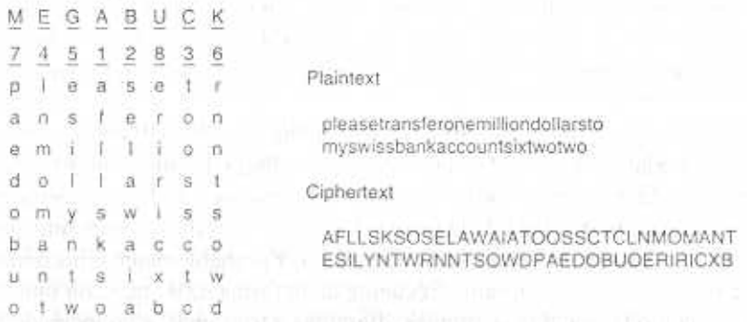


Figura 7-3. Cifrado por transposición.

Para descifrar un cifrado por transposición, el criptoanalista debe primero estar consciente de que está tratando con un cifrado de transposición. Observando la frecuencia de *E, T, A, O, I, N*, etc., es fácil ver si se ajustan al patrón usual del texto normal. De ser así, es evidente que se trata de un cifrado por transposición, pues en tal cifrado cada letra se representa a sí misma.

El siguiente paso es adivinar la cantidad de columnas. En muchos casos, puede adivinarse una palabra o frase probable por el contexto del mensaje. Por ejemplo, supóngase que nuestro criptoanalista sospecha que la frase de texto normal *milliondollars* aparece en algún lugar del mensaje. Observe que los digramas *MO, IL, LL, LA, IR* y *OS* ocurren en el texto cifrado como resultado de que esta frase da la vuelta. La letra de texto cifrado *O* sigue a la letra de texto cifrado *M* (es decir, son adyacentes verticalmente en la columna 4) puesto que están separados en la frase probable por una distancia igual a la longitud de la clave. Si se hubiera usado una clave de longitud siete, habrían ocurrido los digramas *MD, IO, LL, LL, IA, OR* y *NS*. De hecho, para cada longitud de clave, se produce un grupo diferente de digramas en el texto cifrado. Buscando las diferentes posibilidades, el criptoanalista con frecuencia puede determinar fácilmente la longitud de la clave.

El paso restante es ordenar las columnas. Cuando la cantidad de columnas, k, es pequeña, puede examinarse cada uno de los pares de columnas $k(k-1)$ para ver si la frecuencia de sus digramas es igual a la del texto normal. El par con la mejor concordancia se supone correctamente ubicado. Ahora cada columna restante se prueba tentativamente como el sucesor de este par. La columna cuyas frecuencias de digramas y trigramas produce la mejor concordancia se toma tentativamente como correcta. La columna antecesora se encuentra de la misma manera. El proceso completo se repita hasta encontrar un orden potencial. Es probable que el texto normal sea reconocible en este punto (por ejemplo, si aparece *milloin*, quedará claro en dónde está el error).

Algunos cifrados de transposición aceptan un bloque de longitud fija como entrada y producen un bloque de longitud fija como salida. Estos cifrados pueden describirse por completo con sólo dar una lista que indique el orden en el que deben salir los caracteres. Por ejemplo, el cifrado de la figura 7-3 puede verse como un cifrado de bloque de 64 caracteres. Su salida es 4, 12, 20, 28, 36, 44, 52, 60, 5, 13, . . . , 62. En otras palabras, el cuarto carácter de entrada, *a*, es el primero en salir, seguido del décimosegundo, *f*, etcétera.

Rellenos de una sola vez

La construcción de un cifrado inviolable en realidad es bastante sencilla; la técnica se conoce desde hace décadas. Primero se escoge una cadena de bits al azar como clave. Luego se convierte el texto normal en una cadena de bits, por ejemplo usando su representación ASCII. Por último, se calcula el OR EXCLUSIVO de estas dos cadenas, bit por bit. El texto cifrado resultante no puede descifrarse porque cada texto normal posible es un candidato igualmente probable. El texto cifrado no proporciona al criptoanalista ninguna información en absoluto. En una muestra suficientemente grande de texto cifrado, cada letra ocurrirá con la misma frecuencia, al igual que cada digrama y cada trigrama.

Desafortunadamente, este método, conocido como **relleno de una sola vez** (*one-time pad*), tiene varias desventajas prácticas. Para comenzar, la clave no puede memorizarse, por lo que tanto el transmisor como el receptor deben llevar una copia por escrito consigo. Si cualquiera de ellos puede ser capturado, las claves escritas evidentemente son indeseables. Además, la cantidad total de datos que puede transmitirse está limitada a la cantidad de clave disponible. Si el espía tiene suerte y descubre abundante información, podría ser incapaz de transmitirlos todos a su base porque la clave se agotó. Otro problema es la sensibilidad del método a la pérdida o inserción de caracteres. Si el transmisor y el receptor pierden la sincronía, todos los datos a partir de ahí aparecerán alterados.

Con la llegada de las computadoras, el relleno de una sola vez podría volverse práctico para algunas aplicaciones. La fuente de la clave podría ser un CD especial que contenga varios gigabits de información, y si se transporta en una caja de CD de música y va precedida por algunas canciones, ni siquiera resultaría sospechoso. Por supuesto, a velocidades de red de gigabits, tener que introducir un CD nuevo cada cinco segundos podría volverse tedioso. Por esta razón, comenzaremos ahora a ver los algoritmos modernos de cifrado que pueden procesar cantidades arbitrariamente grandes de texto normal.

7.1.2. Dos principios criptográficos fundamentales

Aunque estudiaremos muchos sistemas criptográficos diferentes en las siguientes páginas, hay dos principios que los sostienen a todos y que es importante entender. El primer principio es que todos los mensajes cifrados deben contener redundancia, es decir, información no necesaria para

entender el mensaje. Un ejemplo puede dejar en claro la necesidad de esto. Considere una compañía de compras por teléfono, El Perezoso (EP), que tiene 60,000 productos. Pensando que son muy eficientes, los programadores de EP deciden que los mensajes de pedidos deben consistir en un nombre de cliente de 16 bytes seguido de un campo de datos de 3 bytes (1 byte para la cantidad y 2 para el número del producto). Los últimos 3 bytes deben cifrarse usando una clave muy grande conocida sólo por el cliente y EP.

Inicialmente, esto podría parecer seguro, y en cierto sentido lo es, puesto que los intrusos pasivos no pueden descifrar los mensajes. Desafortunadamente, el sistema también tiene una falla mortal que lo vuelve inútil. Supóngase que un empleado recientemente despedido quiere vengarse de EP por cesarlo. Antes de irse, se lleva con él (parte de) la lista de clientes; entonces trabaja toda la noche escribiendo un programa para generar pedidos ficticios usando nombres reales de los clientes. Dado que no tiene la lista de claves, simplemente pone números aleatorios en los últimos 3 bytes y envía cientos de pedidos a EP.

Al llegar estos mensajes, la computadora de EP usa el nombre del cliente para localizar la clave y descifrar el mensaje. Para mala suerte de EP, casi cada mensaje de 3 bytes es válido, por lo que la computadora comienza a imprimir instrucciones de embarque. Aunque podría parecer extraño que un cliente ordene 137 juegos de columpios para niño, o 240 cajas de arena, en lo que a la computadora concierne el cliente bien podría estar planeando abrir una cadena de lotes con juegos infantiles. De esta manera, un intruso activo (el ex empleado) puede causar muchísimos problemas, aun cuando no pueda entender los mensajes que genera su computadora.

Este problema puede *resolverse* agregando redundancia a todos los mensajes. Por ejemplo, si se extienden los mensajes de pedido a 12 bytes, de los cuales los primeros 9 deben ser ceros, entonces este ataque ya no funciona, porque el ex empleado ya no puede generar una cadena grande de mensajes válidos. La moraleja de esta historia es que todos los mensajes deben contener una cantidad considerable de redundancia para que los intrusos activos no puedan enviar basura al azar y lograr que se interprete como mensajes válidos.

Sin embargo, la adición de redundancia también simplifica a los criptoanalistas el descifrado de los mensajes. Supóngase que el negocio de pedidos por correo es altamente competido, y que la competencia principal de El Perezoso, El Bolsón, estaría encantado de conocer la cantidad de cajas de arena que EP vende. Para ello, ha intervenido la línea telefónica de EP. En el esquema original con mensajes de 3 bytes, el criptoanálisis era prácticamente imposible puesto que, tras adivinar una clave, el criptoanalista no tenía manera de saber si había adivinado correctamente. A fin de cuentas, casi cualquier mensaje era legal. Con el nuevo esquema de 12 bytes, es fácil para el criptoanalista distinguir un mensaje válido de uno no válido.

Por tanto, el principio criptográfico número uno es que todos los mensajes deben contener redundancia para evitar que los intrusos activos engañen al receptor y lo hagan actuar ante un mensaje falso. Sin embargo, esta misma redundancia simplifica mucho la violación del sistema por parte de los intrusos pasivos, por lo que aquí tenemos un poco de tensión. Es más, la redundancia nunca debe tener la forma de n ceros al inicio o al final del mensaje, ya que el análisis de tales mensajes con algunos algoritmos criptográficos da resultados más predecibles, simplificando la tarea del criptoanalista. Una cadena aleatoria de palabras sería mejor para incluir la redundancia.

El segundo principio criptográfico es que deben tomarse algunas medidas para evitar que los intrusos activos reproduzcan mensajes viejos. Si no se toman tales medidas, nuestro ex empleado podría conectarse a la línea telefónica de EP y simplemente continuar repitiendo mensajes válidos enviados previamente. Una de tales medidas es la inclusión en cada mensaje de una marca de tiempo válida durante, digamos, 5 minutos. El receptor puede entonces guardar los mensajes unos 5 minutos, para compararlos con los mensajes nuevos que lleguen y filtrar los duplicados. Los mensajes con mayor antigüedad que 5 minutos pueden descartarse, dado que todas las repeticiones enviadas más de 5 minutos después también se rechazarán como demasiado viejas. Más adelante estudiaremos medidas diferentes de las marcas de tiempo.

7.1.3. Algoritmos de clave secreta

La criptografía moderna usa las mismas ideas básicas que la criptografía tradicional, la transposición y la sustitución, pero su orientación es distinta. Tradicionalmente, los criptógrafos han usado algoritmos sencillos y se han apoyado en claves muy largas para la seguridad. Hoy día es cierto lo inverso: el objetivo es hacer el algoritmo de cifrado tan complicado y rebuscado que inclusive si el criptoanalista obtiene cantidades enormes de texto cifrado a su gusto, no será capaz de entender nada.

Las transposiciones y las sustituciones pueden implantarse mediante circuitos sencillos. En la figura 7-4(a) se muestra un dispositivo, conocido como **caja P** (la P significa permutación), que se usa para efectuar una transposición de una entrada de 8 bits. Si se designan los 8 bits de arriba hacia abajo como 01234567, la salida de esta caja P en particular es 36071245. Mediante el alambrado interno adecuado, puede hacerse que una caja P efectúe cualquier transposición, y lo puede hacer prácticamente a la velocidad de la luz.

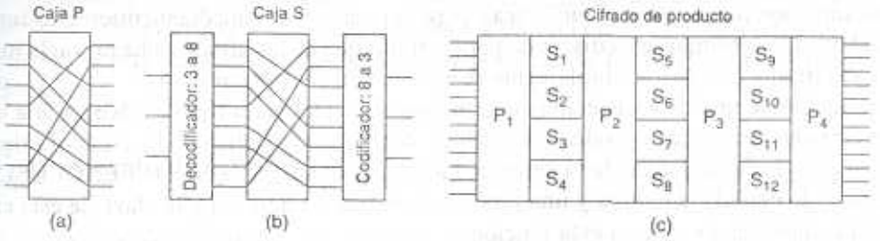


Figura 7-4. Elementos básicos del cifrado de producto. (a) Caja P. (b) Caja S. (c) Producto.

Las sustituciones se llevan a cabo mediante **cajas S**, como se muestra en la figura 7-4(b). En este ejemplo, se ingresa un texto normal de 3 bits y sale un texto cifrado de 3 bits. La entrada de 3 bits selecciona una de las ocho líneas de salida de la primera etapa y la establece en 1; las demás líneas son 0. La segunda etapa es una caja P. La tercera etapa codifica en binario nuevamente la línea de entrada seleccionada. Con el alambrado que se muestra, si los ocho números

octales 01234567 entraran uno tras otro, la secuencia de salida sería de 24506713. En otras palabras se ha reemplazado el 0 por el 2, el 1 por el 4, etc. Nuevamente, mediante el alambrado adecuado de la caja P dentro de la caja S, puede lograrse cualquier sustitución.

La potencia real de estos elementos básicos sólo se hace aparente cuando ponemos en cascada una serie completa de cajas para formar un **cifrado de producto**, como se muestra en la figura 7-4(c). En este ejemplo, se transponen 12 líneas de entrada en la primera etapa. En teoría, sería posible hacer que la segunda etapa sea una caja S que relacione un número de 12 bits con otro número de 12 bits. En vez de ello, tal dispositivo requeriría $2^{12} = 4096$ alambres cruzados en su etapa media. En cambio, la entrada se divide en cuatro grupos de 3 bits, cada uno de los cuales se sustituye independientemente de los demás. Aunque este método es menos general, aún es poderoso. Mediante la inclusión de una cantidad suficiente de etapas en el cifrado de producto, puede hacerse de la salida una función excesivamente complicada de la entrada.

DES

En enero de 1977, el gobierno de Estados Unidos adoptó un cifrado de producto desarrollado por IBM como su estándar oficial para información no secreta. Este cifrado, el **DES** (*Data Encryption Standard*, **estándar de cifrado de datos**), se adoptó ampliamente en la industria para usarse con productos de seguridad. Ya no es seguro en su forma original (Wayner, 1995), pero aún es útil en una forma modificada. Ahora explicaremos el funcionamiento del DES.

En la figura 7-5(a) se muestra un esbozo del DES. El texto normal se cifra en bloques de 64 bits, produciendo 64 bits de texto cifrado. El algoritmo, que se parametriza mediante una clave de 56 bits, tiene 19 etapas diferentes. La primera etapa es una transposición, independiente de la clave, del texto normal de 64 bits. La última etapa es el inverso exacto de esta transposición. La etapa previa a la última intercambia los 32 bits de la izquierda y los 32 bits de la derecha. Las 16 etapas restantes son funcionalmente idénticas, pero se parametrizan mediante diferentes funciones de la clave. El algoritmo se ha diseñado para permitir que el descifrado se haga con la misma clave que el cifrado. Los pasos simplemente se ejecutan en el orden inverso.

La operación de una de estas etapas intermedias se ilustra en la figura 7-5(b). Cada etapa toma dos entradas de 32 bits y produce dos salidas de 32 bits. La salida de la izquierda simplemente es una copia de la entrada de la derecha. La salida de la derecha es el OR EXCLUSIVO a nivel de bit de la entrada izquierda y una función de la entrada derecha y la clave de esta etapa, K_i. Toda la complejidad reside en esta función.

La función consiste en cuatro pasos, ejecutados en secuencia. Primero se construye un número de 48 bits, E, expandiendo el R_{i-1} de 32 bits según una regla fija de transposición y duplicación. Después, se aplica un OR EXCLUSIVO a E y K_i. Esta salida entonces se divide en ocho grupos de 6 bits, cada uno de los cuales se alimenta a una caja S distinta. Cada una de las 64 entradas posibles a la caja S se transforma en una salida de 4 bits. Por último, estos 8×4 bits se pasan a través de una caja P.

En cada una de las 16 iteraciones, se usa una clave diferente. Antes de iniciarse el algoritmo, se aplica una transposición de 56 bits a la clave. Justo antes de cada iteración, la clave se divide

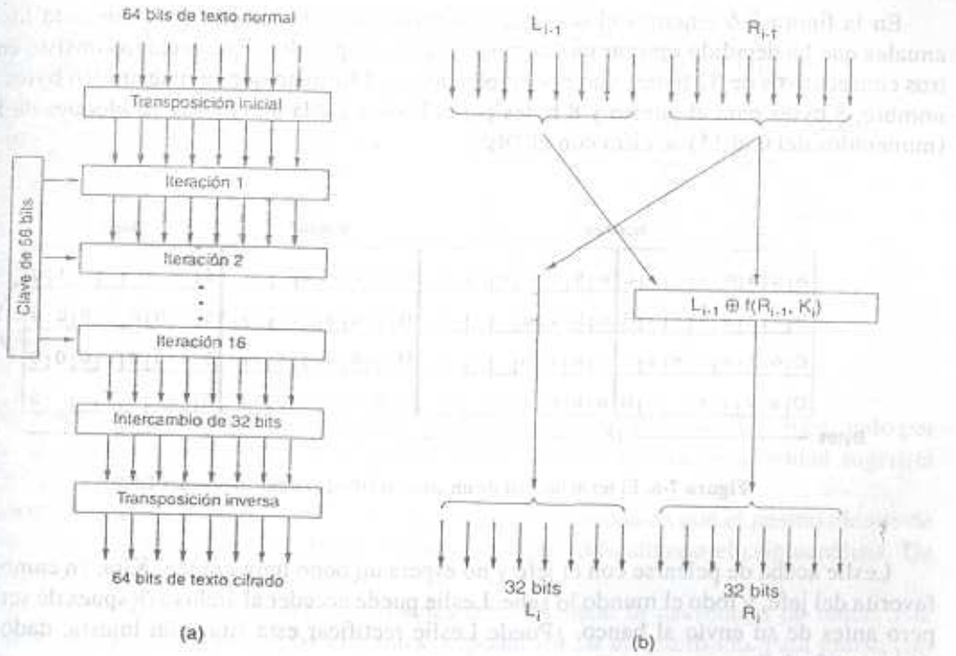


Figura 7-5. El estándar de cifrado de datos. (a) Esbozo general. (b) Detalle de una iteración.

en dos unidades de 28 bits, cada una de las cuales se gira hacia la izquierda una cantidad de bits dependiente del número de iteración. K_i se deriva de esta clave girada aplicándole otra transposición de 56 bits. En cada vuelta se extrae y permuta de los 56 bits un subgrupo de 48 bits diferente.

Encadenamiento DES

A pesar de toda esta complejidad, el DES básicamente es un cifrado por sustitución monoalfabética que usa un carácter de 64 bits. Cada vez que entra el mismo bloque de texto normal de 64 bits por el frente, sale el mismo bloque de texto cifrado de 64 bits por atrás. Un criptoanalista puede explotar esta propiedad como ayuda para violar el DES.

Para ver la manera en que esta propiedad de cifrado por sustitución monoalfabética puede usarse para subvertir el DES, consideremos el cifrado de un mensaje grande de la manera obvia: dividiéndolo en bloques consecutivos de 8 bytes (64 bits) y cifrándolos uno tras otro con la misma clave. El último bloque se rellena a 64 bits, de ser necesario. Esta técnica se conoce como **modo de libro de código electrónico**.

En la figura 7-6 tenemos el comienzo de un archivo de computadora que lista los bonos anuales que ha decidido otorgar una compañía a sus empleados. Este archivo consiste en registros consecutivos de 32 bytes, uno por empleado, en el formato que se muestra: 16 bytes para el nombre, 8 bytes para el puesto y 8 bytes para el bono. Cada uno de los 16 bloques de 8 bytes (numerados del 0 al 15) se cifra con el DES.

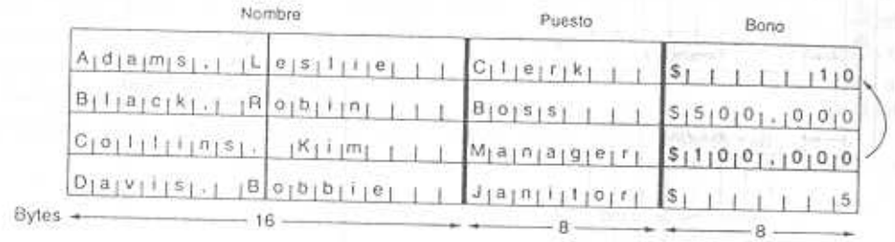


Figura 7-6. El texto normal de un archivo cifrado como 16 bloques DES.

Leslie acaba de pelearse con el jefe y no espera un bono muy grande. Kim, en cambio, es la favorita del jefe, y todo el mundo lo sabe. Leslie puede acceder al archivo después de ser cifrado pero antes de su envío al banco. ¿Puede Leslie rectificar esta situación injusta, dado sólo el archivo cifrado?

No hay problema. Todo lo que necesita Leslie es hacer una copia del bloque de texto cifrado 11 (que contiene el bono de Kim) y usarlo para reemplazar el bloque de texto cifrado 3 (que contiene el bono de Leslie). Aun sin saber lo que dice el bloque 11, Leslie puede esperar tener una Navidad bastante feliz este año. (Copiar el bloque de texto cifrado 7 es otra posibilidad, pero es más fácil su detección; además, Leslie no es una persona codiciosa.)

Para frustrar este tipo de ataque, el DES (y todos los cifrados en bloques) puede encadenarse de varias maneras para que el reemplazo de un bloque de la manera en que lo hizo Leslie haga que el texto normal descifrado comenzando por el bloque reemplazado sea basura. Una forma de encadenar es por **encadenamiento de bloque cifrado**. En este método, que se muestra en la figura 7-7, a cada bloque de texto normal se le hace un OR EXCLUSIVO (#) con el bloque de texto cifrado previo antes de cifrarse. En consecuencia, el mismo bloque de texto normal no corresponde con el mismo bloque de texto cifrado, y el cifrado ya no es un cifrado por sustitución monoalfabética grande. Al primer bloque se le hace un OR EXCLUSIVO con un **vector de inicialización, IV**, seleccionado al azar, que se transmite junto con el texto cifrado.

Podemos ver cómo funciona el encadenamiento de bloques cifrados examinando el ejemplo de la figura 7-7. Comenzamos por calcular $C_0 = E(P_0 \text{ XOR } IV)$. Después calculamos $C_1 = E(P_1 \text{ XOR } C_0)$, etc. El descifrado funciona de la manera opuesta, con $P_0 = IV \text{ XOR } D(C_0)$, etc. Nótese que el cifrado del bloque i es una función de todo el texto normal de los bloques 0 a $i - 1$, por lo que el texto normal genera un texto cifrado diferente dependiendo de dónde ocurre. Una

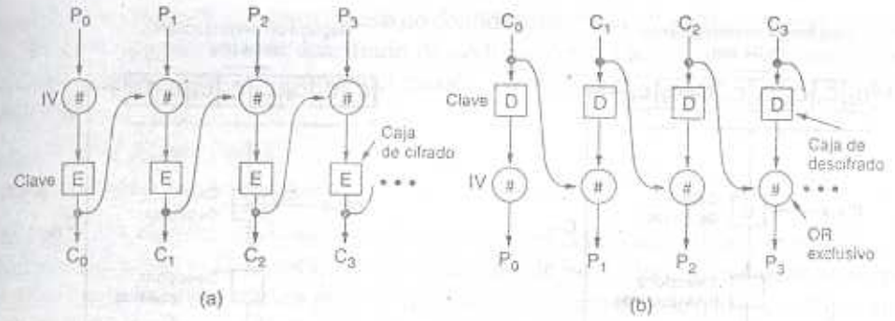


Figura 7-7. Encadenamiento de bloques cifrados.

transformación como la que hizo Leslie produciría basura durante dos bloques, comenzando por el campo de bono de Leslie. Para un oficial de seguridad astuto, esta peculiaridad sugeriría dónde se podría iniciar la investigación subsecuente.

El encadenamiento de bloques cifrados también tiene la ventaja de que el mismo bloque de texto normal no produce el mismo bloque de texto cifrado, dificultando el criptoanálisis. De hecho, ésta es la razón principal de su uso.

Sin embargo, el encadenamiento de bloques cifrados tiene la desventaja de requerir la llegada de un bloque completo de 64 bits antes de poder iniciar el descifrado. Para usarse con terminales interactivas, en las que la gente puede escribir líneas de menos de ocho caracteres y luego detenerse, esperando una respuesta, este modo no es apropiado. Para el cifrado byte por byte puede usarse el **modo de realimentación de cifrado**, que se ilustra en la figura 7-8. En esta figura, se muestra el estado de la máquina de cifrado después de cifrar y enviar los bytes 0 a 9. Al llegar el byte de texto normal 10, como se aprecia en la figura 7-8(a), el algoritmo DES opera con el registro de desplazamiento de 64 bits para generar un texto cifrado de 64 bits. Se extrae el byte de la izquierda de ese texto cifrado y se le aplica un OR EXCLUSIVO con P_{10}. Ese byte se envía por la línea de transmisión. Además, el registro de desplazamiento se desplaza a la izquierda 8 bits, causando la expulsión de C_2 por la izquierda y la introducción de C_{10} en la posición que C_9 dejó vacante a la derecha. Nótese que el contenido del registro de desplazamiento depende de la historia previa completa del texto normal, por lo que un patrón que se repite varias veces en el texto normal se cifrará de manera diferente en cada ocasión. Como ocurre con el encadenamiento de bloques cifrados, se requiere un vector de inicialización para echar a andar el mecanismo.

El descifrado con modo de realimentación de cifrado simplemente hace lo mismo que el cifrado. En particular, el contenido del registro de desplazamiento se *cifra*, no se *descifra*, por lo que el byte seleccionado al que se le hace el OR EXCLUSIVO con C_{10} para obtener P_{10} es el mismo que aquel al que se le hizo el OR EXCLUSIVO con P_{10} para generar C_{10} inicialmente. Mientras los dos registros de desplazamiento permanezcan idénticos, el descifrado funcionará correctamente.

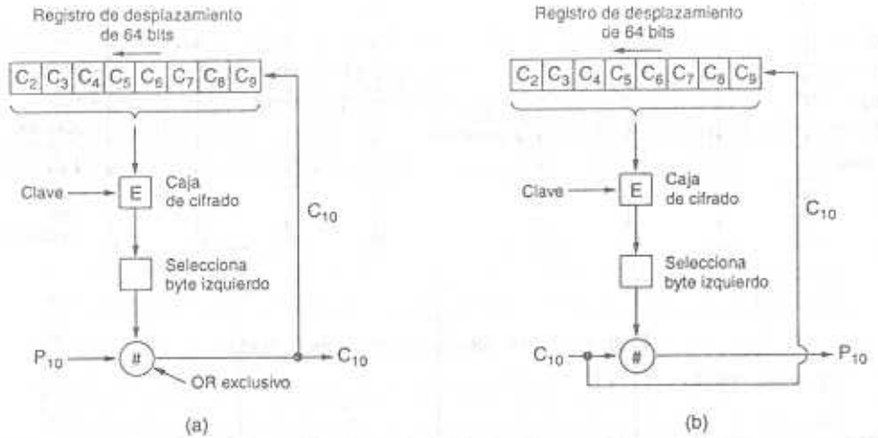


Figura 7-8. Modo de realimentación de cifrado.

Como nota al margen, debe indicarse que, si un bit del texto cifrado accidentalmente se invierte durante la transmisión, los 8 bytes descifrados cuando el byte malo esté en el registro de desplazamiento tendrán error. Una vez que el byte equivocado sea expulsado del registro de desplazamiento, se generará nuevamente texto normal correcto. Por tanto, el efecto de un solo bit invertido está bastante delimitado y no arruina el resto del mensaje.

Sin embargo, existen aplicaciones en las que un error de transmisión de 1 bit que arruina 64 bits de texto normal es un efecto demasiado grande. Para estas aplicaciones existe una cuarta opción, el **modo de realimentación de salida**, que es idéntico al modo de realimentación de cifrado, excepto que el byte realimentado por el lado derecho del registro de desplazamiento se toma justo antes de la caja de OR EXCLUSIVO, no justo después.

El modo de realimentación de salida tiene la propiedad de que un error de 1 bit en el texto cifrado causa un error de 1 solo bit en el texto normal resultante. Por otra parte, es menos seguro que los otros modos, y debe evitarse su uso de propósito general. El modo de libro de código electrónico también debe evitarse excepto en circunstancias especiales (por ejemplo, el cifrado de un solo número aleatorio, como una clave de sesión). Para la operación normal, debe usarse el encadenamiento de bloques cifrados cuando la entrada llega en unidades de 8 bytes (por ejemplo, para cifrar archivos de disco) y el modo de realimentación de cifrado debe usarse para cadenas de entrada irregulares, como la entrada de un teclado.

Descifrado del DES

El DES ha estado rodeado de controversias desde el día en que se lanzó; se basó en un cifrado desarrollado y patentado por IBM, llamado Lucifer, excepto que el cifrado de IBM usaba una clave de 128 bits en lugar de 56 bits. Cuando el gobierno federal de Estados Unidos quiso

estandarizar un método de cifrado para uso no confidencial, "invitó" a IBM a "debatir" el asunto con la NSA, la dependencia de descifrado de códigos del gobierno, que es el empleador de matemáticos y criptólogos más grande del mundo. La NSA es tan secreta que una broma de la industria reza:

Pregunta: ¿Qué significa NSA?
Respuesta: Ninguna Supuesta Agencia.

En realidad, NSA significa *National Security Agency*, Agencia Nacional de Seguridad.

Tras estos debates, la IBM redujo la clave de 128 a 56 bits y decidió mantener en secreto el proceso de diseño del DES. Mucha gente sospechó que la longitud de la clave se redujo para asegurar que la NSA pudiera descifrar el código, pero no ninguna organización de menor presupuesto. El objetivo del diseño secreto supuestamente era esconder una puerta secreta que pudiera facilitar aún más el descifrado del DES por la NSA. Cuando un empleado de la NSA discretamente pidió al IEEE que cancelara una conferencia planeada sobre criptografía, eso incomodó aún más a la gente.

En 1977, dos investigadores de criptografía de Stanford, Diffie y Hellman (1977) diseñaron una máquina para violar el DES y estimaron que podría construirse por unos 20 millones de dólares. Dado un trozo pequeño de texto normal y el texto cifrado correspondiente, esta máquina podría encontrar la clave mediante una búsqueda exhaustiva del espacio de claves de 2^{56} en menos de un día. Hoy día tal máquina costaría tal vez un millón de dólares. En (Wiener, 1994) se presenta un diseño detallado de una máquina que puede descifrar el DES mediante búsqueda exhaustiva en unas cuatro horas.

Aquí presentamos otra estrategia. Aunque el cifrado por *software* es 1000 veces más lento que el cifrado por *hardware*, una computadora casera de alto nivel aún podría efectuar unos 250,000 cifrados/seg en *software*, y es probable que esté ociosa unos 2 millones de segundos al mes. Este tiempo de inactividad podría usarse para descifrar el DES. Si alguien pusiera un mensaje en uno de los populares grupos de noticias de Internet, no debería ser difícil conseguir las 140,000 personas necesarias para revisar las 7×10^{16} claves en un mes.

Probablemente la idea más innovadora para descifrar el DES es la **lotería china** (Quisquater y Girault, 1991). En este diseño, cada radio y televisión tiene que equiparse con un *chip* DES barato capaz de realizar 1 millón de cifrados/seg en *hardware*. Suponiendo que cada una de las 1.2 mil millones de personas de China tiene un radio o televisión, cada vez que el gobierno quiera descifrar un mensaje codificado con DES, simplemente difunde el par texto normal/texto cifrado, y cada uno de los 1.2 mil millones de *chips* comienza a buscar su sección preasignada del espacio de claves. En 60 segundos se encontrarán una o más correspondencias. Para asegurar que se informen, los *chips* podrían programarse para desplegar o anunciar el mensaje:

¡FELICIDADES! ACABA DE GANAR LA LOTERÍA CHINA. PARA COBRAR SU PREMIO, POR FAVOR LLAME AL 1-800-GRAN-PREMIO.

La conclusión que se puede obtener de estos argumentos es que el DES ya no debería usarse para nada importante. Sin embargo, aunque 2^{56} es un miserable 7×10^{16}, 2^{112} es un magnífico

5×10^{33}. Aun con mil millones de *chips* DES efectuando mil millones de operaciones por segundo, se requerirían 100 millones de años para examinar detalladamente un espacio de claves de 112 bits. Por tanto, surge la idea de simplemente ejecutar el DES dos veces, con dos claves de 56 bits diferentes.

Desgraciadamente, Merkle y Hellman (1981) han desarrollado un método que hace sospechoso al doble cifrado. Se llama ataque de **encuentro a la mitad** (*meet-in-the-middle*) y funciona como sigue (Hellman, 1980). Supóngase que alguien ha cifrado doblemente una serie de bloques de texto normal usando el modo de libro de código electrónico. Para unos pocos valores de i, el criptoanalista tiene pares igualados (P_i, C_i) donde

$$Ci = E_{k2}(E_{k1}(P_i))$$

Si ahora aplicamos la función de descifrado, D_{K2}, a cada lado de esta ecuación, obtenemos

$$D_{k2}(C_i) = E_{k1}(P_i) \qquad (7\text{-}1)$$

porque el cifrado de x y su descifrado posterior con la misma clave produce x.

El ataque de encuentro a la mitad usa esta ecuación para encontrar las claves DES, $K1$ y $K2$, como sigue:

1. Calcular $R_i = E_i(P_1)$ para los 2^{56} valores de i, donde E es la función de cifrado DES. Ordenar esta tabla en orden ascendente según R_i.
2. Calcular $S_j = D_j(C_1)$ para todos los 2^{56} valores de j, donde D es la función de descifrado DES. Ordenar esta tabla en orden ascendente según S_j.
3. Barrer la primera tabla en busca de un R_i igual a algún S_j de la segunda tabla. Al encontrar un par, tenemos un par de claves (i, j) tal que $D_j(C_1) = E_i(P_1)$. Potencialmente, i es K_1 y j es K_2.
4. Comprobar si $E_j(E_i(P_2))$ es igual a C_2. Si lo es, intentar todos los demás pares (texto normal, texto cifrado). De no serlo, continuar buscando pares en las dos tablas.

Ciertamente ocurrirán muchas falsas alarmas antes de encontrar las claves reales, pero tarde o temprano se encontrarán. Este ataque requiere sólo 2^{57} operaciones de cifrado o descifrado (para construir las dos tablas), mucho menos que 2^{112}; sin embargo, también requiere un total de 2^{60} bytes de almacenamiento para las dos tablas, por lo que actualmente no es factible en su forma básica, pero Merkle y Hellman han mostrado varias optimizaciones y concesiones que permiten menos almacenamiento a expensas de más cómputo. En conclusión, el cifrado doble usando el DES probablemente no es mucho más seguro que el cifrado sencillo.

El cifrado triple es otro asunto. Ya para 1979, IBM se dio cuenta de que la longitud de la clave DES era demasiado corta y diseñó una manera de aumentarla efectivamente usando codificación triple (Tuchman, 1979). El método seleccionado, que se ha incorporado al estándar internacional 8732, se ilustra en la figura 7-9. Aquí se usan dos claves y tres etapas. En la

primera etapa, el texto normal se cifra con K_1. En la segunda etapa, el DES se ejecuta en modo de descifrado, usando K_2 como clave. Por último, se hace otro cifrado usando K_1.

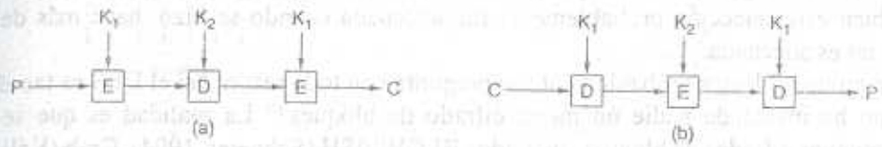


Figura 7-9. Cifrado triple usando el DES.

Este diseño inmediatamente da pie a dos preguntas. Primero, ¿por qué sólo se usan dos claves en lugar de tres? Segundo, ¿por qué se usa EDE (cifrado-descifrado-cifrado) en lugar de EEE (cifrado-cifrado-cifrado)? La razón de que se usen dos claves es que incluso los criptógrafos más paranoicos coinciden en que 112 bits son suficientes para las aplicaciones comerciales por ahora. Subir a 168 bits simplemente agregaría la carga extra innecesaria de administrar y transportar otra clave.

La razón para cifrar, descifrar y luego cifrar de nuevo es la compatibilidad en reversa con los sistemas DES de una sola clave. Tanto las funciones de cifrado como de descifrado son correspondencias entre grupos de números de 64 bits. Desde el punto de vista criptográfico, las dos correspondencias son igualmente robustas. Sin embargo, usando EDE en lugar de EEE, una computadora que usa cifrado triple puede hablar con otra que usa cifrado sencillo simplemente estableciendo $K_1 = K_2$. Esta propiedad permite la introducción gradual del cifrado triple, algo que no interesa a los criptógrafos académicos, pero de importancia considerable para IBM y sus clientes.

No se conoce ningún método para descifrar el DES triple en modo EDE. Van Oorschot y Wiener (1988) han presentado un método para acelerar la búsqueda de EDE en un factor de 16, pero aun con su ataque, el EDE es muy seguro. Para cualquiera que desea nada menos que lo mejor, se recomienda el EEE con tres diferentes claves de 56 bits (168 bits en total).

Antes de dejar el tema del DES, vale la pena cuando menos mencionar dos recientes avances del criptoanálisis. El primero es el **criptoanálisis diferencial** (Biham y Shamir, 1993). Esta técnica puede usarse para atacar cualquier cifrado en bloques; funciona comenzando por un par de bloques de texto normal que difieren sólo en una cantidad pequeña de bits y observando cuidadosamente lo que ocurre en cada iteración interna a medida que avanza el cifrado. En muchos casos, algunos patrones son mucho más comunes que otros, y esta observación conduce a un ataque probabilístico.

El otro avance que vale la pena mencionar es el **criptoanálisis lineal** (Matsui, 1994). Que puede descifrar el DES con sólo 2^{43} textos comunes conocidos. Funciona haciendo un OR EXCLUSIVO entre ciertos bits de texto normal y texto cifrado para generar 1 bit. Al hacerse repetidamente, la mitad de los bits deben ser de ceros y la otra deben ser unos. Sin embargo, con frecuencia los cifrados introducen un sesgo en una dirección o en la otra, y este sesgo, por pequeño que sea, puede explotarse para reducir el factor de trabajo. Para los detalles, véase el trabajo de Matsui.